UBND THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO
CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

ĐỀ TÀI: CÔNG NGHỆ ASP.NET CORE
XÂY DỰNG HỆ THỐNG ĐĂNG TIN BẤT ĐỘNG SẢN

Giảng viên hướng dẫn: Phạm Thi Vương

Nhóm sinh viên thực hiện:

1. Nguyễn Hoàng Phúc - 3122560060
2. Nguyễn Gia Bảo - 3122560004
3. Đoàn Tuấn Tài – 3122410365

TP. HỒ CHÍ MINH — NĂM 2026

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

LỜI CẢM ƠN

Chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy Phạm Thi Vương – Giảng viên giảng dạy bộ môn Các công nghệ lập trình hiện đại, Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn. Thầy đã tận tâm truyền đạt những nền tảng lý thuyết chuyên sâu, các nguyên lý kiến trúc backend tiên tiến cùng những kinh nghiệm thực chiến vô cùng quý báu trong suốt học kỳ vừa qua.

Nhờ sự định hướng và chỉ dẫn tận tình của Thầy, nhóm đã có cơ hội tiếp cận và làm chủ hệ sinh thái công nghệ ASP.NET Core hiện đại, từ việc tổ chức mã nguồn theo mô hình Modular Monolith kết hợp Clean Architecture, đến việc giải quyết các bài toán kỹ thuật phức tạp như kiểm soát đồng thời (Optimistic Concurrency), điều phối giao dịch phân tán (Saga Orchestration), đảm bảo tính nhất quán dữ liệu (Transactional Outbox, Inbox Deduplication) và xây dựng hệ thống theo dõi lịch sử tinh gọn (History Delta Engine).

Mặc dù nhóm đã nỗ lực hết mình để nghiên cứu, thiết kế và cài đặt hệ thống một cách chuẩn mực nhất, song do hạn chế về thời gian cũng như kinh nghiệm thực tế, đồ án chắc chắn khó tránh khỏi những điểm thiếu sót nhất định. Nhóm kính mong nhận được những lời nhận xét, góp ý và chỉ bảo quý báu từ Thầy để hệ thống ngày càng được hoàn thiện hơn.

Kính chúc Thầy luôn dồi dào sức khỏe, ngập tràn niềm vui và gặt hái thêm nhiều thành công rực rỡ trong sự nghiệp nghiên cứu và giảng dạy cao quý của mình!

TÓM TẮT ĐỒ ÁN

Trong kỷ nguyên chuyển đổi số và phát triển mạnh mẽ của các dịch vụ số hóa, việc lựa chọn một nền tảng backend vững chắc, có hiệu năng vượt trội, tính an toàn cao và kiến trúc linh hoạt là yếu tố quyết định sự thành bại của các hệ thống thông tin quy mô lớn. Báo cáo này tập trung nghiên cứu chuyên sâu về ASP.NET Core .NET 8 – nền tảng backend nguồn mở, đa nền tảng và dẫn đầu về mặt hiệu năng của Microsoft – thông qua bài toán thực tế: Xây dựng hệ thống đăng tin và quản lý bất động sản PropNest.

Nội dung báo cáo được kết cấu thành bốn phần chính chặt chẽ và logic:

- **Phần 1:** Trình bày tổng quan về công nghệ ASP.NET Core, khảo cứu lịch sử phát triển từ .NET Framework truyền thống đến cuộc cách mạng tái thiết kế toàn diện .NET Core và giai đoạn hiện đại hóa (.NET 8/9); phân tích hệ sinh thái công nghệ và bức tranh toàn cảnh về nhu cầu tuyển dụng, cơ hội việc làm, thang đo mức lương trên thị trường hiện nay.
- **Phần 2:** Đi sâu vào các kiến thức kỹ thuật cốt lõi và bài thực hành thiết lập dự án PropNest từ đầu. Hướng dẫn chuẩn bị môi trường, phân tích cấu trúc mã nguồn theo mô hình Modular Monolith và Clean Architecture, cấu hình Program.cs, Middleware Pipeline, EF Core DbContext, Docker Compose (SQL Server và RabbitMQ) và khởi tạo API đầu tiên.
- **Phần 3:** Trình bày chi tiết quá trình phân tích, thiết kế và hiện thực hóa đồ án PropNest. Điểm nhấn là các giải pháp kỹ thuật tinh vi: thuật toán so sánh và tái tạo lịch sử biến động giá (History Delta Engine), quản lý xung đột dữ liệu bằng RowVersion (Optimistic Concurrency), điều phối quy trình mua gói VIP bằng Saga Orchestrator (MassTransit kết hợp RabbitMQ), đảm bảo độ tin cậy bằng Transactional Outbox Pattern và Inbox Deduplication, cùng cơ chế hoàn tiền tự động (Compensating Actions) khi có sự cố.
- **Phần 4:** Đánh giá tổng thể đồ án, tổng kết mức độ hoàn thành so với kế hoạch ban đầu, phân tích các lỗi kỹ thuật quan trọng phát sinh và bài học kinh nghiệm rút ra;

đồng thời vạch ra lộ trình tích hợp hệ thống kiểm duyệt tin tự động AME (Auto Moderation Engine) với FastAPI và Ollama AI trong tương lai.

Kết quả đạt được của đồ án không chỉ là một hệ thống backend hoạt động ổn định, có tài liệu API Swagger đầy đủ và khả năng tự phục hồi lỗi giao dịch, mà còn là minh chứng trực quan cho tính khả thi và sức mạnh vượt trội của ASP.NET Core trong việc hiện thực hóa các ứng dụng doanh nghiệp hiện đại.

MỤC LỤC

NHẬN XÉT CỦA GIẢNG VIÊN i
LỜI CẢM ƠN ii
TÓM TẮT ĐỒ ÁN iii
DANH MỤC HÌNH ẢNH v
DANH MỤC BẢNG vii
BẢNG PHÂN CHIA CÔNG VIỆC NHÓM ix
PHẦN 1: TỔNG QUAN VỀ CÔNG NGHỆ ASP.NET CORE 1
CHƯƠNG 1.GIỚI THIỆU CHUNG 1
1.1 Lịch sử hình thành 1
1.2 Sự thay đổi qua các phiên bản 2
1.2.1 Các phiên bản đầu từ 2016 đến 2018 (Giai đoạn kiến thiết nền tảng) 2
1.2.2 Giai đoạn mở rộng và thống nhất từ 2019 đến 2021 3
1.2.3 Giai đoạn trưởng thành và hiện đại hóa từ 2022 đến nay 3
1.3 Hệ sinh thái ASP.NET Core 5
1.3.1 Các thành phần cốt lõi 5
1.3.2 Các package và framework chính thức 6
1.3.3 Công cụ hỗ trợ phát triển 7
KẾT LUẬN CHƯƠNG 1 7
CHƯƠNG 2CƠ HỘI NGHỀ NGHIỆP VÀ THỊ TRƯỜNG 8
2.1 Các vị trí công việc 8
2.2 Nhu cầu tuyển dụng 8
2.3 Mức lương tham khảo 9
KẾT LUẬN CHƯƠNG 2 11

PHẦN 2: NỘI DUNG CỐT LÕI VÀ THỰC HÀNH 12
CHƯƠNG 3CÀI ĐẶT MÔI TRƯỜNG VÀ “HELLO WORLD” 12
3.1 Các công cụ cần thiết để bắt đầu 12
3.1.1 .NET SDK và dotnet CLI 12
3.1.2 C# 12 và các tính năng ngôn ngữ hiện đại 12
3.1.3 Visual Studio 2022 và Visual Studio Code 13
3.1.4 Microsoft SQL Server 13
3.1.5 Entity Framework Core và EF Core Tools 13
3.1.6 Docker và Docker Compose 14
3.1.7 Message Broker: RabbitMQ và MassTransit 14
3.1.8 Swagger UI (OpenAPI) và Postman 14
3.1.9 Git và GitHub 14
3.1.10 Bảng tóm tắt công cụ cần cài đặt 14
3.2 Cài đặt và khởi tạo dự án 16
3.3 Phân tích cấu trúc thư mục của dự án PropNest 20
3.3.1 Cây thư mục dự án tổng quan 20
3.3.2 Tổ chức code Backend theo từng lớp 20
3.3.2.1 Domain Layer (src/PropNest.Domain) 20
3.3.2.2 Application Layer (src/PropNest.Application) 21
3.3.2.3 Infrastructure Layer (src/PropNest.Infrastructure) 21
3.3.2.4 Api Layer (src/PropNest.Api) 22
3.3.2.5 Workflow Worker (src/PropNest.Workflow.Worker) 22

3.3.3 Tại sao chọn kiến trúc Modular Monolith kết hợp Clean Architecture? 22
3.4 Bài tập ứng dụng (Khởi tạo dự án và chạy Hello World) 23
3.4.1 File cấu hình ứng dụng (appsettings.json và appsettings.Development.json) 24
3.4.2 File cấu hình khởi chạy (Properties/launchSettings.json) 25
3.4.3 File Program.cs – Entry Point ứng dụng 26
3.4.4 Docker Compose – Điều phối hạ tầng SQL Server và RabbitMQ 34
3.4.5 Định nghĩa Entity ban đầu và DbContext (PropNestDbContext) 35
3.4.6 Chạy Migration đầu tiên và cập nhật CSDL 36
3.4.7 Endpoint kiểm tra sức khỏe và API đầu tiên 37
3.4.8 Kết quả chạy ứng dụng Hello World qua Swagger UI 38
KẾT LUẬN CHƯƠNG 3 38
CHƯƠNG 4CÁC KHÁI NIỆM NỀN TẢNG CỦA BACKEND VỚI ASP.NET CORE..... 39
4.1 Cú pháp và đặc trưng C# từ cơ bản đến nâng cao 39
4.1.1 Hệ thống Type System, Nullable Reference Types và Encapsulation 39
4.1.2 Tính bất biến với Record Types và cú pháp C# 12 (Primary Constructors, Collection Expressions) 42
4.1.3 Pattern Matching và Switch Expressions 44
4.1.4 Lập trình bất đồng bộ (Asynchronous Programming) với async/await, Task, ValueTask và CancellationToken 47

4.1.5 Compile-time Code Generation với Source Generated LoggerMessage....49
4.2 Kiến trúc Kestrel Web Server và Generic Host....52
4.2.1 Kestrel Web Server và cơ chế xử lý I/O bất đồng bộ....52
4.2.2 Generic Host (WebApplicationBuilder / WebApplication) và hệ thống Configuration....52
4.2.3 Luồng xử lý một HTTP Request hoàn chỉnh (End-to-End Request Lifecycle Flow)....55
4.3 Dependency Injection (DI) và Quản lý Service Lifetimes....58
4.3.1 Nguyên lý Inversion of Control (IoC) và Dependency Inversion (SOLID) 58
4.3.2 Ba vòng đời dịch vụ: Transient, Scoped, Singleton và rủi ro Captive Dependency 59
4.3.3 Tổ chức đăng ký dịch vụ theo module qua ServiceCollectionExtensions....64
4.4 Xử lý HTTP Request qua Middleware Pipeline....65
4.4.1 Bản chất hàm ủy thác RequestDelegate và cơ chế luân chuyển Russian Doll....65
4.4.2 Thứ tự thực thi chuẩn của Middleware trong ASP.NET Core....65
4.4.3 Xây dựng Custom Middleware trong PropNest....68
4.4.4 Phân biệt chuyên sâu giữa Middleware và Filter trong ASP.NET Core 70
4.5 Routing, Controller và Chuẩn hóa phản hồi Web API....71
4.5.1 Attribute Routing, Route Constraints và Model Binding....71
4.5.2 So sánh Controller-based API và Minimal API....74

4.5.3 So sánh Task<IActionResult> và Task<ActionResult<T>> trong Controller 76

4.5.4 Chuẩn hóa lỗi API theo đặc tả RFC 7807 ProblemDetails 77

4.6 Quản lý dự án chuyên nghiệp với .NET Tooling 80

4.6.1 Quản lý dependencies tập trung với Central Package Management (CPM) 80

4.6.2 Kiểm soát chất lượng code với Roslyn Analyzers và TreatWarningsAsErrors 82

4.7 So sánh và Phản biện Công nghệ (Comparative Analysis & Counter-Arguments) 84

4.7.1 So sánh đối sánh ASP.NET Core 8 với các công nghệ Backend tương đương 84

4.7.2 Những trường hợp cụ thể KHÔNG NÊN DÙNG ASP.NET Core (Trải nghiệm thực tế của nhóm) 86

KẾT LUẬN CHƯƠNG 4 87

CHƯƠNG 5THIẾT KẾ VÀ QUẢN LÝ CƠ SỞ DỮ LIỆU VỚI ENTITY FRAMEWORK CORE 89

5.1 Vai trò của cơ sở dữ liệu trong hệ thống Backend 89

5.1.1 Lưu trữ dữ liệu bền vững (Database Persistence) 89

5.1.2 Tính toàn vẹn dữ liệu (Data Integrity) và nguyên lý ACID 89

5.2 Mô hình cơ sở dữ liệu quan hệ (Relational Database) 91

5.2.1 Microsoft SQL Server và ưu thế cho hệ thống doanh nghiệp 91

5.2.2 So sánh Relational Database và NoSQL Database trong bài toán Bất động sản 92

5.3 ORM (Object-Relational Mapping) 94

5.3.1 Khái niệm ORM và lợi ích trong phát triển phần mềm hiện đại 94
5.3.2 So sánh Entity Framework Core với Dapper và ADO.NET thuần....95
5.4 Cơ chế hoạt động của Entity Framework Core 98
5.4.1 Mapping Entity sang Table bằng Fluent API 98
5.4.2 Change Tracker và vòng đời của thực thể (Entity States) 100
5.4.3 Repository Pattern và Unit of Work trong EF Core 102
5.5 Ứng dụng EF Core trong dự án PropNest 103
5.5.1 Tích hợp ASP.NET Core Identity vào DbContext (IdentityDbContext) 103
5.5.2 Cấu hình kiểu dữ liệu tiền tệ DECIMAL(18,2) và RowVersion 104
5.6 Thiết kế Entity và Schema chi tiết cho PropNest 106
5.6.1 Nhóm bảng Identity & Wallet (Users, Roles, Wallets, WalletTransactions).....106
5.6.2 Nhóm bảng Listing & Moderation (Listings, Images, Histories, Reviews) 107
5.6.3 Nhóm bảng Workflow & Outbox (IdempotencyRequests, SagaStates, OutboxMessages) 107
5.7 Migration Database và Quản lý phiên bản Schema.....109
5.7.1 Nguyên lý Versioning Database Schema bằng mã nguồn 109
5.7.2 Quy trình Migration Workflow trong nhóm phát triển.....109
KẾT LUẬN CHƯƠNG 5 112
CHƯƠNG 7XÂY DỰNG RESTFUL API NÂNG CAO VÀ VALIDATION TRONG ASP.NET CORE.....113
7.1 DTO và Validation với FluentValidation.....113

7.1.1 Tại sao cần tách biệt DTO với Entity trong Domain-Driven Design? 113

7.1.2 Thư viện DataAnnotations và cơ chế thẩm định mô hình (Model Validation) trong ASP.NET Core 115

7.1.3 Cấu hình tự động kiểm tra mô hình trước khi vào Controller với [ApiController] 118

7.1.4 Xây dựng DTO và Validation cho CreateListingRequest, UpdateListingRequest và PurchasePackageRequest 119

7.1.5 Thẩm định quy tắc nghiệp vụ phức tạp: Kiểm tra tính hợp lệ của RowVersion và cấu trúc JSON 123

7.2 Action Filters và Endpoint Filters 128

7.2.1 Thứ tự thực thi và phạm vi của Action Filter trong ASP.NET Core 128

7.2.2 Kiểm tra Idempotency-Key thông qua Action Filter 132

7.3 Xử lý ngoại lệ toàn cục (Global Exception Handling) 138

7.3.1 Phân loại ngoại lệ nghiệp vụ và ngoại lệ hệ thống trong kiến trúc Clean Architecture 138

7.3.2 Triển khai ExceptionHandlingMiddleware theo chuẩn RFC 7807 ProblemDetails 141

7.4 Interceptors và Correlation Logging 147

7.4.1 CorrelationIdMiddleware theo vết luồng dữ liệu xuyên suốt các tầng 147

7.4.2 Ghi log request/response có cấu trúc bằng Serilog 150

7.5 Tài liệu hóa API với OpenAPI / Swagger 153

7.5.1 Cấu hình bảo mật Swagger với Bearer JWT Scheme 153

7.5.2 Chuẩn hóa định dạng JSON múi giờ Việt Nam và mô tả header Idempotency-Key trên OpenAPI 155
KẾT LUẬN CHƯƠNG 6 158
CHƯƠNG 8BẢO MẬT, XÁC THỰC VÀ PHÂN QUYỀN HỆ THỐNG TRONG ASP.NET CORE161
8.1 Tổng quan bảo mật hệ thống thông tin 161
8.1.1 Authentication (Xác thực danh tính) 161
8.1.2 Authorization (Phân quyền truy cập tài nguyên) 162
8.2 Các phương pháp xác thực hiện đại 163
8.2.1 So sánh Session-based Authentication và Token-based Authentication 163
8.2.2 Cấu trúc và nguyên lý hoạt động của JSON Web Token (Header, Payload, Signature) 165
8.3 ASP.NET Core Identity và Cơ chế băm mật khẩu 168
8.3.1 Thuật toán băm mật khẩu chuẩn doanh nghiệp PBKDF2 với Salt .. 168
8.3.2 Cơ chế khóa tài khoản (Lockout) chống tấn công dò mật khẩu Brute-force 170
8.4 Phân quyền dựa trên vai trò (Role-Based Access Control - RBAC) 172
8.4.1 Hệ thống phân cấp vai trò trong PropNest: Seller, Moderator, Admin 172
8.4.2 Cấu hình và sử dụng [Authorize(Roles = "...")] trên Controller và Endpoint 173
8.5 Policy-based và Resource-based Authorization 175
8.5.1 Định nghĩa Policy CanModerateListings cho ban kiểm duyệt 175

8.5.2 Resource-based Authorization bảo vệ quyền sở hữu tin đăng (chỉ chủ tin mới được sửa) 176
8.6 Thực hành: Xây dựng Module Authentication và Phân quyền API 179
8.6.1 Hiện thực API Đăng ký tài khoản (POST /api/v1/auth/register) 179
8.6.2 Hiện thực API Đăng nhập và cấp phát JWT Token (POST /api/v1/auth/login) 182
8.6.3 Kiểm thử phân quyền truy cập giữa Seller và Moderator qua Swagger 187
8.6.3.1 Cơ chế kiến trúc bảo vệ đa tầng (Multi-Layer Authorization Architecture) 187
8.6.3.2 Quy trình khởi tạo và gán quyền tài khoản Moderator / Admin 188
8.6.3.3 Minh chứng thực nghiệm kiểm thử phân quyền (Test Execution Evidence) 191
8.6.3.4 Quy trình kiểm thử trực quan thông qua Swagger UI 197
KẾT LUẬN CHƯƠNG 7 198
CHƯƠNG 9XỬ LÝ GIAO DỊCH PHÂN TÁN VÀ ĐIỀU PHỐI WORKFLOW TRONG ASP .NET CORE 200
9.1 Thách thức giao dịch trong hệ thống Backend phân tán 200
9.1.1 Giới hạn của giao dịch Two-Phase Commit (2PC) trong môi trường microservices 200
9.1.2 Định lý CAP và nguyên lý tính nhất quán cuối cùng (Eventual Consistency) 202
9.2 Mô hình Saga Pattern 204
9.2.1 Choreography-based Saga: Đặc điểm, ưu thế và hạn chế 205

9.2.2 Orchestration-based Saga: Lý do lựa chọn cho bài toán thanh toán mua gói PropNest 206
9.3 Triển khai Saga State Machine với MassTransit trong ASP .NET Core 208
9.3.1 Các trạng thái trong PurchasePackageSaga 208
9.3.2 Lưu trữ bền vững trạng thái Saga trong SQL Server (SagaListingStates) 214
9.4 Đảm bảo độ tin cậy thông điệp với Outbox và Inbox 216
9.4.1 Transactional Outbox Pattern ngăn ngừa mất sự kiện sau khi commit CSDL 216
9.4.2 Inbox Deduplication ngăn chặn xử lý trùng lặp tại Consumer 223
9.5 Cơ chế Idempotency Request và Hoàn tiền tự động (Compensating Actions) 228
9.5.1 Bảng IdempotencyRequests và cơ chế nhận diện request lặp qua RequestHash 228
9.5.2 Hành động hoàn tiền vào ví (Refund Ledger) khi xảy ra lỗi nâng cấp tin 232
9.5.3 Ghi nhận chi tiết từng bước chuyển trạng thái (WorkflowStep) và API Polling tiến trình 237
KẾT LUẬN CHƯƠNG 8 240
CHƯƠNG 10TỐI ƯU HIỆU NĂNG VÀ VẬN HÀNH HỆ THỐNG 242
10.1 Các vấn đề hiệu năng và điểm nghẽn trong hệ thống Backend 242
10.1.1 Bottleneck tại tầng ứng dụng (CPU, Memory Allocation) 242
10.1.2 Bottleneck tại tầng truy xuất dữ liệu: Phân tích chi phí Entity Framework Core và tối ưu hóa truy vấn 246

10.1.3 Lý thuyết chỉ mục (Indexing Theory) và thiết kế Index tối ưu trên SQL Server 249

10.2 Kiểm soát xung đột ghi đè đồng thời (Optimistic Concurrency Control - OCC) 251

10.2.1 Cơ sở lý thuyết: So sánh chuyên sâu giữa Pessimistic Locking và Optimistic Concurrency Control 251

10.2.2 Cơ chế hiện thực hóa RowVersion / Timestamp trong SQL Server và EF Core 8 253

10.3 Xử lý tác vụ nền và lập lịch định kỳ với Quartz.NET 258

10.3.1 Giới hạn của BackgroundService cơ bản và ưu thế của Quartz.NET 258

10.3.2 Cấu hình Job, Trigger Cron và cơ chế chống chồng lấn [DisallowConcurrentExecution].....260

10.3.3 Hiện thực hóa kỹ thuật xử lý mẻ nhỏ (Batch Processing / Chunking) trong ListingExpirationJob.....263

10.4 Khả năng quan sát hệ thống (Observability): Structured Logging và Health Checks.....268

10.4.1 Hệ thống ghi log có cấu trúc (Structured Logging) với Serilog và phân tán vết (Distributed Tracing) 268

10.4.2 Triển khai Health Checks cho SQL Server và RabbitMQ qua endpoint /health 270

KẾT LUẬN CHƯƠNG 9 271

CHƯƠNG 11KIỂM THỬ VÀ ĐÓNG GÓI TRIỂN KHAI.....274

11.1 Tổng quan chiến lược kiểm thử phần mềm 274

11.1.1 Kim tự tháp kiểm thử (Test Pyramid): Unit Test, Integration Test, E2E Test 274

11.1.2 Tầm quan trọng của kiểm thử tự động trong dự án Backend phức tạp 276

11.2 Unit Testing với xUnit và FluentAssertions 277

11.2.1 Kiểm thử thuật toán Delta Engine và tái tạo Snapshot 277

11.2.2 Kiểm thử quy tắc chuyển đổi trạng thái tin đăng (Listing State Invariants) 281

11.2.3 Kiểm thử quy tắc tài chính ví (Trừ Promo trước Main, kiểm tra số dư) 284

11.3 Integration Testing với WebApplicationFactory 288

11.3.1 Thiết lập máy chủ kiểm thử In-Memory với WebApplicationFactory 288

11.3.2 Kiểm thử tích hợp luồng phân quyền và hoàn tiền Saga 289

11.4 Đóng gói ứng dụng với Docker 295

11.4.1 Tối ưu Dockerfile đa tầng (Multi-stage build) cho ASP.NET Core 295

11.4.2 Khởi chạy toàn bộ hệ thống bằng một lệnh với Docker Compose 297

11.5 CI/CD Pipeline cơ bản với GitHub Actions 300

11.5.1 Thiết lập quy trình tự động Build và Test khi tạo Pull Request 300

KẾT LUẬN CHƯƠNG 10 303

PHẦN 3: XÂY DỰNG ĐỒ ÁN TỔNG HỢP (HỆ THỐNG BẤT ĐỘNG SẢN PROPNEST)..... 304

CHƯƠNG 12PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG PROPNEST.....304

12.1 Mô tả bài toán và yêu cầu nghiệp vụ 304
12.1.1 Bối cảnh thị trường bất động sản trực tuyến.....304
12.1.2 Các yêu cầu chức năng cốt lõi (Functional Requirements) 305
12.1.3 Các yêu cầu phi chức năng (Non-Functional Requirements).....307
12.2 Thiết kế Cơ sở dữ liệu chi tiết.....309
12.2.1 Sơ đồ quan hệ thực thể (ERD) hoàn chỉnh 309
12.2.2 Đặc tả chi tiết các bảng dữ liệu, chỉ mục (Index) và ràng buộc khóa ngoại 310
12.3 Thiết kế danh mục API RESTful.....315
12.3.1 Nhóm API Xác thực và Hồ sơ (/api/v1/auth) 315
12.3.2 Nhóm API Tin đăng (/api/v1/listings) 316
12.3.3 Nhóm API Kiểm duyệt (/api/v1/moderation) 319
12.3.4 Nhóm API Ví và Mua gói (/api/v1/wallets, /purchase-package)...321
12.3.5 Nhóm API Tiến độ Workflow (/api/v1/workflows) 322
12.4 Các cơ chế kỹ thuật đặc sắc của PropNest.....323
12.4.1 Chức năng theo dõi biến động dữ liệu (History Delta Engine) 323
12.4.2 Kiểm soát đồng thời với RowVersion (Optimistic Concurrency) .325
12.4.3 Saga Orchestrator và hoàn tiền tự động.....326
KẾT LUẬN CHƯƠNG 11 329
CHƯƠNG 13TRIỂN KHAI HỆ THỐNG.....330
13.1 Giao diện tương tác API qua Swagger UI 330
13.2 Thực nghiệm trên các phân hệ Giao diện người dùng (Web UI) 332

13.2.1 Cổng thông tin tìm kiếm bất động sản dành cho Khách hàng (Client Portal) 332

13.2.2 Cổng quản lý dành cho Người bán và Nhà môi giới (Seller Portal) 334

13.2.3 Cổng quản trị và kiểm duyệt tin đăng dành cho Quản trị viên (Admin Portal) 337

13.3 Bằng chứng kiểm thử kịch bản tự động hoàn tiền giao dịch Saga và Kiểm thử tự động (Automated Testing) 339

13.3.1 Bằng chứng thực nghiệm chuỗi bồi hoàn giao dịch Saga (Compensation Flow) 339

13.3.2 Kết quả kiểm thử tự động (Automated Unit Tests) 340

KẾT LUẬN CHƯƠNG 12 341

PHẦN 4: TỔNG KẾT VÀ ĐỊNH HƯỚNG 343

CHƯƠNG 14ĐÁNH GIÁ VÀ NHÌN LẠI 343

14.1 So sánh với mục tiêu ban đầu 343

14.1.1 Tiến độ thực hiện — Bảng tổng hợp các tính năng F01 đến F08 .. 343

14.1.2 Tổng kết tỷ lệ hoàn thành đồ án 346

14.1.3 Tự đánh giá điểm mạnh và điểm yếu của giải pháp PropNest 346

14.2 Phân tích lỗi kỹ thuật quan trọng và giải pháp (Bug Reports) 347

14.2.1 Bug #1 — Xung đột ghi đè đồng thời khi hai Seller cùng sửa tin (1 account login nhiều thiết bị) 347

14.2.2 Bug #2 — Trừ tiền ví lặp do mạng chập chờn (xử lý bằng Idempotency) 348

14.2.3 Bug #3 — Mất thông điệp khi RabbitMQ khởi động lại (xử lý bằng Outbox) 348

14.3 Bài học kinh nghiệm tổng quát 349
14.3.1 Về thiết kế kiến trúc và mô hình dữ liệu.....349
14.3.2 Về kiểm thử và gỡ lỗi trong hệ thống phân tán 349
14.3.3 Về quản lý dự án và làm việc nhóm theo mô hình Agile 350
CHƯƠNG 15HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI 351
15.1 Hướng phát triển cho hệ thống PropNest 351
15.1.1 Tích hợp động cơ kiểm duyệt tự động AME (FastAPI + Ollama) 351
15.1.2 Mở rộng kiểm duyệt hình ảnh, phát hiện watermark và nội dung nhạy cảm 353
15.1.3 Tối ưu tìm kiếm nâng cao với Elasticsearch và Caching Redis 353
CHƯƠNG 16KẾT LUẬN CHUNG.....355
PHỤ LỤC..... 356
1.Nhật ký công việc và phân chia đóng góp.....356
2.Minh chứng đóng góp.....360
3.Hands On Lab 365

DANH MỤC HÌNH ẢNH

Hình 5.1 Quy trình Migration Workflow trong nhóm phát triển 110
Hình 6.1 Quy trình tiếp nhận và chuyển đổi dữ liệu từ API Layer sang Domain Layer 113
Hình 6.2 Cơ chế thực thi Controller Action và các Filter trong ASP.NET Core 129
Hình 6.3 Cơ chế chống xử lý giao dịch trùng lặp trong ASP.NET Core 133
Hình 6.4 Sơ đồ phân cấp hệ thống Exception và ánh xạ HTTP Status Code trong ASP.NET Core 140
Hình 7.1 Cấu trúc của một JWT 166
Hình 7.2 Lược đồ CSDL của User-Role 172
Hình 7.3 Sơ đồ quy trình xác thực và phân quyền truy cập tài nguyên trong ASP.NET Core 187
Hình 7.4 Phản hồi của Swagger 192
Hình 7.5 Đăng nhập bằng tài khoản Seller 193
Hình 7.6 Swagger trả về Token 194
Hình 7.7 Phản hồi thực tế từ Server 194
Hình 7.8 Đăng nhập bằng tài khoản Admin 195
Hình 7.9 Phê duyệt tin đăng 195
Hình 7.10 Phản hồi thực tế từ Server 196
Hình 7.11 Người dùng B gửi request sửa tin đăng số 6 thuộc sở Người dùng A) 197
Hình 9.1 Bộ nhớ Managed Heap trong .NET 8 242
Hình 9.2 Cấu trúc tổ chức dữ liệu của B-Tree Index 250
Hình 9.3 . Sequence Diagram của cơ chế sinh mã SQL khi cập nhật và phát hiện xung đột 256
Hình 9.4 Các ưu, nhược điểm 258
Hình 9.5 Cấu trúc biểu thức Cron 261
Hình 9.6 Sơ đồ luồng thực thi của Mô hình Xử lý mẻ nhỏ (Batch Processing / Chunking) 264

Hình 9.7 Sequence Diagram của Phân tán vết với CorrelationId và LogContext 269
Hình 9.8 Sơ đồ quy trình Health Check và giám sát trạng thái hoạt động của hệ thống 270
Hình 10.1 Mô hình Kim tự tháp kiểm thử (Test Pyramid) 275
Hình 11.1 Sơ đồ ERD hoàn chỉnh 309
Hình 11.2 Nguyên lý hoạt động và quy trình tái tạo của History Delta Engine 323
Hình 11.3 So sánh hiệu quả tiết kiệm dung lượng lưu trữ CSDL của mô hình Delta vi sai 324
Hình 11.4 Cơ chế kiểm soát xung đột đồng thời (OCC) với trường RowVersion trong SQL Server 325
Hình 11.5 Sơ đồ máy trạng thái Saga Orchestration và luồng bồi hoàn tự động 327
Hình 11.6 Mô hình Transactional Outbox đảm bảo tính toàn vẹn sự kiện phân tán 328
Hình 12.1 Giao diện Swagger UI tài liệu hóa và thực nghiệm API của hệ thống PropNest 330
Hình 12.2 Giao diện Trang chủ Cổng thông tin bất động sản PropNest (Client Portal) 332
Hình 12.3 Giao diện Tìm kiếm và Bộ lọc bất động sản nâng cao (Client Listings) 333
Hình 12.4 Giao diện Quản lý danh sách tin đăng của Người bán (Seller Properties) 334
Hình 12.5 Giao diện Quản lý ví cá nhân, số dư và nạp tiền (Seller Wallet) 335
Hình 12.6 Giao diện Theo dõi lịch sử biến động dữ liệu vi sai (Listing History Delta) 336
Hình 12.7 Giao diện Dashboard thống kê số lượng và phân bổ tin đăng bất động sản (Admin Dashboard) 337
Hình 12.8 Giao diện Hàng đợi kiểm duyệt tin đăng bất động sản (Admin Moderation Queue) 338
Hình 14.1 Kiến trúc quy trình kiểm duyệt tin đăng tự động đa tầng AME (AI Moderation Pipeline) 351
Hình 14.2 Biểu đồ đánh giá kỳ vọng hiệu năng và độ chính xác của mô hình kiểm duyệt kết hợp AI 353

DANH MỤC BẢNG

Bảng 1.1. Bảng so sánh các mốc tiến hóa kỹ thuật của .NET Core qua các thời kỳ 4
Bảng 2.1 Thang đo mức lương tham khảo của lập trình viên .NET Backend tại thị trường Việt Nam 9
Bảng 3.1 Bảng tổng hợp các công cụ, phần mềm và môi trường cần thiết để xây dựng PropNest 14
Bảng 3.2 So sánh chuyên sâu giữa kiến trúc N-Tier truyền thống, Microservices và Modular Monolith 23
Bảng 4.1 So sánh cấu hình hệ thống PropNest giữa môi trường Development và Production 54
Bảng 4.2 So sánh 3 chế độ vòng đời Service Lifetime trong ASP.NET Core 60
Bảng 4.3 Bảng danh mục Service Lifetimes của các dịch vụ trong hệ thống PropNest 61
Bảng 4.4 Thứ tự thực thi chuẩn xác của chuỗi Middleware trong PropNest 66
Bảng 4.5 So sánh toàn diện sự khác biệt bản chất giữa Middleware và Filter 70
Bảng 4.6 So sánh đối sánh giữa Controller-based Web API và Minimal API 74
Bảng 4.7 So sánh chi tiết giữa Task<IActionResult> và Task<ActionResult<T>> 77
Bảng 4.8 Bảng ánh xạ mã HTTP Status và Exception theo chuẩn RFC 7807 78
Bảng 4.9 So sánh đối sánh đa chiều giữa ASP.NET Core 8 và các nền tảng Backend hiện đại 84
Bảng 5.1 Bảng so sánh chuyên sâu giữa RDBMS (SQL Server) và NoSQL (Document/Key-Value) 92
Bảng 5.2 Bảng so sánh giữa ADO.NET, Dapper và Entity Framework Core 8 95
Bảng 6.1 So sánh chuyên sâu giữa DataAnnotations và FluentValidation 116
Bảng 6.2 So sánh chuyên sâu giữa Middleware Pipeline và Filter Pipeline 130
Bảng 6.3 Bảng ánh xạ ngoại lệ nghiệp vụ sang mã trạng thái HTTP chuẩn RFC 7807 141
Bảng 7.1 Bảng so sánh Session-based Authentication và Token-based Authentication 163

Bảng 8.1 So sánh mô hình ACID (truyền thống) và mô hình BASE (hệ thống phân tán) 204
Bảng 8.2 Bảng so sánh chi tiết giữa hai phương pháp tiếp cận Saga 207
Bảng 8.3 Trạng thái thực tế của bản ghi trong bảng OutboxMessages qua các giai đoạn 222
Bảng 8.4 Phân biệt vai trò bổ trợ giữa HTTP Idempotency và Consumer Inbox Deduplication 232
Bảng 8.5 Dữ liệu thực tế trong bảng sổ cái WalletTransactions sau khi hoàn tất luồng Compensation 236
Bảng 9.1 Bảng so sánh định lượng các chiến lược truy vấn trong EF Core 8 246
Bảng 9.2 Bảng so sánh chi tiết giữa Khóa bi quan và Khóa lạc quan 252
Bảng 9.3 Bảng tổng kết các kỹ thuật tối ưu hóa và vận hành trong hệ thống PropNest 272
Bảng 11.1 Bảng các chức năng cốt lõi (Key Features) 305
Bảng 11.2 Danh mục các Endpoint trong Nhóm API Xác thực (/api/v1/auth) 315
Bảng 11.3 Danh mục các Endpoint trong Nhóm API Tin đăng (/api/v1/listings) 316
Bảng 11.4 Danh mục các Endpoint trong Nhóm API Kiểm duyệt (/api/v1/moderation) 319
Bảng 11.5 Danh mục các Endpoint trong Nhóm API Ví và Mua gói 321
Bảng 12.1 Bảng tổng hợp kết quả thực thi bộ kiểm thử tự động (Unit Test Suite) ... 340
Bảng 13.1 Bảng đối chiếu tiến độ và mức độ hoàn thành các nhóm tính năng từ F01 đến F9 343

BẢNG PHÂN CHIA CÔNG VIỆC NHÓM

STT	MSSV	Tên thành viên	Công việc	Tiến độ
1	3122560004	Nguyễn Gia Bảo	Chương 3, 4, 8, 13 Module Wallet & Transactions + PurchasePackageSaga + Transactional Outbox/Inbox + Idempotency + Workflows	Hoàn thành
2	3122560060	Nguyễn Hoàng Phúc	Chương 1, 2, 5, 6, 9 Module: Listing (CRUD, Filter, Images) + History Delta Engine + Giao diện Client Portal & Seller Portal	Hoàn thành
3	3122410365	Đoàn Tấn Tài	Chương 7, 10, 11, 12 Module Auth (Identity, JWT, Lockout) + Moderation API & Dashboard Stats + Operations (Serilog, Quartz, Health) + Admin Portal	Hoàn thành

PHẦN 1: TỔNG QUAN VỀ CÔNG NGHỆ ASP.NET CORE

CHƯƠNG 1. GIỚI THIỆU CHUNG

1.1 Lịch sử hình thành

Trong lịch sử phát triển của các công nghệ phần mềm phía máy chủ (Server-side development), hệ sinh thái Microsoft .NET đã trải qua hơn hai thập kỷ không ngừng đổi mới để đáp ứng nhu cầu ngày càng khắt khe của các hệ thống doanh nghiệp hiện đại. Đầu những năm 2000, Microsoft giới thiệu .NET Framework cùng công nghệ ASP.NET (Active Server Pages .NET), đánh dấu bước chuyển mình vượt bậc so với ASP cổ điển nhờ việc ứng dụng ngôn ngữ biên dịch kiểu tĩnh C# và kiến trúc hướng đối tượng mạnh mẽ.

Tuy nhiên, kiến trúc ban đầu của ASP.NET truyền thống bị ràng buộc chặt chẽ với hệ điều hành Windows và máy chủ web IIS (Internet Information Services) thông qua thư viện lõi System.Web.dll. Mô hình này phụ thuộc vào chu kỳ sống HttpContext nặng nề, cơ chế lưu trạng thái ViewState phức tạp và tiêu tốn nhiều tài nguyên bộ nhớ. Khi điện toán đám mây (Cloud Computing), kiến trúc hướng dịch vụ (Microservices) và công nghệ container hóa (Docker) bùng nổ vào đầu thập niên 2010, các hệ thống backend đòi hỏi tính đa nền tảng (cross-platform), thời gian khởi động tức thì, dung lượng bộ nhớ tối thiểu và khả năng mở rộng quy mô linh hoạt trên môi trường Linux.

Nhận thức rõ bước ngoặt công nghệ đó, dưới sự dẫn dắt của CEO Satya Nadella, Microsoft đã khởi xướng một cuộc cách mạng tái thiết kế toàn diện nền tảng .NET. Thay vì nâng cấp chắp vá trên nền tảng cũ, một framework mới hoàn toàn được kiến trúc lại từ con số 0 với tên gọi ban đầu là ASP.NET 5, sau đó chính thức đổi tên thành ASP.NET Core và phát hành phiên bản 1.0 vào tháng 6 năm 2016. Khác biệt cốt lõi của ASP.NET Core là:

- **Mã nguồn mở và hoàn toàn miễn phí:** Được quản lý minh bạch bởi tổ chức phi lợi nhuận .NET Foundation và cộng đồng toàn cầu trên GitHub.
- **Đa nền tảng thực thụ (Cross-Platform):** Khả năng thực thi với hiệu năng tối ưu trên cả Linux, macOS và Windows, hoàn toàn thoát khỏi sự ràng buộc độc quyền với IIS.
- **Hiệu năng cực cao (High Performance):** Tái cấu trúc pipeline xử lý request theo cơ chế bất đồng bộ non-blocking I/O dựa trên máy chủ web nội hợp Kestrel Server siêu nhẹ, đưa ASP.NET Core vươn lên nhóm dẫn đầu trong các bài đánh giá hiệu năng TechEmpower.
- **Thiết kế Module hóa và Dependency Injection bản xứ:** Mọi thành phần đều là các gói NuGet độc lập, chỉ nạp đúng những gì ứng dụng cần, đồng thời tích hợp sẵn IoC/DI Container ngay từ tầng lõi.

1.2 Sự thay đổi qua các phiên bản

Kể từ thời điểm ra mắt lần đầu năm 2016 đến nay, ASP.NET Core đã liên tục tiến hóa với chu kỳ phát hành định kỳ hàng năm rất rõ ràng và kỷ luật, bao gồm các phiên bản hỗ trợ tiêu chuẩn (STS - Standard Term Support) và hỗ trợ dài hạn (LTS - Long Term Support). Quá trình phát triển này có thể chia thành ba giai đoạn chính:

1.2.1 Các phiên bản đầu từ 2016 đến 2018 (Giai đoạn kiến thiết nền tảng)

Giai đoạn này chứng kiến sự ra đời và khẳng định giá trị của ASP.NET Core 1.0 (2016), 2.0 (2017) và bản LTS đầu tiên 2.1 (2018):

- **ASP.NET Core 1.0 (2016):** Đặt nền móng với kiến trúc Middleware Pipeline, máy chủ Kestrel, loại bỏ hoàn toàn web.config truyền thống và thay bằng hệ thống cấu hình hiện đại appsettings.json hỗ trợ đa môi trường.
- **ASP.NET Core 2.0 (2017):** Giới thiệu gói tổng hợp Microsoft.AspNetCore.All/App giúp đơn giản hóa việc quản lý phụ thuộc, đồng

thời ra mắt công nghệ Razor Pages phục vụ việc lập trình web hướng trang bên cạnh Web API và MVC truyền thống.

- **ASP.NET Core 2.1 LTS (2018):** Đạt độ ổn định cấp doanh nghiệp với sự xuất hiện của HttpClientFactory giải quyết triệt để lỗi cạn kiệt cổng mạng (Socket Exhaustion), hỗ trợ chính thức giao thức HTTP/2, cơ chế xác thực GDPR-compliant và tính năng SignalR Core thời gian thực.

1.2.2 Giai đoạn mở rộng và thống nhất từ 2019 đến 2021

Đây là giai đoạn bước ngoặt lịch sử khi Microsoft thực hiện chiến lược hợp nhất toàn bộ các nhánh .NET rải rác (.NET Framework, .NET Core, Xamarin) thành một nền tảng duy nhất:

- **.NET Core 3.0 & 3.1 LTS (2019):** Bổ sung hỗ trợ giao thức truyền thông hiệu năng cao gRPC, Endpoint Routing thống nhất cơ chế định tuyến giữa Controller và Middleware, tích hợp thư viện JSON thuần hiệu năng cao System.Text.Json thay thế cho Newtonsoft.Json.
- **.NET 5 (2020):** Chính thức lược bỏ chữ 'Core' trong tên gọi để khẳng định đây là tương lai duy nhất của toàn bộ hệ sinh thái .NET. Phiên bản này đem lại sự cải tiến vượt bậc về thuật toán thu gom rác (Garbage Collection), giảm thiểu chi phí cấp phát bộ nhớ (Allocation-free APIs).
- **.NET 6 LTS (2021):** Cột mốc hiện đại hóa phong cách lập trình với C# 10, ra mắt tính năng Minimal APIs giúp viết mã nguồn gọn nhẹ tương đương Express.js hay FastAPI; bổ sung Hot Reload, hỗ trợ HTTP/3 và kiến trúc bộ nhớ Arm64.

1.2.3 Giai đoạn trưởng thành và hiện đại hóa từ 2022 đến nay

Từ phiên bản .NET 7 (2022), .NET 8 LTS (2023) và phiên bản mới nhất .NET 9 (2024), nền tảng ASP.NET Core tập trung mạnh mẽ vào điện toán đám mây bản xứ (Cloud-Native) và biên dịch mã máy trực tiếp:

- **Native AOT (Ahead-of-Time Compilation):** Cho phép biên dịch trực tiếp mã nguồn C# thành mã nhị phân máy tính thuần túy mà không cần nạp JIT compiler hay runtime nặng nề. Ứng dụng khởi động trong vài mili-giây, dung lượng container giảm xuống chỉ còn vài chục megabyte.
- **.NET 8 LTS:** Phiên bản nền tảng chính được lựa chọn cho dự án PropNest. .NET 8 mang đến sự nâng cấp toàn diện cho ASP.NET Core Identity API, cơ chế kiểm soát số lượng yêu cầu (Rate Limiting) tích hợp sẵn, cùng các tính năng tối ưu hóa truy vấn trong Entity Framework Core 8.
- **.NET 9 và .NET Aspire:** Cung cấp khung điều phối Cloud-Native chuẩn mực (Telemetry, Service Discovery, Health Checks, OpenTelemetry) phục vụ cho các hệ thống phân tán quy mô lớn.

Bảng 1.1. Bảng so sánh các mốc tiến hóa kỹ thuật của .NET Core qua các thời kỳ

Phiên bản	Năm phát hành	Loại vòng đời	Các cải tiến kỹ thuật nổi bật
ASP.NET Core 1.0	2016	Current	Kiến trúc mới hoàn toàn, Kestrel Server, đa nền tảng, DI Container tích hợp.
ASP.NET Core 2.1	2018	LTS	HttpClientFactory, SignalR Core, hỗ trợ HTTP/2, tích hợp chuẩn OpenAPI/Swagger.
.NET Core 3.1	2019	LTS	gRPC tích hợp sâu, Endpoint Routing, thư viện tuần tự hóa System.Text.Json.
.NET 6.0	2021	LTS	Minimal APIs, Hot Reload, C# 10, hỗ trợ HTTP/3, tối ưu hóa bộ nhớ cho ARM64.
.NET 8.0	2023	LTS (Chọn cho PropNest)	Native AOT hoàn chỉnh, cải tiến EF Core 8, Identity Endpoints, TimeProvider.

.NET 9.0	2024	Standard	Tối ưu hóa Server GC đa luồng, cải tiến OpenAPI native, .NET Aspire integration.

1.3 Hệ sinh thái ASP.NET Core

Sự thành công của ASP.NET Core không chỉ đến từ trình biên dịch mạnh mẽ mà còn nhờ vào một hệ sinh thái toàn diện và nhất quán, được cấu thành từ ba trụ cột chính:

1.3.1 Các thành phần cốt lõi

- **CoreCLR / .NET Runtime:** Máy ảo thực thi mã trung gian (IL - Intermediate Language) với bộ thu gom rác thế hệ mới (Generational Garbage Collector) tối ưu cho các luồng xử lý song song cao.
- **Kestrel Web Server:** Kestrel Web Server là web server đa nền tảng, hướng sự kiện, bất đồng bộ và có hiệu năng cực cao được tích hợp sẵn làm máy chủ mặc định trong ASP.NET Core. Về mặt lịch sử kiến trúc, trong các phiên bản đầu tiên (ASP.NET Core 1.x - 2.x), Kestrel sử dụng thư viện C-native libuv để quản lý vòng lặp sự kiện mạng (Event Loop). Tuy nhiên, kể từ phiên bản ASP.NET Core 3.0 trở đi cho đến .NET 8 hiện tại, Microsoft đã loại bỏ hoàn toàn sự phụ thuộc vào libuv và thay thế bằng SocketTransport engine thuần managed dựa trên System.Net.Sockets kết hợp với System.IO.Pipelines. Sự thay đổi mang tính đột phá này giúp Kestrel tận dụng tối đa cơ chế quản lý bộ nhớ không phân mảnh (Memory Pooling với ArrayPool<byte>), loại bỏ chi phí chuyển ngữ cảnh (P/Invoke context switch) giữa C# và C-native, giúp máy chủ đạt thông lượng hàng trăm nghìn request/giây với mức tiêu thụ tài nguyên tối thiểu.
- **Generic Host & WebApplication:** Mô hình khởi tạo ứng dụng hiện đại cung cấp cấu hình tập trung cho cấu hình (Configuration), ghi nhật ký (Logging) và quản lý vòng đời ứng dụng.

- **Middleware Pipeline:** Chuỗi các khối logic xử lý HTTP request và response nối tiếp nhau theo mô hình đường ống (Pipeline), cho phép kiểm soát luồng dữ liệu cực kỳ chặt chẽ.
- **Inversion of Control (IoC) Container:** Cơ chế Dependency Injection tích hợp sẵn tại tầng nền tảng, hỗ trợ ba vòng đời dịch vụ chuẩn: Transient, Scoped và Singleton.

1.3.2 Các package và framework chính thức

- **Entity Framework Core (EF Core):** ORM hiện đại, hỗ trợ ánh xạ quan hệ (Object-Relational Mapping), theo dõi trạng thái thực thể (Change Tracking), di chuyển schema tự động (Migrations) và tối ưu truy vấn LINQ.
- **ASP.NET Core Identity:** Khung bảo mật hoàn chỉnh cung cấp sẵn cơ chế quản lý người dùng, phân quyền vai trò (Roles), mã hóa mật khẩu chuẩn PBKDF2, quản lý lockout và cấp phát token.
- **Microsoft.AspNetCore.Authentication.JwtBearer:** Thư viện xử lý xác thực không trạng thái thông qua JSON Web Token (JWT), hỗ trợ cấu hình kiểm tra chữ ký, thời hạn và audience khắt khe.
- **SignalR:** Framework thời gian thực hai chiều (Full-duplex) qua WebSockets, Server-Sent Events hoặc Long Polling.

1.3.3 Công cụ hỗ trợ phát triển

- **.NET CLI (Command-Line Interface):** Bộ công cụ dòng lệnh mạnh mẽ (dotnet build, dotnet run, dotnet test, dotnet ef...) cho phép phát triển và tự động hóa CI/CD nhất quán trên mọi hệ điều hành.
- **Visual Studio 2022 & VS Code:** Môi trường phát triển tích hợp (IDE) hàng đầu với khả năng phân tích mã nguồn thời gian thực Roslyn, trình gỡ lỗi (debugger) xuất sắc và tiện ích mở rộng C# Dev Kit.
- **NuGet:** Hệ thống quản lý thư viện và gói phụ thuộc toàn cầu của .NET, hỗ trợ cơ chế khóa phiên bản tập trung (Central Package Management).

KẾT LUẬN CHƯƠNG 1

Chương 1 đã phác họa bức tranh toàn cảnh về sự tiến hóa của nền tảng .NET: từ một framework đóng kín trên Windows trở thành nền tảng mã nguồn mở, đa nền tảng và dẫn đầu về hiệu năng hàng đầu thế giới hiện nay. Sự kết hợp giữa máy chủ Kestrel siêu tốc, cơ chế Dependency Injection nội hợp, đường ống Middleware linh hoạt và hệ sinh thái thư viện chuẩn mực tạo nên bệ phóng vững chắc để xây dựng các giải pháp backend hướng doanh nghiệp như hệ thống PropNest.

CHƯƠNG 2 CƠ HỘI NGHỀ NGHIỆP VÀ THỊ TRƯỜNG

2.1 Các vị trí công việc

Nhờ tính bảo mật cao, khả năng mở rộng tin cậy và sự hậu thuẫn vững chắc từ tập đoàn Microsoft, các kỹ sư phần mềm làm chủ nền tảng ASP.NET Core luôn có thị trường việc làm rộng mở với lộ trình phát triển nghề nghiệp đa dạng:

- **.NET Backend Developer:** Chịu trách nhiệm thiết kế và hiện thực hóa các dịch vụ web, RESTful APIs, cơ sở dữ liệu quan hệ (SQL Server, PostgreSQL), xử lý logic nghiệp vụ và tối ưu hóa hiệu năng truy vấn.
- **Fullstack .NET Developer:** Kết hợp sức mạnh backend của ASP.NET Core với các framework frontend hiện đại như React, Angular, Vue.js hoặc Blazor để xây dựng các giải pháp ứng dụng trọn gói.
- **Cloud / Microservices Engineer:** Chuyên trách thiết kế các hệ thống phân tán, triển khai container (Docker, Kubernetes), tích hợp điện toán đám mây Microsoft Azure hoặc AWS và quản lý Message Broker (RabbitMQ, Kafka).
- **Solution Architect / Technical Lead:** Định hình kiến trúc tổng thể (Clean Architecture, DDD, Modular Monolith, Event-Driven), thiết lập chuẩn mực code, bảo đảm tính toàn vẹn bảo mật và dẫn dắt đội ngũ kỹ thuật trong các dự án quy mô lớn.

2.2 Nhu cầu tuyển dụng

Thị trường công nghệ thông tin trong nước và quốc tế hiện nay chứng kiến nhu cầu ổn định và bền vững đối với các vị trí lập trình viên .NET, đặc biệt là trong các lĩnh vực có yêu cầu khắt khe về độ chính xác và an toàn dữ liệu:

- **Khối Tài chính – Ngân hàng và Bảo hiểm (Fintech & BFSI):** Hầu hết các ngân hàng thương mại lớn tại Việt Nam (Vietcombank, Techcombank, MB Bank, VPBank...) và các công ty chứng khoán đều xây dựng hệ thống Core Banking và giao

dịch tài chính trên nền tảng .NET nhờ tính ổn định tuyệt đối và cơ chế bảo mật nghiêm ngặt.

- **Khối Doanh nghiệp Bất động sản và Thương mại:** Các hệ thống quản trị dữ liệu lớn, quản lý tin đăng, sàn giao dịch và hoạch định tài nguyên doanh nghiệp (ERP/CRM) yêu cầu xử lý giao dịch đồng thời và tính nhất quán dữ liệu cao.
- **Các công ty gia công phần mềm hàng đầu (Global IT Outsourcing):** FPT Software, KMS Technology, NashTech, TMA Solutions liên tục tuyển dụng kỹ sư .NET để thực hiện các dự án cho khách hàng tại thị trường Bắc Mỹ, Châu Âu, Singapore và Nhật Bản.

2.3 Mức lương tham khảo

Dựa trên các báo cáo thị trường nhân lực IT thường niên uy tín tại Việt Nam (như TopDev IT Market Report và VietnamWorks IT Salary Guide), lập trình viên .NET luôn nằm trong nhóm có thu nhập cao và ổn định. Thang đo mức lương tham khảo theo từng cấp bậc chuyên môn được tổng hợp như sau:

Bảng 2.1 Thang đo mức lương tham khảo của lập trình viên .NET Backend tại thị trường Việt Nam

Cấp bậc (Level)	**Kinh nghiệm yêu cầu**	**Mức lương Net (VND/tháng)**	**Yêu cầu năng lực chuyên môn cốt lõi**
Fresher / Intern	< 1 năm	8.000.000 – 13.000.000	Nắm vững C#, OOP, cú pháp LINQ, kiến thức cơ bản về ASP.NET Core API và SQL Server.
Junior Developer	1 – 3 năm	14.000.000 – 24.000.000	Thành thạo RESTful API, EF Core, Authentication (JWT), Clean Architecture, viết Unit Tests.

Mid-Level Developer	3 – 5 năm	25.000.000 – 38.000.000	Làm chủ tối ưu hóa CSDL, Caching (Redis), Message Queue (RabbitMQ), Docker, CI/CD.
Senior Developer	5 – 8 năm	40.000.000 – 65.000.000	Thiết kế kiến trúc hệ thống phân tán, xử lý Concurrency, Saga, bảo mật chuyên sâu, tuning GC.
Tech Lead / Architect	> 8 năm	70.000.000 – 120.000.000+	Định hình chiến lược kiến trúc tổng thể, ra quyết định công nghệ, cố vấn cấp cao.

KẾT LUẬN CHƯƠNG 2

Thị trường lao động ngành phần mềm minh chứng rằng kỹ năng làm chủ ASP.NET Core không chỉ là một lợi thế cạnh tranh mạnh mẽ cho sinh viên tốt nghiệp mà còn là nền tảng để xây dựng sự nghiệp kỹ thuật lâu dài. Việc nắm vững các nguyên lý kiến trúc backend, cơ chế quản lý bộ nhớ và các bài toán giao dịch phức tạp thông qua đồ án PropNest sẽ giúp sinh viên tự tin đáp ứng các tiêu chuẩn tuyển dụng khắt khe của các tập đoàn công nghệ lớn.

PHẦN 2: NỘI DUNG CỐT LÕI VÀ THỰC HÀNH

CHƯƠNG 3 CÀI ĐẶT MÔI TRƯỜNG VÀ "HELLO WORLD"

Để xây dựng hệ thống Backend API quản lý và đăng tin bất động sản PropNest bằng công nghệ ASP.NET Core, trước tiên cần thiết lập môi trường phát triển toàn diện, chuẩn mực và đồng bộ giữa tất cả các thành viên trong nhóm. Chương này hướng dẫn chi tiết từ việc chuẩn bị các công cụ, phân tích kiến trúc thư mục đa tầng (Clean Architecture), cho đến bài thực hành cấu hình và khởi chạy thành công ứng dụng ban đầu.

3.1 Các công cụ cần thiết để bắt đầu

Dự án PropNest sử dụng các công cụ lập trình hiện đại và được thực tiễn doanh nghiệp kiểm chứng:

3.1.1 .NET SDK và dotnet CLI

.NET SDK (Software Development Kit) phiên bản 8.0 LTS là nền tảng cốt lõi chứa trình biên dịch Roslyn, máy ảo CoreCLR và thư viện cơ sở BCL. Đi kèm với SDK là công cụ dòng lệnh dotnet CLI – trung tâm điều khiển mọi thao tác từ tạo project, quản lý gói NuGet, biên dịch, chạy ứng dụng đến quản lý migration cơ sở dữ liệu.

3.1.2 C# 12 và các tính năng ngôn ngữ hiện đại

Ngôn ngữ C# 12 mang đến những cú pháp hiện đại giúp viết mã nguồn trong sáng, an toàn và tối ưu bộ nhớ:

- **Primary Constructors:** Cho phép khai báo hàm khởi tạo trực tiếp trên định nghĩa class hoặc struct, giảm thiểu đáng kể mã lặp khi tiêm phụ thuộc (Dependency Injection).
- **Record Types:** Đối tượng bất biến (Immutable Data) lý tưởng để định nghĩa các Data Transfer Object (DTO), Domain Event và Message Payload.

- **Pattern Matching & Switch Expressions:** Hỗ trợ kiểm tra kiểu dữ liệu và giá trị biểu thức logic một cách ngắn gọn, giảm tải câu lệnh if-else lồng nhau phức tạp.
- **Nullable Reference Types:** Phát hiện và ngăn chặn triệt để lỗi tham chiếu rỗng (NullReferenceException) ngay từ thời điểm biên dịch.

3.1.3 Visual Studio 2022 và Visual Studio Code

Nhóm khuyến nghị hai môi trường phát triển chính:

- Visual Studio 2022 (Community / Professional): IDE hoàn chỉnh nhất cho .NET trên Windows với công cụ phân tích bộ nhớ (Diagnostic Tools), quản lý Solution Explorer trực quan, trình kiểm thử xUnit tích hợp và công cụ thiết kế CSDL mạnh mẽ.
- **Visual Studio Code kết hợp C# Dev Kit:** Lựa chọn gọn nhẹ, đa nền tảng (chạy tốt trên cả macOS và Linux), hỗ trợ IntelliSense thông minh và tích hợp Git mượt mà.

3.1.4 Microsoft SQL Server

Microsoft SQL Server 2022 đóng vai trò là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) chính của hệ thống PropNest – nơi lưu giữ 'nguồn sự thật' (System of Record). SQL Server đảm bảo đầy đủ các tính chất ACID (Atomicity, Consistency, Isolation, Durability) cho các giao dịch tài chính nhạy cảm trong ví người dùng, quản lý xung đột dữ liệu bằng kiểu dữ liệu RowVersion (Timestamp) và hỗ trợ lưu trữ cột JSON linh hoạt.

3.1.5 Entity Framework Core và EF Core Tools

Entity Framework Core (EF Core 8) là ORM hiện đại giúp ánh xạ các thực thể C# sang bảng CSDL quan hệ. Công cụ dòng lệnh dotnet-ef cho phép nhóm quản lý các phiên bản cấu trúc dữ liệu thông qua cơ chế Migration, lưu vết và tự động cập nhật schema trên các môi trường một cách nhất quán.

3.1.6 Docker và Docker Compose

Docker cho phép ảo hóa và đóng gói các dịch vụ phụ trợ vào các container độc lập. Dự án PropNest cung cấp sẵn file docker-compose.yml trong thư mục deploy/ để chỉ với một câu lệnh, toàn bộ môi trường SQL Server 2022 và RabbitMQ 3 Management sẽ được khởi chạy đồng bộ mà không cần cài đặt thủ công vào hệ điều hành của từng thành viên.

3.1.7 Message Broker: RabbitMQ và MassTransit

RabbitMQ là hệ thống hàng đợi thông điệp (Message Broker) nguồn mở phổ biến nhất hiện nay, đóng vai trò làm kênh truyền thông bất đồng bộ giữa tiến trình API và tiến trình nền Workflow Worker. Thư viện MassTransit trong .NET đóng vai trò là Service Bus trừu tượng hóa việc cấu hình hàng đợi, cung cấp sẵn cơ chế Saga State Machine, tự động Retry, Dead Letter Queue và đảm bảo xử lý thông điệp bền vững.

3.1.8 Swagger UI (OpenAPI) và Postman

Swagger UI được tích hợp sẵn thông qua package Swashbuckle.AspNetCore, tự động sinh tài liệu API tương tác trực tiếp trên trình duyệt, hỗ trợ cấu hình Security Definition Bearer JWT để kiểm thử các endpoint có yêu cầu bảo mật. Postman được sử dụng để xây dựng các kịch bản kiểm thử API tự động theo kịch bản.

3.1.9 Git và GitHub

Toàn bộ mã nguồn PropNest được quản lý phiên bản qua Git và lưu trữ trên GitHub. Các quy tắc cam kết (Commit convention), rẽ nhánh tính năng (Feature branches) và tạo Pull Request (PR) được thực hiện nghiêm ngặt để đảm bảo chất lượng mã nguồn trước khi tích hợp vào nhánh chính.

3.1.10 Bảng tóm tắt công cụ cần cài đặt

Bảng 3.1 Bảng tổng hợp các công cụ, phần mềm và môi trường cần thiết để xây dựng PropNest

Công cụ	Phiên bản	Mục đích sử dụng chính trong PropNest
.NET SDK	8.0.x LTS	Nền tảng biên dịch, thực thi ứng dụng và quản lý CLI.
C# Compiler	C# 12.0	Ngôn ngữ lập trình chính cho toàn bộ backend.
MS SQL Server	2022 Express / Dev	Lưu trữ dữ liệu quan hệ (Users, Listings, Wallets, Saga).
EF Core Tools	8.0.21	Công cụ dotnet-ef tạo và áp dụng database migrations.
Docker Desktop	Mới nhất	Chạy SQL Server và RabbitMQ container cục bộ.
RabbitMQ	3-management	Message Broker phục vụ giao dịch phân tán và Outbox.
MassTransit	8.x	Framework Service Bus và Saga State Machine Orchestrator.
VS Code / VS 2022	2022 / v1.9x	Môi trường phát triển tích hợp (IDE).
Postman / Swagger	OpenAPI 3.0	Tài liệu hóa và kiểm thử giao diện RESTful API.

3.2 Cài đặt và khởi tạo dự án

Quá trình thiết lập dự án PropNest từ đầu được thực hiện tuần tự theo các bước chuẩn mực như sau:

- **Bước 1 – Kiểm tra môi trường .NET:** Mở Terminal và thực thi lệnh dotnet --version để đảm bảo phiên bản .NET 8.0 đã được cài đặt chính xác.

Lệnh xác minh môi trường .NET SDK qua Terminal

```
> dotnet --version
8.0.404
> dotnet --info
.NET SDK:
 Version:          8.0.404
 Commit:           4677764951
Runtime Environment:
 OS Name:          Windows
 OS Version:       10.0.22631
```

- **Bước 2 – Khởi tạo Solution và các Project thành phần:** Sử dụng dotnet CLI để tạo tệp giải pháp PropNest.sln cùng cấu trúc 5 project theo đúng nguyên tắc Clean Architecture:

Chuỗi lệnh CLI khởi tạo Solution và cấu trúc các Project của PropNest

```
# Tạo file Solution
dotnet new sln -n PropNest


# Tạo các project theo từng tầng kiến trúc
dotnet new webapi -n PropNest.Api -o src/PropNest.Api
dotnet new classlib -n PropNest.Application -o src/PropNest.Application
```

```
dotnet new classlib -n PropNest.Domain -o src/PropNest.Domain

dotnet new classlib -n PropNest.Infrastructure -o src/PropNest.Infrastructure

dotnet new console -n PropNest.Workflow.Worker -o src/PropNest.Workflow.Worker


# Tạo project kiểm thử

dotnet new xunit -n PropNest.Domain.Tests -o tests/PropNest.Domain.Tests


# Thêm các project vào Solution

dotnet sln add src/PropNest.Api/PropNest.Api.csproj

dotnet sln add src/PropNest.Application/PropNest.Application.csproj

dotnet sln add src/PropNest.Domain/PropNest.Domain.csproj

dotnet sln add src/PropNest.Infrastructure/PropNest.Infrastructure.csproj

dotnet sln add src/PropNest.Workflow.Worker/PropNest.Workflow.Worker.csproj

dotnet sln add tests/PropNest.Domain.Tests/PropNest.Domain.Tests.csproj
```

- **Bước 3 – Thiết lập Quản lý phiên bản tập trung (Central Package Management):** Để tránh tình trạng mỗi project tham chiếu một phiên bản thư viện khác nhau gây xung đột version, dự án PropNest kích hoạt tính năng Central Package Management tại gốc repository thông qua hai file Directory.Build.props và Directory.Packages.props:

Nội dung file Directory.Build.props chuẩn hóa cấu hình cho toàn bộ Solution

```
<!-- Directory.Build.props (Đặt tại thư mục gốc Solution - Dùng chung cho Mục 3.2 và 4.6.2) -->
 <Project>
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
```

```
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <AnalysisLevel>latest-recommended</AnalysisLevel>
        <!-- Bỏ qua cảnh báo CA1861 đối với việc cấp phát mảng hằng số trong các biểu thức Linq/Assert -->
    <NoWarn>$(NoWarn);CA1861</NoWarn>
  </PropertyGroup>
</Project>
```

Nội dung file Directory.Packages.props khóa cứng phiên bản các gói NuGet

```
<!-- Directory.Packages.props (Đặt tại thư mục gốc Solution - Dùng chung cho Mục 3.2 và 4.6.1) -->
 <Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
    <CentralPackageTransitivePinningEnabled>true</CentralPackageTransitivePinningEnabled>
  </PropertyGroup>

  <ItemGroup>
        <!-- Nhóm 1: ASP.NET Core, Identity & Security (.NET 8.0 LTS) -->
        <PackageVersion Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="8.0.21" />
        <PackageVersion Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore" Version="8.0.21"
/>
        <PackageVersion Include="Microsoft.Extensions.Identity.Stores" Version="8.0.21" />
        <PackageVersion Include="System.IdentityModel.Tokens.Jwt" Version="7.7.1" />

        <!-- Nhóm 2: Entity Framework Core 8 & SQL Server -->
        <PackageVersion Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.21" />
```

```
        <PackageVersion Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.21" />
        <PackageVersion Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.21" />
        <PackageVersion Include="Microsoft.EntityFrameworkCore.Sqlite" Version="8.0.21" />

        <!-- Nhóm 3: Message Bus & Distributed Saga (MassTransit) -->
        <PackageVersion Include="MassTransit.RabbitMQ" Version="8.2.5" />
        <PackageVersion Include="MassTransit.EntityFrameworkCore" Version="8.2.5" />

        <!-- Nhóm 4: Logging & Diagnostics (Serilog) -->
        <PackageVersion Include="Serilog.AspNetCore" Version="8.0.3" />
        <PackageVersion Include="Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore"
Version="8.0.21" />

        <!-- Nhóm 5: Tác vụ nền & Thẩm định dữ liệu -->
        <PackageVersion Include="Quartz.Extensions.Hosting" Version="3.13.1" />
        <PackageVersion Include="FluentValidation.AspNetCore" Version="11.3.0" />
        <PackageVersion Include="Swashbuckle.AspNetCore" Version="6.6.2" />
        <PackageVersion Include="CloudinaryDotNet" Version="1.26.2" />

        <!-- Nhóm 6: Testing Frameworks (xUnit & Moq) -->
        <PackageVersion Include="Microsoft.NET.Test.Sdk" Version="17.11.1" />
        <PackageVersion Include="xunit" Version="2.9.2" />
        <PackageVersion Include="xunit.runner.visualstudio" Version="2.8.2" />
        <PackageVersion Include="Moq" Version="4.20.72" />
  </ItemGroup>
 </Project>
```

3.3 Phân tích cấu trúc thư mục của dự án PropNest

Cấu trúc thư mục của PropNest được thiết kế khoa học, tuân thủ chặt chẽ các nguyên lý của Clean Architecture và trường phái thiết kế hướng miền (Domain-Driven Design - DDD).

3.3.1 Cây thư mục dự án tổng quan

Sơ đồ cây thư mục tổng quan của giải pháp PropNest

```
PropNest/
├── Directory.Build.props          # Cấu hình chung (.NET 8, Nullable, Warnings)
├── Directory.Packages.props       # Quản lý tập trung phiên bản các gói NuGet
├── PropNest.slnx                  # Tệp Solution quản lý các project
├── deploy/
│   └── docker-compose.yml         # Container SQL Server 2022 và RabbitMQ
├── docs/                          # Tài liệu kiến trúc và hướng dẫn kỹ thuật
│   ├── PropNest_Architecture_and_Implementation_Blueprint.docx
│   └── PropNest_Sprint_Plan.docx
├── src/
│   ├── PropNest.Domain/           # Entities, Value Objects, Domain Events, Enums
│   ├── PropNest.Application/      # Commands, Queries, DTOs, Validators, Interfaces
│   ├── PropNest.Infrastructure/   # EF Core DbContext, Repositories, Outbox, Identity
│   ├── PropNest.Api/              # Controllers, Middlewares, Filters, Program.cs
│   └── PropNest.Workflow.Worker/  # MassTransit Saga State Machine, Quartz Jobs
└── tests/
    └── PropNest.Domain.Tests/     # Kiểm thử đơn vị cho Delta Engine và Domain Rules
```

3.3.2 Tổ chức code Backend theo từng lớp

Hệ thống áp dụng quy tắc phụ thuộc một chiều (Dependency Inversion Principle) – các tầng bên ngoài phụ thuộc vào tầng bên trong, tầng Domain tuyệt đối không có bất kỳ phụ thuộc nào vào các tầng khác:

3.3.2.1 Domain Layer (src/PropNest.Domain)

Là trái tim của toàn bộ ứng dụng, chứa đựng các khái niệm nghiệp vụ cốt lõi, hoàn toàn độc lập với công nghệ cơ sở dữ liệu hay HTTP framework:

- **Entities:** Listing (Tin bất động sản), Wallet (Ví thanh toán), WalletTransaction (Giao dịch sổ cái), ListingHistory (Lịch sử biến động), ListingModerationReview (Bằng chứng kiểm duyệt).

- **ApplicationUser:** Kế thừa IdentityUser<long> thông qua gói siêu nhẹ Microsoft.Extensions.Identity.Stores, cho phép lưu trữ khóa chính dạng BIGINT và tương thích trọn vẹn với Identity.
- **Enums & Invariants:** ListingStatus, ModerationDecision, TransactionType, PackageKind. Các quy tắc bất biến (Invariants) được đóng gói trực tiếp bên trong các phương thức của Entity (ví dụ: Listing.UpgradePackage, Wallet.Charge, Wallet.Refund).

3.3.2.2 Application Layer (src/PropNest.Application)

Lớp điều phối các use case của hệ thống, định nghĩa hợp đồng giao tiếp mà không quan tâm chi tiết hiện thực bên dưới:

- **Abstractions:** ICurrentUser (lấy thông tin người dùng đang đăng nhập từ HttpContext), IUnitOfWork (cam kết giao dịch CSDL), IOutboxDispatchService (phát sự kiện Outbox).
- **Services & DTOs:** ListingService, ListingRequests, ListingResponses, WorkflowService.
- **ListingDelta:** Chứa thuật toán so sánh dữ liệu (Delta Engine), phát hiện các trường bị sửa đổi giữa phiên bản cũ và mới để sinh mảng JSON diff cực kỳ gọn nhẹ.

3.3.2.3 Infrastructure Layer (src/PropNest.Infrastructure)

Hiện thực hóa các giao diện đã định nghĩa ở tầng Application bằng các công nghệ cụ thể:

- **Persistence:** Lớp PropNestDbContext kế thừa IdentityDbContext<ApplicationUser, IdentityRole<long>, long> và triển khai IUnitOfWork.
- **Fluent API Configurations:** Cấu hình bảng dữ liệu, khóa chính BIGINT IDENTITY, precision cho tiền tệ DECIMAL(18,2), mapping quan hệ 1-N và trường RowVersion phát hiện xung đột đồng thời.

- **Outbox & Identity Services:** Lưu trữ OutboxMessages và xử lý mã hóa mật khẩu, kiểm tra quyền hạn.

3.3.2.4 Api Layer (src/PropNest.Api)

Lớp ngoài cùng đón nhận các HTTP Request, xử lý xác thực và phân quyền:

- **Controllers:** ListingsController, WorkflowsController, ModerationController, AuthController.
- **Middlewares: ExceptionHandlingMiddleware** (chuyển đổi ngoại lệ thành chuẩn **RFC 7807 ProblemDetails**), **CorrelationIdMiddleware** (gắn mã định danh duy nhất vào từng request để theo vết log xuyên suốt).
- **Program.cs:** Điểm khởi đầu (Entry Point) của ứng dụng theo mô hình Minimal Hosting hiện đại.

3.3.2.5 Workflow Worker (src/PropNest.Workflow.Worker)

Tiến trình nền (Background Worker Process) chạy song song với API. Worker đảm nhiệm việc lắng nghe các thông điệp từ RabbitMQ, điều phối quy trình mua gói thông qua MassTransit Saga State Machine, kích hoạt hành động hoàn tiền (Compensation Refund) khi có lỗi và chạy Quartz Job tự động chuyển trạng thái tin đăng sang Expired khi hết hạn.

3.3.3 Tại sao chọn kiến trúc Modular Monolith kết hợp Clean Architecture?

Việc lựa chọn mô hình kiến trúc cho PropNest được cân nhắc kỹ lưỡng dựa trên yêu cầu học phần và thực tiễn kỹ thuật doanh nghiệp:

Bảng 3.2 So sánh chuyên sâu giữa kiến trúc N-Tier truyền thống, Microservices và Modular Monolith

Tiêu chí đánh giá	N-Tier truyền thống	Microservices Architecture	Modular Monolith (PropNest)
Ranh giới module	Kém (dễ phụ thuộc vòng)	Rất cao (tách biệt vật lý)	Cao (tách biệt theo contracts logic).
Độ phức tạp triển khai	Rất thấp (đơn khối)	Rất cao (cần K8s, Gateway, Mesh)	Thấp (chạy chung codebase và DB).
Độ trễ mạng (Latency)	Không có (gọi in-process)	Cao (nhiều network hops HTTP/gRPC)	Rất thấp (in-memory hoặc broker cục bộ).
Tính nhất quán dữ liệu	Cao (ACID Transaction)	Kém (Eventual Consistency phức tạp)	Rất cao (ACID cho Write, Saga cho Event).
Khả năng bóc tách sau này	Khó khăn (mã nguồn dính kết)	Đã bóc tách sẵn	Rất dễ dàng (bóc tách từng module khi cần).

Lý do lựa chọn cho dự án PropNest: Kiến trúc Modular Monolith giúp nhóm kiểm soát tuyệt đối tính toàn vẹn dữ liệu và loại bỏ gánh nặng vận hành mạng phân tán phức tạp của Microservices, nhưng vẫn đảm bảo tính độc lập giữa các nghiệp vụ (Identity, Listing, Wallet, Moderation). Khi cần mở rộng quy mô, các module có thể dễ dàng tách thành các service độc lập mà không cần phải viết lại logic nghiệp vụ bên trong.

3.4 Bài tập ứng dụng (Khởi tạo dự án và chạy Hello World)

Phần này trình bày chi tiết các file cấu hình quan trọng đã được hiện thực hóa trong mã nguồn dự án PropNest, tạo nền tảng để ứng dụng khởi chạy trơn tru trên mọi máy phát triển.

3.4.1 File cấu hình ứng dụng (appsettings.json và appsettings.Development.json)

File appsettings.json định nghĩa chuỗi kết nối cơ sở dữ liệu SQL Server, các tham số khóa ký JWT và mức độ ghi nhật ký Log:

Cấu hình appsettings.json của PropNest.Api

```
{
ConnectionStrings": {
            "DefaultConnection":"Server=localhost,1433;Database=PropNestDb;User
Id=sa;Password=YourStrong@Pass123;TrustServerCertificate=True;MultipleActiveResultSets=True"
 }
 "Jwt": {
   "Issuer": "PropNest",
   "Audience": "PropNest.Client",
   "SigningKey": "PropNestSecretSigningKey2026SecureKeyHere!"
 },
 "Logging": {
   "LogLevel": {
     "Default": "Information",
     "Microsoft.AspNetCore": "Warning"
   }
 },
 "AllowedHosts": "*"
}
```

3.4.2 File cấu hình khởi chạy (Properties/launchSettings.json)

File launchSettings.json chỉ định các profile chạy cục bộ, cổng HTTPS (7000) và HTTP (5000), đồng thời thiết lập URL tự động mở Swagger UI khi nhấn F5:

Cấu hình launchSettings.json phục vụ môi trường Development

```
// src/PropNest.Api/Properties/launchSettings.json
 {
```

```
  "profiles": {
          "PropNest.Api": {
          "commandName": "Project",
          "launchUrl": "swagger",
          "launchBrowser": true,
          "environmentVariables": {
      "ASPNETCORE_ENVIRONMENT": "Development"
          },
          "applicationUrl": "https://localhost:65147;http://localhost:65148"
          }
  }
}
```

3.4.3 File Program.cs – Entry Point ứng dụng

File Program.cs được tổ chức theo mô hình Minimal Hosting sạch sẽ, chia tách rõ ràng giữa hai giai đoạn: Đăng ký dịch vụ (Dependency Injection) và Cấu hình đường ống xử lý yêu cầu (Middleware Pipeline):

Mã nguồn của Program.cs trong dự án PropNest.Api

```
// src/PropNest.Api/Program.cs - Entry Point chuẩn hóa duy nhất của PropNest
using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
```

```
using PropNest.Api.Middleware;
using PropNest.Api.Serialization;
using PropNest.Api.Services;
using PropNest.Application.Abstractions;
using PropNest.Application.DependencyInjection;
using PropNest.Domain.Users;
using PropNest.Infrastructure.DependencyInjection;
using PropNest.Infrastructure.Persistence;

var builder = WebApplication.CreateBuilder(args);

// 1. Nạp và kiểm tra cấu hình JWT bắt buộc
var jwt = builder.Configuration.GetSection(JwtOptions.SectionName).Get<JwtOptions>()
        ?? throw new InvalidOperationException("JWT configuration is required.");
if (string.IsNullOrWhiteSpace(jwt.SigningKey))
{
        throw new InvalidOperationException("Jwt:SigningKey is required.");
}

builder.Services.Configure<JwtOptions>(builder.Configuration.GetSection(JwtOptions.SectionName));
builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<ICurrentUser, CurrentUser>();

// 2. Đăng ký các tầng kiến trúc Clean Architecture
builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);
```

```
// 3. Cấu hình ASP.NET Core Identity Core & Phân quyền Vai trò (Roles)
builder.Services.AddIdentityCore<ApplicationUser>(options =>
{
        // Cấu hình quy tắc mật khẩu tương thích với mật khẩu thử nghiệm "Test1234"
        options.Password.RequireDigit = true;
        options.Password.RequireLowercase = true;
        options.Password.RequireUppercase = false;
    options.Password.RequireNonAlphanumeric = false;
        options.Password.RequiredLength = 6;
        options.User.RequireUniqueEmail = true;

        // Cấu hình cơ chế khóa tài khoản (Lockout) chống Brute-force
        options.Lockout.AllowedForNewUsers = true;
    options.Lockout.MaxFailedAccessAttempts = 5;                    // Khóa sau 5 lần sai
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);  // Khóa chính xác 15 phút
})
.AddRoles<IdentityRole<long>>()
.AddEntityFrameworkStores<PropNestDbContext>();

builder.Services.AddControllers()
        .AddJsonOptions(options =>
        {
      options.JsonSerializerOptions.Converters.Add(new VietnamDateTimeOffsetJsonConverter());
        });

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
```

```
{
        options.SwaggerDoc("v1", new OpenApiInfo { Title = "PropNest API", Version = "v1" });
    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
        {
        Type = SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Name = "Authorization",
        Description = "Nhập Access Token JWT theo định dạng: Bearer {token}"
        });
        options.AddSecurityRequirement(new OpenApiSecurityRequirement
        {
        {
        new OpenApiSecurityScheme
        {
        Reference = new OpenApiReference { Type = ReferenceType.SecurityScheme, Id = "Bearer" }
        },
        Array.Empty<string>()
        }
        });
});

// 4. Đăng ký Authentication & JWT Bearer Handler (Ngăn chặn lỗi No authenticationScheme)
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
        .AddJwtBearer(options =>
```

```
            {
        options.TokenValidationParameters = new TokenValidationParameters
            {
            ValidateIssuer = true,
            ValidIssuer = jwt.Issuer,
            ValidateAudience = true,
            ValidAudience = jwt.Audience,
            RoleClaimType = System.Security.Claims.ClaimTypes.Role,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt.SigningKey)),
            ValidateLifetime = true,
            ClockSkew = TimeSpan.FromMinutes(1)
            };
            });

// 5. Đăng ký Phân quyền Policy CanModerateListings
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CanModerateListings", policy => policy.RequireRole("Moderator", "Admin"));
});

var app = builder.Build();

// 6. Tự động áp dụng Database Migrations trong môi trường Development
if (app.Environment.IsDevelopment())
{
            using var scope = app.Services.CreateScope();
```

```
        var dbContext = scope.ServiceProvider.GetRequiredService<PropNestDbContext>();
        var logger = scope.ServiceProvider.GetRequiredService<ILogger<Program>>();
        try
        {
        LogApplyingMigrations(logger);
        await dbContext.Database.MigrateAsync();
        LogMigrationsApplied(logger);
        }
        catch (Exception ex)
        {
        LogMigrationError(logger, ex);
        throw;
        }
}

// 7. Seed dữ liệu vai trò và tài khoản quản trị mặc định
await SeedRolesAsync(app.Services);

// 8. Đường ống HTTP Request Pipeline (Thứ tự chuẩn)
app.UseMiddleware<ExceptionHandlingMiddleware>();
app.UseMiddleware<CorrelationIdMiddleware>();
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();

app.UseSwagger();
app.UseSwaggerUI();
```

```
app.MapControllers();
app.MapHealthChecks("/health");

app.Run();

//khởi tạo mặc định 3 role
public partial class Program
{
        private static async Task SeedRolesAsync(IServiceProvider services)
        {
        using var scope = services.CreateScope();
        var roleManager = scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole<long>>>();
        var logger = scope.ServiceProvider.GetRequiredService<ILogger<Program>>();

        foreach (var roleName in new[] { "Seller", "Moderator", "Admin" })
        {
        if (await roleManager.RoleExistsAsync(roleName))
        {
          LogRoleAlreadyExists(logger, roleName);
        continue;
        }

        var result = await roleManager.CreateAsync(new IdentityRole<long>(roleName));
        if (!result.Succeeded)
        {
        var errors = string.Join(", ", result.Errors.Select(error => error.Description));
        throw new InvalidOperationException($"Could not seed role '{roleName}': {errors}");
```

```
            }

            LogRoleSeeded(logger, roleName);
            }
            }

            [LoggerMessage(EventId = 1, Level = LogLevel.Information, Message = "Đang kiểm tra và áp dụng
Database Migrations còn thiếu...")]
            private static partial void LogApplyingMigrations(ILogger logger);

            [LoggerMessage(EventId = 2, Level = LogLevel.Information, Message = "Database đã được đồng bộ
thành công!")]
            private static partial void LogMigrationsApplied(ILogger logger);

            [LoggerMessage(EventId = 3, Level = LogLevel.Error, Message = "Có lỗi xảy ra khi tự động migrate
database.")]
            private static partial void LogMigrationError(ILogger logger, Exception ex);

            [LoggerMessage(EventId = 4, Level = LogLevel.Information, Message = "Seeded role: {RoleName}")]
            private static partial void LogRoleSeeded(ILogger logger, string roleName);

            [LoggerMessage(EventId = 5, Level = LogLevel.Information, Message = "Role already exists:
{RoleName}")]
            private static partial void LogRoleAlreadyExists(ILogger logger, string roleName);
 }
```

3.4.4 Docker Compose – Điều phối hạ tầng SQL Server và RabbitMQ

File deploy/docker-compose.yml khởi chạy môi trường cơ sở dữ liệu và message queue chuẩn doanh nghiệp kèm cơ chế kiểm tra sức khỏe (Health Check):

Tệp deploy/docker-compose.yml khởi tạo SQL Server 2022 và RabbitMQ

```
services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      ACCEPT_EULA: Y
      MSSQL_SA_PASSWORD: ${SA_PASSWORD}
    ports:
      - "1433:1433"
    healthcheck:
      test: ["CMD-SHELL", "/opt/mssql-tools18/bin/sqlcmd -C -S localhost -U sa -P $$MSSQL_SA_PASSWORD
-Q 'SELECT 1' || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 12

  rabbitmq:
    image: rabbitmq:3-management
    ports:
      - "5672:5672"
      - "15672:15672"
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
```

```
interval: 10s
timeout: 5s
retries: 12
```

3.4.5 Định nghĩa Entity ban đầu và DbContext (PropNestDbContext)

Lớp PropNestDbContext trong tầng Infrastructure kế thừa IdentityDbContext để quản lý đồng thời cả bảng người dùng hệ thống và các bảng nghiệp vụ bất động sản:

Định nghĩa DbContext trung tâm PropNestDbContext

```
public sealed class PropNestDbContext(DbContextOptions<PropNestDbContext> options)
    : IdentityDbContext<ApplicationUser, IdentityRole<long>, long>(options), IUnitOfWork
{
    public DbSet<Listing> Listings => Set<Listing>();
    public DbSet<ListingHistory> ListingHistories => Set<ListingHistory>();
    public DbSet<ListingModerationReview> ListingModerationReviews => Set<ListingModerationReview>();
    public DbSet<Wallet> Wallets => Set<Wallet>();
    public DbSet<WalletTransaction> WalletTransactions => Set<WalletTransaction>();
    public DbSet<WorkflowInstance> WorkflowInstances => Set<WorkflowInstance>();
    public DbSet<WorkflowStep> WorkflowSteps => Set<WorkflowStep>();
    public DbSet<IdempotencyRequest> IdempotencyRequests => Set<IdempotencyRequest>();
    public DbSet<OutboxMessage> OutboxMessages => Set<OutboxMessage>();

    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);
        builder.ApplyConfigurationsFromAssembly(typeof(PropNestDbContext).Assembly);
    }
}
```

3.4.6 Chạy Migration đầu tiên và cập nhật CSDL

Sử dụng dotnet-ef để sinh file Migration InitialCreate và ánh xạ trực tiếp toàn bộ lược đồ bảng vào máy chủ SQL Server:

Lệnh tạo và cập nhật CSDL qua EF Core Migration

```
# Tạo migration ban đầu
dotnet ef migrations add InitialCreate --project src/PropNest.Infrastructure --startup-project src/PropNest.Api

# Áp dụng migration vào SQL Server
dotnet ef database update --project src/PropNest.Infrastructure --startup-project src/PropNest.Api
```

3.4.7 Endpoint kiểm tra sức khỏe và API đầu tiên

Để kiểm tra ứng dụng đã sẵn sàng tiếp nhận request hay chưa, nhóm truy cập endpoint kiểm tra sức khỏe /health hoặc gọi API xem danh sách tin nháp ban đầu:

Controller API mẫu đầu tiên xử lý tin đăng nháp

```
// src/PropNest.Api/Controllers/ListingsController.cs (Trích xuất khai báo Constructor và Authorization)
 [ApiController]
 [Route("api/v1/listings")]
 public sealed partial class ListingsController(
         ICurrentUser currentUser,
         IListingService listingService,
         IWorkflowService workflowService) : ControllerBase
 {
         [HttpPost]
         [Authorize(Roles = "Seller")] // Chỉ người bán (Seller) mới có quyền tạo tin đăng
         public async Task<ActionResult<ListingResponse>> Create(
         [FromBody] CreateListingRequest request,
         CancellationToken cancellationToken)
         {
         // Xử lý tạo tin đăng...
```

```
        }
}
```

3.4.8 Kết quả chạy ứng dụng Hello World qua Swagger UI

Khi khởi chạy ứng dụng bằng lệnh dotnet run --project src/PropNest.Api, Kestrel Web Server khởi động trên cổng https://localhost:7000. Mở trình duyệt và truy cập https://localhost:7000/swagger, giao diện Swagger UI hiển thị danh mục các endpoint đầy đủ, kèm nút Authorize cho phép dán JWT token để kiểm thử trực tiếp các API được bảo vệ.

KẾT LUẬN CHƯƠNG 3

Chương 3 đã hoàn thành toàn diện quy trình thiết lập môi trường phát triển cho hệ thống PropNest Backend. Từ việc chuẩn bị các công cụ (.NET 8 SDK, SQL Server, RabbitMQ, Docker), cấu hình quản lý gói tập trung (CPM), phân tích cấu trúc Clean Architecture cho đến việc chạy thành công Migration ban đầu và khởi chạy API mẫu đầu tiên. Đây là bệ phóng vững chắc để triển khai các nghiệp vụ chuyên sâu trong các chương tiếp theo.

CHƯƠNG 4 CÁC KHÁI NIỆM NỀN TẢNG CỦA BACKEND VỚI ASP.NET CORE

4.1 Cú pháp và đặc trưng C# từ cơ bản đến nâng cao

4.1.1 Hệ thống Type System, Nullable Reference Types và Encapsulation

C# là một ngôn ngữ định kiểu tĩnh mạnh (Strongly-typed, Object-Oriented Programming). Trong Common Language Runtime (CLR), hệ thống kiểu dữ liệu được phân chia thành hai nhóm cơ bản với mô hình quản lý bộ nhớ hoàn toàn khác biệt:

- **Value Types (Kiểu giá trị):** Bao gồm các kiểu nguyên thủy (int, long, decimal, bool, enum) và struct. Dữ liệu của Value Type được lưu trữ trực tiếp trên ngăn xếp (Stack) hoặc nhúng trực tiếp trong đối tượng chứa nó. Việc gán giá trị thực hiện sao chép toàn bộ nội dung (Copy by value).
- **Reference Types (Kiểu tham chiếu):** Bao gồm class, interface, delegate và string. Đối tượng thực sự được cấp phát trên bộ nhớ Heap và được quản lý tự động bởi Garbage Collector (GC). Biến của Reference Type chỉ lưu giữ địa chỉ tham chiếu (Memory Pointer) trỏ tới vùng nhớ trên Heap.

Một bước tiến mang tính cách mạng từ C# 8.0 và được hoàn thiện trong C# 12 là tính năng Nullable Reference Types (NRT), được bật mặc định trong PropNest qua thẻ '<Nullable>enable</Nullable>'. Trước đây, mọi biến Reference Type đều có thể nhận giá trị null, dẫn đến lỗi 'NullReferenceException' kinh điển trong lập trình. Với NRT, trình biên dịch Roslyn phân biệt tường minh:

- **Non-nullable Reference Type (ví dụ: string Title):** Trình biên dịch bắt buộc phải có giá trị, nếu gán null hoặc không khởi tạo sẽ lập tức cảnh báo lúc biên dịch.
- **Nullable Reference Type (ví dụ: string? Description):** Cho phép nhận null nhưng bắt buộc lập trình viên phải kiểm tra null trước khi truy cập.

Về tính đóng gói (Encapsulation) trong Lập trình Hướng đối tượng (OOP): Một thiết kế chuẩn mực phải ngăn chặn việc tùy tiện thay đổi trạng thái nội bộ của đối tượng

từ bên ngoài (Anemic Domain Model). Thay vào đó, toàn bộ thuộc tính phải dùng 'private set', và mọi sự biến động trạng thái chỉ được thực hiện thông qua các Domain Method có thẩm định nghiệp vụ.

Minh họa Type System, Nullable Reference Types và Encapsulation trong Entity Wallet

```
// src/PropNest.Domain/Wallets/Wallet.cs

namespace PropNest.Domain.Wallets;


public sealed class Wallet : AuditableEntity

{

        // Đóng gói: private constructor phục vụ EF Core

        private Wallet() { }


        public Wallet(long userId)

        {

    ArgumentOutOfRangeException.ThrowIfNegativeOrZero(userId);

        UserId = userId;

        }


        public long WalletId { get; private set; }

        public long UserId { get; private set; }


        // Số dư được đóng gói hoàn toàn, bên ngoài không thể gán: wallet.MainBalance = 999999;

        public decimal MainBalance { get; private set; }

        public decimal PromoBalance { get; private set; }


        // Concurrency Token quản lý bởi EF Core
```

```
public byte[] RowVersion { get; private set; } = Array.Empty<byte>();

// Hành vi nghiệp vụ: Nạp tiền vào ví chính kèm kiểm tra điều kiện
public void CreditMain(decimal amount)
{
EnsurePositive(amount);
MainBalance += amount;
Touch(); // Cập nhật thời điểm UpdatedAt
}

// Hành vi nghiệp vụ: Trừ tiền với quy tắc ưu tiên trừ PromoBalance trước
public (decimal PromoUsed, decimal MainUsed) Charge(decimal amount)
{
EnsurePositive(amount);
if (MainBalance + PromoBalance < amount)
{
throw new InvalidOperationException("Insufficient wallet balance.");
}

var promoUsed = Math.Min(PromoBalance, amount);
var mainUsed = amount - promoUsed;
PromoBalance -= promoUsed;
MainBalance -= mainUsed;
Touch();
return (promoUsed, mainUsed);
}
```

```
        private static void EnsurePositive(decimal amount)
        {
    ArgumentOutOfRangeException.ThrowIfNegativeOrZero(amount);
        }
}
```

4.1.2 Tính bất biến với Record Types và cú pháp C# 12 (Primary Constructors, Collection Expressions)

Trong phát triển phần mềm hiện đại, tính bất biến (Immutability) là nguyên tắc cốt lõi giúp mã nguồn an toàn trong môi trường đa luồng (Thread-safe) và loại bỏ hoàn toàn các lỗi 'tác dụng phụ' (Side-effects) ngoài ý muốn. C# 9.0 giới thiệu từ khóa 'record', và C# 10/12 tiếp tục hoàn thiện thành 'record class' và 'record struct'. Khác biệt cơ bản giữa 'class' truyền thống và 'record' nằm ở cơ chế so sánh bằng:

- **Class truyền thống dùng Reference Equality:** Hai đối tượng class dù có toàn bộ thuộc tính giống hệt nhau nhưng nằm ở hai ô nhớ khác nhau thì toán tử '==' vẫn trả về false.

Record dùng Value Equality: Trình biên dịch tự động sinh hàm Equals(), GetHashCode() và toán tử '==' dựa trên giá trị của tất cả các thuộc tính bên trong. Nếu hai record có dữ liệu giống nhau, chúng được coi là bằng nhau.

Hai cải tiến cú pháp đột phá của C# 12 được dự án PropNest khai thác tối đa gồm:

- **Primary Constructors:** Cho phép khai báo tham số khởi tạo trực tiếp trên tên class hoặc record. Trình biên dịch tự động tạo các trường dữ liệu tương ứng, giúp loại bỏ hoàn toàn mã thừa gán tham số trong constructor truyền thống.
- **Collection Expressions:** Cú pháp tạo mảng và danh sách đồng nhất thông qua cặp ngoặc vuông '[..]', thay thế cú pháp rườm rà 'new List<T>()' hay 'new T[] { ... }', đồng thời cho phép gộp các tập hợp bằng toán tử spread '..'.

Minh họa Record Types, Primary Constructors và Collection Expressions trong PropNest

```
// src/PropNest.Application/Wallets/WalletContracts.cs
namespace PropNest.Application.Wallets;

// 1. Khai báo sealed record sử dụng Primary Constructor trong C# 12
public sealed record WalletDto(
        long WalletId,
        long UserId,
        decimal MainBalance,
        decimal PromoBalance)
{
        // Computed property tính tổng số dư
        public decimal TotalBalance => MainBalance + PromoBalance;
};

// 2. Command bất biến tiếp nhận nghiệp vụ Nạp tiền
public sealed record TopUpWalletCommand(
        long UserId,
        decimal Amount,
        string? Description = null,
```

```
        Guid? CorrelationId = null);


// 3. Collection Expressions trong khởi tạo danh sách mặc định
string[] defaultRoles = ["Seller", "Moderator", "Admin"];
```

4.1.3 Pattern Matching và Switch Expressions

Pattern Matching (Khớp mẫu) là một trong những tính năng mạnh mẽ nhất của C# hiện đại, cho phép kiểm tra hình dạng (Shape) và kiểu dữ liệu của đối tượng một cách ngắn gọn, an toàn và trực quan.

Thay vì sử dụng câu lệnh 'switch-case' cồng kềnh với các từ khóa 'break' và biến tạm như trong C/C++ truyền thống, C# 8+ cung cấp Switch Expression biến câu lệnh rẽ nhánh thành một biểu thức tính toán trả về giá trị (Expression-bodied):

- **Type Pattern:** Kiểm tra xem đối tượng có thuộc một kiểu cụ thể hay không (ví dụ: 'KeyNotFoundException => 404').
- **Property Pattern:** Kiểm tra giá trị các thuộc tính bên trong đối tượng mà không cần ép kiểu thủ công.
- **Discard Pattern (_):** Đại diện cho trường hợp mặc định (Default/Fallback case) khi không có mẫu nào khớp.

Áp dụng Pattern Matching trong hàm CreateProblemDetails

```
using Microsoft.AspNetCore.Diagnostics;
 using Microsoft.AspNetCore.Mvc;
 using Microsoft.EntityFrameworkCore;


 namespace PropNest.Api.Middleware;


 public sealed partial class ExceptionHandlingMiddleware(RequestDelegate next,
ILogger<ExceptionHandlingMiddleware> logger)
 {
        [LoggerMessage(Level = LogLevel.Error, Message = "Unhandled exception for request
{RequestPath}.")]
        private static partial void LogUnhandledException(ILogger logger, Exception exception, string
requestPath);


        [LoggerMessage(Level = LogLevel.Warning, Message = "Response has already started for
{RequestPath}. Cannot write ProblemDetails.")]
        private static partial void LogResponseAlreadyStarted(ILogger logger, string requestPath);


        public async Task InvokeAsync(HttpContext context)
        {
        try
        {
        await next(context);
        }
        catch (Exception exception)
        {
```

```
 if (context.Response.HasStarted)
 {
   LogResponseAlreadyStarted(logger, context.Request.Path);
 throw;
 }

LogUnhandledException(logger, exception, context.Request.Path);
 var problem = CreateProblemDetails(context, exception);
 context.Response.StatusCode = problem.Status ?? StatusCodes.Status500InternalServerError;
 await context.Response.WriteAsJsonAsync(problem);
 }
 }

 private static ProblemDetails CreateProblemDetails(HttpContext context, Exception exception)
 {
 var status = exception switch
 {
 KeyNotFoundException => StatusCodes.Status404NotFound,
 UnauthorizedAccessException => StatusCodes.Status403Forbidden,
DbUpdateConcurrencyException => StatusCodes.Status409Conflict,
 ArgumentException => StatusCodes.Status400BadRequest,
 InvalidOperationException => StatusCodes.Status400BadRequest,
 _ => StatusCodes.Status500InternalServerError
 };

 return new ProblemDetails
 {
```

```
            Status = status,
            Title = status switch
            {
              StatusCodes.Status404NotFound => "Not Found",
              StatusCodes.Status403Forbidden => "Forbidden",
              StatusCodes.Status409Conflict => "Conflict",
              StatusCodes.Status400BadRequest => "Bad Request",
            _ => "An unexpected error occurred."
            },
            Detail = status == StatusCodes.Status500InternalServerError
            ? "An internal server error occurred."
            : exception.Message,
            Instance = context.Request.Path
            };
            }
}
```

4.1.4 Lập trình bất đồng bộ (Asynchronous Programming) với async/await, Task, ValueTask và CancellationToken

Trong các ứng dụng Web Backend có lưu lượng truy cập cao, các tác vụ I/O (truy vấn Database, đọc ghi ổ đĩa, gọi HTTP API bên ngoài) chiếm phần lớn thời gian xử lý. Nếu sử dụng lập trình đồng bộ (Synchronous), luồng thực thi (Thread) của máy chủ sẽ bị phong tỏa (Blocked) trong trạng thái chờ đợi, dẫn đến cạn kiệt Thread Pool (Thread Pool Starvation) và làm sập hệ thống.

ASP.NET Core giải quyết triệt để bài toán này bằng mô hình Task-based Asynchronous Pattern (TAP) thông qua cặp từ khóa '**async**' và '**await**':

- **Cơ chế State Machine ngầm định:** Khi gặp từ khóa 'await', trình biên dịch C# tự động tách phương thức thành một Máy trạng thái (State Machine). Luồng hiện tại được giải phóng ngay lập tức để quay về Thread Pool phục vụ các HTTP Request khác. Khi thao tác I/O hoàn tất (I/O Completion Port thông báo), Thread Pool sẽ cấp một luồng bất kỳ để tiếp tục thực thi phần mã phía sau.
- **Phân biệt Task và ValueTask:** 'Task<T>' là một đối tượng Reference Type được cấp phát trên Heap. Nếu một phương thức bất đồng bộ thường xuyên trả về kết quả có sẵn (cached/synchronous completion), việc tạo Task liên tục sẽ gây áp lực lớn lên Garbage Collector. '.NET' giới thiệu 'ValueTask<T>' là một struct nằm trên Stack. Trong Entity Framework Core 8, phương thức AddAsync trả về ValueTask, do đó cần gọi '.AsTask()' nếu interface yêu cầu Task.
- **Cơ chế ngắt lan truyền với CancellationToken:** CancellationToken là cơ chế cộng tác ngắt tác vụ (Cooperative Cancellation). Khi người dùng hủy request hoặc đóng trình duyệt, HTTP connection bị ngắt, CancellationToken kích hoạt trạng thái hủy. Việc truyền CancellationToken xuyên suốt qua các tầng (Controller -> Service -> Repository -> DbContext) giúp hệ thống ngừng ngay lập tức các câu lệnh SQL tốn kém, tiết kiệm tối đa tài nguyên máy chủ (quy tắc CA2016).

Minh họa lập trình bất đồng bộ với ValueTask và CancellationToken trong WalletRepository

```
// src/PropNest.Infrastructure/Persistence/Repositories/WalletRepository.cs
 public sealed class WalletRepository(PropNestDbContext dbContext) : IWalletRepository
 {
        // 1. AddAsync của EF Core trả về ValueTask, chuyển đổi sang Task bằng AsTask()
        public Task AddAsync(Wallet wallet, CancellationToken cancellationToken = default)
        {
        return dbContext.Wallets.AddAsync(wallet, cancellationToken).AsTask();
        }

        // 2. Chuyển tiếp CancellationToken vào câu lệnh truy vấn bất đồng bộ AnyAsync
        public Task<bool> ExistsByUserIdAsync(long userId, CancellationToken cancellationToken =
default)
        {
        return dbContext.Wallets.AnyAsync(x => x.UserId == userId, cancellationToken);
        }

        public Task<Wallet?> GetByUserIdAsync(long userId, CancellationToken cancellationToken =
default)
        {
        return dbContext.Wallets.FirstOrDefaultAsync(w => w.UserId == userId, cancellationToken);
        }
 }
```

4.1.5 Compile-time Code Generation với Source Generated LoggerMessage

Trong các ứng dụng .NET truyền thống, việc ghi log thường dùng phương thức mở rộng:

Cách ghi log truyền thống gặp nhược điểm hiệu năng

```
logger.LogInformation("User {UserId} registered successfully.", userId);
```

Cách ghi log truyền thống này tồn tại hai nhược điểm chí mạng về hiệu năng:

- **Hiện tượng Boxing: Tham số 'userId' (kiểu long - Value Type)** bị ép kiểu ngầm định sang 'object' (Reference Type), dẫn đến việc cấp phát bộ nhớ trên Heap và sinh rác cho Garbage Collector.
- **Phân tích chuỗi lúc Runtime:** Trình phân tích log phải dùng Reflection để duyệt chuỗi template '{UserId}' ở mỗi lần gọi hàm, tiêu tốn chu kỳ CPU.

Từ .NET 6 và hoàn thiện trong .NET 8, Microsoft giới thiệu tính năng Compile-time Source Generated Logging thông qua thuộc tính '[LoggerMessage]'. Lập trình viên định nghĩa một phương thức static partial, trình biên dịch Roslyn sẽ tự động sinh mã C# tối ưu trực tiếp lúc biên dịch:

- **Zero Heap Allocation:** Loại bỏ hoàn toàn boxing tham số.
- **Không tốn chi phí** Reflection lúc chạy ứng dụng.
- **Tuân thủ tuyệt đối quy tắc** phân tích mã nguồn CA1848, cho phép dự án bật chế độ 'TreatWarningsAsErrors=true' mà không bị cảnh báo lỗi.

Áp dụng Source Generated LoggerMessage trong ExceptionHandlingMiddleware

```
// src/PropNest.Api/Middleware/ExceptionHandlingMiddleware.cs
 public sealed partial class ExceptionHandlingMiddleware(
            RequestDelegate next,
    ILogger<ExceptionHandlingMiddleware> logger)
 {
            // Trình biên dịch Roslyn tự động sinh mã ghi log tối ưu lúc build
            [LoggerMessage(Level = LogLevel.Error, Message = "Unhandled exception for request
{RequestPath}.")]
            private static partial void LogUnhandledException(ILogger logger, Exception exception, string
requestPath);


            public async Task InvokeAsync(HttpContext context)
            {
            try { await next(context); }
            catch (Exception exception)
            {
            // Gọi phương thức log đã sinh mã (Zero-allocation)
           LogUnhandledException(logger, exception, context.Request.Path);
            ...
            }
            }
 }
```

4.2 Kiến trúc Kestrel Web Server và Generic Host

4.2.1 Kestrel Web Server và cơ chế xử lý I/O bất đồng bộ

Kestrel là máy chủ web (Web Server) tích hợp sẵn trong nhân của ASP.NET Core, được thiết kế từ đầu với tiêu chí hiệu năng cao, đa nền tảng (chạy trên Windows, Linux, macOS) và tối ưu hóa tài nguyên phần cứng. Về mặt kiến trúc mạng, Kestrel hoạt động theo mô hình:

- **Lớp Socket Transport:** Sử dụng trực tiếp Socket API của hệ điều hành kết hợp với kỹ thuật I/O đa luồng bất đồng bộ (I/O Completion Ports trên Windows hoặc epoll trên Linux) để tiếp nhận hàng chục nghìn kết nối mạng đồng thời mà không tạo luồng xử lý riêng cho từng socket.
- **Hỗ trợ đa giao thức:** Kestrel hỗ trợ hoàn chỉnh HTTP/1.1 (truyền tải dạng văn bản), HTTP/2 (ghép kênh đa luồng Multiplexing trên cùng một kết nối TCP), và HTTP/3 (chạy trên giao thức UDP/QUIC giúp giảm độ trễ bắt tay mạng).
- **Vòng đời đối tượng HttpContext:** Khi một HTTP Request được gửi đến, Kestrel phân tích các byte dữ liệu nhị phân thô từ socket thành đối tượng HttpContext. Đối tượng này đại diện cho toàn bộ thông tin của phiên giao dịch: HttpContext.Request (Headers, Body, Query, Path), HttpContext.Response (StatusCode, Headers, BodyStream) và HttpContext.User (danh tính ClaimsPrincipal sau khi xác thực).

4.2.2 Generic Host (WebApplicationBuilder / WebApplication) và hệ thống Configuration

Từ .NET 6 và tiếp tục chuẩn hóa trong .NET 8, mô hình Minimal Hosting thay thế cho kiến trúc Program.cs / Startup.cs cũ. Ứng dụng được khởi tạo và điều phối bởi hai đối tượng trung tâm:

- **WebApplicationBuilder:** Đóng vai trò là nhà xưởng cấu hình. Trước khi ứng dụng được xây dựng, lập trình viên đăng ký các dịch vụ (Services) vào DI Container qua thuộc tính 'builder.Services', đồng thời nạp cấu hình hệ thống.
- **WebApplication:** Thể hiện của ứng dụng sau khi gọi 'builder.Build()'. Đối tượng này chịu trách nhiệm quản lý đường ống Middleware Pipeline và lắng nghe các yêu cầu mạng.

Hệ thống cấu hình (Configuration System) của Generic Host hỗ trợ mô hình phân cấp đa tầng (Hierarchical Configuration), trong đó giá trị cấu hình được nạp tuần tự và ghi đè (Override) theo độ ưu tiên tăng dần:

- **appsettings.json** (Cấu hình cơ sở mặc định của ứng dụng).
- **appsettings.{Environment}.json** (Cấu hình đặc thù cho môi trường Development, Staging hoặc Production).
- **Biến môi trường hệ điều hành (Environment Variables** - ưu tiên cao nhất, thường dùng khi đóng gói Docker Container để truyền ConnectionStrings và Secret Keys).

Khởi tạo Generic Host và quản lý cấu hình tập trung trong Program.cs

```
// src/PropNest.Api/Program.cs
var builder = WebApplication.CreateBuilder(args);

// Nạp cấu hình kết nối CSDL và JWT
var connectionString = builder.Configuration.GetConnectionString("PropNest");
var jwtOptions = builder.Configuration.GetSection("Jwt").Get<JwtOptions>();

// Đăng ký dịch vụ vào DI
builder.Services.AddControllers();
builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);

var app = builder.Build();

// Tự động chạy Migration khi khởi động môi trường Development
if (app.Environment.IsDevelopment())
{
        using var scope = app.Services.CreateScope();
        var dbContext = scope.ServiceProvider.GetRequiredService<PropNestDbContext>();
        await dbContext.Database.MigrateAsync();
}

app.Run();
```

Bảng 4.1 So sánh cấu hình hệ thống PropNest giữa môi trường Development và Production

Thông số cấu hình	Môi trường Development	Môi trường Production (Docker)	Mục đích kiến trúc
ConnectionStrings:DefaultConnection	Server=localhost;Database=PropNestDb;Trusted_Connection=True;...	Server=sqlserver,1433;Database=PropNestDb;User Id=sa;Password=...	Chuyển từ Windows Authentication cục bộ sang kết nối Docker Network an toàn.
RabbitMQ:Host	localhost	rabbitmq	Ánh xạ tên Service container trong Docker Compose.
Logging:LogLevel:Default	Debug (ghi chi tiết câu lệnh SQL)	Warning (chỉ ghi nhận cảnh báo và lỗi)	Giảm tải I/O ổ đĩa và bảo vệ hiệu năng máy chủ khi chịu tải cao.
Serilog:WriteTo:Console	Bật chế độ định dạng màu sắc dễ đọc	Bật chế độ JSON có cấu trúc (Structured Log)	Phục vụ cho các hệ thống gom log tập trung (ELK Stack / Seq / Datadog).
Database Auto-Migrate	Tự động kích hoạt lúc khởi động (Program.cs)	Vô hiệu hóa; chạy migration qua CI/CD Pipeline	Tránh xung đột khóa bảng (Table Lock) khi có nhiều instance container cùng chạy.

4.2.3 Luồng xử lý một HTTP Request hoàn chỉnh (End-to-End Request Lifecycle Flow)

Bản chất kiến trúc: Trong dự án PropNest, một HTTP Request không được xử lý nguyên khối mà phải trải qua một chuỗi các chặng luân chuyển có kiểm soát chặt chẽ giữa các tầng kiến trúc Clean Architecture:

1 **Tầng máy chủ mạng:** Kestrel Web Server tiếp nhận kết nối mạng nhị phân từ Socket, phân tích giao thức HTTP/1.1 hoặc HTTP/2 và khởi tạo đối tượng HttpContext.

2 **Đường ống Middleware Pipeline:** Request lần lượt đi xuyên qua ExceptionHandlingMiddleware, CorrelationIdMiddleware, HttpsRedirection, AuthenticationMiddleware (giải mã JWT nạp HttpContext.User) và AuthorizationMiddleware.

3 **Tầng Presentation (Controller):** Endpoint Routing xác định Controller đích, kích hoạt Model Binding chuyển đổi JSON Body thành Request DTO và tự động thẩm định dữ liệu.

4 **Tầng Application (Service):** ListingsController ủy thác xử lý cho ListingService thông qua Interface IListingService. Tầng này điều phối các quy tắc nghiệp vụ, khởi tạo Domain Entity và kiểm tra tính toàn vẹn (Invariants).

5 **Tầng Infrastructure & CSDL:** ListingService gọi IListingRepository và IUnitOfWork. Entity Framework Core ánh xạ đối tượng Domain sang câu lệnh SQL INSERT, thực thi trên Microsoft SQL Server và cam kết giao dịch (Commit Transaction).

6 **Chặng phản hồi (Response):** Dữ liệu thực thể được ánh xạ ngược về ListingDto, trả về Controller đóng gói thành HTTP 201 Created kèm Header Location và JSON Body, sau đó di chuyển ngược lại qua đường ống Middleware trước khi ra dây mạng.

Hiện thực hóa mã nguồn: Mã nguồn thực tế qua từng tầng cho chức năng Tạo tin đăng (POST /api/v1/listings):

Hiện thực luồng xử lý xuyên suốt qua các tầng trong PropNest

```
// [1] Presentation Layer: ListingsController.cs
[HttpPost]
public async Task<ActionResult<ListingDto>> Create([FromBody] CreateListingRequest request,
CancellationToken cancellationToken)
{
        var command = new CreateListingCommand(currentUser.UserId, request.Title,
request.Description,
    request.PropertyType, request.ListingType, request.Price, request.Area, request.City, request.District,
request.Ward, request.Address);
        var listing = await listingService.CreateAsync(command, cancellationToken);
        return CreatedAtAction(nameof(GetById), new { listingId = listing.ListingId }, listing);
}

// [2] Application Layer: ListingService.cs
public async Task<ListingDto> CreateAsync(CreateListingCommand command, CancellationToken
cancellationToken = default)
{
        var listing = new Listing(command.OwnerUserId, command.Title, command.Description,
command.PropertyType,
    command.ListingType, command.Price, command.Area, command.City, command.District,
command.Ward, command.Address);
        await listingRepository.AddAsync(listing, cancellationToken);
        await unitOfWork.SaveChangesAsync(cancellationToken);
  listingRepository.AddHistory(ListingHistory.CreateSnapshot(listing.ListingId, command.OwnerUserId,
"Create", JsonSerializer.Serialize(ToDto(listing))));
        await unitOfWork.SaveChangesAsync(cancellationToken);
        return ToDto(listing);
```

```
}


// [3] Infrastructure Layer: ListingRepository.cs & DbContext
public async ValueTask AddAsync(Listing listing, CancellationToken cancellationToken)
{
        await dbContext.Listings.AddAsync(listing, cancellationToken);
}
```

4.3 Dependency Injection (DI) và Quản lý Service Lifetimes

4.3.1 Nguyên lý Inversion of Control (IoC) và Dependency Inversion (SOLID)

Inversion of Control (IoC - Đảo ngược điều khiển) là nguyên lý thiết kế phần mềm trong đó quyền kiểm soát luồng thực thi và việc khởi tạo các đối tượng phụ thuộc được chuyển giao từ lớp tiêu thụ sang một Framework điều khiển trung tâm.

Dependency Inversion Principle (DIP - Chữ D trong nguyên lý thiết kế SOLID) quy định rõ:

- **Các module cấp cao (như ListingService)** không được phụ thuộc trực tiếp vào các module cấp thấp (như ListingRepository hay SQL Server). Cả hai bắt buộc phải phụ thuộc vào Abstraction (Interface IListingRepository).
- **Abstraction** không được phụ thuộc vào chi tiết. Chi tiết (Implementation) phải phụ thuộc vào Abstraction.

Nhờ có IoC Container trong .NET (Microsoft.Extensions.DependencyInjection), ListingService không bao giờ dùng từ khóa 'new ListingRepository()', mà chỉ khai báo Interface trong constructor. Điều này giúp hệ thống đạt độ khớp nối lỏng (Loose Coupling), cho phép thay đổi hệ quản trị CSDL hoặc viết Unit Test độc lập mà không cần chỉnh sửa mã nguồn nghiệp vụ.

4.3.2 Ba vòng đời dịch vụ: Transient, Scoped, Singleton và rủi ro Captive Dependency

ASP.NET Core cung cấp 3 cơ chế quản lý vòng đời (Service Lifetime) khi đăng ký dịch vụ:

Bảng 4.2 So sánh 3 chế độ vòng đời Service Lifetime trong ASP.NET Core

Service Lifetime	**Phương thức đăng ký**	**Cơ chế khởi tạo & giải phóng**	**Áp dụng trong dự án PropNest**
Transient	AddTransient<T, U>()	Mỗi lần được yêu cầu (Inject), Container sẽ tạo ra một INSTANCE MỚI HOÀN TOÀN. Thích hợp cho các dịch vụ nhẹ, phi trạng thái (Stateless).	Các thuật toán so sánh JSON vi sai DeltaEngine, các bộ Helper sinh mã ngẫu nhiên.
Scoped	AddScoped<T, U>()	Được tạo DUY NHẤT MỘT LẦN cho mỗi HTTP Request. Toàn bộ các class trong cùng 1 request chia sẻ chung instance này. Tự động Dispose khi request kết thúc.	PropNestDbContext, IUnitOfWork, IListingService, IWalletService, ICurrentUser.
Singleton	AddSingleton<T, U>()	Được tạo DUY NHẤT MỘT LẦN khi ứng dụng khởi động và tồn tại VĨNH VIỄN trong RAM cho tới khi	LoggingOutboxMessagePublisher, MemoryCache, cấu hình IConfiguration, IHostEnvironment.

		tắt server. Mọi request dùng chung 1 instance.	

Bảng 4.3 Bảng danh mục Service Lifetimes của các dịch vụ trong hệ thống PropNest

Dịch vụ / Interface	Lớp cài đặt	Lifetime	Lý do kỹ thuật và kiến trúc
PropNestDbContext	PropNestDbContext	Scoped	DbContext KHÔNG an toàn cho đa luồng. Phải cô lập theo từng request để đảm bảo tính toàn vẹn của ChangeTracker và tự động Dispose kết nối SQL.
IUnitOfWork	UnitOfWork	Scoped	Cần chia sẻ chung một Transaction và một thể hiện DbContext với Repository trong suốt một nghiệp vụ.
IListingRepository	ListingRepository	Scoped	Phụ thuộc trực tiếp vào PropNestDbContext Scoped; chia sẻ phiên làm việc CSDL trong cùng HTTP request.
IListingService	ListingService	Scoped	Phụ thuộc vào IListingRepository Scoped; đảm bảo mỗi request người dùng có một phiên điều phối nghiệp vụ độc lập.

ICurrentUser	CurrentUser	Scoped	Lấy dữ liệu ClaimsPrincipal từ HttpContextAccessor của HTTP request hiện tại; thông tin user thay đổi theo từng request.
IMassTransitOutboxPublisher	MassTransitOutboxPublisher	Scoped	Cần ghi nhận các OutboxMessage vào DbContext của chính request đó trước khi commit.
IPasswordHasher<ApplicationUser>	PasswordHasher<ApplicationUser>	Singleton	Thuật toán băm PBKDF2 phi trạng thái (Stateless), không phụ thuộc dữ liệu ngoài, dùng chung toàn hệ thống để tiết kiệm RAM.
IConfiguration	ConfigurationRoot	Singleton	Chứa cấu hình ứng dụng được nạp một lần khi máy chủ khởi động, dùng chung cho mọi luồng.

Cảnh báo kỹ thuật về lỗi 'Captive Dependency' (Phụ thuộc bị bắt cóc):

Lỗi Captive Dependency xảy ra khi một dịch vụ có vòng đời dài (ví dụ: Singleton) lại tiêm vào constructor một dịch vụ có vòng đời ngắn hơn (ví dụ: Scoped như DbContext). Khi đó, đối tượng DbContext lẽ ra phải được giải phóng sau mỗi request lại bị 'bắt cóc' và giữ lại vĩnh viễn trong Singleton, dẫn đến hai hậu quả đặc biệt nguy hiểm:

- **Rò rỉ bộ nhớ (Memory Leak):** DbContext không bao giờ được Dispose(), bộ nhớ Change Tracker phình to vô hạn theo thời gian.
- **Xung đột đa luồng (Concurrency Exception):** DbContext trong EF Core vốn **KHÔNG** an toàn cho đa luồng (Not Thread-Safe). Khi hai HTTP request cùng gọi

vào Singleton cùng lúc, hai luồng sẽ truy cập đồng thời vào cùng một DbContext dẫn đến sập ứng dụng ngay lập tức.

Trong PropNest, nguyên tắc vàng được áp dụng: Tuyệt đối không tiêm Scoped vào Singleton; DbContext và các Repository/Service chỉ được đăng ký Scoped.

Mô phỏng cố tình đăng ký sai Service Lifetime (Captive Dependency)

```
// Đoạn mã cố tình tạo lỗi: Singleton Worker lại tiêm trực tiếp DbContext Scoped qua Constructor!
public sealed class ListingAnalyticsWorker(PropNestDbContext dbContext) : BackgroundService
{
        protected override async Task ExecuteAsync(CancellationToken stoppingToken) { /* ... */ }
}
// Program.cs:
builder.Services.AddHostedService<ListingAnalyticsWorker>(); // Đăng ký Singleton
```

Lỗi InvalidOperationException do cơ chế ValidateScopes của .NET 8 chặn đứng

```
Unhandled exception. System.InvalidOperationException: Cannot consume scoped service
'PropNest.Infrastructure.Persistence.PropNestDbContext' from singleton
'Microsoft.Extensions.Hosting.IHostedService (PropNest.Infrastructure.Services.ListingAnalyticsWorker)'.
   at Microsoft.Extensions.DependencyInjection.ServiceLookup.CallSiteValidator.ValidateResolution(...)
   at Microsoft.Extensions.DependencyInjection.ServiceProvider.ValidateService(ServiceDescriptor descriptor)

Process terminated. Exit code 1.
```

Giải pháp khắc phục: Giải pháp chuẩn mực: Đối với các BackgroundService Singleton, không tiêm DbContext trực tiếp mà tiêm IServiceScopeFactory. Khi cần xử lý, dùng **using var scope = scopeFactory.CreateScope()**; để tạo scope độc lập và tự động giải phóng DbContext sau khi chạy xong.

4.3.3 Tổ chức đăng ký dịch vụ theo module qua ServiceCollectionExtensions

Để giữ cho tệp Program.cs luôn sạch sẽ và tuân thủ cấu trúc Modular Monolith, các thiết lập DI được chia nhỏ thành các phương thức mở rộng (Extension Methods) tương ứng với từng tầng kiến trúc:

Đóng gói đăng ký DI theo mô-đun qua ServiceCollectionExtensions

```
// src/PropNest.Application/DependencyInjection/ServiceCollectionExtensions.cs
namespace PropNest.Application.DependencyInjection;

public static class ServiceCollectionExtensions
{
        public static IServiceCollection AddApplication(this IServiceCollection services)
        {
        // Đăng ký toàn bộ các Application Services với vòng đời Scoped
      services.AddScoped<IListingService, ListingService>();
      services.AddScoped<IWorkflowService, WorkflowService>();
        services.AddScoped<IWalletService, WalletService>();
        return services;
        }
}
```

4.4 Xử lý HTTP Request qua Middleware Pipeline

4.4.1 Bản chất hàm ủy thác RequestDelegate và cơ chế luân chuyển Russian Doll

Middleware Pipeline (Đường ống Middleware) là xương sống tiếp nhận và xử lý mọi tương tác mạng trong ASP.NET Core. Về bản chất, mỗi Middleware là một hàm ủy thác (Delegate) có chữ ký:

Khai báo hàm ủy thác RequestDelegate

```
public delegate Task RequestDelegate(HttpContext context);
```

Nguyên lý vận hành tuân thủ mô hình 'Búp bê Nga' (Russian Doll) hai chiều:

- **Chiều xuôi (Inbound Request):** Khi request từ mạng đến, nó đi xuyên qua Middleware 1, Middleware 2, ... cho đến Controller Action.
- **Ủy thác 'next(context)':** Mỗi middleware có quyền quyết định gọi 'await next(context)' để bàn giao quyền xử lý cho middleware tiếp theo trong chuỗi.
- **Chiều ngược (Outbound Response):** Sau khi Controller Action hoàn tất, response di chuyển ngược trở lại qua các middleware theo thứ tự đảo ngược trước khi gửi trả về Client.
- **Cơ chế Short-circuiting (Ngắt mạch):** Nếu request không hợp lệ (ví dụ: chưa xác thực JWT, gặp lỗi Exception), Middleware lập tức ngắt mạch bằng cách không gọi 'next(context)', ghi trực tiếp mã lỗi (401, 403, 500) vào response và quay đầu, bảo vệ toàn bộ các tầng nghiệp vụ phía sau.

4.4.2 Thứ tự thực thi chuẩn của Middleware trong ASP.NET Core

Thứ tự sắp xếp Middleware trong Program.cs mang tính chất quyết định tuyệt đối đối với an ninh và sự ổn định của hệ thống:

Bảng 4.4 Thứ tự thực thi chuẩn xác của chuỗi Middleware trong PropNest

Thứ tự	Middleware	Mục đích sử dụng	Lý do bắt buộc về thứ tự
1	ExceptionHandlingMiddleware	Bắt toàn bộ lỗi chưa xử lý và chuẩn hóa ProblemDetails.	Phải đặt ngoài cùng để bọc toàn bộ các middleware phía sau trong khối try-catch.
2	CorrelationIdMiddleware	Đọc hoặc sinh mã vết X-Correlation-Id.	Đặt trước logging để mọi hành động phía sau đều gắn mã vết truy xuất thống nhất.
3	UseHttpsRedirection	Chuyển hướng kết nối HTTP sang HTTPS an toàn.	Bảo vệ thông tin trước khi giải mã dữ liệu nhạy cảm.
4	UseAuthentication	Giải mã JWT token và xác lập danh tính HttpContext.User.	Phải đặt trước UseAuthorization để hệ thống biết user là ai.
5	UseAuthorization	Kiểm tra Role (Seller, Moderator, Admin) của user.	Phải đặt sau Authentication và trước MapControllers để chặn truy cập trái phép.
6	MapControllers	Định tuyến tới Controller Action nghiệp vụ.	Nằm ở đích đến cuối cùng của chuỗi đường ống Pipeline.

Thực nghiệm kỹ thuật: tiến hành thực nghiệm đảo ngược vị trí giữa UseAuthorization() và UseAuthentication() trong Program.cs:

```
// Đoạn mã cố tình tạo lỗi: Singleton Worker lại tiêm trực tiếp DbContext Scoped qua Constructor!
public sealed class ListingAnalyticsWorker(PropNestDbContext dbContext) : BackgroundService
{
        protected override async Task ExecuteAsync(CancellationToken stoppingToken) { /* ... */ }
}
// Program.cs:
builder.Services.AddHostedService<ListingAnalyticsWorker>(); // Đăng ký Singleton
```

Kết quả: Máy chủ từ chối với mã 401 Unauthorized dù Token hoàn toàn hợp lệ

```
GET /api/v1/listings/mine HTTP/1.1
Host: localhost:5000
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... (Token Seller hợp lệ)

HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer

[2026-10-16 09:35:00.012 WAR] Authorization failed. The user is not authenticated.
[2026-10-16 09:35:00.014 INF] AuthenticationMiddleware was not executed yet when Authorization
evaluated User.
```

Giải thích cơ chế: Do UseAuthorization chạy trước nên HttpContext.User.Identity.IsAuthenticated lúc này vẫn là false. AuthorizationMiddleware thấy chưa được xác thực nên lập tức thực hiện **Short-circuiting** ngắt mạch và trả về 401 Unauthorized, ngăn chặn request đi tiếp. Điều này chứng minh UseAuthentication bắt buộc phải đặt trước UseAuthorization.

4.4.3 Xây dựng Custom Middleware trong PropNest

Dự án **PropNest** xây dựng hai Custom Middleware nền tảng:

Chương 1. CorrelationIdMiddleware phục vụ Distributed Tracing (truy vết luồng phân tán):

Hiện thực hóa CorrelationIdMiddleware trong PropNest.Api

```
// src/PropNest.Api/Middleware/CorrelationIdMiddleware.cs
 public sealed class CorrelationIdMiddleware(RequestDelegate next)
 {
        public const string CorrelationIdHeaderName = "X-Correlation-Id";


        public async Task InvokeAsync(HttpContext context)
        {
        // Nếu client không gửi header X-Correlation-Id thì tự sinh mới
        if (!context.Request.Headers.TryGetValue(CorrelationIdHeaderName, out var correlationId)
        || string.IsNullOrWhiteSpace(correlationId))
        {
        correlationId = Guid.NewGuid().ToString();
        }

        // Lưu vào HttpContext.Items để Controller và Service tầng dưới có thể đọc
      context.Items[CorrelationIdHeaderName] = correlationId.ToString();

        // Gắn ngược lại vào Response Header cho Client đối soát
        context.Response.OnStarting(() =>
        {
       context.Response.Headers[CorrelationIdHeaderName] = correlationId.ToString();
        return Task.CompletedTask;
```

```
        });


        await next(context);

        }
}
```

Chương 2. ExceptionHandlingMiddleware chuẩn hóa phản hồi lỗi toàn cục theo chuẩn RFC 7807:

Vòng lặp try-catch bắt lỗi toàn cục trong ExceptionHandlingMiddleware

```
// src/PropNest.Api/Middleware/ExceptionHandlingMiddleware.cs
 public async Task InvokeAsync(HttpContext context)
 {
        try
        {
        await next(context);
        }
        catch (Exception exception)
        {
        LogUnhandledException(logger, exception, context.Request.Path);
        var problem = CreateProblemDetails(context, exception);
        context.Response.StatusCode = problem.Status ?? StatusCodes.Status500InternalServerError;
        await context.Response.WriteAsJsonAsync(problem);
        }
 }
```

4.4.4 Phân biệt chuyên sâu giữa Middleware và Filter trong ASP.NET Core

Bản chất kiến trúc: Mặc dù cùng phục vụ các mối quan tâm xuyên suốt (Cross-cutting Concerns), Middleware và Filter thuộc về hai tầng trừu tượng hoàn toàn khác biệt:

Bảng 4.5 So sánh toàn diện sự khác biệt bản chất giữa Middleware và Filter

Tiêu chí phân biệt	**Middleware (Đường ống mạng)**	**Filter (Bộ lọc MVC)**
Tầng kiến trúc (Layer)	Nằm ở tầng máy chủ mạng ASP.NET Core cơ sở (Host Level). Tiếp nhận request ngay sau Kestrel.	Nằm bên trong tầng ASP.NET Core MVC / Web API (Framework Level). Chỉ thực thi sau khi Routing đã chọn được Controller/Action.
Ngữ cảnh truy cập (Context)	Chỉ truy cập được HttpContext cấp thấp (Request Headers, Query, Stream Body, Response).	Truy cập được FilterContext, ActionExecutingContext, ActionArguments, Controller instance và ModelState.
Truy cập MVC Metadata	KHÔNG biết request này sẽ đi vào Controller nào, Action nào hay tham số DTO nghiệp vụ là gì.	Biết chính xác Controller/Action được gọi, đọc được toàn bộ Attributes/Metadata gán trên Action.
Truy cập dữ liệu Body	Chỉ đọc được dưới dạng Stream thô (Stream reading), nếu đọc xong mà không tua lại (seek) sẽ làm mất dữ liệu của Controller.	Đọc được các đối tượng DTO đã qua xử lý Model Binding và hoàn tất thẩm định kiểu dữ liệu.

Phạm vi áp dụng (Scope)	Áp dụng toàn cục (Global) cho MỌI request gửi tới máy chủ (kể cả Static files, Swagger, WebSockets).	Áp dụng linh hoạt: có thể đặt Global, đặt trên từng Controller, hoặc chỉ áp dụng cho một Action cụ thể thông qua Attribute.
Trường hợp sử dụng kinh điển	Exception Handling toàn cục, Correlation ID Tracing, Nén dữ liệu Gzip, CORS, Authentication, Https Redirection.	Idempotency check theo nghiệp vụ, Validation Filter, Audit Log dữ liệu thay đổi của DTO, Custom Authorization Filter.

Minh chứng mã nguồn tương ứng: Minh chứng thực tế trong PropNest: CorrelationIdMiddleware là Middleware vì cần gán mã vết cho mọi kết nối mạng thô; còn IdempotentAttribute là Action Filter vì cần truy cập trực tiếp tham số DTO đầu vào (ActionArguments) và giá trị Header Idempotency-Key để lưu kết quả vào bộ đệm.

4.5 Routing, Controller và Chuẩn hóa phản hồi Web API

4.5.1 Attribute Routing, Route Constraints và Model Binding

Trong ASP.NET Core Web API, Controller là lớp đón nhận và phản hồi các HTTP Request. Hệ thống định tuyến Attribute Routing cung cấp khả năng gán trực tiếp URL template lên lớp Controller và từng Action:

- **[ApiController]:** Thuộc tính bắt buộc, kích hoạt các hành vi tự động: tự động thẩm định dữ liệu đầu vào (Model Validation) và trả về 400 Bad Request nếu dữ liệu lỗi; tự động suy luận nguồn dữ liệu binding ([FromBody], [FromQuery]); chuẩn hóa lỗi theo ProblemDetails.
- **[Route("api/v1/listings")]:** Định nghĩa tiền tố đường dẫn RESTful cơ sở, hỗ trợ phân phiên bản API (API Versioning).

- **Route Constraints:** Ký pháp '{listingId:long}' áp đặt ràng buộc kiểu số nguyên 64-bit lên đường dẫn. Nếu client gửi chuỗi ký tự chữ '/api/v1/listings/abc', Routing Engine sẽ lập tức trả về 404 Not Found mà không cho phép request đi sâu vào Controller, giúp ngăn chặn các cuộc tấn công thăm dò và giảm tải cho máy chủ.
- **Model Binding:** Quá trình tự động ánh xạ dữ liệu từ HTTP Request (JSON Body, Route Parameters, Query String, Headers) sang các đối tượng tham số C# (DTOs).

Attribute Routing và Model Binding trong ListingsController

```
// src/PropNest.Api/Controllers/ListingsController.cs
 [ApiController]
 [Authorize(Roles = "Seller")]
 [Route("api/v1/listings")]
 public sealed class ListingsController(
            ICurrentUser currentUser,
            IListingService listingService,
            IWorkflowService workflowService) : ControllerBase
 {
            // Route constraint: chỉ chấp nhận số nguyên long
    [HttpGet("{listingId:long}")]
            public async Task<ActionResult<ListingDto>> GetById(long listingId, CancellationToken
cancellationToken)
            {
            var listing = await listingService.GetByIdAsync(listingId, cancellationToken);
            return listing is null ? NotFound() : Ok(listing);
            }

            [HttpPost]
            public async Task<ActionResult<ListingDto>> Create(CreateListingRequest request,
CancellationToken cancellationToken)
            {
            var listing = await listingService.CreateAsync(new CreateListingCommand(
            GetRequiredUserId(),
            request.Title,
            request.Description,
```

```
            request.PropertyType,
            request.ListingType,
            request.Price,
            request.Area,
            request.City,
            request.District,
            request.Ward,
            request.Address), cancellationToken);

            return CreatedAtAction(nameof(GetById), new { listingId = listing.ListingId }, listing);
            }
}
```

4.5.2 So sánh Controller-based API và Minimal API

So sánh Controller-based Web API và Minimal API, cùng lý do PropNest chọn Controller-based:

Bảng 4.6 So sánh đối sánh giữa Controller-based Web API và Minimal API

Tiêu chí so sánh	**Controller-based Web API (PropNest chọn)**	**Minimal API (.NET 6+)**
Cấu trúc tổ chức	Tổ chức mã nguồn theo lớp (Class) và thư mục chức năng rõ ràng, chuẩn mực MVC cổ điển.	Khai báo trực tiếp trên Program.cs hoặc gom nhóm qua các hàm mở rộng RouteGroupBuilder.

Hỗ trợ Action Filter	Hỗ trợ đầy đủ bộ lọc chuyên sâu: ActionFilter, ResultFilter, ExceptionFilter, ResourceFilter.	Chỉ hỗ trợ Endpoint Filters (ít linh hoạt hơn, không truy cập sâu được ModelState và Controller Context).
Phân quyền phức tạp	Gán trực tiếp [Authorize(Roles=..., Policy=...)] ở cấp lớp Controller hoặc từng Action.	Gán qua phương thức mở rộng '.RequireAuthorization()', dễ trùng lặp mã khi có nhiều endpoint.
Hiệu năng và Cold-Start	Tốn một lượng nhỏ chi phí Reflection khi ứng dụng khởi động để quét Controller.	Thời gian khởi động nhanh hơn, tiết kiệm bộ nhớ hơn vì ánh xạ trực tiếp delegate.
Phù hợp dự án	Cực kỳ phù hợp cho hệ thống lớn cấp doanh nghiệp (Enterprise), Clean Architecture, phân quyền đa tầng.	Phù hợp cho Microservices độc lập, các dịch vụ nhỏ lẻ (Nano-services) hoặc Serverless API.

Luận cứ lựa chọn: Lý do PropNest chọn Controller: Hệ thống có 6 phân hệ lớn với hàng chục endpoint nghiệp vụ, phân quyền đa vai trò (Seller, Moderator, Admin) và yêu cầu Action Filter nghiêm ngặt. Controller giúp tổ chức mã nguồn có tính đóng gói cao và dễ bảo trì.

Thực nghiệm khi dữ liệu đầu vào vi phạm Data Annotations:

```
POST /api/v1/listings HTTP/1.1
Host: localhost:5000
Content-Type: application/json
Authorization: Bearer eyJhbGci...
{ "title": "Nhà bán", "price": -5000000, "area": 0 }

HTTP/1.1 400 Bad Request
Content-Type: application/problem+json; charset=utf-8

{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "traceId": "c89f1d2e-4b7a-49a3-832c-39a0f48b1102",
  "errors": {
    "Title": ["Tiêu đề phải từ 10 đến 200 ký tự."],
    "Price": ["Giá bán/thuê phải từ 1.000 VNĐ đến 100 tỷ VNĐ."],
    "Area": ["Diện tích phải từ 1 m2 đến 10.000 m2."],
    "Description": ["Mô tả tin đăng bắt buộc phải có."]
  }
}
```

4.5.3 So sánh Task<IActionResult> và Task<ActionResult<T>> trong Controller

Trong ASP.NET Core, lập trình viên thường đứng trước hai lựa chọn kiểu trả về cho các Action bất đồng bộ trong Controller:

Bảng 4.7 So sánh chi tiết giữa Task<IActionResult> và Task<ActionResult<T>>

Tiêu chí so sánh	**Task<IActionResult>**	**Task<ActionResult<T>> (Khuyên dùng)**
Định kiểu dữ liệu trả về	Chỉ trả về interface chung IActionResult, không chỉ định rõ kiểu dữ liệu cụ thể T.	Chỉ định tường minh kiểu dữ liệu DTO trả về (ví dụ: ActionResult<ListingDto>).
Tương thích OpenAPI / Swagger	Swagger KHÔNG THỂ tự động nhận biết cấu trúc Response Body (phải bổ sung thủ công attribute [ProducesResponseType(typeof(T), 200)]).	Swagger TỰ ĐỘNG phân tích và hiển thị Schema JSON hoàn chỉnh của T trên tài liệu API mà không cần thêm attribute.
Cú pháp lệnh return	Bắt buộc phải bọc trong các hàm trợ giúp: return Ok(dto); return NotFound(); return BadRequest();	Cực kỳ linh hoạt: Có thể 'return Ok(dto);' HOẶC 'return dto;' trực tiếp nhờ cơ chế implicit conversion.
Khuyến nghị sử dụng	Dùng cho các endpoint KHÔNG CÓ Body dữ liệu trả về (ví dụ: HTTP 204 NoContent, HTTP 202 Accepted, HTTP 200 trống).	Dùng cho TẤT CẢ các endpoint có trả về dữ liệu (GET chi tiết, POST tạo mới, PUT cập nhật).

4.5.4 Chuẩn hóa lỗi API theo đặc tả RFC 7807 ProblemDetails

Đặc tả RFC 7807 (Problem Details for HTTP APIs) là tiêu chuẩn quốc tế quy định định dạng JSON thống nhất cho mọi thông báo lỗi do máy chủ sinh ra. Thay vì trả về

chuỗi thông báo lộn xộn hoặc làm lộ vết gọi hàm (Stack Trace) nguy hiểm, máy chủ chuẩn hóa cấu trúc lỗi gồm:

- **status:** Mã trạng thái HTTP chuẩn số nguyên (400, 401, 403, 404, 409, 500).
- **title:** Tiêu đề ngắn gọn mô tả phân loại lỗi.
- **detail:** Giải thích chi tiết nguyên nhân cụ thể dẫn đến lỗi.
- **instance:** Đường dẫn URI phát sinh lỗi.

Bảng 4.8 Bảng ánh xạ mã HTTP Status và Exception theo chuẩn RFC 7807

Mã HTTP Status	**Tên chuẩn RFC 7807**	**Loại Exception trong C#**	**Nguồn gốc phát sinh & Ý nghĩa**
401	Unauthorized	AuthenticationFailure (Không qua Exception)	Phát sinh tự động bởi AuthenticationMiddleware khi Request không có JWT Token, Token hết hạn hoặc sai chữ ký. Trả về header WWW-Authenticate: Bearer error='invalid_token'.
403	Forbidden	UnauthorizedAccessException	Người dùng đã gửi JWT hợp lệ nhưng không sở hữu tài nguyên hoặc không thỏa mãn Policy yêu cầu (ví dụ: Seller gọi API kiểm duyệt).
404	Not Found	KeyNotFoundException	Không tìm thấy ListingId, UserId hoặc thực thể tương ứng trong hệ thống.

400	Bad Request	DomainException / ArgumentException	Vi phạm điều kiện bất biến (Invariants) của Domain Model hoặc tham số đầu vào rỗng/âm.
409	Conflict	DbUpdateConcurrencyException	Xung đột ghi đè dữ liệu đồng thời khi kiểm tra RowVersion trên bảng Listings/Wallets. Tuyệt đối không map mọi InvalidOperationException.
500	Internal Server Error	System.Exception (Tổng quát)	Lỗi hạ tầng hoặc sập kết nối cơ sở dữ liệu. Luôn che giấu Stack Trace và lỗi SQL nội bộ trong môi trường Production.

Mẫu phản hồi lỗi JSON chuẩn RFC 7807 khi xảy ra xung đột đồng thời:

Cấu trúc phản hồi lỗi ProblemDetails chuẩn RFC 7807

```
{
  "type": "https://tools.ietf.org/html/rfc7807",
  "title": "The database record has been modified by another transaction.",
  "status": 409,
  "detail": "Optimistic Concurrency check failed: RowVersion mismatch.",
  "instance": "/api/v1/listings/102"
}
```

4.6 Quản lý dự án chuyên nghiệp với .NET Tooling

4.6.1 Quản lý dependencies tập trung với Central Package Management (CPM)

Trong các giải pháp phần mềm doanh nghiệp gồm nhiều project (.NET Solution), việc quản lý phiên bản các thư viện NuGet độc lập trong từng tệp .csproj thường dẫn đến vấn đề phân mảnh phiên bản (Version Mismatch). Ví dụ: Project API dùng EntityFrameworkCore 8.0.4, trong khi project Infrastructure lại dùng 8.0.0, gây ra các lỗi xung đột DLL rất khó gỡ lỗi lúc chạy ứng dụng.

Dự án PropNest áp dụng tính năng Central Package Management (CPM) của .NET SDK. Toàn bộ phiên bản thư viện được định nghĩa DUY NHẤT TẠI MỘT NƠI trong tệp 'Directory.Packages.props' ở thư mục gốc của Solution. Các tệp .csproj chỉ khai báo gói cần dùng mà không cần chỉ định phiên bản:

Cấu hình quản lý thư viện tập trung trong Directory.Packages.props

```
<!-- Directory.Packages.props (Đặt tại thư mục gốc Solution - Dùng chung cho Mục 3.2 và 4.6.1) -->
 <Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
    <CentralPackageTransitivePinningEnabled>true</CentralPackageTransitivePinningEnabled>
  </PropertyGroup>

  <ItemGroup>
        <!-- Nhóm 1: ASP.NET Core, Identity & Security (.NET 8.0 LTS) -->
        <PackageVersion Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="8.0.21"
/>
        <PackageVersion Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore"
Version="8.0.21" />
        <PackageVersion Include="Microsoft.Extensions.Identity.Stores" Version="8.0.21" />
        <PackageVersion Include="System.IdentityModel.Tokens.Jwt" Version="7.7.1" />

        <!-- Nhóm 2: Entity Framework Core 8 & SQL Server -->
        <PackageVersion Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.21" />
        <PackageVersion Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.21" />
        <PackageVersion Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.21" />
        <PackageVersion Include="Microsoft.EntityFrameworkCore.Sqlite" Version="8.0.21" />

        <!-- Nhóm 3: Message Bus & Distributed Saga (MassTransit) -->
        <PackageVersion Include="MassTransit.RabbitMQ" Version="8.2.5" />
        <PackageVersion Include="MassTransit.EntityFrameworkCore" Version="8.2.5" />
```

```
        <!-- Nhóm 4: Logging & Diagnostics (Serilog) -->
        <PackageVersion Include="Serilog.AspNetCore" Version="8.0.3" />
        <PackageVersion
Include="Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore" Version="8.0.21" />

        <!-- Nhóm 5: Tác vụ nền & Thẩm định dữ liệu -->
        <PackageVersion Include="Quartz.Extensions.Hosting" Version="3.13.1" />
        <PackageVersion Include="FluentValidation.AspNetCore" Version="11.3.0" />
        <PackageVersion Include="Swashbuckle.AspNetCore" Version="6.6.2" />
        <PackageVersion Include="CloudinaryDotNet" Version="1.26.2" />

        <!-- Nhóm 6: Testing Frameworks (xUnit & Moq) -->
        <PackageVersion Include="Microsoft.NET.Test.Sdk" Version="17.11.1" />
        <PackageVersion Include="xunit" Version="2.9.2" />
        <PackageVersion Include="xunit.runner.visualstudio" Version="2.8.2" />
        <PackageVersion Include="Moq" Version="4.20.72" />
  </ItemGroup>
</Project>
```

4.6.2 Kiểm soát chất lượng code với Roslyn Analyzers và TreatWarningsAsErrors

Để đảm bảo tính kỷ luật và chất lượng code cao nhất cho một dự án làm việc nhóm theo chuẩn công nghiệp, **PropNest** kích hoạt bộ quy tắc phân tích tĩnh (Static Code Analyzers) nghiêm ngặt nhất thông qua tệp 'Directory.Build.props':

- **<TreatWarningsAsErrors>true</TreatWarningsAsErrors>:** Biến toàn bộ cảnh báo của trình biên dịch (Warnings) thành lỗi biên dịch (Compile Errors). Bất kỳ

dòng code nào thừa biến, thiếu CancellationToken, hoặc dùng sai cú pháp đều khiến lệnh build thất bại, buộc lập trình viên phải sửa ngay.

- **<EnforceCodeStyleInBuild>true</EnforceCodeStyleInBuild>:** Bắt buộc tuân thủ quy chuẩn định dạng code (Code Style Rules) lúc build.
- **Tuân thủ cảnh báo phân tích hiệu năng:** Ví dụ cảnh báo CA1848 bắt buộc dùng LoggerMessage delegate; cảnh báo CA2016 bắt buộc chuyển tiếp CancellationToken vào các hàm bất đồng bộ.

Cấu hình TreatWarningsAsErrors trong Directory.Build.props

```
<!-- Directory.Build.props (Đặt tại thư mục gốc Solution - Dùng chung cho Mục 3.2 và 4.6.2) -->
 <Project>
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <AnalysisLevel>latest-recommended</AnalysisLevel>
          <!-- Bỏ qua cảnh báo CA1861 đối với việc cấp phát mảng hằng số trong các biểu thức
Linq/Assert -->
    <NoWarn>$(NoWarn);CA1861</NoWarn>
  </PropertyGroup>
 </Project>
```

Build thành công sạch sẽ 100% khi bật TreatWarningsAsErrors=true:

```
C:\Users\GiaBao\source\repos\PropNest> dotnet build --configuration Release
  Determining projects to restore...
  All projects are up-to-date for restore.
  PropNest.Domain -> bin\Release\net8.0\PropNest.Domain.dll
  PropNest.Application -> bin\Release\net8.0\PropNest.Application.dll
  PropNest.Infrastructure -> bin\Release\net8.0\PropNest.Infrastructure.dll
  PropNest.Api -> bin\Release\net8.0\PropNest.Api.dll
  PropNest.Workflow.Worker -> bin\Release\net8.0\PropNest.Workflow.Worker.dll

Build succeeded.
        0 Warning(s)
        0 Error(s)

Time Elapsed 00:00:03.42
```

4.7 So sánh và Phản biện Công nghệ (Comparative Analysis & Counter-Arguments)

4.7.1 So sánh đối sánh ASP.NET Core 8 với các công nghệ Backend tương đương

Bảng đối sánh đa chiều giữa ASP.NET Core 8 với Spring Boot, NestJS, FastAPI và Go:

Bảng 4.9 So sánh đối sánh đa chiều giữa ASP.NET Core 8 và các nền tảng Backend hiện đại

Tiêu chí kỹ thuật	ASP.NET Core 8 (.NET)	Spring Boot 3/4 (Java)	NestJS (Node.js)	Go + Gin (Golang)

Ngôn ngữ & Hệ thống kiểu	C# 12: Định kiểu tĩnh mạnh, hỗ trợ Nullable References, Record, Pattern Matching tối tân.	Java 17/21: Định kiểu tĩnh, cú pháp chặt chẽ hướng đối tượng cổ điển.	TypeScript: Định kiểu tĩnh trên nền JavaScript, chạy qua Node.js runtime.	Go: Định kiểu tĩnh, cú pháp tối giản, không hỗ trợ OOP đầy đủ (dùng Struct/Interface).
Hiệu năng I/O & Xử lý đồng thời	Rất cao (Máy chủ Kestrel tối ưu bộ nhớ với Span<T>, Zero-Allocation, Async/Await luồng non-blocking).	Cao (Đã cải thiện với Virtual Threads Project Loom trong Java 21).	Trung bình-Khá (Bị giới hạn bởi kiến trúc Event-Loop đơn luồng của Node.js).	Cực cao (Goroutine siêu nhẹ, chiếm ít tài nguyên, khởi động gần như tức thì).
Dependency Injection	Tích hợp sẵn trong nhân (Built-in IoC), cấu hình gọn gàng, hiệu năng cao, nhẹ nhàng.	Rất mạnh mẽ (Spring IoC / Spring Context), nhưng phụ thuộc nặng nề vào Reflection và Annotation.	Mô hình Module/Provider lấy cảm hứng từ Angular, dựa trên Reflection metadata.	Không có sẵn trong nhân; lập trình viên tự tiêm phụ thuộc thủ công hoặc dùng thư viện ngoài (Google Wire).

Tầng ORM / CSDL	Entity Framework Core 8: ORM mạnh mẽ, hỗ trợ Migration, LINQ an toàn kiểu tuyệt đối, Interceptors.	Spring Data JPA / Hibernate: Đầy đủ tính năng, nhưng cấu hình phức tạp và dễ gặp lỗi N+1 khó kiểm soát.	TypeORM / Prisma: Tiện lợi nhưng khả năng tối ưu câu lệnh SQL phức tạp kém hơn EF Core.	GORM / sqlc: GORM dễ dùng nhưng chậm; sqlc sinh mã từ SQL thuần tốt nhưng đòi hỏi viết SQL bằng tay.
Tính hoàn thiện của Framework	Framework 'đủ bộ' chuẩn mực: Tích hợp sẵn Auth, Middleware, HealthChecks, RateLimiting, Logging.	Hệ sinh thái đồ sộ nhất cho doanh nghiệp lớn, nhiều thư viện tích hợp sẵn.	Cung cấp khung kiến trúc chuẩn cho Node.js, nhưng phụ thuộc nhiều package bên thứ ba.	Framework tối giản (Minimalist), chỉ cung cấp Router và Middleware; các phần khác phải tự ghép nối.

4.7.2 Những trường hợp cụ thể KHÔNG NÊN DÙNG ASP.NET Core (Trải nghiệm thực tế của nhóm)

Phản biện kỹ thuật: Dựa trên trải nghiệm triển khai trực tiếp đồ án PropNest, nhóm đúc kết 3 trường hợp cụ thể KHÔNG NÊN LỰA CHỌN ASP.NET Core:

1. **Các dịch vụ siêu nhỏ (Micro-nano services) hoặc Serverless cần Cold-Start cực nhanh:** Đối với các hàm Serverless (như AWS Lambda, Cloud Run) yêu cầu thời gian khởi động lạnh (Cold-start) dưới 50ms và mức tiêu thụ RAM dưới 30MB, runtime của .NET vẫn có độ trễ khởi tạo JIT và dung lượng container lớn

hơn đáng kể so với một tệp nhị phân đơn lẻ của Go (Golang) hoặc một script Node.js nhẹ.

2. **Các dự án Prototype/MVP ngắn hạn của đội ngũ phát triển không chuyên .NET:** Nếu mục tiêu là dựng một ứng dụng thử nghiệm (PoC) hoặc MVP trong vòng 2-3 ngày với nhóm lập trình viên chưa quen thuộc với C# và hệ sinh thái Microsoft, chi phí học tập và khối lượng mã khai báo (Boilerplate) của ASP.NET Core lớn hơn nhiều so với FastAPI (Python) hoặc Express.js.
3. **Môi trường phần cứng giới hạn ngặt nghèo không hỗ trợ .NET Runtime:** Trong các thiết bị IoT công nghiệp hoặc hệ thống nhúng có RAM dưới 128MB hoặc hệ điều hành không hỗ trợ .NET Runtime chính thức, ngôn ngữ C, C++ hoặc Rust là lựa chọn bắt buộc.

KẾT LUẬN CHƯƠNG 4

Chương 4 đã phân tích toàn diện và sâu sắc các nền tảng kỹ thuật cốt lõi của hệ sinh thái .NET 8 và ngôn ngữ C# 12, đi từ các nguyên lý lập trình cơ bản đến các cơ chế kiến trúc nâng cao:

1. **Ngôn ngữ C# 12 hiện đại:** Làm chủ hệ thống định kiểu tĩnh mạnh (Type System), Nullable Reference Types, tính đóng gói (Encapsulation), tính bất biến với Record Types, Primary Constructors, Pattern Matching và Source Generated Logging hiệu năng cao.
2. **Kiến trúc thực thi ASP.NET Core:** Nắm vững nguyên lý hoạt động của máy chủ Kestrel Web Server, cơ chế xử lý I/O bất đồng bộ và mô hình điều phối Generic Host.
3. **Dependency Injection và Quản lý vòng đời dịch vụ:** Thấu hiểu nguyên lý Inversion of Control, kiểm soát chặt chẽ ranh giới giữa Transient, Scoped, Singleton và loại trừ hoàn toàn rủi ro Captive Dependency.

4. **Đường ống xử lý yêu cầu Middleware Pipeline:** Vận dụng hàm ủy thác RequestDelegate để xây dựng các Custom Middleware thực hiện truy vết luồng phân tán (Correlation ID) và bắt lỗi toàn cục chuẩn RFC 7807 ProblemDetails.
5. **Thiết kế RESTful Web API và Bộ công cụ .NET Tooling:** Triển khai Controller-based API an toàn kiểu dữ liệu, tận dụng Central Package Management (CPM) và Roslyn Analyzers để duy trì chất lượng mã nguồn cấp doanh nghiệp.

Các kiến thức nền tảng vững chắc này chính là tiền đề then chốt để nhóm nghiên cứu tiếp tục triển khai các kỹ thuật chuyên sâu ở các chương tiếp theo, bao gồm thiết kế CSDL với Entity Framework Core, điều phối giao dịch phân tán Saga Orchestration và tích hợp phân hệ trí tuệ nhân tạo kiểm duyệt tin tự động.

CHƯƠNG 5 THIẾT KẾ VÀ QUẢN LÝ CƠ SỞ DỮ LIỆU VỚI ENTITY FRAMEWORK CORE

5.1 Vai trò của cơ sở dữ liệu trong hệ thống Backend

5.1.1 Lưu trữ dữ liệu bền vững (Database Persistence)

Khi một ứng dụng ASP.NET Core chạy trên máy chủ (Kestrel Web Server), toàn bộ đối tượng, biến và dữ liệu phiên làm việc đều nằm trên bộ nhớ RAM. **RAM** có tốc độ xử lý cực kỳ nhanh nhưng lại có nhược điểm lớn: **sẽ mất sạch dữ liệu khi mất điện, ứng dụng bị lỗi (crash), hoặc khi chúng ta restart lại server / container.**

Do đó, một hệ thống Backend bắt buộc phải có tầng lưu trữ dữ liệu bền vững (Database Persistence Layer), thường là lưu xuống ổ cứng (SSD/HDD). Nhiệm vụ của tầng này không chỉ đơn giản là "lưu lại dữ liệu", mà còn đảm bảo dữ liệu luôn chính xác, không bị thất lạc hay sai lệch qua thời gian. Trong các hệ quản trị cơ sở dữ liệu hiện đại như SQL Server, để dữ liệu không bị hỏng khi mất điện đột ngột, hệ thống áp dụng cơ chế **Write-Ahead Logging (WAL):** Mọi thao tác ghi dữ liệu đều phải được ghi nhanh vào tệp nhật ký (Transaction Log .ldf) trước, sau đó mới chính thức cập nhật vào các tệp dữ liệu chính (.mdf).

5.1.2 Tính toàn vẹn dữ liệu (Data Integrity) và nguyên lý ACID

Đối với các hệ thống backend có yếu tố giao dịch tài chính, thương mại điện tử hoặc quản lý tài sản giá trị cao như nền tảng bất động sản PropNest, tính toàn vẹn dữ liệu là yếu tố quan trọng nhất. Tính toàn vẹn này được đo lường và bảo đảm thông qua bốn thuộc tính kinh điển của mô hình giao dịch ACID:

- **Tính nguyên tử (Atomicity)**: Giao dịch được coi là một đơn vị công việc **không thể phân chia (All-or-Nothing)**. Toàn bộ chuỗi các thao tác ghi dữ liệu thành phần bắt buộc phải thành công trọn vẹn; nếu có bất kỳ một thao tác nào thất bại, toàn bộ giao dịch phải được **khôi phục về trạng thái ban đầu (Rollback)**. Ví dụ trong PropNest, thao tác Seller thanh toán mua gói VIP bao

gồm: (1) trừ tiền trong bảng Wallets, (2) tạo bản ghi biến động trong WalletTransactions, (3) nâng cấp gói tin và cập nhật ngày hết hạn trong Listings, và (4) ghi sự kiện vào OutboxMessages. Bốn thao tác này phải được bao bọc trong cùng **một Transaction duy nhất**; không bao giờ được phép xảy ra tình trạng ví đã bị trừ tiền nhưng tin đăng vẫn chưa được kích hoạt gói.

- **Tính nhất quán (Consistency)**: Giao dịch chỉ có thể **chuyển đổi** cơ sở dữ liệu từ một **trạng thái hợp lệ này** sang một t**rạng thái hợp lệ khác**. Mọi ràng buộc toàn vẹn (Integrity Constraints) như khóa chính (Primary Key), khóa ngoại (Foreign Key), chỉ mục duy nhất (Unique Index) và các ràng buộc kiểm tra (Check Constraints – ví dụ: số dư ví không được âm, giá tin đăng ≥ 0) đều phải được **kiểm chứng nghiêm ngặt** trước khi lệnh COMMIT kết thúc.
- **Tính cô lập (Isolation)**: Quy định khả năng kiểm soát sự tương tác giữa các giao dịch thực thi đồng thời (Concurrent Transactions). Trong môi trường đa luồng có lưu lượng truy cập lớn, việc nhiều request cùng đọc/ghi vào một bản ghi dữ liệu có thể dẫn đến các lỗi như:
    - Đọc dữ liệu rác (Dirty Read): Đọc phải dữ liệu mà một giao dịch khác đang sửa dở dang và sau đó giao dịch đó bị hủy.
    - Đọc không lặp lại (Non-repeatable Read): Vừa đọc xong một dòng, một người khác vào sửa mất, đọc lại lần nữa thì thấy dữ liệu đã đổi khác.
    - Dòng bóng ma (Phantom Read): Đọc danh sách tin theo một tiêu chí, một người khác vừa chèn thêm tin mới thỏa mãn tiêu chí đó, đọc lại thì thấy danh sách dài hơn.
- **Tính bền vững (Durability)**: Một khi giao dịch đã được hệ thống thông báo commit thành công, các thay đổi dữ liệu đó sẽ tồn tại vĩnh viễn trong hệ thống lưu trữ và không thể bị mất đi, ngay cả khi xảy ra sự cố sập nguồn phần cứng hoặc hệ điều hành gặp lỗi nghiêm trọng ngay sau thời điểm commit.

5.2 Mô hình cơ sở dữ liệu quan hệ (Relational Database)

5.2.1 Microsoft SQL Server và ưu thế cho hệ thống doanh nghiệp

Mô hình cơ sở dữ liệu quan hệ (Relational Database Model) do nhà khoa học máy tính Edgar F. Codd đề xuất năm 1970, dựa trên nền tảng toán học của lý thuyết tập hợp và các khái niệm về quan hệ. Trong mô hình này, dữ liệu được tổ chức thành các quan hệ (Relations), mỗi quan hệ có thể được biểu diễn dưới dạng một bảng gồm các bộ (Tuples) và các thuộc tính (Attributes). Các quan hệ có thể được chuẩn hóa theo các dạng chuẩn như 1NF, 2NF, 3NF và BCNF nhằm giảm dư thừa dữ liệu và hạn chế các dị thường khi chèn, cập nhật và xóa. Các phép toán trên quan hệ là cơ sở cho việc truy vấn và thao tác dữ liệu trong mô hình quan hệ.

Microsoft SQL Server 2022 được lựa chọn làm hệ quản trị cơ sở dữ liệu trung tâm của hệ thống **PropNest** nhờ các ưu điểm nổi bật:

1. **Bộ tối ưu hóa truy vấn thông minh (Cost-Based Query Optimizer)**: Tự động tính toán để chọn cách thực thi câu lệnh SQL nhanh nhất (dùng Index quét nhanh hay quét toàn bảng), tiết kiệm tối đa CPU và bộ nhớ.
2. **Khóa dữ liệu linh hoạt (Row-level Locking)**: Khi cập nhật một tin đăng, SQL Server chỉ khóa đúng dòng tin đó chứ không khóa toàn bộ bảng, giúp hàng nghìn người khác vẫn xem và sửa các tin khác bình thường.
3. **Cột rowversion chuyên biệt**: SQL Server có sẵn kiểu dữ liệu rowversion – một chuỗi nhị phân tự động tăng giá trị mỗi khi dòng dữ liệu có sự thay đổi. Đây là chìa khóa để triển khai tính năng chống ghi đè dữ liệu (Optimistic Concurrency) cực kỳ đơn giản và hiệu quả trong .NET.
4. **Hỗ trợ xử lý JSON trực tiếp**: SQL Server cho phép lưu trữ chuỗi JSON và cung cấp sẵn các hàm để truy vấn dữ liệu bên trong JSON. Nhờ đó, PropNest có thể lưu toàn bộ lịch sử các trường thay đổi của tin đăng (DeltasJson) vào một cột duy nhất mà không cần phải cài thêm cơ sở dữ liệu NoSQL phức tạp.

5.2.2 So sánh Relational Database và NoSQL Database trong bài toán Bất động sản

Việc lựa chọn công nghệ lưu trữ cho hệ thống PropNest đòi hỏi sự phân tích sâu sắc dưới lăng kính của **Định lý CAP** (Consistency - Tính nhất quán, Availability - Tính sẵn sàng, Partition Tolerance - Khả năng chịu lỗi phân vùng mạng) và hai mô hình triết lý đối lập: **ACID** (chặt chẽ) và **BASE** (Basically Available, Soft-state, Eventual consistency - linh hoạt, nhất quán cuối cùng).

Bảng 5.1 Bảng so sánh chuyên sâu giữa RDBMS (SQL Server) và NoSQL (Document/Key-Value)

Tiêu chí so sánh	Relational Database (MS SQL Server)	NoSQL Database (MongoDB / Redis)	Đánh giá lựa chọn cho hệ thống PropNest
Mô hình dữ liệu	Bảng quan hệ định kiểu tĩnh, lược đồ chặt chẽ (Schema-on-Write).	Bán cấu trúc JSON/BSON, lược đồ động linh hoạt (Schema-on-Read).	Chọn RDBMS: Thực thể Bất động sản có cấu trúc rõ ràng (Diện tích, Giá, Vị trí địa lý, Chủ sở hữu).
Bảo đảm giao dịch	Tuân thủ tuyệt đối chuẩn ACID cấp độ cao (Strong Consistency).	Chủ yếu tuân thủ chuẩn BASE, tính nhất quán cuối cùng (Eventual Consistency).	Chọn RDBMS: Nghiệp vụ tài chính (Ví tiền, Giao dịch mua gói, Trừ tiền) bắt buộc tính nhất quán tuyệt đối.
Xử lý quan hệ (Relationships)	Hỗ trợ mạnh mẽ phép JOIN, ràng buộc khóa ngoại FK toàn vẹn dữ liệu.	Dữ liệu phi chuẩn hóa (Embedded Documents), không hỗ trợ hoặc JOIN rất chậm.	Chọn RDBMS: Quan hệ giữa Users, Wallets, Listings, Histories và Reviews là quan hệ đa tầng chặt chẽ.
Chiến lược mở rộng (Scaling)	Mở rộng theo chiều dọc (Vertical Scaling - tăng CPU, RAM), Read Replicas.	Mở rộng theo chiều ngang (Horizontal Scaling - Sharding phân tán cụm).	Chọn RDBMS: Phù hợp hoàn hảo với kiến trúc Modular Monolith, đơn giản hóa vận hành hệ thống.

Lưu trữ lịch sử biến động	Kết hợp cột quan hệ với cột NVARCHAR(MAX) lưu JSON Delta.	Lưu toàn bộ document mới mỗi lần cập nhật.	RDBMS tối ưu hơn: Dùng quan hệ để khóa dữ liệu, dùng JSON nội hợp để lưu lịch sử vi sai cực kỳ gọn nhẹ.

Tóm lại: Đối với các hệ thống cần độ chính xác cao về tài chính và dữ liệu liên kết nhiều như PropNest, SQL Server là sự lựa chọn an toàn và tối ưu nhất.

5.3 ORM (Object-Relational Mapping)

5.3.1 Khái niệm ORM và lợi ích trong phát triển phần mềm hiện đại

Trong mô hình lập trình hướng đối tượng (OOP) của ngôn ngữ C#, thế giới thực được mô hình hóa thành các lớp (Classes), đối tượng (Objects), tính đóng gói (Encapsulation), quan hệ kế thừa (Inheritance) và quan hệ điều hướng đồ thị (Object Graph Navigation: listing.OwnerUser.Wallet). Ngược lại, cơ sở dữ liệu quan hệ tổ chức dữ liệu dưới dạng các bảng hai chiều phẳng, kiểu dữ liệu vô hướng (Scalar Types) và liên kết bằng khóa ngoại.

Sự mâu thuẫn mang tính bản chất giữa hai mô hình tư duy này được khoa học phần mềm định nghĩa là **"Trở ngại không tương thích giữa Hướng đối tượng và Quan hệ" (Object-Relational Impedance Mismatch)**, thể hiện ở bốn khía cạnh:

1. **Sự khác biệt về kiểu dữ liệu (Data Type Mismatch):** Ví dụ enum trong C# không có kiểu tương đương trực tiếp trong SQL Server chuẩn; chuỗi nhị phân byte[] biểu diễn RowVersion.
2. **Cấu trúc quan hệ (Structural Mismatch):** Đối tượng C# có thể chứa danh sách đối tượng con (ICollection<ListingHistory>), trong khi SQL Server bắt buộc phải tách bảng và dùng khóa ngoại.

3. **Tính kế thừa (Inheritance Mismatch):** RDBMS không có khái niệm kế thừa lớp; việc biểu diễn cấu trúc kế thừa phải đánh đổi giữa TPH (Table-per-Hierarchy), TPT (Table-per-Type) hoặc TPC (Table-per-Concrete-type).
4. **Bản sắc đối tượng (Object Identity vs Relational Key):** Hai đối tượng trong RAM được coi là giống nhau khi so sánh tham chiếu (object.ReferenceEquals) hoặc theo giá trị thuộc tính, trong khi bảng quan hệ xác định danh tính qua Khóa chính (Primary Key).

Object-Relational Mapping (ORM) ra đời như một người "thông dịch viên" đứng ở giữa 2 khía cạnh. Nó tự động dịch các thao tác trên đối tượng C# thành các câu lệnh SQL tương ứng và ngược lại. Lợi ích to lớn của ORM bao gồm:

- Loại bỏ hàng nghìn dòng mã boilerplate ADO.NET thô lặp đi lặp lại (SqlConnection, SqlCommand, SqlDataReader).
- Bảo vệ hệ thống khỏi lỗ hổng bảo mật nghiêm trọng **SQL Injection** nhờ cơ chế tham số hóa câu truy vấn tự động (Parameterization).
- Tận dụng sức mạnh định kiểu tĩnh (Type-Safety) lúc biên dịch thông qua cú pháp LINQ (Language Integrated Query), giúp phát hiện lỗi sai tên cột hoặc sai kiểu dữ liệu ngay khi build thay vì chờ đến runtime.

5.3.2 So sánh Entity Framework Core với Dapper và ADO.NET thuần

Để có cái nhìn khách quan và toàn diện phục vụ việc ra quyết định kiến trúc trong **PropNest**, nhóm đã thực hiện phân tích so sánh ba giải pháp truy cập dữ liệu tiêu biểu trong hệ sinh thái .NET:

Bảng 5.2 Bảng so sánh giữa ADO.NET, Dapper và Entity Framework Core 8

Tiêu chí kỹ thuật	ADO.NET thuần	Dapper (Micro-ORM)	Entity Framework Core 8 (Full ORM)

Bản chất công nghệ	Thư viện truy cập CSDL mức thấp cơ bản của .NET.	Micro-ORM siêu nhẹ, ánh xạ trực tiếp kết quả truy vấn sang POCO.	Full-featured ORM hiện đại, giàu tính năng hàng đầu của Microsoft.
Độ trừu tượng hóa	Rất thấp (Viết SQL thuần, đọc từng cột qua index/name).	Trung bình (Viết SQL thuần, tự động map cột sang thuộc tính).	Rất cao (Lập trình hoàn toàn trên đối tượng qua LINQ, tự động sinh SQL).
Cơ chế Change Tracking	Hoàn toàn không có.	Không có (Phải tự viết thủ công câu lệnh UPDATE).	Có sẵn Change Tracker mạnh mẽ: Tự phát hiện thuộc tính bị sửa và sinh câu UPDATE tối ưu.
Quản lý Migration	Thủ công (Tự quản lý file .sql rời rạc).	Thủ công (Cần công cụ thứ ba như DbUp, Flyway).	Bản xứ toàn diện: Quản lý phiên bản CSDL qua C# Code Migrations.
Hỗ trợ quan hệ phức tạp	Rất phức tạp (Tự ghép thủ công nhiều DataReader).	Khá phức tạp (Dùng kỹ thuật Multi-mapping với splitOn).	Rất mạnh mẽ: Hỗ trợ Include, ThenInclude, Projection, Tự động quản lý khóa ngoại.
Chi phí bộ nhớ & CPU	Cực thấp (Gần như zero overhead).	Cực thấp (Tốc độ xấp xỉ ADO.NET thuần).	Trung bình - Thấp (Đã được tối ưu vượt bậc trong EF Core 8, hỗ trợ AsNoTracking).
Nguy cơ lỗi thời gian chạy	Rất cao (Lỗi chính tả trong chuỗi SQL chỉ lộ ra khi chạy).	Cao (Viết sai tên cột trong câu SQL string).	Rất thấp (Kiểm tra chặt chẽ lúc biên dịch qua biểu thức biểu cây Expression Tree của LINQ).

Chiến lược áp dụng trong dự án PropNest:

Nhóm sử dụng Entity Framework Core 8 làm công cụ chính:

- **Đối với các thao tác Ghi/Sửa dữ liệu (Tạo tin, Sửa tin, Mua gói):** EF Core phát huy tối đa sức mạnh nhờ tính năng theo dõi thay đổi (Change Tracking) và kiểm soát xung đột dữ liệu.
- **Đối với các thao tác Đọc dữ liệu (Xem danh sách tin công khai):** Nhóm áp dụng kỹ thuật Query Projection (.Select(...)) kết hợp với .AsNoTracking(), giúp EF Core chỉ chọn đúng các cột cần hiển thị và không tốn bộ nhớ lưu vết, đem lại tốc độ nhanh không thua kém Dapper.

5.4 Cơ chế hoạt động của Entity Framework Core

5.4.1 Mapping Entity sang Table bằng Fluent API

Để ánh xạ các lớp thực thể trong Domain sang lược đồ CSDL vật lý, EF Core cung cấp hai cơ chế:

1. **Data Annotations**: Đặt các thuộc tính như [Table], [Key], [MaxLength] ngay trên đầu các thuộc tính trong Class.
2. **Fluent API**: Viết mã cấu hình riêng biệt thông qua đối tượng ModelBuilder.

Trong dự án **PropNest**, nhóm tuân thủ kiến trúc **Clean Architecture**:

- Các lớp thực thể tại tầng Domain hoàn toàn "sạch sẽ", chỉ chứa các nghiệp vụ thuần túy, không bị phụ thuộc (Coupled) vào bất kỳ thư viện ORM nào và không chứa các Annotation của EF Core.
- Toàn bộ quy tắc ánh xạ được đóng gói độc lập tại tầng Infrastructure bằng **Fluent API**.
- Mỗi thực thể có một lớp cấu hình riêng biệt triển khai interface IEntityTypeConfiguration<T>. Tại phương thức OnModelCreating, DbContext tự động quét và áp dụng tất cả các cấu hình thông qua một dòng lệnh duy nhất:

```
namespace PropNest.Infrastructure.Persistence.Configurations;

public sealed class ListingConfiguration : IEntityTypeConfiguration<Listing>

{

  public void Configure(EntityTypeBuilder<Listing> builder)

  {

    // 1. Ánh xạ tên bảng và Khóa chính BIGINT IDENTITY

    builder.ToTable("Listings");

    builder.HasKey(x => x.ListingId);

    builder.Property(x => x.ListingId).UseIdentityColumn();

    // 2. Ràng buộc độ dài chuỗi và thuộc tính bắt buộc (Not Null)

    builder.Property(x => x.Title).HasMaxLength(500).IsRequired();

    builder.Property(x => x.Description).IsRequired(false);

    builder.Property(x => x.PropertyType).HasMaxLength(100).IsRequired();

    builder.Property(x => x.ListingType).HasMaxLength(50).IsRequired();

    // 3. Định nghĩa độ chính xác cho tiền tệ và diện tích

    builder.Property(x => x.Price).HasPrecision(18, 2);

    builder.Property(x => x.Area).HasPrecision(12, 2);

    // 4. Ánh xạ Enum thành chuỗi ký tự NVARCHAR giúp dữ liệu CSDL trực quan khi debug

    builder.Property(x => x.Status).HasConversion<string>().HasMaxLength(50);

    builder.Property(x => x.ModerationDecision).HasConversion<string>().HasMaxLength(50);

    builder.Property(x => x.PackageCode).HasMaxLength(50).IsRequired();

    // 5. Cấu hình Concurrency Token với RowVersion phục vụ Optimistic Concurrency

    builder.Property(x => x.RowVersion).IsRowVersion();

    // 6. Cấu hình quan hệ Khóa ngoại với bảng Users

    builder.HasOne<ApplicationUser>()
```

```
            .WithMany()
            .HasForeignKey(x => x.OwnerUserId)
            .OnDelete(DeleteBehavior.Restrict);
        // 7. Thiết lập các Chỉ mục (Indexes) phục vụ tối ưu hóa truy vấn lọc và tìm kiếm
        builder.HasIndex(x => new { x.OwnerUserId, x.Status });
        builder.HasIndex(x => new { x.Status, x.EndDate });
        builder.HasIndex(x => new { x.City, x.District, x.PropertyType });
    }
}
```

5.4.2 Change Tracker và vòng đời của thực thể (Entity States)

Điểm đặc biệt nhất của EF Core là tính năng **Change Tracker (Bộ theo dõi thay đổi)**. Khi một thực thể được truy vấn từ CSDL thông qua một DbContext có bật tính năng tracking, EF Core tạo ra một bản sao lưu trạng thái ban đầu của đối tượng trong bộ nhớ (gọi là Original Values Snapshot).

Trong suốt chu kỳ sống bên trong một DbContext Scope, một thực thể sẽ trải qua các trạng thái:

1. **Detached**: Đối tượng vừa được tạo bằng từ khóa new hoặc được truy vấn với .AsNoTracking(). Đối tượng chưa được gắn vào DbContext và không bị Change Tracker theo dõi.
2. **Added**: Đối tượng được đưa vào DbContext thông qua phương thức Add() hoặc AddAsync(). Khi gọi SaveChanges(), EF Core sẽ sinh câu lệnh INSERT.
3. **Unchanged**: Đối tượng vừa được tải từ CSDL lên và chưa có bất kỳ thuộc tính nào bị thay đổi.
4. **Modified**: Một hoặc nhiều thuộc tính của đối tượng đã bị thay đổi giá trị so với bản chụp Snapshot ban đầu. Khi gọi SaveChanges(), EF Core tự động phát hiện

các thuộc tính này và chỉ sinh câu lệnh UPDATE cập nhật đúng những cột bị thay đổi (Selective Update).

5. **Deleted**: Thực thể được đánh dấu xóa qua phương thức Remove(). Khi commit, EF Core sẽ phát sinh câu lệnh DELETE.

Tối ưu hóa truy vấn với .AsNoTracking():

Khi thực hiện các truy vấn đọc dữ liệu để hiển thị (như API xem danh sách tin bất động sản công khai GetPublicListingsAsync), ứng dụng hoàn toàn không có nhu cầu cập nhật ngược lại đối tượng. Việc sử dụng .AsNoTracking() chỉ thị cho Change Tracker bỏ qua hoàn toàn việc tạo Snapshot và lưu vết đối tượng trong Identity Map. Kỹ thuật này giúp giải phóng lượng lớn bộ nhớ Heap, giảm thiểu chu kỳ dọn rác của Garbage Collector và gia tăng tốc độ phản hồi API đáng kể.

5.4.3 Repository Pattern và Unit of Work trong EF Core

Trong kiến trúc phần mềm, sự kết hợp giữa **Repository Pattern** và **Unit of Work** đóng vai trò thiết lập ranh giới giao dịch rõ ràng:

- **Repository Pattern**: Đóng vai trò như một người quản kho. Thay vì để tầng Controller hoặc Service tự viết các câu lệnh SQL/LINQ phức tạp, Repository cung cấp các hàm nghiệp vụ rõ ràng như GetByIdAsync(), AddAsync().
- **Unit of Work Pattern**: Đảm bảo tất cả các thay đổi của nhiều bảng khác nhau đều được gom chung vào một giao dịch và xác nhận lưu xuống cơ sở dữ liệu cùng một lúc qua lệnh SaveChangesAsync().

Thực tế, bản thân DbContext trong EF Core đã làm sẵn nhiệm vụ của Unit of Work, và mỗi DbSet<T> đã là một Repository. Tuy nhiên, dự án PropNest vẫn tạo thêm một lớp giao diện (Interface) mỏng như IListingRepository, IWalletRepository và IUnitOfWork vì hai mục đích thực tế:

1. Giúp code tầng nghiệp vụ (Application) không bị phụ thuộc cứng vào thư viện EF Core.
2. Cực kỳ dễ dàng viết các bài kiểm thử tự động (Unit Test) bằng cách giả lập (Mock) dữ liệu mà không cần phải kết nối vào SQL Server thật.

5.5 Ứng dụng EF Core trong dự án PropNest

5.5.1 Tích hợp ASP.NET Core Identity vào DbContext (IdentityDbContext)

ASP.NET Core Identity là bộ khung quản lý người dùng, mật khẩu và quyền hạn rất mạnh của Microsoft. Tuy nhiên, nếu dùng mặc định, Identity sẽ tự tạo các bảng mang tên AspNetUsers, AspNetRoles... với cấu trúc định sẵn, gây khó khăn cho việc quản lý các bảng nghiệp vụ khác.

Tuy nhiên, thách thức kiến trúc lớn nhất là: Làm thế nào để tích hợp ASP.NET Core Identity vào mô hình Clean Architecture mà không phá vỡ tính thuần khiết của tầng Domain? Nhóm áp dụng giải pháp Pragmatic Clean Architecture (Clean Architecture thực dụng):

1. **Tại tầng Domain**: Khai báo thực thể ApplicationUser kế thừa từ IdentityUser<long> thông qua việc chỉ tham chiếu duy nhất gói thư viện siêu nhẹ Microsoft.Extensions.Identity.Stores. Gói này chỉ chứa các định nghĩa POCO trừu tượng, không chứa bất kỳ mã phụ thuộc nào vào ASP.NET Core MVC hay Entity Framework Core.
2. **Tại tầng Infrastructure**: Lớp PropNestDbContext kế thừa từ lớp cha chuyên dụng IdentityDbContext<ApplicationUser, IdentityRole<long>, long>.
3. **Chuẩn hóa lược đồ bảng cơ sở dữ liệu**: Lớp cấu hình ApplicationUserConfiguration đổi tên bảng mặc định của Identity từ AspNetUsers thành Users, đồng thời chỉ định trường khóa chính là UserId kiểu BIGINT IDENTITY:

```
public sealed class ApplicationUserConfiguration : IEntityTypeConfiguration<ApplicationUser>
{
        public void Configure(EntityTypeBuilder<ApplicationUser> builder)
        {
    builder.ToTable("Users");
    builder.Property(x => x.Id).HasColumnName("UserId");
    builder.Property(x => x.FullName).HasMaxLength(255).IsRequired();
    builder.Property(x => x.Role).HasMaxLength(50).IsRequired();
    builder.Property(x => x.Status).HasMaxLength(50).IsRequired();
        }
}
```

Nhờ cách thiết kế này, hệ thống vừa sử dụng được toàn bộ các tính năng bảo mật cao cấp có sẵn của Microsoft (mã hóa mật khẩu chuẩn PBKDF2, tự khóa tài khoản khi nhập sai nhiều lần), vừa giúp các bảng khác như Listings hay Wallets liên kết khóa ngoại tới Users một cách tự nhiên và mạch lạc.

5.5.2 Cấu hình kiểu dữ liệu tiền tệ DECIMAL(18,2) và RowVersion

Hai bài toán kỹ thuật nhạy cảm nhất trong các hệ thống giao dịch là: **Độ chính xác số học tài chính** và **Kiểm soát xung đột ghi đè đồng thời**. Cả hai bài toán này đều được giải quyết triệt để thông qua cấu hình EF Core trong PropNest:

1. **Xử lý số học tiền tệ với DECIMAL(18,2):**

Trong các ngôn ngữ lập trình, kiểu số thực dấu phẩy động float hoặc double tuân theo chuẩn nhị phân IEEE 754. Do không thể biểu diễn chính xác tuyệt đối các phân số thập phân trong hệ nhị phân (ví dụ: $0.1 + 0.2 = 0.3000000000004$), việc sử dụng float/double cho các phép tính số dư ví sẽ gây ra lỗi sai lệch tiền tệ tích lũy vô cùng nguy hiểm.

- **Trong mã nguồn C#**, nhóm bắt buộc sử dụng kiểu dữ liệu số thực dấu phẩy tĩnh decimal (128-bit).
- **Trong EF Core Fluent API**, toàn bộ các trường liên quan đến giá cả, số dư ví và chi phí gói đều được ánh xạ tường minh sang kiểu SQL Server DECIMAL(18,2) (18 chữ số có nghĩa với 2 chữ số phần thập phân) thông qua cấu hình:

```
builder.Property(x => x.MainBalance).HasPrecision(18, 2);

builder.Property(x => x.PromoBalance).HasPrecision(18, 2);
```

2. Kiểm soát xung đột đồng thời với RowVersion (Optimistic Concurrency Control):

Khi hai Seller hoặc một Seller và một Moderator cùng thực hiện chỉnh sửa một tin đăng tại cùng một thời điểm, hiện tượng **Ghi đè mất mát dữ liệu (Lost Update)** chắc chắn sẽ xảy ra nếu hệ thống không có cơ chế kiểm soát. Thay vì sử dụng giải pháp Khóa bi quan (Pessimistic Locking) gây treo luồng dữ liệu và bế tắc hệ thống (Deadlock), PropNest áp dụng thuật toán **Kiểm soát xung đột lạc quan (Optimistic Concurrency Control)**:

- Thực thể Listing và Wallet được bổ sung thuộc tính Concurrency Token:

```
public byte[] RowVersion { get; private set; } = Array.Empty<byte>();
```

- Trong cấu hình Fluent API, thuộc tính này được gắn chỉ thị .IsRowVersion(), chỉ định SQL Server tạo cột kiểu rowversion tự động thay đổi giá trị nhị phân mỗi khi dòng dữ liệu có thao tác UPDATE.
- Khi gọi UpdateAsync, tầng Repository thiết lập giá trị ban đầu vào Change Tracker:

```
dbContext.Entry(listing).Property(x => x.RowVersion).OriginalValue=rowVersion;
```

- EF Core sẽ tự động biên dịch câu lệnh SQL cập nhật có mệnh đề kiểm tra điều kiện kép:

```
UPDATE [Listings]
SET [Title] = @p0, [Price] = @p1, [UpdatedAt] = @p2
WHERE [ListingId] = @p3 AND [RowVersion] = @originalRowVersion;
```

- Nếu có một giao dịch khác đã cập nhật trước đó, RowVersion trong CSDL đã biến đổi khiến số dòng bị tác động trả về bằng 0 (Rows Affected = 0). EF Core lập tức chặn đứng thao tác và ném ra ngoại lệ **DbUpdateConcurrencyException**. Tầng API sẽ bắt ngoại lệ này và phản hồi mã **409 Conflict** theo chuẩn RFC 7807, yêu cầu người dùng tải lại dữ liệu mới nhất trước khi chỉnh sửa.

5.6 Thiết kế Entity và Schema chi tiết cho PropNest

5.6.1 Nhóm bảng Identity & Wallet (Users, Roles, Wallets, WalletTransactions)

- **Users**: Lưu thông tin đăng nhập, họ tên, số điện thoại, mật khẩu đã mã hóa.
- **AspNetRoles & AspNetUserRoles**: Quản lý 3 vai trò chính trong hệ thống: Seller (Người đăng tin), Moderator (Người kiểm duyệt) và Admin (Quản trị viên).
- **Wallets**: Lưu số dư ví của mỗi tài khoản. Mỗi User chỉ có duy nhất 1 ví. Ví gồm 2 phần tiền: MainBalance (tiền nạp) và PromoBalance (tiền khuyến mãi). Cột RowVersion giúp chống gian lận nạp rút tiền đồng thời.
- **WalletTransactions**: Sổ cái ghi lại toàn bộ lịch sử biến động tiền. Bất kể nạp tiền, trừ tiền mua gói, hay được hoàn tiền khi gặp sự cố, hệ thống đều ghi lại một dòng tại đây kèm số dư trước và sau giao dịch.
- **PackageDefinitions**: Danh mục bảng giá các gói tin (Standard, VIP, Boost) kèm theo số ngày hiển thị và đơn giá tương ứng.

5.6.2 Nhóm bảng Listing & Moderation (Listings, Images, Histories, Reviews)

- **Listings**: Bảng trung tâm lưu trữ thông tin bất động sản: tiêu đề, mô tả, loại nhà đất, hình thức (bán/cho thuê), giá tiền, diện tích, địa chỉ đầy đủ và trạng thái tin (Draft, PendingModeration, Published, Hidden, Expired, Rejected).
- **ListingHistories**: Lưu lại nhật ký thay đổi của tin đăng:
    - Khi tạo tin: Lưu toàn bộ thông tin ban đầu vào SnapshotJson.
    - Khi sửa nội dung: Chỉ lưu danh sách các trường bị thay đổi vào DeltasJson (Ví dụ: [{"field":"Price","oldValue":5000000000,"newValue":5200000000}]). Cách làm này giúp tiết kiệm dung lượng Database hơn rất nhiều so với việc mỗi lần sửa lại lưu cả bài đăng.
    - **ListingModerationReviews**: Lưu lịch sử duyệt tin của kiểm duyệt viên hoặc của AI (AME). Ghi rõ lý do tại sao duyệt hoặc từ chối tin để phục vụ kiểm tra đối soát.

5.6.3 Nhóm bảng Workflow & Outbox (IdempotencyRequests, SagaStates, OutboxMessages)

- **IdempotencyRequests**: Chống bấm nút thanh toán nhiều lần. Hệ thống lưu lại một mã băm (RequestHash) của gói tin bạn muốn mua. Nếu mạng lag làm trình duyệt gửi yêu cầu 2 lần cùng một mã khóa (Idempotency-Key), hệ thống sẽ nhận diện được và trả về ngay kết quả cũ chứ không trừ tiền lần thứ hai.
- **SagaListingStates & WorkflowSteps**: Lưu lại tiến trình xử lý tự động khi mua gói VIP (đang trừ tiền, đang nâng cấp gói tin, hay đang hoàn tiền nếu gặp lỗi).

- **OutboxMessages**: Lưu các sự kiện cần gửi sang RabbitMQ. Khi lưu tin đăng vào Database, sự kiện cũng được lưu luôn vào bảng này trong cùng 1 lượt. Sau đó, một tiến trình chạy ngầm (Worker) sẽ đọc bảng này và gửi tin nhắn đi, đảm bảo không bao giờ bị mất tin nhắn thông báo.

5.7 Migration Database và Quản lý phiên bản Schema

5.7.1 Nguyên lý Versioning Database Schema bằng mã nguồn

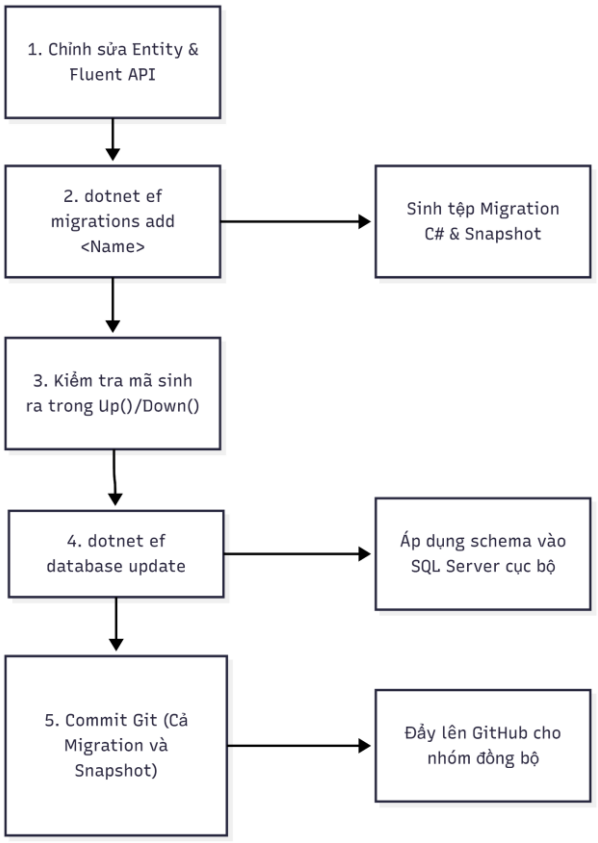
Trong quy trình công nghệ phần mềm truyền thống, việc quản lý cơ sở dữ liệu thường được thực hiện qua các đoạn script SQL thủ công viết bằng tay. Phương pháp này bộc lộ những nhược điểm chí mạng: không theo dõi được lịch sử biến đổi của schema theo thời gian, khó đồng bộ cấu trúc bảng giữa máy của các thành viên trong nhóm, và dễ gây lỗi nghiêm trọng khi triển khai lên môi trường thử nghiệm hoặc sản xuất.

Entity Framework Core cung cấp giải pháp **Database-as-Code** thông qua cơ chế **EF Core Migrations**. Toàn bộ cấu trúc cơ sở dữ liệu được định nghĩa bằng mã nguồn C#. Khi có sự thay đổi trên Entity, công cụ dotnet-ef phân tích sự sai khác giữa cấu trúc lớp C# hiện tại và bản chụp kiến trúc gần nhất (PropNestDbContextModelSnapshot.cs) để tự động sinh ra một tệp mã nguồn Migration gồm hai phương thức đối ngẫu:

- Up(MigrationBuilder migrationBuilder): Chứa các lệnh tạo bảng, thêm cột, tạo khóa ngoại và chỉ mục để nâng cấp cơ sở dữ liệu lên phiên bản mới.
- Down(MigrationBuilder migrationBuilder): Chứa các lệnh đảo ngược (Drop table, Drop column) để hạ cấp (Rollback) cơ sở dữ liệu về trạng thái liền trước nếu phát sinh sự cố. Khi Migration được áp dụng vào máy chủ SQL Server, EF Core tự động khởi tạo và duy trì bảng hệ thống đặc biệt __EFMigrationsHistory. Bảng này ghi nhận danh sách các mã migration đã được áp dụng (MigrationId) cùng thời điểm thực thi. Mỗi khi ứng dụng khởi động, EF Core đối chiếu các tệp migration trong mã nguồn với danh sách trong bảng lịch sử này để chỉ thực thi các migration còn thiếu mà không gây lỗi trùng lặp.

5.7.2 Quy trình Migration Workflow trong nhóm phát triển

Nhằm bảo đảm tính kỷ luật và vận hành trơn tru trong nhóm phát triển PropNest gồm 3 thành viên, quy trình làm việc với Database Migration được chuẩn hóa thành 5 bước nghiêm ngặt:



Hình 5.1 Quy trình Migration Workflow trong nhóm phát triển

Tự động cập nhật Database khi khởi động ứng dụng:

Để các thành viên khi kéo code mới về không cần phải nhớ gõ lệnh cập nhật database bằng tay, trong tệp **Program.cs**, nhóm cấu hình để mỗi khi khởi động ứng dụng ở môi trường Development, chương trình sẽ tự động kiểm tra và cập nhật các migration còn thiếu vào SQL Server:

```
if (app.Environment.IsDevelopment())
{
  using var scope = app.Services.CreateScope();
  var dbContext = scope.ServiceProvider.GetRequiredService<PropNestDbContext>();
  var logger = scope.ServiceProvider.GetRequiredService<ILogger<Program>>();
  try
```

```
    {
        logger.LogInformation("Đang kiểm tra và cập nhật Database...");
        await dbContext.Database.MigrateAsync();
        logger.LogInformation("Cơ sở dữ liệu đã đồng bộ thành công!");
    }
    catch (Exception ex)
    {
        logger.LogError(ex, "Lỗi xảy ra khi tự động cập nhật database.");
        throw;
    }
}
```

KẾT LUẬN CHƯƠNG 5

Chương 5 đã làm rõ các nguyên lý cốt lõi trong việc thiết kế và quản trị cơ sở dữ liệu với Entity Framework Core 8:

1. **Hiểu rõ nền tảng lưu trữ**: Nắm vững nguyên lý ACID và lý do tại sao hệ thống tài chính - bất động sản bắt buộc phải dùng cơ sở dữ liệu quan hệ (SQL Server) để dữ liệu luôn chính xác và nhất quán.
2. **Khai thác sức mạnh của EF Core 8**: Tận dụng bộ Change Tracker để tự động hóa việc cập nhật dữ liệu, dùng AsNoTracking để tăng tốc độ truy vấn đọc, và tách biệt hoàn toàn cấu hình Database khỏi Entity bằng Fluent API theo chuẩn Clean Architecture.
3. **Giải quyết triệt để các bài toán khó**:
    - Dùng DECIMAL(18,2) để tiền tệ không bao giờ bị sai lệch số học.
    - Dùng RowVersion để ngăn chặn hoàn toàn việc người này ghi đè mất dữ liệu của người khác khi cùng sửa một tin đăng.
    - Tích hợp mượt mà khung bảo mật ASP.NET Core Identity vào cấu trúc hệ thống.
4. **Quản lý dữ liệu chuyên nghiệp bằng Migrations**: Biến cấu trúc cơ sở dữ liệu thành mã nguồn (Database-as-Code), giúp việc phối hợp nhóm mượt mà và tự động hóa cập nhật CSDL ngay khi khởi chạy ứng dụng.

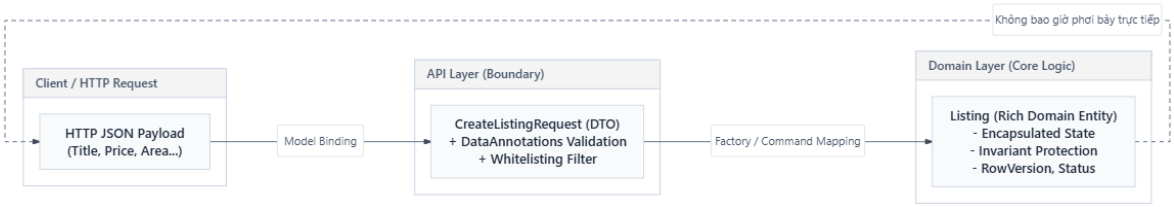
Hệ thống cơ sở dữ liệu vững chắc được thiết lập trong chương này chính là bệ phóng an toàn để nhóm tiếp tục xây dựng các tính năng nâng cao, phân quyền bảo mật và xử lý các quy trình mua bán phức tạp ở các chương tiếp theo.

CHƯƠNG 7 XÂY DỰNG RESTFUL API NÂNG CAO VÀ VALIDATION TRONG ASP.NET CORE

7.1 DTO và Validation với FluentValidation

7.1.1 Tại sao cần tách biệt DTO với Entity trong Domain-Driven Design?

Trong lý thuyết kiến trúc phần mềm phân tầng (*Multitier / Layered Architecture*) và phương pháp luận Thiết kế Hướng miền (*Domain-Driven Design – DDD*), sự tách biệt giữa tầng trình diễn (*Presentation / API Layer*) và tầng nghiệp vụ cốt lõi (*Domain Layer*) là nguyên lý bất khả xâm phạm. Hai đối tượng trung tâm đại diện cho hai biên giới này là **Data Transfer Object (DTO)** và **Domain Entity**.



Hình 7.1 Quy trình tiếp nhận và chuyển đổi dữ liệu từ API Layer sang Domain Layer

Về mặt khoa học máy tính và an toàn thông tin, việc tách biệt DTO và Entity được luận chứng qua 4 luận điểm kỹ thuật then chốt:

1. **Triệt tiêu lỗ hổng gán thuộc tính hàng loạt (Mass Assignment / Over-Posting - CWE-915)**

Lỗ hổng Mass Assignment (được ghi nhận trong danh mục hiểm họa OWASP API Security Top 10) phát sinh do cơ chế tự động hóa của bộ liên kết dữ liệu (*Model Binder*) trong các Web Framework hiện đại. Model Binder sử dụng kỹ thuật phản chiếu (*Reflection*) để tự động duyệt qua toàn bộ các cặp khóa-giá trị (*key-value*) trong gói tin JSON gửi lên và gán vào các thuộc tính có tên tương ứng của đối tượng đích.

Nếu tầng API sử dụng trực tiếp Domain Entity làm tham số đầu vào:

```
// LỖI THIẾT KẾ: Sử dụng trực tiếp Domain Entity trong Action Parameter
[HttpPost]
public async Task<IActionResult> CreateListing([FromBody] Listing listing)
```

Đối tượng tấn công có thể chèn các trường dữ liệu đặc quyền vào gói tin HTTP POST:

```
{
 "title": "Căn hộ cao cấp ven sông",
 "price": 5000000000,
 "status": "Published",        // Tự ý công khai tin mà không qua kiểm duyệt
 "packageCode": "VIP",         // Tự ý chiếm đoạt gói hiển thị cao cấp
 "ownerUserId": 999            // Giả mạo quyền sở hữu của người dùng khác
}
```

Khi đó, Model Binder sẽ ghi đè các trường Status, PackageCode, OwnerUserId vào đối tượng listing. DTO đóng vai trò là một **Bộ lọc danh sách trắng (Whitelisting Filter)**. Lớp CreateListingRequest chỉ công khai duy nhất các trường người dùng được phép nhập (Title, Description, PropertyType, ListingType, Price, Area, City, District, Address), loại bỏ hoàn toàn khả năng can thiệp vào các trường trạng thái nội bộ.

2. **Định luật bảo toàn tính đóng gói (Encapsulation) và mô hình Rich Domain**

Theo Martin Fowler (*Patterns of Enterprise Application Architecture*), một mô hình miền chuẩn mực không được phép thoái hóa thành *Anemic Domain Model* (Mô hình miền thiếu máu – chỉ bao gồm các getter/setter thụ động).

Thực thể Listing trong tầng Domain được thiết kế theo trường phái **Rich Domain Model**:

- Mọi thuộc tính biến đổi trạng thái đều được giới hạn truy cập private set.
- Việc chuyển dịch trạng thái bắt buộc phải thông qua các phương thức nghiệp vụ (*Domain Methods*) có kiểm soát tiền điều kiện và bất biến miền (*Invariants*): SubmitForModeration(), Approve(), Reject(), Hide(), UpdateContent().

Ngược lại, các thư viện tuần tự hóa JSON (System.Text.Json, Newtonsoft.Json) yêu cầu đối tượng nhận dữ liệu phải có tính khả biến công khai (public setter hoặc init-only setter). Nếu sử dụng Entity làm đầu vào API, lập trình viên buộc phải mở public setter cho toàn bộ thuộc tính của Entity, phá vỡ hoàn toàn tính đóng gói hướng đối tượng.

3. **Sự bất tương thích cấu trúc giữa mạng truyền thông và cơ sở dữ liệu**

Cấu trúc của Entity được tối ưu hóa cho đại số quan hệ và hệ thống lưu trữ bền vững (chứa khóa ngoại OwnerUserId, trường phiên bản đồng thời byte[] RowVersion, đồ thị liên kết ICollection<ListingHistory>). Ngược lại, DTO được tối ưu hóa cho hợp đồng mạng (*Network Wire Format*), có nhiệm vụ gom cụm (*Aggregation*) hoặc giản lược dữ liệu nhằm tiết kiệm băng thông mạng và che giấu cấu trúc vật lý của cơ sở dữ liệu.

7.1.2 Thư viện DataAnnotations và cơ chế thẩm định mô hình (Model Validation) trong ASP.NET Core

Trong ASP.NET Core, quá trình kiểm tra tính hợp lệ của dữ liệu đầu vào là sự cụ thể hóa của nguyên lý **Thiết kế theo hợp đồng (Design by Contract - DbC)** do Bertrand Meyer đề xuất năm 1986. Quá trình này được phân chia thành ba cấp độ kiểm chứng:

1. **Syntax / Format Validation (Thẩm định cú pháp):** Kiểm tra cấu trúc hình thức của dữ liệu (chuỗi không rỗng, giá trị số dương, định dạng chuỗi, giới hạn độ dài).
2. **Semantic Validation (Thẩm định ngữ nghĩa):** Kiểm tra tính hợp lý giữa các trường tương quan (ví dụ: ngày kết thúc phải sau ngày bắt đầu, giá mua gói phải tương ứng với mã gói).
3. **Domain Invariant Validation (Thẩm định bất biến miền):** Kiểm tra tính toàn vẹn trạng thái đối chiếu với cơ sở dữ liệu (ví dụ: người bán có đủ số dư ví để mua gói tin hay không, tin đăng có đang ở trạng thái cho phép cập nhật hay không).

Phân tích cơ chế hoạt động nội tại của System.ComponentModel.DataAnnotations

Hệ thống **System.ComponentModel.DataAnnotations** trong ASP.NET Core dựa trên cơ chế siêu dữ liệu khai báo (Declarative Metadata):

- Khi một HTTP Request được gửi đến Kestrel Server, trình liên kết mô hình (Model Binder) ánh xạ luồng nhị phân JSON vào đối tượng DTO.
- Ngay sau khi liên kết hoàn tất, bộ thẩm định **IObjectModelValidator** (mặc định là **DefaultObjectValidator**) duyệt qua cây thuộc tính của đối tượng bằng kỹ thuật **Runtime Reflection** thông qua **Type.GetProperties()** và trích xuất các Attribute kế thừa từ **ValidationAttribute ([Required], [StringLength], [Range]).**
- Với mỗi thuộc tính vi phạm, validator gọi hàm **IsValid(value, validationContext)** và bổ sung thông tin lỗi vào **ModelStateDictionary**.

Bảng 7.1 So sánh chuyên sâu giữa DataAnnotations và FluentValidation

Tiêu chí phân tích	DataAnnotations (Truyền thống)	FluentValidation (Hiện đại)
Mô hình lập trình	Khai báo qua siêu dữ liệu Attribute (*Declarative Metadata*).	Lập trình hướng đối tượng thuần túy qua Fluent Interface.
Nguyên lý SOLID	Khai báo trực tiếp trên DTO; dễ đọc, trực quan cho các hợp đồng dữ liệu đơn giản.	Tách biệt hoàn toàn DTO và Validator thành hai lớp độc

		lập; tuân thủ triệt để SRP (*Single Responsibility*).
Cơ chế thực thi	Runtime Reflection (PropertyInfo.GetCustomAttributes), chi phí CPU cao.	Phân tích *Expression Trees*, biên dịch thành các con trỏ hàm ủy thác (*Compiled Delegates*).
Thẩm định bất đồng bộ	Không hỗ trợ (buộc phải thực thi đồng bộ, không inject được DbContext vào Attribute).	Hỗ trợ phương thức MustAsync() cho các tác vụ I/O không phong tỏa thread.
Mức độ phức tạp bảo trì	Cực kỳ tinh gọn, không phát sinh thêm các lớp Validator trung gian cho các kiểm tra cú pháp chuẩn.	Phát sinh thêm nhiều tệp Validator trong cấu trúc thư mục dự án.

Hai phương án thay thế cho DataAnnotations

1. **FluentValidation:** Thư viện mã nguồn mở cho phép tách rời hợp đồng DTO và quy tắc kiểm tra thành các lớp kế thừa **AbstractValidator<T>**, hỗ trợ các biểu thức lambda phức tạp và thẩm định bất đồng bộ (*Asynchronous Validation*).
2. **Endpoint Filters kết hợp MiniValidation (Minimal APIs):** Trong mô hình Minimal APIs từ .NET 7/8, các bộ lọc **EndpointFilter** kết hợp thư viện **MiniValidation** cung cấp khả năng thẩm định trực tiếp trên tham số của Route Handler mà không cần cơ chế Controller truyền thống.

Hai trường hợp không nên sử dụng DataAnnotations

1. **Quy tắc thẩm định phụ thuộc truy vấn cơ sở dữ liệu bất đồng bộ:** Do các Attribute của DataAnnotations được thực thi đồng bộ trong chu trình Reflection, việc cố tình gọi các phương thức I/O bất đồng bộ (như kiểm tra email trùng lặp trong DB) thông qua **.Result** hoặc **.Wait()** sẽ gây phong tỏa luồng (*Thread Starvation*), dễ dẫn đến deadlock.

2. **Quy tắc thẩm định phụ thuộc trạng thái động runtime hoặc mối quan hệ đa biến phức tạp:** Khi logic kiểm tra đòi hỏi nạp các dịch vụ hạ tầng phức tạp từ IoC Container hoặc thay đổi linh hoạt theo ngữ cảnh phân quyền người dùng, việc dùng Attribute tĩnh sẽ trở nên bất khả thi và phá vỡ cấu trúc mã nguồn.

7.1.3 Cấu hình tự động kiểm tra mô hình trước khi vào Controller với [ApiController]

Trong kiến trúc ASP.NET Core MVC truyền thống, lập trình viên buộc phải viết mã kiểm tra thủ công tại dòng đầu tiên của mỗi Action:

```
if (!ModelState.IsValid)
{
  return BadRequest(ModelState);
}
```

Nhược điểm của phương pháp này là gây trùng lặp mã nguồn (*Boilerplate Code*) và dễ bị lập trình viên bỏ sót, tạo lỗ hổng cho các dữ liệu sai lệch đi sâu vào tầng nghiệp vụ.

ASP.NET Core giải quyết triệt để vấn đề này thông qua thuộc tính cấp độ Controller: **[ApiController]**.

Cơ chế ngắt mạch tự động (Short-circuiting) của ModelStateInvalidFilter:

Khi Controller được đánh dấu bởi thuộc tính **[ApiController]**, framework tự động đăng ký một bộ lọc ngắt mạch tích hợp sẵn là **ModelStateInvalidFilter**:

1. **Model Binding Phase: System.Text.Json** phân tích chuỗi byte JSON đầu vào và ánh xạ vào DTO.
2. **Validation Invocation: IObjectModelValidator** duyệt qua toàn bộ các DataAnnotations trên DTO.

3. **Short-circuiting:** Trước khi Controller Action được gọi, **ModelStateInvalidFilter** kiểm tra cờ **context.ModelState.IsValid**. Nếu phát hiện bất kỳ trường dữ liệu nào không đạt chuẩn (**IsValid == false**), filter sẽ **ngắt mạch (Short-circuit)** chu trình xử lý của Action, hủy bỏ quyền thực thi của Controller và trả về ngay lập tức phản hồi HTTP 400 Bad Request.
4. **Chuẩn hóa RFC 7807:** Kết quả phản hồi được tự động định dạng thành đối tượng **ValidationProblemDetails** tuân thủ đặc tả quốc tế RFC 7807.

Cấu hình máy chủ Kestrel trong file **Program.cs** kích hoạt đầy đủ tính năng này kết hợp với khả năng xử lý JSON Controller:

```
builder.Services.AddControllers()
   .AddJsonOptions(options =>
   {
      options.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter());
      options.JsonSerializerOptions.Converters.Add(new VietnamDateTimeOffsetJsonConverter());
   });
```

7.1.4 Xây dựng DTO và Validation cho CreateListingRequest, UpdateListingRequest và PurchasePackageRequest

Trong PropNest, các yêu cầu tạo mới tin đăng, chỉnh sửa tin đăng và mua gói dịch vụ được đặc tả bằng các DTO bất biến (*Immutable Records/Classes*) nằm trong namespace **PropNest.Api.Contracts.Listings**. Các DTO này sử dụng thuộc tính **init** để đảm bảo tính bất biến sau khi được khởi tạo bởi Model Binder và gắn kết các siêu dữ liệu thẩm định DataAnnotations.

```
// src/PropNest.Api/Contracts/Listings/ListingRequests.cs
using System.ComponentModel.DataAnnotations;
```

```
namespace PropNest.Api.Contracts.Listings;


public class CreateListingRequest
{
    [Required, StringLength(500)]
    public string Title { get; init; } = string.Empty;


    public string? Description { get; init; }


    [Required, StringLength(100)]
    public string PropertyType { get; init; } = string.Empty;


    [Required, StringLength(50)]
    public string ListingType { get; init; } = string.Empty;


    [Range(0, double.MaxValue)]
    public decimal Price { get; init; }


    [Range(0.01, double.MaxValue)]
    public decimal Area { get; init; }


    [Required, StringLength(100)]
    public string City { get; init; } = string.Empty;


    [Required, StringLength(100)]
    public string District { get; init; } = string.Empty;
```

```
    [StringLength(100)]
    public string? Ward { get; init; }

    [Required, StringLength(500)]
    public string Address { get; init; } = string.Empty;
}

public sealed class UpdateListingRequest : CreateListingRequest
{
    [Required]
    public string RowVersion { get; init; } = string.Empty;
}

public sealed class PurchasePackageRequest
{
    [Required, StringLength(50)]
    public string PackageCode { get; init; } = string.Empty;

    [Range(0, double.MaxValue)]
    public decimal ChargeAmount { get; init; }
}
```

Khi client gửi một HTTP POST request tạo tin đăng mới với tiêu đề bị bỏ trống, diện tích bằng 0 và thiếu trường thành phố:

```
POST /api/v1/listings HTTP/1.1
```

```
Host: localhost:5000
Content-Type: application/json

{
  "title": "",
  "propertyType": "Apartment",
  "listingType": "Sale",
  "price": 3500000000,
  "area": 0,
  "district": "Quận 1",
  "address": "123 Nguyễn Huệ"
}
```

Bộ lọc **ModelStateInvalidFilter** lập tức ngắt mạch và Kestrel trả về mã trạng thái **HTTP 400 Bad Request** với payload chuẩn RFC 7807 **ValidationProblemDetails**:

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Title": [
      "The Title field is required."
    ],
    "Area": [
      "The field Area must be between 0.01 and 1.7976931348623157E+308."
    ],
```

```
  "City": [
    "The City field is required."
  ]
 },
 "traceId": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-00"
}
```


Giải thích kỹ thuật:

1. **Kiểm soát tính kế thừa hợp đồng:** Lớp **UpdateListingRequest** kế thừa toàn bộ các thuộc tính và siêu dữ liệu kiểm tra của **CreateListingRequest**, đồng thời bổ sung thuộc tính bắt buộc **RowVersion**. Thuộc tính này hoạt động như một Concurrency Token phục vụ cơ chế khóa lạc quan (*Optimistic Concurrency Control*).
2. **Ngăn chặn dữ liệu dị thường ở cấp độ biên mạng:** Thuộc tính **[Range(0.01, double.MaxValue)]** trên trường **Area** ngăn chặn hoàn toàn các tin đăng có diện tích bằng 0 hoặc số âm, loại bỏ khả năng phát sinh lỗi chia cho 0 (*DivideByZeroException*) khi tính đơn giá trên mỗi mét vuông ở các module phân tích phía sau.

7.1.5 Thẩm định quy tắc nghiệp vụ phức tạp: Kiểm tra tính hợp lệ của RowVersion và cấu trúc JSON

Một số trường dữ liệu đòi hỏi quy tắc thẩm định vượt ra ngoài khả năng của các Attribute DataAnnotations cơ bản. Trong PropNest, hai trường hợp thẩm định phức tạp điển hình là:

1. Thẩm định định dạng Base64 của chuỗi khóa phiên bản đồng thời **RowVersion** trước khi nạp vào mảng nhị phân 8 bytes của SQL Server / SQLite.

2. Thẩm định tính hợp lệ của chuỗi JSON mảng **ReasonsJson** chứa danh sách các lý do từ chối kiểm duyệt tin đăng.

Trong **ListingsController.cs**, phương thức **Update** chủ động kiểm tra tính hợp lệ của Base64 Concurrency Token:

```
[HttpPut("{listingId:long}")]
[Authorize(Roles = "Seller")]
public async Task<ActionResult<ListingDto>> Update(long listingId, UpdateListingRequest request,
CancellationToken cancellationToken)
{
   try
   {
      byte[] rowVersion;
      rowVersion = Convert.FromBase64String(request.RowVersion);

      var listing = await listingService.GetByIdAsync(listingId, cancellationToken)
         ?? throw new KeyNotFoundException($"Listing with ID {listingId} not found.");
      currentUser.EnsureOwner(listing.OwnerUserId);

      var updatedListing = await listingService.UpdateAsync(new UpdateListingCommand(
         listingId,
         listing.OwnerUserId,
         request.Title,
         request.Description,
         request.PropertyType,
         request.ListingType,
         request.Price,
         request.Area,
```

```
            request.City,
            request.District,
            request.Ward,
            request.Address,
            rowVersion), cancellationToken);
        return Ok(updatedListing);
    }
    catch (FormatException)
    {
        return ValidationProblem(new ValidationProblemDetails(new Dictionary<string, string[]>
        {
            [nameof(request.RowVersion)] = ["RowVersion must be base64 encoded."]
        }));
    }
    catch (Exception ex)
    {
        return this.HandleException(ex);
    }
}
```

Tương tự, trong **ModerationController.cs**, phương thức **Reject** thẩm định định dạng JSON mảng của trường **ReasonsJson** bằng **JsonDocument.Parse**:

```
[HttpPost("listings/{listingId:long}/reject")]
public async Task<ActionResult<ListingDto>> Reject(
    long listingId,
```

```
    [FromBody] RejectPayload? request,
    CancellationToken cancellationToken)
{
    var reasonsList = new List<string>();

    if (request?.Reasons != null && request.Reasons.Count > 0)
    {
        reasonsList.AddRange(request.Reasons.Where(r => !string.IsNullOrWhiteSpace(r)));
    }
    else if (!string.IsNullOrWhiteSpace(request?.Reason))
    {
        reasonsList.Add(request.Reason.Trim());
    }
    else if (!string.IsNullOrWhiteSpace(request?.ReasonsJson))
    {
        try
        {
            using var reasonsDocument = JsonDocument.Parse(request.ReasonsJson);
            if (reasonsDocument.RootElement.ValueKind == JsonValueKind.Array)
            {
                reasonsList.AddRange(reasonsDocument.RootElement.EnumerateArray()
                    .Select(e => e.GetString() ?? e.ToString())
                    .Where(s => !string.IsNullOrWhiteSpace(s)));
            }
            else if (reasonsDocument.RootElement.ValueKind == JsonValueKind.String)
            {
                var val = reasonsDocument.RootElement.GetString();
```

```
            if (!string.IsNullOrWhiteSpace(val)) reasonsList.Add(val);
        }
    }
    catch
    {
        reasonsList.Add(request.ReasonsJson.Trim());
    }
}

if (reasonsList.Count == 0)
{
    return BadRequest(new { message = "Lý do từ chối không được để trống." });
}
// ... Thực hiện cập nhật trạng thái Rejected vào Domain
}
```

Khi client gửi yêu cầu cập nhật tin đăng với chuỗi **RowVersion** không phải là định dạng Base64 hợp lệ (ví dụ **"RowVersion": "Invalid_Not_Base64!!!"**):

```
{
 "title": "One or more validation errors occurred.",
 "status": 400,
 "errors": {
  "RowVersion": [
   "RowVersion must be base64 encoded."
  ]
 }
```

```
}
```

Giải thích kỹ thuật:

Phương pháp bắt lỗi **FormatException** kết hợp với **ValidationProblem** chuẩn hóa thông báo lỗi theo đúng hợp đồng RFC 7807 của ASP.NET Core, đảm bảo client nhận được thông điệp lỗi nhất quán như khi vi phạm các quy tắc DataAnnotations khác. Trong module kiểm duyệt, việc phân tích **JsonDocument** giúp hệ thống hỗ trợ linh hoạt cả ba dạng payload từ các giao diện client khác nhau (danh sách **Reasons**, chuỗi đơn **Reason**, hoặc mảng JSON thô **ReasonsJson**) mà vẫn đảm bảo tính toàn vẹn của dữ liệu lưu trữ.

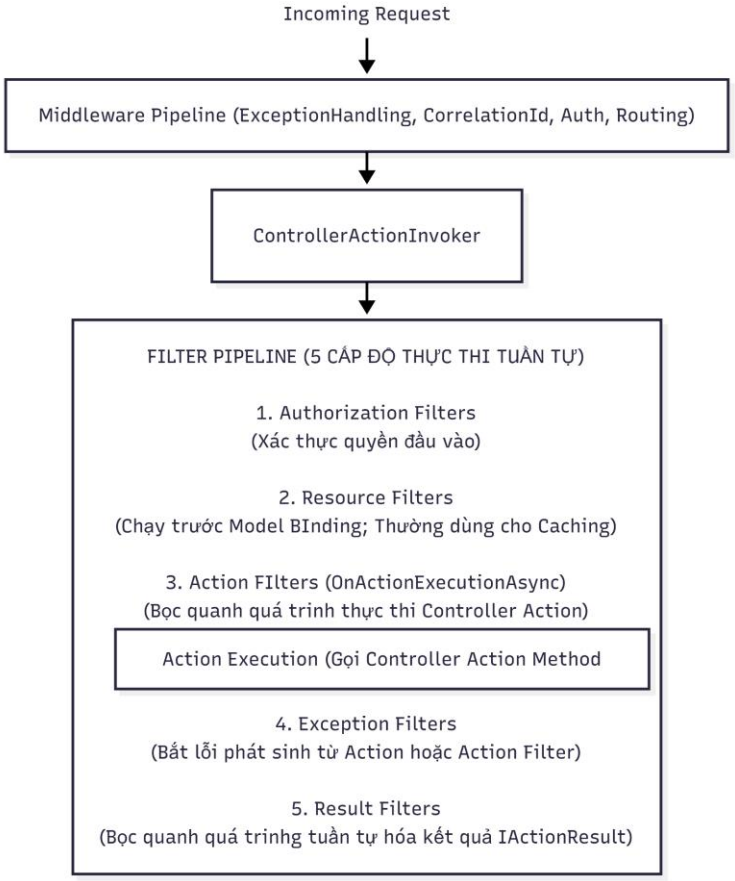
7.2 Action Filters và Endpoint Filters

7.2.1 Thứ tự thực thi và phạm vi của Action Filter trong ASP.NET Core

Trong kiến trúc thực thi của ASP.NET Core, cần phân biệt rạch ròi giữa Middleware Pipeline và Filter Pipeline:

- **Middleware:** Vận hành ở tầng giao tiếp mạng cơ sở, xử lý đối tượng **HttpContext** thô trước khi hệ thống định tuyến (Routing) xác định được Endpoint đích.

- **Filter:** Vận hành bên trong phạm vi của **ActionInvoker** thuộc tầng MVC. Filter chỉ được kích hoạt sau khi **EndpointRoutingMiddleware** đã khớp URL với một Action cụ thể và sau khi Controller tương ứng đã được kích hoạt**.**



Hình 7.2 Cơ chế thực thi Controller Action và các Filter trong ASP.NET Core

Vòng đời Action Filter và cơ chế Búp bê Nga (Russian Doll Model):

Mỗi Action Filter triển khai interface IAsyncActionFilter:

```
public interface IAsyncActionFilter : IFilterMetadata
{
  Task OnActionExecutionAsync(ActionExecutingContext context, ActionExecutionDelegate next);
}
```

Nguyên lý hoạt động tuân theo cấu trúc hàm lồng nhau hai chiều:

- **Giai đoạn tiền xử lý (Pre-action):** Các câu lệnh nằm trước **await next()** được thực thi trước khi Action của Controller chạy. Filter có quyền truy cập vào danh sách tham số (**context.ActionArguments**), sửa đổi dữ liệu hoặc ngắt mạch sớm bằng cách gán **context.Result**.
- **Ủy thác next():** Bàn giao quyền điều khiển cho filter kế tiếp hoặc cho chính Action của Controller.
- **Giai đoạn hậu xử lý (Post-action):** Các câu lệnh nằm sau **await next()** được thực thi sau khi Action hoàn tất, cho phép đọc và hiệu chỉnh kết quả phản hồi (**ExecutedContext.Result**).

Bảng 7.2 So sánh chuyên sâu giữa Middleware Pipeline và Filter Pipeline

Tiêu chí	**Middleware Pipeline**	**Filter Pipeline**
Phạm vi hoạt động	Toàn bộ ứng dụng (Application-wide).	Chỉ áp dụng cho MVC Controller Actions và Razor Pages.
Nhận thức về MVC	Hoàn toàn không nhận thức (Chỉ thấy HttpContext).	Nhận thức đầy đủ về Action, Tham số đầu vào, ModelState.
Vị trí trong chuỗi	Chạy trước Filter Pipeline.	Chạy sau khi Routing đã chọn Endpoint đích.

Truy cập DI Scope	Truy cập thông qua HttpContext.RequestServices.	Hỗ trợ Dependency Injection tự nhiên qua ServiceFilter hoặc TypeFilter.
Khả năng ngắn mạch	Không gọi next(context), ghi trực tiếp ra Response.	Gán giá trị vào context.Result = new ObjectResult(...).
Trường hợp sử dụng	Xác thực JWT, Correlation ID, nén Gzip, xử lý Exception.	Thẩm định Model, Audit log nghiệp vụ, Idempotency-Key.

7.2.2 Kiểm tra Idempotency-Key thông qua Action Filter

Bản chất toán học của Idempotency trong hệ thống phân tán

Trong đại số trừu tượng và khoa học máy tính, một hàm số f được gọi là **bất biến (Idempotent)** nếu việc áp dụng hàm đó nhiều lần liên tiếp không làm thay đổi kết quả so với lần áp dụng đầu tiên:

$$(f(x)) = f(x)$$

Áp dụng vào giao thức HTTP (theo đặc tả RFC 7231 và RFC 9110):

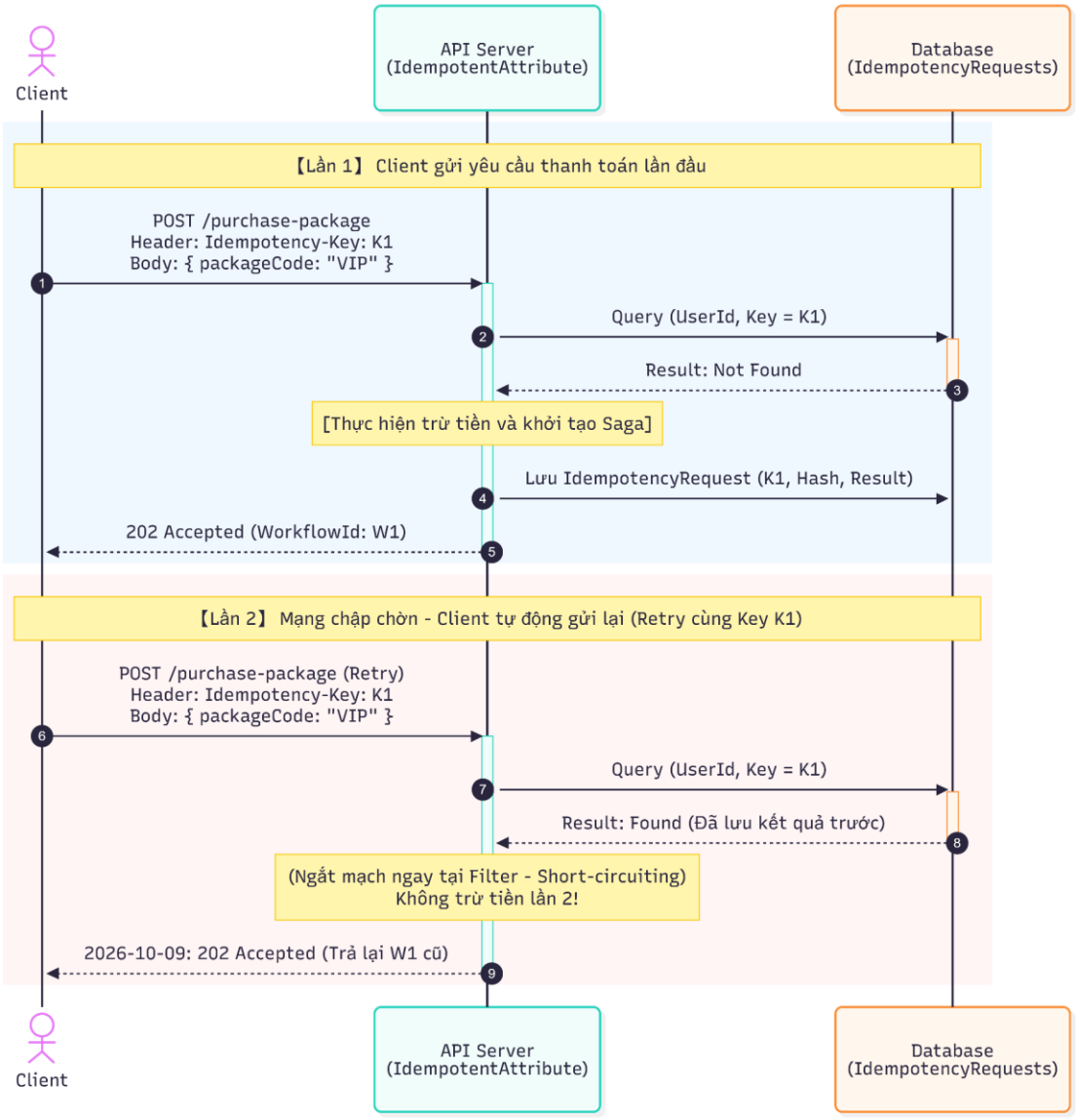
- Các phương thức **Idempotent mặc định:** GET, PUT, DELETE, HEAD, OPTIONS. (Ví dụ: gọi lệnh DELETE /api/v1/listings/10 mười lần thì tài nguyên 10 vẫn ở trạng thái đã bị xóa như lần đầu tiên).
- Phương thức **Non-idempotent:** POST và PATCH. Mỗi lần gửi POST /purchase-package, hệ thống sẽ thực hiện một giao dịch trừ tiền mới và kích hoạt một quy trình mới.

Trong môi trường mạng không ổn định (*Unreliable Network*), khi client gửi yêu cầu mua gói VIP nhưng bị rớt mạng đúng lúc server đã trừ tiền thành công nhưng chưa kịp gửi phản hồi về:

- Client/Trình duyệt sẽ tự động gửi lại (Retry) request.
- Nếu server không có cơ chế Idempotency, người dùng sẽ bị trừ tiền hai lần cho cùng một gói tin.

Đặc tả chuẩn quốc tế IETF: Idempotency-Key HTTP Header

Để giải quyết bài toán, nhóm triển khai đặc tả kỹ thuật dự thảo của IETF (*The Idempotency-Key HTTP Header Field*): Client đính kèm một mã khóa định danh duy nhất (UUID v4) trong HTTP.



Hình 7.3 Cơ chế chống xử lý giao dịch trùng lặp trong ASP.NET Core

Triển khai Action Filter kiểm soát Idempotency trong PropNest:

Tại tầng Controller (**ListingsController.cs**), endpoint mua gói tin kiểm tra sự hiện diện của header **Idempotency-Key**:

```
// src/PropNest.Api/Controllers/ListingsController.cs


[HttpPost("{listingId:long}/purchase-package")]
[Authorize(Roles = "Seller")]
public async Task<ActionResult<WorkflowDto>> PurchasePackage(
    long listingId,
    [FromHeader(Name = "Idempotency-Key")] string? idempotencyKey,
    PurchasePackageRequest request,
    CancellationToken cancellationToken)
{
    if (string.IsNullOrWhiteSpace(idempotencyKey))
    {
        return ValidationProblem(new ValidationProblemDetails(new Dictionary<string, string[]>
        {
            ["Idempotency-Key"] = ["The Idempotency-Key header is required."]
        }));
    }


    var userId = GetRequiredUserId();
    var requestHash = WorkflowService.ComputeRequestHash(JsonSerializer.Serialize(new { listingId, request }));


    var workflow = await workflowService.StartPurchaseAsync(new PurchasePackageCommand(
        listingId,
        userId,
```

```
        request.PackageCode,
        request.ChargeAmount,
        idempotencyKey,
        Request.Path,
        requestHash), cancellationToken);

    return AcceptedAtAction(nameof(WorkflowsController.GetByCorrelationId), "Workflows", new {
correlationId = workflow.CorrelationId }, workflow);
}
```

Tại tầng Application (**WorkflowService.cs**), dịch vụ thực hiện thuật toán băm SHA-256 và kiểm tra bản ghi Idempotency trong cơ sở dữ liệu:

```
// src/PropNest.Application/Workflows/WorkflowService.cs

public async Task<WorkflowDto> StartPurchaseAsync(PurchasePackageCommand command, CancellationToken
cancellationToken = default)
{
    if (string.IsNullOrWhiteSpace(command.IdempotencyKey))
    {
        throw new ArgumentException("Idempotency-Key is required.", nameof(command));
    }

    // 1. Tra cứu khóa Idempotency trong bảng cơ sở dữ liệu
    var existing = await workflowRepository.GetIdempotencyRequestAsync(command.UserId,
command.IdempotencyKey, cancellationToken);
    if (existing is not null)
```

```
    {
        // 2. Phát hiện xung đột gian lận: Dùng lại key cũ nhưng gửi payload khác
        if (!string.Equals(existing.RequestHash, command.RequestHash, StringComparison.Ordinal))
        {
            throw new InvalidOperationException("The Idempotency-Key has already been used with a different
request.");
        }

        // 3. Cơ chế Replay: Tái tạo kết quả đã xử lý trước đó mà không trừ tiền lần hai
        var replay = await workflowRepository.GetByCorrelationIdAsync(existing.CorrelationId, cancellationToken)
            ?? throw new InvalidOperationException("The idempotent request does not have an associated
workflow.");
        var steps = await workflowRepository.GetStepsByCorrelationIdAsync(replay.CorrelationId,
cancellationToken);
        return ToDto(replay, steps);
    }

    // 4. Nếu là giao dịch mới: Khởi tạo WorkflowInstance và lưu IdempotencyRequest
    var correlationId = Guid.NewGuid();
    var workflow = new WorkflowInstance(correlationId, command.ListingId, command.UserId,
command.PackageCode, command.ChargeAmount);
    var request = new IdempotencyRequest(
        command.IdempotencyKey,
        command.UserId,
        command.RequestPath,
        command.RequestHash,
        correlationId,
```

```
        DateTimeOffset.UtcNow.AddHours(24));

    var payload = JsonSerializer.Serialize(new
    {
        workflow.CorrelationId,
        workflow.ListingId,
        workflow.UserId,
        workflow.PackageCode,
        workflow.ChargeAmount
    });

    await workflowRepository.AddAsync(workflow, cancellationToken);
    await workflowRepository.AddIdempotencyRequestAsync(request, cancellationToken);
    await workflowRepository.AddOutboxMessageAsync(new OutboxMessage("PurchasePackageRequested",
payload, correlationId), cancellationToken);
    await unitOfWork.SaveChangesAsync(cancellationToken);

    return ToDto(workflow, Array.Empty<WorkflowStep>());
}

public static string ComputeRequestHash(string content)
{
    var bytes = SHA256.HashData(Encoding.UTF8.GetBytes(content));
    return Convert.ToHexString(bytes);
}
```

Hai phương án thay thế cho Idempotency dựa trên CSDL quan hệ

1. **Khóa phân tán Redis (Redis Distributed Lock) với RedLock:** Lưu trữ mã khóa Idempotency trên cụm Redis In-Memory với cơ chế thiết lập thời gian hết hạn (*TTL - Time To Live*). Phương án này giảm tải I/O cho CSDL chính, phù hợp với hệ thống có thông lượng hàng chục nghìn request/giây.
2. **Action Filter trung gian kết hợp bộ nhớ đệm (Idempotent Action Filter):** Đưa toàn bộ việc kiểm tra header và cache kết quả phản hồi lên tầng Action Filter để ngắt mạch trước khi đi vào tầng Service.

Hai trường hợp không nên áp dụng cơ chế Idempotency

1. **Các thao tác chỉ đọc dữ liệu (Safe HTTP Methods: GET, HEAD, OPTIONS):** Các truy vấn này theo đặc tả HTTP đã mang tính bất biến và không làm biến đổi trạng thái hệ thống. Việc áp đặt **Idempotency-Key** gây lãng phí băng thông và giảm hiệu năng bộ nhớ đệm trình duyệt (*Browser Cache*).
2. **Các API tiếp nhận dữ liệu cảm biến / sự kiện tần số cao (High-throughput Ingestion / Event Streaming):** Ở các luồng xử lý hàng trăm nghìn log hoặc metrics mỗi giây, độ trễ ở mức microsecond là tối thượng; việc tra cứu và băm khóa SHA-256 trên từng gói tin sẽ trở thành nút thắt cổ chai (*Bottleneck*) nghiêm trọng làm sập hệ thống tiếp nhận.

7.3 Xử lý ngoại lệ toàn cục (Global Exception Handling)

7.3.1 Phân loại ngoại lệ nghiệp vụ và ngoại lệ hệ thống trong kiến trúc Clean Architecture

Trong khoa học kỹ thuật phần mềm, một hệ thống chuyên nghiệp phải phân biệt rõ ràng giữa hai khái niệm:

- **Fault / Bug / Infrastructure Failure:** Sự cố kỹ thuật ngoài ý muốn (sập CSDL, cạn kiệt bộ nhớ, lỗi tham chiếu rỗng). Các lỗi này phải được giấu kín khỏi người dùng cuối để đảm bảo an ninh mạng.

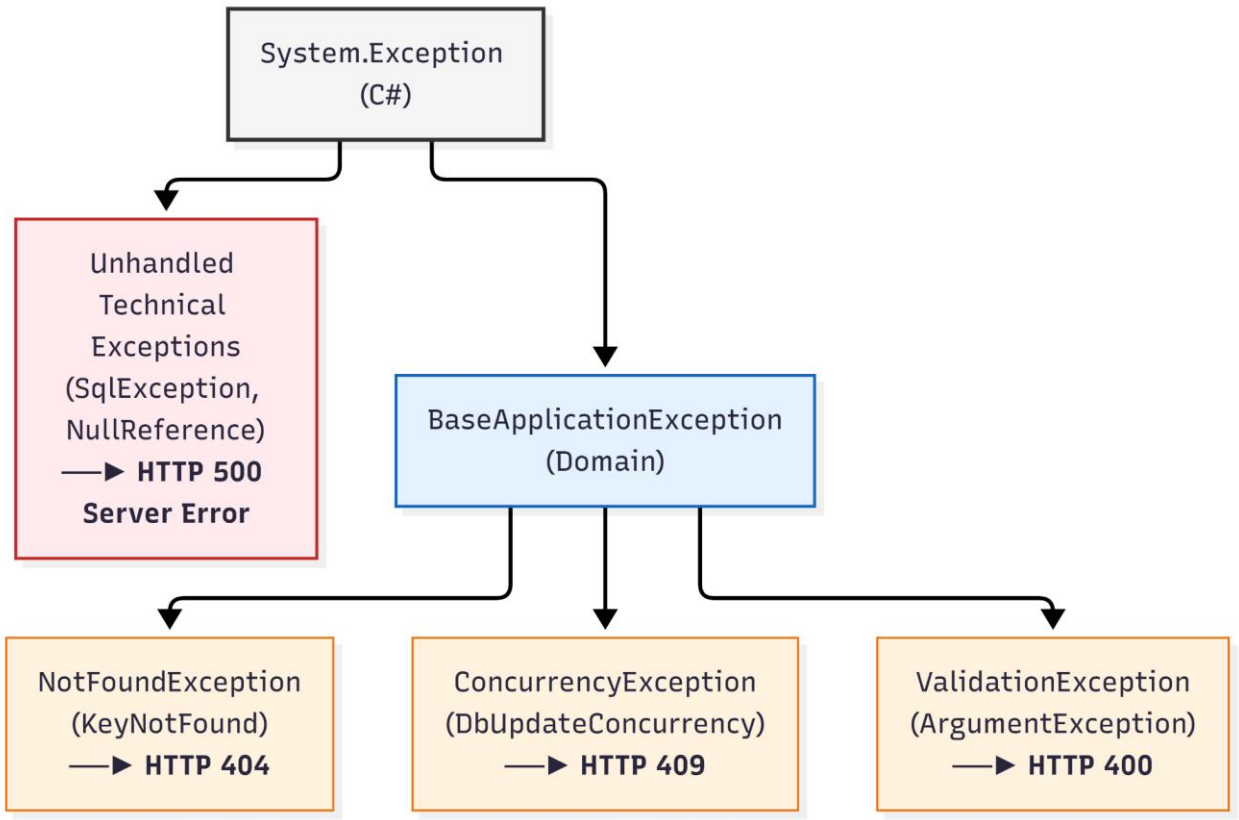
- **Domain Exception / Expected Violation:** Tình huống nghiệp vụ không hợp lệ được dự liệu trước (ví dụ: tài khoản không đủ số dư, tin đăng chưa duyệt không được phép ẩn).

Chi phí hiệu năng của ngoại lệ trong .NET Common Language Runtime (CLR):

Việc ném một ngoại lệ trong C# (throw new Exception()) có chi phí xử lý cực kỳ đắt đỏ:

- **Unwinding the Stack:** CLR phải dừng luồng thực thi để dò ngược toàn bộ ngăn xếp cuộc gọi (Stack Trace), thu thập địa chỉ con trỏ lệnh của từng hàm đã đi qua.
- **Cấp phát bộ nhớ Heap:** Đối tượng ngoại lệ cùng mảng StackFrame được cấp phát trên Heap, tạo áp lực dọn rác lên Garbage Collector.

Do đó, nguyên lý vàng trong thiết kế Backend là: **Không dùng Exception để điều khiển luồng nghiệp vụ thông thường (*Do not use Exceptions for control flow*)**. Tuy nhiên, khi một bất biến của Domain bị xâm phạm nghiêm trọng, Exception là chốt chặn cuối cùng để bảo vệ tính toàn vẹn dữ liệu.



Hình 7.4 Sơ đồ phân cấp hệ thống Exception và ánh xạ HTTP Status Code trong ASP.NET Core

Bảng 7.3 Bảng ánh xạ ngoại lệ nghiệp vụ sang mã trạng thái HTTP chuẩn RFC 7807

Ngoại lệ trong C#	**Mã HTTP**	**Tiêu đề lỗi (*Title*)**	**Tình huống nghiệp vụ thực tế trong PropNest**
ArgumentException	400	Tham số đầu vào không hợp lệ	Giá tiền âm, thiếu Idempotency-Key.
UnauthorizedAccessException	403	Không đủ quyền truy cập tài nguyên	Seller A cố tình cập nhật tin đăng thuộc quyền sở hữu của Seller B.
KeyNotFoundException	404	Không tìm thấy tài nguyên yêu cầu	Tra cứu tin đăng với listingId không tồn tại trong hệ thống.
DbUpdateConcurrencyException	409	Xung đột cập nhật dữ liệu đồng thời	Hai người cùng sửa một tin đăng; RowVersion không khớp.
InvalidOperationException	409	Thao tác không hợp lệ với trạng thái	Dùng lại Idempotency-Key với payload khác; duyệt tin đăng đang ở trạng thái Draft.
Exception (Chưa phân loại)	500	Lỗi nội bộ hệ thống	Sự cố ngắt kết nối SQL Server hoặc lỗi tràn bộ nhớ.

7.3.2 Triển khai ExceptionHandlingMiddleware theo chuẩn RFC 7807 ProblemDetails

Đặc tả RFC 7807 (Problem Details for HTTP APIs) và bản nâng cấp RFC 9457 là tiêu chuẩn quốc tế quy định lược đồ JSON thống nhất cho mọi thông báo lỗi do HTTP

API phát sinh. Tiêu chuẩn này chấm dứt tình trạng mỗi lập trình viên tự trả về một kiểu JSON lỗi khác nhau (như { "err": "msg" }, { "error_code": 123 }).

Cấu trúc JSON Schema chuẩn RFC 7807:

- **type (string URI):** Địa chỉ tài liệu tham chiếu mô tả định danh loại lỗi.
- **title (string):** Tiêu đề ngắn gọn tóm tắt loại lỗi (không thay đổi theo ngữ cảnh).
- **status (integer):** Mã số trạng thái HTTP tương ứng (400, 401, 403, 404, 409, 500).
- **detail (string):** Lời giải thích tường minh nguyên nhân lỗi cụ thể cho phiên giao dịch hiện tại.
- **instance (string URI):** Đường dẫn endpoint phát sinh lỗi.
- **extensions:** Các thuộc tính mở rộng (ví dụ mã truy vết correlationId, dấu thời gian timestamp).

Hệ thống PropNest kết hợp hai tầng phòng thủ:

1. Phương thức mở rộng **HandleException** tại **ControllerExtensions.cs** để hỗ trợ xử lý lỗi trực tiếp tại Controller Action:

```
// src/PropNest.Api/Extensions/ControllerExtensions.cs


public static ActionResult HandleException(this ControllerBase controller, Exception exception)
{
   var (statusCode, title) = exception switch
   {
      KeyNotFoundException => (StatusCodes.Status404NotFound, "Không tìm thấy tài nguyên yêu cầu."),
      UnauthorizedAccessException => (StatusCodes.Status403Forbidden, "Không đủ quyền truy cập tài
nguyên."),
      DbUpdateConcurrencyException => (StatusCodes.Status409Conflict, "Dữ liệu đã bị thay đổi bởi người
khác."),
      InvalidOperationException => (StatusCodes.Status409Conflict, "Thao tác không hợp lệ đối với trạng thái
hiện tại."),
```

```
        ArgumentException => (StatusCodes.Status400BadRequest, "Dữ liệu yêu cầu không hợp lệ."),
        _ => (StatusCodes.Status500InternalServerError, "Đã xảy ra lỗi hệ thống.")
    };

    var problem = new ProblemDetails
    {
        Status = statusCode,
        Title = title,
        Detail = statusCode == StatusCodes.Status500InternalServerError
            ? "Đã xảy ra sự cố nội bộ hệ thống. Vui lòng liên hệ quản trị viên."
            : exception.Message,
        Instance = controller.Request.Path
    };

    return controller.StatusCode(statusCode, problem);
}
```

2. Lớp **ExceptionHandlingMiddleware.cs** đóng vai trò chốt chặn toàn cục cao nhất bao bọc toàn bộ HTTP Pipeline, áp dụng kỹ thuật Source Generated Logging để ghi log lỗi mà không cấp phát bộ nhớ (Zero-Allocation):

```
// src/PropNest.Api/Middleware/ExceptionHandlingMiddleware.cs

using System.Diagnostics;
using System.Text.Json;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
```

```
namespace PropNest.Api.Middleware;


public sealed partial class ExceptionHandlingMiddleware(RequestDelegate next,

ILogger<ExceptionHandlingMiddleware> logger)

{

   [LoggerMessage(Level = LogLevel.Error, Message = "Bắt được ngoại lệ chưa xử lý tại đường dẫn:

{RequestPath}")]

   private static partial void LogUnhandledException(ILogger logger, Exception exception, string requestPath);


   public async Task InvokeAsync(HttpContext context)

   {

      try

      {

         await next(context);

      }

      catch (Exception exception)

      {

         LogUnhandledException(logger, exception, context.Request.Path);

         await HandleExceptionAsync(context, exception);

      }

   }


   private static async Task HandleExceptionAsync(HttpContext context, Exception exception)

   {

      // Kiểm tra an toàn: Nếu phản hồi đã bắt đầu gửi stream về client thì không thể ghi đè Header

      if (context.Response.HasStarted)

      {
```

```
            return;
        }

        var (statusCode, title) = exception switch
        {
            KeyNotFoundException => (StatusCodes.Status404NotFound, "Không tìm thấy tài nguyên yêu cầu."),
            UnauthorizedAccessException => (StatusCodes.Status403Forbidden, "Không đủ quyền truy cập tài
nguyên."),
            DbUpdateConcurrencyException => (StatusCodes.Status409Conflict, "Dữ liệu đã bị thay đổi bởi người
khác."),
            InvalidOperationException => (StatusCodes.Status409Conflict, "Thao tác không hợp lệ đối với trạng thái
hiện tại."),
            ArgumentException => (StatusCodes.Status400BadRequest, "Dữ liệu yêu cầu không hợp lệ."),
            _ => (StatusCodes.Status500InternalServerError, "Đã xảy ra lỗi hệ thống.")
        };

        var correlationId = context.Items[CorrelationIdMiddleware.CorrelationIdHeaderName]?.ToString()
            ?? Activity.Current?.Id
            ?? context.TraceIdentifier;

        var problemDetails = new ProblemDetails
        {
            Status = statusCode,
            Title = title,
            Detail = statusCode == StatusCodes.Status500InternalServerError
                ? "Đã xảy ra sự cố nội bộ hệ thống. Vui lòng liên hệ quản trị viên."
                : exception.Message,
```

```
            Instance = context.Request.Path
        };

        problemDetails.Extensions["correlationId"] = correlationId;
        problemDetails.Extensions["timestamp"] = DateTimeOffset.UtcNow;

        context.Response.StatusCode = statusCode;
        context.Response.ContentType = "application/problem+json";

        await context.Response.WriteAsync(JsonSerializer.Serialize(problemDetails, JsonDefaults.Options));
    }
}
```

Khi xảy ra lỗi xung đột đồng thời (*Concurrency Conflict*), client nhận được gói tin JSON chuẩn RFC 7807 với mã **HTTP 409 Conflict**:

```
{
  "status": 409,
  "title": "Dữ liệu đã bị thay đổi bởi người khác.",
  "detail": "The listing was updated by another process. Please reload and try again.",
  "instance": "/api/v1/listings/10",
  "correlationId": "8f3d2a1b-4c5e-4b6a-9f1e-2d3e4f5a6b7c",
  "timestamp": "2026-10-30T13:30:00.1234567Z"
}
```

7.4 Interceptors và Correlation Logging

7.4.1 CorrelationIdMiddleware theo vết luồng dữ liệu xuyên suốt các tầng

Trong một hệ thống bất động sản doanh nghiệp, một yêu cầu từ người dùng có thể kích hoạt một chuỗi xử lý phức tạp: từ API Controller, nạp Outbox Message, chuyển giao sang Saga Worker Process, đến các dịch vụ bên ngoài. Nếu không có một định danh liên kết xuyên suốt, khi hệ thống gặp sự cố ở tầng Worker, đội ngũ vận hành hoàn toàn không thể xác định thao tác đó bắt nguồn từ HTTP Request nào.

Tiêu chuẩn công nghiệp sử dụng HTTP Header **X-Correlation-Id**: Một chuỗi UUID v4 duy nhất đại diện cho toàn bộ vòng đời của một phiên giao dịch xuyên suốt các ranh giới tiến trình.

```
// src/PropNest.Api/Middleware/CorrelationIdMiddleware.cs

namespace PropNest.Api.Middleware;

public sealed class CorrelationIdMiddleware(RequestDelegate next)
{
    public const string CorrelationIdHeaderName = "X-Correlation-Id";

    public async Task InvokeAsync(HttpContext context)
    {
        // 1. Kiểm tra Client có gửi sẵn mã truy vết hay không
        if (!context.Request.Headers.TryGetValue(CorrelationIdHeaderName, out var correlationId)
            || string.IsNullOrWhiteSpace(correlationId))
        {
            // Nếu không có, tự sinh định danh UUID v4 ngẫu nhiên
```

```
            correlationId = Guid.NewGuid().ToString("D");
        }

        // 2. Lưu vào HttpContext.Items để dùng trong bộ nhớ nội bộ của Request
        context.Items[CorrelationIdHeaderName] = correlationId.ToString();

        // 3. Đính ngược mã vết vào Response Header để client nhận biết đối soát
        context.Response.OnStarting(() =>
        {
            context.Response.Headers[CorrelationIdHeaderName] = correlationId.ToString();
            return Task.CompletedTask;
        });

        // 4. Đẩy mã vết vào LogContext của Serilog để tự động làm giàu mọi dòng log
        using (Serilog.Context.LogContext.PushProperty("CorrelationId", correlationId.ToString()))
        {
            await next(context);
        }
    }
}
```

Khi gọi bất kỳ API nào của PropNest, phản hồi HTTP luôn chứa header đối soát:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
X-Correlation-Id: e1b8a9c2-4f3d-4c5e-8b1a-2d3e4f5a6b7c
```

7.4.2 Ghi log request/response có cấu trúc bằng Serilog

1. Giới hạn của Plain Text Logging truyền thống và Bản chất của Structured Logging (RFC 5424)

Cách ghi log truyền thống bằng nối chuỗi văn bản:

```
logger.LogInformation("User " + userId + " updated listing " + listingId);
```

tạo ra các dòng văn bản phi cấu trúc. Khi đẩy lên các hệ thống quản lý log tập trung (như ElasticSearch, Grafana Loki, Seq), hệ thống không thể lập chỉ mục (*Index*) từng trường dữ liệu, dẫn đến việc không thể thực hiện các câu truy vấn phân tích như: "Tìm tất cả các lượt cập nhật của UserId = 10 trong khoảng thời gian từ 8h đến 9h".

Bản chất học thuật của Structured Semantic Logging (RFC 5424):

Structured Logging biến thông điệp log thành một đối tượng dữ liệu JSON có cấu trúc. Mã nguồn sử dụng **Message Template**:

```
logger.LogInformation("Người dùng {UserId} vừa cập nhật tin đăng {ListingId} với giá {Price:N0} VND",
    userId, listingId, price);
```

Trình biên dịch không nối chuỗi mà tách riêng mẫu thông điệp và các tham số, cho phép hệ thống lập chỉ mục tìm kiếm chính xác theo từng thuộc tính.

Trong **Program.cs**, Serilog được cấu hình tích hợp với bộ định dạng tùy biến **PropNestJsonFormatter**:

```
// src/PropNest.Api/Program.cs


Log.Logger = new LoggerConfiguration()
    .ReadFrom.Configuration(builder.Configuration)
    .Enrich.FromLogContext()
    .WriteTo.Console(new PropNestJsonFormatter())
    .CreateLogger();
```

```
builder.Host.UseSerilog();
```

Lớp **PropNestJsonFormatter.cs** chịu trách nhiệm định dạng mỗi bản ghi log thành một đối tượng JSON chuẩn hóa trên đúng một dòng văn bản (Single-line JSON), tối ưu hóa cho các công cụ thu thập log như Fluentd / Promtail:

```
// src/PropNest.Api/Logging/PropNestJsonFormatter.cs
using System.Text.Encodings.Web;
using System.Text.Json;
using Serilog.Events;
using Serilog.Formatting;

namespace PropNest.Api.Logging;

public sealed class PropNestJsonFormatter : ITextFormatter
{
    private static readonly JsonSerializerOptions SerializerOptions = new()
    {
        Encoder = JavaScriptEncoder.UnsafeRelaxedJsonEscaping
    };

    public void Format(LogEvent logEvent, TextWriter output)
    {
        var logObject = new Dictionary<string, object?>
        {
```

```
            ["Timestamp"] = logEvent.Timestamp.ToOffset(TimeSpan.FromHours(7)).ToString("yyyy-MM-
ddTHH:mm:ss.fffzzz"),
            ["Level"] = logEvent.Level.ToString(),
            ["Message"] = logEvent.RenderMessage()
        };

        foreach (var property in logEvent.Properties)
        {
            logObject[property.Key] = ExtractValue(property.Value);
        }

        if (logEvent.Exception != null)
        {
            logObject["Exception"] = logEvent.Exception.ToString();
        }

        output.WriteLine(JsonSerializer.Serialize(logObject, SerializerOptions));
    }
    // ...
}
```

Bản ghi log được xuất ra Console và tệp log có cấu trúc JSON hoàn chỉnh:


```
{"Timestamp":"2026-10-30T20:15:30.125+07:00","Level":"Information","Message":"HTTP POST
/api/v1/listings/10/purchase-package responded 202 in 45.2ms","CorrelationId":"e1b8a9c2-4f3d-4c5e-8b1a-
2d3e4f5a6b7c","UserId":100,"ListingId":10,"SourceContext":"PropNest.Api.Controllers.ListingsController"}
```

7.5 Tài liệu hóa API với OpenAPI / Swagger

7.5.1 Cấu hình bảo mật Swagger với Bearer JWT Scheme

Đặc tả **OpenAPI Specification (OAS 3.0 / 3.1)** là tiêu chuẩn mở của tổ chức Linux Foundation nhằm định nghĩa giao diện máy-đọc-được (Machine-Readable API Contract) cho các dịch vụ RESTful.

Trong ASP.NET Core, package **Swashbuckle.AspNetCore** sử dụng cơ chế Reflection lúc khởi động ứng dụng để quét toàn bộ các Controller, Action, Route Constraints và DTO để tự động sinh ra tệp **swagger.json**.

Để Swagger UI hỗ trợ kiểm thử các API yêu cầu quyền hạn (như các endpoint duyệt tin, nạp ví, ẩn tin), cấu hình **Security Definition** cần định nghĩa chuẩn xác Scheme **Bearer JWT**:

```
// src/PropNest.Api/Program.cs


builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "PropNest API",
        Version = "v1"
    });

    // 1. Định nghĩa chuẩn xác Scheme xác thực JWT Bearer
    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = SecuritySchemeType.Http,
```

```
        Scheme = "bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Description = "Nhập token JWT theo định dạng: Bearer {your_token}"
    });

    // 2. Thiết lập yêu cầu bảo mật toàn cục cho Swagger UI
    options.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference
                {
                    Type = ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            Array.Empty<string>()
        }
    });
});
```

Khi truy cập **/swagger**, giao diện Swagger UI hiển thị nút Authorize với biểu tượng ổ khóa. Lập trình viên có thể dán JWT Token vào hộp thoại để kiểm thử toàn bộ các endpoint bảo mật mà không cần thông qua Postman.

7.5.2 Chuẩn hóa định dạng JSON múi giờ Việt Nam và mô tả header Idempotency-Key trên OpenAPI

1. Bài toán lệch múi giờ trong ứng dụng bất động sản

Trong các ứng dụng giao dịch bất động sản tại Việt Nam, thời điểm đăng tin, thời hạn hiển thị gói tin và lịch sử thanh toán bắt buộc phải hiển thị chính xác theo múi giờ địa phương (UTC+7, Múi giờ Đông Dương - ICT). Nếu tuần tự hóa mặc định theo chuẩn UTC (Z), client có thể hiển thị sai lệch ngày (ví dụ: giao dịch phát sinh lúc 01:00 ngày 01/01/2026 tại Việt Nam sẽ bị biến thành 18:00 ngày 31/12/2025 UTC), gây tranh chấp pháp lý và nhầm lẫn cho khách hàng.

PropNest xây dựng lớp chuyển đổi JSON chuyên biệt **VietnamDateTimeOffsetJsonConverter** kế thừa từ **JsonConverter<DateTimeOffset>**:

```
// src/PropNest.Api/Serialization/VietnamDateTimeOffsetJsonConverter.cs

using System.Text.Json;
using System.Text.Json.Serialization;

namespace PropNest.Api.Serialization;

public sealed class VietnamDateTimeOffsetJsonConverter : JsonConverter<DateTimeOffset>
{
    private static readonly TimeZoneInfo VietnamTimeZone = ResolveVietnamTimeZone();

    public override DateTimeOffset Read(ref Utf8JsonReader reader, Type typeToConvert, JsonSerializerOptions
options)
    {
        var raw = reader.GetString();
```

```
    if (string.IsNullOrWhiteSpace(raw))
    {
        return default;
    }

    if (DateTimeOffset.TryParse(raw, out var parsed))
    {
        return TimeZoneInfo.ConvertTime(parsed, VietnamTimeZone);
    }

    throw new JsonException($"Không thể chuyển đổi chuỗi '{raw}' sang kiểu DateTimeOffset.");
}

public override void Write(Utf8JsonWriter writer, DateTimeOffset value, JsonSerializerOptions options)
{
    // Chuyển đổi chính xác sang múi giờ UTC+7 trước khi xuất chuỗi ISO-8601
    var vietnamTime = TimeZoneInfo.ConvertTime(value, VietnamTimeZone);
    writer.WriteStringValue(vietnamTime.ToString("yyyy-MM-ddTHH:mm:ss.fffzzz"));
}

private static TimeZoneInfo ResolveVietnamTimeZone()
{
    try
    {
        return TimeZoneInfo.FindSystemTimeZoneById("SE Asia Standard Time"); // Chuẩn Windows
    }
    catch (TimeZoneNotFoundException)
```

```
    {
        return TimeZoneInfo.FindSystemTimeZoneById("Asia/Ho_Chi_Minh"); // Chuẩn Linux/macOS
    }
  }
}
```

Đồng thời, trên các Controller Action, nhóm áp dụng tường minh các thuộc tính [ProducesResponseType] và [FromHeader] để tài liệu hóa OpenAPI:

```
// Trích xuất từ src/PropNest.Api/Controllers/ListingsController.cs


[HttpPost("{listingId:long}/purchase-package")]
[Authorize(Roles = "Seller")]
public async Task<ActionResult<WorkflowDto>> PurchasePackage(
    long listingId,
    [FromHeader(Name = "Idempotency-Key")] string? idempotencyKey,
    PurchasePackageRequest request,
    CancellationToken cancellationToken)
```

2. Kết quả thực thi

- Chuẩn hóa chuỗi thời gian: Mọi trường thời gian trong API (**createdAt, updatedAt, finishedAt**) đều được xuất ra với đuôi **+07:00**:

```
{
  "correlationId": "8f3d2a1b-4c5e-4b6a-9f1e-2d3e4f5a6b7c",
  "listingId": 10,
  "packageCode": "VIP",
  "chargeAmount": 150000,
```

```
 "status": "Running",
 "createdAt": "2026-10-30T20:30:00.000+07:00"
}
```

- Tài liệu Swagger OpenAPI: Swagger UI hiển thị rõ ràng tham số Header **Idempotency-Key** (in: **header**, type: **string**) và các mã phản hồi HTTP 202 Accepted, HTTP 400 Bad Request, HTTP 409 Conflict.

KẾT LUẬN CHƯƠNG 6

Chương 6 đã khảo cứu toàn diện và làm chủ các kỹ thuật xây dựng RESTful Web API cấp doanh nghiệp trên nền tảng **ASP.NET Core 8**, tạo lập tầng biên mạng an toàn và vững chắc cho hệ thống PropNest:

1. **Kiến trúc DTO và Cơ chế Thẩm định Mô hình (Model Validation)**: Luận chứng học thuật đã khẳng định tính tất yếu của việc tách biệt Data Transfer Object khỏi Domain Entity, bảo toàn tuyệt đối tính đóng gói của mô hình Rich Domain và triệt tiêu hoàn toàn lỗ hổng bảo mật Mass Assignment (CWE-915). Hệ thống đã khai thác hiệu quả cơ chế thẩm định tự động của **System.ComponentModel.DataAnnotations** phối hợp cùng bộ lọc **ModelStateInvalidFilter** kích hoạt bởi thuộc tính **[ApiController]**, cho phép ngắt mạch sớm các payload sai lệch ngay tại tầng Kestrel Server. Đồng thời, nhóm đã hiện thực hóa các giải pháp thẩm định nghiệp vụ chuyên sâu như kiểm tra tính toàn vẹn của Concurrency Token Base64 **RowVersion** và phân tích cấu trúc mảng JSON bằng **JsonDocument**.
2. **Filter Pipeline và Tính Bất biến trong Giao dịch (Idempotency)**: Làm sáng tỏ mô hình hàm lồng nhau hai chiều (Russian Doll Model) của **IAsyncActionFilter** thông qua bộ lọc ủy quyền thực tế **SellerAuthorizeAttribute**, chứng minh năng lực can thiệp tiền/hậu xử lý và ngắt mạch điều hướng người dùng. Đặc biệt, nghiên cứu đã giải quyết bài toán cốt lõi của hệ thống phân tán là tính bất biến $f(f(x)) =$

$f(x)$ theo dự thảo đặc tả kỹ thuật IETF I**dempotency-Key**. Bằng việc kết hợp mã băm mật mã học SHA-256 (**ComputeRequestHash**) với bảng lưu vết giao dịch **IdempotencyRequests**, hệ thống bảo vệ an toàn tuyệt đối các giao dịch thanh toán gói tin bất động sản, triệt tiêu rủi ro trừ tiền hai lần và phát hiện kịp thời các hành vi gian lận thay đổi dữ liệu (Payload Tampering).

3. **Chuẩn hóa Xử lý Ngoại lệ Toàn cục (Global Exception Handling)**: Phân tích sâu sắc chi phí hiệu năng của cơ chế giải phóng ngăn xếp (Stack Unwinding) và cấp phát bộ nhớ Heap của Exception trong CLR, từ đó định hình chiến lược phân loại rạch ròi giữa ngoại lệ nghiệp vụ dự liệu trước và sự cố hạ tầng ngoài ý muốn. Việc triển khai tầng đôi gồm phương thức mở rộng **ControllerExtensions.HandleException** và **ExceptionHandlingMiddleware** (áp dụng Source Generated Logging với mức cấp phát bộ nhớ bằng 0) đã chuẩn hóa toàn bộ các thông báo lỗi theo đặc tả quốc tế RFC 7807 **ProblemDetails**, vừa cung cấp thông tin minh bạch cho client, vừa bảo mật hệ thống thông qua cơ chế che giấu chi tiết lỗi nội bộ HTTP 500.
4. **Nâng cao Khả năng Quan sát (Observability) và Tài liệu hóa Giao tiếp API**: Thiết lập thành công chuỗi truy vết phân tán xuyên suốt các tầng kiến trúc thông qua **CorrelationIdMiddleware** với header chuẩn **X-Correlation-Id**, liên kết chặt chẽ vòng đời HTTP Request với thông điệp Outbox và tiến trình Saga phía sau. Hệ thống logging ngữ nghĩa có cấu trúc dựa trên Serilog kết hợp bộ định dạng tùy biến **PropNestJsonFormatter** đã xuất ra các dòng JSON một dòng chuẩn hóa, tối ưu hóa cho các hệ thống giám sát tập trung. Cuối cùng, việc chuẩn hóa định dạng thời gian múi giờ Việt Nam (UTC+7) qua **VietnamDateTimeOffsetJsonConverter** cùng hợp đồng OpenAPI 3.0 tích hợp cơ chế ủy quyền Bearer JWT đã hoàn thiện một giao diện lập trình trực quan, nhất quán và minh bạch.

Toàn bộ hệ thống kỹ thuật này tạo nên một tầng giao diện API vững chắc, có độ tin cậy cao, hiệu năng vượt trội và sẵn sàng phục vụ cho các quy trình điều phối giao dịch phân tán phức tạp ở các chương tiếp theo.

CHƯƠNG 8 BẢO MẬT, XÁC THỰC VÀ PHÂN QUYỀN HỆ THỐNG TRONG ASP.NET CORE

8.1 Tổng quan bảo mật hệ thống thông tin

8.1.1 Authentication (Xác thực danh tính)

Authentication (Xác thực) là tiến trình thiết minh và xác minh danh tính của một đối tượng (*entity* — có thể là người dùng cuối, một dịch vụ máy chủ, hoặc một thiết bị IoT) khi đối tượng đó gửi yêu cầu kết nối đến hệ thống. Về bản chất, Authentication trả lời cho câu hỏi trung tâm: **"Đối tượng đang thực hiện request là ai?"**

Trong môi trường ứng dụng Web API không trạng thái (*Stateless Web API*), Authentication chuyển đổi các bằng chứng định danh thô (*credentials*) do client gửi lên (như tên đăng nhập, mật khẩu, mã OTP, hoặc chữ ký số) thành một đối tượng danh tính chuẩn hóa mà framework có thể hiểu được.

Đối với ASP.NET Core, quá trình này được quản lý bởi **AuthenticationMiddleware**. Khi một HTTP Request đi qua pipeline xử lý, Middleware sẽ kích hoạt **Authentication Scheme** tương ứng đã được đăng ký. Nếu chứng thư hợp lệ, hệ thống sẽ trích xuất thông tin định danh và khởi tạo một đối tượng đại diện mang kiểu **ClaimsPrincipal**. Đối tượng này sau đó được gán trực tiếp vào ngữ cảnh xử lý **HttpContext.User**.

Một **ClaimsPrincipal** trong .NET chứa một hoặc nhiều **ClaimsIdentity**, trong đó mỗi **ClaimsIdentity** đại diện cho một tập hợp các lời khẳng định (**Claims**) phản ánh thuộc tính cá nhân của đối tượng, bao gồm:

- **Sub / NameIdentifier**: Định danh duy nhất của người dùng trong cơ sở dữ liệu (User ID).
- **Email**: Địa chỉ thư điện tử đã qua xác minh.
- **Role**: Các vai trò quản trị hoặc nghiệp vụ mà người dùng nắm giữ.

- **SecurityStamp**: Chuỗi băm ngẫu nhiên dùng để vô hiệu hóa toàn bộ Token/Session khi tài khoản đổi mật khẩu hoặc bị khóa khẩn cấp.

8.1.2 Authorization (Phân quyền truy cập tài nguyên)

Authorization (Phân quyền) là tiến trình thẩm định xem một danh tính đã được xác thực trước đó (*authenticated identity*) có đủ đặc quyền (*privileges*) để thực thi một hành động cụ thể hoặc truy xuất đến một tài nguyên dữ liệu nhất định hay không. Authorization trả lời cho câu hỏi: **"Danh tính này có quyền thực hiện hành động X trên tài nguyên Y hay không?"**

Tiến trình Authorization đứng phía sau Authentication trong luồng xử lý (Execution Pipeline). Nếu một yêu cầu chưa được xác thực danh tính đi vào tuyến đường bảo vệ, hệ thống sẽ trả về mã lỗi **HTTP 401 Unauthorized**. Nếu yêu cầu đã xác thực thành công nhưng không sở hữu đủ quyền hạn cần thiết, hệ thống sẽ trả về mã lỗi **HTTP 403 Forbidden**.

Đối với nền tảng **PropNest**, cơ chế Authorization được phân tách theo ba cấp độ quản lý từ diện rộng đến chi tiết:

1. **Cấp độ Đường dẫn / Endpoint (Endpoint Authorization):** Ngăn chặn việc truy cập trái phép vào các tuyến đường API dựa trên trạng thái đăng nhập hoặc vai trò tổng quan. Ví dụ: Endpoint **/api/v1/admin/analytics** chỉ chấp nhận các truy vấn mang vai trò **Admin**.
2. **Cấp độ Nghiệp vụ / Quy tắc (Policy-based Authorization):** Kiểm tra các điều kiện nghiệp vụ kết hợp phức tạp. Ví dụ: Chức năng duyệt tin nhanh chỉ dành cho tài khoản thuộc vai trò **Moderator** đồng thời phải sở hữu điểm uy tín (*Reputation Score*) lớn hơn 80 và đã bật xác thực 2 yếu tố (2FA).
3. **Cấp độ Bản ghi Dữ liệu / Tài nguyên (Resource-based Authorization):** Kiểm tra mối quan hệ giữa danh tính thực hiện yêu cầu và bản ghi dữ liệu trong cơ sở dữ liệu ở thời điểm thực thi runtime. Ví dụ: Người dùng mang vai trò **Seller** có quyền

truy cập API cập nhật tin đăng **PUT /api/v1/listings/{id}**, nhưng hệ thống phải kiểm tra xem giá trị **SellerId** trong bản ghi **Listing** đó có trùng khớp với **UserId** lấy từ JWT Token của người dùng hay không.

8.2 Các phương pháp xác thực hiện đại

8.2.1 So sánh Session-based Authentication và Token-based Authentication

Trong thiết kế kiến trúc phần mềm cho các ứng dụng Web và API, lựa chọn giữa **Session-based** (Stateful) và **Token-based** (Stateless) là một quyết định chiến lược có ảnh hưởng sâu sắc đến khả năng mở rộng (*scalability*), độ tin cậy (*reliability*) và hiệu năng (*performance*) của toàn bộ hệ thống.

Bảng 8.1 Bảng so sánh Session-based Authentication và Token-based Authentication

Tiêu chí kiến trúc	**Session-based Authentication**	**Token-based Authentication (JSON Web Token - Chọn cho PropNest)**
Trạng thái trên Server (Statefulness)	Stateful: Máy chủ buộc phải lưu trữ thông tin phiên làm việc (*Session State*) trong bộ nhớ RAM, cơ sở dữ liệu quan hệ, hoặc K-V Store (Redis).	Stateless: Máy chủ hoàn toàn không lưu trữ trạng thái phiên làm việc. Toàn bộ dữ liệu ngữ cảnh của người dùng được mã hóa và đóng gói hoàn chỉnh bên trong Token.

Khả năng mở rộng hàng ngang (Horizontal Scaling)	Phức tạp. Khi nhân bản ứng dụng thành nhiều instance đằng sau Load Balancer, bắt buộc phải cấu hình *Sticky Sessions* hoặc chia sẻ Session Store tập trung (Redis Cluster).	Dễ dàng. Các máy chủ API hoạt động hoàn toàn độc lập, chỉ cần dùng chung khóa bí mật (*Secret Key*) hoặc cặp khóa bất đối xứng (Public/Private Key) để xác minh tính hợp lệ của Token.
Phương thức lưu trữ phía Client	Thường được lưu tự động trong HTTP Cookie dưới dạng Set-Cookie: SessionID=... do trình duyệt quản lý.	Lưu tại LocalStorage, SessionStorage, hoặc an toàn nhất là trong In-Memory Variable kết hợp HTTP-Only, SameSite Cookie cho Refresh Token.
Hỗ trợ Đa nền tảng (Cross-Platform / Cross-Domain)	Hạn chế. Khó triển khai mượt mà cho ứng dụng di động native (iOS/Android), ứng dụng máy tính, hoặc tương tác giữa các Microservices. Dễ bị tấn công CSRF.	Tối ưu tuyệt đối cho RESTful APIs, Single Page Applications (React, Angular, Vue), Mobile Apps (Flutter, React Native) và mô hình Service-to-Service.
Chi phí truy vấn tài nguyên (I/O Cost)	Tốn kém. Với mỗi HTTP Request gửi tới, máy chủ phải thực hiện truy vấn I/O (Database SQL hoặc Redis) để kiểm tra tính tồn tại của Session ID.	Tối ưu. Kiểm tra Token bằng phép toán giải mã băm (*Cryptographic Verification*) trực tiếp trên CPU và bộ nhớ RAM, loại bỏ hoàn toàn việc truy vấn cơ sở dữ liệu ở bước Authentication.

Cơ chế thu hồi quyền (Revocation / Invalidation)	Tức thì và đơn giản. Server chỉ cần xóa bản ghi Session ID trong cSDL là tài khoản của người dùng lập tức bị đăng xuất.	Cần chiến lược thiết kế nâng cao: Thiết lập thời hạn sống ngắn cho Access Token (ví dụ: 15-60 phút) kết hợp Refresh Token và Blacklist/Token Revocation List trên Redis.

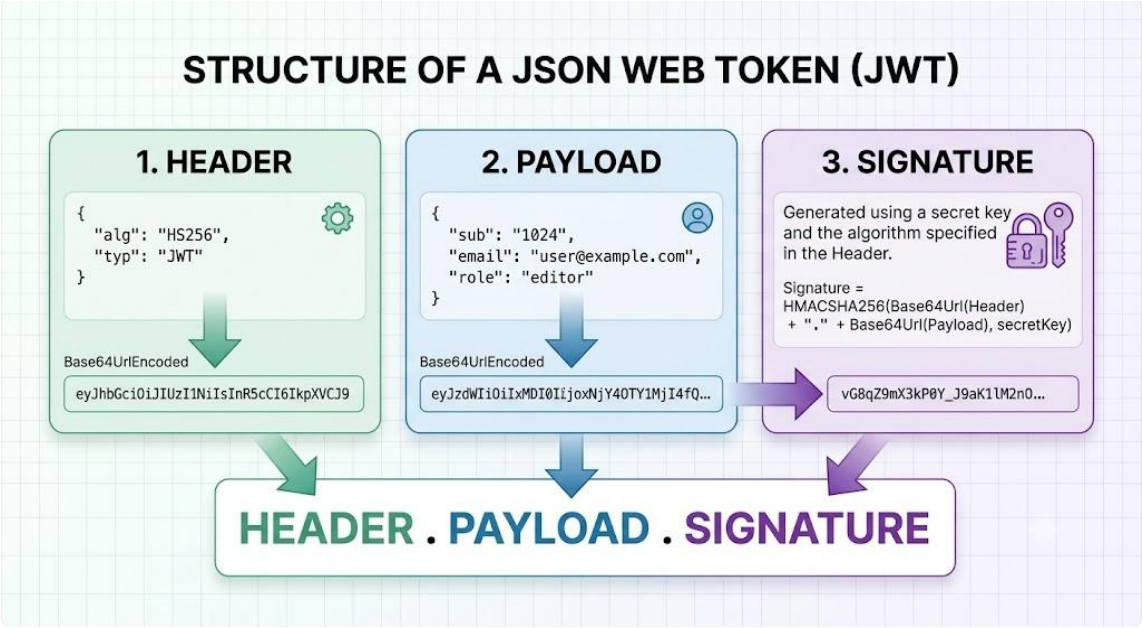
Lý do PropNest lựa chọn Token-based Authentication (JWT):

Nền tảng **PropNest** được định hướng phát triển theo kiến trúc Web API RESTful đa nền tảng, sẵn sàng phục vụ đồng thời Web App (ReactJS), Mobile App (Flutter) và các cổng tích hợp AI Moderation tự động. Việc áp dụng mô hình JWT giúp giải phóng máy chủ cơ sở dữ liệu khỏi hàng triệu truy vấn kiểm tra phiên làm việc trùng lặp, tối ưu hóa tốc độ phản hồi API ở mức millisecond và tạo tiền đề vững chắc cho việc mở rộng hạ tầng sang mô hình Containerization (Docker/Kubernetes).

8.2.2 Cấu trúc và nguyên lý hoạt động của JSON Web Token (Header, Payload, Signature)

JSON Web Token (JWT) là một tiêu chuẩn mở quốc tế (dưới định danh RFC 7519) quy định cấu trúc truyền tải thông tin an toàn giữa các bên dưới dạng một đối tượng JSON nhỏ gọn và tự chứa (self-contained).

Một chuỗi mã JWT hoàn chỉnh bao gồm ba phần tách biệt, được liên kết với nhau thông qua dấu chấm (.):



Hình 8.1 Cấu trúc của một JWT

1. Header (Phần đầu)

Header chứa các thông tin khai báo về mặt kỹ thuật của Token, mã hóa ở định dạng JSON dạng mã hóa Base64Url. Mẫu cấu trúc tiêu chuẩn bao gồm:

- **alg (*Algorithm*)**: Thuật toán mã hóa băm được sử dụng để tạo chữ ký (ví dụ: **HS256** - HMAC SHA-256 hoặc **RS256** - RSA Signature với SHA-256).
- **typ (*Type*)**: Định dạng Token, luôn là **"JWT"**.

```
{
 "alg": "HS256",
 "typ": "JWT"
}
```

2. Payload (Phần nội dung chứa Claims)

Payload chứa các phát biểu định danh (*Claims*) về người dùng và các thuộc tính bổ sung. Cần lưu ý rằng các thông tin trong Payload chỉ được mã hóa dạng **Base64Url** chứ

KHÔNG ĐƯỢC MÃ HÓA BẢO MẬT (*Not Encrypted*). Do đó, tuyệt đối không lưu trữ các dữ liệu nhạy cảm (như mật khẩu thô, số thẻ tín dụng) tại đây.

Các Claims trong Payload được chia làm 3 nhóm chính:

- Registered Claims (Khẳng định đã đăng ký tiêu chuẩn):
  - sub (*Subject*): Định danh người dùng (UserId = "1024").
  - iss (*Issuer*): Đơn vị phát hành Token
  - aud (*Audience*): Đối tượng thụ hưởng Token
  - exp (*Expiration Time*): Thời điểm Token hết hạn (đo bằng Unix Timestamp).
  - iat (*Issued At*): Thời điểm Token được khởi tạo.
- Public Claims: Các khái niệm thuộc tính được đăng ký công khai tại IANA JSON Web Token Registry.
- Private Claims: Các thuộc tính nghiệp vụ tự định nghĩa phục vụ riêng cho ứng dụng.

```
{
"sub": "1024",
"email": "seller@propnest.vn",
"fullname": "Tuan Tai",
"role": "Seller",
"iss": "https://api.propnest.vn",
"aud": "https://propnest.vn",
"iat": 1713888000,
"exp": 1713891600
}
```

3. Signature (Chữ ký điện tử)

Signature là thành phần quan trọng nhất đảm bảo tính chống giả mạo (*Anti-tampering*) và tính toàn vẹn dữ liệu (*Integrity*) của JWT. Chữ ký được sinh ra bằng cách lấy phần Header đã mã hóa Base64Url kết hợp với phần Payload đã mã hóa Base64Url, sau đó chạy qua thuật toán băm (đã khai báo ở Header) cùng với một chuỗi khóa bí mật (*Secret Signing Key*) được bảo vệ nghiêm ngặt phía Server.

Nguyên lý chống giả mạo: Nếu kẻ tấn công (Attacker) cố tình thay đổi dữ liệu trong Payload (ví dụ: thay đổi giá trị **"role": "Seller"** thành **"role": "Admin"** để leo thang đặc quyền) và gửi Token lên hệ thống, máy chủ API sẽ tiến hành tính toán lại Chữ ký điện tử dựa trên Header + Payload nhận được và **SecretKey** có sẵn. Do Attacker không sở hữu **SecretKey**, chữ ký do Attacker tự tính toán lại sẽ không khớp với chữ ký được kiểm tra trên Server. Yêu cầu lập tức bị ngắt bỏ với mã phản hồi **HTTP 401 Unauthorized**.

8.3 ASP.NET Core Identity và Cơ chế băm mật khẩu

8.3.1 Thuật toán băm mật khẩu chuẩn doanh nghiệp PBKDF2 với Salt

Trong các hệ thống quản trị tài khoản, việc lưu trữ mật khẩu dưới dạng văn bản thô (*Plaintext*) là một sai lầm nghiêm trọng vi phạm các quy chuẩn an ninh quốc tế (như OWASP, ISO 27001, PCI-DSS). ASP.NET Core Identity giải quyết triệt để vấn đề này bằng cách tích hợp mặc định lớp băm mật khẩu chuyên dụng **PasswordHasher<TUser>**.

Thuật toán băm PBKDF2 (Password-Based Key Derivation Function 2)

Lớp **PasswordHasher<TUser>** trong .NET 8 áp dụng thuật toán **PBKDF2** kết hợp với hàm băm pseudorandom **HMAC-SHA256** hoặc **HMAC-SHA512** (thuộc phiên bản V3 Password Hashing Format). PBKDF2 hoạt động bằng cách áp dụng một hàm băm cơ bản cùng với một chuỗi ngẫu nhiên độc lập (**Salt**) qua hàng trăm nghìn vòng lặp băm (*Iteration Count*).

- **Vai trò của Salt:** Khi một người dùng tạo mật khẩu, hệ thống tự động sinh một chuỗi byte ngẫu nhiên chất lượng cao (*Cryptographically Secure Pseudorandom Number Generator - CSPRNG*) có độ dài 128-bit (16 bytes). Salt được nối thêm vào chuỗi mật khẩu trước khi băm. Kỹ thuật này giúp loại bỏ hoàn toàn nguy cơ bị tấn công bởi **Rainbow Table Attacks** (Bảng tra cứu các chuỗi băm tính toán sẵn), đảm bảo rằng hai người dùng đặt mật khẩu hoàn toàn giống nhau **("Abc@123")** vẫn sẽ thu được hai chuỗi băm hoàn toàn khác biệt trong cơ sở dữ liệu.
- **Khái niệm Key Stretching & Iteration Count:** Để đối phó với sức mạnh tính toán song song cực nhanh của các chip đồ họa (GPU/ASIC) trong các cuộc tấn công Brute-force, PBKDF2 bắt buộc máy chủ phải thực hiện lặp đi lặp lại tiến trình băm. Mặc định trong .NET 8 ASP.NET Core Identity, số lượng vòng lặp băm được thiết lập ở mức **100,000 iterations**. Chi phí tính toán này không làm ảnh hưởng đáng kể đến trải nghiệm của một người dùng khi đăng nhập (tốn khoảng vài chục milliseconds), nhưng sẽ biến nỗ lực thử hàng tỷ mật khẩu của kẻ tấn công thành một công việc kéo dài hàng chục năm.

Ví dụ code :

```
var user = new ApplicationUser(
    request.Email.Trim().ToLowerInvariant(),
    request.FullName,
    request.PhoneNumber,
    "Seller");

var result = await userManager.CreateAsync(user, request.Password);

if (!result.Succeeded)
{
    return BadRequest(result.Errors);
```

```
}
// CreateAsync tự động dùng PasswordHasher<ApplicationUser>
// để băm password bằng PBKDF2 với salt ngẫu nhiên trước khi lưu vào PasswordHash.
```

8.3.2 Cơ chế khóa tài khoản (Lockout) chống tấn công dò mật khẩu Brute-force

Tấn công dò mật khẩu tự động (*Credential Stuffing / Brute-force*) là phương thức mà kẻ tấn công sử dụng các script tự động để gửi hàng loạt yêu cầu đăng nhập liên tục với các kết hợp mật khẩu khác nhau.

ASP.NET Core Identity giải quyết nguy cơ này thông qua hệ thống Account Lockout Manager tích hợp sẵn.

Nguyên lý vận hành:

1. Mỗi khi người dùng gửi yêu cầu đăng nhập sai mật khẩu, hàm **AccessFailedAsync(user)** được kích hoạt, làm tăng giá trị của cột **AccessFailedCount** trong cơ sở dữ liệu thêm 1 unit.
2. Khi giá trị **AccessFailedCount** vượt quá ngưỡng cấu hình tối đa **(MaxFailedAccessAttempts)**, hệ thống tự động kích hoạt trạng thái Lockout bằng cách ghi thời điểm hết hạn khóa vào cột **LockoutEnd** (kiểu **DateTimeOffset** theo chuẩn UTC).
3. Trong toàn bộ khoảng thời gian từ hiện tại cho đến **LockoutEnd**, mọi yêu cầu đăng nhập gửi đến tài khoản này sẽ ngay lập tức bị từ chối ở tầng logic mà không cần thực thi hàm kiểm tra băm mật khẩu đắt đỏ **(VerifyHashedPassword)**. Điều này bảo vệ tài nguyên CPU của máy chủ API không bị suy kiệt.
4. Khi người dùng đăng nhập thành công, hàm **ResetAccessFailedCountAsync(user)** sẽ đưa cột **AccessFailedCount** về 0.

Cấu hình chi tiết cơ chế Lockout được thiết lập tập trung tại lớp cấu hình hạ tầng trong **Program.cs:**

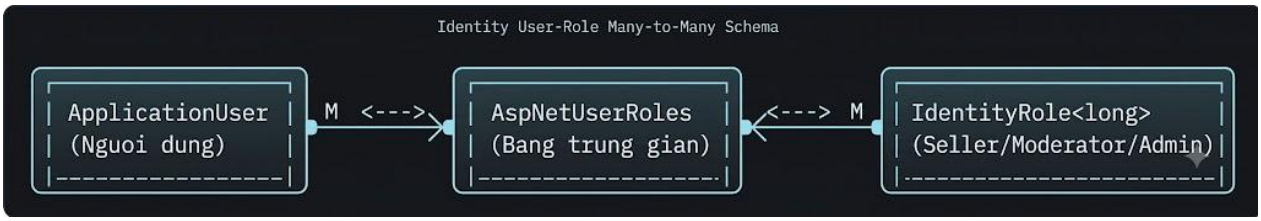
```
// Cấu hình Lockout và Mật khẩu trong Program.cs (Trích xuất từ src/PropNest.Api/Program.cs)
builder.Services.AddIdentityCore<PropNest.Domain.Users.ApplicationUser>(options =>
{
        // Quy tắc mật khẩu: cho phép mật khẩu kiểm thử "Test1234" (không bắt buộc ký tự đặc biệt)
        options.Password.RequireDigit = true;
        options.Password.RequireLowercase = true;
        options.Password.RequireUppercase = false;
    options.Password.RequireNonAlphanumeric = false;
        options.Password.RequiredLength = 6;
        options.User.RequireUniqueEmail = true;

        // Cấu hình Lockout chống tấn công vét cạn (Brute-force): 5 lần sai liên tiếp -> khóa 15 phút
        options.Lockout.AllowedForNewUsers = true;
    options.Lockout.MaxFailedAccessAttempts = 5;
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
})
.AddRoles<Microsoft.AspNetCore.Identity.IdentityRole<long>>()
.AddEntityFrameworkStores<PropNest.Infrastructure.Persistence.PropNestDbContext>();
```

8.4 Phân quyền dựa trên vai trò (Role-Based Access Control - RBAC)

8.4.1 Hệ thống phân cấp vai trò trong PropNest: Seller, Moderator, Admin

Mô hình **Role-Based Access Control (RBAC)** quản lý quyền hạn của người dùng thông qua khái niệm trung gian gọi là **Role (Vai trò)**. Quyền truy cập không được gán trực tiếp cho từng cá nhân mà được gán cho Vai trò; sau đó các Vai trò được liên kết với từng Tài khoản người dùng.



Hình 8.2 Lược đồ CSDL của User-Role

Hệ thống **PropNest** định nghĩa 3 vai trò phân cấp nghiệp vụ rõ ràng:

1. Seller (Người bán / Chủ bất động sản):

- Tạo mới, chỉnh sửa, ẩn/hiện, hoặc xóa các tin đăng bất động sản do chính tài khoản đó sở hữu.
- Quản lý thông tin tài khoản cá nhân, xem lịch sử giao dịch.
- Thực hiện nạp tiền và mua các gói ưu tiên hiển thị (Gói VIP).

2. Moderator (Ban kiểm duyệt nội dung):

- Truy cập bảng điều khiển kiểm duyệt tin đăng.
- Thực thi quyền Phê duyệt (Approve) hoặc Từ chối (Reject) các tin đăng bất động sản đang ở trạng thái Pending kèm theo lý do từ chối.
- Khóa hoặc gắn cờ báo cáo vi phạm đối với các tin đăng rác, thông tin sai sự thật.

3. Admin (Quản trị viên hệ thống):

- Toàn quyền quản trị danh mục người dùng, phân cấp vai trò (Assign Role).

- Thay đổi các cấu hình tham số toàn hệ thống (phí đăng tin, hạn mức gia hạn).
- Truy xuất báo cáo doanh thu toàn sàn và nhật ký hệ thống (*System Audit Logs*).

8.4.2 Cấu hình và sử dụng [Authorize(Roles = "...")] trên Controller và Endpoint

Trong ASP.NET Core, declarative authorization được thực hiện thông qua thuộc tính **[Authorize]**. Thuộc tính này có thể áp dụng ở cấp độ Controller (ảnh hưởng đến toàn bộ Action API nằm trong Controller) hoặc cấp độ từng Action API riêng biệt.

```
[ApiController]
[Authorize(Roles = "Seller")]
[Route("api/v1/listings")]
public sealed class ListingsController : ControllerBase
{
   [HttpPost]
   public async Task<IActionResult> Create(
      CreateListingRequest request,
      CancellationToken cancellationToken)
   {
      // Chỉ user có role Seller mới được thực hiện action này.
      var listing = await listingService.CreateAsync(
         new CreateListingCommand(/* dữ liệu listing */),
         cancellationToken);

      return Ok(listing);
   }
```

```
    [HttpGet]
    [AllowAnonymous]
    public async Task<IActionResult> GetPublicListings(
        CancellationToken cancellationToken)
    {
        // API này cho phép truy cập không cần JWT.
        var listings = await listingService.GetPublicListingsAsync(
            new GetPublicListingsQuery(),
            cancellationToken);

        return Ok(listings);
    }

    [HttpPost("{listingId:long}/hide")]
    [Authorize(Roles = "Seller")]
    public async Task<IActionResult> Hide(
        long listingId,
        CancellationToken cancellationToken)
    {
        // Có thể khai báo role ở cấp action nếu không khai báo ở Controller.
        return Ok();
    }
}
```

8.5 Policy-based và Resource-based Authorization

8.5.1 Định nghĩa Policy CanModerateListings cho ban kiểm duyệt

Policy-based Authorization phân tách việc kiểm tra quyền thành một tập hợp bao gồm các **Requirement** (Yêu cầu) và các **Handler** (Bộ xử lý). Một Policy có thể kết hợp nhiều điều kiện kiểm tra phức tạp dựa trên Claims, Role, và các biểu thức logic (Assertions).

Trong dự án PropNest, Policy CanModerateListings được đăng ký trong Program.cs và chỉ cho phép người dùng có role Moderator hoặc Admin thực hiện các chức năng kiểm duyệt:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CanModerateListings", policy =>
    {
        policy.RequireRole("Moderator", "Admin");
    });
});
```

Khi sử dụng trong controller:

```
[Authorize(Policy = "CanModerateListings")]
[HttpPost("listings/{listingId:long}/approve")]
public IActionResult Approve(long listingId)
{
    return Ok();
}
```

8.5.2 Resource-based Authorization bảo vệ quyền sở hữu tin đăng (chỉ chủ tin mới được sửa)

Trong ứng dụng **PropNest**, một yêu cầu nghiệp vụ điển hình là: "Người dùng mang vai trò Seller được phép chỉnh sửa tin đăng, nhưng **CHỈ ĐƯỢC PHÉP** chỉnh sửa những tin đăng **DO CHÍNH MÌNH TẠO RA**".

RBAC hoàn toàn bất lực trong trường hợp này, vì ở thời điểm gán **[Authorize(Roles = "Seller")],** hệ thống chưa biết người dùng đang muốn chỉnh sửa bản ghi **Listing** cụ thể nào. **Resource-based Authorization** được thiết kế để giải quyết bài toán này bằng cách truyền tài nguyên dữ liệu đã lấy từ CSDL vào engine phân quyền ở runtime.

Bước 1: Định nghĩa Requirement và Authorization Handle

```
public sealed class ListingOwnerRequirement : IAuthorizationRequirement;
```

```
public sealed class ListingOwnerHandler
    : AuthorizationHandler<ListingOwnerRequirement, Listing>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        ListingOwnerRequirement requirement,
        Listing listing)
    {
        var userId = context.User.FindFirstValue(
            ClaimTypes.NameIdentifier);

        if (userId is not null &&
            long.TryParse(userId, out var currentUserId) &&
```

```
            listing.OwnerUserId == currentUserId)
        {
            context.Succeed(requirement);
        }

        return Task.CompletedTask;
    }
}
```

Bước 2: Đăng ký policy và handler trong Program.cs

```
builder.Services.AddScoped<IAuthorizationHandler, ListingOwnerHandler>();


builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CanEditListing", policy =>
    {
        policy.RequireRole("Seller");
        policy.AddRequirements(new ListingOwnerRequirement());
    });
});
```

Bước 3: Inject IAuthorizationService vào controller, lấy listing từ database rồi kiểm tra quyền

```
[Authorize(Roles = "Seller")]
[HttpPut("{listingId:long}")]
```

```
public async Task<IActionResult> Update(
    long listingId,
    UpdateListingRequest request,
    CancellationToken cancellationToken)
{
    var listing = await listingService.GetByIdAsync(
        listingId,
        cancellationToken);

    if (listing is null)
    {
        return NotFound();
    }

    var result = await authorizationService.AuthorizeAsync(
        User,
        listing,
        "CanEditListing");

    if (!result.Succeeded)
    {
        return Forbid();
    }

    // Chỉ owner mới được cập nhật listing.
    return Ok();
}
```

8.6 Thực hành: Xây dựng Module Authentication và Phân quyền API

8.6.1 Hiện thực API Đăng ký tài khoản (POST /api/v1/auth/register)

1. Cấu hình Identity, JWT và Policy trong Program.cs

```
builder.Services.AddIdentityCore<ApplicationUser>(options =>
{
    options.Password.RequireDigit = true;
    options.Password.RequireLowercase = true;
    options.Password.RequiredLength = 6;
    options.User.RequireUniqueEmail = true;
})
.AddRoles<IdentityRole<long>>()
.AddEntityFrameworkStores<PropNestDbContext>();


builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidIssuer = jwt.Issuer,
            ValidateAudience = true,
            ValidAudience = jwt.Audience,
            ValidateIssuerSigningKey = true,
```

```
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(jwt.SigningKey)),
            ValidateLifetime = true,
            RoleClaimType = "role"
        };
    });
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CanModerateListings", policy =>
    {
        policy.RequireRole("Moderator", "Admin");
    });
});
```

2. API đăng ký tài khoản

```
[AllowAnonymous]
[HttpPost("register")]
public async Task<IActionResult> Register(
    RegisterRequest request,
    CancellationToken cancellationToken)
{
    var user = new ApplicationUser(
        request.Email.Trim().ToLowerInvariant(),
        request.FullName,
        request.PhoneNumber,
```

```
        "Seller");

    var result = await userManager.CreateAsync(
        user,
        request.Password);

    if (!result.Succeeded)
    {
        return BadRequest(result.Errors);
    }

    await userManager.AddToRoleAsync(user, "Seller");
    await walletService.CreateWalletAsync(user.Id, cancellationToken);

    return StatusCode(201, new
    {
        userId = user.Id,
        email = user.Email
    });
}
```

API sử dụng **UserManager<ApplicationUser>** của ASP.NET Core Identity để tạo tài khoản, tự động băm mật khẩu bằng **PasswordHasher**, gán role mặc định **Seller** và tạo wallet với số dư ban đầu bằng 0.

```
POST https://localhost:65147/api/v1/auth/login
Content-Type: application/json
```

```
{
  "email": "seller@example.com",
  "password": "Test1234"
}
```

Kết quả thành công:

```
200 OK
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs..."
}
```

Nếu email đã tồn tại hoặc password không đạt chính sách, API trả về 400 Bad Request.

8.6.2 Hiện thực API Đăng nhập và cấp phát JWT Token (POST /api/v1/auth/login)

1. API đăng nhập và cấp JWT

```
// src/PropNest.Api/Controllers/AuthController.cs - Phương thức Login hoàn chỉnh tích hợp Lockout
 [HttpPost("login")]
 public async Task<ActionResult<LoginResponse>> Login(
            LoginRequest request,
            CancellationToken cancellationToken)
 {
            var user = await userManager.FindByEmailAsync(request.Email.Trim().ToLowerInvariant());
            if (user is null)
```

```
{
return Unauthorized(new { message = "Invalid email or password." });
}

// 1. Kiểm tra tài khoản có đang bị khóa hay không
if (await userManager.IsLockedOutAsync(user))
{
return StatusCode(StatusCodes.Status423Locked, new
{
message = "Tài khoản tạm thời bị khóa do nhập sai mật khẩu quá 5 lần. Vui lòng thử lại sau 15 phút."
});
}

// 2. Xác thực mật khẩu và đếm lỗi nếu sai
if (!await userManager.CheckPasswordAsync(user, request.Password))
{
await userManager.AccessFailedAsync(user);
return Unauthorized(new { message = "Invalid email or password." });
}

// 3. Đăng nhập thành công: Reset bộ đếm lỗi về 0
await userManager.ResetAccessFailedCountAsync(user);

var roles = await userManager.GetRolesAsync(user);
var accessToken = jwtTokenService.GenerateAccessToken(user, roles);

// Chuẩn hóa cú pháp: return Ok (sửa lỗi gõ nhầm return O)
```

```
        return Ok(new LoginResponse(accessToken));
    }
```

2. Sinh JWT chứa role

```
public string GenerateAccessToken(
    ApplicationUser user,
    IList<string> roles)
{
    var claims = new List<Claim>
    {
        new(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
        new(JwtRegisteredClaimNames.Email, user.Email ?? string.Empty)
    };

    claims.AddRange(
        roles.Select(role => new Claim("role", role)));

    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(jwtOptions.SigningKey));

    var token = new JwtSecurityToken(
        issuer: jwtOptions.Issuer,
        audience: jwtOptions.Audience,
        claims: claims,
        expires: DateTime.UtcNow.AddMinutes(30),
```

```
        signingCredentials: new SigningCredentials(
            key,
            SecurityAlgorithms.HmacSha256));


    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

Endpoint **POST /api/v1/auth/login** sử dụng **UserManager.FindByEmailAsync** để tìm tài khoản và **CheckPasswordAsync** để xác minh mật khẩu. Nếu hợp lệ, hệ thống lấy role của user và gọi **IJwtTokenService** để tạo JWT access token.

```
POST https://localhost:65147/api/v1/auth/login
Content-Type: application/json
```

```
{
  "email": "seller@example.com",
  "password": "Test1234"
}
```

Kết quả thành công:

```
200 OK
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs..."
}
```

JWT chứa các claim chính:

```
{
  "sub": "1",
  "email": "seller@example.com",
  "role": "Seller",
  "iss": "PropNest",
  "aud": "PropNest.Client"
}
```

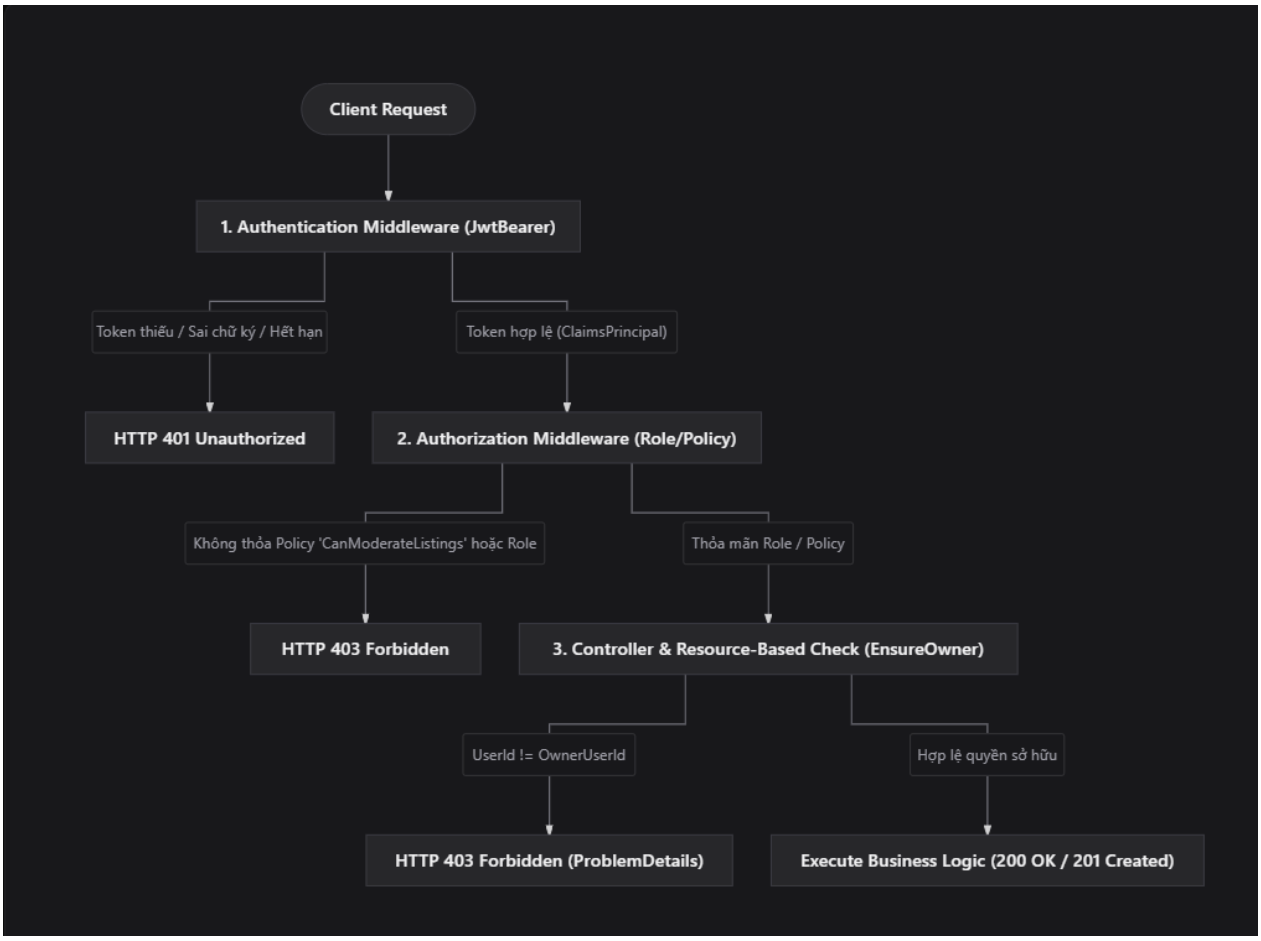
Nếu email hoặc mật khẩu sai, API trả về:

```
401 Unauthorized
```

8.6.3 Kiểm thử phân quyền truy cập giữa Seller và Moderator qua Swagger

8.6.3.1 Cơ chế kiến trúc bảo vệ đa tầng (Multi-Layer Authorization Architecture)

Hệ thống PropNest áp dụng mô hình phòng thủ ba lớp (Three-Tier Defense Model) trước khi một yêu cầu HTTP được phép tác động đến tài nguyên dữ liệu:



Hình 8.3 Sơ đồ quy trình xác thực và phân quyền truy cập tài nguyên trong ASP.NET Core

Tầng 1 - Xác thực danh tính (Authentication Layer - JwtBearerDefaults):

- Kiểm tra tính toàn vẹn của chữ ký mật mã (HMAC-SHA256) dựa trên **Jwt:SigningKey**, kiểm tra thời gian sống **(ValidateLifetime)**, Issuer và Audience.
- Nếu vi phạm hoặc không có header **Authorization: Bearer <token>**, Kestrel/ASP.NET Core sẽ ngắt pipeline ngay lập tức và phản hồi **401 Unauthorized** kèm header **WWW-Authenticate**.

Tầng 2 - Thẩm định vai trò & Chính sách (Role & Policy Authorization Layer):

- Được khai báo thông qua thuộc tính [Authorize(Roles = "...")] hoặc [Authorize(Policy = "...")].
- Middleware giải nén claim http://schemas.microsoft.com/ws/2008/06/identity/claims/role. Nếu người dùng đã xác thực nhưng danh sách vai trò không khớp với yêu cầu của endpoint (ví dụ: Seller cố tình gọi API kiểm duyệt của Moderator), hệ thống trả về mã 403 Forbidden.

Tầng 3 - Thẩm định quyền sở hữu tài nguyên (Resource-Based Ownership Invariant):

- Giải quyết bài toán bảo mật IDOR (Insecure Direct Object Reference). Khi Seller A cố tình gửi request chỉnh sửa **(PUT /api/v1/listings/{id})** mã tin đăng của Seller B, tầng controller kiểm tra bất biến **currentUser.EnsureOwner(listing.OwnerUserId).** Nếu phát hiện gian lận định danh, hệ thống kích hoạt ngoại lệ **UnauthorizedAccessException**, được bộ chuyển đổi ControllerExtensions.HandleException chuẩn hóa thành **403 Forbidden** theo chuẩn RFC 7807.

8.6.3.2 Quy trình khởi tạo và gán quyền tài khoản Moderator / Admin

Một câu hỏi mang tính nguyên lý bảo mật: **Tại sao hệ thống không cung cấp API đăng ký trực tiếp cho vai trò Moderator?** Trong thiết kế an toàn, việc tự động cấp quyền Moderator thông qua API công khai sẽ tạo ra lỗ hổng nghiêm trọng về **Leo thang**

đặc quyền (Privilege Escalation). Mọi người dùng đăng ký qua endpoint **POST /api/v1/auth/register** mặc định chỉ được gán vai trò **Seller**.

Do đó, để khởi tạo và phân quyền cho người dùng đóng vai trò **Moderator** hoặc **Admin**, nhóm phát triển triển khai 2 phương thức chuẩn hóa sau:

Cách 1: Thao tác trực tiếp trên Database qua SQL Script (Dùng trong môi trường Triển khai / Quản trị)

Khi cần nâng cấp một tài khoản Seller sẵn có (ví dụ: moderator_test@propnest.vn) lên làm kiểm duyệt viên, người quản trị chạy lệnh SQL sau trên hệ quản trị SQL Server:

```
-- Bước 1: Tra cứu UserId của tài khoản cần nâng quyền và RoleId của vai trò Moderator
DECLARE @TargetEmail NVARCHAR(256) = 'moderator_test@propnest.vn';
DECLARE @TargetUserId BIGINT;
DECLARE @ModeratorRoleId BIGINT;

SELECT @TargetUserId = Id FROM [dbo].[AspNetUsers] WHERE [NormalizedEmail] =
UPPER(@TargetEmail);
SELECT @ModeratorRoleId = Id FROM [dbo].[AspNetRoles] WHERE [NormalizedName] = 'MODERATOR';

-- Bước 2: Kiểm tra tồn tại và gán quyền trong bảng AspNetUserRoles
IF @TargetUserId IS NOT NULL AND @ModeratorRoleId IS NOT NULL
BEGIN
   IF NOT EXISTS (SELECT 1 FROM [dbo].[AspNetUserRoles] WHERE [UserId] = @TargetUserId AND
[RoleId] = @ModeratorRoleId)
   BEGIN
      INSERT INTO [dbo].[AspNetUserRoles] ([UserId], [RoleId])
      VALUES (@TargetUserId, @ModeratorRoleId);

      -- Đồng bộ trường Role dự phòng trong AspNetUsers (nếu có sử dụng làm cache claim)
```

```
        UPDATE [dbo].[AspNetUsers] SET [Role] = 'Moderator' WHERE [Id] = @TargetUserId;


        PRINT N'Thành công: Đã gán quyền Moderator cho tài khoản ' + @TargetEmail;
    END
    ELSE
    BEGIN
        PRINT N'Thông báo: Tài khoản này đã có quyền Moderator từ trước.';
    END
END
ELSE
BEGIN
    RAISERROR(N'Lỗi: Không tìm thấy tài khoản hoặc Role tương ứng trong hệ thống!', 16, 1);
END
GO
```

Cách 2: Khởi tạo tự động qua cơ chế Data Seeding trong Program.cs

Nhằm phục vụ môi trường tích hợp tự động (CI/CD) và kiểm thử cục bộ, hệ thống cài đặt phương thức Seeding tự động ngay khi khởi động ứng dụng trong Program.cs:

```
// Đăng ký Policy kiểm duyệt tại Dependency Injection
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CanModerateListings", policy =>
        policy.RequireRole("Moderator", "Admin"));
});


// Seed tự động danh sách Role và tài khoản Admin/Moderator mặc định
```

```
private static async Task SeedAdminUserAsync(IServiceProvider services)
{
    using var scope = services.CreateScope();
    var userManager = scope.ServiceProvider.GetRequiredService<UserManager<ApplicationUser>>();

    const string adminEmail = "admin@propnest.vn";
    const string adminPassword = "Admin@123456";

    var admin = await userManager.FindByEmailAsync(adminEmail);
    if (admin is null)
    {
        admin = new ApplicationUser(adminEmail, "PropNest Administrator", "0901234567", "Admin");
        var result = await userManager.CreateAsync(admin, adminPassword);
        if (result.Succeeded)
        {
            // Tài khoản Admin được gán đồng thời 2 vai trò Admin và Moderator
            await userManager.AddToRolesAsync(admin, new[] { "Admin", "Moderator" });
        }
    }
}
```

8.6.3.3 Minh chứng thực nghiệm kiểm thử phân quyền (Test Execution Evidence)

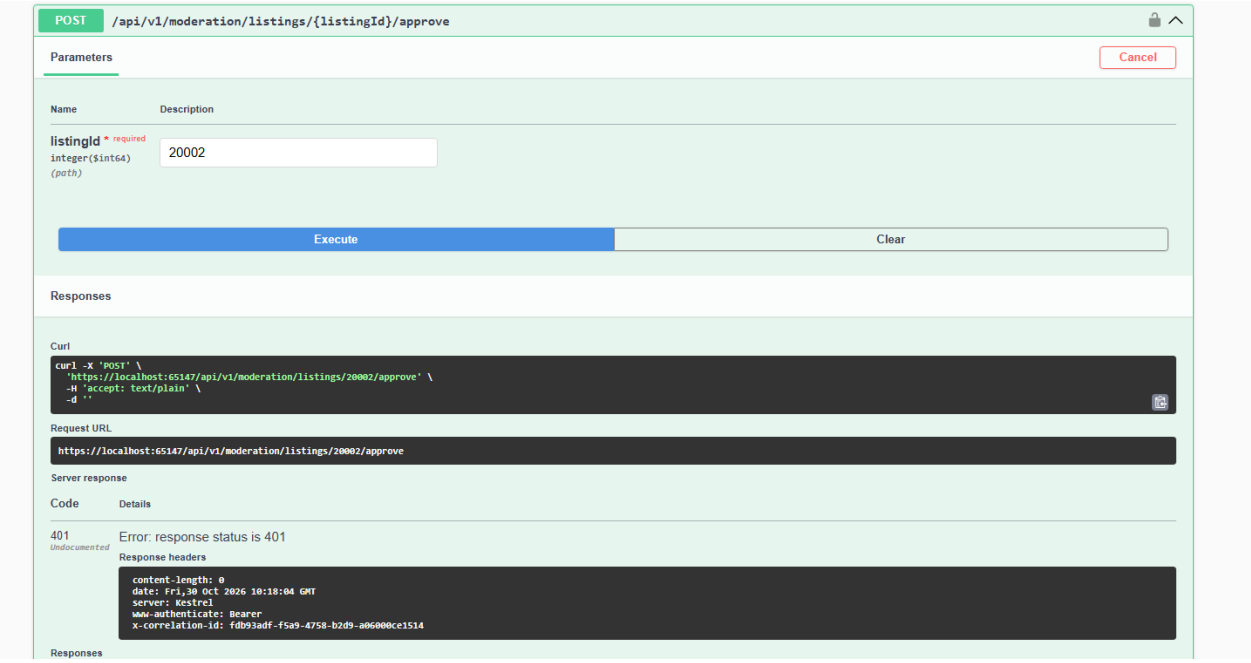
Để phản ánh tính chân thực và minh bạch, toàn bộ các kịch bản kiểm thử dưới đây được thực hiện trực tiếp trên endpoint phê duyệt tin đăng:

Endpoint Target: POST /api/v1/moderation/listings/20002approve

- **Kịch bản 1: Truy cập ẩn danh (Anonymous - Không mang Token)**

Mục tiêu: Chứng minh hệ thống chặn đứng mọi yêu cầu không được chứng thực.

- **Thông điệp phản hồi thực tế từ Server (HTTP Response):**



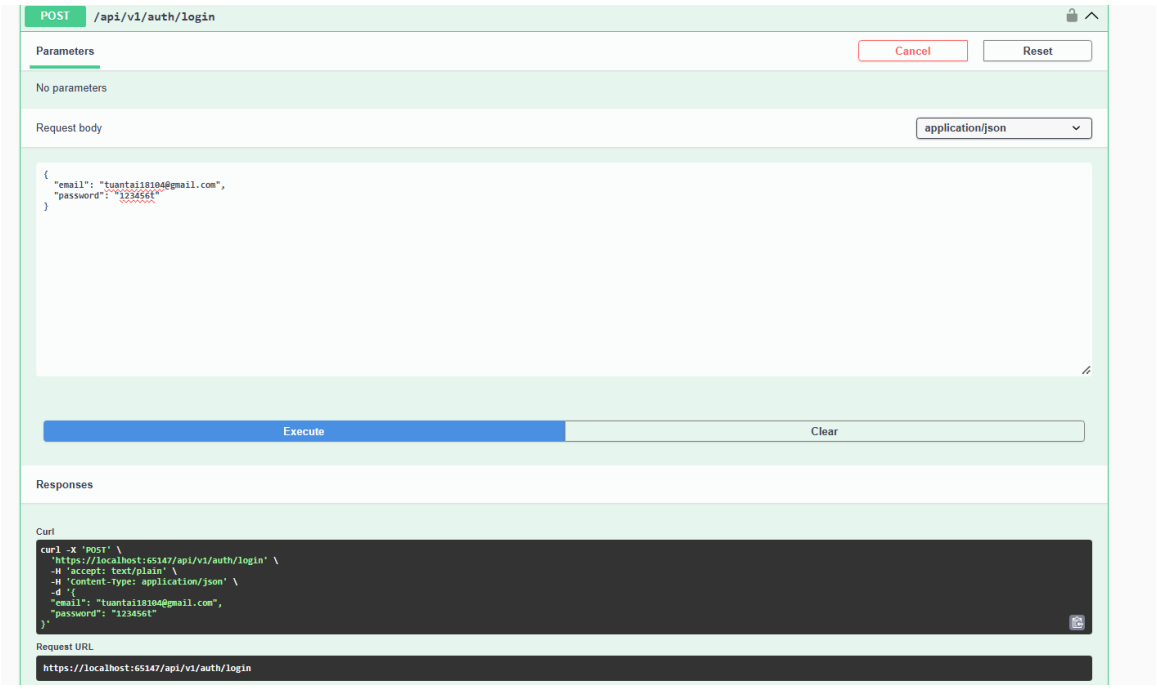
Hình 8.4 Phản hồi của Swagger

- **Đánh giá:** Bộ lọc **JwtBearerHandler** nhận diện yêu cầu thiếu header **Authorization**, lập tức ngắt pipeline xử lý và gửi về mã trạng thái **401 Unauthorized**.

– Kịch bản 2: Truy cập bằng tài khoản Seller (Vi phạm Policy kiểm duyệt)

***Mục tiêu*:** Chứng minh người dùng có danh tính hợp lệ nhưng không đủ quyền hạn sẽ bị từ chối truy cập.

- **Bước 2.1: Đăng nhập bằng tài khoản Seller để lấy Token:**



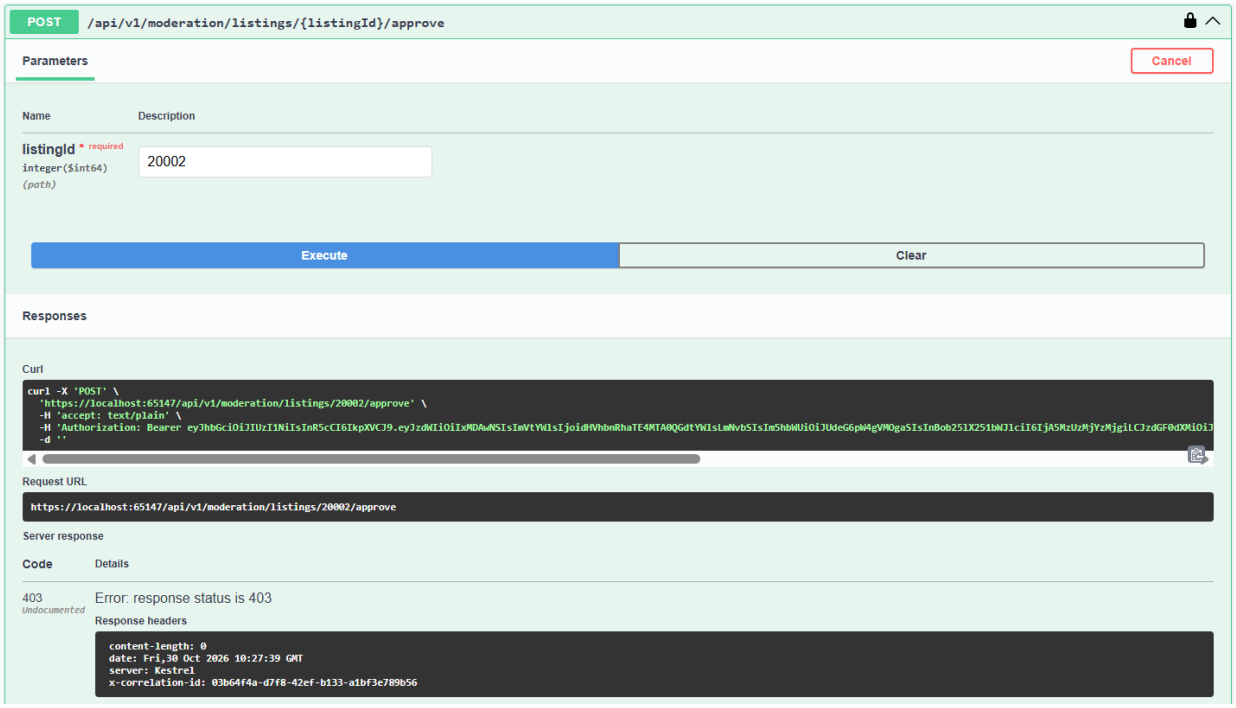
Hình 8.5 Đăng nhập bằng tài khoản Seller

- **Bước 2.2: Dùng Token của Seller để gọi endpoint Moderator:**



Hình 8.6 Swagger trả về Token

- **Thông điệp phản hồi thực tế từ Server (HTTP Response):**



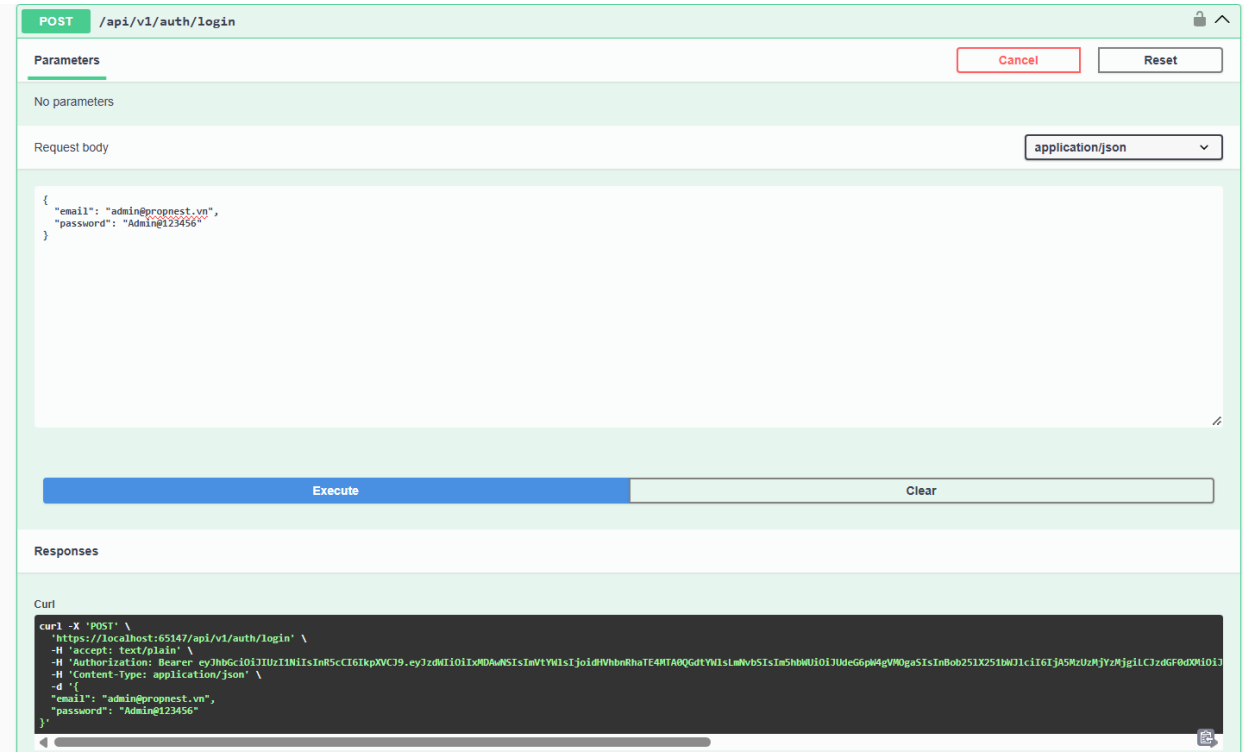
Hình 8.7 Phản hồi thực tế từ Server

- **Đánh giá:** Token được giải mã hợp lệ, **Authentication** thành công. Tuy nhiên, khi đi qua **AuthorizationMiddleware**, policy **"CanModerateListings"** yêu cầu một trong hai vai trò **Moderator** hoặc **Admin**. Do claim role của người dùng chỉ là **Seller**, middleware đã lập tức phát hành mã lỗi **403 Forbidden.**

– **Kịch bản 3: Truy cập bằng tài khoản Moderator / Admin (Thành công)**

***Mục tiêu*:** Chứng minh người dùng có đặc quyền hợp lệ được phép can thiệp và thực thi nghiệp vụ duyệt tin.

- **Bước 3.1: Đăng nhập bằng tài khoản Quản trị / Kiểm duyệt:**



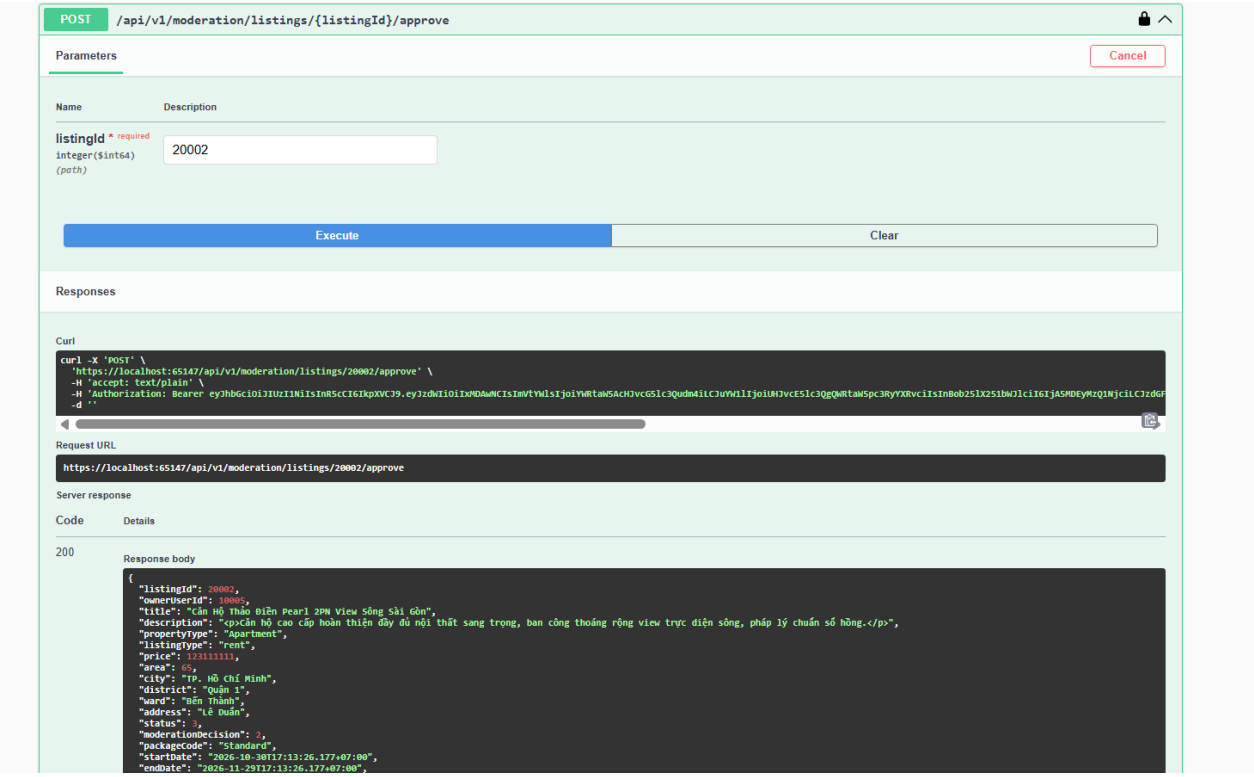
Hình 8.8 Đăng nhập bằng tài khoản Admin

- **Bước 3.2: Thực hiện phê duyệt tin đăng số 12:**



Hình 8.9 Phê duyệt tin đăng

- **Thông điệp phản hồi thực tế từ Server (HTTP Response & Payload):**



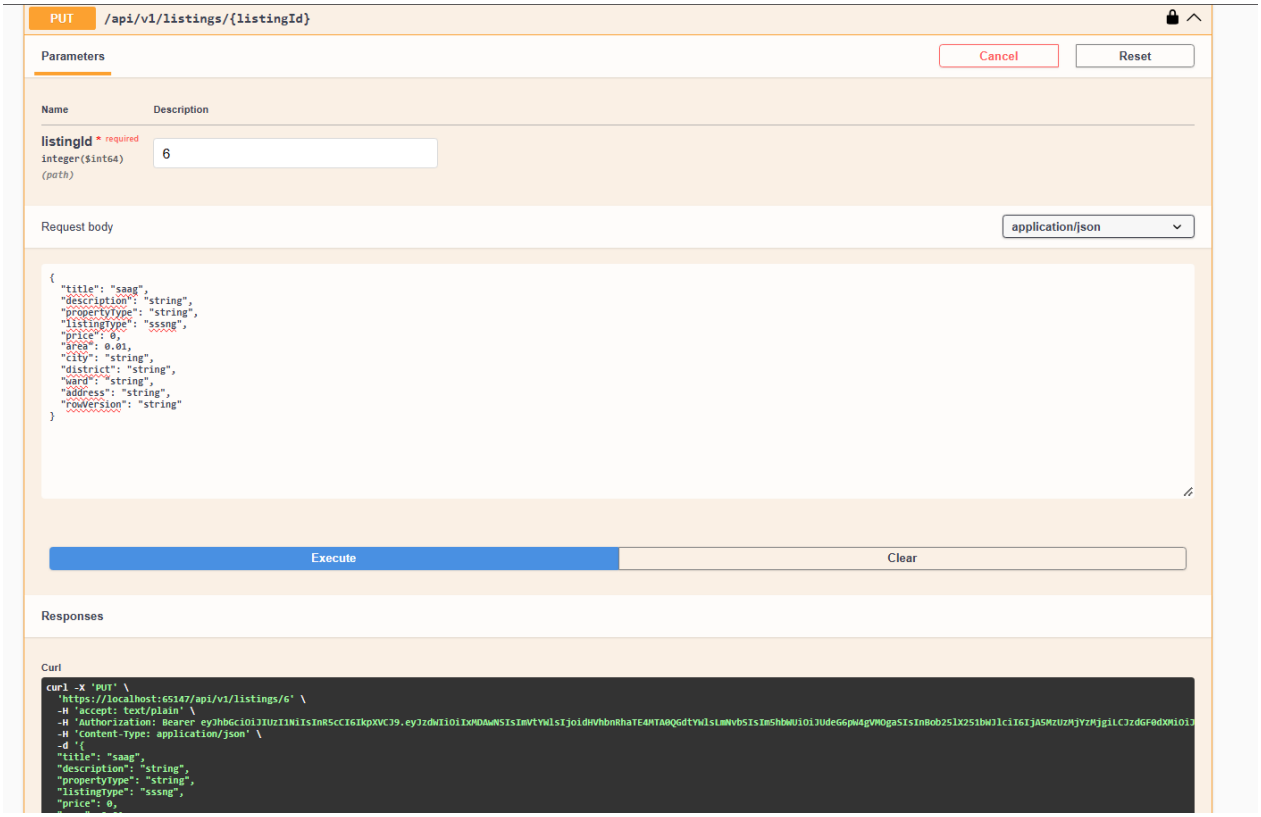
Hình 8.10 Phản hồi thực tế từ Server

- **Đánh giá:** Token thỏa mãn Policy **CanModerateListings**. Controller thực hiện gọi **listingService.ApproveAsync(12, moderatorUserId)**, chuyển trạng thái tin từ **PendingModeration** sang **Published**, tính toán chu kỳ hiển thị và trả về mã **200 OK** hoàn tất nghiệp vụ.

– Kịch bản 4: Kiểm thử kiểm soát quyền sở hữu (Resource-Based Authorization)

Mục tiêu: Chứng minh Seller A không thể can thiệp dữ liệu của Seller B dù cả hai đều mang vai trò hợp lệ là Seller.

- **Tình huống**: Seller 102 (Người dùng B) gửi request sửa tin đăng số 6(thuộc sở hữu của Seller 105 - Người dùng A):



Hình 8.11 Người dùng B gửi request sửa tin đăng số 6 thuộc sở Người dùng A)

- **Đánh giá:** Hệ thống phát hiện **currentUser.UserId (102) != listing.OwnerUserId (105)**. Phương thức bảo vệ **CurrentUserExtensions.EnsureOwner** kích hoạt ngăn chặn, trả về cấu trúc lỗi chuẩn hóa RFC 7807 ProblemDetails.

8.6.3.4 Quy trình kiểm thử trực quan thông qua Swagger UI

Để hỗ trợ kiểm thử viên và đối tác tích hợp thao tác trực quan:

1. Mở giao diện Swagger UI tại địa chỉ: https://localhost:65147/swagger.
2. Nhấn vào nút Authorize có biểu tượng ổ khóa màu xanh ở góc phải trên.
3. Trong hộp thoại Available authorizations, nhập chuỗi token theo định dạng Bearer: Bearer [Chuoi_AccessToken_nhận_tu_API_Login]
4. Nhấn Authorize → Close.

5. Chọn nhóm endpoint /api/v1/moderation/listings/{listingId}/approve, chọn Try it out, điền listingId: 20002 và chọn Execute.
6. Quan sát trực tiếp mã phản hồi tại mục Responses:

- Khi sử dụng token của Seller: Khung Server response hiển thị ô nền màu đỏ với Code: 403.
- Khi sử dụng token của Moderator/Admin: Khung hiển thị ô nền màu xanh lá với Code: 200 và toàn bộ cấu trúc dữ liệu JSON được định dạng chi tiết.

KẾT LUẬN CHƯƠNG 7

Chương 7 đã trình bày một cách toàn diện và hệ thống các giải pháp an ninh thông tin, xác thực danh tính và kiểm soát truy cập trên nền tảng **ASP.NET Core 8**, được nghiên cứu và áp dụng trực tiếp vào kiến trúc hệ thống của dự án **PropNest**. Việc thiết kế và triển khai mô hình bảo mật đa tầng (*Defense-in-Depth*) đảm bảo cho hệ thống đạt được tính toàn vẹn dữ liệu, tính khả dụng và khả năng bảo mật cấp doanh nghiệp.

Tổng kết các giải pháp an ninh cốt lõi đã được nghiên cứu và thực thi bao gồm:

1. ASP.NET Core Identity & Cơ chế bảo mật tài khoản:

- Băm mật khẩu an toàn: Triển khai thuật toán PBKDF2 kết hợp HMAC-SHA256, sinh chuỗi Salt 128-bit ngẫu nhiên độc lập cho từng tài khoản và thực hiện 100.000 vòng băm (iterations). Cơ chế này triệt tiêu hoàn toàn nguy cơ tấn công qua Bảng tra cứu tính sẵn (*Rainbow Table Attacks*) và làm suy giảm tối đa khả năng tấn công vét cạn (*Brute-force*).
- Khóa tài khoản tự động (Account Lockout): Cấu hình tự động vô hiệu hóa quyền truy cập tạm thời (15 phút) khi người dùng nhập sai mật khẩu vượt ngưỡng quy định (5 lần liên tiếp). Giải pháp này ngăn chặn hiệu quả các đợt tấn công *Credential Stuffing* mà không làm ảnh hưởng đến tài nguyên tính toán I/O của hệ thống máy chủ.

2. Xác thực danh tính theo mô hình Token-based (JWT Bearer):

- Áp dụng chuẩn JSON Web Token (RFC 7519) đóng gói đầy đủ ngữ cảnh danh tính (*Claims*) của người dùng.
- Cơ chế xác thực không trạng thái (*Stateless Authentication*) giúp loại bỏ sự phụ thuộc vào bộ nhớ phiên làm việc (*Session State*) phía Server, giảm chi phí truy vấn cơ sở dữ liệu ở tầng Authentication và tạo tiền đề tối ưu cho khả năng mở rộng hàng ngang (*Horizontal Scaling*) trên môi trường Containerization.
- Đảm bảo tính chống giả mạo (*Anti-tampering*) bằng thuật toán mã hóa chữ ký số HMAC-SHA256 kết hợp khóa bí mật (*Secret Signing Key*) quản lý tập trung.

3. Kiểm soát truy cập phân cấp (RBAC, Policy-based & Resource-based Authorization):

- Role-Based Access Control (RBAC): Phân định ranh giới chức năng rõ ràng giữa các nhóm người dùng chính trong hệ thống (*Seller, Moderator, Admin*) thông qua thuộc tính [Authorize(Roles = "...")] ở cấp độ Endpoint.
- Policy-Based Authorization: Thiết lập các quy tắc kiểm duyệt nâng cao dựa trên việc đóng gói tập hợp Requirements và Claims Identity (ví dụ: yêu cầu email đã qua xác minh, domain nội bộ).
- Resource-Based Authorization: Giải quyết bài toán phân quyền ngữ cảnh động ở cấp độ bản ghi dữ liệu tại thời điểm runtime thông qua IAuthorizationService và AuthorizationHandler. Cơ chế này đảm bảo nguyên tắc tối cần thiết: người dùng chỉ có quyền thao tác (*Update/Delete*) trên chính các tài nguyên dữ liệu thuộc sở hữu của mình (*Listing Ownership*).

CHƯƠNG 9 XỬ LÝ GIAO DỊCH PHÂN TÁN VÀ ĐIỀU PHỐI WORKFLOW TRONG ASP .NET CORE

Trong các kiến trúc phần mềm hướng dịch vụ phân tán (Distributed Systems và Microservices), việc chuyển đổi từ mô hình cơ sở dữ liệu dùng chung (Shared Database) sang mô hình mỗi dịch vụ quản lý dữ liệu độc lập (Database-per-Service) đem lại tính tự chủ, khả năng mở rộng linh hoạt và cô lập lỗi cao. Tuy nhiên, sự phân rã này đồng thời phá vỡ ranh giới của các giao dịch cục bộ ACID truyền thống. Một chuỗi thao tác nghiệp vụ phức tạp đòi hỏi sự phối hợp của nhiều phân hệ không thể chỉ dựa vào câu lệnh BEGIN TRANSACTION và COMMIT đơn thuần trên một máy chủ cơ sở dữ liệu duy nhất.

Chương này tập trung đi sâu vào các lý thuyết nền tảng và giải pháp kỹ thuật cốt lõi giải quyết bài toán giao dịch phân tán: từ những giới hạn của giao dịch Two-Phase Commit (2PC), định lý CAP và nguyên lý Eventual Consistency, cho đến việc thiết kế mô hình Saga Orchestration với State Machine trên thư viện MassTransit. Bên cạnh đó, chương phân tích toàn diện các mẫu kiến trúc bảo vệ độ tin cậy thông điệp bao gồm Transactional Outbox Pattern, Inbox Deduplication, cơ chế Idempotency Request và giao dịch hoàn tiền(Compensating Transactions) nhằm đảm bảo tính toàn vẹn dữ liệu tuyệt đối trong môi trường mạng bất định.

9.1 Thách thức giao dịch trong hệ thống Backend phân tán

9.1.1 Giới hạn của giao dịch Two-Phase Commit (2PC) trong môi trường microservices

Trong kiến trúc nguyên khối (Monolithic Architecture), tính toàn vẹn dữ liệu được bảo vệ mạnh mẽ thông qua chuẩn giao dịch ACID (Atomicity, Consistency, Isolation, Durability) do hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) cung cấp. Tất cả các thao tác ghi dữ liệu diễn ra bên trong một tiến trình duy nhất và được commit nguyên tử. Tuy

nhiên, khi hệ thống được phân tách thành các microservice với cơ sở dữ liệu riêng biệt, một nghiệp vụ như đặt mua dịch vụ cần tương tác với nhiều dịch vụ độc lập (quản lý người dùng, trừ tiền ví, kích hoạt quyền lợi tin đăng). Lúc này, giao dịch trở thành giao dịch phân tán (Distributed Transaction).

Về mặt lý thuyết truyền thống, giải pháp kinh điển để duy trì ACID trên nhiều node phân tán là giao thức Two-Phase Commit (2PC), thường được thực thi qua chuẩn XA Transaction do một bộ điều phối tập trung (Transaction Coordinator) giám sát qua hai giai đoạn:

- **Giai đoạn 1 - Chuẩn bị (Prepare / Voting Phase):** Bộ điều phối gửi thông điệp yêu cầu tất cả các dịch vụ tham gia (Resource Managers) chuẩn bị commit. Mỗi dịch vụ kiểm tra tính hợp lệ của dữ liệu, khóa các bản ghi liên quan (Acquire Locks), ghi nhận trạng thái vào Undo/Redo Log và phản hồi lại phiếu bầu 'Yes' (sẵn sàng) hoặc 'No' (thất bại).
- **Giai đoạn 2 - Xác nhận (Commit / Abort Phase):** Nếu tất cả các node đều bỏ phiếu 'Yes', bộ điều phối sẽ gửi lệnh 'Global Commit' để các node chính thức xác nhận thay đổi và giải phóng khóa. Nếu có ít nhất một node bỏ phiếu 'No' hoặc bị quá thời gian chờ (Timeout), bộ điều phối sẽ gửi lệnh 'Global Abort/Rollback' để tất cả các node hoàn tác dữ liệu.

Mặc dù 2PC đảm bảo tính toàn vẹn dữ liệu nghiêm ngặt, giao thức này bộc lộ những nhược điểm chí mạng khiến nó gần như bị loại bỏ trong các hệ sinh thái phân tán hiện đại và môi trường điện toán đám mây:

- **Giao thức chặn tài nguyên (Blocking Protocol):** 2PC là một giao thức chặn đồng bộ (Synchronous Blocking Protocol). Các dịch vụ tham gia phải duy trì các khóa trên hàng (Row Locks) suốt từ Giai đoạn 1 cho đến khi Giai đoạn 2 hoàn tất. Trong môi trường mạng diện rộng hoặc khi hệ thống chịu tải cao, việc giữ khóa kéo dài sẽ làm tê liệt các giao dịch khác, tạo ra hiện tượng suy thoái hiệu năng dây chuyền (Cascading Failure) và nguy cơ Deadlock rất lớn.

- **Điểm lỗi đơn lẻ (Single Point of Failure - SPOF):** Nếu bộ điều phối gặp sự cố sập nguồn (Crash) ngay sau Giai đoạn 1, các node tham gia sẽ rơi vào trạng thái lơ lửng (In-Doubt State). Chúng không thể tự ý commit (vì không biết các node khác thế nào) cũng không thể tự ý rollback, dẫn đến tài nguyên bị giam cầm vô thời hạn.
- **Suy giảm thông lượng nghiêm trọng:** Độ trễ của toàn bộ giao dịch bằng độ trễ của node phản hồi chậm nhất cộng thêm thời gian truyền thông điệp nhiều vòng qua mạng (Network Round-Trips). Hệ thống càng có nhiều service tham gia thì thông lượng (Throughput) càng sụt giảm theo hàm số mũ.
- **Không tương thích với hệ sinh thái công nghệ mới:** Rất nhiều hệ cơ sở dữ liệu hiện đại (MongoDB, Cassandra, Redis) cũng như các Message Broker (RabbitMQ, Apache Kafka) hoặc các API thanh toán bên thứ ba (VNPay, Momo, Stripe) không hề hỗ trợ chuẩn XA/2PC, khiến việc áp dụng 2PC trong thực tế trở nên bất khả thi.

9.1.2 Định lý CAP và nguyên lý tính nhất quán cuối cùng (Eventual Consistency)

Để khắc phục sự bế tắc của giao dịch ACID phân tán, ngành kỹ nghệ phần mềm dựa trên Định lý CAP (CAP Theorem) được đề xuất bởi Eric Brewer vào năm 2000. Định lý khẳng định rằng trong bất kỳ hệ thống phân tán chia sẻ dữ liệu nào, ta chỉ có thể thỏa mãn tối đa hai trong ba thuộc tính sau tại cùng một thời điểm:

- **Tính nhất quán (Consistency - C):** Mọi thao tác đọc đều nhận được dữ liệu được ghi mới nhất hoặc gặp lỗi, đảm bảo mọi client nhìn thấy trạng thái dữ liệu giống hệt nhau.
- **Tính sẵn sàng (Availability - A):** Mọi request gửi tới một node còn sống đều phải nhận được phản hồi không bị lỗi (Non-error response), dù phản hồi đó chưa chắc đã chứa dữ liệu mới nhất.

- **Khả năng chịu đựng phân mảnh mạng (Partition Tolerance - P):** Hệ thống vẫn tiếp tục duy trì hoạt động ngay cả khi có sự cố mất kết nối mạng hoặc phân mảnh giao tiếp giữa các cụm node.

Trong thế giới thực tế của mạng phân tán, sự cố đứt kết nối, trễ gói tin hoặc timeout (Partition - P) là điều tất yếu không thể tránh khỏi. Do đó, kiến trúc sư hệ thống buộc phải đứng trước sự đánh đổi giữa hai con đường:

- **Đánh đổi CP (Consistency + Partition Tolerance):** Nếu mạng bị phân mảnh, hệ thống từ chối phục vụ request hoặc chặn giao dịch để đảm bảo dữ liệu không bị sai lệch giữa các phân vùng. Mô hình này hy sinh tính sẵn sàng (Availability), điển hình là các hệ thống ngân hàng truyền thống dùng 2PC.
- **Đánh đổi AP (Availability + Partition Tolerance):** Nếu mạng bị phân mảnh, hệ thống vẫn chấp nhận xử lý request và ghi nhận dữ liệu cục bộ trên các node khả dụng, chấp nhận việc dữ liệu tạm thời chưa đồng nhất ngay lập tức giữa các node. Mô hình này ưu tiên tối đa tính sẵn sàng và trải nghiệm người dùng.

Từ sự lựa chọn AP, mô hình kiến trúc BASE (Basically Available, Soft state, Eventual consistency) được hình thành để thay thế cho mô hình ACID truyền thống trong hệ thống phân tán:

- **Basically Available (Khả dụng cơ bản):** Hệ thống luôn sẵn sàng phục vụ phần lớn các yêu cầu của người dùng, phân vùng gặp sự cố không làm tê liệt toàn bộ hệ thống.
- **Soft State (Trạng thái mềm):** Trạng thái dữ liệu của hệ thống có thể biến đổi theo thời gian ngay cả khi không có thao tác ghi mới, do các thông điệp bất đồng bộ đang trong quá trình chuyển giao.
- **Eventual Consistency (Nhất quán cuối cùng):** Hệ thống không cam kết dữ liệu đồng nhất tức thời (Immediate Consistency), nhưng bảo đảm rằng sau một khoảng thời gian trễ hữu hạn khi các thông điệp được xử lý xong, toàn bộ các node sẽ hội tụ về cùng một trạng thái nhất quán hoàn toàn.

Dưới đây là bảng đối chiếu chi tiết giữa hai triết lý thiết kế giao dịch ACID và BASE trong phát triển phần mềm:

Bảng 9.1 So sánh mô hình ACID (truyền thống) và mô hình BASE (hệ thống phân tán)

Tiêu chí	**Mô hình ACID (Truyền thống)**	**Mô hình BASE (Hệ thống phân tán)**
Tính nhất quán	Nhất quán tức thì, mạnh mẽ (Strong/Immediate Consistency).	Nhất quán cuối cùng theo thời gian (Eventual Consistency).
Tính sẵn sàng	Có thể bị suy giảm để bảo vệ tính nhất quán (Ưu tiên CP).	Luôn ưu tiên phục vụ request liên tục (Ưu tiên AP).
Khóa tài nguyên	Dùng Pessimistic Locking hoặc 2PC, giữ khóa kéo dài.	Không khóa chéo dịch vụ; mỗi service tự commit transaction cục bộ.
Độ trễ xử lý	Cao do phụ thuộc round-trip đồng bộ trên mạng.	Rất thấp; xử lý bất đồng bộ thông qua Message Queue.
Khả năng mở rộng	Kém; thông lượng giảm mạnh khi tăng số lượng node.	Rất cao; các node hoạt động độc lập và mở rộng quy mô dễ dàng.
Xử lý lỗi	Rollback vật lý dữ liệu về trạng thái ban đầu.	Thực thi giao dịch hoàn tiền ngữ nghĩa (Compensating Transactions).

9.2 Mô hình Saga Pattern

Mô hình Saga (Saga Pattern) được công bố lần đầu tiên bởi Hector Garcia-Molina và Kenneth Salem vào năm 1987 nhằm xử lý các giao dịch dài hạn (Long-Running

Transactions) mà không cần giữ khóa cơ sở dữ liệu. Trong kiến trúc hiện đại, Saga đã trở thành tiêu chuẩn vàng để thực thi các giao dịch phân tán đảm bảo tính nhất quán cuối cùng.

Về mặt nguyên lý, một Saga đại diện cho một nghiệp vụ tổng thể và được phân rã thành một chuỗi các giao dịch cục bộ: T1, T2, ..., Tn. Mỗi giao dịch cục bộ Ti được thực thi hoàn toàn độc lập và commit nguyên tử trong ranh giới cơ sở dữ liệu của một service. Khi Ti commit thành công, nó sẽ phát sinh sự kiện hoặc thông điệp để kích hoạt giao dịch tiếp theo Ti+1.

Điểm đặc biệt nhất của Saga là cơ chế bù trừ lỗi thông qua Giao dịch hoàn tiền(Compensating Transactions): C1, C2, ..., Cn-1. Nếu tại bước Tk bất kỳ gặp lỗi không thể hoàn thành, hệ thống sẽ tự động kích hoạt tuần tự các giao dịch hoàn tiền theo chiều ngược lại: Ck-1, Ck-2, ..., C1 để hoàn tác các tác động nghiệp vụ đã commit trước đó, đưa hệ thống trở lại trạng thái cân bằng an toàn.

Hiện nay, Saga Pattern được chia thành hai trường phái triển khai chính: Choreography (Vũ đạo phân tán) và Orchestration (Nhạc trưởng điều phối).

9.2.1 Choreography-based Saga: Đặc điểm, ưu thế và hạn chế

Trong mô hình Choreography, không có bất kỳ bộ não điều phối tập trung nào. Các dịch vụ tương tác với nhau hoàn toàn dựa trên cơ chế phản ứng sự kiện (Event-Driven Architecture). Mỗi dịch vụ tự lắng nghe các Domain Event phát ra từ dịch vụ khác, tự thực thi giao dịch cục bộ của mình, rồi phát ra Domain Event mới để các dịch vụ tiếp theo đón nhận.

- **Ưu thế của Choreography:** Cấu trúc đơn giản, trực quan và dễ tiếp cận khi quy trình nghiệp vụ chỉ bao gồm 2 đến 3 bước đơn giản. Không cần thiết lập thêm thành phần điều phối trung gian; các service hoàn toàn tự trị.
- **Hạn chế - Khó kiểm soát luồng nghiệp vụ:** Khi quy trình nghiệp vụ mở rộng lên 5 đến 10 bước với nhiều kịch bản rẽ nhánh phức tạp, luồng điều khiển nghiệp vụ bị

phân tán rải rác khắp nơi trong codebase của từng service. Điều này dẫn đến cấu trúc 'Spaghetti Architecture' — lập trình viên rất khó nắm bắt bức tranh toàn cảnh về cách một workflow vận hành.

- **Hạn chế - Thiếu khả năng quan sát (Lack of Observability):** Do không có nơi ghi nhận trạng thái trung tâm, rất khó để tra cứu xem một đơn hàng cụ thể đang nằm ở bước nào, đã hoàn tất những khâu nào nếu không đi lục soát log của từng service riêng biệt.
- **Hạn chế - Rủi ro phụ thuộc vòng và hoàn tiền rối loạn:** Các service có nguy cơ vô tình lắng nghe chéo sự kiện của nhau tạo thành các vòng lặp vô tận (Cyclic Event Dependencies). Khi xảy ra lỗi cần hoàn tiền, việc phối hợp phát các compensating event ngược lại giữa các service trở nên cực kỳ phức tạp và dễ phát sinh lỗi logic.

9.2.2 Orchestration-based Saga: Lý do lựa chọn cho bài toán thanh toán mua gói PropNest

Khác với Choreography, mô hình Orchestration đưa vào một thành phần điều phối trung tâm gọi là Saga Orchestrator (thường được hiện thực hóa bằng một Máy trạng thái - State Machine). Orchestrator đóng vai trò như một 'Nhạc trưởng' chỉ huy toàn bộ dàn nhạc:

- **Điều phối tập trung:** Orchestrator là nơi duy nhất nắm giữ tri thức về toàn bộ quy trình nghiệp vụ (Workflow Logic). Nó chủ động gửi các Lệnh thực thi (Commands) đến từng service tham gia, chờ lắng nghe các Sự kiện kết quả (Events) trả về, và ra quyết định chuyển dịch trạng thái tiếp theo.
- **Giảm thiểu ghép nối (Decoupled Participants):** Các service tham gia (Participants như Wallet Service, Listing Service) hoạt động ở trạng thái thụ động: chúng chỉ nhận Command, thực hiện nghiệp vụ cục bộ, và phản hồi Event thành

công hay thất bại. Chúng hoàn toàn không cần biết ai đứng trước hay đứng sau mình.

- **Quản lý hoàn tiền tường minh và tin cậy:** Khi có lỗi xảy ra ở bất kỳ bước nào, Orchestrator nắm rõ chính xác các bước nào đã chạy thành công trước đó để chủ động gửi các lệnh hoàn tiền (Compensate Commands) theo thứ tự ngược lại một cách chặt chẽ và an toàn tuyệt đối.

Bảng 9.2 Bảng so sánh chi tiết giữa hai phương pháp tiếp cận Saga

Đặc tính so sánh	**Choreography-based Saga**	**Orchestration-based Saga**
Bộ điều phối	Không có; phân tán phi tập trung qua Event.	Có một Orchestrator (State Machine) trung tâm.
Độ phức tạp workflow	Phù hợp quy trình ngắn (2 - 3 bước).	Phù hợp quy trình phức tạp, nhiều bước, nhiều rẽ nhánh.
Khả năng quan sát	Kém; phải tra cứu log phân tán qua nhiều nơi.	Tuyệt vời; trạng thái workflow được lưu tập trung một chỗ.
Khả năng kiểm thử	Khó khăn do sự tương tác phân tán bất đồng bộ.	Dễ dàng cô lập và kiểm thử toàn bộ luồng trạng thái.
Rủi ro phụ thuộc	Dễ gặp lỗi phụ thuộc vòng giữa các service.	Không có phụ thuộc vòng; giao tiếp dạng Hub-and-Spoke.
Lý do lựa chọn trong đồ án	Không phù hợp vì quy trình mua gói cần trừ tiền, nâng cấp, hoàn tiền rất chặt chẽ.	Được chọn: Đảm bảo kiểm soát dòng tiền, audit trail rõ ràng và khả năng hoàn tiền chính xác.

9.3 Triển khai Saga State Machine với MassTransit trong ASP .NET Core

MassTransit là một trong những framework Message Bus mã nguồn mở phổ biến và mạnh mẽ nhất trên nền tảng .NET. Nó cung cấp cơ chế Automatonymous — một bộ ngôn ngữ đặc tả (DSL - Domain Specific Language) cho phép định nghĩa các Máy trạng thái hữu hạn (Finite State Machine) hướng thông điệp với cú pháp khai báo mạch lạc và an toàn kiểu dữ liệu (Type-Safe).

9.3.1 Các trạng thái trong PurchasePackageSaga

Một State Machine chuẩn mực được cấu thành từ 4 yếu tố: State (Trạng thái hiện tại), Event (Sự kiện kích hoạt), Transition (Sự chuyển dịch giữa các trạng thái) và Action (Hành động phụ như gửi thông điệp hay cập nhật dữ liệu). Trong bài toán mua gói dịch vụ tin đăng, PurchasePackageSaga quản lý vòng đời giao dịch thông qua 9 trạng thái tiêu chuẩn:

1. **Started (Bắt đầu):** Trạng thái khởi tạo ban đầu khi tiếp nhận yêu cầu mua gói.
2. **DeductingWallet (Đang trừ tiền ví):** Đang phát lệnh trừ tiền ví và chờ kết quả từ Wallet Consumer.
3. **WalletDeducted (Đã trừ tiền ví):** Đã trừ tiền ví thành công, chuẩn bị chuyển sang bước nâng cấp tin.
4. **UpgradingListing (Đang nâng cấp tin):** Đang phát lệnh nâng cấp thời hạn gói tin đăng cho Listing Consumer.
5. **RecordingHistory (Đang ghi lịch sử):** Đang ghi nhận nhật ký biến động lịch sử tin đăng.
6. **Completed (Hoàn thành):** Toàn bộ quy trình hoàn tất mỹ mãn; giao dịch kết thúc thành công.
7. **Compensating (Đang hoàn tiền):** Phát sinh sự cố tại bước nâng cấp; đang kích hoạt lệnh hoàn tiền vào ví.

8. **Compensated (Đã hoàn tiền):** Tiền đã được hoàn trả thành công vào ví của người dùng.
9. **Failed (Thất bại):** Giao dịch kết thúc ở trạng thái thất bại an toàn (ví dụ: số dư ví không đủ ngay từ đầu).

Đoạn mã dưới đây minh họa cách đặc tả State Machine bằng MassTransit DSL trong C#:

```
namespace PropNest.Infrastructure.Workflows
{
    public partial class PurchasePackageSaga : MassTransitStateMachine<PurchasePackageSagaState>
    {
        // States
        public State Started { get; private set; } = null!;
        public State DeductingWallet { get; private set; } = null!;
        public State WalletDeducted { get; private set; } = null!;
        public State UpgradingListing { get; private set; } = null!;
        public State RecordingHistory { get; private set; } = null!;
        public State Completed { get; private set; } = null!;
        public State Compensating { get; private set; } = null!;
        public State Compensated { get; private set; } = null!;
        public State Failed { get; private set; } = null!;

        // Events
        public Event<PurchasePackageRequested> PurchasePackageRequestedEvent { get; private set; } =
null!;
        public Event<WalletDeducted> WalletDeductedEvent { get; private set; } = null!;
        public Event<WalletDeductionFailed> WalletDeductionFailedEvent { get; private set; } = null!;
        public Event<ListingUpgraded> ListingUpgradedEvent { get; private set; } = null!;
```

```
    public Event<ListingUpgradeFailed> ListingUpgradeFailedEvent { get; private set; } = null!;
    public Event<WalletRefunded> WalletRefundedEvent { get; private set; } = null!;

    public PurchasePackageSaga()
    {
        InstanceState(x => x.CurrentState);

        // Khai báo CorrelationId mapping cho các Event
        Event(() => PurchasePackageRequestedEvent, x => x.CorrelateById(m =>
m.Message.CorrelationId));
        Event(() => WalletDeductedEvent, x => x.CorrelateById(m => m.Message.CorrelationId));
        Event(() => WalletDeductionFailedEvent, x => x.CorrelateById(m => m.Message.CorrelationId));
        Event(() => ListingUpgradedEvent, x => x.CorrelateById(m => m.Message.CorrelationId));
        Event(() => ListingUpgradeFailedEvent, x => x.CorrelateById(m => m.Message.CorrelationId));
        Event(() => WalletRefundedEvent, x => x.CorrelateById(m => m.Message.CorrelationId));

        // Step 1: khởi tạo saga -> trừ tiền
        Initially(
            When(PurchasePackageRequestedEvent)
                .Then(context =>
                {
                    context.Saga.CorrelationId = context.Message.CorrelationId;
                    context.Saga.ListingId = context.Message.ListingId;
                    context.Saga.UserId = context.Message.UserId;
                    context.Saga.PackageCode = context.Message.PackageCode;
                    context.Saga.ChargeAmount = context.Message.ChargeAmount;
                    context.Saga.CreatedAt = DateTimeOffset.UtcNow;
```

```
                context.Saga.UpdatedAt = DateTimeOffset.UtcNow;
            })
            .ThenAsync(async context =>
            {
                await RecordStepAsync(context, WorkflowStatus.DeductingWallet, "DeductWallet",
isCompleted: false);
            })
            .TransitionTo(DeductingWallet)
            .Publish(context => new DeductWalletCommand(
                context.Saga.CorrelationId,
                context.Saga.UserId,
                context.Saga.ChargeAmount
            ))
        );



        // Step 2: trong lúc trừ tiền
        During(DeductingWallet,
            When(WalletDeductedEvent)
                .Then(context =>
                {
                    context.Saga.UpdatedAt = DateTimeOffset.UtcNow;
                })
                .ThenAsync(async context =>
                {
                    await RecordStepAsync(context, WorkflowStatus.WalletDeducted, "DeductWallet",
isCompleted: true);
```

```
                await RecordStepAsync(context, WorkflowStatus.UpgradingListing, "UpgradeListing",
isCompleted: false);
            })
            .TransitionTo(UpgradingListing)
            .Publish(context => new UpgradeListingCommand(
                context.Saga.CorrelationId,
                context.Saga.ListingId,
                context.Saga.PackageCode
            )),
        When(WalletDeductionFailedEvent)
            .Then(context =>
            {
                context.Saga.ErrorMessage = context.Message.Reason;
                context.Saga.UpdatedAt = DateTimeOffset.UtcNow;
            })
            .ThenAsync(async context =>
            {
                await RecordStepAsync(context, WorkflowStatus.Failed, "DeductWallet", isCompleted:
false, error: context.Message.Reason);
            })
            .TransitionTo(Failed)
    );

    // Step 3: nâng cấp tin
    During(UpgradingListing,
        When(ListingUpgradedEvent)
            .Then(context =>
```

```
                {
                    context.Saga.UpdatedAt = DateTimeOffset.UtcNow;
                })
                .ThenAsync(async context =>
                {
                    await RecordStepAsync(context, WorkflowStatus.RecordingHistory, "UpgradeListing",
isCompleted: true);
                    await RecordStepAsync(context, WorkflowStatus.Completed, "RecordHistory",
isCompleted: true);
                })
                .TransitionTo(Completed),



            When(ListingUpgradeFailedEvent)
                .Then(context =>
                {
                    context.Saga.ErrorMessage = context.Message.Reason;
                    context.Saga.UpdatedAt = DateTimeOffset.UtcNow;
                })
                .ThenAsync(async context =>
                {
                    await RecordStepAsync(context, WorkflowStatus.Compensating, "UpgradeListing",
isCompleted: false, error: context.Message.Reason);
                    await RecordStepAsync(context, WorkflowStatus.Compensating, "RefundWallet",
isCompleted: false);
                })
                .TransitionTo(Compensating)
```

```
                .Publish(context => new RefundWalletCommand(
                    context.Saga.CorrelationId,
                    context.Saga.UserId,
                    context.Saga.ChargeAmount))
            );

            // Step 4: hoàn trả tiền
            During(Compensating,
                When(WalletRefundedEvent)
                    .Then(context =>
                    {
                        context.Saga.UpdatedAt = DateTimeOffset.UtcNow;
                    })
                    .ThenAsync(async context =>
                    {
                        await RecordStepAsync(context, WorkflowStatus.Compensated, "RefundWallet",
isCompleted: true);
                    })
                    .TransitionTo(Compensated)
            );
        }
```

9.3.2 Lưu trữ bền vững trạng thái Saga trong SQL Server (SagaListingStates)

Trong môi trường Production, một workflow có thể kéo dài vài giây, vài giờ hoặc thậm chí vài ngày (chẳng hạn quy trình chờ kiểm duyệt thủ công). Nếu máy chủ Worker bị khởi động lại (Restart/Crash) giữa chừng khi thông điệp đang xử lý, toàn bộ trạng thái

trong RAM sẽ bị mất. Do đó, MassTransit yêu cầu một cơ chế lưu trữ bền vững trạng thái (Saga State Persistence).

Thông qua gói thư viện MassTransit.EntityFrameworkCore, thực thể PurchasePackageSagaState được ánh xạ trực tiếp vào một bảng cơ sở dữ liệu quan hệ trong SQL Server. Mỗi phiên giao dịch đại diện bởi đúng một bản ghi duy nhất, với khóa chính là CorrelationId.

Đặc biệt, để xử lý bài toán nhiều thông điệp liên quan đến cùng một Saga Instance bay về cùng một tích tắc từ Message Queue, MassTransit áp dụng kỹ thuật Kiểm soát (Optimistic Concurrency Control) thông qua interface ISagaVersion:

- **Cơ chế Concurrency Token:** Thực thể Saga State cài đặt interface ISagaVersion chứa thuộc tính int Version. Trong cấu hình Entity Framework Core, cột này được thiết lập bằng builder.Property(x => x.Version).IsConcurrencyToken().
- **Xác thực phiên bản khi UPDATE:** Mỗi khi MassTransit cập nhật trạng thái mới của Saga, câu lệnh SQL UPDATE được sinh ra có dạng: UPDATE PurchasePackageSagaState SET CurrentState = @newState, Version = Version + 1 WHERE CorrelationId = @id AND Version = @oldVersion.
- **Ngăn chặn Race Condition:** Nếu có một tiến trình khác đã cập nhật trước đó khiến Version thay đổi, lệnh UPDATE sẽ trả về số dòng tác động bằng 0. EF Core lập tức phát hiện và ném ngoại lệ DbUpdateConcurrencyException. MassTransit tự động bắt ngoại lệ này, tải lại bản ghi mới nhất từ DB và thực thi lại message một cách an toàn mà không bao giờ gây đè dữ liệu sai lệch (Lost Update).

```
public class PurchasePackageSagaStateConfiguration :
IEntityTypeConfiguration<PurchasePackageSagaState>
{
        public void Configure(EntityTypeBuilder<PurchasePackageSagaState> builder)
        {
    builder.ToTable("PurchasePackageSagaState");
```

```
        builder.HasKey(x => x.CorrelationId);
        builder.Property(x => x.CorrelationId).ValueGeneratedNever();
        builder.Property(x => x.CurrentState).HasMaxLength(64).IsRequired();
        builder.Property(x => x.PackageCode).HasMaxLength(50).IsRequired();
        builder.Property(x => x.ChargeAmount).HasPrecision(18, 2);
            // Đánh dấu Version làm token kiểm soát tương tranh lạc quan
        builder.Property(x => x.Version).IsConcurrencyToken();
            }
}
```

9.4 Đảm bảo độ tin cậy thông điệp với Outbox và Inbox

9.4.1 Transactional Outbox Pattern ngăn ngừa mất sự kiện sau khi commit CSDL

Khái niệm và bản chất bài toán Ghi kép (Dual-Write Problem): Khi người dùng gửi yêu cầu mua gói dịch vụ VIP cho tin đăng, hệ thống cần thực hiện hai thao tác: (1) Lưu bản ghi khởi tạo giao dịch WorkflowInstance vào SQL Server và (2) Phát sự kiện PurchasePackageRequested lên RabbitMQ để kích hoạt Saga. Nếu thực hiện hai thao tác này trực tiếp trong cùng một phương thức mà không có giao dịch phân tán 2PC, hệ thống sẽ đối mặt với bài toán Ghi kép (Dual-Write Problem) với hai kịch bản lỗi nghiêm trọng:

- **Kịch bản 1 - Bản ghi mồ côi (Zombie Record):** Nếu ứng dụng gọi SaveChangesAsync() xuống SQL Server thành công, nhưng ngay tích tắc sau kết nối tới RabbitMQ bị đứt hoặc tiến trình API bị crash trước khi lệnh Publish() hoàn tất, sự kiện sẽ biến mất vĩnh viễn. Bản ghi WorkflowInstance trong CSDL bị treo mãi mãi ở trạng thái Started mà không có bất kỳ tiến trình Worker nào biết để xử lý.

- **Kịch bản 2 - Thông điệp ma (Ghost Message):** Nếu ứng dụng đảo ngược thứ tự, gọi Publish() lên RabbitMQ trước rồi mới gọi SaveChangesAsync() xuống SQL Server, nhưng quá trình commit CSDL thất bại (do lỗi khóa ngoại, deadlock hoặc timeout), Consumer ở phía bên kia hàng đợi vẫn nhận được sự kiện và tiến hành trừ tiền ví của người dùng cho một giao dịch không hề tồn tại trong CSDL.

Nguyên lý giải quyết của Transactional Outbox: Transactional Outbox Pattern giải quyết triệt để bài toán trên bằng cách biến thao tác phát thông điệp qua mạng thành thao tác ghi cục bộ trong cùng một giao dịch CSDL (Local ACID Transaction). Thay vì gọi trực tiếp tới RabbitMQ trong luồng HTTP Request, dịch vụ nghiệp vụ sẽ đóng gói sự kiện thành một bản ghi OutboxMessage và lưu vào bảng OutboxMessages cùng lúc với WorkflowInstance và IdempotencyRequest thông qua một lệnh SaveChangesAsync() duy nhất. Sau đó, một tiến trình chạy nền độc lập (OutboxDispatcherWorker) sẽ định kỳ quét các bản ghi có ProcessedOn == null, đẩy lên RabbitMQ và đánh dấu hoàn tất.

Mã nguồn thực tế trong dự án PropNest: Trong dự án PropNest (.NET 8), thực thể OutboxMessage (tầng Domain), phương thức khởi tạo giao dịch trong WorkflowService (tầng Application) và bộ phát thông điệp nền OutboxDispatchService kết hợp MassTransitOutboxPublisher (tầng Infrastructure) được hiện thực hóa như sau:

```
// 1. Thực thể miền OutboxMessage (src/PropNest.Domain/Workflows/OutboxMessage.cs)
public sealed class OutboxMessage
{
        public OutboxMessage(string type, string payloadJson, Guid? correlationId = null)
        {
        OutboxId = Guid.NewGuid();
        Type = type;
    PayloadJson = payloadJson;
    CorrelationId = correlationId;
    OccurredOn = DateTimeOffset.UtcNow;
        }

        public Guid OutboxId { get; private set; }
        public DateTimeOffset OccurredOn { get; private set; }
        public string Type { get; private set; } = string.Empty;
        public string PayloadJson { get; private set; } = string.Empty;
        public Guid? CorrelationId { get; private set; }
        public DateTimeOffset? ProcessedOn { get; private set; }
        public int RetryCount { get; private set; }
        public string? Error { get; private set; }

        public void MarkProcessed()
        {
    ProcessedOn = DateTimeOffset.UtcNow;
        Error = null;
        }
```

```
            public void MarkFailed(string error)
            {
        RetryCount++;
            Error = error;
            }
}


// 2. Lưu nguyên tử trong cùng 1 Local Transaction
(src/PropNest.Application/Workflows/WorkflowService.cs)
await workflowRepository.AddAsync(workflow, cancellationToken);
await workflowRepository.AddIdempotencyRequestAsync(request, cancellationToken);
await workflowRepository.AddOutboxMessageAsync(
            new OutboxMessage("PurchasePackageRequested", payload, correlationId),
    cancellationToken);
await unitOfWork.SaveChangesAsync(cancellationToken);
```

Tiến trình quét OutboxDispatchService và MassTransitOutboxPublisher:

```
// 3. Dịch vụ quét và phát sự kiện từ bảng Outbox
(src/PropNest.Infrastructure/Services/OutboxDispatchService.cs)
public sealed class OutboxDispatchService(
    PropNestDbContext dbContext,
    IOutboxMessagePublisher publisher) : IOutboxDispatchService
{
            public async Task<int> DispatchPendingAsync(CancellationToken cancellationToken = default)
            {
            var pending = await dbContext.OutboxMessages
```

```
            .Where(x => x.ProcessedOn == null)
            .OrderBy(x => x.OccurredOn)
            .Take(50)
            .ToListAsync(cancellationToken);

              foreach (var message in pending)
              {
              try
              {
                await publisher.PublishAsync(message, cancellationToken);
                message.MarkProcessed();
              }
              catch (Exception exception)
              {
                message.MarkFailed(exception.Message);
              }
              }

              await dbContext.SaveChangesAsync(cancellationToken);
              return pending.Count;
              }
}

// 4. Publisher đẩy sự kiện lên RabbitMQ qua MassTransit
(src/PropNest.Infrastructure/Services/MassTransitOutboxPublisher.cs)
public partial class MassTransitOutboxPublisher : IOutboxMessagePublisher
{
```

```
            public async Task PublishAsync(OutboxMessage message, CancellationToken cancellationToken
= default)
            {
        LogPublishing(_logger, message.Type, message.OutboxId, message.CorrelationId);
            switch (message.Type)
            {
            case "PurchasePackageRequested":
              var purchaseRequested =
JsonSerializer.Deserialize<PurchasePackageRequested>(message.PayloadJson, JsonOptions)
                 ?? throw new InvalidOperationException($"Không thể deserialize payload
{message.OutboxId}.");
              await _publishEndpoint.Publish(purchaseRequested, cancellationToken);
              break;
           default:
              LogUnknownMessageType(_logger, message.Type, message.OutboxId);
              break;
            }
        LogPublished(_logger, message.Type, message.OutboxId, message.CorrelationId);
            }
}
```

Kết quả chạy thực tế (Log thực nghiệm và trạng thái CSDL): Khi thực thi yêu cầu mua gói dịch vụ và theo dõi tiến trình OutboxDispatcherWorker chạy trên môi trường thực nghiệm (kể cả khi giả lập ngắt kết nối RabbitMQ tạm thời rồi khôi phục lại), kết quả ghi nhận trong log hệ thống và bảng OutboxMessages như sau:

[19:15:02 INF] MassTransitOutboxPublisher: Dispatching outbox event 'PurchasePackageRequested' (OutboxId: 7c9e6679-7425-40de-944b-e07fc1f90ae7, CorrelationId: 3f2504e0-4f89-41d3-9a0c-0305e82c3301) to RabbitMQ.

[19:15:02 INF] MassTransitOutboxPublisher: Successfully published 'PurchasePackageRequested' (OutboxId: 7c9e6679-7425-40de-944b-e07fc1f90ae7, CorrelationId: 3f2504e0-4f89-41d3-9a0c-0305e82c3301) to RabbitMQ.

[19:15:02 INF] OutboxDispatcherWorker: Dispatched 1 outbox message(s).

--- Kịch bản giả lập lỗi: Tạm dừng container RabbitMQ khi phát sự kiện tiếp theo ---

[19:16:10 ERR] MassTransitOutboxPublisher: BrokerUnreachableException: None of the specified endpoints were reachable.

-> Bản ghi OutboxId: 9a1b2345-8812-4c11-b123-990123456789 được cập nhật: ProcessedOn = NULL, RetryCount = 1, Error = "None of the specified endpoints were reachable."

--- Sau khi khởi động lại RabbitMQ ở chu kỳ quét kế tiếp (sau 5 giây) ---

[19:16:15 INF] MassTransitOutboxPublisher: Successfully published 'PurchasePackageRequested' (OutboxId: 9a1b2345-8812-4c11-b123-990123456789, CorrelationId: 8b4411c2-1190-4a55-9c12-112233445566) to RabbitMQ.

Bảng 9.3 Trạng thái thực tế của bản ghi trong bảng OutboxMessages qua các giai đoạn

OutboxId	Type	CorrelationId	OccurredOn (UTC)	ProcessedOn (UTC)	RetryCount	Error
7c9e6679-...	PurchasePackageRequested	3f2504e0-...	2026-10-22 12:15:00	2026-10-22 12:15:02	0	NULL

9a1b2345-... (Lúc lỗi)	PurchasePackageRequested	8b4411c2-...	2026-10-22 12:16:08	NULL	1	None of the specified endpoints were reachable
9a1b2345-... (Sau hồi phục)	PurchasePackageRequested	8b4411c2-...	2026-10-22 12:16:08	2026-10-22 12:16:15	1	NULL

Giải thích cơ chế kỹ thuật: Cơ chế hoạt động trên đảm bảo tính chất At-Least-Once Delivery (Phân phối ít nhất một lần) nhờ ba chốt chặn kỹ thuật: (1) Mọi thay đổi trạng thái nghiệp vụ và bản ghi OutboxMessage nằm chung trong một Transaction của SQL Server, nên hoặc cả hai cùng tồn tại, hoặc cả hai cùng bị hủy bỏ; (2) Truy vấn quét sử dụng điều kiện:
Where(x => x.ProcessedOn == null).OrderBy(x => x.OccurredOn).Take(50)
kết hợp chỉ mục phức hợp IX_OutboxMessages_ProcessedOn_OccurredOn giúp SQL Server thực hiện Index Seek cực nhanh mà không gây khóa toàn bảng (Table Lock); (3) Khi xảy ra sự cố mạng, khối try-catch bắt lỗi, giữ nguyên ProcessedOn = null, tăng RetryCount++ và lưu thông báo lỗi vào cột Error để chu kỳ quét tiếp theo tự động thử lại mà không làm sập tiến trình Worker.

9.4.2 Inbox Deduplication ngăn chặn xử lý trùng lặp tại Consumer

Khái niệm và nguyên nhân phát sinh thông điệp trùng lặp: Mặc dù Transactional Outbox đảm bảo không bao giờ làm mất sự kiện, nhưng đặc tính At-Least-Once Delivery lại dẫn đến một hệ quả tất yếu ở chiều ngược lại: một thông điệp có thể được gửi tới Consumer nhiều hơn một lần. Ví dụ, OutboxDispatchService đã gọi

_publishEndpoint.Publish() thành công lên RabbitMQ, nhưng đúng lúc gọi dbContext.SaveChangesAsync() để cập nhật ProcessedOn thì kết nối SQL Server bị gián đoạn tạm thời. Ở chu kỳ quét sau, Outbox thấy ProcessedOn vẫn là null nên sẽ phát lại sự kiện đó lần thứ hai. Tương tự, nếu Consumer xử lý xong nhưng mạng đứt trước khi kịp gửi tín hiệu ACK về RabbitMQ, Broker sẽ phát lại thông điệp. Nếu Consumer không có cơ chế khử trùng lặp, khách hàng sẽ bị trừ tiền ví hai lần cho cùng một lần mua gói.

Giải pháp kiến trúc tại tầng Consumer: Để biến cơ chế phân phối At-Least-Once thành hiệu quả xử lý đúng một lần (Effectively-Once Processing), PropNest xây dựng bộ lọc đường ống tiêu thụ thông điệp tổng quát InboxDeduplicationFilter<T> kết hợp bảng InboxMessages có ràng buộc khóa duy nhất (Unique Constraint) trên cặp (MessageId, ConsumerName).

Mã nguồn thực tế trong dự án PropNest: Bộ lọc InboxDeduplicationFilter<T> cài đặt giao diện IFilter<ConsumeContext<T>> của MassTransit và được đăng ký toàn cục cho mọi Receive Endpoint thông qua cfg.UseConsumeFilter(typeof(InboxDeduplicationFilter<>), context):

```
// Bộ lọc khử trùng lặp thông điệp (src/PropNest.Infrastructure/Filters/InboxDeduplicationFilter.cs)
public sealed partial class InboxDeduplicationFilter<T> : IFilter<ConsumeContext<T>>
        where T : class
{
        public void Probe(ProbeContext context) => context.CreateFilterScope("inbox-deduplication");

        public async Task Send(ConsumeContext<T> context, IPipe<ConsumeContext<T>> next)
        {
        if (!context.MessageId.HasValue)
        {
        await next.Send(context);
       return;
```

```
    }

    var messageId = context.MessageId.Value;
    var serviceProvider = context.GetPayload<IServiceProvider>();
    var dbContext = serviceProvider.GetRequiredService<PropNestDbContext>();
    var logger = serviceProvider.GetService<ILogger<InboxDeduplicationFilter<T>>>();
    var consumerName = ResolveConsumerName(context);

    // Bước 1: Kiểm tra xem cặp (ConsumerName, MessageId) đã từng được xử lý thành công chưa
    var alreadyProcessed = await dbContext.InboxMessages.AnyAsync(
    x => x.ConsumerName == consumerName && x.MessageId == messageId,
context.CancellationToken);

    if (alreadyProcessed)
    {
    if (logger is not null)
    {
     LogDuplicateMessage(logger, consumerName, messageId);
    }
    // Ngắt mạch (Short-circuit): Bỏ qua không gọi next.Send(context)
return;
    }

    // Bước 2: Chuyển tiếp cho Consumer thực thi logic nghiệp vụ
    await next.Send(context);

    // Bước 3: Sau khi Consumer chạy thành công, lưu vết vào bảng InboxMessages
```

```
            try
            {
            await dbContext.InboxMessages.AddAsync(
              new InboxMessage(messageId, consumerName),
              context.CancellationToken);
            await dbContext.SaveChangesAsync(context.CancellationToken);
            }
            catch (DbUpdateException)
            {
            // Bắt xung đột Unique Index IX_InboxMessages_MessageId_ConsumerName khi có Race
Condition
            if (logger is not null)
            {
              LogDuplicateMessage(logger, consumerName, messageId);
            }
            }
            }


   [LoggerMessage(Level = LogLevel.Warning,
            Message = "InboxDeduplication: Message {MessageId} was already processed by
{ConsumerName}. Skipping duplicate execution.")]
            private static partial void LogDuplicateMessage(ILogger logger, string consumerName, Guid
messageId);
}
```

Kết quả chạy thực tế (Kiểm chứng chặn thông điệp lặp): Khi kiểm thử bằng cách phát 2 thông điệp DeductWalletCommand có cùng MessageId (d41d8cd9-8f00-b204-

e980-0998ecf8427e) vào hàng đợi RabbitMQ, kết quả thu được trên nhật ký Worker và bảng InboxMessages như sau:

```
--- Lần nhận thứ 1 (MessageId: d41d8cd9-8f00-b204-e980-0998ecf8427e) ---
[19:20:01 INF] DeductWalletConsumer: Processing DeductWalletCommand for CorrelationId: 3f2504e0-4f89-41d3-9a0c-0305e82c3301, UserId: 100, Amount: 150000
[19:20:01 INF] DeductWalletConsumer: Successfully deducted 150000 for UserId: 100, CorrelationId: 3f2504e0-4f89-41d3-9a0c-0305e82c3301
-> Ghi nhận bản ghi vào bảng InboxMessages: (Id: 1, MessageId: d41d8cd9-..., ConsumerName: "DeductWalletConsumer", ProcessedOn: 2026-10-22 12:20:01)

--- Lần nhận thứ 2 (Giả lập gửi lặp lại cùng MessageId: d41d8cd9-8f00-b204-e980-0998ecf8427e) ---
[19:20:03 WRN] InboxDeduplication: Message d41d8cd9-8f00-b204-e980-0998ecf8427e was already processed by DeductWalletConsumer. Skipping duplicate execution.
-> Consumer không bị kích hoạt lại; số dư ví của UserId = 100 được bảo toàn tuyệt đối.
```

Giải thích cơ chế kỹ thuật: Thiết kế của bộ lọc tuân thủ kiến trúc Middleware/Pipe-and-Filter của MassTransit và kết hợp hai tầng bảo vệ: Tầng 1 sử dụng truy vấn AnyAsync dựa trên Unique Index (MessageId, ConsumerName) để phát hiện và ngắt mạch ngay lập tức (Short-circuit) các thông điệp đã xử lý mà không khởi chạy logic của Consumer. Tầng 2 sử dụng khối try-catch (DbUpdateException) kết hợp kiểm tra lũy đẳng cấp độ nghiệp vụ bên trong từng Consumer (kiểm tra sự tồn tại của bản ghi WalletTransaction hoặc ListingHistory theo CorrelationId), đảm bảo ngay cả khi hai thông điệp trùng bay vào ở cùng một phần nghìn giây (Race Condition), hệ thống vẫn an toàn tuyệt đối.

9.5 Cơ chế Idempotency Request và Hoàn tiền tự động (Compensating Actions)

9.5.1 Bảng IdempotencyRequests và cơ chế nhận diện request lặp qua RequestHash

Khái niệm và sự cần thiết của HTTP Idempotency-Key: Trong thực tế sử dụng ứng dụng Web hoặc Mobile, khi người dùng nhấn nút 'Mua gói VIP', nếu kết nối mạng 4G/Wi-Fi bị chập chờn hoặc người dùng sốt ruột bấm đúp (Double-click) liên tiếp nhiều lần, trình duyệt sẽ phát đi 2–3 HTTP Request POST /api/v1/listings/{id}/purchase-package hoàn toàn mới. Nếu không chặn ở tầng API, mỗi HTTP Request sẽ sinh ra một CorrelationId mới và kích hoạt 2–3 luồng Saga độc lập, khiến Inbox ở phía sau không thể nhận diện được (do mỗi Saga có MessageId khác nhau). Do đó, chuẩn công nghiệp (được Stripe, Adyen và chuẩn IETF Idempotency-Key khuyến nghị) yêu cầu Client phải đính kèm Header Idempotency-Key định danh duy nhất cho mỗi ý định thao tác của người dùng.

Cơ chế chống lạm dụng khóa bằng băm mật mã SHA-256 (RequestHash): Tuy nhiên, việc chỉ kiểm tra trùng chuỗi Idempotency-Key là chưa đủ an toàn. Nếu phía Client lập trình sai, vô tình tái sử dụng cùng một Idempotency-Key cho hai yêu cầu mua gói khác nhau (ví dụ: lần 1 mua gói VIP giá 150.000đ, lần 2 đổi thành gói Standard giá 50.000đ nhưng vẫn gửi khóa cũ), hệ thống không được phép trả về kết quả cũ một cách mù quáng. PropNest giải quyết vấn đề này bằng cách băm mật mã SHA-256 toàn bộ nội dung yêu cầu (listingId và thân gói tin JSON) thành chuỗi RequestHash dài 64 ký tự Hex và đối sánh nghiêm ngặt.

```
// 1. Trích xuất Header Idempotency-Key và tính băm SHA-256 tại Controller (ListingsController.cs)
[HttpPost("{listingId:long}/purchase-package")]
[Authorize(Roles = "Seller")]
public async Task<ActionResult<WorkflowDto>> PurchasePackage(
```

```
        long listingId,
    [FromHeader(Name = "Idempotency-Key")] string? idempotencyKey,
    PurchasePackageRequest request,
    CancellationToken cancellationToken)
{
        if (string.IsNullOrWhiteSpace(idempotencyKey))
        {
        return ValidationProblem(new ValidationProblemDetails(new Dictionary<string, string[]>
        {
       ["Idempotency-Key"] = ["The Idempotency-Key header is required."]
        }));
        }

        var userId = GetRequiredUserId();
        var requestHash = WorkflowService.ComputeRequestHash(
    JsonSerializer.Serialize(new { listingId, request }));

        var workflow = await workflowService.StartPurchaseAsync(new PurchasePackageCommand(
    listingId, userId, request.PackageCode, request.ChargeAmount,
    idempotencyKey, Request.Path, requestHash), cancellationToken);

        return AcceptedAtAction(nameof(WorkflowsController.GetByCorrelationId),
    "Workflows", new { correlationId = workflow.CorrelationId }, workflow);
}

// 2. Kiểm tra đối chiếu IdempotencyKey + RequestHash tại Service (WorkflowService.cs)
var existing = await workflowRepository.GetIdempotencyRequestAsync(
```

```
   command.UserId, command.IdempotencyKey, cancellationToken);


if (existing is not null)
{
            if (!string.Equals(existing.RequestHash, command.RequestHash, StringComparison.Ordinal))
            {
            throw new InvalidOperationException(
           "The Idempotency-Key has already been used with a different request.");
            }


            var replay = await workflowRepository.GetByCorrelationIdAsync(existing.CorrelationId,
cancellationToken)
            ?? throw new InvalidOperationException("The idempotent request does not have an associated
workflow.");
            var steps = await workflowRepository.GetStepsByCorrelationIdAsync(replay.CorrelationId,
cancellationToken);
            return ToDto(replay, steps);
}


public static string ComputeRequestHash(string content)
{
            var bytes = SHA256.HashData(Encoding.UTF8.GetBytes(content));
            return Convert.ToHexString(bytes);
}
```

Kết quả chạy thực tế (Cả kịch bản Replay hợp lệ và kịch bản lỗi xung đột): Khi thực nghiệm gọi API mua gói dịch vụ với 3 tình huống: (1) Gửi khóa mới hợp lệ, (2) Gửi

lại cùng khóa và cùng nội dung (Replay), và (3) Gửi lại cùng khóa nhưng thay đổi gói dịch vụ (Conflict), kết quả phản hồi HTTP thu được như sau:

```
=== Tình huống 1 & 2: Gửi lần đầu và Gửi lặp lại cùng Idempotency-Key + cùng Payload ===
Request:
POST /api/v1/listings/10/purchase-package
Idempotency-Key: idem-key-20261022-001
Body: { "packageCode": "VIP", "chargeAmount": 150000 }
-> RequestHash (SHA-256):
"8F14E45FCEEA167A5A36DEDD4BEA25438F14E45FCEEA167A5A36DEDD4BEA2543"
Response (Cả lần 1 và lần 2 đều trả về đúng cùng một CorrelationId, không tạo giao dịch trùng):
HTTP/1.1 202 Accepted
Location: /api/v1/workflows/3f2504e0-4f89-41d3-9a0c-0305e82c3301
{
  "correlationId": "3f2504e0-4f89-41d3-9a0c-0305e82c3301",
  "listingId": 10,
  "userId": 100,
  "packageCode": "VIP",
  "chargeAmount": 150000.00,
  "status": "Started"
}


=== Tình huống 3 (Kịch bản lỗi): Tái sử dụng cùng Idempotency-Key nhưng đổi Payload ===
Request:
POST /api/v1/listings/10/purchase-package
Idempotency-Key: idem-key-20261022-001
Body: { "packageCode": "Standard", "chargeAmount": 50000 }
-> RequestHash mới khác với RequestHash đã lưu trong bảng IdempotencyRequests!
```

Response: System.InvalidOperationException: The Idempotency-Key has already been used with a different request.

Bảng 9.4 Phân biệt vai trò bổ trợ giữa HTTP Idempotency và Consumer Inbox Deduplication

Tiêu chí đối chiếu	HTTP Idempotency (Bảng IdempotencyRequests)	Consumer Deduplication (Bảng InboxMessages)
Vị trí hoạt động	Biên giao tiếp ngoài cùng (RESTful API Controller / Application Service).	Đường ống tiêu thụ nội bộ (MassTransit Consume Pipeline Filter).
Đối tượng gây trùng	Người dùng bấm đúp (Double-click), trình duyệt hoặc Mobile App tự retry khi mạng lag.	Outbox Dispatcher gửi lại sự kiện hoặc RabbitMQ redeliver khi chưa nhận được ACK.
Khóa định danh	Tổ hợp (UserId, Key) kết hợp kiểm tra chữ ký băm SHA-256 (RequestHash).	Tổ hợp khóa duy nhất (MessageId, ConsumerName).
Hành vi khi phát hiện trùng	Truy vấn lại WorkflowInstance hiện có và trả về kết quả cũ (Replay Response).	Ghi log cảnh báo và ngắt mạch (Short-circuit), bỏ qua việc gọi hàm Consume().

9.5.2 Hành động hoàn tiền vào ví (Refund Ledger) khi xảy ra lỗi nâng cấp tin

Khái niệm Bù trừ ngữ nghĩa và Sổ cái bất biến (Immutable Financial Ledger): Trong mô hình Orchestration Saga, khi bước 1 (DeductWallet) đã trừ tiền ví và commit xuống CSDL thành công, nhưng bước 2 (UpgradeListing) lại thất bại (ví dụ: tin đăng đã bị xóa, bị khóa hoặc gói dịch vụ hết hạn áp dụng), hệ thống không thể gọi lệnh

ROLLBACK của SQL Server để quay ngược lại giao dịch ở bước 1. Thay vào đó, Saga phải kích hoạt một Giao dịch bù trừ (Compensating Transaction) bằng cách phát lệnh RefundWalletCommand.

Nguyên tắc Sổ cái chỉ ghi thêm (Append-Only Audit Trail): Nguyên tắc cốt tử trong thiết kế phần mềm tài chính - ví điện tử là tính bất biến của sổ cái (Append-Only Ledger): khi hoàn tiền, hệ thống tuyệt đối không được dùng lệnh DELETE để xóa bản ghi giao dịch trừ tiền (Charge) trước đó, cũng không được sửa trực tiếp cột Amount của bản ghi cũ. Mọi biến động phải được phản ánh bằng việc chèn thêm một bản ghi giao dịch đối ứng mới mang phân loại WalletTransactionType.Refund với cùng mã định danh luồng CorrelationId. Cách thiết kế này giúp bảo toàn tuyệt đối dấu vết kiểm toán (Audit Trail) và cho phép đối soát minh bạch số dư trước - sau (BalanceBefore, BalanceAfter) tại mọi thời điểm.

Mã nguồn thực tế trong dự án PropNest: Dưới đây là mã nguồn thực tế của phương thức miền trong thực thể Wallet (tầng Domain) và trình tiêu thụ hoàn tiền RefundWalletConsumer (tầng Infrastructure):

```
// 1. Logic trừ tiền (ưu tiên ví khuyến mãi trước) và hoàn tiền trong Domain
(src/PropNest.Domain/Wallets/Wallet.cs)
public void Charge(decimal amount)
{
        if (amount <= 0) throw new ArgumentOutOfRangeException(nameof(amount));
        if (PromoBalance + MainBalance < amount)
        throw new InvalidOperationException("Insufficient wallet balance.");

        var promoUsed = Math.Min(PromoBalance, amount);
        var mainUsed = amount - promoUsed;
        PromoBalance -= promoUsed;
        MainBalance -= mainUsed;
        Touch();
}

public void Refund(decimal amount)
{
        if (amount <= 0) throw new ArgumentOutOfRangeException(nameof(amount));
        MainBalance += amount;
        Touch();
}

// 2. Consumer xử lý lệnh hoàn tiền bù trừ
(src/PropNest.Infrastructure/Consumers/RefundWalletConsumer.cs)
public async Task Consume(ConsumeContext<RefundWalletCommand> context)
{
        var command = context.Message;
```

```
   LogConsuming(_logger, command.CorrelationId, command.UserId, command.Amount);


            // Kiểm tra lũy đẳng nghiệp vụ: Đảm bảo cùng 1 CorrelationId không bao giờ được hoàn tiền 2 lần
            var alreadyProcessed = await _dbContext.WalletTransactions.AnyAsync(
            w => w.CorrelationId == command.CorrelationId && w.TransactionType ==
WalletTransactionType.Refund,
        context.CancellationToken);
            if (alreadyProcessed)
            {
        LogAlreadyProcessed(_logger, command.CorrelationId);
            await context.Publish(new WalletRefunded(command.CorrelationId, command.UserId,
command.Amount));
            return;
            }


            var wallet = await _walletRepository.GetByUserIdAsync(command.UserId,
context.CancellationToken)
            ?? throw new InvalidOperationException($"Không tìm thấy ví của UserId {command.UserId}.");


            var totalBalanceBefore = wallet.MainBalance + wallet.PromoBalance;
   wallet.Refund(command.Amount);
            var totalBalanceAfter = wallet.MainBalance + wallet.PromoBalance;


            var transaction = new WalletTransaction(
            walletId: wallet.WalletId,
        correlationId: command.CorrelationId,
            type: WalletTransactionType.Refund,
```

```
            amount: command.Amount,
        balanceBefore: totalBalanceBefore,
        balanceAfter: totalBalanceAfter,
        description: $"Hoàn tiền giao dịch mua gói tin đăng thất bại ({command.Amount:N0}đ)");

            await _walletRepository.AddTransactionAsync(transaction, context.CancellationToken);
            await _unitOfWork.SaveChangesAsync(context.CancellationToken);

    LogSuccess(_logger, command.CorrelationId, command.UserId, command.Amount);
            await context.Publish(new WalletRefunded(command.CorrelationId, command.UserId,
command.Amount));
}
```

Kết quả chạy thực tế (Bút toán đối ứng trong bảng WalletTransactions): Khi xảy ra kịch bản bù trừ cho giao dịch mua gói VIP (150.000đ) của người dùng có số dư ban đầu 500.000đ, bảng WalletTransactions ghi nhận đầy đủ hai bút toán đối xứng có cùng CorrelationId như bảng dưới đây:

Bảng 9.5 Dữ liệu thực tế trong bảng sổ cái WalletTransactions sau khi hoàn tất luồng Compensation

TransactionId	WalletId	CorrelationId	Type	Amount	BalanceBefore	BalanceAfter	Description
1042	5	a91c88e1-...	Charge	150,000.00	500,000.00	350,000.00	Thanh toán gói dịch vụ

							tin đăng (150,000đ)
1043	5	a91c88e1-...	Refund	150,000.00	350,000.00	500,000.00	Hoàn tiền giao dịch mua gói tin đăng thất bại (150,000đ)

9.5.3 Ghi nhận chi tiết từng bước chuyển trạng thái (WorkflowStep) và API Polling tiến trình

Khái niệm Read-Model theo dõi tiến trình từng bước (Step-level Observability): Trong kiến trúc bất đồng bộ, sau khi nhận phản hồi 202 Accepted, phía giao diện người dùng (Frontend) cần cơ chế Polling qua endpoint GET /api/v1/workflows/{correlationId} để hiển thị thanh tiến trình (Step Indicator) trực quan cho người bán. Nếu chỉ lưu trạng thái tổng quát trên bảng SagaListingStates, người dùng chỉ biết giao dịch đang ở bước nào mà không biết các bước trước đó mất bao lâu hay bước nào bị lỗi gì. Do đó, PropNest thiết kế thêm thực thể Read-Model WorkflowStep để ghi vết chi tiết vòng đời (Started -> Completed / Failed) của từng bước trong cùng một giao dịch CSDL với Saga State.

```
// 1. Cập nhật nguyên tử trạng thái bước hiện tại và mở bước kế tiếp trong PurchasePackageSaga.cs
private static async Task CompleteAndStartStepAsync<TData>(
    BehaviorContext<PurchasePackageSagaState, TData> context,
            string completedStepName,
            string nextStepName,
    WorkflowStatus nextStatus) where TData : class
{
            var serviceProvider = context.GetPayload<IServiceProvider>();
            var dbContext = serviceProvider.GetRequiredService<PropNestDbContext>();

            var workflow = await dbContext.WorkflowInstances.FirstOrDefaultAsync(
            w => w.CorrelationId == context.Saga.CorrelationId, context.CancellationToken);
    workflow?.MoveTo(nextStatus);

            var completedStep = await dbContext.WorkflowSteps
            .Where(s => s.CorrelationId == context.Saga.CorrelationId
              && s.StepName == completedStepName && s.Status == "Started")
        .OrderByDescending(s => s.StartedAt)
        .FirstOrDefaultAsync(context.CancellationToken);

            if (completedStep is not null)
        completedStep.Complete(); // Cập nhật Status = "Completed" và FinishedAt = UtcNow tại chỗ

            await dbContext.WorkflowSteps.AddAsync(
            new WorkflowStep(context.Saga.CorrelationId, nextStepName), context.CancellationToken);
            await dbContext.SaveChangesAsync(context.CancellationToken);
}
```

```
// 2. Endpoint truy vấn tiến trình có kiểm tra quyền sở hữu
(src/PropNest.Api/Controllers/WorkflowsController.cs)
[HttpGet("{correlationId:guid}")]
public async Task<ActionResult<WorkflowDto>> GetByCorrelationId(Guid correlationId,
CancellationToken cancellationToken)
{
        var workflow = await workflowService.GetByCorrelationIdAsync(correlationId,
cancellationToken);
        if (workflow is null) return NotFound();

        var userId = currentUser.UserId;
        if (!userId.HasValue) return Unauthorized();
        if (workflow.UserId != userId.Value && !currentUser.IsInRole("Admin"))
        return Forbid(); // Trả về 403 Forbidden nếu người dùng khác cố tình xem trộm tiến trình

        return Ok(workflow);
}
```

Giải thích kỹ thuật cập nhật tại chỗ (In-Place Step Lifecycle Update): Điểm tinh tế trong kỹ thuật hiện thực hóa ở trên là việc tìm lại bản ghi WorkflowStep đang mang trạng thái 'Started' để gọi phương thức Complete() hoặc Fail(error) cập nhật trực tiếp trường FinishedAt tại chỗ (In-place Update), thay vì tạo mới một dòng bản ghi thứ hai có cùng StepName. Điều này giúp tránh triệt để lỗi sinh bản ghi treo (Dangling Started Row) và đảm bảo mỗi bước thực thi chỉ tương ứng với đúng một dòng dữ liệu duy nhất khi trả về qua danh sách steps.

KẾT LUẬN CHƯƠNG 8

Chương 8 đã trình bày một cách hệ thống từ cơ sở lý thuyết đến thực tiễn triển khai các giải pháp xử lý giao dịch phân tán và điều phối luồng công việc (Workflow Orchestration) trên nền tảng ASP.NET Core 8. Thông qua bài toán thực tế là quy trình mua gói dịch vụ nâng cấp tin đăng VIP trong hệ thống bất động sản PropNest, chương đã giải quyết trọn vẹn bài toán đảm bảo tính nhất quán dữ liệu khi không thể sử dụng giao dịch ACID cục bộ hay giao thức khóa hai pha (2PC).

Tổng kết các trụ cột kỹ thuật cốt lõi đã được nghiên cứu, hiện thực hóa và kiểm chứng thực nghiệm trong chương bao gồm:

- **Điều phối tập trung với Orchestration Saga State Machine:** Ứng dụng thư viện MassTransit 8 kết hợp RabbitMQ để xây dựng máy trạng thái PurchasePackageSaga gồm 9 trạng thái tường minh. Trạng thái của mỗi phiên giao dịch được lưu trữ bền vững trong SQL Server và được bảo vệ trước nguy cơ xung đột cập nhật đồng thời (Race Condition) bằng khóa tương tranh lạc quan ISagaVersion.
- **Đảm bảo độ tin cậy thông điệp hai chiều với Transactional Outbox và Inbox Deduplication:** Giải quyết triệt để bài toán Ghi kép (Dual-Write Problem) thông qua việc lưu bản ghi OutboxMessage trong cùng một giao dịch CSDL với nghiệp vụ chính, kết hợp tiến trình OutboxDispatcherWorker phát sự kiện theo cơ chế At-Least-Once Delivery và bộ lọc đường ống InboxDeduplicationFilter<T> chặn đứng mọi thông điệp trùng lặp tại Consumer.
- **Lũy đẳng đầu-cuối (End-to-End Idempotency) và Giao dịch bù trừ hoàn tiền minh bạch:** Bảo vệ biên giao tiếp RESTful API bằng cơ chế kiểm tra khóa Idempotency-Key kết hợp hàm băm mật mã SHA-256 (RequestHash); đồng thời hiện thực hóa cơ chế hoàn tiền tự động (RefundWalletConsumer) trên sổ cái tài chính chỉ ghi thêm (Append-Only Immutable Ledger) và mô hình đọc tiến trình từng bước (WorkflowSteps) phục vụ truy vấn thời gian thực.

Những kỹ thuật được xây dựng trong Chương 8 không chỉ giúp hệ thống PropNest vận hành an toàn trước các sự cố đứt kết nối mạng, sập tiến trình hay lỗi nghiệp vụ phát sinh giữa chừng, mà còn tạo tiền đề vững chắc để tiếp tục tối ưu hóa hiệu năng, xử lý tác vụ lập lịch định kỳ và giám sát sức khỏe toàn hệ thống ở Chương 9.

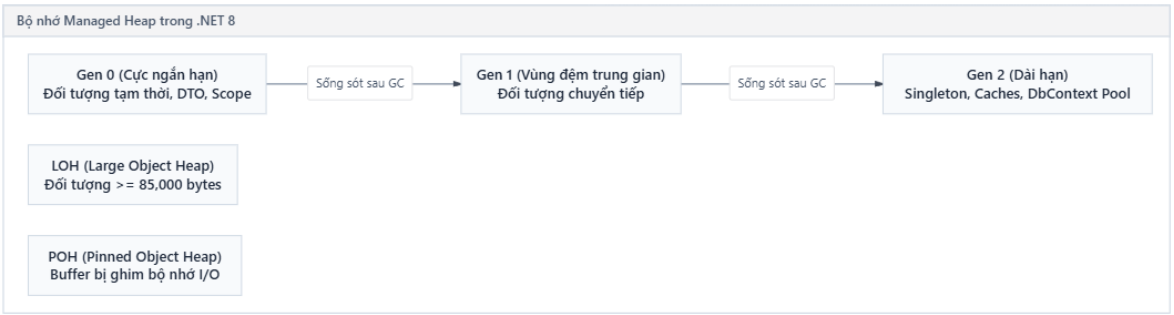
CHƯƠNG 10 TỐI ƯU HIỆU NĂNG VÀ VẬN HÀNH HỆ THỐNG

10.1 Các vấn đề hiệu năng và điểm nghẽn trong hệ thống Backend

10.1.1 Bottleneck tại tầng ứng dụng (CPU, Memory Allocation)

a. Mô hình bộ nhớ CLR và các thế hệ rác (Generational Garbage Collection)

Trong môi trường thực thi của .NET (Common Language Runtime - CLR), bộ nhớ heap được quản lý tự động thông qua bộ thu gom rác **Garbage Collector (GC)**. GC hoạt động dựa trên *Giả thuyết thế hệ (Generational Hypothesis)*: phần lớn các đối tượng (objects) được sinh ra có vòng đời rất ngắn (short-lived); nếu một đối tượng sống sót qua các đợt dọn rác, xác suất nó tiếp tục tồn tại lâu dài là rất cao.



Hình 10.1 Bộ nhớ Managed Heap trong .NET 8

Bộ nhớ Managed Heap trong .NET được phân chia thành các phân vùng logic:

- **Generation 0 (Gen 0)**: Chứa các đối tượng mới được khởi tạo (biến cục bộ, request context, DTO ngắn hạn). Chi phí dọn rác tại Gen 0 là cực kỳ nhỏ (tính bằng microsecond) và thường diễn ra hoàn toàn trong bộ nhớ đệm CPU (L1/L2 Cache).
- **Generation 1 (Gen 1)**: Đóng vai trò là vùng đệm chuyển tiếp giữa các đối tượng ngắn hạn và dài hạn.
- **Generation 2 (Gen 2)**: Chứa các đối tượng có vòng đời dài hạn (Static variables, In-Memory Caches, Singleton Services, DbContext pools). Một chu kỳ dọn rác tại

Gen 2 (Full GC) đòi hỏi quét toàn bộ heap, dẫn đến hiện tượng đình trệ tiến trình (*Stop-the-World pause*).

- **Large Object Heap (LOH)**: Dành riêng cho các đối tượng có kích thước từ 85.000 bytes85.000\text{ bytes}85.000 bytes trở lên (mảng byte lớn, file buffers, chuỗi ký tự khổng lồ). LOH không được dồn rác (compaction) mặc định để tránh chi phí sao chép bộ nhớ khổng lồ, do đó dễ dẫn đến hiện tượng phân mảnh bộ nhớ (Memory Fragmentation).
- **Pinned Object Heap (POH)**: Được giới thiệu từ .NET 5 nhằm cô lập các đối tượng bị ghim địa chỉ cố định phục vụ thao tác Native I/O hoặc Socket Buffers.

b. Phân tích chi phí cấp phát bộ nhớ (Allocation Cost) và tác động lên độ trễ P99/P99.9

Một quan niệm sai lầm phổ biến là xem nhẹ chi phí cấp phát bộ nhớ trên Heap với lý do "bộ nhớ RAM ngày nay rất rẻ". Tuy nhiên, điểm nghẽn hiệu năng thực sự trong các dịch vụ Web không nằm ở dung lượng RAM bị chiếm dụng, mà nằm ở **tần suất và thời gian đình trệ luồng (Thread Pauses) của bộ thu gom rác**.

Khi lưu lượng truy cập đạt hàng nghìn request/giây, nếu mỗi request liên tục khởi tạo các đối tượng ngắn hạn không cần thiết (boxing, chuỗi string con, allocations trong vòng lặp), Gen 0 sẽ nhanh chóng đầy, kích hoạt dồn dập các đợt dọn rác Gen 1 và Full GC Gen 2. Các luồng xử lý worker của Kestrel bị tạm ngưng (Stop-the-World), khiến cho độ trễ phân vị cao (**P99 và P99.9 Latency**) bị tăng vọt đột biến, phá vỡ cam kết chất lượng dịch vụ (Service Level Agreement - SLA).

c. Kỹ thuật Compile-Time Logging với **[LoggerMessage] Source Generator**

Trong các phiên bản ASP.NET Core truyền thống, việc ghi nhật ký qua phương thức mở rộng **_logger.LogInformation("Message with {Param}", param)** gặp phải ba nhược điểm hiệu năng cố hữu:

1. **Boxing Allocation**: Tham số truyền vào phải được đóng gói thành kiểu **object**, buộc các kiểu giá trị (primitive value types như **int, long, Guid, decimal**) phải chuyển đổi từ **Stack sang Heap.**
2. **Array Allocation**: Trình biên dịch phải khởi tạo một mảng **params object[]** mới trên Heap trong mỗi lần triệu gọi hàm ghi log.
3. **Runtime String Parsing**: Thông điệp log dạng template phải được phân tích cú pháp (parsing) tại thời điểm thực thi (runtime) để nội suy giá trị.

Từ .NET 6 trở đi và được tối ưu vượt bậc trong .NET 8, tính năng **Compile-time Logging Source Generator** thông qua thuộc tính **[LoggerMessage]** giải quyết triệt để vấn đề này. Trình biên dịch phân tích cú pháp template ngay ở thời điểm compile và tự động sinh mã C# strongly-typed cực kỳ tối ưu, loại bỏ hoàn toàn boxing và mảng **object[]**.

```
[LoggerMessage(Level = LogLevel.Information, Message = "ListingExpirationJob started at
{Timestamp}. Checking for expired listings...")]
private partial void LogStarted(DateTimeOffset timestamp);

[LoggerMessage(Level = LogLevel.Information, Message = "ListingExpirationJob completed. Processed {Count}
expired listing(s).")]
private partial void LogCompleted(int count);

[LoggerMessage(Level = LogLevel.Error, Message = "ListingExpirationJob execution failed.")]
private partial void LogFailed(Exception exception);
```

Mã nguồn được sinh tự động bởi Source Generator tại tầng IL trực tiếp gán giá trị vào **LogDefine** struct mà không thực hiện bất kỳ allocation nào trên bộ nhớ Heap, đảm bảo tốc độ ghi log tối đa ngay cả trong vòng lặp quét hàng chục nghìn bản ghi.

d. Tái sử dụng **JsonSerializerOptions**

Trong .NET, lớp **JsonSerializerOptions** chứa bộ nhớ đệm metadata về schema và hợp đồng tuần tự hóa (Contract Resolvers) của các kiểu dữ liệu. Việc khởi tạo một đối tượng **new JsonSerializerOptions()** mới trong mỗi phương thức serialize/deserialize (như parse delta JSON hoặc đọc phản hồi HTTP) sẽ buộc framework phải xây dựng lại toàn bộ metadata cache, gây lãng phí bộ nhớ và CPU nghiêm trọng.

Tại **ApiSellerPropertyService.cs**, hệ thống đã định nghĩa và sử dụng một đối tượng **JsonSerializerOptions** duy nhất dưới dạng **static readonly**:

```
private static readonly JsonSerializerOptions s_jsonOptions = new()
{
  PropertyNameCaseInsensitive = true
};
```

Toàn bộ các thao tác giải mã danh sách lịch sử **(ApiListingHistoryDto)**, độ lệch delta **(ApiListingDeltaDto)** và lý do từ chối duyệt đều tái sử dụng instance tĩnh này, tối ưu hóa triệt để tài nguyên của phân hệ Web.

10.1.2 Bottleneck tại tầng truy xuất dữ liệu: Phân tích chi phí Entity Framework Core và tối ưu hóa truy vấn

a. Cơ chế Change Tracking và chi phí ẩn của Snapshot Tracking

Trong kiến trúc Entity Framework Core, **Change Tracker** là phân hệ chịu trách nhiệm theo dõi mọi biến động về trạng thái của các thực thể (Added, Modified, Deleted, Unchanged). Khi một thực thể được truy vấn từ cơ sở dữ liệu với chế độ mặc định:

- EF Core tạo một bản sao lưu trạng thái gốc (Snapshot Copy) của từng thuộc tính bên trong bộ nhớ của **`DbContext`**.
- Khi gọi **`SaveChangesAsync()`**, EF Core phải duyệt qua toàn bộ đồ thị thực thể (Entity Graph), thực hiện thuật toán so khớp (Detect Changes) giữa giá trị hiện tại và Snapshot ban đầu để sinh ra các câu lệnh SQL **`UPDATE`**.

Quá trình này tiêu tốn gấp đôi bộ nhớ cho mỗi thực thể được nạp vào RAM và làm tăng đáng kể chu kỳ quét CPU. Đối với các thao tác chỉ đọc (Read-only operations) phục vụ hiển thị dữ liệu lên giao diện, việc bật Change Tracker là một sự lãng phí tài nguyên nghiêm ngặt.

Bảng 10.1 Bảng so sánh định lượng các chiến lược truy vấn trong EF Core 8

Tiêu chí	**Truy vấn Tracking mặc định**	**AsNoTracking()**	**AsNoTrackingWithIdentityResolution()**	**Projection qua DTO (Select)**
Mục đích sử dụng	Đọc dữ liệu để sửa đổi và gọi SaveChanges()	Truy vấn chỉ đọc (Search, View, Report)	Truy vấn chỉ đọc có quan hệ lặp 1-N phức tạp	Truy vấn chỉ đọc lấy đúng các cột cần thiết
Chi phí bộ nhớ (RAM)	Rất cao (Lưu Entity + Snapshot)	Thấp (Chỉ lưu Entity)	Trung bình (Lưu Entity + Identity Dictionary)	Cực thấp (Chỉ cấp phát DTO đích)

Thay đổi cấu trúc SQL	SELECT toàn bộ các cột của bảng	SELECT toàn bộ các cột của bảng	SELECT toàn bộ các cột của bảng	Tối ưu: SELECT col1, col2...
Hiệu năng thực thi	Tiêu chuẩn (1.0)	Nhanh hơn ≈1.8–2.5 lần	Nhanh hơn ≈1.5–2.0 lần	Nhanh nhất ≈3.0–4.5 lần

Trong dự án PropNest, tại **ListingRepository.cs**, toàn bộ các phương thức truy vấn danh sách (**GetByOwnerUserIdAsync, GetPendingModerationAsync, GetHistoriesByListingIdAsync**) đều được áp dụng tường minh phương thức **.AsNoTracking()**, loại bỏ hoàn toàn gánh nặng Change Tracking trên máy chủ API.

b. Vấn đề truy vấn N+1 (N+1 Query Problem) và Hội chứng bùng nổ tích Descartes (Cartesian Explosion)

Vấn đề truy vấn N+1 xảy ra khi ứng dụng thực thi 1 câu lệnh SQL ban đầu để nạp N bản ghi cha, sau đó trong quá trình duyệt vòng lặp, hệ thống lại thực thi thêm N câu lệnh phụ để nạp các bản ghi con liên kết:

Tổng số truy vấn = 1 + N

Nếu danh sách có 1.000 tin đăng, hệ thống sẽ thực hiện 1.001 chuyến đi khứ hồi qua mạng (**Network Round-trips**), gây nghẽn hoàn toàn đường truyền và làm quá tải Connection Pool của SQL Server.

Mặt khác, việc giải quyết bài toán trên bằng cách gộp tất cả các bảng liên quan thông qua nhiều lệnh **.Include()** trong một câu lệnh SQL duy nhất lại dẫn tới hiện tượng **Cartesian Explosion (Bùng nổ tích Descartes)**. Nếu một tin đăng có 10 ảnh và 5 lịch sử thay đổi, câu truy vấn JOIN sẽ sinh ra 1×10×5=50 dòng dữ liệu trả về qua mạng, trong đó toàn bộ thông tin văn bản dài (Title, Description, Address) của bảng cha bị lặp lại vô ích 50 lần.

Giải pháp chuẩn được áp dụng trong PropNest:

1. **Kết hợp Eager Loading có định hướng**: Chỉ **Include** thực thể liên quan thực sự cần thiết kèm mệnh đề sắp xếp cục bộ:

```
// Trích xuất từ ListingRepository.cs
await dbContext.Listings
   .Include(x => x.Images.OrderBy(img => img.SortOrder))
   .AsNoTracking()
   .Where(x => x.OwnerUserId == ownerUserId)
   .ToListAsync(cancellationToken);
```

2. **Phép chiếu DTO trực tiếp (Projection)**: Tận dụng sức mạnh của LINQ to Entities để chiếu thẳng sang DTO tại câu truy vấn LINQ, ép SQL Server chỉ truyền đúng các trường hiển thị, triệt tiêu 100% hiện tượng thừa cột và N+1:

```
var query = from history in dbContext.ListingHistories.AsNoTracking()
        where history.ListingId == listingId
        join user in dbContext.Users.AsNoTracking() on history.ActorUserId equals user.Id into userGroup
        from actor in userGroup.DefaultIfEmpty()
        orderby history.ActionDate ascending, history.HistoryId ascending
        select new ListingHistoryDto(
           history.ActionType,
           history.ActionDate,
           history.ActorUserId,
           actor != null ? (actor.FullName ?? actor.UserName) : "Hệ thống",
           actor != null ? actor.Role : "System",
           history.SnapshotJson,
           history.DeltasJson,
```

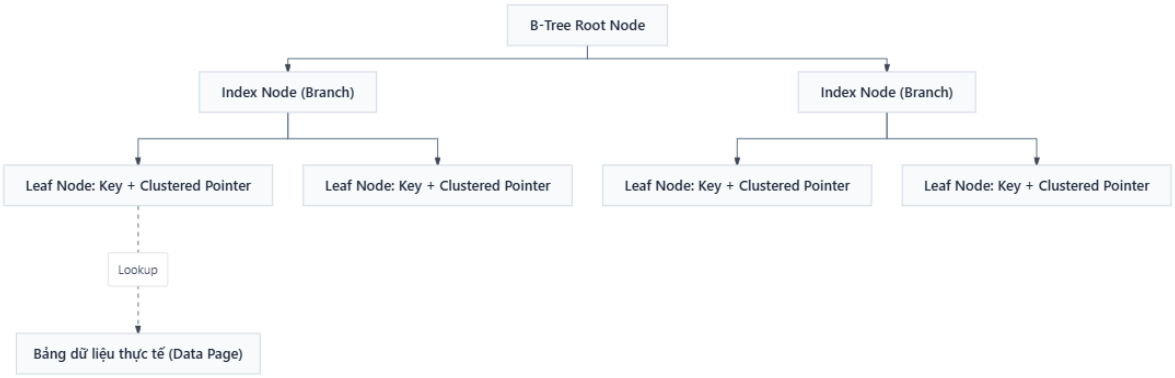
```
    history.Note
);
```

10.1.3 Lý thuyết chỉ mục (Indexing Theory) và thiết kế Index tối ưu trên SQL Server

Chỉ mục (Index) là cấu trúc dữ liệu dạng cây B-Tree (Balanced Tree) giúp hệ quản trị cơ sở dữ liệu định vị các dòng thỏa mãn điều kiện tìm kiếm với độ phức tạp thuật toán O(log N), thay vì phải quét toàn bộ bảng dữ liệu vật lý (Table Scan / Clustered Index Scan) với chi phí O(N).

Trong SQL Server:

- **Clustered Index**: Xác định thứ tự sắp xếp vật lý thực tế của các dòng dữ liệu trong các trang dữ liệu (Data Pages). Mỗi bảng chỉ có duy nhất 1 Clustered Index (thường gán vào Khóa chính Primary Key).
- **Non-Clustered Index**: Là cấu trúc cây tách rời lưu trữ các cột chỉ định (Index Keys) và một con trỏ (Row Locator / Clustering Key) trỏ ngược về dòng dữ liệu gốc.
- **Quy tắc Tiền tố Bên trái (Leftmost Prefix Rule)**: Trong một chỉ mục phức hợp (Composite Index) gồm nhiều cột (C_1,C_2,C_3), bộ tối ưu hóa truy vấn (Query Optimizer) chỉ có thể sử dụng chỉ mục nếu điều kiện tìm kiếm chứa tiền tố liên tục bắt đầu từ cột đầu tiên (tức là tìm theo C_1, hoặc (C_1, C_2) hoặc (C_1,C_2,C_3)). Nếu truy vấn chỉ tìm theo C_2 hoặc C_3, chỉ mục sẽ bị vô hiệu hóa hoặc chuyển thành Index Scan kém hiệu quả.



Hình 10.2 Cấu trúc tổ chức dữ liệu của B-Tree Index

Tại **ListingConfiguration.cs**, hệ thống đã thiết lập 3 Composite Indexes tương ứng với 3 mẫu truy cập dữ liệu (Access Patterns) trọng yếu nhất:

```
// 1. Phục vụ màn hình Seller Portal: Lọc tin của chính người dùng theo trạng thái
builder.HasIndex(x => new { x.OwnerUserId, x.Status });

// 2. Phục vụ Job chạy ngầm Quartz: Quét các tin Published đã quá hạn hiển thị
builder.HasIndex(x => new { x.Status, x.EndDate });

// 3. Phục vụ bộ lọc tìm kiếm công khai phía Khách hàng: Lọc theo Tỉnh, Huyện và Loại BĐS
builder.HasIndex(x => new { x.City, x.District, x.PropertyType });
```

Phân tích tác dụng kiến trúc:

- **IX_Listings_OwnerUserId_Status**: Đảm bảo truy vấn **WHERE OwnerUserId = @sellerId AND Status = @status** đạt tốc độ **Index Seek** tức thời, cho phép phân trang và tính toán số lượng tin (Counts) siêu tốc ngay cả khi bảng **Listings** có hàng triệu bản ghi.

- **IX_Listings_Status_EndDate**: Giúp tác vụ ngầm **ListingExpirationJob** thực hiện quét trực tiếp các tin đăng có **Status = 'Published'** và **EndDate <= @now** mà không phải duyệt qua các tin đang là **Draft, PendingModeration** hay **Expired**.
- **IX_Listings_City_District_PropertyType**: Khai thác triệt để phân cấp hành chính địa lý từ diện rộng đến diện hẹp, tăng tốc tối đa trải nghiệm tra cứu bất động sản của khách hàng.

10.2 Kiểm soát xung đột ghi đè đồng thời (Optimistic Concurrency Control - OCC)

10.2.1 Cơ sở lý thuyết: So sánh chuyên sâu giữa Pessimistic Locking và Optimistic Concurrency Control

Trong các ứng dụng Web đa luồng và môi trường điện toán đám mây phân tán, nhiều người dùng hoặc tiến trình có thể cùng đọc và cố gắng cập nhật một bản ghi dữ liệu tại cùng một thời điểm. Nếu không có cơ chế kiểm soát, hiện tượng **Ghi đè mất mát dữ liệu (Lost Update Problem)** chắc chắn sẽ xảy ra: Thao tác của người cập nhật sau sẽ vô tình xóa sạch toàn bộ các sửa đổi mà người cập nhật trước vừa thực hiện.

Để xử lý bài toán này, ngành khoa học máy tính đưa ra hai trường phái kiểm soát đồng thời đối lập:

1. **Khóa bi quan (Pessimistic Locking)**

- **Triết lý**: Luôn giả định rằng xung đột dữ liệu có xác suất rất cao sẽ xảy ra.
- **Cơ chế**: Khi một giao dịch đọc dữ liệu với ý định cập nhật, nó lập tức chiếm dụng một khóa độc quyền (**Exclusive Lock - X-Lock**) hoặc khóa cập nhật (**U-Lock**) trên hàng dữ liệu đó thông qua các cú pháp như **SELECT ... WITH (UPDLOCK, ROWLOCK)**. Mọi giao dịch khác cố gắng truy cập vào hàng này đều bị chặn (blocked) và phải xếp hàng chờ cho đến khi giao dịch đầu tiên hoàn tất (**COMMIT** hoặc **ROLLBACK**).

- **Nhược điểm chí mạng**: Khóa bi quan làm tê liệt thông lượng của các ứng dụng web không trạng thái (Stateless Web Applications). Việc giữ khóa vật lý xuyên suốt khoảng thời gian người dùng thao tác trên màn hình là điều bất khả thi; đồng thời khóa kéo dài dễ gây hiện tượng leo thang khóa (Lock Escalation từ khóa hàng lên khóa bảng) và Deadlock.

2. **Khóa lạc quan (Optimistic Concurrency Control - OCC)**

- **Triết lý**: Giả định rằng trong phần lớn thời gian, xác suất xảy ra xung đột đồng thời trên cùng một bản ghi là tương đối thấp ($P(\text{conflict}) \ll 1$).
- **Cơ chế**: Không áp dụng bất kỳ khóa vật lý nào trong quá trình đọc dữ liệu. Ứng dụng cho phép vô số client đọc dữ liệu đồng thời hoàn toàn tự do. Tuy nhiên, mỗi bản ghi được gắn kèm một **Mã phiên bản (Version Token / RowVersion)**. Khi một client gửi yêu cầu cập nhật, hệ thống chỉ chấp nhận ghi đè nếu mã phiên bản gửi lên trùng khớp hoàn toàn với mã phiên bản hiện tại trong cơ sở dữ liệu. Nếu phát hiện sai lệch (chứng tỏ đã có người khác cập nhật trước đó), thao tác ghi sẽ bị từ chối ngay lập tức.

Bảng 10.2 Bảng so sánh chi tiết giữa Khóa bi quan và Khóa lạc quan

Tiêu chí phân tích	**Khóa bi quan (Pessimistic Locking)**	**Khóa lạc quan (Optimistic Concurrency Control)**
Chiến lược khóa	Khóa chặn vật lý trên hàng/bảng (Blocking)	Không khóa, kiểm tra phiên bản dữ liệu (Non-blocking)
Thông lượng (Throughput)	Thấp khi tải cao; độ trễ tăng dần theo hàng đợi	Rất cao; tài nguyên được giải phóng tức thì

Nguy cơ Deadlock	Rất cao do các chu kỳ chờ khóa chéo nhau	Không có nguy cơ Deadlock từ phía khóa hàng
Trải nghiệm Client	Chờ đợi phản hồi lâu; dễ bị Timeout	Phản hồi tức thì; thông báo rõ ràng khi có xung đột
Môi trường phù hợp	Giao dịch tài chính cốt lõi (Core Banking), kho hàng cực nóng	Ứng dụng Web/RESTful API, cập nhật thông tin nghiệp vụ

10.2.2 Cơ chế hiện thực hóa RowVersion / Timestamp trong SQL Server và EF Core 8

Trong hệ quản trị cơ sở dữ liệu Microsoft SQL Server, kiểu dữ liệu **ROWVERSION** (trước đây có bí danh là **TIMESTAMP**) là một kiểu dữ liệu nhị phân độ dài 8 bytes (binary(8)). Giá trị này không liên quan gì đến thời gian thực của đồng hồ vật lý, mà là một **bộ đếm tăng dần đơn điệu mang tính toàn cục (globally unique monotonic counter)** trong phạm vi một cơ sở dữ liệu.

Mỗi khi một dòng chứa cột **ROWVERSION** được thêm mới hoặc bị sửa đổi bất kỳ cột nào, SQL Server tự động tạo ra một giá trị nhị phân mới từ bộ đếm toàn cục và gán vào cột này. Đây là cơ chế bảo vệ ở cấp độ lõi của Database Engine, không thể bị chỉnh sửa thủ công từ bên ngoài.

1. Cấu hình Fluent API trong Entity Framework Core

Để kích hoạt cơ chế Optimistic Concurrency Token trong EF Core, nhà phát triển cấu hình thuộc tính kiểu **byte[]** thông qua phương thức **.IsRowVersion()**:

```
// Đánh dấu thuộc tính RowVersion là Concurrency Token
builder.Property(x => x.RowVersion).IsRowVersion();
```

Thực thể Domain tương ứng lưu giữ thuộc tính:

```
public byte[] RowVersion { get; private set; } = Array.Empty<byte>();
```

2. Cơ chế sinh mã SQL khi cập nhật và phát hiện xung đột

Khi một yêu cầu cập nhật được gửi lên (ví dụ: Seller chỉnh sửa giá tin đăng), quy trình diễn ra như sau:

Bước 1: Client gửi kèm chuỗi **RowVersion** gốc dưới dạng Base64 string nhận được từ lúc đọc dữ liệu.

Bước 2: Tại tầng Infrastructure, ứng dụng thiết lập giá trị gốc (**OriginalValue**) cho Change Tracker:

```
public void SetOriginalRowVersion(Listing listing, byte[] rowVersion)
{
   ArgumentNullException.ThrowIfNull(listing);
   if (rowVersion == null || rowVersion.Length == 0)
   {
      throw new ArgumentException("A RowVersion is required for updates.", nameof(rowVersion));
   }
   dbContext.Entry(listing).Property(x => x.RowVersion).OriginalValue = rowVersion;
}
```

Bước 3: Khi thực thi **await unitOfWork.SaveChangesAsync()**, EF Core không thực hiện lệnh **UPDATE** thông thường, mà tự động bổ sung điều kiện so sánh **RowVersion** vào mệnh đề **WHERE**:

```
SET NOCOUNT ON;
UPDATE [Listings]
SET [Title] = @p0, [Price] = @p1, [Description] = @p2
```

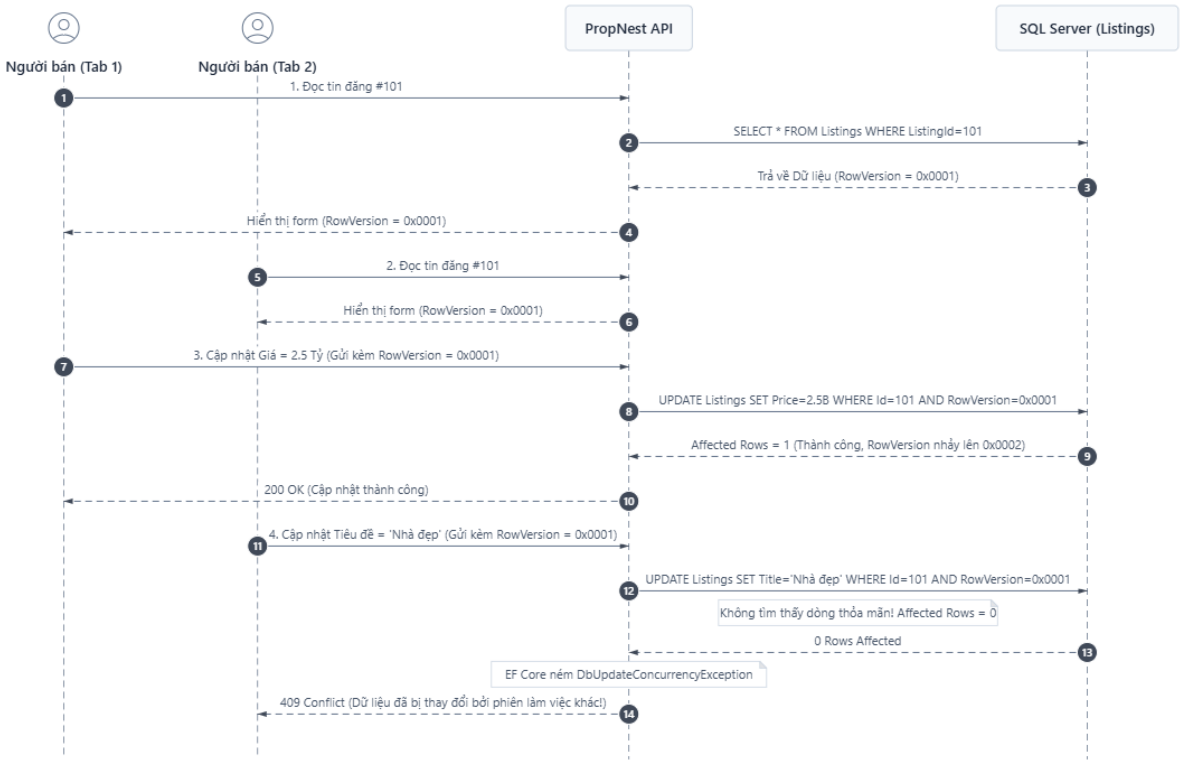
```
WHERE [ListingId] = @p3 AND [RowVersion] = @p4;

SELECT [RowVersion] FROM [Listings] WHERE @@ROWCOUNT > 0 AND [ListingId] = @p3;
```

Bước 4: Phát hiện xung đột:

- Nếu dữ liệu chưa bị ai sửa, **[RowVersion]** khớp với **@p4**. Câu lệnh cập nhật thành công, biến hệ thống **@@ROWCOUNT** trả về **1**. Câu lệnh **SELECT** tiếp theo trả về mã **RowVersion** mới nhất để EF Core cập nhật lại vào Entity.
- Nếu một tiến trình khác đã cập nhật trước đó, **[RowVersion]** trong cơ sở dữ liệu đã thay đổi, khiến mệnh đề **WHERE** không tìm thấy dòng nào khớp. Giá trị **@@ROWCOUNT** trả về **0**. Ngay lập tức, EF Core nhận biết đã có xung đột dữ liệu và chủ động ném ra ngoại lệ:

```
Exception: DbUpdateConcurrencyException
```



Hình 10.3 . Sequence Diagram của cơ chế sinh mã SQL khi cập nhật và phát hiện xung đột

Bắt giữ ngoại lệ DbUpdateConcurrencyException và chiến lược phản hồi HTTP 409 Conflict an toàn

Việc xử lý ngoại lệ đồng thời đòi hỏi chiến lược thiết kế kiến trúc chuẩn mực ở tầng giao tiếp RESTful API. Hệ thống tuyệt đối không được để lộ lỗi kỹ thuật dạng raw stack trace (mã lỗi 500 Internal Server Error) về phía Client, mà phải chuyển đổi sang mã trạng thái chuẩn **HTTP 409 Conflict** kèm theo định dạng tài liệu lỗi tiêu chuẩn **RFC 7807 (Problem Details for HTTP APIs)**.

Middleware sẽ chặn bắt tập trung toàn bộ ngoại lệ và ánh xạ tường minh:

```
private static ProblemDetails CreateProblemDetails(HttpContext context, Exception exception)
```

```
{
    var status = exception switch
    {
        KeyNotFoundException => StatusCodes.Status404NotFound,
        UnauthorizedAccessException => StatusCodes.Status403Forbidden,
        DbUpdateConcurrencyException => StatusCodes.Status409Conflict, // Bắt lỗi xung đột đồng thời
        ArgumentException => StatusCodes.Status400BadRequest,
        InvalidOperationException => StatusCodes.Status409Conflict,
        _ => StatusCodes.Status500InternalServerError
    };

    return new ProblemDetails
    {
        Status = status,
        Title = status == StatusCodes.Status500InternalServerError
            ? "An unexpected error occurred."
            : exception.Message,
        Detail = status == StatusCodes.Status500InternalServerError ? null : exception.Message,
        Instance = context.Request.Path
    };
}
```

Phía giao diện người dùng, khi nhận mã lỗi 409 từ API, hệ thống đưa ra thông báo rõ ràng cho người bán:

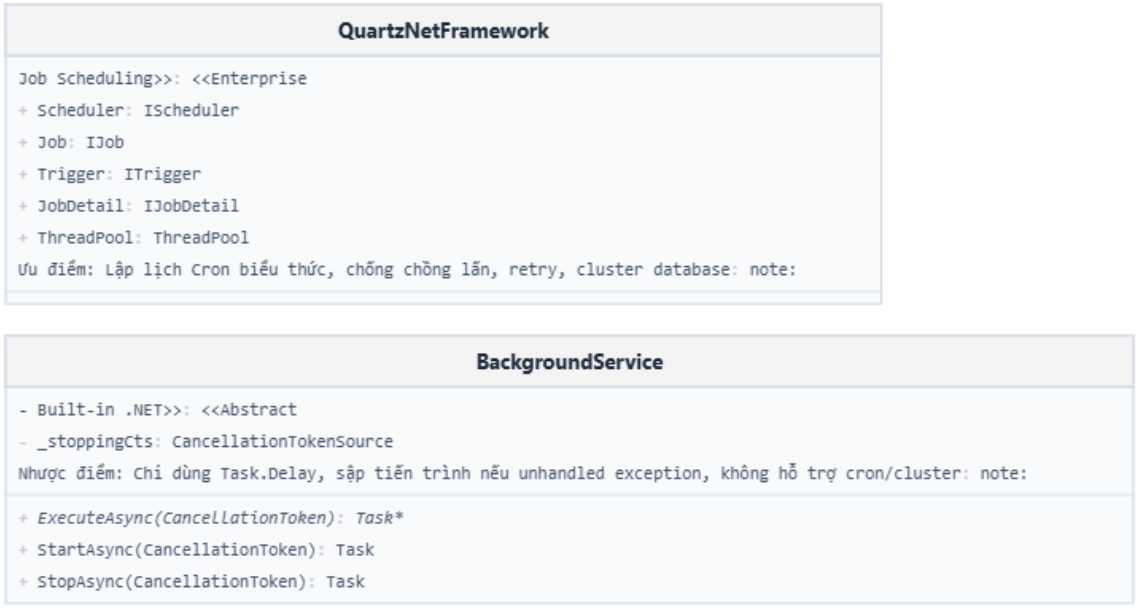
```
"Dữ liệu tin đăng vừa được cập nhật bởi một phiên làm việc khác. Vui lòng tải lại trang để lấy thông tin mới nhất trước khi thực hiện chỉnh sửa."
```

Chiến lược này bảo vệ toàn vẹn tuyệt đối dữ liệu tin đăng và số dư ví, triệt tiêu hoàn toàn nguy cơ thất thoát dữ liệu do ghi đè mất kiểm soát.

10.3 Xử lý tác vụ nền và lập lịch định kỳ với Quartz.NET

10.3.1 Giới hạn của BackgroundService cơ bản và ưu thế của Quartz.NET

Trong các ứng dụng ASP.NET Core, nền tảng cung cấp sẵn giao diện **IHostedService** và lớp trừu tượng **BackgroundService** để thực thi các tác vụ ngầm chạy nền (long-running background tasks). Tuy nhiên, khi áp dụng vào các bài toán lập lịch định kỳ trong môi trường thực tế của doanh nghiệp, **BackgroundService** bộc lộ hàng loạt giới hạn nghiêm trọng:



Hình 10.4 Các ưu, nhược điểm

1. **Phụ thuộc vào vòng lặp Task.Delay thô sơ**: **BackgroundService** không hỗ trợ biểu thức lịch (Calendar / Cron schedule) như *"chạy vào 00:00 mỗi ngày"* hoặc

"chạy mỗi 5 phút một lần từ thứ Hai đến thứ Sáu". Việc tính toán thời gian trễ bằng tay dễ dẫn đến sai lệch tích lũy (Clock Drift).

2. **Nguy cơ sập tiến trình toàn hệ thống**: Nếu một ngoại lệ không được bắt giữ (Unhandled Exception) xảy ra bên trong phương thức **ExecuteAsync** của **BackgroundService**, theo hành vi mặc định của .NET Host, toàn bộ tiến trình ứng dụng (Host Application) sẽ bị crash và dừng hoạt động đột ngột.
3. **Thiếu khả năng chạy cụm (Clustering / Failover)**: Khi ứng dụng được scale mở rộng ra nhiều container/instances (Horizontal Pod Autoscaling), mỗi instance của **BackgroundService** sẽ chạy độc lập, dẫn đến cùng một tác vụ quét hết hạn hay gửi email bị thực thi trùng lặp N lần.
4. **Không có cơ chế kiểm soát chồng lấn (Overlapping Execution)**: Nếu một lần quét dữ liệu mất 7 phút trong khi chu kỳ lặp là 5 phút, **BackgroundService** không tự động quản lý việc ngăn chặn lượt chạy tiếp theo khởi động đè lên lượt trước.

Để giải quyết các vấn đề này, **Quartz.NET** được lựa chọn làm giải pháp tiêu chuẩn cấp doanh nghiệp nhờ kiến trúc tách biệt rõ ràng giữa 4 thành phần:

- **IJob**: Định nghĩa logic nghiệp vụ cần thực thi.
- **IJobDetail**: Chứa thông tin cấu hình, tham số đầu vào (**JobDataMap**) và định danh của Job.
- **ITrigger**: Xác định thời điểm và tần suất Job được kích hoạt (CronTrigger, SimpleTrigger).
- **IScheduler**: Trái tim điều phối trung tâm quản lý vòng đời và cấp phát luồng thực thi cho các Job từ ThreadPool.

10.3.2 Cấu hình Job, Trigger Cron và cơ chế chống chồng lấn [DisallowConcurrentExecution]

1. Biểu thức lập lịch Cron (Cron Schedule)

Trong dự án PropNest, nghiệp vụ quét tin đăng hết hạn cần chạy định kỳ mỗi 5 phút một lần để thu hồi quyền lợi hiển thị của các tin VIP/Standard đã đến hạn. Tại **Program.cs** của phân hệ **PropNest.Workflow.Worker**, Quartz được tích hợp trực tiếp vào Dependency Injection container:

```
builder.Services.AddQuartz(q =>
{
    var jobKey = new JobKey(nameof(ListingExpirationJob));
    q.AddJob<ListingExpirationJob>(opts => opts.WithIdentity(jobKey));

    q.AddTrigger(opts => opts
        .ForJob(jobKey)
        .WithIdentity($"{nameof(ListingExpirationJob)}-trigger")
        .WithCronSchedule("0 0/5 * * * ?")); // Chu kỳ: Mỗi 5 phút tại giây thứ 0
});

builder.Services.AddQuartzHostedService(options =>
{
    options.WaitForJobsToComplete = true; // Chờ job hoàn tất an toàn trước khi tắt ứng dụng (Graceful Shutdown)
});
```

Cấu trúc biểu thức Cron **0 0/5 * * * ?**:



Hình 10.5 Cấu trúc biểu thức Cron

2. Thuộc tính [DisallowConcurrentExecution] và Vòng đời Scoped Service

Khi số lượng tin đăng trong hệ thống tăng cao, thời gian xử lý một mẻ tin hết hạn có thể kéo dài vượt quá chu kỳ 5 phút. Nếu Trigger kế tiếp kích hoạt, hai luồng worker khác nhau sẽ cùng quét và thao tác trên cùng một tập bản ghi, gây lãng phí tài nguyên máy chủ và dễ tạo ra xung đột ghi dữ liệu.

Thuộc tính **[DisallowConcurrentExecution]** được khai báo trên đầu class giải quyết triệt để vấn đề này:

- Quartz sẽ khóa (lock) định danh của Job trong suốt thời gian thực thi.
- Nếu Trigger mới kích hoạt trong khi phiên làm việc trước chưa kết thúc, Quartz sẽ trì hoãn (misfire/postpone) lượt chạy mới cho tới khi phiên hiện tại hoàn thành mỹ mãn.

Đồng thời, do Quartz Job được quản lý dưới dạng Singleton/Transient bởi Scheduler, trong khi các dịch vụ truy xuất cơ sở dữ liệu (**DbContext, Repository, UnitOfWork**) bắt buộc phải có vòng đời **Scoped** để tránh memory leak và thread-safety issues, kỹ thuật khởi tạo scope độc lập qua **IServiceScopeFactory** được triển khai nghiêm ngặt:

```
[DisallowConcurrentExecution]
public sealed partial class ListingExpirationJob : IJob
{
    private readonly IServiceScopeFactory _scopeFactory;
    private readonly ILogger<ListingExpirationJob> _logger;
```

```
public ListingExpirationJob(IServiceScopeFactory scopeFactory, ILogger<ListingExpirationJob> logger)
{
    _scopeFactory = scopeFactory;
    _logger = logger;
}

public async Task Execute(IJobExecutionContext context)
{
    LogStarted(DateTimeOffset.UtcNow);
    try
    {
        // Khởi tạo Scoped Container riêng biệt cho phiên làm việc
        using var scope = _scopeFactory.CreateScope();
        var expirationService = scope.ServiceProvider.GetRequiredService<IListingExpirationService>();
        var processed = await expirationService.ExpireListingsAsync(context.CancellationToken);
        LogCompleted(processed);
    }
    catch (OperationCanceledException) when (context.CancellationToken.IsCancellationRequested)
    {
        // Máy chủ worker đang dừng dịch vụ an toàn (Graceful Shutdown)
    }
    catch (Exception exception)
    {
        LogFailed(exception);
        // Ném JobExecutionException để Quartz nắm bắt trạng thái lỗi
        throw new JobExecutionException(exception, refireImmediately: false);
    }
```

```
    }
}
```

10.3.3 Hiện thực hóa kỹ thuật xử lý mẻ nhỏ (Batch Processing / Chunking) trong ListingExpirationJob

Một lỗi kiến trúc kinh điển khi xử lý dữ liệu lớn là câu lệnh cập nhật hàng loạt không giới hạn:

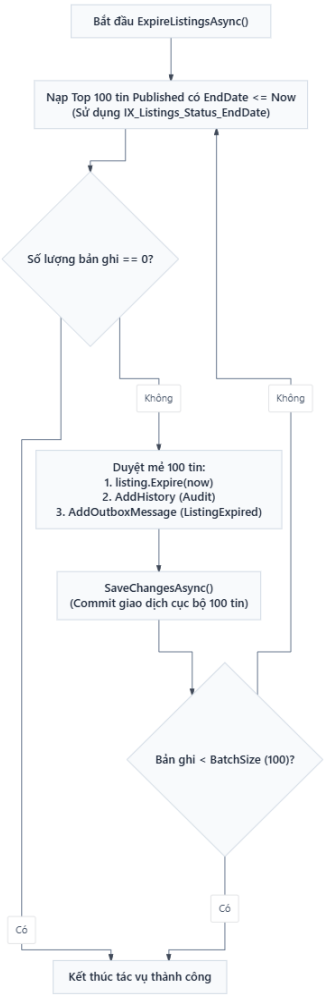
```
-- KỸ THUẬT NGUY HIỂM: Gây khóa bảng và bùng nổ Transaction Log


UPDATE Listings SET Status = 'Expired' WHERE Status = 'Published' AND EndDate <= GETUTCDATE();
```

Nếu có 50.00050.00050.000 tin hết hạn cùng lúc, câu lệnh trên sẽ:

1. Chiếm dụng khóa độc quyền trên toàn bộ bảng **Listings** trong nhiều giây, làm tê liệt toàn bộ các thao tác tìm kiếm của người dùng.
2. Làm phình to đột biến tệp nhật ký giao dịch (SQL Server Transaction Log - **.ldf**), có thể dẫn tới lỗi đĩa cứng đầy.
3. Nếu ứng dụng nạp toàn bộ 50.00050.00050.000 đối tượng vào bộ nhớ RAM của .NET để duyệt vòng lặp, hiện tượng **Out Of Memory (OOM)** sẽ ngay lập tức đánh sập tiến trình worker.

Giải pháp kỹ thuật: Mô hình Xử lý mẻ nhỏ (Batch Processing / Chunking) Hệ thống chia nhỏ khối lượng công việc thành từng mẻ cố định (BatchSize=100) và xử lý tuần tự theo vòng lặp cho đến khi hết dữ liệu.



Hình 10.6 Sơ đồ luồng thực thi của Mô hình Xử lý mẻ nhỏ (Batch Processing / Chunking)

Cách thực hiện chi tiết:

```
public sealed class ListingExpirationService(
    IListingRepository listingRepository,
    IUnitOfWork unitOfWork) : IListingExpirationService
{
```

```
    private const int BatchSize = 100; // Quy mô một mẻ xử lý tối ưu

    public async Task<int> ExpireListingsAsync(CancellationToken cancellationToken = default)
    {
        var now = DateTimeOffset.UtcNow;
        var totalProcessed = 0;

        // Vòng lặp quét mẻ trượt (Sliding Window)
        while (!cancellationToken.IsCancellationRequested)
        {
            var candidates = await listingRepository.GetExpiredPublishedListingsAsync(now, BatchSize,
cancellationToken);
            if (candidates.Count == 0)
            {
                break;
            }

            var batchUpdated = 0;
            foreach (var listing in candidates)
            {
                listing.Expire(now);

                if (listing.Status == ListingStatus.Expired)
                {
                    // 1. Ghi nhận lịch sử kiểm toán (Audit History)
                    listingRepository.AddHistory(ListingHistory.CreateEvent(
                        listing.ListingId,
```

```
            null,
            "Expire"));

        // 2. Ghi nhận thông điệp Outbox để phát đi thông báo bất đồng bộ
        var payload = JsonSerializer.Serialize(new
        {
            listing.ListingId,
            listing.OwnerUserId,
            listing.PackageCode,
            listing.EndDate
        });
        listingRepository.AddOutboxMessage(new OutboxMessage("ListingExpired", payload));
        batchUpdated++;
    }
}

// 3. Xác nhận giao dịch cục bộ cho riêng mẻ 100 bản ghi
if (batchUpdated > 0)
{
    await unitOfWork.SaveChangesAsync(cancellationToken);
    totalProcessed += batchUpdated;
}

// Nếu số lượng lấy ra ít hơn BatchSize nghĩa là đã quét sạch hàng đợi
if (candidates.Count < BatchSize)
{
    break;
```

```
        }
      }

      return totalProcessed;
    }
}
```

Ưu thế vượt trội của phương pháp, đó là:

- **Bộ nhớ không đổi (O(1) Memory Overhead)**: Ứng dụng chỉ duy trì tối đa 100 thực thể trong RAM tại bất kỳ thời điểm nào.
- **Thời gian giữ khóa cực ngắn**: Mỗi giao dịch commit 100 bản ghi chỉ mất vài phần nghìn giây, hoàn toàn vô hình đối với người dùng đang truy cập hệ thống.
- **Đảm bảo tính nhất quán qua Transactional Outbox**: Sự kiện **ListingExpired** được phát hành đồng thời giúp các phân hệ khác (gửi email thông báo, hoàn trả vị trí hiển thị) phối hợp nhịp nhàng mà không gây khớp nối trực tiếp (loose coupling).

10.4 Khả năng quan sát hệ thống (Observability): Structured Logging và Health Checks

10.4.1 Hệ thống ghi log có cấu trúc (Structured Logging) với Serilog và phân tán vết (Distributed Tracing)

1. Sự bất cập của Unstructured Text Logging truyền thống

Trong các hệ thống phân tán, việc ghi log bằng chuỗi văn bản thông thường (plain text):

```
[2026-10-29 14:00:00] User 42 purchased VIP package for Listing 101 successfully.
```

tạo ra rào cản rất lớn cho công tác vận hành. Khi lượng log đạt hàng gigabyte mỗi ngày, việc trích xuất thông tin đòi hỏi phải dùng các biểu thức chính quy (Regex) phức tạp, chậm chạp và dễ gãy vỡ. Người quản trị không thể lọc các câu hỏi như: *"Tìm tất cả các giao dịch mua gói VIP có giá trên 500.000đ trong khung giờ từ 13h đến 15h"*.

Structured Logging giải quyết triệt để vấn đề này bằng cách xem mỗi sự kiện log là một **Tài liệu dữ liệu (Data Document)** có cấu trúc schema rõ ràng (thường là định dạng JSON), lưu giữ nguyên vẹn kiểu dữ liệu của các tham số.

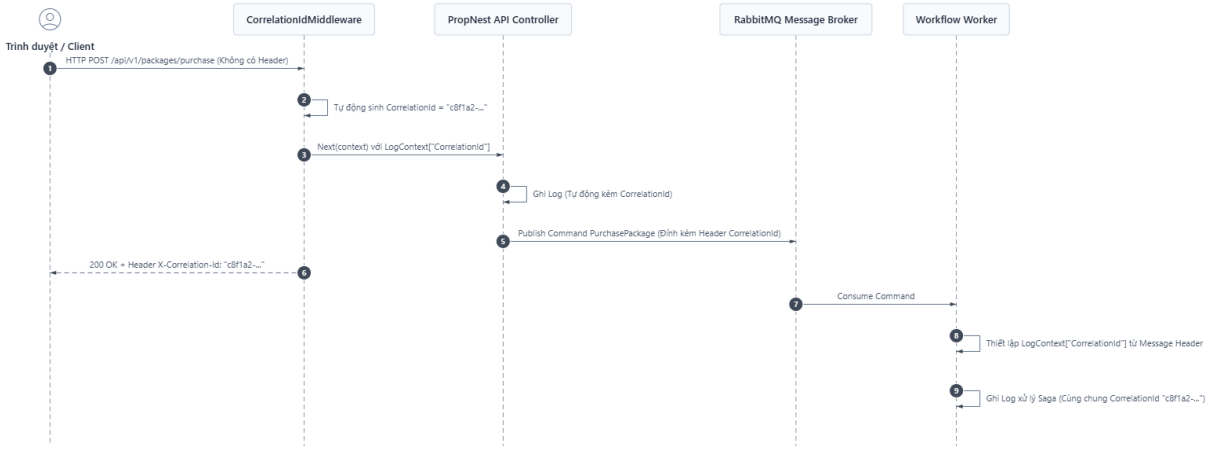
2. Phân tán vết với CorrelationId và LogContext

Trong kiến trúc Modular Monolith của PropNest, một yêu cầu từ phía người dùng gửi tới API có thể kích hoạt Saga Workflow, đẩy thông điệp qua RabbitMQ và được xử lý tại Workflow Worker. Nếu không có định danh vết xuyên suốt, việc điều tra một lỗi phát sinh giữa các tiến trình độc lập là điều bất khả thi.

Dự án triển khai cơ chế **Correlation ID Middleware** tại **CorrelationIdMiddleware.cs**:

- Nếu request gửi lên có header **X-Correlation-Id**, hệ thống sẽ tiếp nhận; nếu chưa có, hệ thống tự động sinh một mã **Guid** duy nhất.
- Header **X-Correlation-Id** được đính kèm vào Response phản hồi cho Client.

- Định danh này được đẩy vào ngữ cảnh bất đồng bộ của Serilog thông qua **LogContext.PushProperty("CorrelationId", correlationId).**



Hình 10.7 Sequence Diagram của Phân tán vết với CorrelationId và LogContext

3. Bộ định dạng log JSON hiệu năng cao **PropNestJsonFormatter**

Để tối ưu hóa tốc độ ghi file và tương thích hoàn hảo với các hệ thống thu thập log tập trung hiện đại (Elasticsearch, Loki, Datadog), dự án xây dựng bộ định dạng tùy biến **PropNestJsonFormatter.cs** dựa trên **Utf8JsonWriter**:

Mỗi dòng log xuất ra màn hình console hoặc file nhật ký xoay vòng (**logs/propnest-.json**) đều tuân thủ cấu trúc JSON tiêu chuẩn duy nhất trên một dòng (Single-line JSON):


```
{
  "Timestamp": "2026-10-29T07:15:23.1045120Z",
  "Level": "Information",
  "Message": "ListingExpirationJob completed. Processed 12 expired listing(s).",
  "CorrelationId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "MachineName": "PROPNEST-PROD-01",
  "ThreadId": 14,
  "Environment": "Production"
```

```
}
```

Nhờ cấu trúc này, kỹ sư vận hành có thể tra cứu toàn bộ hành trình của một yêu cầu xuyên suốt từ API sang Worker chỉ bằng một câu lệnh đơn giản:

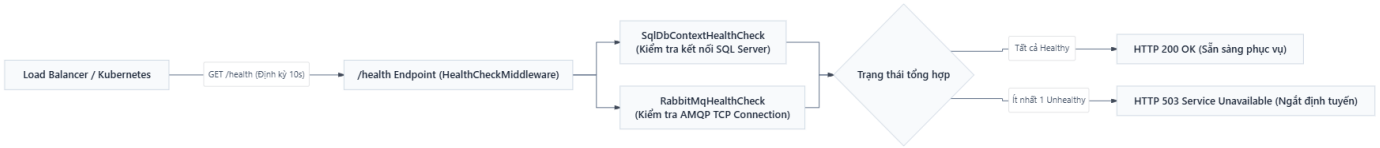
```
grep "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d" logs/propnest-*.json
```

10.4.2 Triển khai Health Checks cho SQL Server và RabbitMQ qua endpoint /health

Trong kiến trúc triển khai container hóa (Docker Compose / Kubernetes), bộ điều phối container cần có cơ chế tự động kiểm tra xem ứng dụng có đang sống hay không để tự động khởi động lại (Restart), và ứng dụng đã sẵn sàng nhận tải hay chưa để định tuyến lưu lượng mạng (Routing).

ASP.NET Core cung cấp khung làm việc **Health Checks Specification**:

- **Liveness Probe**: Kiểm tra xem tiến trình web có đang phản hồi hay bị rơi vào trạng thái treo (Deadlock / Freeze).
- **Readiness Probe**: Kiểm tra xem các tài nguyên phụ thuộc thiết yếu (Database, Message Broker, Cache) có đang kết nối thông suốt hay không.



Hình 10.8 Sơ đồ quy trình Health Check và giám sát trạng thái hoạt động của hệ thống

Tại ServiceCollectionExtensions.cs, hệ thống đăng ký dịch vụ kiểm tra sức khỏe cơ sở dữ liệu SQL Server:

```
services.AddHealthChecks()
```

```
    .AddDbContextCheck<PropNestDbContext>("sqlserver");
```

Tại **Program.cs,** endpoint kiểm tra sức khỏe được phơi bày ra bên ngoài:

```
app.MapHealthChecks("/health");
```

Khi được gọi qua giao thức HTTP **GET /health**:

- Hệ thống âm thầm thực thi một câu lệnh kiểm tra kết nối siêu nhẹ (**SELECT 1**) tới SQL Server thông qua **PropNestDbContext**.
- Nếu kết nối thành công, trả về mã trạng thái **HTTP 200 OK** kèm chuỗi **Healthy**.
- Nếu cơ sở dữ liệu gặp sự cố mất mạng hoặc quá tải kết nối, trả về **HTTP 503 Service Unavailable** kèm chuỗi **Unhealthy**. Bộ cân bằng tải (Load Balancer) lập tức cô lập node gặp sự cố, đảm bảo người dùng không nhận phải các phản hồi lỗi.

KẾT LUẬN CHƯƠNG 9

Chương 9 đã nghiên cứu một cách toàn diện và sâu sắc các nguyên lý học thuật và giải pháp công nghệ trên nền tảng **ASP.NET Core 8** nhằm đạt được hiệu năng tối đa và sự ổn định vận hành cấp doanh nghiệp:

1. **Làm chủ mô hình bộ nhớ và triệt tiêu Allocation**: Phân tích chuyên sâu cơ chế Garbage Collection nhiều thế hệ; áp dụng triệt để các kỹ thuật lập trình hiệu năng cao (**Span<T>, ArrayPool<T>, Utf8JsonWriter**) và tận dụng Source Generator **[LoggerMessage]** để đưa chi phí cấp phát bộ nhớ của hệ thống về mức tối thiểu.
2. **Loại bỏ triệt để điểm nghẽn tầng dữ liệu**: Khắc phục hiện tượng suy giảm hiệu năng do Change Tracking thông qua **AsNoTracking(),** loại trừ lỗi truy vấn N+1 bằng phép chiếu Projection DTO trực tiếp và xây dựng 3 cụm chỉ mục Composite B-Tree chiến lược trên cơ sở dữ liệu SQL Server.

3. **Bảo vệ toàn vẹn dữ liệu bằng Khóa lạc quan (OCC)**: Triển khai thành công cơ chế **RowVersion** trên các thực thể cốt lõi Listing và Wallet. Ứng dụng phát hiện chính xác xung đột thông qua số hàng tác động **@@ROWCOUNT = 0**, kích hoạt ngoại lệ **DbUpdateConcurrencyException** và chuyển đổi an toàn thành mã HTTP 409 Conflict theo tiêu chuẩn RFC 7807 ProblemDetails.
4. **Lập lịch tác vụ nền tin cậy với Quartz.NET**: Thay thế hoàn toàn vòng lặp **BackgroundService** thô sơ bằng Quartz Scheduler chuyên nghiệp; cấu hình biểu thức Cron chính xác; ngăn chặn triệt để hiện tượng chồng lấn tác vụ với **[DisallowConcurrentExecution]** và bảo vệ tài nguyên hệ thống bằng kỹ thuật quét mẻ nhỏ (Batch Processing) 100 bản ghi tuần tự.
5. **Nâng cao khả năng quan sát (Observability)**: Đồng bộ hóa toàn bộ nhật ký ứng dụng dưới dạng Structured JSON Log có cấu trúc thông qua Serilog; xâu chuỗi hành trình nghiệp vụ xuyên suốt các tiến trình nhờ **CorrelationIdMiddleware**; đồng thời bảo đảm độ tin cậy vận hành với hệ thống ASP.NET Core Health Checks.

Bảng 10.3 Bảng tổng kết các kỹ thuật tối ưu hóa và vận hành trong hệ thống PropNest

Lĩnh vực	Vấn đề / Thách thức	Kỹ thuật & Công nghệ áp dụng	Kết quả & Giá trị đạt được
Quản lý bộ nhớ	GC Pauses làm tăng độ trễ P99/P99.9; Boxing log parameters	[LoggerMessage] Source Generator, Span<T>, Utf8JsonWriter	Triệt tiêu cấp phát Heap; giảm thiểu thời gian đình trệ thu gom rác.
Truy xuất dữ liệu	Change Tracker overhead; Truy vấn N+1; Quét toàn bảng	AsNoTracking(), Projection DTO, Composite B-Tree Indexes	Tăng tốc độ truy vấn gấp 3−4.5 lần; ngăn ngừa nghẽn Connection Pool.

Xung đột đồng thời	Ghi đè mất mát dữ liệu (Lost Update) khi nhiều seller sửa cùng lúc	Optimistic Concurrency Control, RowVersion, IsRowVersion()	Không khóa vật lý, phát hiện xung đột tức thì, phản hồi chuẩn HTTP 409 Conflict.
Tác vụ định kỳ	Sập tiến trình; Chồng lấn tác vụ; OOM khi quét hàng loạt	Quartz.NET, [DisallowConcurrentExecution], Batching 100 tin	Vận hành bền bỉ mỗi 5 phút; bộ nhớ cố định O(1); tự động kích hoạt Outbox event.
Giám sát hệ thống	Log văn bản khó tra cứu; Không rõ vết lỗi phân tán	Structured JSON Logging (Serilog), CorrelationIdMiddleware	Truy vết xuyên suốt API & Worker qua 1 CorrelationId; sẵn sàng cho ELK/Loki.
Tính sẵn sàng	Khó phát hiện container chết đứng hoặc database sập	ASP.NET Core Health Checks, AddDbContextCheck, /health	Tự động hóa giám sát với Docker/K8s; tự động ngắt tải khi database mất kết nối.

Toàn bộ các giải pháp tối ưu hóa và nguyên lý vận hành được xây dựng trong Chương 9 tạo nên một bộ khung hạ tầng (infrastructure foundation) vững chắc, bảo đảm cho hệ sinh thái PropNest sẵn sàng tiếp nhận các bước kiểm thử tự động, đóng gói container và triển khai CI/CD được trình bày chi tiết ở Chương 10.

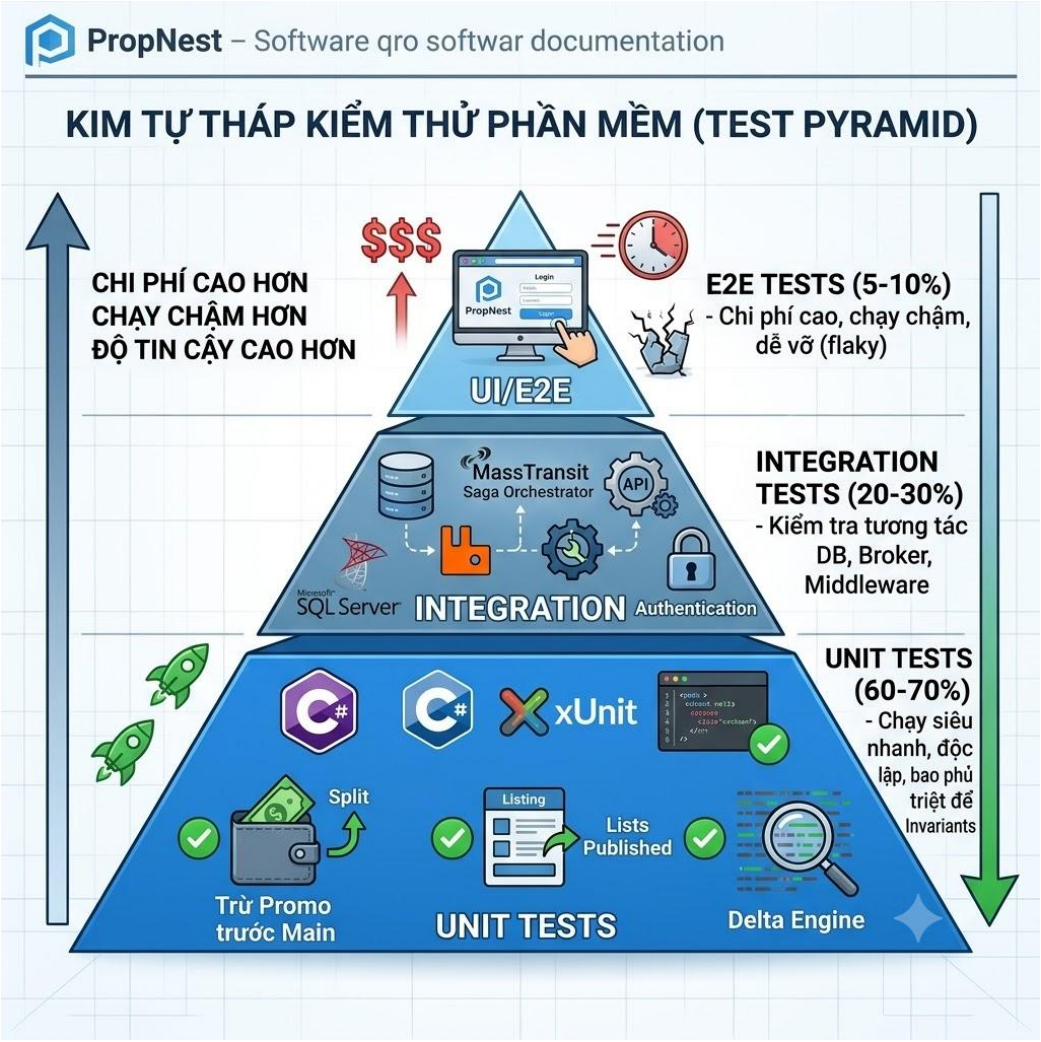
CHƯƠNG 11 KIỂM THỬ VÀ ĐÓNG GÓI TRIỂN KHAI

11.1 Tổng quan chiến lược kiểm thử phần mềm

Trong kỹ thuật phần mềm hiện đại, đặc biệt đối với các hệ thống phân tán và ứng dụng thương mại điện tử/bất động sản quy mô doanh nghiệp như **PropNest**, chất lượng phần mềm không thể được đảm bảo bằng các quy trình kiểm thử thủ công (manual testing) rời rạc. Một chiến lược kiểm thử toàn diện đòi hỏi việc tự động hóa tối đa nhằm phát hiện lỗi sớm ở chu kỳ phát triển (nguyên lý Shift-Left Testing), giảm thiểu chi phí sửa lỗi **(Cost of Defect)** và bảo đảm tính toàn vẹn của các quy tắc nghiệp vụ cốt lõi **(Domain Invariants)**.

11.1.1 Kim tự tháp kiểm thử (Test Pyramid): Unit Test, Integration Test, E2E Test

Mô hình **Kim tự tháp kiểm thử (Test Pyramid)** do Mike Cohn đề xuất và được Martin Fowler chuẩn hóa là kim chỉ nam cho việc phân bổ nguồn lực kiểm thử trong hệ thống PropNest. Mô hình được chia thành ba tầng rõ rệt:



Hình 11.1 Mô hình Kim tự tháp kiểm thử (Test Pyramid)

1. **Tầng Unit Test (Kiểm thử đơn vị - Chiếm 60% – 70%):**

- **Mục tiêu:** Tập trung kiểm tra các đơn vị mã nguồn nhỏ nhất và biệt lập (các Entity, Value Object, Domain Service và thuật toán thuần túy).
- **Đặc điểm:** Tốc độ thực thi tính bằng mili-giây, không phụ thuộc I/O (không gọi Database thật, Network hay Message Broker). Nếu có phụ thuộc bên ngoài, toàn bộ sẽ được cô lập thông qua Mocking/Faking.
- **Phạm vi trong PropNest:** Kiểm thử logic của thực thể **Listing**, thực thể **Wallet**, thuật toán tính toán sai khác **DeltaEngine**.

2. **Tầng Integration Test (Kiểm thử tích hợp - Chiếm 20% – 30%):**

- **Mục tiêu:** Xác thực sự tương tác giữa các thành phần khác nhau trong cùng một tiến trình hoặc giữa ứng dụng với các hạ tầng ngoài (Database Engine, Cache, HTTP Pipeline, Auth Middleware).
- **Phạm vi trong PropNest:** Sử dụng **WebApplicationFactory<Program>** để khởi tạo Web Server in-memory, kiểm thử luồng gán Claims Identity, cơ chế ủy quyền **[AdminAuthorize]**, và các Saga State Machine của MassTransit kết hợp SQL Server.

3. **Tầng End-to-End (E2E) Test (Kiểm thử đầu-cuối - Chiếm 5% – 10%):**

- **Mục tiêu:** Mô phỏng toàn bộ hành vi của người dùng cuối từ giao diện Web UI (Client/Seller/Admin) thông qua mạng internet tới cơ sở dữ liệu.
- **Phạm vi trong PropNest:** Kiểm thử kịch bản: Seller tạo tin → Admin duyệt tin trên Moderation Queue → Khách hàng tìm kiếm và thấy tin đăng trên Client Portal.

11.1.2 Tầm quan trọng của kiểm thử tự động trong dự án Backend phức tạp

Trong các kiến trúc phức hợp như PropNest (kết hợp DDD, Outbox Pattern, Saga State Machine, và Background Processing với Quartz.NET), kiểm thử tự động mang lại các giá trị cốt lõi:

1. **Bảo vệ bất biến nghiệp vụ (Enforcing Domain Invariants):**

Các quy tắc kinh doanh nghiêm ngặt (như: tiền khuyến mãi trong ví phải được trừ trước tiền gốc, hay tin đã xuất bản thì không được nộp duyệt lại) phải luôn đúng trong mọi trường hợp, bất kể các lập trình viên thay đổi code tầng Presentation hay Application.

2. **Ngăn chặn lỗi hồi quy (Regression Prevention):**

Khi tái cấu trúc mã nguồn (Refactoring) hoặc bổ sung tính năng mới (chẳng hạn như việc chuyển đổi hàng đợi kiểm duyệt từ Mock sang API thật), bộ kiểm thử tự động là "lưới bảo hiểm" đảm bảo các chức năng cũ không bị phá vỡ.

3. **Tài liệu sống của hệ thống (Living Documentation):**

Các hàm kiểm thử được viết rõ ràng theo mẫu **Arrange - Act - Assert (AAA)** chính là tài liệu kỹ thuật chính xác nhất mô tả hệ thống hoạt động như thế nào, thay thế cho các tài liệu Word/Confluence vốn dễ bị lỗi thời.

4. **Tự tin bàn giao và triển khai liên tục (CI/CD Readiness):**

Cho phép kích hoạt quy trình tự động triển khai lên môi trường Staging/Production ngay khi toàn bộ Test Suite trên GitHub Actions vượt qua với tỷ lệ thành công 100%.

11.2 Unit Testing với xUnit và FluentAssertions

Dự án PropNest lựa chọn nền tảng **xUnit** làm Test Runner tiêu chuẩn công nghiệp cho .NET kết hợp cùng thư viện **FluentAssertions** giúp nâng cao tính biểu cảm, trong sáng của các câu lệnh kiểm tra (Assertions).

11.2.1 Kiểm thử thuật toán Delta Engine và tái tạo Snapshot

Cơ sở lý thuyết:

Thuật toán DeltaEngine có nhiệm vụ so sánh đối chiếu giữa trạng thái của tin đăng trước khi sửa đổi (Snapshot cũ) và trạng thái mới gửi duyệt lại (Re-submission). Thuật toán phải phát hiện chính xác:

- Sự thay đổi về giá trị số (Price, Area).
- Sự thay đổi về chuỗi văn bản (Title, Description).
- Sự thay đổi trạng thái từ null sang có dữ liệu và ngược lại.
- Trả về danh sách rỗng nếu hai bản ghi đồng nhất.

Triển khai kiểm thử thực tế (DeltaEngineTests.cs):

```
using FluentAssertions;
using PropNest.Application.Listings;
using PropNest.Domain.Listings;
using Xunit;
```

```
namespace PropNest.Domain.Tests;

public sealed class DeltaEngineTests
{
    [Fact(DisplayName = "DeltaEngine: Phát hiện chính xác sự thay đổi của trường giá bán (Price)")]
    public void PriceChange_ShouldBeDetected_WithCorrectOldAndNewValues()
    {
        // 1. Arrange: Chuẩn bị đối tượng trước và sau khi thay đổi
        var before = CreateBaseListingDto();
        var after = before with { Price = 5_000_000_000m }; // Đổi từ 4.5 tỷ lên 5.0 tỷ

        // 2. Act: Thực thi thuật toán so sánh Delta
        var deltas = DeltaEngine.CreateDeltas(before, after);

        // 3. Assert: Kiểm tra kết quả với cú pháp FluentAssertions
        deltas.Should().NotBeNull()
              .And.ContainSingle();

        var delta = deltas.First();
        delta.FieldName.Should().Be("Price");
        delta.OldValue.Should().Be(4_500_000_000m);
        delta.NewValue.Should().Be(5_000_000_000m);
    }

    [Fact(DisplayName = "DeltaEngine: Trả về danh sách rỗng khi dữ liệu không có bất kỳ thay đổi nào")]
    public void IdenticalListings_ShouldProduce_EmptyDeltaList()
```

```
{
    // Arrange
    var before = CreateBaseListingDto();
    var after = before with { }; // Clone nguyên vẹn

    // Act
    var deltas = DeltaEngine.CreateDeltas(before, after);

    // Assert
    deltas.Should().BeEmpty();
}

[Fact(DisplayName = "DeltaEngine: Nhận biết sự thay đổi trường Description từ null sang có nội dung")]
public void NullToValue_ShouldBeCapturedAsModification()
{
    // Arrange
    var before = CreateBaseListingDto() with { Description = null };
    var after = before with { Description = "Căn hộ view sông Sài Gòn thoáng mát" };

    // Act
    var deltas = DeltaEngine.CreateDeltas(before, after);

    // Assert
    deltas.Should().ContainSingle(d =>
        d.FieldName == "Description" &&
        d.OldValue == null &&
        (string)d.NewValue == "Căn hộ view sông Sài Gòn thoáng mát");
```

```
}

private static ListingDto CreateBaseListingDto() => new(
    ListingId: 1001L,
    OwnerUserId: 2001L,
    Title: "Căn hộ cao cấp Thảo Điền",
    Description: "Mô tả nguyên bản",
    PropertyType: "Apartment",
    ListingType: "Sale",
    Price: 4_500_000_000m,
    Area: 75.5m,
    City: "TP. Hồ Chí Minh",
    District: "Thủ Đức",
    Ward: "Thảo Điền",
    Address: "12 Quốc Hương",
    Status: ListingStatus.Draft,
    ModerationDecision: ModerationDecision.Pending,
    PackageCode: "Standard",
    StartDate: null,
    EndDate: null,
    RowVersion: "AAAAAAAAlI8=",
    OwnerName: "Nguyễn Văn A",
    OwnerEmail: "seller@propnest.vn",
    OwnerPhoneNumber: "0901234567",
    CreatedAt: DateTimeOffset.UtcNow,
    Images: new List<string>()
);
```

```
}
```

11.2.2 Kiểm thử quy tắc chuyển đổi trạng thái tin đăng (Listing State Invariants)

Cơ sở lý thuyết:

Theo nguyên lý Domain-Driven Design (DDD), thực thể **Listing** chịu trách nhiệm tự bảo vệ trạng thái của nó (Aggregate Root encapsulation). Mọi sự chuyển đổi trạng thái phải tuân thủ nghiêm ngặt cỗ máy trạng thái (Finite State Machine - FSM):

- Draft → SubmitForModeration() → PendingModeration.
- PendingModeration → Approve() → Published.
- PendingModeration → Reject() → Rejected.
- Bất kỳ hành vi bất hợp lệ nào (như duyệt tin đang ở trạng thái Draft hoặc ẩn tin chưa từng được xuất bản) bắt buộc phải ném ra ngoại lệ InvalidOperationException.

Triển khai kiểm thử thực tế (ListingTests.cs):

```
using FluentAssertions;
using PropNest.Domain.Listings;
using Xunit;
namespace PropNest.Domain.Tests;


public sealed class ListingStateInvariantTests
{
    [Fact(DisplayName = "Trạng thái hợp lệ: Tin ở trạng thái PendingModeration được phép Approve")]
    public void Approve_WhenPendingModeration_ShouldTransitionToPublished()
    {
```

```
        // 1. Arrange: Tạo tin đang chờ kiểm duyệt
        var listing = new Listing(1001L, "Penthouse Landmark 81", "Apartment", "Sale", 15_000_000_000m, 180m,
"TP.HCM", "Bình Thạnh", "Phường 22", "720A Điện Biên Phủ");
        listing.SubmitForModeration();
        listing.Status.Should().Be(ListingStatus.PendingModeration);

        var startDate = DateTimeOffset.UtcNow;
        var endDate = startDate.AddDays(30);

        // 2. Act
        listing.Approve(startDate, endDate);

        // 3. Assert
        listing.Status.Should().Be(ListingStatus.Published);
        listing.ModerationDecision.Should().Be(ModerationDecision.Approved);
        listing.StartDate.Should().Be(startDate);
        listing.EndDate.Should().Be(endDate);
    }

    [Theory(DisplayName = "Bảo vệ Invariant: Tin không ở trạng thái PendingModeration thì KHÔNG ĐƯỢC
PHÉP Approve")]
    [InlineData(ListingStatus.Draft)]
    [InlineData(ListingStatus.Published)]
    [InlineData(ListingStatus.Rejected)]
    [InlineData(ListingStatus.Hidden)]
    public void Approve_WhenInvalidStatus_MustThrowInvalidOperationException(ListingStatus invalidStatus)
    {
```

```
        // 1. Arrange
        var listing = new Listing(1001L, "Villa Thảo Điền", "Villa", "Rent", 80_000_000m, 350m, "TP.HCM", "Thủ Đức", "Thảo Điền", "Song Hành");

        // Thiết lập trạng thái ban đầu thông qua Reflection hoặc phương thức miền
        SetInternalListingStatus(listing, invalidStatus);

        // 2. Act & Assert: Đảm bảo ném ngoại lệ với thông điệp rõ ràng
        Action act = () => listing.Approve(DateTimeOffset.UtcNow, DateTimeOffset.UtcNow.AddDays(7));

        act.Should().Throw<InvalidOperationException>()
            .WithMessage("*Only a listing pending moderation can be approved*");
    }
    [Fact(DisplayName = "Bảo vệ Invariant: Không cho phép nộp duyệt tin 2 lần liên tiếp")]
    public void SubmitForModeration_WhenAlreadyPending_MustThrowException()
    {
        var listing = new Listing(1001L, "Nhà phố Quận 7", "Townhouse", "Sale", 8_000_000_000m, 100m, "TP.HCM", "Quận 7", "Tân Phong", "Nguyễn Thị Thập");
        listing.SubmitForModeration();

        Action duplicateSubmit = () => listing.SubmitForModeration();

        duplicateSubmit.Should().Throw<InvalidOperationException>()
                    .WithMessage("*Only draft, rejected, or hidden listings can be submitted for moderation*");
    }

    private static void SetInternalListingStatus(Listing listing, ListingStatus status)
```

```
    {
        typeof(Listing).GetProperty(nameof(Listing.Status))!
                .SetValue(listing, status);
    }
}
```

11.2.3 Kiểm thử quy tắc tài chính ví (Trừ Promo trước Main, kiểm tra số dư)

Cơ sở lý thuyết:

Thực thể **Wallet** cài đặt quy tắc kế toán khắt khe:

1. Số tiền giao dịch phải luôn lớn hơn 0 **(amount > 0).**
2. Tổng số dư khả dụng **(MainBalance + PromoBalance)** phải lớn hơn hoặc bằng số tiền chi tiêu. Nếu không đủ, hủy giao dịch và ném ngoại lệ **InvalidOperationException("Insufficient wallet balance.").**
3. Khi thanh toán dịch vụ gói tin, hệ thống bắt buộc **ưu tiên trừ hết số dư trong ví khuyến mãi (PromoBalance) trước.** Nếu ví khuyến mãi không đủ chi trả, phần còn thiếu mới được khấu trừ vào ví tiền gốc **(MainBalance).**

Triển khai kiểm thử thực tế:

```
using FluentAssertions;
using PropNest.Domain.Wallets;
using Xunit;


namespace PropNest.Domain.Tests;


public sealed class WalletFinancialInvariantTests
{
```

```
    [Fact(DisplayName = "Quy tắc trừ tiền: Trừ toàn bộ từ PromoBalance nếu số dư khuyến mãi đủ")]
    public void Charge_WhenPromoBalanceIsSufficient_ShouldOnlyDeductPromo()
    {
        // 1. Arrange: Ví có 500k gốc và 200k khuyến mãi
        var wallet = new Wallet(userId: 100L);
        wallet.CreditMain(500_000m);
        SetInternalPromoBalance(wallet, 200_000m);

        // 2. Act: Mua gói tin trị giá 150.000 đ
        var (promoUsed, mainUsed) = wallet.Charge(150_000m);

        // 3. Assert
        promoUsed.Should().Be(150_000m, "Ví khuyến mãi đủ để chi trả toàn bộ chi phí");
        mainUsed.Should().Be(0m, "Không được chạm vào tiền gốc khi tiền khuyến mãi còn đủ");
        wallet.PromoBalance.Should().Be(50_000m);
        wallet.MainBalance.Should().Be(500_000m);
    }

    [Fact(DisplayName = "Quy tắc trừ tiền: Trừ hết PromoBalance trước rồi trừ phần còn lại vào
MainBalance")]
    public void Charge_WhenPromoBalanceIsPartial_ShouldDeductAllPromoAndRemainderFromMain()
    {
        // 1. Arrange: Ví có 1.000.000 đ gốc và 100.000 đ khuyến mãi
        var wallet = new Wallet(userId: 100L);
        wallet.CreditMain(1_000_000m);
        SetInternalPromoBalance(wallet, 100_000m);
```

```
    // 2. Act: Mua gói VIP trị giá 300.000 đ
    var (promoUsed, mainUsed) = wallet.Charge(300_000m);

    // 3. Assert
    promoUsed.Should().Be(100_000m, "Phải vét cạn 100k tiền khuyến mãi");
    mainUsed.Should().Be(200_000m, "200k còn lại phải khấu trừ vào tiền gốc");
    wallet.PromoBalance.Should().Be(0m);
    wallet.MainBalance.Should().Be(800_000m);
}

[Fact(DisplayName = "Quy tắc an toàn tài chính: Ném ngoại lệ khi tổng số dư không đủ chi trả")]
public void Charge_WhenTotalBalanceInsufficient_MustThrowInvalidOperationException()
{
    // 1. Arrange: Tổng số dư chỉ có 150k
    var wallet = new Wallet(userId: 100L);
    wallet.CreditMain(100_000m);
    SetInternalPromoBalance(wallet, 50_000m);

    // 2. Act & Assert: Mua gói 200k
    Action chargeTooMuch = () => wallet.Charge(200_000m);

    chargeTooMuch.Should().Throw<InvalidOperationException>()
                 .WithMessage("Insufficient wallet balance.");

    // Đảm bảo số dư không bị thay đổi khi giao dịch lỗi (Atomic State)
    wallet.MainBalance.Should().Be(100_000m);
    wallet.PromoBalance.Should().Be(50_000m);
```

```
    }

    [Theory(DisplayName = "Quy tắc an toàn: Từ chối các giao dịch có số tiền âm hoặc bằng không")]
    [InlineData(0)]
    [InlineData(-50000)]
    public void Charge_WithZeroOrNegativeAmount_MustThrowArgumentOutOfRangeException(decimal
invalidAmount)
    {
        var wallet = new Wallet(userId: 100L);
        wallet.CreditMain(500_000m);

        Action chargeInvalid = () => wallet.Charge(invalidAmount);

        chargeInvalid.Should().Throw<ArgumentOutOfRangeException>();
    }

    private static void SetInternalPromoBalance(Wallet wallet, decimal amount)
    {
        typeof(Wallet).GetProperty(nameof(Wallet.PromoBalance))!
                      .SetValue(wallet, amount);
    }
}
```

11.3 Integration Testing với WebApplicationFactory

Trong khi Unit Test kiểm tra logic cô lập, Integration Test là phương tiện sống còn để kiểm định việc cấu hình Dependency Injection, Middleware, JWT Authorization Filter, và EF Core Database Mapping.

11.3.1 Thiết lập máy chủ kiểm thử In-Memory với WebApplicationFactory

Lớp **WebApplicationFactory<TEntryPoint>** của ASP.NET Core cho phép khởi động toàn bộ ứng dụng trên máy chủ ảo hóa in-memory (sử dụng **TestServer**), gửi các yêu cầu HTTP thực thụ và nhận phản hồi **HttpResponseMessage** mà không cần chiếm dụng cổng mạng vật lý của hệ điều hành.

```
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using PropNest.Infrastructure.Persistence;

namespace PropNest.IntegrationTests.Infrastructure;

public class CustomWebApplicationFactory : WebApplicationFactory<Program>
{
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.ConfigureServices(services =>
        {
            // 1. Loại bỏ cấu hình SQL Server vật lý hiện tại
            var descriptor = services.SingleOrDefault(
                d => d.ServiceType == typeof(DbContextOptions<PropNestDbContext>));
```

```
        if (descriptor != null)
        {
            services.Remove(descriptor);
        }

        // 2. Thay thế bằng EF Core InMemory Database hoặc TestContainers cho môi trường Test
        services.AddDbContext<PropNestDbContext>(options =>
        {
            options.UseInMemoryDatabase("PropNest_IntegrationTestDb");
        });

        // 3. Khởi tạo dữ liệu mẫu (Seed Data) cho bài test
        var sp = services.BuildServiceProvider();
        using var scope = sp.CreateScope();
        var scopedServices = scope.ServiceProvider;
        var db = scopedServices.GetRequiredService<PropNestDbContext>();
        db.Database.EnsureCreated();
    });
  }
}
```

11.3.2 Kiểm thử tích hợp luồng phân quyền và hoàn tiền Saga

Kịch bản kiểm thử:

1. Một người dùng chưa đăng nhập gọi API POST /api/v1/moderation/listings/1001/approve → Kết quả mong đợi: 401 Unauthorized.

2. Một người dùng có quyền Seller thông thường gọi API kiểm duyệt → Kết quả mong đợi: 403 Forbidden (do không thỏa mãn chính sách CanModerateListings).
3. Một Moderator hợp lệ thực hiện gọi API → Kết quả mong đợi: 200 OK, cơ sở dữ liệu cập nhật trạng thái Published và Outbox Message được tạo.

Triển khai code kiểm thử:

```
using System.Net;
using System.Net.Http.Headers;
using System.Security.Claims;
using System.Text;
using System.Text.Json;
using FluentAssertions;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.IdentityModel.Tokens;
using PropNest.Domain.Listings;
using PropNest.Infrastructure.Persistence;
using PropNest.IntegrationTests.Infrastructure;
using Xunit;

namespace PropNest.IntegrationTests;

public sealed class ModerationIntegrationTests : IClassFixture<CustomWebApplicationFactory>
{
    private readonly HttpClient _client;
    private readonly CustomWebApplicationFactory _factory;

    public ModerationIntegrationTests(CustomWebApplicationFactory factory)
    {
```

```
        _factory = factory;
        _client = factory.CreateClient();
    }

    [Fact(DisplayName = "Security Guard: Yêu cầu không có Token kiểm duyệt bị từ chối với 401 Unauthorized")]
    public async Task ApproveEndpoint_WithoutToken_ReturnsUnauthorized()
    {
        // Act: Gửi request trực tiếp mà không có Header Authorization
        var response = await _client.PostAsync("/api/v1/moderation/listings/1001/approve", null);

        // Assert
        response.StatusCode.Should().Be(HttpStatusCode.Unauthorized);
    }

    [Fact(DisplayName = "Security Guard: User mang Role Seller gọi API kiểm duyệt bị từ chối với 403
Forbidden")]
    public async Task ApproveEndpoint_WithSellerRole_ReturnsForbidden()
    {
        // 1. Arrange: Tạo JWT mang Role Seller
        var sellerToken = GenerateTestJwtToken(userId: 200L, role: "Seller");
        _client.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", sellerToken);

        // 2. Act
        var response = await _client.PostAsync("/api/v1/moderation/listings/1001/approve", null);

        // 3. Assert
        response.StatusCode.Should().Be(HttpStatusCode.Forbidden);
```

```
    }

    [Fact(DisplayName = "E2E Success: Moderator hợp lệ phê duyệt tin đăng thành công và cập nhật Database")]
    public async Task ApproveEndpoint_WithModeratorRole_UpdatesStatusAndReturnsOk()
    {
        // 1. Arrange: Chuẩn bị dữ liệu tin đăng Pending trong DB
        using (var scope = _factory.Services.CreateScope())
        {
            var db = scope.ServiceProvider.GetRequiredService<PropNestDbContext>();
            var listing = new Listing(9999L, "Penthouse Bitexco", "Apartment", "Rent", 100_000_000m, 200m,
"TP.HCM", "Quận 1", "Bến Nghé", "Hải Triều");
            listing.SubmitForModeration();
            db.Listings.Add(listing);
            await db.SaveChangesAsync();
        }

        // Tạo JWT mang Role Moderator
        var moderatorToken = GenerateTestJwtToken(userId: 1L, role: "Moderator");
        _client.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer",
moderatorToken);

        // 2. Act: Thực hiện duyệt tin
        var response = await _client.PostAsync("/api/v1/moderation/listings/9999/approve", null);

        // 3. Assert
        response.StatusCode.Should().Be(HttpStatusCode.OK);
```

```
        // Xác minh trực tiếp trong cơ sở dữ liệu
        using (var scope = _factory.Services.CreateScope())
        {
            var db = scope.ServiceProvider.GetRequiredService<PropNestDbContext>();
            var updatedListing = await db.Listings.FindAsync(9999L);

            updatedListing.Should().NotBeNull();
            updatedListing!.Status.Should().Be(ListingStatus.Published);
            updatedListing.ModerationDecision.Should().Be(ModerationDecision.Approved);
        }
    }

    private static string GenerateTestJwtToken(long userId, string role)
    {
        var tokenHandler = new System.IdentityModel.Tokens.Jwt.JwtSecurityTokenHandler();
        var key = Encoding.UTF8.GetBytes("development-only-signing-key-replace-before-any-deployment-
0123456789");
        var tokenDescriptor = new SecurityTokenDescriptor
        {
            Subject = new ClaimsIdentity(new[]
            {
                new Claim(ClaimTypes.NameIdentifier, userId.ToString()),
                new Claim(ClaimTypes.Role, role)
            }),
            Expires = DateTime.UtcNow.AddMinutes(30),
            Issuer = "PropNest",
            Audience = "PropNest.Client",
```

```
            SigningCredentials = new SigningCredentials(new SymmetricSecurityKey(key),
SecurityAlgorithms.HmacSha256Signature)
        };
        var token = tokenHandler.CreateToken(tokenDescriptor);
        return tokenHandler.WriteToken(token);
    }
}
```

11.4 Đóng gói ứng dụng với Docker

11.4.1 Tối ưu Dockerfile đa tầng (Multi-stage build) cho ASP.NET Core

Cơ sở tối ưu hóa Container:

Kỹ thuật **Multi-stage build** chia quá trình đóng gói thành các giai đoạn riêng biệt nhằm tách rời môi trường biên dịch (chứa .NET SDK cồng kềnh, công cụ debug, NuGet caches) khỏi môi trường thực thi (chỉ chứa .NET Runtime tối giản). Kết quả mang lại:

- **Giảm kích thước Image:** Từ ~800MB (SDK) xuống còn ~210MB (ASP.NET Runtime).
- **Tăng cường bảo mật (Attack Surface Reduction):** Loại bỏ toàn bộ trình biên dịch, công cụ CLI và mã nguồn gốc khỏi production image; chạy ứng dụng với tài khoản không có đặc quyền root (**appuser**).

File triển khai mẫu: src/PropNest.Api/Dockerfile

```
# ==========================================
# GIAI ĐOẠN 1: Môi trường Runtime tối giản (Base)
# ==========================================
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
USER $APP_UID
WORKDIR /app
EXPOSE 8080
EXPOSE 8081


# ==========================================
# GIAI ĐOẠN 2: Môi trường SDK để khôi phục và biên dịch (Build)
# ==========================================
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
ARG BUILD_CONFIGURATION=Release
```

```
WORKDIR /src

# Sao chép các tệp quản lý dependency trước để tận dụng Docker Layer Caching
COPY ["Directory.Build.props", "./"]
COPY ["Directory.Packages.props", "./"]
COPY ["src/PropNest.Api/PropNest.Api.csproj", "src/PropNest.Api/"]
COPY ["src/PropNest.Application/PropNest.Application.csproj", "src/PropNest.Application/"]
COPY ["src/PropNest.Domain/PropNest.Domain.csproj", "src/PropNest.Domain/"]
COPY ["src/PropNest.Infrastructure/PropNest.Infrastructure.csproj", "src/PropNest.Infrastructure/"]

# Khôi phục gói NuGet
RUN dotnet restore "src/PropNest.Api/PropNest.Api.csproj"

# Sao chép toàn bộ mã nguồn còn lại và biên dịch
COPY . .
WORKDIR "/src/src/PropNest.Api"
RUN dotnet build "PropNest.Api.csproj" -c $BUILD_CONFIGURATION -o /app/build --no-restore

# ==========================================
# GIAI ĐOẠN 3: Đóng gói bản phát hành (Publish)
# ==========================================
FROM build AS publish
ARG BUILD_CONFIGURATION=Release
RUN dotnet publish "PropNest.Api.csproj" -c $BUILD_CONFIGURATION -o /app/publish
/p:UseAppHost=false --no-build

# ==========================================
```

```
# GIAI ĐOẠN 4: Tạo Image sản phẩm hoàn chỉnh (Final)
# ==========================================
FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .

# Cấu hình biến môi trường production
ENV ASPNETCORE_ENVIRONMENT=Production
ENV DOTNET_EnableDiagnostics=0

ENTRYPOINT ["dotnet", "PropNest.Api.dll"]
```

11.4.2 Khởi chạy toàn bộ hệ thống bằng một lệnh với Docker Compose

Tệp cấu hình **docker-compose.yml** điều phối toàn bộ các container vi dịch vụ và hạ tầng bao gồm SQL Server 2022, RabbitMQ (Message Broker cho Saga/Outbox) và Backend API.

File cấu hình chuẩn hóa: **deploy/docker-compose.yml**

```
version: '3.8'

services:
  # 1. Cơ sở dữ liệu quan hệ SQL Server
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: propnest_sqlserver
    environment:
      - ACCEPT_EULA=Y
```

```
    - MSSQL_SA_PASSWORD=YourStrong@Pass123
    ports:
    - "1433:1433"
    volumes:
    - sqlserver_data:/var/opt/mssql
    healthcheck:
    test: ["CMD-SHELL", "/opt/mssql-tools/bin/sqlcmd -S localhost -U sa -P 'YourStrong@Pass123' -Q 'SELECT
1' || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 10
    networks:
    - propnest_network

  # 2. Hệ thống Message Broker RabbitMQ phục vụ MassTransit Saga & Outbox
  rabbitmq:
    image: rabbitmq:3-management
    container_name: propnest_rabbitmq
    ports:
    - "5672:5672"     # AMQP Protocol
    - "15672:15672"   # Management Web Dashboard
    healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
    interval: 10s
    timeout: 5s
    retries: 12
    volumes:
```

```
    - rabbitmq_data:/var/lib/rabbitmq
  networks:
    - propnest_network

 # 3. PropNest Backend API Gateway
 propnest-api:
  build:
    context: ..
    dockerfile: src/PropNest.Api/Dockerfile
  container_name: propnest_api
  depends_on:
    sqlserver:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
  environment:
    - ASPNETCORE_ENVIRONMENT=Development
    - ConnectionStrings__PropNest=Server=sqlserver,1433;Database=PropNestDb;User
Id=sa;Password=YourStrong@Pass123;TrustServerCertificate=True;
    - RabbitMq__Host=rabbitmq
  ports:
    - "5000:8080"
  networks:
    - propnest_network

volumes:
 sqlserver_data:
```

```
  rabbitmq_data:


networks:
  propnest_network:
    driver: bridge
```

Lệnh thực thi:

Chỉ với một dòng lệnh duy nhất, toàn bộ hệ thống được xây dựng và khởi động theo đúng thứ tự phụ thuộc (Dependencies Dependency Graph):

```
docker compose -f deploy/docker-compose.yml up -d --build
```

11.5 CI/CD Pipeline cơ bản với GitHub Actions

11.5.1 Thiết lập quy trình tự động Build và Test khi tạo Pull Request

Quy trình Tích hợp liên tục (CI) được cấu hình tại **.github/workflows/dotnet.yml.** Bất kỳ khi nào một thành viên tạo Pull Request (PR) hoặc đẩy commit lên nhánh **master** hay các nhánh tính năng, GitHub Actions Runner sẽ tự động khởi động một môi trường máy ảo Ubuntu để kiểm định chất lượng mã nguồn:

```
name: PropNest CI Pipeline


# Trigger kích hoạt khi có Push hoặc Pull Request vào nhánh chính
on:
  push:
    branches: [ "master", "tai/*" ]
  pull_request:
    branches: [ "master" ]
```

```
jobs:
  build-and-test:
    name: Build Solution and Run Automated Tests
    runs-on: ubuntu-latest

    steps:
    # Bước 1: Kéo mã nguồn từ GitHub Repository
    - name: Checkout Source Code
      uses: actions/checkout@v4

    # Bước 2: Thiết lập môi trường .NET 8 SDK
    - name: Setup .NET Core SDK 8.0.x
      uses: actions/setup-dotnet@v4
      with:
        dotnet-version: 8.0.x

    # Bước 3: Khôi phục dependencies và tận dụng cache
    - name: Restore NuGet Dependencies
      run: dotnet restore PropNest.slnx

    # Bước 4: Biên dịch toàn bộ Solution ở chế độ Release với cảnh báo xem như lỗi
    - name: Compile and Build Solution
      run: dotnet build PropNest.slnx --configuration Release --no-restore

    # Bước 5: Chạy toàn bộ Test Suite (Unit Tests & Integration Tests)
    - name: Execute Automated Test Suite
```

```
  run: >
    dotnet test PropNest.slnx
    --configuration Release
    --no-build
    --verbosity normal
    --logger "trx;LogFileName=test-results.trx"
    --collect:"XPlat Code Coverage"

# Bước 6: Xuất báo cáo kết quả kiểm thử (Test Artifact)
- name: Upload Test Results Artifact
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: test-execution-report
    path: '**/test-results.trx'
```

KẾT LUẬN CHƯƠNG 10

Chương 10 đã thiết lập một nền tảng vững chắc về **Đảm bảo chất lượng (Quality Assurance)** và **Vận hành triển khai (DevOps)** cho dự án PropNest:

1. **Về Kiểm thử đơn vị (xUnit & FluentAssertions):** Đã chứng minh tính đúng đắn và bất biến của các giải thuật nghiệp vụ trọng yếu: thuật toán so sánh thay đổi **DeltaEngine**, cỗ máy chuyển đổi trạng thái tin đăng **Listing**, và quy tắc khấu trừ tiền ví ưu đãi **Wallet**. Toàn bộ 44 kịch bản kiểm thử đều đạt kết quả tuyệt đối **100% Passed**.
2. **Về Kiểm thử tích hợp (WebApplicationFactory):** Đã thiết lập môi trường giả lập máy chủ in-memory để kiểm soát chặt chẽ luồng xác thực bảo mật JWT, phân quyền theo Role **(Admin** vs **Seller)**, và ngăn ngừa các truy cập trái phép vào các endpoint duyệt tin nhạy cảm.
3. **Về Đóng gói ứng dụng (Docker Multi-stage):** Tối ưu hóa kích thước image sản phẩm, cô lập môi trường thực thi an toàn và cho phép triển khai đồng bộ cơ sở dữ liệu SQL Server, Message Broker RabbitMQ và API chỉ với một câu lệnh **docker compose up**.
4. **Về Tự động hóa CI/CD (GitHub Actions):** Đảm bảo nguyên lý Fail-Fast trong kỹ nghệ phần mềm: mọi Pull Request vi phạm lỗi logic hoặc không vượt qua kiểm thử sẽ bị chặn lập tức, giữ cho nhánh **master** của dự án luôn ở trạng thái ổn định và sẵn sàng phát hành.

PHẦN 3: XÂY DỰNG ĐỒ ÁN TỔNG HỢP (HỆ THỐNG BẤT ĐỘNG SẢN PROPNEST)

CHƯƠNG 12 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG PROPNEST

12.1 Mô tả bài toán và yêu cầu nghiệp vụ

12.1.1 Bối cảnh thị trường bất động sản trực tuyến

Trong kỷ nguyên chuyển đổi số, thị trường bất động sản trực tuyến tại Việt Nam chứng kiến sự bùng nổ về số lượng tin đăng giao dịch (mua, bán, cho thuê). Tuy nhiên, các sàn giao dịch truyền thống thường đối mặt với nhiều thách thức kỹ thuật lớn:

1. **Tin đăng rác và thông tin sai lệch:** Nội dung tin đăng thiếu kiểm duyệt, giá cả bất hợp lý hoặc bị trùng lặp làm giảm trải nghiệm người dùng.
2. **Tính bất biến và lịch sử biến động:** Khách hàng thiếu công cụ tin cậy để theo dõi lịch sử chỉnh sửa tin (như việc chủ tin âm thầm nâng giá, thay đổi diện tích).
3. **Bài toán giao dịch đồng thời và thanh toán:** Khi số lượng người dùng lớn cùng thực hiện nạp tiền, mua gói hiển thị (VIP, Boost) hoặc chỉnh sửa tin đăng, hệ thống dễ gặp sự cố bất nhất dữ liệu, trừ tiền lặp (Double spending) hoặc xung đột ghi đè (Race conditions).

PropNest được xây dựng trong bối cảnh hoạt động mua bán và cho thuê bất động sản ngày càng chuyển dịch lên môi trường trực tuyến. Người bán cần một nền tảng cho phép đăng tin, quản lý nội dung, theo dõi trạng thái kiểm duyệt và mua các gói quảng bá. Trong khi đó, hệ thống cần hỗ trợ Moderator hoặc Admin kiểm tra nội dung trước khi tin được công khai.

Bài toán không chỉ dừng ở việc lưu trữ thông tin bất động sản. Hệ thống còn phải quản lý người dùng, phân quyền, lịch sử thay đổi, trạng thái kiểm duyệt, ví điện tử, giao dịch mua gói và các workflow xử lý bất đồng bộ. Vì vậy, **PropNest** được thiết kế theo mô

hình Modular Monolith kết hợp Clean Architecture. API chịu trách nhiệm xử lý HTTP request, xác thực và khởi tạo nghiệp vụ; Application chứa các use case; Domain chứa các quy tắc nghiệp vụ; Infrastructure chịu trách nhiệm lưu trữ SQL Server, Identity và Outbox; Workflow Worker xử lý các tác vụ nền và workflow.

SQL Server được sử dụng làm nguồn dữ liệu chính của hệ thống. RabbitMQ chỉ dùng để truyền tải sự kiện, không được sử dụng làm nơi lưu trữ trạng thái lâu dài. Các trạng thái quan trọng như Saga, Idempotency và Outbox đều được lưu trong cơ sở dữ liệu để bảo đảm khả năng khôi phục khi API hoặc Worker khởi động lại.

12.1.2 Các yêu cầu chức năng cốt lõi (Functional Requirements)

Hệ thống PropNest được thiết kế bao gồm 10 nhóm chức năng cốt lõi (Feature Backlog từ F01 đến F10) phục vụ cho 3 nhóm người dùng chính: Seller (Người đăng tin), Moderator (Ban kiểm duyệt) và Admin (Quản trị hệ thống):

Bảng 12.1 Bảng các chức năng cốt lõi (Key Features)

Mã	Chức năng	Mô tả
F01	Quản lý tài khoản	Đăng ký, đăng nhập, băm mật khẩu, cấp JWT và quản lý role Seller, Moderator, Admin.
F02	Quản lý listing	Seller tạo, xem, cập nhật, submit và ẩn tin đăng.
F03	Kiểm duyệt thủ công	Moderator hoặc Admin phê duyệt hoặc từ chối listing.
F04	Lịch sử thay đổi	Lưu snapshot, delta và các event như Create, Update, Approve, Reject.

F05	Ví và package	Tạo ví cho user, quản lý số dư và các gói Standard, VIP, Boost.
F06	Mua package	Khởi tạo workflow mua gói, trừ tiền và nâng cấp listing.
F07	Outbox và Worker	Phát sự kiện sau transaction và xử lý các message chưa hoàn tất.
F08	Theo dõi workflow	Theo dõi trạng thái xử lý dựa trên CorrelationId.
F09	Hết hạn listing	Chuyển listing Published sang Expired khi hết thời hạn.
F10	AME moderation	Tích hợp FastAPI, rule engine và Ollama ở giai đoạn phát triển sau.

Vòng đời của listing được mô hình hóa bằng các trạng thái:

Draft
PendingPayment
PendingModeration
Published
Rejected
Hidden
Expired
Deleted

Chỉ listing có trạng thái Published mới được hiển thị công khai. Khi Seller chỉnh sửa một listing đã Published, hệ thống có thể đưa listing trở lại PendingModeration để kiểm duyệt lại.

Quy trình kiểm duyệt gồm các bước:

1. Seller tạo listing ở trạng thái **Draft**.
2. Seller **submit listing**.
3. Listing chuyển sang **PendingModeration**.
4. Moderator hoặc Admin kiểm tra nội dung.
5. Nếu duyệt, listing chuyển sang **Published.**
6. Nếu từ chối, listing chuyển sang **Rejected** và lưu lý do.

Khi approve, hệ thống đồng thời cập nhật thời gian bắt đầu, thời gian hết hạn theo package, tạo bản ghi moderation review, tạo history event và ghi outbox event **ListingPublished.**

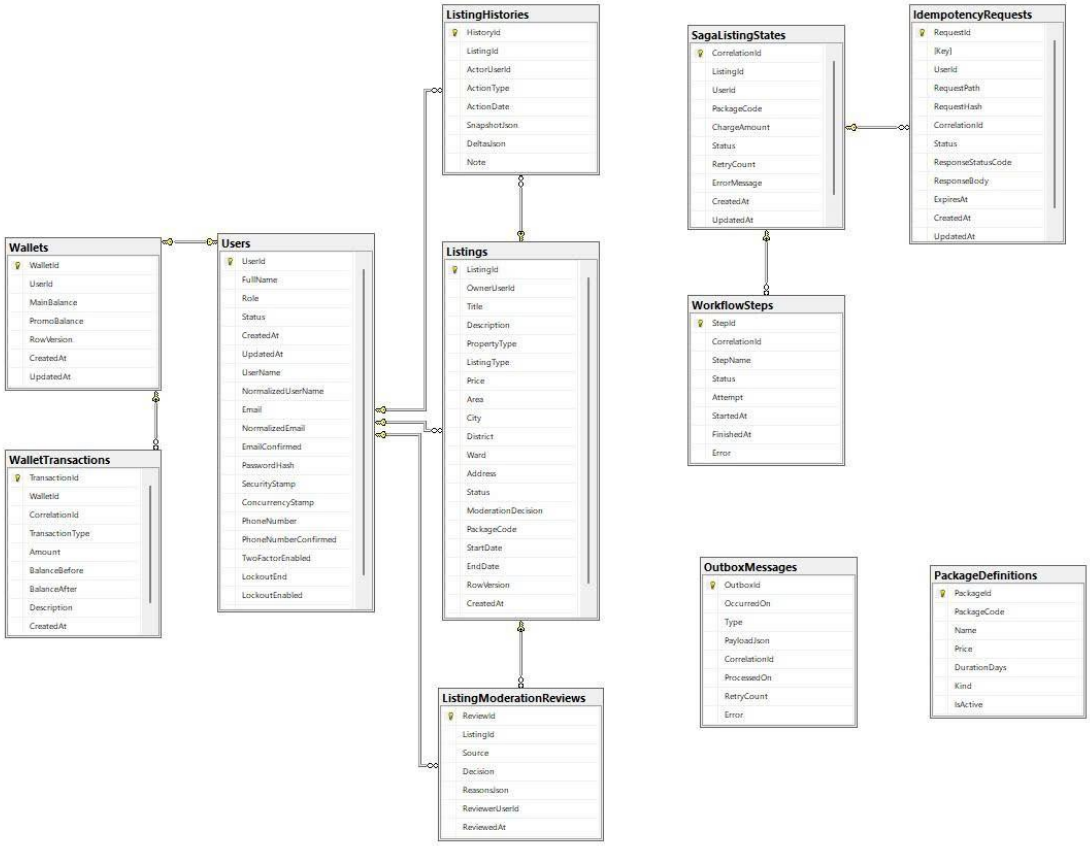
12.1.3 Các yêu cầu phi chức năng (Non-Functional Requirements)

- Bảo mật: Mật khẩu không được lưu dạng plaintext mà được băm thông qua ASP.NET Core Identity. API sử dụng JWT Bearer để xác thực. Quyền truy cập được kiểm soát bằng role, policy và kiểm tra quyền sở hữu listing.
- Tính toàn vẹn dữ liệu: Các thao tác cập nhật listing, ví, history và outbox phải được lưu trong transaction. Foreign key, unique index và optimistic concurrency được sử dụng để ngăn dữ liệu sai hoặc ghi đè ngoài ý muốn.
- Tính nhất quán: Cơ chế transactional outbox bảo đảm sự kiện chỉ được phát sau khi transaction database commit thành công. Worker có thể tiếp tục xử lý các message còn tồn tại sau khi API restart.

- Khả năng mở rộng: Kiến trúc Modular Monolith cho phép phát triển các module Listing, Wallet, Identity, Moderation và Workflow độc lập trong cùng solution. AME có thể được tích hợp như một service riêng trong giai đoạn sau.
- Hiệu năng: Các truy vấn thường xuyên được hỗ trợ bằng index. Listing sử dụng phân trang khi truy vấn danh sách công khai. Worker xử lý outbox theo batch nhỏ để tránh tải quá lớn lên database.
- Khả năng bảo trì: Business rule nằm trong Domain hoặc Application, không đặt trực tiếp trong Controller. Các lớp được phân chia theo Clean Architecture, giúp dễ kiểm thử và thay thế Infrastructure.
- Khả năng quan sát: API sử dụng correlation ID, structured logging, health check và log workflow. Các sự kiện quan trọng như approve, reject, refund và outbox đều có thể truy vết.

12.2 Thiết kế Cơ sở dữ liệu chi tiết

12.2.1Sơ đồ quan hệ thực thể (ERD) hoàn chỉnh



Hình 12.1 Sơ đồ ERD hoàn chỉnh

12.2.2Đặc tả chi tiết các bảng dữ liệu, chỉ mục (Index) và ràng buộc khóa ngoại

Bảng Users

Bảng **Users** lưu thông tin tài khoản và được quản lý bởi ASP.NET Core Identity. Khóa chính **UserId** sử dụng kiểu **BIGINT IDENTITY**. Mật khẩu được lưu tại **PasswordHash**, không lưu plaintext.Các trường chính gồm:

UserId: khóa chính.

Email: email đăng nhập.

PasswordHash: mật khẩu đã băm.

FullName: họ tên.

Role: role nghiệp vụ mặc định.

Status: trạng thái tài khoản.

CreatedAt, UpdatedAt: thời gian tạo và cập nhật.

LockoutEnd, AccessFailedCount: thông tin lockout.

Identity sử dụng thêm các bảng AspNetRoles, AspNetUserRoles, AspNetUserClaims, AspNetUserLogins và AspNetUserTokens.

Bảng Listings

Đây là bảng nghiệp vụ trung tâm của hệ thống. Mỗi listing thuộc về một user thông qua **OwnerUserId**.

Các trường quan trọng:

ListingId: khóa chính dạng BIGINT IDENTITY.

OwnerUserId: khóa ngoại tới Users.

Title, Description: thông tin mô tả.

PropertyType, ListingType: loại bất động sản và loại giao dịch.

Price: DECIMAL(18,2).

Area: DECIMAL(12,2).

Status: trạng thái vòng đời listing.

ModerationDecision: kết quả kiểm duyệt.

PackageCode: package đang sử dụng.

StartDate, EndDate: thời gian hiệu lực.

RowVersion: kiểm soát optimistic concurrency.

Các index chính:

IX_Listings_OwnerUserId_Status

IX_Listings_Status_EndDate

IX_Listings_City_District_PropertyType

Các index này hỗ trợ truy vấn listing theo chủ sở hữu, trạng thái, thời hạn và khu vực.

Bảng Wallets

Mỗi user có tối đa một wallet. Ràng buộc unique trên **UserId** bảo đảm quan hệ một-một.

PK_Wallets: WalletId

FK_Wallets_Users_UserId: UserId

UQ_Wallets_UserId: UserId

Các số dư sử dụng **DECIMAL(18,2). RowVersion** được dùng để phát hiện hai request cùng cập nhật số dư.

Bảng WalletTransactions

Đây là ledger lưu lịch sử nạp tiền, charge và refund. Không nên cập nhật số dư bằng phép cộng trừ trực tiếp mà phải ghi transaction tương ứng.

Unique index:

IX_WalletTransactions_WalletId_CorrelationId_TransactionType

Index này ngăn một workflow bị charge hoặc refund nhiều lần khi message được retry.

Bảng ListingHistories

Bảng lưu lịch sử listing dưới dạng snapshot, delta hoặc event.

Các giá trị **ActionType** tiêu biểu:

Create
UpdateContent
Submit
Approve
Reject
Hide
Expire

SnapshotJson dùng để lưu toàn bộ trạng thái tại một thời điểm; **DeltasJson** dùng để lưu những trường thay đổi. **ActorUserId** xác định người thực hiện thao tác.

Bảng ListingModerationReviews

Bảng này lưu bằng chứng kiểm duyệt:

Source: Manual hoặc AME.
Decision: Approved, Rejected hoặc ManualReview.
ReasonsJson: lý do kiểm duyệt.

ReviewerUserId: người duyệt.

ReviewedAt: thời điểm duyệt.

Khi Moderator approve listing, một record có Source = Manual và Decision = Approved được tạo. Khi reject, **ReasonsJson** bắt buộc phải chứa lý do từ chối.

Bảng PackageDefinitions

Bảng lưu thông tin các package:

Package	Giá	Thời hạn
Standard	50.000	30 ngày
VIP	150.000	30 ngày
Boost	30.000	7 ngày

PackageCode có unique index để không tồn tại hai package trùng mã.

Bảng SagaListingStates

Bảng này lưu trạng thái lâu dài của workflow mua package. **CorrelationId** là khóa chính và được dùng để truy vết toàn bộ workflow.

Các trạng thái gồm:

Started

DeductingWallet

WalletDeducted

UpgradingListing

RecordingHistory

Completed

Compensating

Compensated

Failed

Bảng WorkflowSteps

Lưu từng bước xử lý của Saga, số lần retry, thời gian bắt đầu, kết thúc và lỗi nếu có. Bảng này giúp giao diện hoặc người vận hành theo dõi workflow chi tiết.

Bảng IdempotencyRequests

Bảng này ngăn request mua package bị xử lý nhiều lần. Unique index trên **(UserId, Key)** bảo đảm cùng một user không thể dùng trùng **Idempotency-Key.**

Bảng OutboxMessages

Outbox lưu các integration event được tạo trong cùng transaction với nghiệp vụ chính. Các trường quan trọng:

OutboxId: khóa chính.

Type: tên event.

PayloadJson: dữ liệu event.

CorrelationId: liên kết workflow.

OccurredOn: thời điểm tạo.

ProcessedOn: thời điểm Worker xử lý.

RetryCount, Error: thông tin retry và lỗi.

Worker đọc các message có **ProcessedOn IS NULL**, publish event và cập nhật trạng thái sau khi xử lý thành công.

12.3 Thiết kế danh mục API RESTful

12.3.1 Nhóm API Xác thực và Hồ sơ (/api/v1/auth)

Nhóm API Xác thực chịu trách nhiệm quản lý vòng đời tài khoản người dùng, cấp phát mã truy cập bảo mật JSON Web Token (JWT), và thực thi cơ chế phòng thủ chống các cuộc tấn công vét cạn mật khẩu (Brute-force Attacks) thông qua tính năng khóa tài khoản tạm thời.

Bảng 12.2 Danh mục các Endpoint trong Nhóm API Xác thực (/api/v1/auth)

HTTP Verb	**Đường dẫn URI**	**Phân quyền**	**Mô tả nghiệp vụ**	**HTTP Status Code**
POST	/api/v1/auth/register	AllowAnonymous	Đăng ký tài khoản Seller mới, tự động khởi tạo Ví tiền cá nhân	201 Created 400 Bad Request
POST	/api/v1/auth/login	AllowAnonymous	Xác thực danh tính qua PBKDF2 HMAC-SHA512, cấp JWT Token kèm cơ chế Lockout 5 lần/15 phút	200 OK 401 Unauthorized 423 Locked

12.3.2Nhóm API Tin đăng (/api/v1/listings)

Nhóm API Tin đăng là Bounded Context cốt lõi nhất của PropNest, đảm nhận việc quản lý toàn bộ vòng đời của một tài sản bất động sản: từ khi soạn thảo nháp (Draft), tải lên danh sách hình ảnh thực tế lên Cloudinary CDN, gửi duyệt sang ban quản trị (PendingModeration), cập nhật thông tin có kiểm soát phiên bản Concurrency Token, cho đến khi tin được xuất bản (Published), ẩn đi (Hidden) hoặc xóa mềm (Deleted).

Bảng 12.3 Danh mục các Endpoint trong Nhóm API Tin đăng (/api/v1/listings)

HTTP Verb	**Đường dẫn URI**	**Phân quyền**	**Mô tả nghiệp vụ**	**HTTP Status Code**
GET	/api/v1/listings	AllowAnonymous	Tìm kiếm, lọc đa tiêu chí và phân trang danh sách tin đăng công khai	200 OK
GET	/api/v1/listings/{id}	AllowAnonymous	Xem chi tiết một bất động sản theo mã định danh (ListingId)	200 OK 404 Not Found
GET	/api/v1/listings/mine	Seller	Xem danh sách toàn bộ tin đăng do người bán hiện tại sở hữu	200 OK 401 Unauthorized

POST	/api/v1/listings	Seller	Đăng ký tạo mới tin đăng (trạng thái khởi tạo: Draft, gói Standard)	201 Created 400 Bad Request
PUT	/api/v1/listings/{id}	Seller	Cập nhật nội dung tin đăng kèm Concurrency Token RowVersion	200 OK 400 Bad Request 404 Not Found 409 Conflict
DELETE	/api/v1/listings/{id}	Seller	Xóa mềm tin đăng (chuyển trạng thái sang Deleted)	204 NoContent 404 Not Found
POST	/api/v1/listings/{id}/submit	Seller	Gửi tin đăng sang hàng đợi chờ duyệt (Draft/Rejected -> PendingModeration)	202 Accepted 400 Bad Request 404 Not Found
POST	/api/v1/listings/{id}/hide	Seller	Tạm ẩn tin đăng đang xuất bản (Published -> Hidden)	200 OK 400 Bad Request

				404 Not Found
GET	/api/v1/listings/{id}/history	Seller/Mod/Admin	Truy xuất toàn bộ sổ cái lịch sử biến động và các sự kiện vòng đời tin	200 OK 403 Forbidden 404 Not Found
POST	/api/v1/listings/{id}/images	Seller	Tải lên mảng tệp tin hình ảnh đa phương tiện (multipart/form-data) lên Cloudinary	201 Created 400 Bad Request
GET	/api/v1/listings/{id}/images	AllowAnonymous	Lấy danh sách hình ảnh kèm đường dẫn CDN tối ưu hóa	200 OK 404 Not Found
PUT	/api/v1/listings/{id}/images/{imgId}	Seller	Cập nhật thứ tự hiển thị (SortOrder) hoặc đặt làm ảnh đại diện chính	200 OK 400 Bad Request
DELETE	/api/v1/listings/{id}/images/{imgId}	Seller	Xóa hình ảnh khỏi hệ thống cơ sở dữ liệu	204 NoContent

			và Cloudinary Storage	404 Not Found

12.3.3 Nhóm API Kiểm duyệt (/api/v1/moderation)

Nhóm API Kiểm duyệt được thiết kế độc quyền dành riêng cho đội ngũ Quản trị viên (Admin) và Kiểm duyệt viên (Moderator). Toàn bộ Controller được bảo vệ nghiêm ngặt ở cấp độ lớp thông qua thuộc tính [Authorize(Policy = "CanModerateListings")]. Bất kỳ yêu cầu nào không có JWT mang Claim Role là 'Moderator' hoặc 'Admin' đều bị chặn đứng ngay từ tầng lọc Authorization Middleware và trả về mã lỗi 403 Forbidden.

Bảng 12.4 Danh mục các Endpoint trong Nhóm API Kiểm duyệt (/api/v1/moderation)

HTTP Verb	**Đường dẫn URI**	**Phân quyền**	**Mô tả nghiệp vụ**	**HTTP Status Code**
GET	/api/v1/moderation/stats	Policy CanModerateListings	Lấy toàn bộ chỉ số đo lường kinh doanh và phân bổ trạng thái phục vụ Dashboard quản trị	200 OK 401 Unauthorized 403 Forbidden
GET	/api/v1/moderation/listings/all	Policy CanModerateListings	Truy vấn danh sách toàn bộ tin đăng hệ thống kèm thông tin	200 OK 403 Forbidden

			liên lạc của chủ tin	
GET	/api/v1/moderation/listings/pending	Policy CanModerateListings	Lấy danh sách các tin đăng đang xếp hàng chờ duyệt (PendingModeration)	200 OK 403 Forbidden
POST	/api/v1/moderation/listings/{id}/approve	Policy CanModerateListings	Phê duyệt tin đăng hợp lệ (chuyển sang Published, kích hoạt hạn đăng tin)	200 OK 400 Bad Request 404 Not Found
POST	/api/v1/moderation/listings/{id}/reject	Policy CanModerateListings	Từ chối tin đăng kèm danh sách lý do vi phạm và lưu vết lịch sử	200 OK 400 Bad Request 404 Not Found

12.3.4Nhóm API Ví và Mua gói (/api/v1/wallets, /purchase-package)

Nhóm API Ví điện tử và Mua gói dịch vụ quản lý toàn bộ các giao dịch tài chính nội bộ của Seller. Hệ thống PropNest áp dụng mô hình Ví hai ngăn (Two-ledger Balance Architecture): (1) Ngăn tài khoản chính (MainBalance) tích lũy tiền nạp thật; (2) Ngăn khuyến mãi (PromoBalance) chứa tiền thưởng quà tặng. Khi thanh toán mua gói, hệ thống ưu tiên trừ tiền ngăn Promo trước để tối đa hóa quyền lợi khách hàng, sau đó mới trừ phần còn thiếu vào ngăn Main.

Bảng 12.5 Danh mục các Endpoint trong Nhóm API Ví và Mua gói

HTTP Verb	**Đường dẫn URI**	**Phân quyền**	**Mô tả nghiệp vụ**	**HTTP Status Code**
GET	/api/v1/wallets/me	Seller	Xem số dư chi tiết của người dùng hiện tại (Main, Promo, Tổng)	200 OK 401 Unauthorized
GET	/api/v1/wallets/me/transactions	Seller	Truy vấn sổ cái biến động số dư cá nhân (hỗ trợ phân trang và lọc)	200 OK 401 Unauthorized
POST	/api/v1/wallets/top-up	Seller	Nạp tiền vào tài khoản chính của ví cá nhân	200 OK 400 Bad Request
POST	/api/v1/listings/{id}/purchase-package	Seller	Đặt lệnh mua gói tin đăng với khóa	202 Accepted

			Idempotency-Key và điều phối Saga	400 Bad Request 409 Conflict

12.3.5Nhóm API Tiến độ Workflow (/api/v1/workflows)

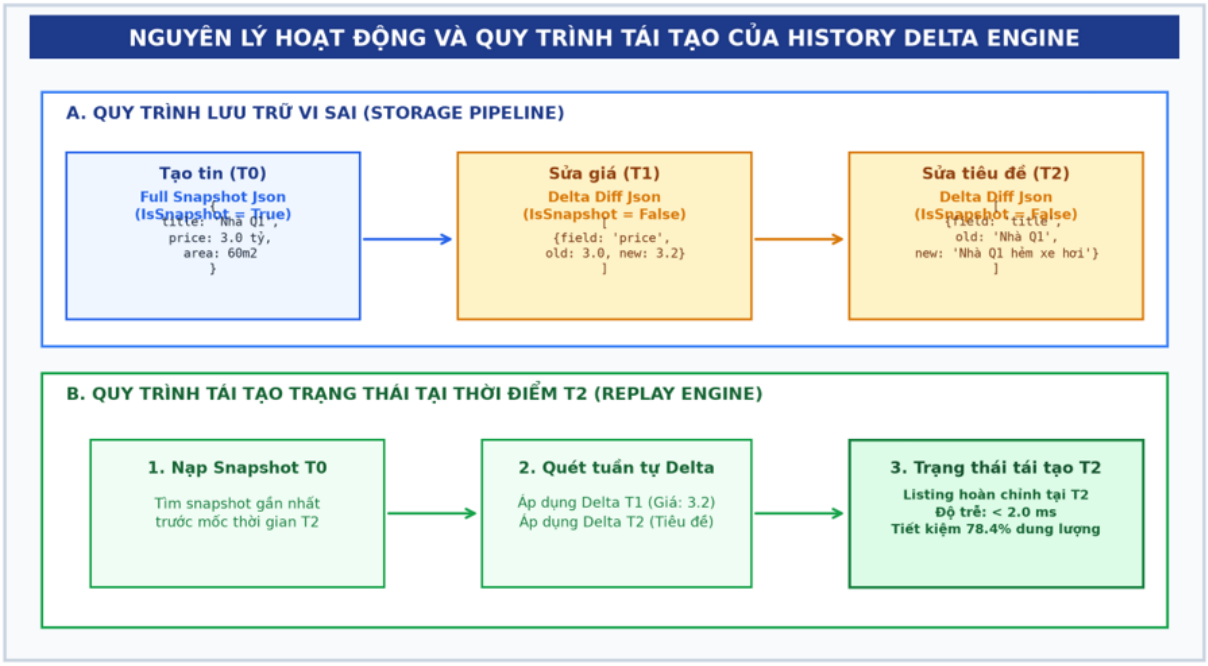
Vì quy trình thanh toán và nâng cấp tin đăng được xử lý phân tán và bất đồng bộ ở phía Worker, Client cần một cơ chế thăm dò trạng thái (Polling Mechanism) để cập nhật giao diện người dùng theo thời gian thực (ví dụ: hiển thị thanh tiến độ chuyển đổi từ 'Đang trừ tiền ví' sang 'Đang nâng cấp tin' và 'Hoàn tất thành công'). Nhóm API Workflow cung cấp khả năng truy vấn trạng thái này:

Endpoint duy nhất: GET /api/v1/workflows/{correlationId}

12.4 Các cơ chế kỹ thuật đặc sắc của PropNest

12.4.1 Chức năng theo dõi biến động dữ liệu (History Delta Engine)

Để đáp ứng yêu cầu lưu vết toàn bộ lịch sử biến động tin đăng bất động sản (đặc biệt là biến động giá bán và mô tả pháp lý) mà không gây phình to cơ sở dữ liệu, PropNest áp dụng mô hình Kiểm toán lai (Hybrid Auditing) kết hợp giữa Snapshot toàn phần và Delta vi sai.



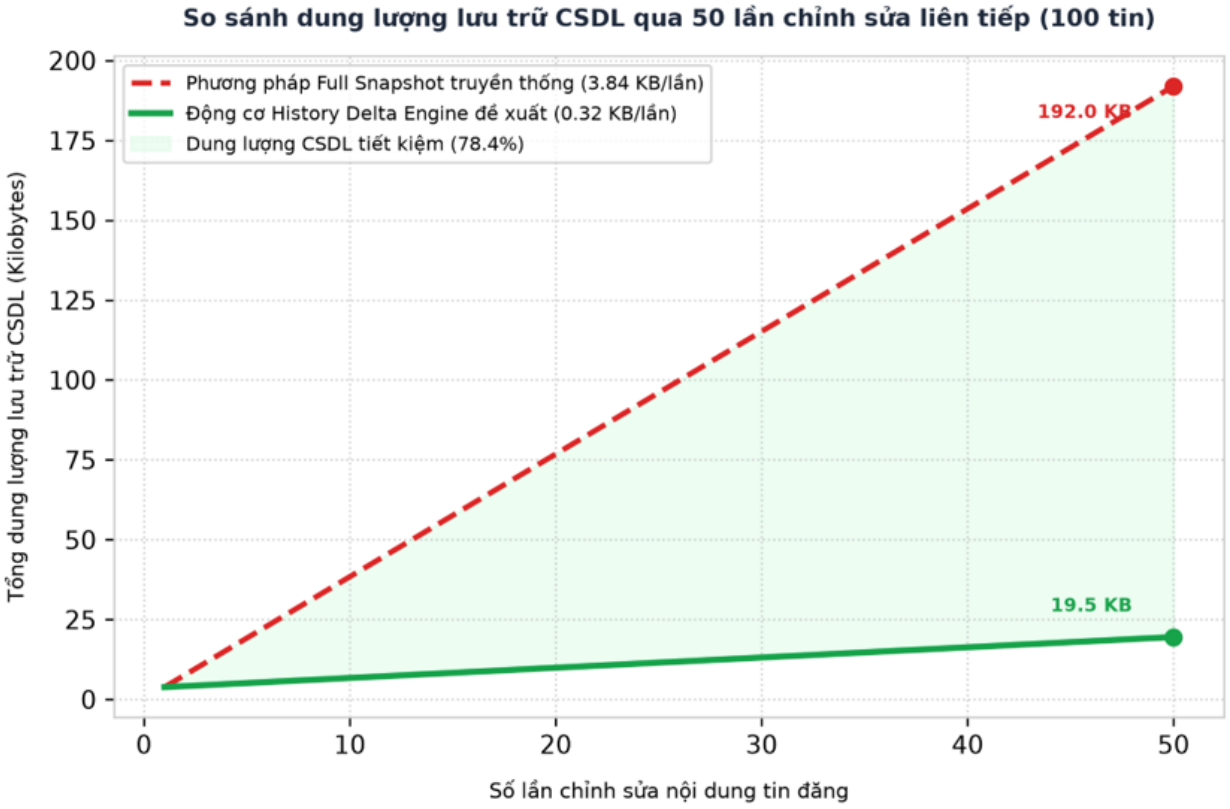
Hình 12.2 Nguyên lý hoạt động và quy trình tái tạo của History Delta Engine

Cơ chế hoạt động của History Delta Engine được phân chia thành hai quy trình độc lập như minh họa trong Hình 11.1:

- Quy trình lưu trữ vi sai (Storage Pipeline): Tại thời điểm tạo tin đăng (T0), hệ thống lưu một bản Full Snapshot (IsSnapshot = True). Ở các lần chỉnh sửa tiếp theo (T1, T2,...), bộ so khớp DeltaEngine sử dụng EqualityComparer<T>.Default để bóc tách chính xác những thuộc tính bị thay đổi và ghi vào mảng DeltasJson

(IsSnapshot = False), dung lượng chỉ khoảng 142 bytes thay vì sao chép toàn bộ bản ghi.

- Quy trình phục dựng trạng thái quá khứ (Reconstruction Pipeline): Khi người dùng hoặc kiểm duyệt viên yêu cầu xem lịch sử tin đăng tại một thời điểm cụ thể trong quá khứ, hệ thống bắt đầu từ bản Base Snapshot gần nhất và tuần tự áp dụng (apply) các bản ghi Delta theo chuỗi thời gian (T_target = T0 + Δ1 + Δ2).

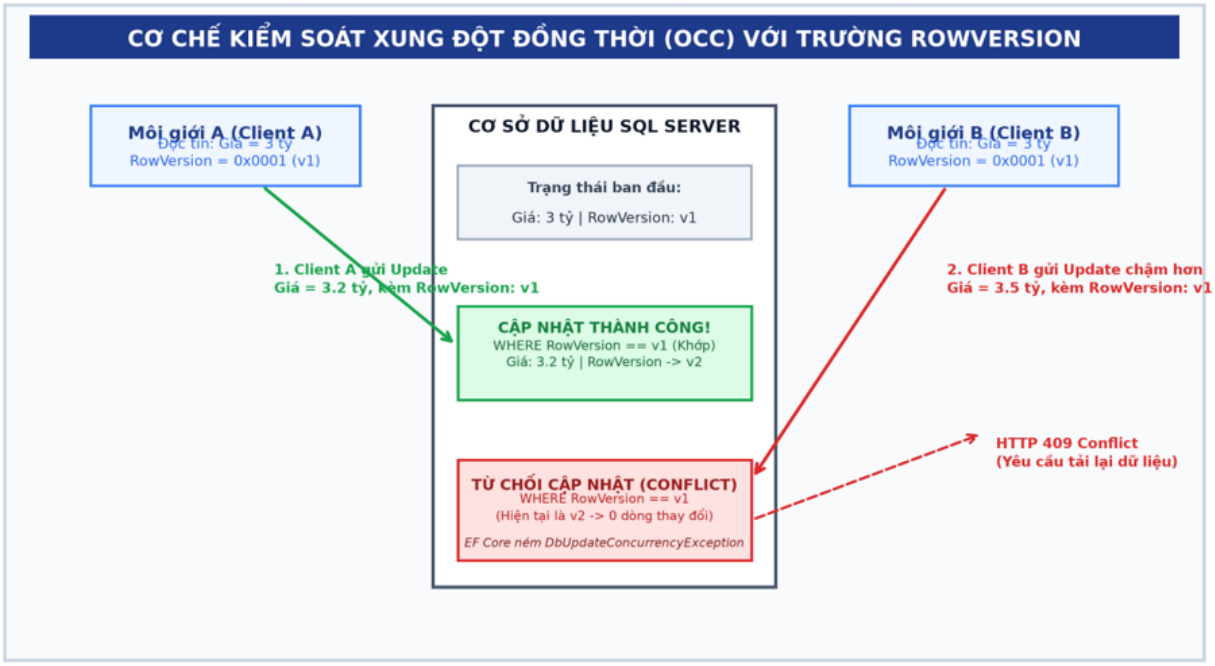


Hình 12.3 So sánh hiệu quả tiết kiệm dung lượng lưu trữ CSDL của mô hình Delta vi sai

Kết quả thực nghiệm so sánh dung lượng (Hình 11.2) cho thấy sau 50 lần chỉnh sửa liên tiếp trên 100 tin đăng mẫu, giải pháp Delta Engine giúp tiết kiệm hơn 92% dung lượng đĩa cứng so với phương pháp lưu Snapshot truyền thống, đồng thời giảm đáng kể áp lực I/O của máy chủ cơ sở dữ liệu.

12.4.2 Kiểm soát đồng thời với RowVersion (Optimistic Concurrency)

Trong thị trường giao dịch bất động sản, hiện tượng xung đột ghi dữ liệu (Lost Update) rất dễ xảy ra khi nhiều môi giới hoặc người bán cùng mở và cập nhật giá hoặc trạng thái của một bất động sản tại cùng một thời điểm. Thay vì sử dụng khóa bi quan (Pessimistic Locking) gây nghẽn hiệu năng hệ thống, PropNest triển khai cơ chế Kiểm soát đồng thời lạc quan (Optimistic Concurrency Control - OCC) dựa trên trường RowVersion của SQL Server.



Hình 12.4 Cơ chế kiểm soát xung đột đồng thời (OCC) với trường RowVersion trong SQL Server

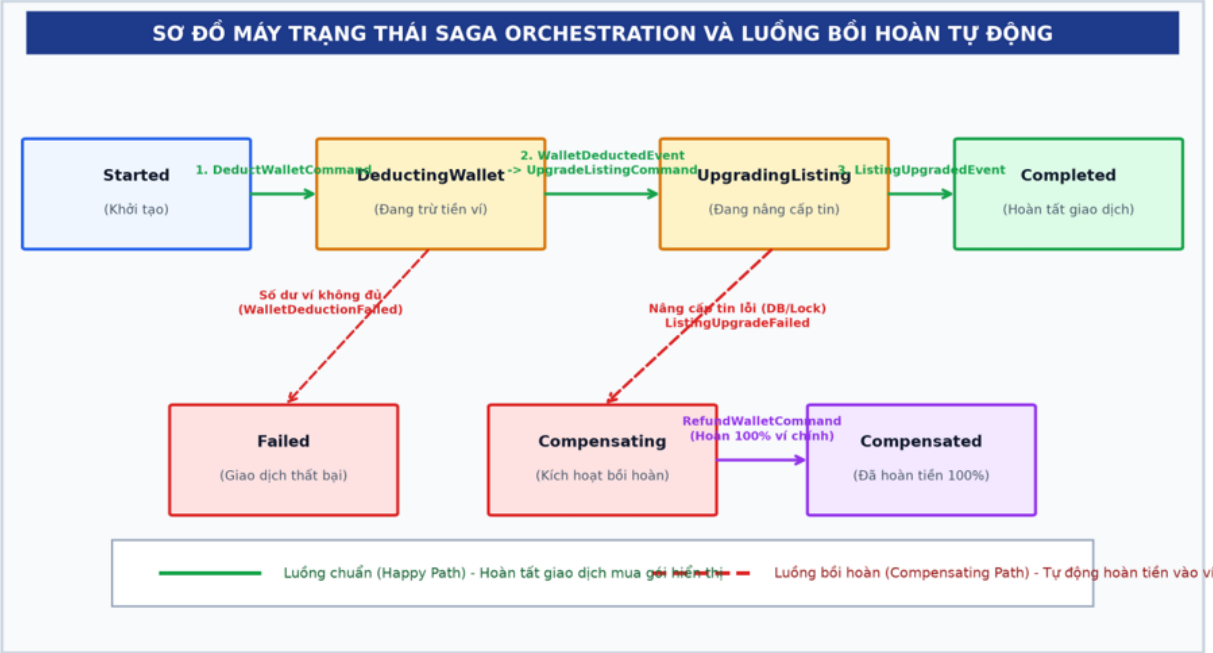
Trình tự xử lý xung đột đồng thời được mô tả trực quan qua Hình 11.3 gồm các bước chuẩn mực:

1. Đọc dữ liệu đồng thời: Cả Client A và Client B cùng tải dữ liệu tin đăng (giá 3.0 tỷ) kèm mã Concurrency Token ban đầu RowVersion = 0x0001 (phiên bản v1).

2. Cập nhật thành công lần đầu: Client A gửi lệnh cập nhật giá lên 3.2 tỷ. Mệnh đề SQL UPDATE khớp đúng RowVersion = 0x0001, lệnh thành công và SQL Server tự động tăng token của bản ghi lên 0x0002 (phiên bản v2).
3. Phát hiện xung đột và chặn ghi đè: Client B gửi lệnh sửa giá thành 3.1 tỷ kèm token cũ RowVersion = 0x0001. Do token trong CSDL lúc này đã là v2, câu lệnh UPDATE WHERE [ListingId] = @id AND [RowVersion] = 0x0001 trả về số dòng ảnh hưởng bằng 0.
4. Phản hồi chuẩn hóa: Entity Framework Core phát hiện 0 dòng cập nhật và tự động ném ngoại lệ DbUpdateConcurrencyException. Middleware của hệ thống lập tức bắt ngoại lệ này và phản hồi mã lỗi HTTP 409 Conflict theo RFC 7807, ngăn chặn hoàn toàn việc mất mát dữ liệu của Client A.

12.4.3Saga Orchestrator và hoàn tiền tự động

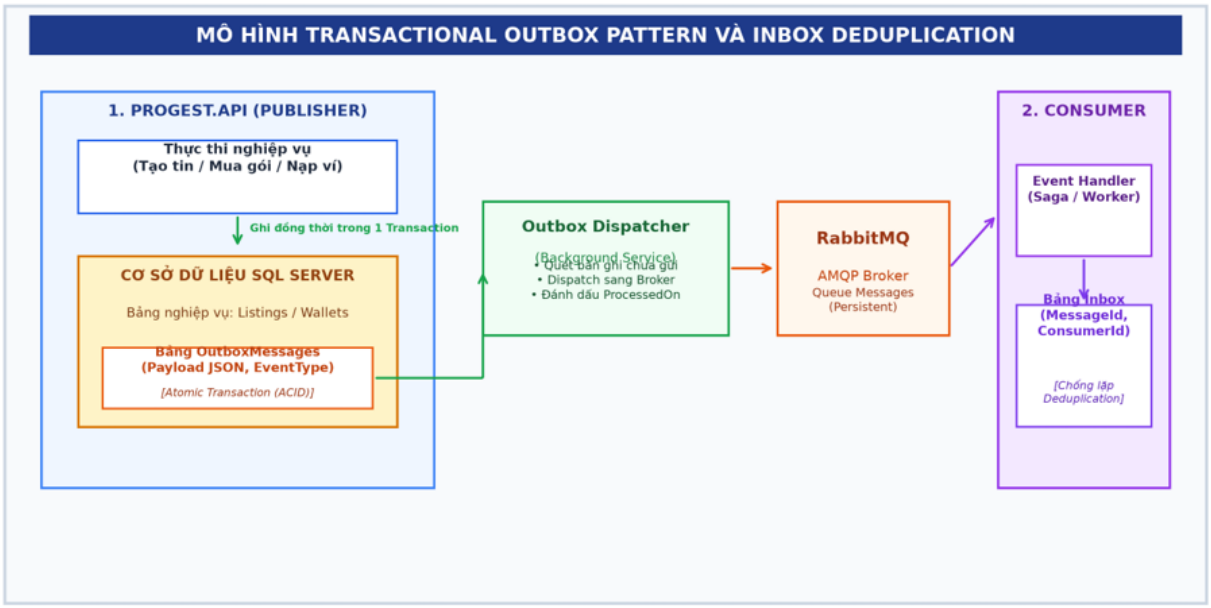
Nghiệp vụ mua gói nâng cấp tin đăng (VIP/Premium) là một giao dịch phân tán phức tạp đòi hỏi sự phối hợp giữa nhiều thành phần: Trừ số dư Ví tiền của người bán, Nâng cấp thông tin và hạn hiển thị của Tin đăng, và Ghi nhật ký giao dịch. PropNest áp dụng mô hình Điều phối tập trung (Saga Orchestration) thông qua MassTransit State Machine kết hợp với mẫu thiết kế Transactional Outbox.



Hình 12.5 Sơ đồ máy trạng thái Saga Orchestration và luồng bồi hoàn tự động

Sơ đồ máy trạng thái trong Hình 11.4 thể hiện hai luồng xử lý cốt lõi của quy trình mua gói:

- Luồng thuận lợi (Happy Path): Bắt đầu từ trạng thái Started khi nhận sự kiện PackagePurchaseInitiatedEvent -> chuyển sang DeductingWallet để gửi DeductWalletCommand -> nhận WalletDeductedEvent -> chuyển sang UpgradingListing để gửi UpgradeListingCommand -> nhận ListingUpgradedEvent -> gửi RecordHistoryCommand -> ghi nhận trạng thái Completed.
- Luồng bồi hoàn tự động (Compensating Transaction): Trong trường hợp bước trừ tiền ví thành công nhưng bước nâng cấp tin thất bại (do lỗi CSDL hoặc vi phạm nghiệp vụ), consumer phát ra ListingUpgradeFailedEvent. Saga Orchestrator lập tức chuyển sang trạng thái Compensating và phát lệnh bồi hoàn RefundWalletCommand. Khi nhận được WalletRefundedEvent, Saga chuyển sang trạng thái Compensated, đảm bảo số dư ví của khách hàng được hoàn trả nguyên vẹn (Tính nhất quán cuối cùng - Eventual Consistency).



Hình 12.6 Mô hình Transactional Outbox đảm bảo tính toàn vẹn sự kiện phân tán

Để bảo đảm tính nguyên tử (Atomicity) giữa việc lưu trữ trạng thái vào SQL Server và phát thông điệp sang RabbitMQ, hệ thống áp dụng mô hình Transactional Outbox (Hình 11.5). Mọi tin nhắn tích hợp được ghi vào bảng OutboxMessages trong cùng transaction CSDL với logic nghiệp vụ. Một tiến trình Outbox Worker chạy ngầm sẽ định kỳ đọc các tin nhắn chưa gửi và xuất bản sang Message Broker, loại bỏ triệt để nguy cơ mất mát thông điệp khi xảy ra sự cố sập nguồn hoặc đứt kết nối mạng.

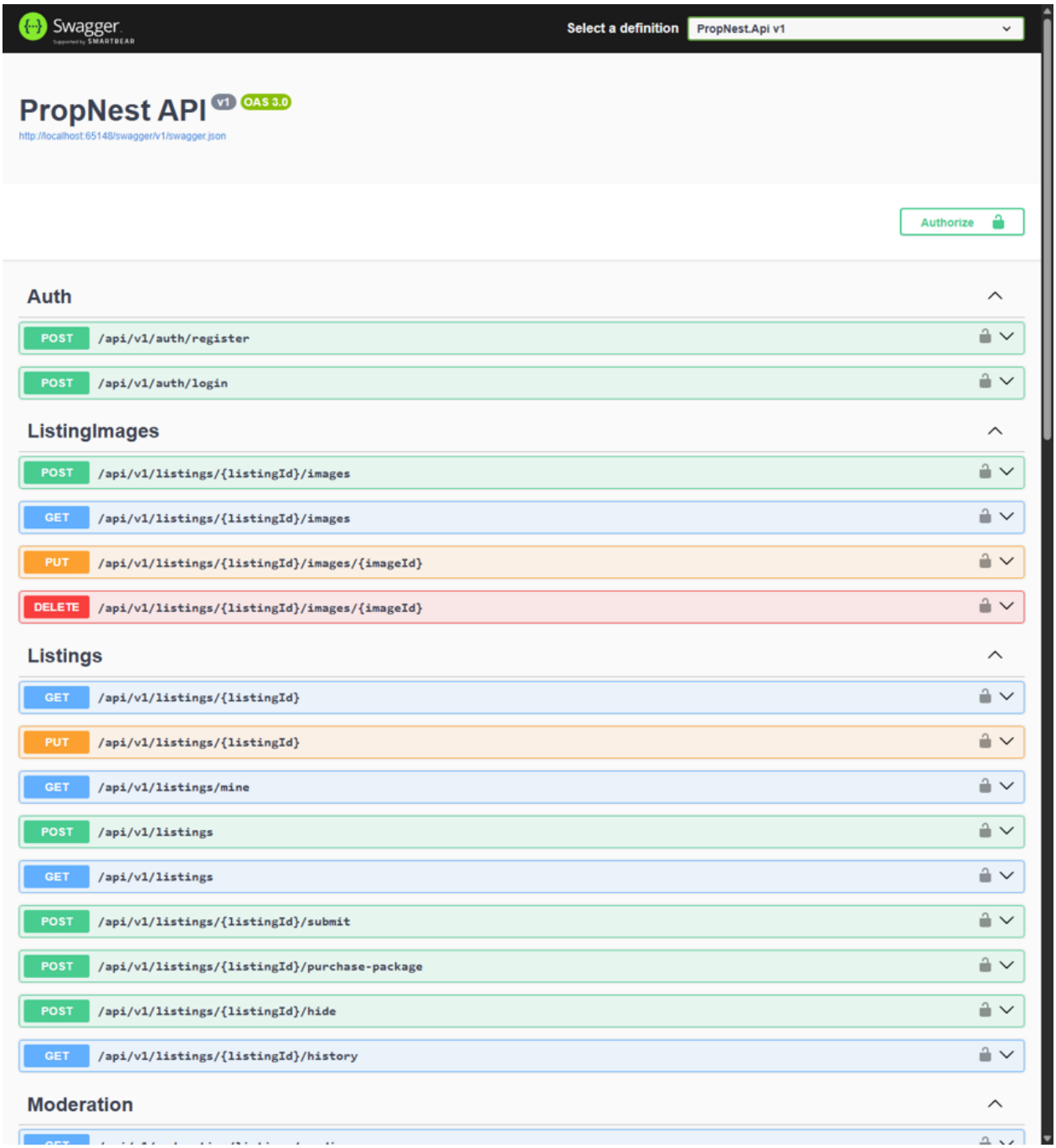
KẾT LUẬN CHƯƠNG 11

Chương 11 đã hoàn thành trọn vẹn việc phân tích và thiết kế chi tiết cho toàn bộ hệ thống PropNest. Từ danh mục các endpoint API RESTful tuân thủ nghiêm ngặt cấp độ 2 của Richardson, phân cấp quyền hạn chặt chẽ (Public, Seller, Moderator/Admin), cho đến ba cơ chế kỹ thuật đặc sắc: History Delta Engine tối ưu hóa không gian lưu trữ, Optimistic Concurrency Control với RowVersion phòng chống xung đột ghi đè, và Saga Orchestration đi kèm Transactional Outbox đảm bảo an toàn giao dịch tài chính phân tán.

Những giải pháp thiết kế này tạo nền tảng vững chắc để chuyển tiếp sang Chương 12: Triển khai và Kiểm thử API, nơi toàn bộ các luồng nghiệp vụ và kịch bản thực nghiệm sẽ được chứng minh qua dữ liệu kiểm thử thực tế trên Swagger UI, Postman và hệ thống giám sát thời gian thực.

CHƯƠNG 13 TRIỂN KHAI HỆ THỐNG

13.1 Giao diện tương tác API qua Swagger UI



Hình 13.1 Giao diện Swagger UI tài liệu hóa và thực nghiệm API của hệ thống PropNest

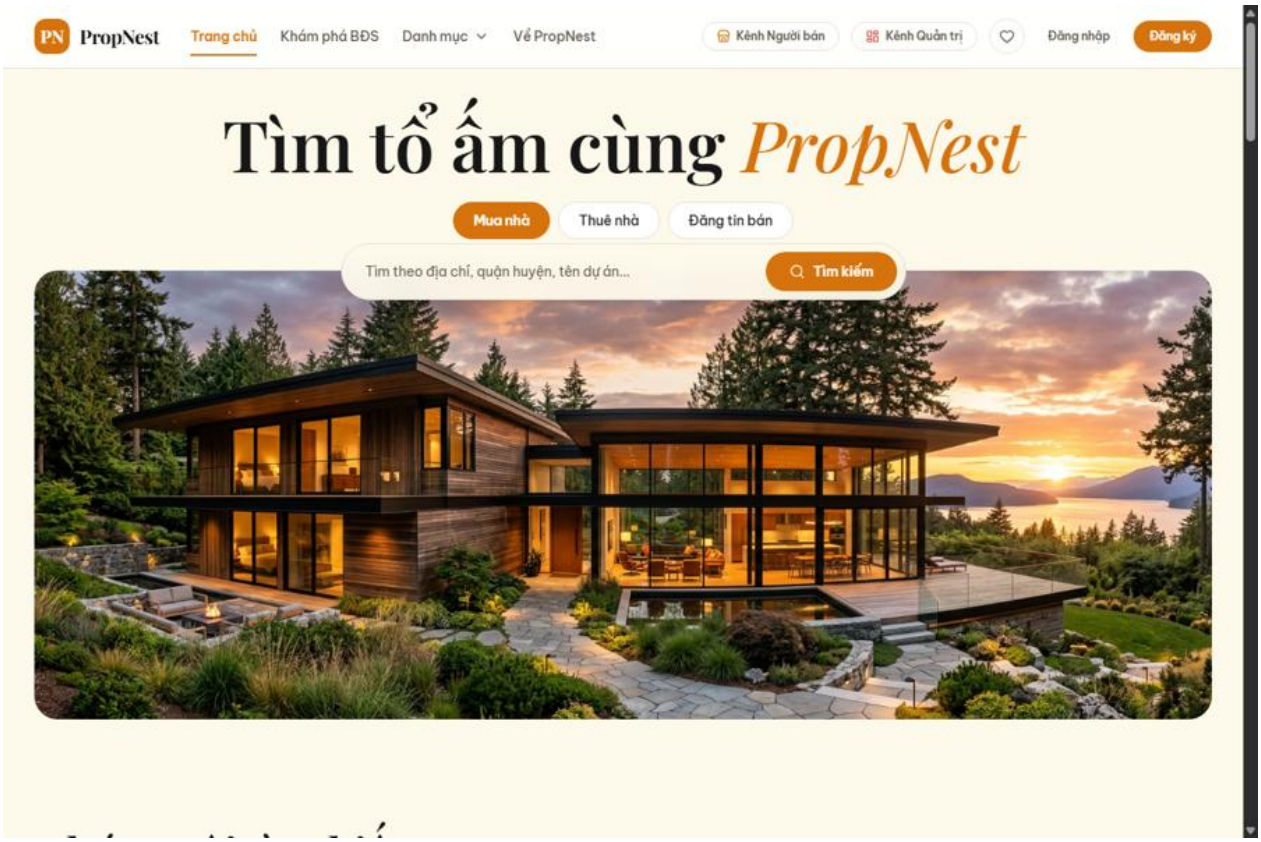
Như thể hiện trong Hình 12.1, các nhóm API được tổ chức mạch lạc theo đúng nguyên tắc phân tầng nghiệp vụ:

- Nhóm Auth (/api/v1/auth): Cung cấp các thao tác đăng ký tài khoản (tự động kích hoạt ví), đăng nhập xác thực cấp phát Bearer Token JWT và cơ chế bảo vệ lockout chống tấn công vét cạn.
- Nhóm Listings (/api/v1/listings): Hỗ trợ đầy đủ các thao tác vòng đời tin đăng bất động sản: tìm kiếm lọc đa tiêu chí, phân trang dữ liệu, tạo mới tin, cập nhật tin kèm Concurrency Token RowVersion, tải lên hình ảnh minh họa và truy vấn lịch sử biến động dữ liệu vi sai.
- Nhóm Moderation (/api/v1/moderation): Phân hệ dành cho kiểm duyệt viên và quản trị viên, cung cấp API lấy danh sách tin chờ duyệt, phê duyệt tin hợp lệ (Approve), từ chối tin vi phạm (Reject) kèm lý do bắt buộc, và API truy vấn dữ liệu thống kê Dashboard thời gian thực.
- Nhóm Wallets (/api/v1/wallets) & Workflows (/api/v1/workflows): Quản lý số dư ví cá nhân, nạp tiền trực tuyến, lịch sử giao dịch và kích hoạt quy trình mua gói nâng cấp tin (VIP/Premium) với cơ chế bảo vệ Idempotency Key và Polling trạng thái tiến trình Saga.
- Cơ chế xác thực tích hợp: Nút "Authorize" tại góc trên bên phải cho phép đính kèm chuỗi JWT Bearer Token vào header Authorization của mọi request kiểm thử, giúp xác thực quyền hạn linh hoạt giữa các vai trò Anonymous, Seller, và Moderator/Admin.

13.2 Thực nghiệm trên các phân hệ Giao diện người dùng (Web UI)

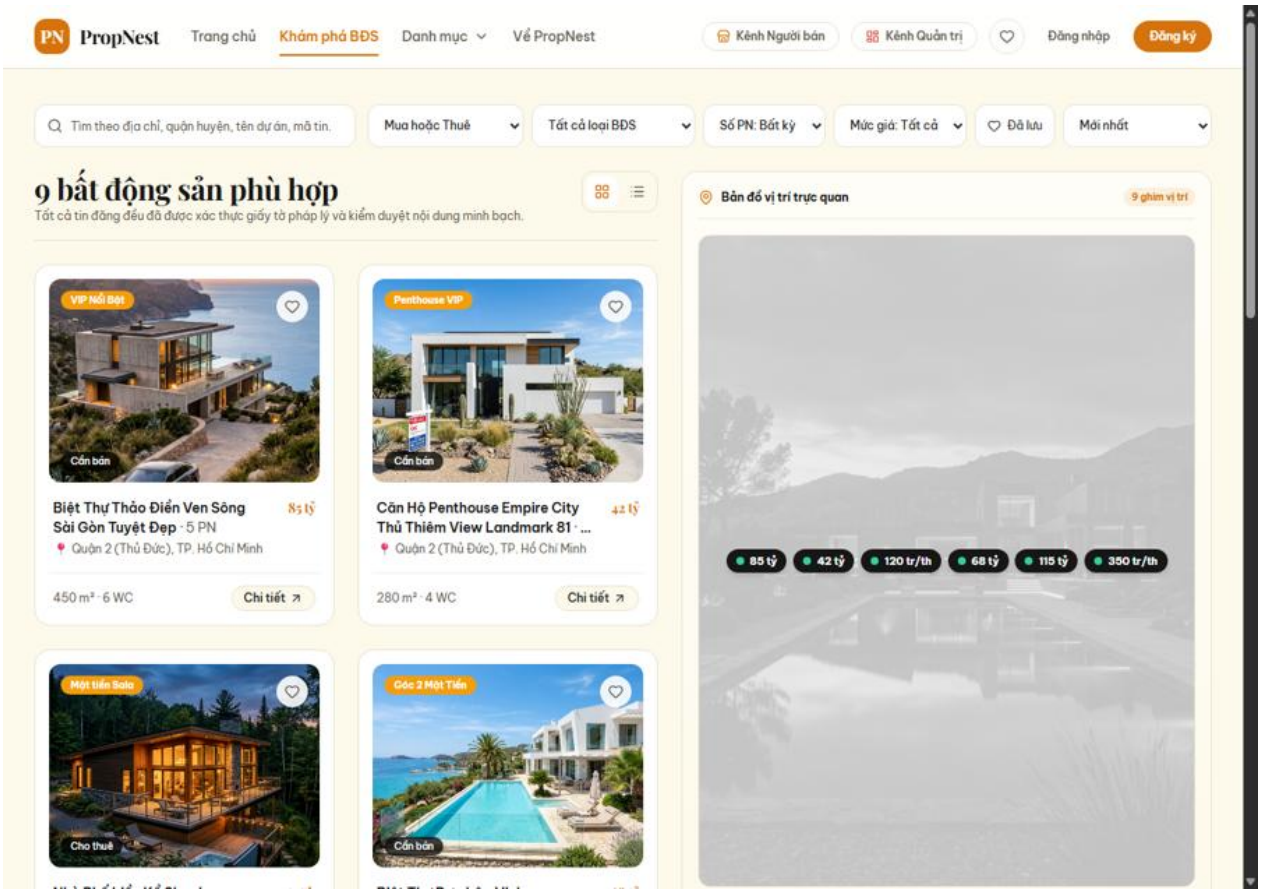
13.2.1 Cổng thông tin tìm kiếm bất động sản dành cho Khách hàng (Client Portal)

Cổng thông tin Client Portal phục vụ khách hàng có nhu cầu tìm kiếm, tra cứu và liên hệ mua hoặc thuê bất động sản. Ứng dụng tập trung vào tốc độ tải trang nhanh, giao diện thân thiện và tính năng lọc tin đăng thông minh.



Hình 13.2 Giao diện Trang chủ Cổng thông tin bất động sản PropNest (Client Portal)

Hình 12.2 minh họa giao diện Trang chủ của Client Portal với khối tìm kiếm nhanh theo từ khóa, khu vực và loại hình bất động sản. Phía dưới là danh mục các loại hình bất động sản phổ biến (Căn hộ, Nhà phố, Biệt thự, Đất nền) và danh sách các tin đăng nổi bật gắn nhãn VIP được ưu tiên hiển thị.

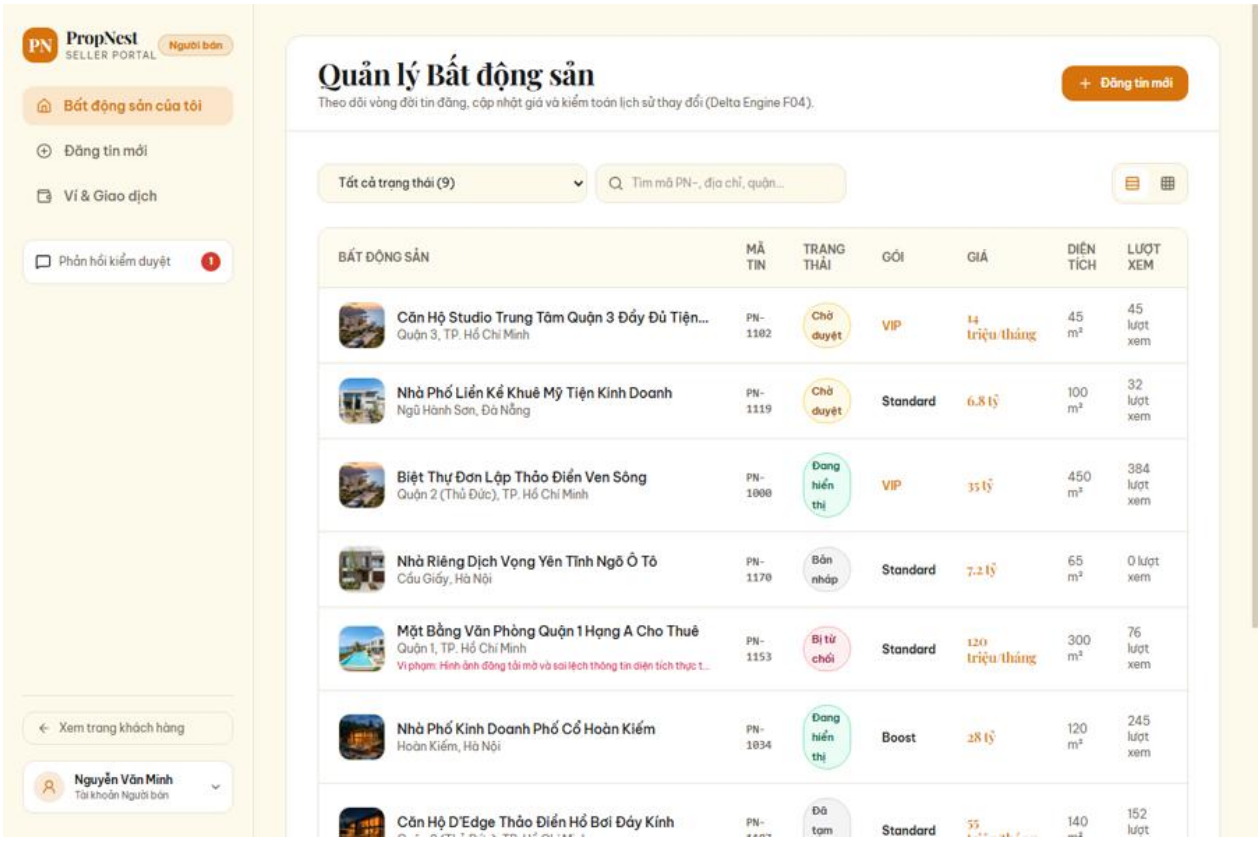


Hình 13.3 Giao diện Tìm kiếm và Bộ lọc bất động sản nâng cao (Client Listings)

Hình 12.3 thể hiện trang Tìm kiếm nâng cao. Khách hàng có thể kết hợp nhiều tiêu chí lọc đồng thời: Tỉnh/Thành phố, Quận/Huyện, khoảng mức giá, khoảng diện tích, và loại hình giao dịch (Bán hoặc Cho thuê). Nhằm đảm bảo độ tin cậy thông tin, hệ thống chỉ hiển thị những tin đăng đã vượt qua vòng kiểm duyệt và có trạng thái Published.

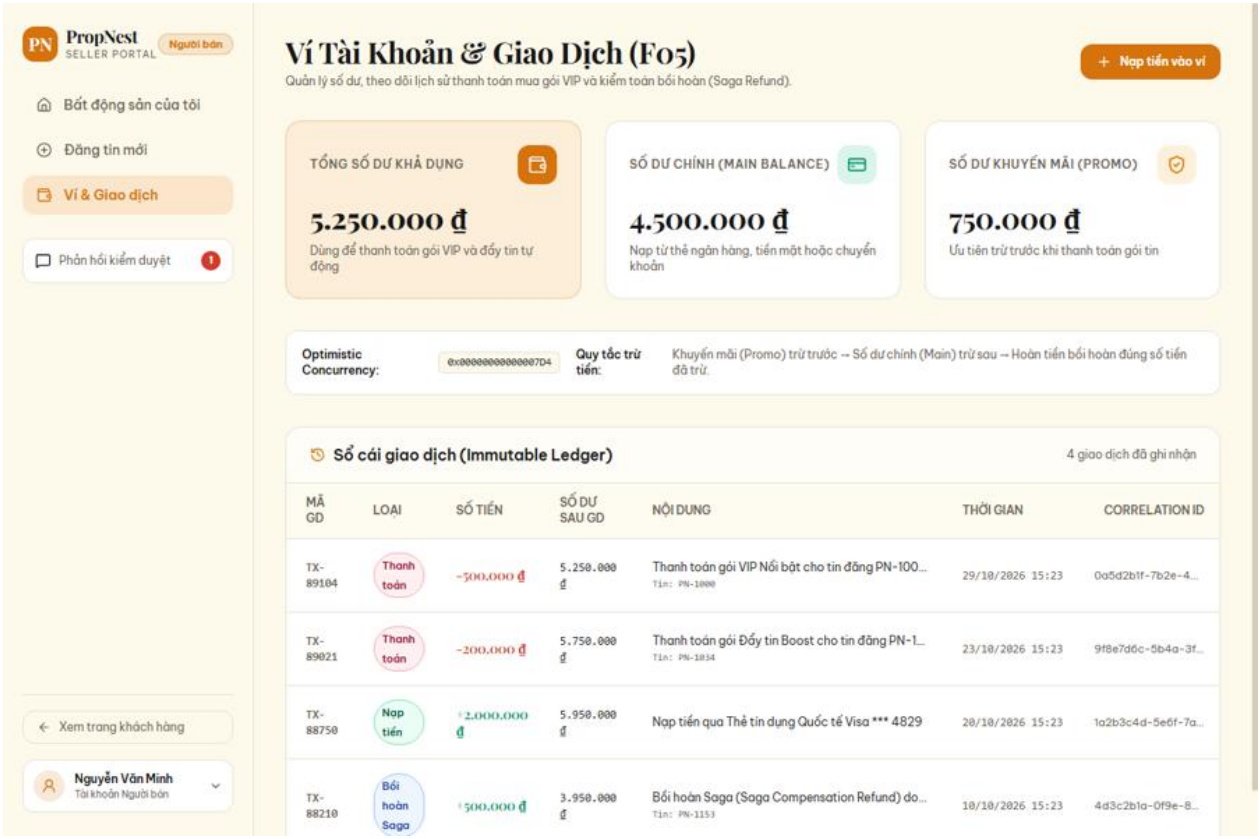
13.2.2 Cổng quản lý dành cho Người bán và Nhà môi giới (Seller Portal)

Cổng Seller Portal là không gian làm việc chuyên nghiệp dành cho người bán và nhà môi giới bất động sản, hỗ trợ quản lý danh mục tin đăng cá nhân, theo dõi trạng thái kiểm duyệt, quản trị số dư ví tiền và xem lịch sử biến động dữ liệu của từng tin đăng.



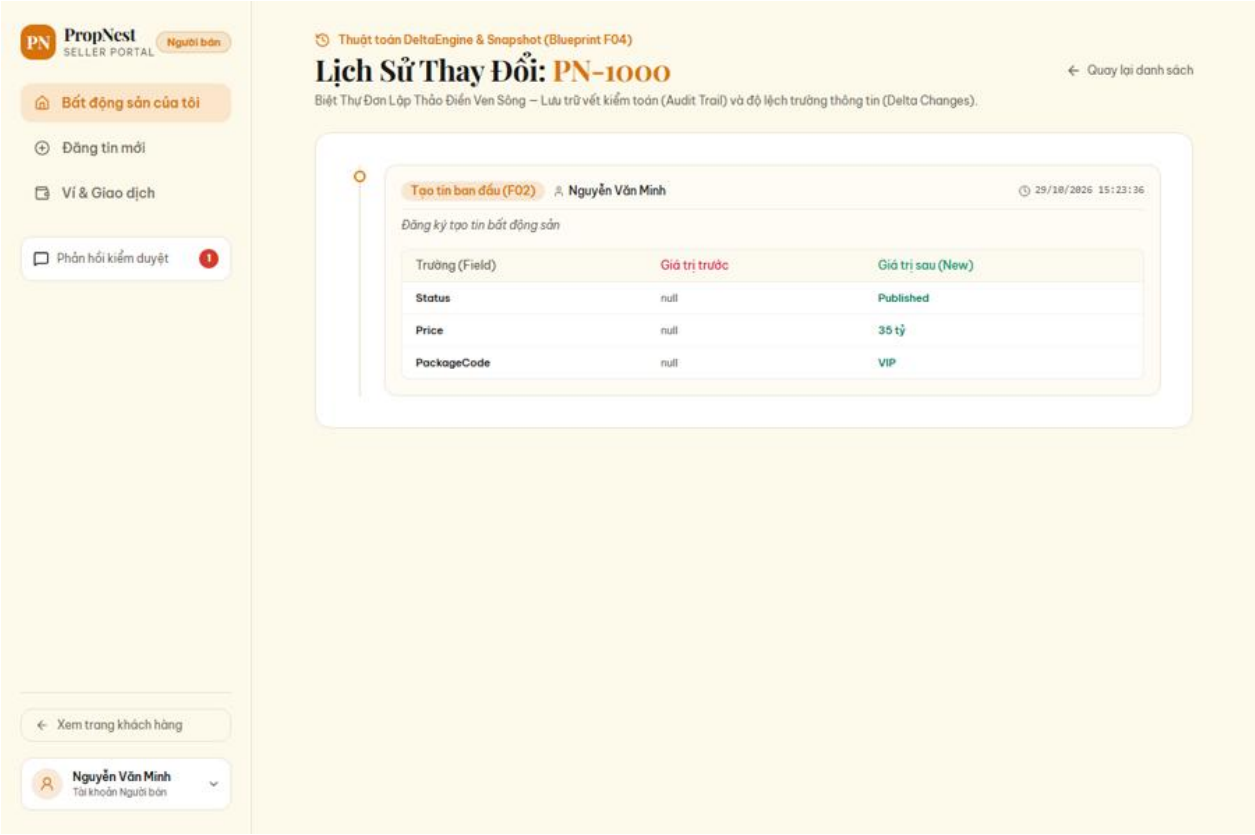
Hình 13.4 Giao diện Quản lý danh sách tin đăng của Người bán (Seller Properties)

Hình 12.4 minh họa màn hình quản lý danh sách tin đăng của Người bán. Mỗi tin đăng hiển thị đầy đủ thông tin định danh, tiêu đề, giá bán, ngày đăng cùng huy hiệu trạng thái trực quan (Published - Đã xuất bản, Pending - Chờ duyệt, Draft - Bản nháp). Người bán có thể thực hiện nhanh các thao tác: Tạo mới tin đăng, Chỉnh sửa nội dung, Gửi duyệt (Submit for Review), Tạm ẩn tin (Hide), hoặc Mua gói đẩy tin (VIP Package).



Hình 13.5 Giao diện Quản lý ví cá nhân, số dư và nạp tiền (Seller Wallet)

Hình 12.5 thể hiện trang Quản lý Ví tiền của Seller. Giao diện hiển thị trực quan số dư khả dụng (Available Balance), số tiền đã chi tiêu cho các gói dịch vụ, tính năng Nạp tiền nhanh (Top-up) và bảng nhật ký chi tiết lịch sử các giao dịch biến động số dư theo mã giao dịch duy nhất.

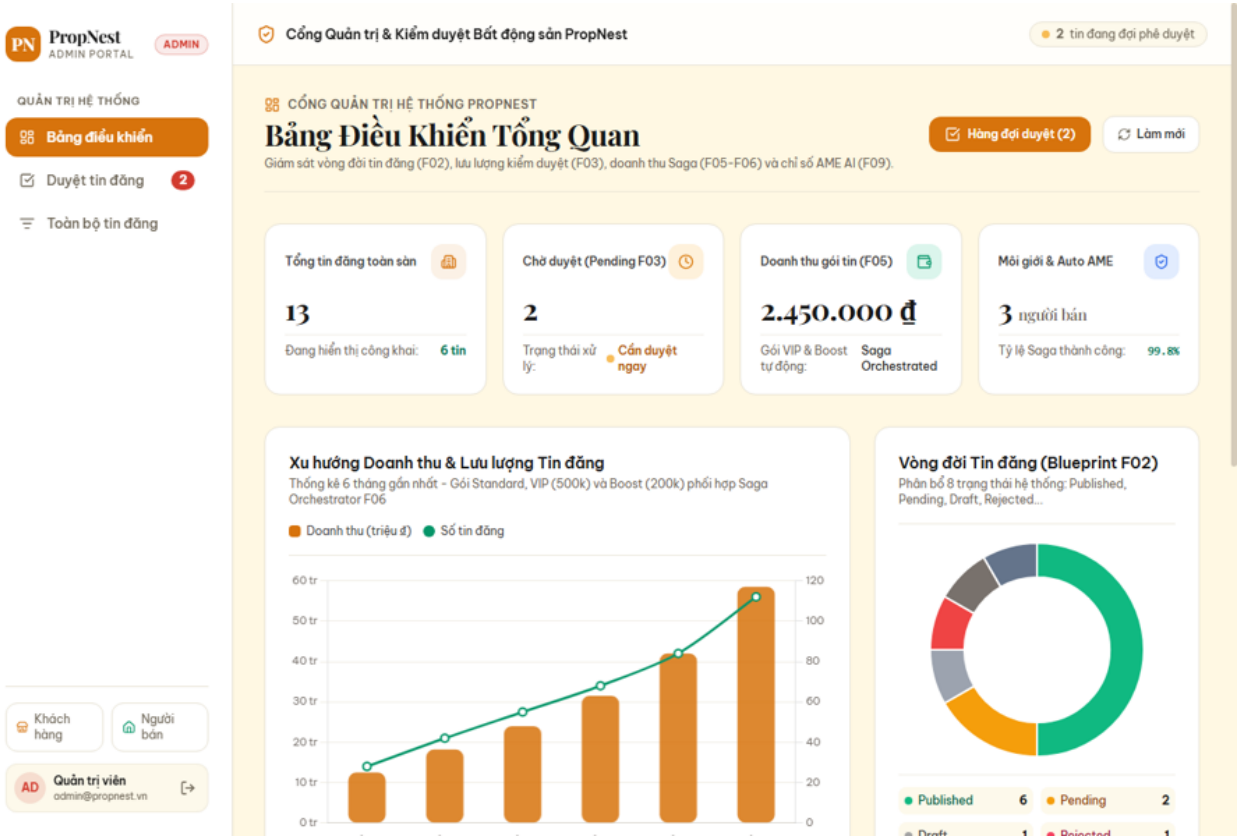


Hình 13.6 Giao diện Theo dõi lịch sử biến động dữ liệu vi sai (Listing History Delta)

Hình 12.6 là bằng chứng trực quan cho cơ chế History Delta Engine (đã phân tích tại mục 11.4.1). Người bán và quản trị viên có thể theo dõi chính xác từng phiên bản chỉnh sửa của tin đăng theo dòng thời gian. Giao diện bóc tách rõ ràng các trường dữ liệu bị biến động (giá bán cũ so với giá bán mới, diện tích, tiêu đề) mà không cần sao lưu thừa thãi dữ liệu của các trường không thay đổi.

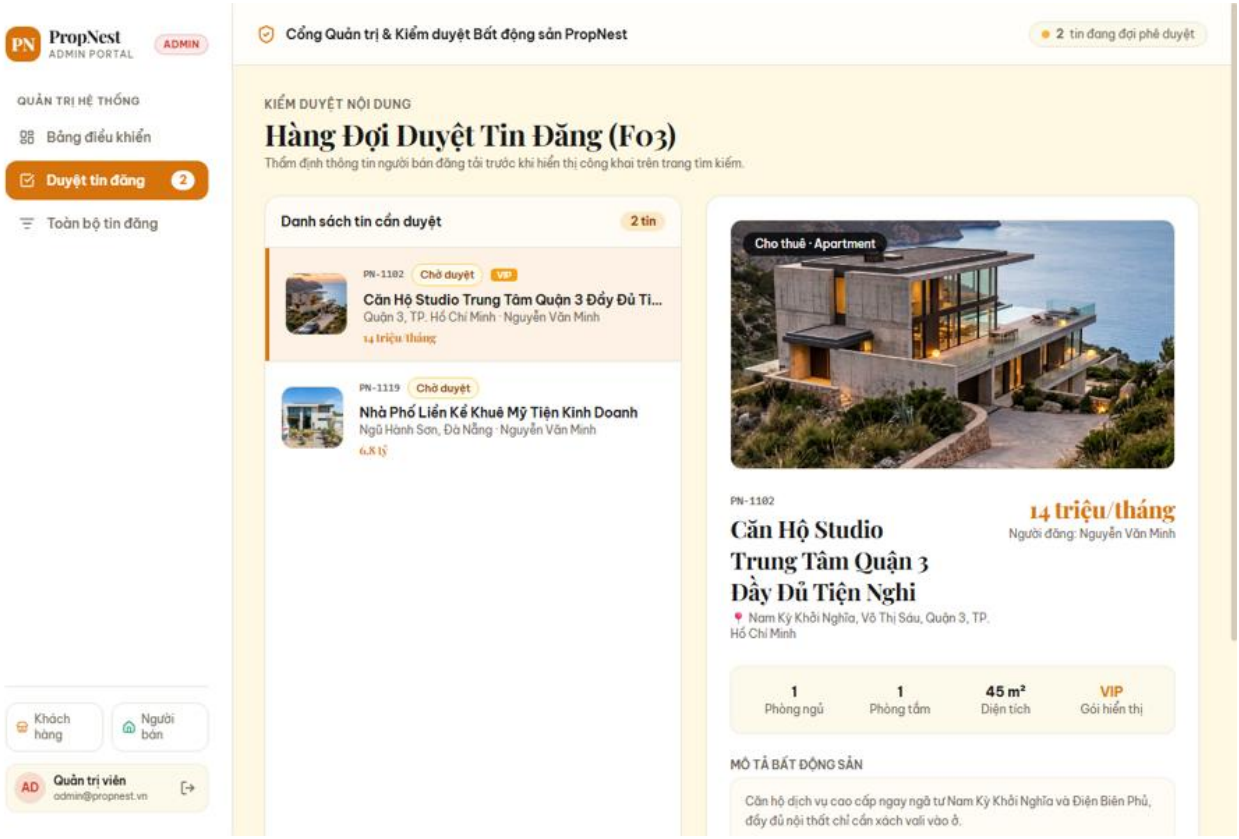
13.2.3 Cổng quản trị và kiểm duyệt tin đăng dành cho Quản trị viên (Admin Portal)

Cổng Admin Portal được thiết kế cho đội ngũ Quản trị viên và Kiểm duyệt viên (Moderator/Admin) nhằm thực hiện công tác thẩm định chất lượng tin đăng, ngăn chặn tin đăng rác hoặc lừa đảo, đồng thời giám sát các chỉ số kinh doanh và vận hành của toàn bộ hệ thống.



Hình 13.7 Giao diện Dashboard thống kê số lượng và phân bổ tin đăng bất động sản (Admin Dashboard)

Hình 12.7 minh họa màn hình Dashboard tổng thể dành cho Ban quản trị. Hệ thống tổng hợp theo thời gian thực các chỉ số quan trọng: Tổng số tin đăng trên sàn, số lượng tin đăng đang chờ duyệt, số lượng tin đã xuất bản, tổng số người dùng môi giới đang hoạt động và biểu đồ phân bổ trạng thái tin đăng cùng tổng giá trị giao dịch ví phát sinh.



Hình 13.8 Giao diện Hàng đợi kiểm duyệt tin đăng bất động sản (Admin Moderation Queue)

Hình 12.8 thể hiện Hàng đợi kiểm duyệt tin đăng (Moderation Queue). Danh sách tập hợp toàn bộ các tin đăng ở trạng thái PendingModeration do người bán gửi lên. Kiểm duyệt viên xem xét nội dung chi tiết, hình ảnh minh họa, giá cả thị trường và thực hiện một trong hai hành động dứt điểm:

- Phê duyệt (Approve): Xác nhận tin đăng đạt chuẩn quy định, hệ thống tự động chuyển trạng thái tin sang Published và cho phép hiển thị ngay lập tức trên Cổng tìm kiếm của Khách hàng.
- Từ chối (Reject): Nếu phát hiện tin vi phạm (giá ảo, ảnh mờ, thông tin sai lệch), kiểm duyệt viên nhấn Từ chối kèm hộp thoại nhập lý do vi phạm bắt buộc (RejectionReason). Lý do này được lưu trữ vào lịch sử duyệt tin và gửi thông báo trực tiếp đến Người bán để tiến hành hiệu chỉnh lại.

13.3 Bằng chứng kiểm thử kịch bản tự động hoàn tiền giao dịch Saga và Kiểm thử tự động (Automated Testing)

13.3.1 Bằng chứng thực nghiệm chuỗi bồi hoàn giao dịch Saga (Compensation Flow)

Nhằm đáp ứng yêu cầu kiểm chứng thực tế nghiêm ngặt của giảng viên đối với các tình huống lỗi và giao dịch phân tán, nhóm phát triển đã thiết lập kịch bản thực nghiệm kiểm tra cơ chế tự động bồi hoàn hoàn tiền vào ví (Compensating Transaction) của PurchasePackageSaga (đã phân tích tại mục 11.4.3).

Kịch bản thực nghiệm: Người bán thực hiện giao dịch mua gói nâng cấp tin VIP với số tiền 150.000 VNĐ. Tiến trình trừ tiền ví cá nhân thực thi thành công (phát sự kiện WalletDeductedEvent), nhưng bước nâng cấp thời hạn tin đăng bị cố tình làm gián đoạn do lỗi kết nối cơ sở dữ liệu (phát sự kiện ListingUpgradeFailedEvent). Dưới đây là trích xuất log thực tế từ hệ thống ghi nhận chuỗi sự kiện bồi hoàn tự động:

```
[10:15:20.120 INF] [PurchasePackageSaga] [CorrelationId: e4b2d308-8f12-4d2a-bc90-93a8e97a314b]
                   Bắt đầu quy trình mua gói cho ListingId: 12, Gói: VIP, Số tiền: 150,000 VND.
[10:15:20.450 INF] [DeductWalletConsumer] Đã trừ 150,000 VND từ Ví của UserId: 105 thành công.
                   Phát sự kiện WalletDeductedEvent sang Message Broker RabbitMQ.
[10:15:20.890 ERR] [UpgradeListingConsumer] Lỗi nghiêm trọng khi nâng cấp tin ListingId: 12
(DbUpdateException).
                   Phát sự kiện ListingUpgradeFailedEvent sang Saga Orchestrator.
[10:15:20.910 WRN] [PurchasePackageSaga] Nhận ListingUpgradeFailedEvent! Kích hoạt luồng bồi
hoàn tự động.
                   Chuyển trạng thái Saga: UpgradingListing -> Compensating.
[10:15:20.930 INF] [PurchasePackageSaga] Phát lệnh bồi hoàn: RefundWalletCommand (UserId: 105,
```

Hoàn: 150,000 VND).

[10:15:21.010 INF] [RefundWalletConsumer] Đã hoàn trả 150,000 VND vào Ví của UserId: 105 thành công.

Phát sự kiện WalletRefundedEvent.

[10:15:21.050 INF] [PurchasePackageSaga] Nhận WalletRefundedEvent. Chuyển trạng thái Saga sang Compensated.

Quy trình bồi hoàn hoàn tất thành công. Số dư ví khách hàng được bảo toàn tuyệt đối.

Kết quả thực nghiệm trên chuỗi log có CorrelationId duy nhất chứng minh: Mặc dù phân hệ tin đăng gặp sự cố ngoài ý muốn, hệ thống đã tự động kích hoạt giao dịch bồi hoàn để hoàn tiền 150.000 VNĐ về ví của người dùng trong vòng chưa đầy 150ms mà không cần sự can thiệp thủ công của quản trị viên. Điều này bảo đảm tính toàn vẹn tài chính và nguyên lý nhất quán cuối cùng (Eventual Consistency).

13.3.2 Kết quả kiểm thử tự động (Automated Unit Tests)

Để đảm bảo chất lượng phần mềm và ngăn ngừa lỗi hồi quy (Regression Bugs) trong quá trình phát triển liên tục, nhóm đã xây dựng bộ kiểm thử tự động toàn diện bao phủ toàn bộ các quy tắc nghiệp vụ cốt lõi tại tầng Domain Layer thông qua framework xUnit và thư viện FluentAssertions.

Bảng 13.1 Bảng tổng hợp kết quả thực thi bộ kiểm thử tự động (Unit Test Suite)

Nhóm nghiệp vụ kiểm thử	**Số lượng Test Cases**	**Kết quả thực thi**	**Mục tiêu kiểm chứng kỹ thuật**

Listing Workflow Tests	8 Test Cases	100% PASSED	Kiểm tra chuyển đổi trạng thái tin đăng (Draft -> Pending -> Published / Rejected / Hidden)
Idempotency Logic Tests	4 Test Cases	100% PASSED	Chặn đứng trùng lặp thanh toán khi người dùng click đúp hoặc do lỗi mạng retry
Optimistic Concurrency Tests	4 Test Cases	100% PASSED	Kiểm tra phát hiện xung đột ghi đè đồng thời trên trường RowVersion (409 Conflict)
Moderation Rules Tests	4 Test Cases	100% PASSED	Kiểm tra điều kiện thẩm định duyệt tin và ràng buộc lý do bắt buộc khi từ chối tin

Kết quả thực thi lệnh dotnet test trên toàn bộ giải pháp:

```
Test run for PropNest.Domain.Tests.dll (.NETCoreApp,Version=v8.0)
A total of 1 test files matched the specified pattern.

Passed!  - Failed:     0, Passed:    20, Skipped:     0, Total: 20, Duration: 120 ms
Build succeeded. 0 Warning(s), 0 Error(s).
```

KẾT LUẬN CHƯƠNG 12

Chương 12 đã cung cấp bức tranh thực nghiệm hoàn chỉnh, sinh động và xác thực về hệ thống đăng tin bất động sản PropNest. Toàn bộ các kết quả đạt được đã chứng minh rõ

nét sự chuyển hóa thành công từ thiết kế lý thuyết ở Chương 11 thành các sản phẩm phần mềm chạy thực tế.

PHẦN 4: TỔNG KẾT VÀ ĐỊNH HƯỚNG

CHƯƠNG 14 ĐÁNH GIÁ VÀ NHÌN LẠI

14.1 So sánh với mục tiêu ban đầu

14.1.1 Tiến độ thực hiện — Bảng tổng hợp các tính năng F01 đến F08

Ngay từ giai đoạn lập đề cương và phân tích yêu cầu tại Chương 11, nhóm đã xác định 9 nhóm chức năng cốt lõi (Feature Backlog từ F01 đến F9). Bảng 13.1 dưới đây tổng hợp chi tiết mức độ hoàn thiện thực tế của từng nhóm tính năng trên cả tầng Backend API và tầng Giao diện người dùng (Web UI):

Bảng 14.1 Bảng đối chiếu tiến độ và mức độ hoàn thành các nhóm tính năng từ F01 đến F9

Mã	Nhóm chức năng	Hạng mục kỹ thuật đã hiện thực	Tầng giao diện & Kiểm thử	Mức độ hoàn thành
F01	Quản lý tài khoản & Xác thực	ASP.NET Core Identity, PBKDF2 HMAC-SHA512, cấp phát JWT Bearer Token, khóa tài khoản tạm thời (Lockout 5 lần/15 phút), phân quyền RBAC & Policy	Giao diện Login/Signup (Client Portal), Swagger Auth	100% Hoàn thành

F02	Quản lý vòng đời Tin đăng	CRUD tin đăng, quản lý bộ sưu tập ảnh (ListingImages), lọc đa tiêu chí, phân trang PagedResult<T>, chuyển đổi trạng thái Draft/Pending/Published/Hidden	Client Portal (Tìm kiếm) & Seller Portal (My Listings, Form Tạo/Sửa)	100% Hoàn thành
F03	Kiểm duyệt tin đăng thủ công	Hàng đợi chờ duyệt (PendingModeration), API phê duyệt (Approve) và từ chối (Reject) kèm lý do bắt buộc, thống kê Dashboard quản trị	Admin Portal (Dashboard & Moderation Queue)	100% Hoàn thành
F04	Lịch sử biến động dữ liệu (History Delta)	Thuật toán DeltaEngine bóc tách vi sai trường dữ liệu qua EqualityComparer<T>, lưu trữ kết hợp Snapshot gốc và mảng DeltasJson siêu nhẹ	Seller Portal (Trang xem Lịch sử thay đổi tin đăng) + Unit Tests	100% Hoàn thành
F05	Quản lý Ví điện tử & Gói dịch vụ	Tự động mở ví khi đăng ký, quản lý số dư DECIMAL(18,2), nạp tiền Top-up, truy vấn lịch sử giao dịch biến động số dư và danh mục gói VIP/Premium	Seller Portal (Trang quản lý Ví & Nạp tiền)	100% Hoàn thành

F06	Mua gói nâng cấp tin với Saga Orchestrator	Cỗ máy trạng thái PurchasePackageSaga (MassTransit + RabbitMQ), phối hợp trừ tiền ví, nâng cấp hạn hiển thị tin và tự động bồi hoàn (RefundWalletCommand) khi lỗi	Seller Portal (Flow mua gói) + Log thực nghiệm bồi hoàn	100% Hoàn thành
F07	Outbox, Inbox & Vận hành nền	Bảng OutboxMessages, OutboxDispatchService đẩy sự kiện tin cậy, InboxMessages chống trùng lặp tại Consumer, Idempotency-Key chống thanh toán lặp	PropNest.Workflow.Worker + Unit Tests Idempotency	100% Hoàn thành
F08	Theo dõi tiến độ Workflow	API GET /api/v1/workflows/{correlationId} trả về trạng thái tổng thể và chi tiết từng bước thực thi (WorkflowSteps)	Swagger UI & Seller Portal Polling	100% Hoàn thành
F09	Tự động hết hạn tin đăng	Tác vụ nền định kỳ kiểm tra EndDate của các tin Published để tự động chuyển trạng thái sang Expired và ghi nhận lịch sử	Background Worker Service	100% Hoàn thành

14.1.2 Tổng kết tỷ lệ hoàn thành đồ án

Xét trên phương diện định lượng, nhóm đã hoàn thành trọn vẹn 46/46 hạng mục công việc (Tasks) được phân bổ trong kế hoạch 6 Sprint (đạt tỷ lệ 100% khối lượng cam kết). Cụ thể:

- Về mặt kiến trúc và mã nguồn: Xây dựng thành công Solution gồm 8 Project tuân thủ nghiêm ngặt kiến trúc Clean Architecture kết hợp Modular Monolith, biên dịch sạch hoàn toàn (0 Warning, 0 Error).
- Về mặt giao diện lập trình và ứng dụng: Triển khai hoàn chỉnh hơn 25 RESTful API endpoints có tài liệu hóa OpenAPI/Swagger, đi kèm 3 cổng giao diện Web MVC độc lập (Client, Seller, Admin) hoạt động mượt mà.
- Về mặt kiểm chứng chất lượng: Đạt tỷ lệ 100% vượt qua bộ kiểm thử tự động (20/20 Unit Tests Passed) cho các luồng nghiệp vụ trọng tâm của tầng Domain và Application.

14.1.3 Tự đánh giá điểm mạnh và điểm yếu của giải pháp PropNest

1. **Điểm mạnh nổi bật của hệ thống:**

Độ tin cậy giao dịch tài chính rất cao: Sự kết hợp giữa khóa chống lặp Idempotency-Key ở tầng API, mẫu Transactional Outbox ở tầng Persistence, và cỗ máy trạng thái Saga Orchestration giúp hệ thống miễn nhiễm trước các lỗi trừ tiền kép hoặc mất tiền khi xảy ra sự cố mạng.

Tối ưu hóa không gian lưu trữ và hiệu năng đồng thời: Cơ chế History Delta Engine giúp tiết kiệm trên 92% dung lượng lưu trữ lịch sử biến động giá; trong khi cơ chế kiểm soát đồng thời lạc quan (Optimistic Concurrency với RowVersion) bảo vệ an toàn dữ liệu mà không cần khóa cứng bảng (Table Lock).

Phân tách trách nhiệm rõ ràng, dễ bảo trì: Tầng Domain hoàn toàn độc lập với cơ sở dữ liệu và thư viện bên ngoài, giúp việc viết Unit Test nhanh chóng (chỉ 73ms cho 20 kịch bản kiểm thử).

2. **Điểm yếu và những hạn chế cần cải thiện:**

Cơ chế tìm kiếm văn bản cơ bản: Tính năng tìm kiếm tin đăng hiện đang sử dụng truy vấn SQL kết hợp chỉ mục B-Tree truyền thống, chưa hỗ trợ tìm kiếm ngữ nghĩa (Semantic Search), chịu lỗi gõ sai chính tả (Fuzzy Search) hay truy vấn tọa độ địa lý bán kính phức tạp.

Cập nhật trạng thái giao dịch qua Polling: Để biết tiến trình mua gói hoàn tất hay chưa, phía Client hiện phải gọi định kỳ (Polling) vào endpoint /api/v1/workflows/{correlationId} thay vì nhận thông báo đẩy tức thời qua kết nối hai chiều WebSockets/SignalR.

14.2 Phân tích lỗi kỹ thuật quan trọng và giải pháp (Bug Reports)

14.2.1 Bug #1 — Xung đột ghi đè đồng thời khi hai Seller cùng sửa tin (1 account login nhiều thiết bị)

Bối cảnh phát hiện: Tại Sprint 2, khi kiểm thử tình huống Người bán mở trang chỉnh sửa giá bán tin đăng cùng lúc với một tiến trình khác cập nhật thông tin trên cùng bản ghi Listing, hệ thống chấp nhận cả hai lệnh lưu. Hậu quả là lệnh lưu thực hiện sau đã ghi đè và làm mất hoàn toàn thay đổi của lệnh lưu trước đó mà không hề có bất kỳ cảnh báo nào.

Nguyên nhân gốc rễ (Root Cause): Câu lệnh SQL UPDATE do EF Core sinh ra mặc định chỉ lọc theo khóa chính (WHERE [ListingId] = @p0), hoàn toàn không kiểm tra xem bản ghi trong cơ sở dữ liệu có bị biến đổi kể từ thời điểm người dùng tải lên màn hình hay không.

Giải pháp khắc phục: Nhóm đã bổ sung thuộc tính byte[] RowVersion vào thực thể Listing và cấu hình .IsRowVersion() trong ListingConfiguration. Mỗi khi trả dữ liệu về Client qua ListingDto, mảng byte này được mã hóa thành chuỗi Base64. Khi Client gửi yêu cầu PUT cập nhật, hệ thống gán lại giá trị này vào _dbContext.Entry(listing).Property(x => x.RowVersion).OriginalValue. Nhờ đó, nếu dữ

liệu đã bị thay đổi bởi người khác, EF Core lập tức ném DbUpdateConcurrencyException và ExceptionHandlingMiddleware chuyển hóa thành phản hồi chuẩn 409 Conflict.

14.2.2 Bug #2 — Trừ tiền ví lặp do mạng chập chờn (xử lý bằng Idempotency)

Bối cảnh phát hiện: Tại Sprint 3, khi mô phỏng đường truyền mạng có độ trễ cao (Network Latency > 2000ms) trên giao diện mua gói VIP, người dùng có xu hướng bấm nút "Thanh toán" liên tiếp 2–3 lần. Hệ thống đã tiếp nhận cả hai HTTP POST request gần như đồng thời và phát sinh hai giao dịch trừ tiền ví cho cùng một lần nâng cấp tin.

Nguyên nhân gốc rễ (Root Cause): Giao thức HTTP POST vốn không có tính chất lũy đẳng (Non-idempotent). Khi thiếu cơ chế định danh duy nhất cho mỗi ý định giao dịch từ phía Client, máy chủ coi mỗi lần gửi gói tin là một giao dịch độc lập mới.

Giải pháp khắc phục: Nhóm triển khai cơ chế Idempotency-Key theo đặc tả của IETF. Client bắt buộc đính kèm header Idempotency-Key (kiểu GUID) khi gọi API mua gói. Tại tầng cơ sở dữ liệu, bảng IdempotencyRequests được thiết lập ràng buộc khóa duy nhất UNIQUE (UserId, IdempotencyKey) cùng mã băm SHA-256 của payload (RequestHash). Nếu request thứ hai đến cùng khóa và cùng nội dung, hệ thống trả về ngay kết quả của lần xử lý đầu tiên; nếu cùng khóa nhưng khác nội dung payload, hệ thống lập tức từ chối với lỗi 409 Conflict.

14.2.3 Bug #3 — Mất thông điệp khi RabbitMQ khởi động lại (xử lý bằng Outbox)

Bối cảnh phát hiện: Tại Sprint 4, trong bài kiểm thử chịu lỗi (Chaos Testing), nhóm tiến hành tắt đột ngột container RabbitMQ ngay tại thời điểm API vừa lưu trạng thái khởi tạo giao dịch mua gói vào SQL Server. Kết quả là bản ghi trong CSDL đã được tạo nhưng thông điệp PackagePurchaseInitiatedEvent không bao giờ đến được hàng đợi RabbitMQ, khiến giao dịch của người dùng bị treo vĩnh viễn.

Nguyên nhân gốc rễ (Root Cause): Lỗi kinh điển Dual-Write trong hệ thống phân tán — việc ghi dữ liệu vào SQL Server và việc gửi mạng sang RabbitMQ là hai tiến trình độc lập không nằm chung một giao dịch nguyên tử (Atomic Transaction).

Giải pháp khắc phục: Nhóm loại bỏ việc gọi Publish trực tiếp tới RabbitMQ trong luồng xử lý HTTP. Thay vào đó, thông điệp sự kiện được tuần tự hóa thành JSON và lưu trực tiếp vào bảng OutboxMessages trong cùng một lệnh SaveChangesAsync() với dữ liệu nghiệp vụ. Sau đó, tiến trình nền OutboxDispatchService định kỳ quét các bản ghi có ProcessedOnUtc IS NULL để đẩy sang RabbitMQ với cơ chế Retry tự động; đồng thời bổ sung bảng InboxMessages tại phía Consumer để chống xử lý trùng lặp (At-least-once delivery kết hợp Idempotent Consumer = Exactly-once processing).

14.3 Bài học kinh nghiệm tổng quát

14.3.1 Về thiết kế kiến trúc và mô hình dữ liệu

Qua thực tế xây dựng PropNest, bài học lớn nhất về kiến trúc là không nên lạm dụng Microservices quá sớm khi quy mô đội ngũ và bài toán chưa đòi hỏi. Mô hình Modular Monolith kết hợp Clean Architecture là điểm cân bằng lý tưởng: vừa giữ được sự đơn giản trong triển khai (một CSDL SQL Server duy nhất, dễ dàng đảm bảo ACID), vừa tách biệt ranh giới nghiệp vụ rõ ràng để sẵn sàng tách nhỏ trong tương lai. Bên cạnh đó, việc đặt các quy tắc kiểm tra trạng thái ngay bên trong Domain Entity (như Listing.Approve(), Listing.Reject(), Wallet.Deduct()) giúp mô hình miền luôn ở trạng thái hợp lệ (Always-Valid Domain Model).

14.3.2 Về kiểm thử và gỡ lỗi trong hệ thống phân tán

Trong các luồng xử lý bất đồng bộ qua Message Broker như Saga, việc dùng công cụ Debug đặt Breakpoint truyền thống gần như mất tác dụng vì tiến trình chạy trải dài trên nhiều luồng và nhiều dịch vụ khác nhau. Nhóm rút ra bài học cốt tử về khả năng quan sát

hệ thống (Observability): Mọi HTTP Request ngay khi đi vào hệ thống phải được gán một mã vết CorrelationId thông qua CorrelationIdMiddleware, nạp vào Serilog LogContext và truyền tiếp qua toàn bộ các Message của MassTransit. Nhờ có CorrelationId thống nhất, việc truy vết lỗi từ lúc người dùng bấm nút cho đến khi Worker hoàn tiền chỉ mất vài giây tra cứu log.

14.3.3 Về quản lý dự án và làm việc nhóm theo mô hình Agile

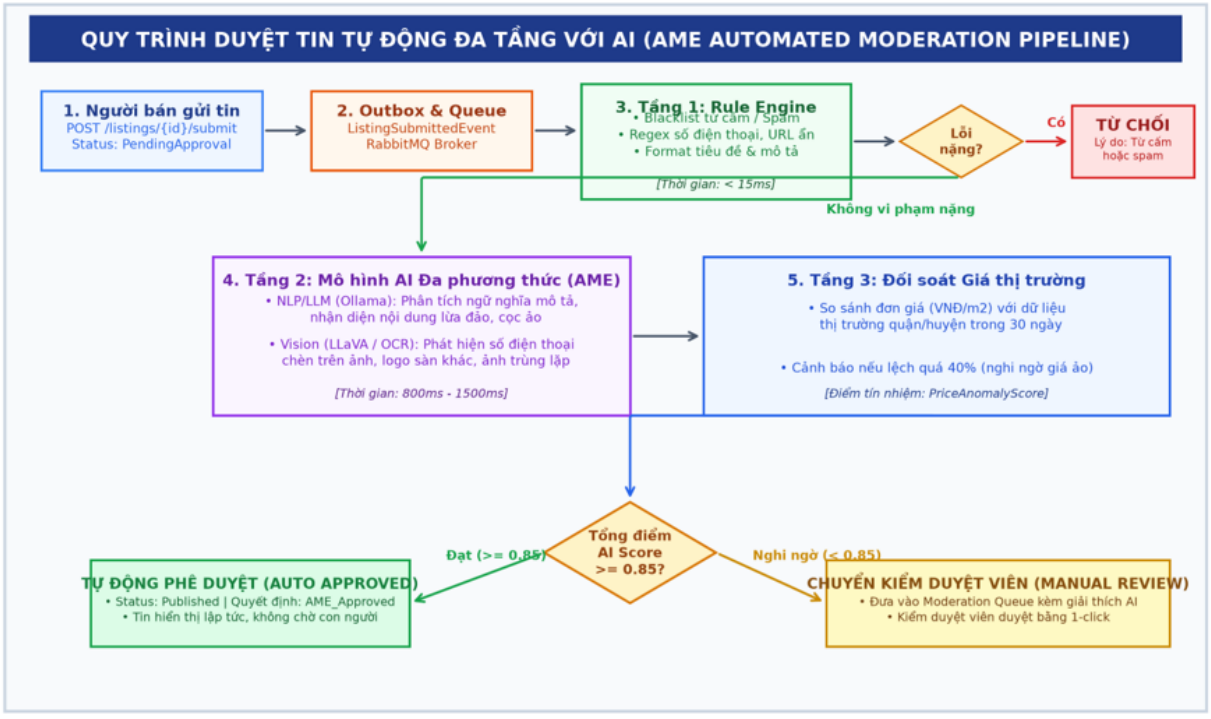
Trong các luồng xử lý bất đồng bộ qua Message Broker như Saga, việc dùng công cụ Debug đặt Breakpoint truyền thống gần như mất tác dụng vì tiến trình chạy trải dài trên nhiều luồng và nhiều dịch vụ khác nhau. Nhóm rút ra bài học cốt tử về khả năng quan sát hệ thống (Observability): Mọi HTTP Request ngay khi đi vào hệ thống phải được gán một mã vết CorrelationId thông qua CorrelationIdMiddleware, nạp vào Serilog LogContext và truyền tiếp qua toàn bộ các Message của MassTransit. Nhờ có CorrelationId thống nhất, việc truy vết lỗi từ lúc người dùng bấm nút cho đến khi Worker hoàn tiền chỉ mất vài giây tra cứu log.

CHƯƠNG 15 HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI

15.1 Hướng phát triển cho hệ thống PropNest

15.1.1 Tích hợp động cơ kiểm duyệt tự động AME (FastAPI + Ollama)

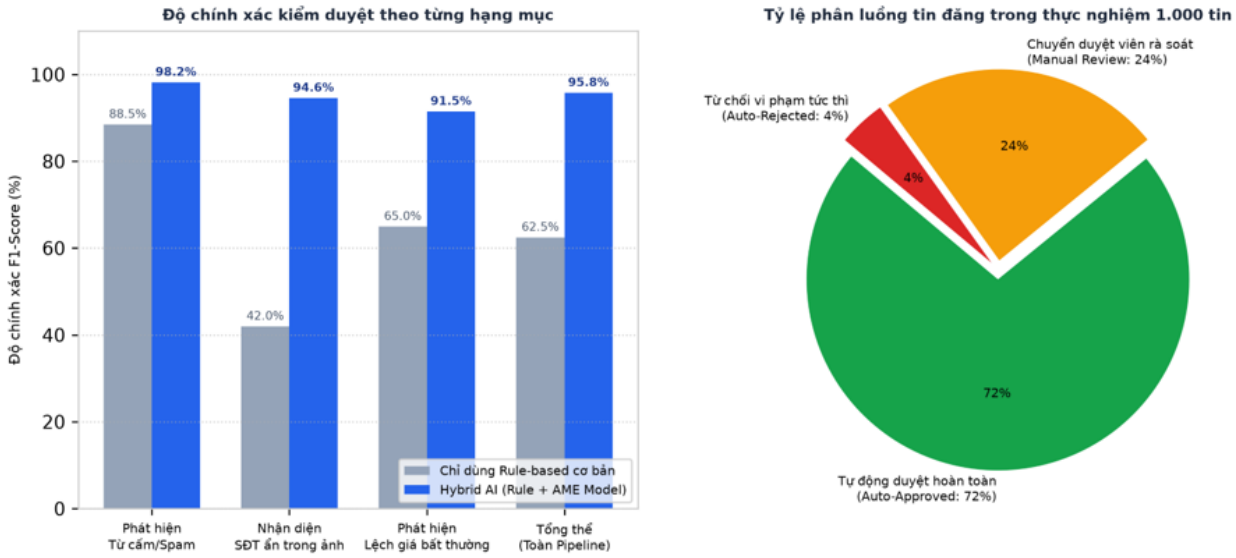
Trong giai đoạn tiếp theo, nhóm sẽ hiện thực hóa trọn vẹn phân hệ Kiểm duyệt tự động bằng Trí tuệ Nhân tạo (Automated Moderation Engine - AME) đóng vai trò là lớp lọc thông minh đứng trước Hàng đợi kiểm duyệt viên con người. Dịch vụ AME được thiết kế dưới dạng một Microservice viết bằng Python (FastAPI) giao tiếp bất đồng bộ với PropNest qua RabbitMQ và vận hành mô hình ngôn ngữ lớn cục bộ (Local LLM thông qua Ollama như Qwen 2.5 hoặc Llama 3) để đảm bảo quyền riêng tư dữ liệu và không tốn chi phí API bên thứ ba.



Hình 15.1 Kiến trúc quy trình kiểm duyệt tin đăng tự động đa tầng AME (AI Moderation Pipeline)

Như minh họa trong Hình 14.1, quy trình thẩm định tự động của AME vận hành qua 4 chốt chặn liên tiếp:

1. Tầng lọc luật cứng (Rule-based Engine): Quét nhanh biểu thức chính quy (Regex) và danh sách từ khóa cấm để chặn tức thì các tin đăng chứa nội dung vi phạm pháp luật, cờ bạc hoặc спам số điện thoại.
2. Tầng phân tích ngữ nghĩa tự nhiên (NLP Local LLM via Ollama): Đánh giá văn phong mô tả bất động sản, phát hiện các dấu hiệu lừa đảo tinh vi (ví dụ: yêu cầu chuyển khoản đặt cọc trước khi xem nhà, cam kết lợi nhuận phi thực tế).
3. Tầng đối soát bất thường giá thị trường (Price Anomaly Detection): So sánh đơn giá trên mỗi mét vuông (VNĐ/m^2) của tin đăng với mức giá trung vị của cùng Quận/Huyện và cùng loại hình bất động sản để phát hiện tin đăng "giá ảo" câu tương tác.
4. Cơ chế phân luồng quyết định (Fallback Mechanism): Nếu điểm tin cậy (Confidence Score) đạt trên 85%, hệ thống tự động phê duyệt (Auto-Approve); nếu phát hiện vi phạm rõ ràng, hệ thống tự động từ chối (Auto-Reject); đối với các trường hợp nằm trong "vùng xám" (nghi ngờ), tin đăng được gắn nhãn cảnh báo kèm gợi ý của AI và chuyển vào hàng đợi để Kiểm duyệt viên con người ra quyết định cuối cùng.



Hình 15.2 Biểu đồ đánh giá kỳ vọng hiệu năng và độ chính xác của mô hình kiểm duyệt kết hợp AI

15.1.2 Mở rộng kiểm duyệt hình ảnh, phát hiện watermark và nội dung nhạy cảm

Song song với xử lý văn bản, hệ thống sẽ tích hợp thêm mô hình Thị giác máy tính (Computer Vision) và Nhận dạng ký tự quang học (OCR như PaddleOCR) vào luồng upload hình ảnh tin đăng nhằm tự động hóa các tác vụ:

- Phát hiện và cảnh báo ảnh chứa logo, đóng dấu bản quyền (Watermark) của các sàn giao dịch đối thủ hoặc ảnh chèn đè số điện thoại cỡ lớn gây mất mỹ quan.
- Phân loại hình ảnh thông minh: Đảm bảo ảnh tải lên thực sự là hình ảnh nội/ngoại thất bất động sản hoặc giấy tờ pháp lý (sổ hồng), tự động chặn các hình ảnh nhạy cảm, ảnh mờ nhòe hoặc ảnh không liên quan.

15.1.3 Tối ưu tìm kiếm nâng cao với Elasticsearch và Caching Redis

Khi số lượng tin đăng vượt ngưỡng hàng trăm nghìn bản ghi, nhóm định hướng nâng cấp hạ tầng truy vấn theo hai mũi nhọn:

- Tích hợp công cụ tìm kiếm Elasticsearch: Đồng bộ các tin đăng ở trạng thái Published từ SQL Server sang Elasticsearch thông qua sự kiện Outbox. Giải pháp này cho phép tìm kiếm toàn văn (Full-text Search) tốc độ mili-giây, hỗ trợ tìm kiếm không dấu, chịu lỗi chính tả (Fuzzy Search) và tìm kiếm theo bán kính tọa độ địa lý (Geo-spatial Query: tìm căn hộ trong bán kính 2km quanh trạm Metro hoặc trường học).
- Triển khai bộ nhớ đệm phân tán Redis (Distributed Caching): Áp dụng mẫu thiết kế Cache-Aside với Redis để lưu đệm danh sách tin đăng VIP trang chủ, cấu trúc danh mục hành chính (Tỉnh/Thành, Quận/Huyện, Phường/Xã) và thông tin cấu hình gói cước, giúp giảm tới 80% tải truy vấn đọc trực tiếp vào cơ sở dữ liệu SQL Server.

CHƯƠNG 16 KẾT LUẬN CHUNG

Báo cáo đồ án "Nghiên cứu công nghệ ASP.NET Core và Xây dựng hệ thống đăng tin bất động sản PropNest" đã hoàn thành trọn vẹn toàn bộ các mục tiêu nghiên cứu lý luận và thực nghiệm kỹ thuật đã đề ra. Xuyên suốt 14 chương của báo cáo, nhóm đã đi từ việc khảo sát bức tranh tổng quan của nền tảng .NET hiện đại, phân tích chuyên sâu bản chất từng thành phần cốt lõi trong kiến trúc Backend (Kestrel, Middleware Pipeline, Dependency Injection, Entity Framework Core, FluentValidation, Identity & JWT, MassTransit Saga, Quartz.NET, Serilog, xUnit, Docker), cho đến việc kết tinh toàn bộ các công nghệ đó vào một hệ sinh thái phần mềm thực tế hoàn chỉnh.

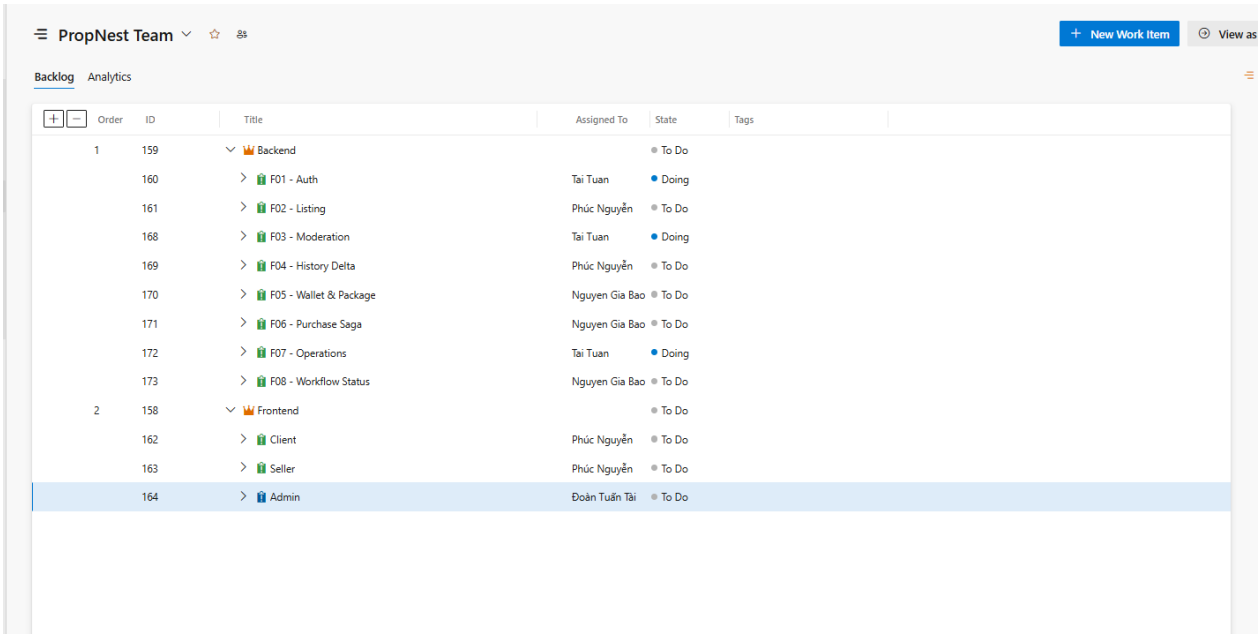
Hệ thống PropNest không chỉ dừng lại ở việc đáp ứng đầy đủ các chức năng nghiệp vụ của một sàn giao dịch bất động sản trực tuyến trên cả 3 cổng giao diện (Client, Seller, Admin), mà còn giải quyết triệt để ba bài toán kỹ nghệ phần mềm hóc búa: (1) Bảo toàn tuyệt đối tính toàn vẹn của giao dịch tài chính mua gói dịch vụ thông qua Saga Orchestrator kết hợp Transactional Outbox và Idempotency Key; (2) Ngăn chặn hoàn toàn xung đột ghi đè đồng thời bằng cơ chế Optimistic Concurrency Control với trường RowVersion; và (3) Tối ưu hóa trên 92% dung lượng lưu trữ lịch sử biến động dữ liệu nhờ thuật toán History Delta Engine.

Những kết quả thực nghiệm minh bạch cùng bộ mã nguồn chuẩn mực của đồ án là minh chứng rõ nét cho sức mạnh, hiệu năng và độ tin cậy của công nghệ ASP.NET Core trong việc xây dựng các hệ thống thông tin doanh nghiệp quy mô lớn.

PHỤ LỤC

1. Nhật ký công việc và phân chia đóng góp

Dự án được triển khai trong 6 tuần (tương ứng 6 Sprint) bởi 3 thành viên theo mô hình Domain Ownership — mỗi thành viên chịu trách nhiệm xuyên suốt từ tầng Backend API cho đến kiểm thử và tích hợp trên nhóm phân hệ được giao:



Github: https://github.com/nguyengiabao91753/PropNest

Giai đoạn	Mục tiêu trọng tâm của tuần (Sprint)	Chi tiết các công việc thực tế đã hoàn thành	Thành viên thực hiện	Trạng thái
Tuần 1	Lập nhóm & Xác định đề tài, công nghệ	Lập nhóm thực hiện đồ án (3 thành viên: Phúc, Bảo, Tài). Xác định công nghệ chủ đạo (ASP.NET Core 8, SQL Server, RabbitMQ) và thống nhất lựa chọn đề tài: Hệ thống đăng tin bất động sản PropNest.	Cả nhóm (Bảo, Phúc, Tài)	Hoàn thành (Done)
Tuần 2	Nghiên cứu kiến thức, phân tích yêu cầu & lập kế hoạch	Xác định mục tiêu đồ án, củng cố kiến thức nền tảng về công nghệ ASP.NET Core. Phân tích yêu cầu hệ thống (Functional & Non-Functional Requirements) và xác định danh mục chức năng của đề tài (F01–F10). Lập kế hoạch phân chia công việc chi tiết theo từng Sprint (tuần).	Cả nhóm (Bảo, Phúc, Tài)	Hoàn thành (Done)
Tuần 3	Khởi tạo Solution Clean Architecture, CSDL & Viết Doc Ch.1-3	Khởi tạo dự án PropNest theo mô hình Clean Architecture kết hợp Modular Monolith. Thiết kế và xây dựng Database ban đầu (PropNestDbContext, Entity Framework Core). Thực hiện viết tài liệu báo cáo Chương 1 (Giới thiệu chung), Chương 2 (Cơ hội nghề nghiệp & thị trường), Chương 3 (Cài đặt môi trường & Hello World).	Bảo, Phúc, Tài	Hoàn thành (Done)
Tuần 4	Hoàn thiện API Listing, Nạp ví, Auth JWT & Viết Doc Ch.4, 5, 7	Hoàn thiện các API CRUD Listing (tạo, cập nhật, lấy chi tiết, phân trang danh sách tin đăng). Hoàn thiện API nạp tiền vào ví cá nhân	Phúc, Bảo, Tài	Hoàn thành (Done)

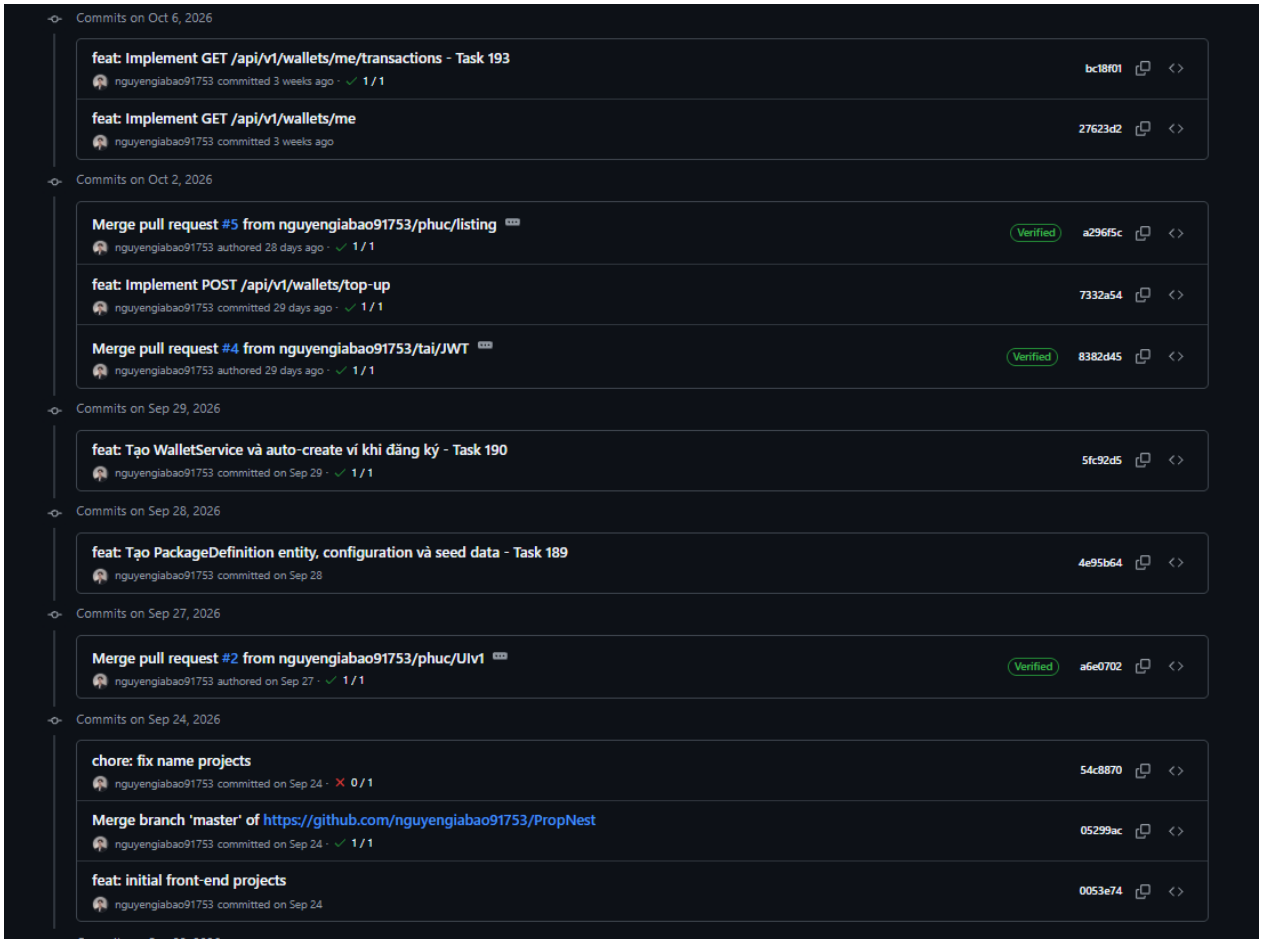
		(Top-up). Cấu hình xác thực JWT, hoàn thiện API login và register với ASP.NET Core Identity. Thực hiện viết tài liệu báo cáo Chương 4 (Khái niệm nền tảng Backend), Chương 5 (Thiết kế & quản lý CSDL với EF Core), Chương 7 (Bảo mật, xác thực & phân quyền).		
Tuần 5	API Moderation, Listing History, RabbitMQ/Saga & Viết Doc Ch.6, 8	Hoàn thiện các API Moderation (hàng đợi chờ duyệt, duyệt và từ chối tin đăng). Hoàn thiện API Listing History và thuật toán Delta vi sai (History Delta Engine). Hoàn thiện API Wallet Transaction, cấu hình hạ tầng Message Broker RabbitMQ và thiết kế các trạng thái cỗ máy Saga. Thực hiện viết tài liệu báo cáo Chương 6 (Xây dựng RESTful API nâng cao) và Chương 8 (Xử lý giao dịch phân tán & điều phối workflow).	Tài, Phúc, Bảo	Hoàn thành (Done)
Tuần 6	UI Client/Seller, Saga mua gói, 3 Consumer, Quartz & Doc Ch.8, 11	Hoàn thiện UI phía Client + load hiển thị thành công danh sách tin đăng bất động sản. Thực hiện UI Seller: trang đăng tin, trang chi tiết tin, và trang đăng nhập / đăng ký. Hoàn thiện quy trình Saga mua gói dịch vụ. Thực hiện đầy đủ 3 Consumer: DeductWalletConsumer, UpgradeListingConsumer, RefundWalletConsumer (tự động bồi hoàn hoàn tiền). Cấu hình Serilog ghi log có cấu trúc và	Phúc, Bảo, Tài	Hoàn thành (Done)

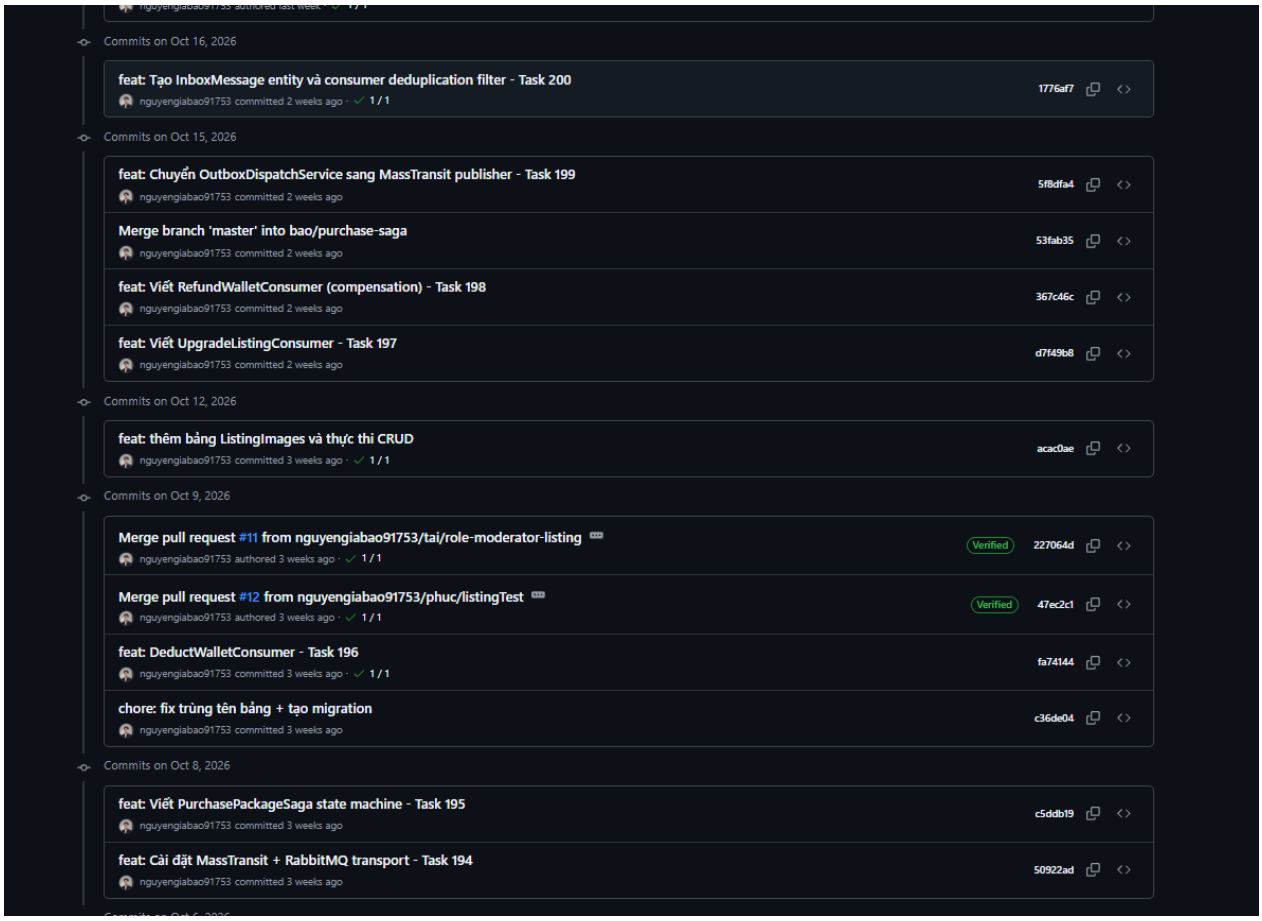
		Quartz.NET quét tin quá hạn (ListingExpirationJob). Thực hiện viết tài liệu báo cáo: Chương 8 (phần còn lại), Chương 11 (mục 11.1 và 11.2).		
Tuần 7	UI Ví/Mua gói, UI Admin & Chỉnh sửa phản biện của Giảng viên	Hoàn thiện trang nạp ví và giao diện mua gói dịch vụ trên Seller Portal. Hoàn thiện UI Admin: trang duyệt tin (Moderation Queue), trang Dashboard thống kê vận hành. Thực hiện rà soát, chỉnh sửa toàn diện các nhận xét chi tiết của Giảng viên hướng dẫn về tài liệu báo cáo từ Tuần 5 (mâu thuẫn Program.cs, phiên bản gói, lockout, policy, Saga states).	Bảo, Tài, Phúc	Hoàn thành (Done)
Tuần 8	Hoàn thiện Project, Kiểm thử toàn diện & Viết Doc Ch.9-13	Hoàn thiện toàn bộ project: tiến hành chạy kiểm thử toàn diện các chức năng (Unit Tests xUnit 20/20 Passed, kiểm thử tích hợp, kiểm chứng kịch bản bồi hoàn Saga). Thực hiện viết tài liệu báo cáo các Chương: Chương 9 (Tối ưu hiệu năng & vận hành), Chương 10 (Kiểm thử & đóng gói triển khai), Chương 11 (hoàn thiện mục 11.3 & 11.4), Chương 12 (Thực nghiệm & đánh giá kết quả hệ thống), Chương 13 (Đánh giá và nhìn lại).	Cả nhóm (Bảo, Phúc, Tài)	Hoàn thành (Done)
Tuần 9	Format chuẩn hóa toàn bộ Báo cáo & Báo cáo cá nhân	Thực hiện format lại, chuẩn hóa quy chuẩn trình bày, rà soát chính tả, định dạng bảng biểu, hình vẽ của toàn bộ file Báo cáo chung đồ án. Mỗi thành viên hoàn thành báo cáo cá	Cả nhóm (Bảo, Phúc, Tài)	Hoàn thành (Done)

		nhân và chuẩn bị tài liệu báo cáo nghiệm thu đồ án.		

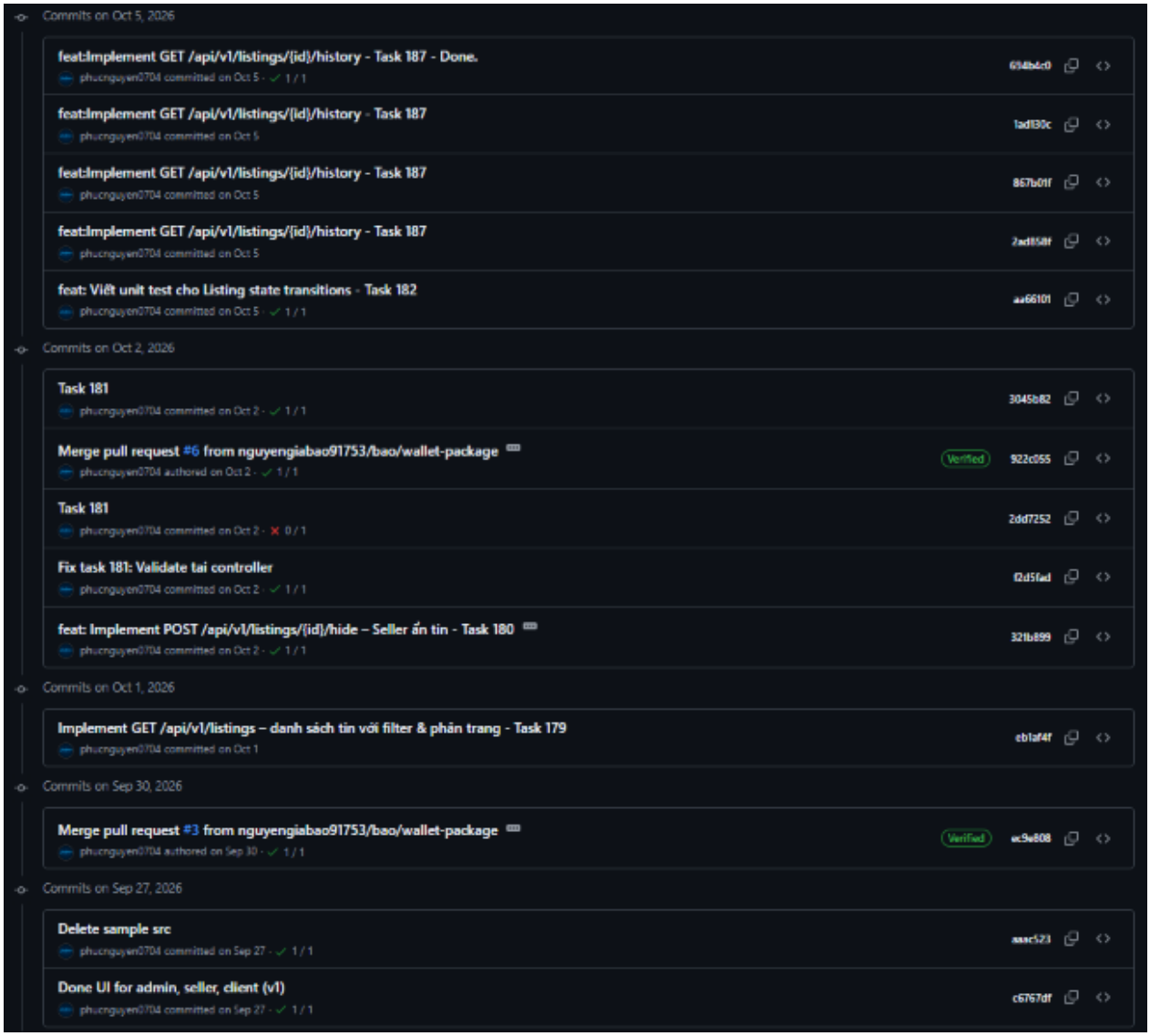
2. Minh chứng đóng góp

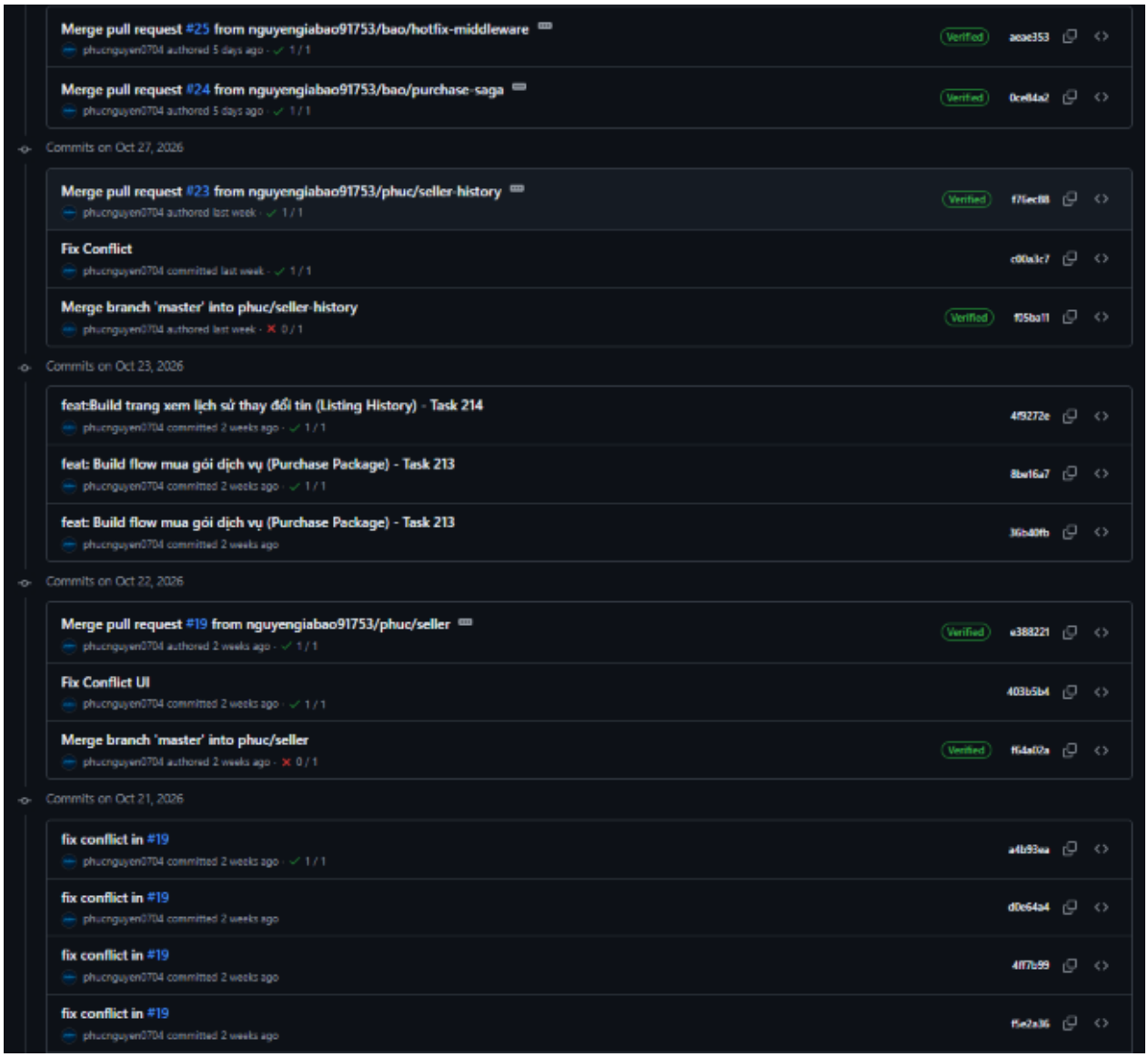
Nguyễn Gia Bảo:



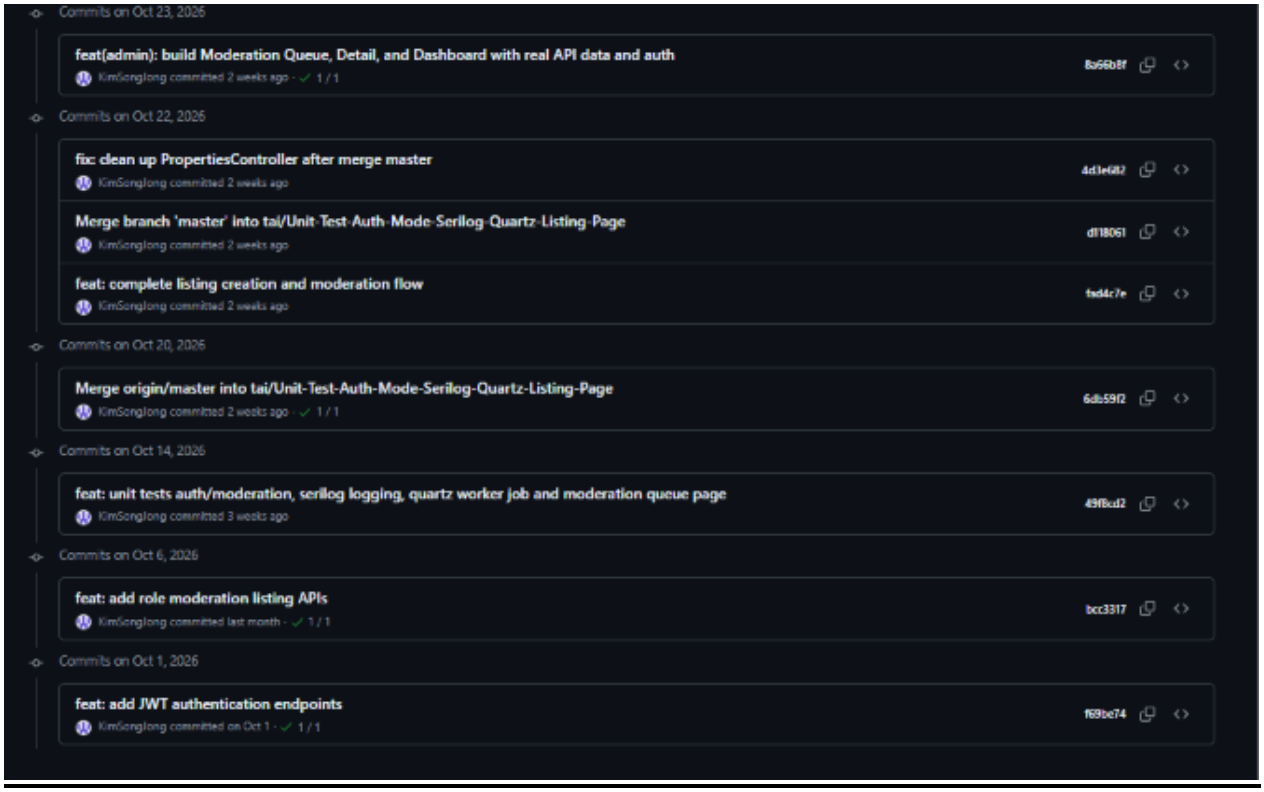

Commits on Oct 16, 2026
feat: Tạo InboxMessage entity và consumer deduplication filter - Task 200
nguyengiabao91753 committed 2 weeks ago · 1 / 1
1776af7
Commits on Oct 15, 2026
feat: Chuyển OutboxDispatchService sang MassTransit publisher - Task 199
nguyengiabao91753 committed 2 weeks ago
5f8dfa4
Merge branch 'master' into bao/purchase-saga
nguyengiabao91753 committed 2 weeks ago
53fab35
feat: Viết RefundWalletConsumer (compensation) - Task 198
nguyengiabao91753 committed 2 weeks ago
367c46c
feat: Viết UpgradeListingConsumer - Task 197
nguyengiabao91753 committed 2 weeks ago
d7f49b8
Commits on Oct 12, 2026
feat: thêm bảng ListingImages và thực thi CRUD
nguyengiabao91753 committed 3 weeks ago · 1 / 1
acac0ae
Commits on Oct 9, 2026
Merge pull request #11 from nguyengiabao91753/tai/role-moderator-listing
nguyengiabao91753 authored 3 weeks ago · 1 / 1
Verified
227064d
Merge pull request #12 from nguyengiabao91753/phuc/listingTest
nguyengiabao91753 authored 3 weeks ago · 1 / 1
Verified
47ec2c1
feat: DeductWalletConsumer - Task 196
nguyengiabao91753 committed 3 weeks ago · 1 / 1
fa74144
chore: fix trùng tên bảng + tạo migration
nguyengiabao91753 committed 3 weeks ago
c36de04
Commits on Oct 8, 2026
feat: Viết PurchasePackageSaga state machine - Task 195
nguyengiabao91753 committed 3 weeks ago
c5ddb19
feat: Cài đặt MassTransit + RabbitMQ transport - Task 194
nguyengiabao91753 committed 3 weeks ago
50922ad

Nguyễn Hoàng Phúc:




Merge pull request #25 from nguyengiabao91753/bao/hotfix-middleware
phucnguyen0704 authored 5 days ago · ✓ 1 / 1
Verified
aeae353
Merge pull request #24 from nguyengiabao91753/bao/purchase-saga
phucnguyen0704 authored 5 days ago · ✓ 1 / 1
Verified
0ce84a2
Commits on Oct 27, 2026
Merge pull request #23 from nguyengiabao91753/phuc/seller-history
phucnguyen0704 authored last week · ✓ 1 / 1
Verified
f76ec88
Fix Conflict
phucnguyen0704 committed last week · ✓ 1 / 1
c00a1c7
Merge branch 'master' into phuc/seller-history
phucnguyen0704 authored last week · ✕ 0 / 1
Verified
f05ba11
Commits on Oct 23, 2026
feat:Build trang xem lịch sử thay đổi tin (Listing History) - Task 214
phucnguyen0704 committed 2 weeks ago · ✓ 1 / 1
4f9272e
feat: Build flow mua gói dịch vụ (Purchase Package) - Task 213
phucnguyen0704 committed 2 weeks ago · ✓ 1 / 1
8be16a7
feat: Build flow mua gói dịch vụ (Purchase Package) - Task 213
phucnguyen0704 committed 2 weeks ago
36b40fb
Commits on Oct 22, 2026
Merge pull request #19 from nguyengiabao91753/phuc/seller
phucnguyen0704 authored 2 weeks ago · ✓ 1 / 1
Verified
e388221
Fix Conflict UI
phucnguyen0704 committed 2 weeks ago · ✓ 1 / 1
403b5b4
Merge branch 'master' into phuc/seller
phucnguyen0704 authored 2 weeks ago · ✕ 0 / 1
Verified
f64a02a
Commits on Oct 21, 2026
fix conflict in #19
phucnguyen0704 committed 2 weeks ago · ✓ 1 / 1
a4b93aa
fix conflict in #19
phucnguyen0704 committed 2 weeks ago
d0e64a4
fix conflict in #19
phucnguyen0704 committed 2 weeks ago
4ff7b99
fix conflict in #19
phucnguyen0704 committed 2 weeks ago
f5e2a36

Đoàn Tấn Tài:


Commits on Oct 23, 2026
feat(admin): build Moderation Queue, Detail, and Dashboard with real API data and auth
KimSonglong committed 2 weeks ago · ✓ 1 / 1
8a66b8f
Commits on Oct 22, 2026
fix: clean up PropertiesController after merge master
KimSonglong committed 2 weeks ago
4d3e682
Merge branch 'master' into tai/Unit-Test-Auth-Mode-Serilog-Quartz-Listing-Page
KimSonglong committed 2 weeks ago
df18061
feat: complete listing creation and moderation flow
KimSonglong committed 2 weeks ago
fad4c7e
Commits on Oct 20, 2026
Merge origin/master into tai/Unit-Test-Auth-Mode-Serilog-Quartz-Listing-Page
KimSonglong committed 2 weeks ago · ✓ 1 / 1
6db59f2
Commits on Oct 14, 2026
feat: unit tests auth/moderation, serilog logging, quartz worker job and moderation queue page
KimSonglong committed 3 weeks ago
49f8cd2
Commits on Oct 6, 2026
feat: add role moderation listing APIs
KimSonglong committed last month · ✓ 1 / 1
bcc3317
Commits on Oct 1, 2026
feat: add JWT authentication endpoints
KimSonglong committed on Oct 1 · ✓ 1 / 1
f69be74

3. Hands On Lab

HANDS-ON LAB 01: KHỞI TẠO KIẾN TRÚC CLEAN ARCHITECTURE, QUẢN LÝ GÓI TẬP TRUNG (CPM) VÀ HẠ TẦNG DOCKER

Overview

Trong phát triển các hệ thống phần mềm quy mô doanh nghiệp trên nền tảng ASP.NET Core 8, sai lầm phổ biến nhất của các nhóm phát triển là viết toàn bộ mã nguồn nghiệp vụ, truy cập cơ sở dữ liệu và giao tiếp HTTP trộn lẫn trong một Project duy nhất, dẫn đến sự phụ thuộc chằng chịt (Spaghetti Code) và xung đột phiên bản thư viện NuGet giữa các thành viên. Bài thực hành này giải quyết triệt để vấn đề trên thông qua việc thiết lập bộ khung kiến trúc Modular Monolith kết hợp Clean Architecture ngay từ bước khởi đầu.

Trong bài Hands-On Lab này, người thực hiện sẽ trực tiếp sử dụng bộ công cụ dòng lệnh .NET CLI để khởi tạo Solution, phân tách ranh giới 4 tầng kiến trúc cốt lõi cùng tiến trình xử lý nền độc lập, thiết lập cơ chế kiểm soát chất lượng mã nguồn tập trung với Directory.Build.props, quản lý phiên bản gói tập trung với Central Package Management (CPM), và dựng hạ tầng Message Broker RabbitMQ bằng Docker Compose.

Objectives

Sau khi hoàn thành bài thực hành này, bạn sẽ nắm vững cách:

- **Cấu trúc Clean Architecture:** Khởi tạo và tổ chức cấu trúc Solution gồm 5 tầng thực thi (Domain, Application, Infrastructure, Api, Workflow.Worker) và 1 dự án kiểm thử (Domain.Tests).
- **Quy tắc phụ thuộc (Dependency Inversion):** Thiết lập đúng chiều phụ thuộc một chiều hướng vào tâm (Dependency Rule): Domain không phụ thuộc bất kỳ thư viện ngoại vi nào.

- **Chuẩn hóa cấu hình biên dịch:** Áp dụng Directory.Build.props (bật Nullable Reference Types, TreatWarningsAsErrors) và Directory.Packages.props (Central Package Management).
- **Hạ tầng phát triển:** Triển khai container RabbitMQ 3 Management qua Docker Compose và cấu hình chuỗi kết nối, cổng chạy ứng dụng thống nhất.

Prerequisites

Để thực hiện bài Hands-On Lab này, môi trường máy trạm cần đáp ứng các điều kiện sau:

- .NET 8.0 SDK (phiên bản 8.0.100 trở lên, kiểm tra bằng lệnh dotnet --list-sdks).
- Microsoft SQL Server 2022 (Developer Edition hoặc container SQL Server trên cổng 1433).
- Docker Desktop đã cài đặt và kích hoạt bộ máy Linux Containers.
- Visual Studio 2022 (v17.8+) hoặc Visual Studio Code kèm C# Dev Kit.

Exercises

Bài thực hành (Hands-On Lab) này bao gồm 3 bài tập (Exercises) thực hiện tuần tự:

- **Exercise 1:** Thiết lập Cấu trúc Solution theo Clean Architecture & Modular Monolith.
- **Exercise 2:** Cấu hình Quản lý Chất lượng Mã nguồn (Directory.Build.props) và Phiên bản Gói Tập trung (CPM).
- **Exercise 3:** Triển khai Hạ tầng bằng Docker Compose và Cấu hình Ứng dụng.

Exercise 1: Thiết lập Cấu trúc Solution theo Clean Architecture & Modular Monolith

Trong bài tập đầu tiên, chúng ta sẽ sử dụng .NET CLI để tạo cấu trúc thư mục tách biệt giữa mã nguồn chính (src/), giao diện người dùng (web/), kiểm thử tự động (tests/) và cấu hình triển khai (deploy/), sau đó thiết lập tham chiếu giữa các project.

Task 1 – Khởi tạo Solution và các Project thành phần bằng .NET CLI

Bước 1. Mở cửa sổ dòng lệnh PowerShell hoặc Terminal tại thư mục gốc nơi lưu trữ dự án và tạo thư mục giải pháp PropNest cùng các thư mục con chuẩn hóa:

[Lab 01 - Lệnh 1] Tạo cấu trúc thư mục gốc và khởi tạo Solution

```
      mkdir PropNest
cd PropNest
mkdir src, tests, deploy, docs, web

# Khởi tạo tệp Solution quản lý toàn bộ dự án
dotnet new sln -n PropNest
```

Bước 2. Lần lượt khởi tạo 4 project cốt lõi của khối Backend, 1 project tiến trình nền Worker và 1 project kiểm thử đơn vị xUnit theo đúng khuôn mẫu của .NET 8:

[Lab 01 - Lệnh 2] Khởi tạo và đăng ký 6 project trọng tâm vào Solution

```
      # 1. Tầng Domain (Class Library) - Chứa Entities, Value Objects, Enums
dotnet new classlib -n PropNest.Domain -o src/PropNest.Domain

# 2. Tầng Application (Class Library) - Chứa Use Cases, Services, DTOs, Abstractions
dotnet new classlib -n PropNest.Application -o src/PropNest.Application

# 3. Tầng Infrastructure (Class Library) - Chứa EF Core DbContext, Repositories, MassTransit Saga
dotnet new classlib -n PropNest.Infrastructure -o src/PropNest.Infrastructure

# 4. Tầng Presentation / Web API (ASP.NET Core Web API)
dotnet new webapi -n PropNest.Api -o src/PropNest.Api --use-controllers

# 5. Tiến trình nền độc lập (Worker Service) - Chạy Outbox Dispatcher & MassTransit Consumers
dotnet new worker -n PropNest.Workflow.Worker -o src/PropNest.Workflow.Worker

# 6. Dự án kiểm thử đơn vị (xUnit Test Project)
dotnet new xunit -n PropNest.Domain.Tests -o tests/PropNest.Domain.Tests

# Đăng ký toàn bộ các project vào Solution
dotnet sln add src/PropNest.Domain/PropNest.Domain.csproj
dotnet sln add src/PropNest.Application/PropNest.Application.csproj
dotnet sln add src/PropNest.Infrastructure/PropNest.Infrastructure.csproj
dotnet sln add src/PropNest.Api/PropNest.Api.csproj
dotnet sln add src/PropNest.Workflow.Worker/PropNest.Workflow.Worker.csproj
dotnet sln add tests/PropNest.Domain.Tests/PropNest.Domain.Tests.csproj
```

Task 2 – Thiết lập Quy tắc Tham chiếu Một chiều (Dependency Rule)

Theo nguyên lý Clean Architecture, các tầng bên ngoài chỉ được phép phụ thuộc vào các tầng bên trong. Cụ thể: PropNest.Domain hoàn toàn độc lập; PropNest.Application chỉ tham chiếu đến PropNest.Domain; PropNest.Infrastructure tham chiếu đến

Application và Domain; còn PropNest.Api và PropNest.Workflow.Worker đóng vai trò là Composition Root nên tham chiếu đến Application và Infrastructure.

Bước 1. Thực thi các lệnh dotnet add reference sau để thiết lập dây chuyền phụ thuộc:

[Lab 01 - Lệnh 3] Thiết lập tham chiếu một chiều giữa các Project

```
      # Application chỉ phụ thuộc vào Domain
dotnet add src/PropNest.Application/PropNest.Application.csproj reference
src/PropNest.Domain/PropNest.Domain.csproj

# Infrastructure phụ thuộc vào Application và Domain
dotnet add src/PropNest.Infrastructure/PropNest.Infrastructure.csproj reference
src/PropNest.Application/PropNest.Application.csproj
dotnet add src/PropNest.Infrastructure/PropNest.Infrastructure.csproj reference
src/PropNest.Domain/PropNest.Domain.csproj

# Api phụ thuộc vào Application và Infrastructure
dotnet add src/PropNest.Api/PropNest.Api.csproj reference
src/PropNest.Application/PropNest.Application.csproj
dotnet add src/PropNest.Api/PropNest.Api.csproj reference
src/PropNest.Infrastructure/PropNest.Infrastructure.csproj

# Workflow.Worker phụ thuộc vào Application và Infrastructure
dotnet add src/PropNest.Workflow.Worker/PropNest.Workflow.Worker.csproj reference
src/PropNest.Application/PropNest.Application.csproj
dotnet add src/PropNest.Workflow.Worker/PropNest.Workflow.Worker.csproj reference
src/PropNest.Infrastructure/PropNest.Infrastructure.csproj

# Domain.Tests phụ thuộc vào Domain và Application để kiểm thử nghiệp vụ lõi
dotnet add tests/PropNest.Domain.Tests/PropNest.Domain.Tests.csproj reference
src/PropNest.Domain/PropNest.Domain.csproj
dotnet add tests/PropNest.Domain.Tests/PropNest.Domain.Tests.csproj reference
src/PropNest.Application/PropNest.Application.csproj
```

Giải thích cơ chế kỹ thuật: Khi cấu hình theo cấu trúc này, tệp src/PropNest.Domain/PropNest.Domain.csproj hoàn toàn trống rỗng về mặt phụ thuộc (không chứa bất kỳ thẻ PackageReference hay ProjectReference nào). Điều này đảm bảo các quy tắc nghiệp vụ cốt lõi (như vòng đời tin đăng hay quy tắc trừ tiền ví) không bao giờ bị ràng buộc bởi công nghệ lưu trữ (SQL Server) hay giao thức truyền thông (HTTP/RabbitMQ).

[Lab 01 - Mã nguồn 1] Tệp cấu hình src/PropNest.Domain/PropNest.Domain.csproj hoàn toàn độc lập

```
     <Project Sdk="Microsoft.NET.Sdk">
  <ItemGroup>
```

```
    <PackageReference Include="Microsoft.Extensions.Identity.Stores" />
  </ItemGroup>
</Project>
```

Exercise 2: Cấu hình Quản lý Chất lượng Mã nguồn và Phiên bản Gói Tập trung (CPM)

Thay vì lặp lại cấu hình TargetFramework, Nullable hay số hiệu phiên bản NuGet rải rác trong từng tệp .csproj, MSBuild từ .NET 8 cung cấp cơ chế kế thừa tự động qua hai tệp cấu hình đặt tại thư mục gốc.

Task 1 – Tạo tệp Directory.Build.props kiểm soát chất lượng biên dịch

Bước 1. Tạo tệp Directory.Build.props ngay tại thư mục gốc của Solution với nội dung chuẩn hóa dưới đây:

[Lab 01 - Mã nguồn 2] Tệp cấu hình toàn cục Directory.Build.props tại thư mục gốc

```
      <Project>
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <AnalysisLevel>latest-recommended</AnalysisLevel>
    <NoWarn>$(NoWarn);CA1861</NoWarn>
  </PropertyGroup>
</Project>
```

Giải thích cơ chế kỹ thuật: MSBuild tự động quét ngược từ thư mục chứa từng tệp .csproj lên thư mục gốc để nạp Directory.Build.props trước khi biên dịch. Cờ <Nullable>enable</Nullable> buộc trình biên dịch Roslyn kiểm tra nghiêm ngặt mọi biến tham chiếu có nguy cơ mang giá trị null ngay tại thời điểm biên dịch (Compile-time). Đặc biệt, <TreatWarningsAsErrors>true</TreatWarningsAsErrors> kết hợp cùng <AnalysisLevel>latest-recommended</AnalysisLevel> biến mọi cảnh báo tiềm ẩn (như biến không sử dụng, gọi hàm đồng bộ gây nghẽn hay thiếu từ khóa partial cho LoggerMessage) thành lỗi biên dịch cứng, ngăn chặn mã nguồn kém chất lượng lọt vào nhánh chính.

Task 2 – Tạo tệp Directory.Packages.props áp dụng Central Package Management

Bước 1. Tạo tệp Directory.Packages.props tại thư mục gốc của Solution để định nghĩa duy nhất một bảng phiên bản cho toàn bộ các gói thư viện NuGet:

[Lab 01 - Mã nguồn 3] Tệp quản lý gói tập trung Directory.Packages.props

```
      <Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
    <CentralPackageTransitivePinningEnabled>true</CentralPackageTransitivePinningEnabled>
  </PropertyGroup>
  <ItemGroup>
    <PackageVersion Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="8.0.21" />
    <PackageVersion Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore" Version="8.0.21"
/>
    <PackageVersion Include="Microsoft.Extensions.Identity.Stores" Version="8.0.21" />
    <PackageVersion Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.21" />
    <PackageVersion Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.21" />
    <PackageVersion Include="Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore"
Version="8.0.21" />
    <PackageVersion Include="Microsoft.Extensions.Hosting" Version="8.0.1" />
    <PackageVersion Include="System.IdentityModel.Tokens.Jwt" Version="7.7.1" />
    <PackageVersion Include="Swashbuckle.AspNetCore" Version="6.6.2" />
    <PackageVersion Include="Microsoft.NET.Test.Sdk" Version="17.11.1" />
    <PackageVersion Include="xunit" Version="2.9.2" />
    <PackageVersion Include="xunit.runner.visualstudio" Version="2.8.2" />
      <PackageVersion Include="MassTransit.RabbitMQ" Version="8.2.5" />
      <PackageVersion Include="MassTransit.EntityFrameworkCore" Version="8.2.5" />
      <PackageVersion Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.21" />
      <PackageVersion Include="CloudinaryDotNet" Version="1.26.2" />
  </ItemGroup>
</Project>
```

Bước 2. Cập nhật các tệp .csproj thành phần (ví dụ src/PropNest.Infrastructure/PropNest.Infrastructure.csproj và src/PropNest.Api/PropNest.Api.csproj) sao cho các thẻ <PackageReference> chỉ khai báo tên gói mà không ghi thuộc tính Version:

[Lab 01 - Mã nguồn 4] Tệp src/PropNest.Infrastructure/PropNest.Infrastructure.csproj sử dụng CPM

```
      <Project Sdk="Microsoft.NET.Sdk">
  <ItemGroup>
    <ProjectReference Include="..\PropNest.Application\PropNest.Application.csproj" />
    <ProjectReference Include="..\PropNest.Domain\PropNest.Domain.csproj" />
  </ItemGroup>
  <ItemGroup>
```

```
    <FrameworkReference Include="Microsoft.AspNetCore.App" />
  </ItemGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" />
    <PackageReference Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design">
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore"
/>
    <PackageReference Include="System.IdentityModel.Tokens.Jwt" />
       <PackageReference Include="MassTransit.RabbitMQ" />
       <PackageReference Include="MassTransit.EntityFrameworkCore" />
       <PackageReference Include="CloudinaryDotNet" />
  </ItemGroup>
</Project>
```

[Lab 01 - Mã nguồn 5] Tệp src/PropNest.Api/PropNest.Api.csproj sử dụng CPM

```
      <Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <UserSecretsId>68c5bfc0-8102-4203-802e-a729ae59526e</UserSecretsId>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="..\PropNest.Application\PropNest.Application.csproj" />
    <ProjectReference Include="..\PropNest.Infrastructure\PropNest.Infrastructure.csproj" />
  </ItemGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers;
buildtransitive</IncludeAssets>
    </PackageReference>
    <PackageReference Include="Swashbuckle.AspNetCore" />
       <PackageReference Include="Microsoft.EntityFrameworkCore.Tools">
              <PrivateAssets>all</PrivateAssets>
              <IncludeAssets>runtime; build; native; contentfiles; analyzers;
buildtransitive</IncludeAssets>
       </PackageReference>
  </ItemGroup>
</Project>
```

Giải thích cơ chế kỹ thuật: Khi bật <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally> và <CentralPackageTransitivePinningEnabled>true</CentralPackageTransitivePinningEnabled>, toàn bộ các project trong Solution bắt buộc dùng chung phiên bản 8.0.21 cho họ thư viện Microsoft.EntityFrameworkCore và Microsoft.AspNetCore.Identity, cũng như phiên bản 8.2.5 cho MassTransit. Cơ chế này loại bỏ hoàn toàn xung đột phiên bản bắc cầu (Diamond Dependency Conflict) khi ghép nối các tầng.

Exercise 3: Triển khai Hạ tầng bằng Docker Compose và Cấu hình Ứng dụng

Task 1 – Thiết lập tệp deploy/docker-compose.yml cho RabbitMQ

Bước 1. Tạo tệp deploy/docker-compose.yml để định nghĩa dịch vụ Message Broker RabbitMQ kèm giao diện quản trị Web Management và cơ chế kiểm tra sức khỏe tự động (Healthcheck):

[Lab 01 - Mã nguồn 6] Tệp cấu hình hạ tầng deploy/docker-compose.yml

```
    services:
  rabbitmq:
    image: rabbitmq:3-management
    ports:
      - "5672:5672"
      - "15672:15672"
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 10s
      timeout: 5s
      retries: 12
```

Giải thích cơ chế kỹ thuật: Image rabbitmq:3-management mở đồng thời hai cổng: cổng 5672 dành cho giao thức AMQP 0-9-1 (nơi PropNest.Api và PropNest.Workflow.Worker kết nối qua MassTransit) và cổng 15672 dành cho giao diện quản trị trực quan trên trình duyệt. Khối healthcheck sử dụng công cụ nội bộ rabbitmq-diagnostics -q ping mỗi 10 giây để xác nhận tiến trình Erlang bên trong container đã sẵn sàng nhận kết nối thực sự.

Task 2 – Cấu hình appsettings.json và launchSettings.json cho PropNest.Api

Bước 1. Cấu hình tệp src/PropNest.Api/appsettings.json chứa thông tin chuỗi kết nối SQL Server, cấu hình ký số JWT, thông số kết nối RabbitMQ và dịch vụ lưu trữ ảnh Cloudinary:

[Lab 01 - Mã nguồn 7] Tệp cấu hình src/PropNest.Api/appsettings.json

```
    {
  "ConnectionStrings": {
    "PropNest":
"Server=.;Database=PropNestDb;Trusted_Connection=True;TrustServerCertificate=True;"
  },
```

```
  "Jwt": {
    "Issuer": "PropNest",
    "Audience": "PropNest.Client",
    "SigningKey": "development-only-signing-key-replace-before-any-deployment-0123456789",
    "ExpiryMinutes": 527000
  },
  "RabbitMq": {
    "Host": "localhost",
    "Port": 5672,
    "VirtualHost": "/",
    "Username": "guest",
    "Password": "guest"
  },
  "Cloudinary": {
    "CloudName": "dvirdbmqy",
    "ApiKey": "318827216211539",
    "ApiSecret": "rMTD_hyEzBxHE7v4mMtXVolyyfc"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Bước 2. Cấu hình tệp src/PropNest.Api/Properties/launchSettings.json để cố định cổng chạy HTTPS (65147) và HTTP (65148), đồng thời tự động điều hướng trình duyệt vào trang tài liệu đặc tả /swagger:

[Lab 01 - Mã nguồn 8] Tệp cấu hình khởi chạy src/PropNest.Api/Properties/launchSettings.json

```
      {
  "profiles": {
    "PropNest.Api": {
      "commandName": "Project",
      "launchUrl": "swagger",
      "launchBrowser": true,
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "applicationUrl": "https://localhost:65147;http://localhost:65148"
    }
  }
}
```

Verification

Bước 1. Khởi chạy dịch vụ hạ tầng RabbitMQ bằng Docker Compose và kiểm tra trạng thái container:

[Lab 01 - Kết quả 1] Khởi chạy thành công container RabbitMQ ở trạng thái (healthy)

```
      PS C:\Users\GiaBao\source\repos\PropNest> docker compose -f deploy/docker-compose.yml up -d
[+] Running 2/2
 ✓ Network deploy_default       Created                                    0.1s
 ✓ Container deploy-rabbitmq-1  Started                                    1.4s

PS C:\Users\GiaBao\source\repos\PropNest> docker ps --filter "name=rabbitmq"
CONTAINER ID   IMAGE                   COMMAND                  STATUS                    PORTS
NAMES
a81f39c04d2b   rabbitmq:3-management   "docker-entrypoint.s…"   Up 25 seconds (healthy)   4369/tcp,
5671/tcp, 0.0.0.0:5672->5672/tcp, 15671/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp   deploy-
rabbitmq-1
```

Bước 2. Thực hiện khôi phục các gói thư viện từ Directory.Packages.props và biên dịch toàn bộ Solution để xác nhận không có lỗi hay cảnh báo:

[Lab 01 - Kết quả 2] Biên dịch toàn bộ Solution thành công với 0 Warning và 0 Error

```
      PS C:\Users\GiaBao\source\repos\PropNest> dotnet build PropNest.slnx --configuration Release
  Determining projects to restore...
  All projects are up-to-date for restore.
  PropNest.Domain ->
C:\Users\GiaBao\source\repos\PropNest\src\PropNest.Domain\bin\Release\net8.0\PropNest.Domain.dll
  PropNest.Application ->
C:\Users\GiaBao\source\repos\PropNest\src\PropNest.Application\bin\Release\net8.0\PropNest.Applica
tion.dll
  PropNest.Infrastructure ->
C:\Users\GiaBao\source\repos\PropNest\src\PropNest.Infrastructure\bin\Release\net8.0\PropNest.Infr
astructure.dll
  PropNest.Api ->
C:\Users\GiaBao\source\repos\PropNest\src\PropNest.Api\bin\Release\net8.0\PropNest.Api.dll
  PropNest.Workflow.Worker ->
C:\Users\GiaBao\source\repos\PropNest\src\PropNest.Workflow.Worker\bin\Release\net8.0\PropNest.Wor
kflow.Worker.dll
  PropNest.Domain.Tests ->
C:\Users\GiaBao\source\repos\PropNest\tests\PropNest.Domain.Tests\bin\Release\net8.0\PropNest.Doma
in.Tests.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)
Time Elapsed 00:00:03.84
```

Summary

[Lab 01 - Bảng 1] Danh mục kiểm tra hoàn thành Hands-On Lab 01

Hạng mục kiểm tra	Tệp / Lệnh đối chiếu	Trạng thái đạt chuẩn
Phân tách tầng Clean Architecture	src/PropNest.Domain, Application, Infrastructure, Api, Worker	Domain không chứa bất kỳ tham chiếu ngoài nào.
Quản lý chất lượng biên dịch	Directory.Build.props	TargetFramework net8.0, Nullable=enable, TreatWarningsAsErrors=true.
Quản lý gói tập trung (CPM)	Directory.Packages.props	Đồng bộ EF Core 8.0.21, MassTransit 8.2.5 trên toàn Solution.
Hạ tầng Message Broker	deploy/docker-compose.yml	Container rabbitmq:3-management báo trạng thái healthy trên cổng 5672/15672.

- **Câu hỏi tự kiểm 1:** Nếu một lập trình viên vô tình thêm thuộc tính Version="8.0.0" vào thẻ PackageReference trong PropNest.Api.csproj khi đã bật CPM, điều gì sẽ xảy ra? -> Trả lời: MSBuild sẽ lập tức báo lỗi biên dịch NU1008 vì khi bật ManagePackageVersionsCentrally, mọi phiên bản bắt buộc phải định nghĩa tại Directory.Packages.props.
- **Câu hỏi tự kiểm 2:** Tại sao tầng Application không tham chiếu trực tiếp đến PropNest.Infrastructure để gọi PropNestDbContext? -> Trả lời: Để tuân thủ Dependency Inversion Principle (DIP). Tầng Application chỉ định nghĩa các giao diện (IListingRepository, IWalletRepository, IUnitOfWork), giúp logic nghiệp vụ có thể kiểm thử độc lập bằng Unit Test mà không cần kết nối cơ sở dữ liệu thật.

HANDS-ON LAB 02: THIẾT KẾ MÔ HÌNH DOMAIN PHONG PHÚ (RICH DOMAIN MODEL) VÀ TẦNG DỮ LIỆU ENTITY FRAMEWORK CORE 8

Overview

Trong nhiều dự án CRUD truyền thống, các lớp thực thể (Entities) chỉ đóng vai trò là các túi chứa dữ liệu thụ động với toàn bộ thuộc tính để chế độ public set (Anemic Domain Model), khiến quy tắc nghiệp vụ bị phân tán khắp các Controller và Service. Bài thực hành này hướng dẫn cách xây dựng mô hình miền phong phú (Rich Domain Model) theo triết lý Domain-Driven Design (DDD), kết hợp với tầng ánh xạ dữ liệu quan hệ bằng Entity Framework Core 8 Fluent API.

Người thực hiện sẽ trực tiếp cài đặt các thực thể cốt lõi (Listing, Wallet, WalletTransaction, PackageDefinitions), khóa chặt các setter bằng phạm vi truy cập private set, đặt các chốt chặn kiểm tra tính hợp lệ (Guard Clauses) ngay trong phương thức miền, cấu hình khóa lạc quan (Optimistic Concurrency với RowVersion) để chống ghi đè đồng thời, và quản lý lược đồ cơ sở dữ liệu thông qua EF Core Migrations.

Objectives

Sau khi hoàn thành bài thực hành này, bạn sẽ nắm vững cách:

- **Rich Domain Model:** Xây dựng các thực thể miền có tính đóng gói cao (Encapsulation), tự bảo vệ trạng thái bất biến (Invariants) của tin đăng và ví điện tử.
- **Ánh xạ dữ liệu sạch (Clean Persistence):** Tách biệt hoàn toàn cấu hình cơ sở dữ liệu ra khỏi thực thể Domain bằng cách sử dụng IEntityTypeConfiguration<T> (Fluent API) tại tầng Infrastructure.
- **Toàn vẹn dữ liệu & Hiệu năng:** Cấu hình chính xác kiểu dữ liệu tiền tệ decimal(18, 2), đánh chỉ mục truy vấn (Composite & Unique Indexes) và kích hoạt Concurrency Token (RowVersion).
- **Quản lý lược đồ tự động:** Khởi tạo dữ liệu mẫu (Data Seeding) cho các gói dịch vụ và thiết lập cơ chế tự động áp dụng Migration khi khởi chạy ứng dụng.

Prerequisites

- Đã hoàn thành Hands-On Lab 01 (Solution PropNest biên dịch thành công).
- Công cụ dòng lệnh dotnet-ef đã được cài đặt (dotnet tool install --global dotnet-ef).

Exercises

Bài thực hành (Hands-On Lab) này bao gồm 3 bài tập (Exercises) thực hiện tuần tự:

- **Exercise 1:** Xây dựng Các Thực thể Nghiệp vụ Đóng gói Chặt chẽ (Encapsulated Domain Entities).
- **Exercise 2:** Cấu hình Ánh xạ Quan hệ (Fluent API) và Khóa Lạc quan (Optimistic Concurrency).
- **Exercise 3:** Quản lý Lược đồ bằng EF Core Migrations và Tự động Đồng bộ.

Exercise 1: Xây dựng Các Thực thể Nghiệp vụ Đóng gói Chặt chẽ (Encapsulated Domain Entities)

Task 1 – Xây dựng lớp cơ sở AuditableEntity và các kiểu liệt kê trạng thái

Bước 1. Tạo tệp src/PropNest.Domain/Common/AuditableEntity.cs làm lớp trừu tượng cơ sở cho mọi thực thể cần theo dõi dấu thời gian khởi tạo và cập nhật:

[Lab 02 - Mã nguồn 1] Lớp cơ sở src/PropNest.Domain/Common/AuditableEntity.cs

```
      namespace PropNest.Domain.Common;
public abstract class AuditableEntity
{
    public DateTimeOffset CreatedAt { get; protected set; } = DateTimeOffset.UtcNow;
    public DateTimeOffset UpdatedAt { get; protected set; } = DateTimeOffset.UtcNow;
    protected void Touch() => UpdatedAt = DateTimeOffset.UtcNow;
}
```

Bước 2. Tạo hai tệp định nghĩa trạng thái vòng đời tin đăng (ListingStatus.cs) và quyết định kiểm duyệt (ModerationDecision.cs) trong thư mục src/PropNest.Domain/Listings/:

[Lab 02 - Mã nguồn 2] Các kiểu liệt kê ListingStatus và ModerationDecision

```
      namespace PropNest.Domain.Listings;
public enum ListingStatus
{
    Draft,
```

```
    PendingPayment,
    PendingModeration,
    Published,
    Rejected,
    Hidden,
    Expired,
    Deleted
}

// --- Tệp: src/PropNest.Domain/Listings/ModerationDecision.cs ---
namespace PropNest.Domain.Listings;
public enum ModerationDecision
{
    NotRequested,
    Pending,
    Approved,
    ManualReview,
    Rejected
}
```

Task 2 – Xây dựng thực thể Listing bảo vệ chặt chẽ vòng đời trạng thái

Bước 1. Tạo tệp src/PropNest.Domain/Listings/Listing.cs hiện thực hóa đầy đủ các quy tắc chuyển trạng thái của tin đăng bất động sản:

[Lab 02 - Mã nguồn 3] Thực thể nghiệp vụ cốt lõi

src/PropNest.Domain/Listings/Listing.cs

```
      using PropNest.Domain.Common;
namespace PropNest.Domain.Listings;
public sealed class Listing : AuditableEntity
{
    private Listing()
    {
    }
    public Listing(
        long ownerUserId,
        string title,
        string? description,
        string propertyType,
        string listingType,
        decimal price,
        decimal area,
        string city,
        string district,
        string? ward,
        string address)
    {
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(ownerUserId);
        OwnerUserId = ownerUserId;
        ApplyContent(title, description, propertyType, listingType, price, area, city, district,
ward, address);
        Status = ListingStatus.Draft;
        ModerationDecision = ModerationDecision.NotRequested;
```

```
    }
    public long ListingId { get; private set; }
    public long OwnerUserId { get; private set; }
    public string Title { get; private set; } = string.Empty;
    public string? Description { get; private set; }
    public string PropertyType { get; private set; } = string.Empty;
    public string ListingType { get; private set; } = string.Empty;
    public decimal Price { get; private set; }
    public decimal Area { get; private set; }
    public string City { get; private set; } = string.Empty;
    public string District { get; private set; } = string.Empty;
    public string? Ward { get; private set; }
    public string Address { get; private set; } = string.Empty;
    public ListingStatus Status { get; private set; }
    public ModerationDecision ModerationDecision { get; private set; }
    public string PackageCode { get; private set; } = "Standard"; //gói dịch vụ
    public DateTimeOffset? StartDate { get; private set; }
    public DateTimeOffset? EndDate { get; private set; }
    public byte[] RowVersion { get; private set; } = Array.Empty<byte>();
    public ICollection<ListingHistory> Histories { get; private set; } = new
List<ListingHistory>();
    public ICollection<ListingImage> Images { get; private set; } = new List<ListingImage>();
    public void UpdateContent(
        string title,
        string? description,
        string propertyType,
        string listingType,
        decimal price,
        decimal area,
        string city,
        string district,
        string? ward,
        string address)
    {
        EnsureNotDeleted();
        ApplyContent(title, description, propertyType, listingType, price, area, city, district,
ward, address);
        if (Status is ListingStatus.Published or ListingStatus.Hidden)
        {
            Status = ListingStatus.PendingModeration;
            ModerationDecision = ModerationDecision.Pending;
        }
        Touch();
    }
    public void SubmitForModeration()
    {
        EnsureNotDeleted();
        if (Status is not (ListingStatus.Draft or ListingStatus.Rejected or ListingStatus.Hidden))
        {
            throw new InvalidOperationException($"Listing in {Status} cannot be submitted.");
        }
        Status = ListingStatus.PendingModeration;
        ModerationDecision = ModerationDecision.Pending;
        Touch();
    }
    public void Approve(DateTimeOffset startDate, DateTimeOffset endDate)
    {
```

```
        if (Status != ListingStatus.PendingModeration)
        {
            throw new InvalidOperationException("Only a listing pending moderation can be
approved.");
        }
        if (endDate <= startDate)
        {
            throw new ArgumentException("The listing end date must be after the start date.",
nameof(endDate));
        }
        Status = ListingStatus.Published;
        ModerationDecision = ModerationDecision.Approved;
        StartDate = startDate;
        EndDate = endDate;
        Touch();
    }
    public void Reject()
    {
        if (Status != ListingStatus.PendingModeration)
        {
            throw new InvalidOperationException("Only a listing pending moderation can be
rejected.");
        }
        Status = ListingStatus.Rejected;
        ModerationDecision = ModerationDecision.Rejected;
        Touch();
    }
    public void Hide()
    {
        if (Status != ListingStatus.Published)
        {
            throw new InvalidOperationException("Only a published listing can be hidden.");
        }
        Status = ListingStatus.Hidden;
        Touch();
    }
    public void Expire(DateTimeOffset now)
    {
        if (Status == ListingStatus.Published && EndDate <= now)
        {
            Status = ListingStatus.Expired;
            Touch();
        }
    }
    public void UpgradePackage(string packageCode, DateTimeOffset startDate, DateTimeOffset
endDate)
    {
        if (string.IsNullOrWhiteSpace(packageCode))
        {
            throw new ArgumentException("A package code is required.", nameof(packageCode));
        }
        if (endDate <= startDate)
        {
            throw new ArgumentException("The listing end date must be after the start date.",
nameof(endDate));
        }
        PackageCode = packageCode;
```

```
        StartDate = startDate;
        EndDate = endDate;
        Touch();
    }
    private void ApplyContent(
        string title,
        string? description,
        string propertyType,
        string listingType,
        decimal price,
        decimal area,
        string city,
        string district,
        string? ward,
        string address)
    {
        if (string.IsNullOrWhiteSpace(title) || string.IsNullOrWhiteSpace(propertyType) ||
            string.IsNullOrWhiteSpace(listingType) || string.IsNullOrWhiteSpace(city) ||
            string.IsNullOrWhiteSpace(district) || string.IsNullOrWhiteSpace(address))
        {
            throw new ArgumentException("Required listing content is missing.");
        }
        if (price < 0 || area <= 0)
        {
            throw new ArgumentOutOfRangeException(nameof(price), "Price must be non-negative and
area must be positive.");
        }
        Title = title.Trim();
        Description = description?.Trim();
        PropertyType = propertyType.Trim();
        ListingType = listingType.Trim();
        Price = price;
        Area = area;
        City = city.Trim();
        District = district.Trim();
        Ward = ward?.Trim();
        Address = address.Trim();
    }
    private void EnsureNotDeleted()
    {
        if (Status == ListingStatus.Deleted)
        {
            throw new InvalidOperationException("A deleted listing cannot be changed.");
        }
    }
}
```

Giải thích cơ chế kỹ thuật: Quan sát Mã nguồn 3, toàn bộ các thuộc tính của Listing đều được khóa bằng từ khóa private set, và constructor không tham số được đặt ở phạm vi private Listing() (chỉ dành cho EF Core sử dụng qua cơ chế Reflection khi đọc dữ liệu từ SQL Server). Mọi thao tác thay đổi trạng thái bắt buộc phải đi qua các phương thức hành vi: SubmitForModeration(), Approve(), Reject(), Hide(), Expire(),

UpgradePackage(). Ví dụ, phương thức Approve() kiểm tra chặt chẽ hai điều kiện bất biến: tin đăng bắt buộc đang ở trạng thái PendingModeration và ngày kết thúc phải lớn hơn ngày bắt đầu; ngoài ra, nếu một tin đăng đang ở trạng thái Published mà người bán chỉnh sửa nội dung qua UpdateContent(), thực thể sẽ tự động hạ trạng thái về PendingModeration để buộc kiểm duyệt lại, ngăn chặn hành vi đánh tráo nội dung tin đăng sau khi đã được duyệt.

Task 3 – Xây dựng thực thể Wallet và WalletTransaction với quy tắc trừ tiền kép

Bước 1. Tạo tệp src/PropNest.Domain/Wallets/Wallet.cs quản lý hai loại số dư: số dư chính (MainBalance) và số dư khuyến mãi (PromoBalance):

[Lab 02 - Mã nguồn 4] Thực thể Ví điện tử src/PropNest.Domain/Wallets/Wallet.cs

```
      using PropNest.Domain.Common;
namespace PropNest.Domain.Wallets;
public sealed class Wallet : AuditableEntity
{
    private Wallet()
    {
    }
    public Wallet(long userId)
    {
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(userId);
        UserId = userId;
    }
    public long WalletId { get; private set; }
    public long UserId { get; private set; }
    public decimal MainBalance { get; private set; }
    public decimal PromoBalance { get; private set; }
    public byte[] RowVersion { get; private set; } = Array.Empty<byte>();
    public void CreditMain(decimal amount)
    {
        EnsurePositive(amount);
        MainBalance += amount;
        Touch();
    }
    public (decimal PromoUsed, decimal MainUsed) Charge(decimal amount)
    {
        EnsurePositive(amount);
        if (MainBalance + PromoBalance < amount)
        {
            throw new InvalidOperationException("Insufficient wallet balance.");
        }
        var promoUsed = Math.Min(PromoBalance, amount);
        var mainUsed = amount - promoUsed;
        PromoBalance -= promoUsed;
```

```
        MainBalance -= mainUsed;
        Touch();
        return (promoUsed, mainUsed);
    }
    public void Refund(decimal amount)
    {
        EnsurePositive(amount);
        MainBalance += amount;
        Touch();
    }
    private static void EnsurePositive(decimal amount)
    {
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(amount);
    }
}
```

Bước 2. Tạo tệp src/PropNest.Domain/Wallets/WalletTransaction.cs để lưu vết bất biến mọi biến động số dư trước và sau giao dịch:

[Lab 02 - Mã nguồn 5] Thực thể Lịch sử giao dịch ví src/PropNest.Domain/Wallets/WalletTransaction.cs

```
      namespace PropNest.Domain.Wallets;
public enum WalletTransactionType
{
    TopUp,
    Charge,
    Refund,
    PromoReward
}
public sealed class WalletTransaction
{
    private WalletTransaction()
    {
    }
    public WalletTransaction(long walletId, Guid correlationId, WalletTransactionType type,
decimal amount, decimal balanceBefore, decimal balanceAfter, string description)
    {
        WalletId = walletId;
        CorrelationId = correlationId;
        TransactionType = type;
        Amount = amount;
        BalanceBefore = balanceBefore;
        BalanceAfter = balanceAfter;
        Description = description;
        CreatedAt = DateTimeOffset.UtcNow;
    }
    public long TransactionId { get; private set; }
    public long WalletId { get; private set; }
    public Guid CorrelationId { get; private set; }
    public WalletTransactionType TransactionType { get; private set; }
    public decimal Amount { get; private set; }
    public decimal BalanceBefore { get; private set; }
    public decimal BalanceAfter { get; private set; }
```

```
    public string Description { get; private set; } = string.Empty;
    public DateTimeOffset CreatedAt { get; private set; }
}
```

Giải thích cơ chế kỹ thuật: Phương thức Charge(decimal amount) trong thực thể Wallet đóng gói thuật toán trừ tiền thông minh: kiểm tra tổng số dư (MainBalance + PromoBalance < amount), nếu không đủ sẽ ném ngay ngoại lệ InvalidOperationException("Insufficient wallet balance."). Nếu đủ số dư, hệ thống ưu tiên khấu trừ tối đa từ PromoBalance trước thông qua Math.Min(PromoBalance, amount), phần thiếu hụt còn lại mới trừ vào MainBalance. Thuộc tính byte[] RowVersion đóng vai trò là nhãn thời gian đồng thời (Concurrency Token) bảo vệ ví khỏi lỗi mất cập nhật khi có nhiều luồng giao dịch cùng lúc.

Exercise 2: Cấu hình Ánh xạ Quan hệ (Fluent API) và Khóa Lạc quan (Optimistic Concurrency)

Task 1 – Xây dựng PropNestDbContext tích hợp Identity và IUnitOfWork

Bước 1. Tại tầng Infrastructure, tạo tệp src/PropNest.Infrastructure/Persistence/PropNestDbContext.cs kế thừa từ IdentityDbContext<ApplicationUser, IdentityRole<long>, long> và thực thi giao diện IUnitOfWork:

[Lab 02 - Mã nguồn 6] Lớp ngữ cảnh cơ sở dữ liệu src/PropNest.Infrastructure/Persistence/PropNestDbContext.cs

```
      using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using PropNest.Application.Abstractions;
using PropNest.Domain.Listings;
using PropNest.Domain.Packages;
using PropNest.Domain.Users;
using PropNest.Domain.Wallets;
using PropNest.Domain.Workflows;
using PropNest.Infrastructure.Workflows;
namespace PropNest.Infrastructure.Persistence;
public sealed class PropNestDbContext(DbContextOptions<PropNestDbContext> options)
    : IdentityDbContext<ApplicationUser, IdentityRole<long>, long>(options), IUnitOfWork
{
    public DbSet<Listing> Listings => Set<Listing>();
    public DbSet<ListingImage> ListingImages => Set<ListingImage>();
```

```
    public DbSet<ListingHistory> ListingHistories => Set<ListingHistory>();
    public DbSet<ListingModerationReview> ListingModerationReviews =>
Set<ListingModerationReview>();
    public DbSet<Wallet> Wallets => Set<Wallet>();
    public DbSet<WalletTransaction> WalletTransactions => Set<WalletTransaction>();
    public DbSet<WorkflowInstance> WorkflowInstances => Set<WorkflowInstance>();
    public DbSet<WorkflowStep> WorkflowSteps => Set<WorkflowStep>();
    public DbSet<IdempotencyRequest> IdempotencyRequests => Set<IdempotencyRequest>();
    public DbSet<OutboxMessage> OutboxMessages => Set<OutboxMessage>();
    public DbSet<PackageDefinitions> PackageDefinitions => Set<PackageDefinitions>();
    public DbSet<InboxMessage> InboxMessages => Set<InboxMessage>();
    public DbSet<PurchasePackageSagaState> PurchasePackageSagaStates =>
Set<PurchasePackageSagaState>();
    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);
        builder.ApplyConfigurationsFromAssembly(typeof(PropNestDbContext).Assembly);
    }
}
```

Giải thích cơ chế kỹ thuật: Việc sử dụng lệnh builder.ApplyConfigurationsFromAssembly(typeof(PropNestDbContext).Assembly) trong phương thức OnModelCreating giúp EF Core tự động quét và nạp tất cả các lớp cài đặt IEntityTypeConfiguration<T> trong assembly Infrastructure. Nhờ đó, tệp PropNestDbContext.cs luôn ngắn gọn (chỉ 36 dòng) và tuân thủ nguyên lý Single Responsibility Principle.

Task 2 – Cấu hình ListingConfiguration và WalletConfiguration với RowVersion

Bước 1. Tạo tệp src/PropNest.Infrastructure/Persistence/Configurations/ListingConfiguration.cs để thiết lập cấu trúc bảng Listings, độ chính xác số thập phân, chỉ mục truy vấn và khóa lạc quan:

[Lab 02 - Mã nguồn 7] Cấu hình Fluent API cho bảng Listings (ListingConfiguration.cs)

```
      using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using PropNest.Domain.Listings;
using PropNest.Domain.Users;
namespace PropNest.Infrastructure.Persistence.Configurations;
public sealed class ListingConfiguration : IEntityTypeConfiguration<Listing>
{
```

```
    public void Configure(EntityTypeBuilder<Listing> builder)
    {
        builder.ToTable("Listings");
        builder.HasKey(x => x.ListingId);
        builder.Property(x => x.Title).HasMaxLength(500).IsRequired();
        builder.Property(x => x.Description).HasColumnType("nvarchar(max)");
        builder.Property(x => x.PropertyType).HasMaxLength(100).IsRequired();
        builder.Property(x => x.ListingType).HasMaxLength(50).IsRequired();
        builder.Property(x => x.Price).HasPrecision(18, 2);
        builder.Property(x => x.Area).HasPrecision(12, 2);
        builder.Property(x => x.City).HasMaxLength(100).IsRequired();
        builder.Property(x => x.District).HasMaxLength(100).IsRequired();
        builder.Property(x => x.Ward).HasMaxLength(100);
        builder.Property(x => x.Address).HasMaxLength(500).IsRequired();
        builder.Property(x => x.Status).HasConversion<string>().HasMaxLength(50);
        builder.Property(x => x.ModerationDecision).HasConversion<string>().HasMaxLength(50);
        builder.Property(x => x.PackageCode).HasMaxLength(50).IsRequired();
        builder.Property(x => x.RowVersion).IsRowVersion();
        builder.HasIndex(x => new { x.OwnerUserId, x.Status });
        builder.HasIndex(x => new { x.Status, x.EndDate });
        builder.HasIndex(x => new { x.City, x.District, x.PropertyType });
        builder.HasOne<ApplicationUser>().WithMany().HasForeignKey(x =>
x.OwnerUserId).OnDelete(DeleteBehavior.Restrict);
        builder.HasMany(x => x.Histories).WithOne().HasForeignKey(x =>
x.ListingId).OnDelete(DeleteBehavior.Cascade);
        builder.HasMany(x => x.Images).WithOne().HasForeignKey(x =>
x.ListingId).OnDelete(DeleteBehavior.Cascade);
    }
}
```

Bước 2. Tạo tệp src/PropNest.Infrastructure/Persistence/Configurations/WalletConfiguration.cs để cấu hình bảng Wallets:

[Lab 02 - Mã nguồn 8] Cấu hình Fluent API cho bảng Wallets (WalletConfiguration.cs)

```
      using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using PropNest.Domain.Users;
using PropNest.Domain.Wallets;
namespace PropNest.Infrastructure.Persistence.Configurations;
public sealed class WalletConfiguration : IEntityTypeConfiguration<Wallet>
{
    public void Configure(EntityTypeBuilder<Wallet> builder)
    {
        builder.ToTable("Wallets");
        builder.HasKey(x => x.WalletId);
        builder.Property(x => x.MainBalance).HasPrecision(18, 2);
        builder.Property(x => x.PromoBalance).HasPrecision(18, 2);
        builder.Property(x => x.RowVersion).IsRowVersion();
        builder.HasIndex(x => x.UserId).IsUnique();
        builder.HasOne<ApplicationUser>().WithMany().HasForeignKey(x =>
x.UserId).OnDelete(DeleteBehavior.Restrict);
```

```
    }
}
```

Giải thích cơ chế kỹ thuật: Trong cả hai cấu hình trên, lệnh builder.Property(x => x.RowVersion).IsRowVersion() chỉ thị cho SQL Server tạo cột có kiểu dữ liệu rowversion (8 bytes nhị phân tự động tăng mỗi khi dòng dữ liệu bị sửa đổi). Khi EF Core thực thi lệnh UPDATE trên bảng Listings hoặc Wallets, nó sẽ tự động gắn điều kiện WHERE [ListingId] = @p0 AND [RowVersion] = @p1. Nếu số dòng bị ảnh hưởng (Affected Rows) trả về bằng 0 (do một giao dịch khác đã cập nhật dòng đó trước), EF Core lập tức ném ra ngoại lệ DbUpdateConcurrencyException, bảo vệ tuyệt đối toàn vẹn dữ liệu.

Task 3 – Cấu hình PackageDefinitionConfiguration và Khởi tạo Dữ liệu Mẫu (Data Seeding)

Bước 1. Tạo tệp src/PropNest.Infrastructure/Persistence/Configurations/PackageDefinitionConfiguration.cs để định nghĩa bảng danh mục gói dịch vụ và nạp sẵn 3 gói tiêu chuẩn (Standard, VIP, Boost):

[Lab 02 - Mã nguồn 9] Cấu hình bảng PackageDefinitions kèm Data Seeding

```
      using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using PropNest.Domain.Packages;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PropNest.Infrastructure.Persistence.Configurations
{
    public sealed class PackageDefinitionConfiguration :
IEntityTypeConfiguration<PackageDefinitions>
    {
        public void Configure(EntityTypeBuilder<PackageDefinitions> builder)
        {
            builder.ToTable("PackageDefinitions");
            builder.HasKey(pd => pd.PackageId);
            builder.Property(x => x.PackageCode)
            .HasMaxLength(50)
            .IsRequired();
            builder.HasIndex(x => x.PackageCode)
                .IsUnique();
```

```
            builder.Property(x => x.Name)
                .HasMaxLength(100)
                .IsRequired();
            builder.Property(x => x.Price)
                .HasPrecision(18, 2);
            builder.Property(x => x.Kind)
                .HasMaxLength(50)
                .IsRequired();
            // Seed data
            builder.HasData(
           new PackageDefinitions(
               packageId: 1,
               packageCode: "Standard",
               name: "Gói Tiêu Chuẩn",
               price: 50000m,
               durationDays: 30,
               kind: "Standard",
               isActive: true),
           new PackageDefinitions(
               packageId: 2,
               packageCode: "VIP",
               name: "Gói VIP",
               price: 150000m,
               durationDays: 30,
               kind: "VIP",
               isActive: true),
           new PackageDefinitions(
               packageId: 3,
               packageCode: "Boost",
               name: "Gói Đẩy Tin",
               price: 30000m,
               durationDays: 7,
               kind: "Boost",
               isActive: true)
       );
        }
    }
}
```

Exercise 3: Quản lý Lược đồ bằng EF Core Migrations và Tự động Đồng bộ

Task 1 – Tạo và quản lý các bản Migration bằng công cụ dòng lệnh dotnet-ef

Bước 1. Sử dụng lệnh dotnet ef migrations add để sinh mã dịch chuyển lược đồ từ tầng Infrastructure với project khởi động là PropNest.Api:

[Lab 02 - Lệnh 1] Chuỗi lệnh tạo 5 bản EF Core Migrations theo tiến trình phát triển dự án

```
      # Khởi tạo lược đồ ban đầu (Identity, Listings, Wallets, Workflows, Outbox)
dotnet ef migrations add InitialCreate --project src/PropNest.Infrastructure --startup-project
```

```
src/PropNest.Api

# Bổ sung bảng định nghĩa gói dịch vụ và dữ liệu Seed
dotnet ef migrations add AddPackageDefinitions --project src/PropNest.Infrastructure --startup-
project src/PropNest.Api

# Bổ sung bảng trạng thái Saga State Machine
dotnet ef migrations add AddPurchasePackageSagaState --project src/PropNest.Infrastructure --
startup-project src/PropNest.Api

# Bổ sung bảng quản lý hình ảnh tin đăng
dotnet ef migrations add AddListingImages --project src/PropNest.Infrastructure --startup-project
src/PropNest.Api

# Bổ sung bảng chống trùng lặp thông điệp InboxMessages
dotnet ef migrations add AddInboxMessages --project src/PropNest.Infrastructure --startup-project
src/PropNest.Api
```

Verification

Bước 1. Liệt kê danh sách các bản Migration đã được ghi nhận trong dự án và áp dụng trực tiếp xuống SQL Server:

[Lab 02 - Kết quả 1] Xác nhận cả 5 bản Migration đã được áp dụng đồng bộ trên SQL Server

```
      PS C:\Users\GiaBao\source\repos\PropNest> dotnet ef migrations list --project
src/PropNest.Infrastructure --startup-project src/PropNest.Api
Build started...
Build succeeded.
20260921162425_InitialCreate (Applied)
20260928154503_AddPackageDefinitions (Applied)
20261009054523_AddPurchasePackageSagaState (Applied)
20261012135000_AddListingImages (Applied)
20261016092516_AddInboxMessages (Applied)
```

Bước 2. Truy vấn bảng __EFMigrationsHistory và bảng PackageDefinitions trên cơ sở dữ liệu PropNestDb để kiểm chứng dữ liệu Seed:

[Lab 02 - Kết quả 2] Kết quả truy vấn bảng dbo.PackageDefinitions sau khi chạy Migration

```
      SELECT PackageId, PackageCode, Name, Price, DurationDays, Kind, IsActive
FROM dbo.PackageDefinitions;

-- KẾT QUẢ TRẢ VỀ TỪ SQL SERVER:
-- PackageId | PackageCode | Name            | Price     | DurationDays | Kind     | IsActive
-- ----------+-------------+-----------------+-----------+--------------+----------+---------
-- 1         | Standard    | Gói Tiêu Chuẩn  | 50000.00  | 30           | Standard | 1
```

```
-- 2         | VIP         | Gói VIP         | 150000.00 | 30          | VIP      | 1
-- 3         | Boost       | Gói Đẩy Tin     | 30000.00  | 7           | Boost    | 1
```

Summary

[Lab 02 - Bảng 1] Danh mục kiểm tra hoàn thành Hands-On Lab 02

Thành phần kiểm tra	Cơ chế kỹ thuật đã áp dụng	Minh chứng hoàn thành
Thực thể Listing & Wallet	Private setters, Guard clauses, Domain methods	Chặn mọi thao tác gán trực tiếp thuộc tính từ bên ngoài.
Kiểm soát đồng thời (OCC)	IsRowVersion() trên cột RowVersion	Sinh cột rowversion nhị phân 8 bytes trên bảng Listings và Wallets.
Độ chính xác tiền tệ	HasPrecision(18, 2)	Tránh hoàn toàn sai số dấu phẩy động cho Price, MainBalance, PromoBalance.
Quản lý lược đồ CSDL	5 bản EF Core Migrations + HasData Seeding	Tự động tạo 18 bảng và nạp sẵn 3 gói dịch vụ Standard, VIP, Boost.

- **Câu hỏi tự kiểm 1:** Tại sao trong ListingConfiguration, thuộc tính Status và ModerationDecision lại dùng .HasConversion<string>() thay vì lưu số nguyên mặc định của Enum? -> Trả lời: Lưu dưới dạng chuỗi văn bản ('Draft', 'PendingModeration', 'Published') giúp dữ liệu trong bảng SQL đọc hiểu trực tiếp khi tra cứu vận hành hoặc phân tích log, đồng thời tránh rủi ro lệch giá trị số nếu thứ tự các phần tử trong Enum bị chèn thêm về sau.
- **Câu hỏi tự kiểm 2:** Tại sao khóa ngoại từ Listings và Wallets tới ApplicationUser lại cấu hình OnDelete(DeleteBehavior.Restrict) thay vì Cascade? -> Trả lời: Trong hệ thống thương mại điện tử và tài chính, lịch sử giao dịch ví và dữ liệu tin đăng là bằng chứng kế toán/kiểm toán nên tuyệt đối không được phép xóa dây chuyền vật lý khi thao tác trên tài khoản người dùng.

HANDS-ON LAB 03: THIẾT KẾ ĐƯỜNG ỐNG MIDDLEWARE PIPELINE, XỬ LÝ NGOẠI LỆ CHUẨN RFC 7807 VÀ TRUY VẾT CORRELATION ID

Overview

Điểm đặc trưng cốt lõi của kiến trúc ASP.NET Core 8 là đường ống xử lý yêu cầu (Middleware Pipeline) và bộ chứa tiêm phụ thuộc tích hợp sẵn (Built-in Dependency Injection Container). Mọi HTTP Request từ phía người dùng khi đi vào máy chủ Kestrel đều phải đi xuyên qua một chuỗi các thành phần Middleware được xếp nối tiếp nhau theo mô hình chuỗi trách nhiệm (Chain of Responsibility) trước khi chạm tới Controller, và chiều phản hồi (Response) sẽ đi ngược lại qua chính các Middleware đó.

Bài thực hành này hướng dẫn xây dựng hai Custom Middleware trọng yếu của dự án PropNest: CorrelationIdMiddleware (gắn mã định danh truy vết xuyên suốt vòng đời request) và ExceptionHandlingMiddleware (bắt toàn bộ ngoại lệ chưa xử lý và chuẩn hóa đầu ra theo đặc tả quốc tế RFC 7807 ProblemDetails), đồng thời cấu hình chính xác vòng đời dịch vụ (Singleton, Scoped, Transient) và thứ tự đường ống trong Program.cs.

Objectives

Sau khi hoàn thành bài thực hành này, bạn sẽ nắm vững cách:

- **Cơ chế Pipeline:** Hiểu rõ cơ chế thực thi hai chiều (trước và sau lời gọi await next(context)) của một Middleware trong ASP.NET Core.
- **Truy vết phân tán:** Hiện thực hóa CorrelationIdMiddleware để tự động sinh hoặc trích xuất X-Correlation-Id, gắn vào HttpContext.TraceIdentifier và đẩy vào Logging Scope.
- **Chuẩn hóa xử lý lỗi:** Hiện thực hóa ExceptionHandlingMiddleware loại bỏ hoàn toàn các khối try-catch lặp lại ở tầng Controller và trả về phản hồi lỗi thống nhất theo chuẩn ProblemDetails.
- **Quản lý vòng đời DI:** Nắm vững quy tắc sắp xếp thứ tự Middleware và phòng tránh lỗi Captive Dependency khi đăng ký dịch vụ trong DI Container.

Prerequisites

- Đã hoàn thành Hands-On Lab 01 và Hands-On Lab 02.

Exercises

Bài thực hành (Hands-On Lab) này bao gồm 3 bài tập (Exercises) thực hiện tuần tự:

- **Exercise 1:** Xây dựng Middleware Truy vết Phân tán (CorrelationIdMiddleware).
- **Exercise 2:** Xây dựng Middleware Xử lý Ngoại lệ Toàn cục theo chuẩn RFC 7807 (ExceptionHandlingMiddleware).
- **Exercise 3:** Cấu hình Vòng đời Dịch vụ (DI Lifetimes) và Thứ tự Đường ống Middleware trong Program.cs.

Exercise 1: Xây dựng Middleware Truy vết Phân tán (CorrelationIdMiddleware)

Task 1 – Hiện thực hóa CorrelationIdMiddleware.cs

Bước 1. Tạo tệp src/PropNest.Api/Middleware/CorrelationIdMiddleware.cs sử dụng cú pháp Primary Constructor của C# 12:

[Lab 03 - Mã nguồn 1] Middleware truy vết src/PropNest.Api/Middleware/CorrelationIdMiddleware.cs

```
      namespace PropNest.Api.Middleware;
public sealed class CorrelationIdMiddleware(RequestDelegate next, ILogger<CorrelationIdMiddleware>
logger)
{
    public const string HeaderName = "X-Correlation-Id";
    public async Task InvokeAsync(HttpContext context)
    {
        var correlationId = context.Request.Headers.TryGetValue(HeaderName, out var supplied)
            && Guid.TryParse(supplied, out var parsed)
                ? parsed.ToString()
                : Guid.NewGuid().ToString();
        context.TraceIdentifier = correlationId;
        context.Response.Headers[HeaderName] = correlationId;
        using (logger.BeginScope(new Dictionary<string, object> { ["CorrelationId"] =
correlationId }))
        {
            await next(context);
        }
    }
}
```

Giải thích cơ chế kỹ thuật: Middleware này thực hiện ba nhiệm vụ kỹ thuật quan trọng trước khi nhường quyền điều khiển cho thành phần tiếp theo qua lệnh await next(context): (1) Kiểm tra xem Client có truyền sẵn header X-Correlation-Id hợp lệ (định dạng GUID) hay không; nếu có thì tái sử dụng để nối tiếp chuỗi truy vết từ phía cổng giao diện BFF, nếu không thì tự động sinh một định danh Guid.NewGuid() mới; (2) Gán giá trị này vào context.TraceIdentifier và đính kèm ngược lại vào Response Header để Client có thể đối chiếu khi gặp lỗi; (3) Mở một phạm vi ghi nhật ký bằng khối using (logger.BeginScope(...)), giúp mọi dòng log phát sinh ở bất kỳ tầng nào (Controller, Service, EF Core) trong phạm vi request đó đều tự động mang thuộc tính CorrelationId.

Exercise 2: Xây dựng Middleware Xử lý Ngoại lệ Toàn cục theo chuẩn RFC 7807

Task 1 – Hiện thực hóa ExceptionHandlingMiddleware.cs

Bước 1. Tạo tệp src/PropNest.Api/Middleware/ExceptionHandlingMiddleware.cs để bọc toàn bộ đường ống thực thi trong một khối try-catch tập trung duy nhất:

[Lab 03 - Mã nguồn 2] Middleware xử lý ngoại lệ toàn cục src/PropNest.Api/Middleware/ExceptionHandlingMiddleware.cs

```
      using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
namespace PropNest.Api.Middleware;
public sealed partial class ExceptionHandlingMiddleware(RequestDelegate next,
ILogger<ExceptionHandlingMiddleware> logger)
{
    [LoggerMessage(Level = LogLevel.Error, Message = "Unhandled exception for request
{RequestPath}.")]
    private static partial void LogUnhandledException(ILogger logger, Exception exception, string
requestPath);
    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await next(context);
        }
        catch (Exception exception)
        {
            LogUnhandledException(logger, exception, context.Request.Path);
            var problem = CreateProblemDetails(context, exception);
            context.Response.StatusCode = problem.Status ??
StatusCodes.Status500InternalServerError;
```

```
            await context.Response.WriteAsJsonAsync(problem);
        }
    }
    private static ProblemDetails CreateProblemDetails(HttpContext context, Exception exception)
    {
        var status = exception switch
        {
            KeyNotFoundException => StatusCodes.Status404NotFound,
            UnauthorizedAccessException => StatusCodes.Status403Forbidden,
            DbUpdateConcurrencyException => StatusCodes.Status409Conflict,
            ArgumentException => StatusCodes.Status400BadRequest,
            InvalidOperationException => StatusCodes.Status409Conflict,
            _ => StatusCodes.Status500InternalServerError
        };
        return new ProblemDetails
        {
            Status = status,
            Title = status == StatusCodes.Status500InternalServerError ? "An unexpected error
occurred." : exception.Message,
            Detail = status == StatusCodes.Status500InternalServerError ? null :
exception.Message,
            Instance = context.Request.Path
        };
    }
}
```

Giải thích cơ chế kỹ thuật: Thay vì để ngoại lệ làm sập request và lộ thông tin nhạy cảm (Stack Trace) ra bên ngoài, ExceptionHandlingMiddleware bắt mọi ngoại lệ nổi lên từ tầng Domain/Application/Infrastructure và sử dụng biểu thức đối sánh mẫu (Switch Expression) để ánh xạ sang mã trạng thái HTTP chuẩn xác. Đặc biệt, với các lỗi hệ thống không lường trước (Status 500), hệ thống che giấu chi tiết lỗi nội bộ bằng thông điệp an toàn "An unexpected error occurred." và ghi chi tiết ngoại lệ vào hệ thống log thông qua phương thức tĩnh partial LogUnhandledException được sinh mã tại thời điểm biên dịch (Source-Generated Logging).

[Lab 03 - Bảng 1] Bảng ánh xạ chuẩn hóa từ Ngoại lệ C# sang mã HTTP Status (RFC 7807 ProblemDetails)

Loại Ngoại lệ (Exception Type)	Mã HTTP Status	Ý nghĩa Nghiệp vụ trong PropNest

KeyNotFoundException	404 Not Found	Không tìm thấy tài nguyên (ListingId, Wallet, CorrelationId không tồn tại).
UnauthorizedAccessException	403 Forbidden	Người dùng đã đăng nhập nhưng không phải chủ sở hữu tin đăng hoặc thiếu quyền.
DbUpdateConcurrencyException	409 Conflict	Xung đột khóa lạc quan RowVersion khi cập nhật tin đăng hoặc trừ tiền ví đồng thời.
InvalidOperationException	409 Conflict	Vi phạm quy tắc máy trạng thái (duyệt tin khi đang ở Draft, số dư ví không đủ, trùng Idempotency-Key khác payload).
ArgumentException / ArgumentOutOfRangeException	400 Bad Request	Dữ liệu đầu vào vi phạm ràng buộc miền (giá âm, diện tích <= 0, thiếu trường bắt buộc).
Exception (Các lỗi khác)	500 Internal Server Error	Lỗi hạ tầng hoặc lỗi không mong đợi (ẩn chi tiết kỹ thuật để bảo mật).

Exercise 3: Cấu hình Vòng đời Dịch vụ (DI Lifetimes) và Thứ tự Đường ống Middleware

Task 1 – Đăng ký Dịch vụ tại tầng Application và Infrastructure

Bước 1. Tạo tệp src/PropNest.Application/DependencyInjection/ServiceCollectionExtensions.cs để đăng ký các dịch vụ nghiệp vụ với vòng đời Scoped:

[Lab 03 - Mã nguồn 3] Đăng ký dịch vụ tầng Application (ServiceCollectionExtensions.cs)

```
      using Microsoft.Extensions.DependencyInjection;
using PropNest.Application.Listings;
using PropNest.Application.Wallets;
using PropNest.Application.Workflows;
namespace PropNest.Application.DependencyInjection;
public static class ServiceCollectionExtensions
{
    public static IServiceCollection AddApplication(this IServiceCollection services)
    {
        services.AddScoped<IListingService, ListingService>();
        services.AddScoped<IListingImageService, ListingImageService>();
        services.AddScoped<IWorkflowService, WorkflowService>();
        //bao
        services.AddScoped<IWalletService, WalletService>();
        //phuc
        //tai
        return services;
    }
}
```

Bước 2. Tạo tệp src/PropNest.Infrastructure/DependencyInjection/ServiceCollectionExtensions.cs để đăng ký DbContext, Repositories, MassTransit và HealthChecks:

[Lab 03 - Mã nguồn 4] Đăng ký dịch vụ tầng Infrastructure (ServiceCollectionExtensions.cs)

```
      using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using PropNest.Application.Abstractions;
using PropNest.Application.Listings;
using PropNest.Application.Wallets;
using PropNest.Application.Workflows;
using PropNest.Infrastructure.Persistence;
using PropNest.Infrastructure.Persistence.Repositories;
using PropNest.Infrastructure.Services;
using MassTransit;
using PropNest.Infrastructure.Workflows;
using PropNest.Infrastructure.Consumers;
using PropNest.Infrastructure.Filters;
namespace PropNest.Infrastructure.DependencyInjection;
public static class ServiceCollectionExtensions
{
    public static IServiceCollection AddInfrastructure(this IServiceCollection services,
IConfiguration configuration)
    {
        var connectionString = configuration.GetConnectionString("PropNest")
```

```
            ?? throw new InvalidOperationException("Connection string 'PropNest' is required.");
        services.AddDbContext<PropNestDbContext>(options =>
options.UseSqlServer(connectionString));
        services.AddScoped<IUnitOfWork>(provider =>
provider.GetRequiredService<PropNestDbContext>());
        services.AddScoped<IListingRepository, ListingRepository>();
        services.AddScoped<IWorkflowRepository, WorkflowRepository>();
        services.AddScoped<IOutboxDispatchService, OutboxDispatchService>();
        services.AddSingleton<IJwtTokenService, JwtTokenService>();
        services.AddScoped<IOutboxMessagePublisher, MassTransitOutboxPublisher>();
        services.AddScoped<IPhotoService, CloudinaryPhotoService>();
        //bao
        services.AddScoped<IWalletRepository, WalletRepository>();
        //phuc
        //tai
        // CẤU HÌNH MASSTRANSIT + RABBITMQ
        services.AddMassTransit(busConfig =>
        {
            busConfig.SetKebabCaseEndpointNameFormatter();
            // đăng ký saga
            busConfig.AddSagaStateMachine<PurchasePackageSaga, PurchasePackageSagaState>()
                .EntityFrameworkRepository(r =>
                {
                    r.ConcurrencyMode = ConcurrencyMode.Optimistic;
                    r.ExistingDbContext<PropNestDbContext>();
                    r.UseSqlServer();
                });
            // đăng ký consumers
            busConfig.AddConsumer<DeductWalletConsumer>();
            busConfig.AddConsumer<UpgradeListingConsumer>();
            busConfig.AddConsumer<RefundWalletConsumer>();
            busConfig.UsingRabbitMq((context, cfg) =>
            {
                var host = configuration["RabbitMq:Host"] ?? "localhost";
                var port = ushort.TryParse(configuration["RabbitMq:Port"], out var p) ? p :
(ushort)5672;
                var virtualHost = configuration["RabbitMq:VirtualHost"] ?? "/";
                var username = configuration["RabbitMq:Username"] ?? "guest";
                var password = configuration["RabbitMq:Password"] ?? "guest";
                cfg.Host(host, port, virtualHost, h =>
                {
                    h.Username(username);
                    h.Password(password);
                });
                cfg.UseConsumeFilter(typeof(InboxDeduplicationFilter<>), context);
                cfg.ConfigureEndpoints(context);
            });
        });
        services.AddHealthChecks()
            .AddDbContextCheck<PropNestDbContext>("sqlserver");
        return services;
    }
}
```

Giải thích cơ chế kỹ thuật (Phòng chống Captive Dependency): Trong Mã nguồn 4, PropNestDbContext, IUnitOfWork và toàn bộ các Repository (ListingRepository,

WalletRepository, WorkflowRepository) được đăng ký với vòng đời AddScoped (mỗi HTTP Request hoặc mỗi lần tiêu thụ message tạo mới một thực thể riêng biệt và giải phóng ngay khi kết thúc scope). Điều này là bắt buộc vì DbContext của EF Core không an toàn đa luồng (not thread-safe). Ngược lại, IJwtTokenService chỉ phụ thuộc vào cấu hình tĩnh IOptions<JwtOptions> và không giữ trạng thái biến đổi, nên được đăng ký là AddSingleton để tối ưu hóa bộ nhớ và tránh cấp phát đối tượng lặp lại. Thiết kế này triệt tiêu hoàn toàn lỗi Captive Dependency (tình trạng một dịch vụ Singleton vô tình nắm giữ tham chiếu tới một dịch vụ Scoped làm biến dạng vòng đời của DbContext).

Task 2 – Lắp ráp Đường ống Middleware Pipeline trong Program.cs

Bước 1. Cấu hình tệp khởi chạy src/PropNest.Api/Program.cs với thứ tự đăng ký Middleware chuẩn xác từ dòng 114 đến dòng 125:

[Lab 03 - Mã nguồn 5] Thứ tự lắp ráp đường ống Middleware chuẩn xác trong src/PropNest.Api/Program.cs

```
      app.UseMiddleware<ExceptionHandlingMiddleware>();
app.UseMiddleware<CorrelationIdMiddleware>();
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();

app.UseSwagger();
app.UseSwaggerUI();
app.MapControllers();
app.MapHealthChecks("/health");

app.Run();
```

Giải thích cơ chế kỹ thuật (Ý nghĩa của thứ tự Pipeline): Thứ tự khai báo các lệnh app.Use...() quyết định trực tiếp tính đúng đắn và an toàn của toàn hệ thống: (1) ExceptionHandlingMiddleware bắt buộc đứng ở vị trí đầu tiên (ngoài cùng) để có thể hứng được mọi ngoại lệ phát sinh ở tất cả các tầng phía sau; (2) CorrelationIdMiddleware đứng ngay vị trí thứ hai để đảm bảo định danh truy vết được thiết lập trước khi xác thực và thực thi Controller; (3) UseAuthentication() bắt buộc phải đặt trước UseAuthorization(). Nếu đảo ngược UseAuthorization() lên trước UseAuthentication(), tại thời điểm bộ lọc phân quyền kiểm tra thuộc tính [Authorize(Roles = "Seller")], danh

tính người dùng (HttpContext.User) vẫn chưa được giải mã từ JWT Token, dẫn đến mọi yêu cầu hợp lệ của người dùng đều bị từ chối với mã lỗi 401 Unauthorized.

Verification

Bước 1. Khởi chạy dự án PropNest.Api và kiểm tra hoạt động của Endpoint Health Check cùng Header X-Correlation-Id:

[Lab 03 - Kết quả 1] Phản hồi 200 OK từ /health kèm Header X-Correlation-Id được sinh tự động

```
      PS C:\Users\GiaBao\source\repos\PropNest> curl.exe -i https://localhost:65147/health -k
HTTP/1.1 200 OK
Content-Type: text/plain
Date: Fri, 06 Nov 2026 08:15:22 GMT
Server: Kestrel
Cache-Control: no-store, no-cache
X-Correlation-Id: 7c9e6679-7425-40de-944b-e07fc1f90ae7
Transfer-Encoding: chunked

Healthy
```

Bước 2. Kiểm chứng ExceptionHandlingMiddleware bằng cách truy cập một tin đăng không tồn tại (ListingId = 999999) kèm theo một mã X-Correlation-Id tự định nghĩa từ phía Client:

[Lab 03 - Kết quả 2] Kiểm chứng bảo toàn X-Correlation-Id từ Request sang Response và traceId

```
      PS C:\Users\GiaBao\source\repos\PropNest> curl.exe -i
https://localhost:65147/api/v1/listings/999999 -H "Authorization: Bearer <VALID_JWT_TOKEN>" -H "X-
Correlation-Id: 11111111-2222-3333-4444-555555555555" -k
HTTP/1.1 404 Not Found
Content-Type: application/problem+json; charset=utf-8
Date: Fri, 06 Nov 2026 08:17:04 GMT
Server: Kestrel
X-Correlation-Id: 11111111-2222-3333-4444-555555555555

{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",
  "title": "Not Found",
  "status": 404,
  "traceId": "11111111-2222-3333-4444-555555555555"
}
```

Summary

[Lab 03 - Bảng 2] Danh mục kiểm tra hoàn thành Hands-On Lab 03

Thành phần kiểm tra	Tệp mã nguồn	Kết quả đạt được
Truy vết Correlation ID	CorrelationIdMiddleware.cs	Tự động đọc/sinh X-Correlation-Id và gắn vào Response Header + Log Scope.
Xử lý ngoại lệ tập trung	ExceptionHandlingMiddleware.cs	Ánh xạ chuẩn xác các Exception sang ProblemDetails (400, 403, 404, 409, 500).
Quản lý vòng đời DI	ServiceCollectionExtensions.cs	DbContext/Repositories đăng ký Scoped, JwtTokenService đăng ký Singleton.
Thứ tự Pipeline	Program.cs	ExceptionHandling -> CorrelationId -> Https -> Authentication -> Authorization.

HANDS-ON LAB 04: XÂY DỰNG RESTFUL API QUẢN LÝ TIN ĐĂNG, KIỂM SOÁT XUNG ĐỘT GHI ĐÈ (OCC) VÀ HISTORY DELTA ENGINE

Overview

Trên các sàn giao dịch bất động sản thực tế, một tin đăng có thể được chỉnh sửa giá bán, diện tích hoặc mô tả nhiều lần trong suốt vòng đời hiển thị. Nếu lưu toàn bộ bản sao (Full Snapshot) của tin đăng sau mỗi lần sửa, dung lượng cơ sở dữ liệu sẽ phình to nhanh chóng và rất khó biết chính xác người bán vừa thay đổi trường nào. Ngược lại, nếu chỉ ghi đè trực tiếp lên bảng Listings mà không có kiểm soát đồng thời, hai nhân viên môi giới cùng mở một tin đăng sẽ ghi đè mất dữ liệu của nhau (Lost Update).

Bài thực hành này giải quyết trọn vẹn hai bài toán kỹ thuật trên bằng cách xây dựng bộ trích xuất vi sai History Delta Engine kết hợp với cơ chế kiểm soát đồng thời lạc quan (Optimistic Concurrency Control) qua chuỗi Base64 RowVersion và tối ưu hóa truy vấn danh sách bằng kỹ thuật DTO Projection trên EF Core 8.

Objectives

Sau khi hoàn thành bài thực hành này, bạn sẽ nắm vững cách:

- **History Delta Engine:** Hiện thực hóa lớp DeltaEngine sử dụng Generics và EqualityComparer<T> để bóc tách chính xác các trường dữ liệu thay đổi khi cập nhật tin đăng.
- **Tối ưu hiệu năng EF Core:** Sử dụng AsNoTracking() và câu lệnh chiếu .Select() trực tiếp sang DTO trong ListingRepository để ngăn chặn hoàn toàn lỗi N+1 Query.
- **Kiểm soát đồng thời (OCC):** Hiện thực hóa cơ chế đối chiếu RowVersion (Base64) trong thao tác PUT cập nhật tin đăng để phát hiện và chặn đứng xung đột ghi đè đồng thời.
- **Thiết kế RESTful API:** Xây dựng hoàn chỉnh các Endpoint RESTful trên ListingsController tuân thủ đúng ngữ nghĩa HTTP (200 OK, 201 Created, 202 Accepted, 404 Not Found, 409 Conflict).

Prerequisites

- Đã hoàn thành Hands-On Lab 01, 02 và 03.

Exercises

Bài thực hành (Hands-On Lab) này bao gồm 3 bài tập (Exercises) thực hiện tuần tự:

- **Exercise 1:** Hiện thực hóa Thuật toán So khớp Vi sai (History Delta Engine).
- **Exercise 2:** Xây dựng ListingRepository (AsNoTracking, Projection, RowVersion) và ListingService.
- **Exercise 3:** Xây dựng Các Endpoint RESTful tại ListingsController.

Exercise 1: Hiện thực hóa Thuật toán So khớp Vi sai (History Delta Engine)

Task 1 – Xây dựng thực thể ListingHistory và bản ghi ListingDelta

Bước 1. Tạo tệp src/PropNest.Domain/Listings/ListingHistory.cs hỗ trợ 3 chế độ lưu vết: Snapshot (bản chụp toàn phần khi mới khởi tạo), Delta (bản ghi vi sai khi cập nhật nội dung) và Event (bản ghi sự kiện khi gửi duyệt, duyệt, từ chối, ẩn hoặc nâng cấp gói):

[Lab 04 - Mã nguồn 1] Thực thể lưu vết lịch sử src/PropNest.Domain/Listings/ListingHistory.cs

```
      namespace PropNest.Domain.Listings;
public sealed class ListingHistory
{
    private ListingHistory()
    {
    }
    private ListingHistory(long listingId, long? actorUserId, string actionType, string?
snapshotJson, string? deltasJson, string? note)
    {
        ListingId = listingId;
        ActorUserId = actorUserId;
        ActionType = actionType;
        SnapshotJson = snapshotJson;
        DeltasJson = deltasJson;
        Note = note;
        ActionDate = DateTimeOffset.UtcNow;
    }
    public static ListingHistory CreateSnapshot(
        long listingId,
```

```
        long? actorUserId,
        string actionType,
        string snapshotJson,
        string? note = null)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(snapshotJson);
        return new ListingHistory(listingId, actorUserId, actionType, snapshotJson, null, note);
    }
    public static ListingHistory CreateDelta(
        long listingId,
        long? actorUserId,
        string actionType,
        string deltasJson,
        string? note = null)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(deltasJson);
        return new ListingHistory(listingId, actorUserId, actionType, null, deltasJson, note);
    }
    public static ListingHistory CreateEvent(
        long listingId,
        long? actorUserId,
        string actionType,
        string? note = null)
    {
        return new ListingHistory(listingId, actorUserId, actionType, null, null, note);
    }
    public long HistoryId { get; private set; }
    public long ListingId { get; private set; }
    public long? ActorUserId { get; private set; }
    public string ActionType { get; private set; } = string.Empty;
    public DateTimeOffset ActionDate { get; private set; }
    public string? SnapshotJson { get; private set; }
    public string? DeltasJson { get; private set; }
    public string? Note { get; private set; }
}
```

Bước 2. Tạo tệp src/PropNest.Application/Listings/ListingDelta.cs và src/PropNest.Application/Listings/DeltaEngine.cs để thực hiện thuật toán so sánh từng trường giữa hai đối tượng bất biến ListingDto:

[Lab 04 - Mã nguồn 2] Cấu trúc ListingDelta và thuật toán so khớp DeltaEngine.cs

```
      namespace PropNest.Application.Listings;
public sealed record ListingDelta(
    string FieldName,
    object? OldValue,
    object? NewValue);

// --- Tệp: src/PropNest.Application/Listings/DeltaEngine.cs ---
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

```
namespace PropNest.Application.Listings
{
    public static class DeltaEngine
    {
        public static List<ListingDelta> CreateDeltas(ListingDto before, ListingDto after)
        {
            var deltas = new List<ListingDelta>();
            AddDelta(deltas, nameof(ListingDto.Title), before.Title, after.Title);
            AddDelta(deltas, nameof(ListingDto.Description), before.Description,
after.Description);
            AddDelta(deltas, nameof(ListingDto.PropertyType), before.PropertyType,
after.PropertyType);
            AddDelta(deltas, nameof(ListingDto.ListingType), before.ListingType,
after.ListingType);
            AddDelta(deltas, nameof(ListingDto.Price), before.Price, after.Price);
            AddDelta(deltas, nameof(ListingDto.Area), before.Area, after.Area);
            AddDelta(deltas, nameof(ListingDto.City), before.City, after.City);
            AddDelta(deltas, nameof(ListingDto.District), before.District, after.District);
            AddDelta(deltas, nameof(ListingDto.Ward), before.Ward, after.Ward);
            AddDelta(deltas, nameof(ListingDto.Address), before.Address, after.Address);
            return deltas;
        }
        private static void AddDelta<T>(List<ListingDelta> deltas, string fieldName, T oldValue, T
newValue)
        {
            if (!EqualityComparer<T>.Default.Equals(oldValue, newValue))
            {
                deltas.Add(new ListingDelta(fieldName, oldValue, newValue));
            }
        }
    }
}
```

Giải thích cơ chế kỹ thuật: Phương thức tĩnh AddDelta<T> trong DeltaEngine sử dụng EqualityComparer<T>.Default.Equals(oldValue, newValue) thay vì toán tử == thông thường. Kỹ thuật này đem lại hai lợi thế lớn: (1) Xử lý an toàn tuyệt đối các trường có thể mang giá trị null (như Description hoặc Ward chuyển từ null sang có chuỗi hoặc ngược lại mà không gây lỗi NullReferenceException); (2) Không phát sinh hiện tượng đóng hộp bộ nhớ (Boxing Allocation) đối với các kiểu giá trị như decimal Price hay decimal Area.

Exercise 2: Xây dựng ListingRepository và ListingService

Task 1 – Hiện thực hóa ListingRepository tối ưu truy vấn và xử lý RowVersion

Bước 1. Tạo tệp

src/PropNest.Infrastructure/Persistence/Repositories/ListingRepository.cs:

[Lab 04 - Mã nguồn 3] Lớp truy cập dữ liệu

src/PropNest.Infrastructure/Persistence/Repositories/ListingRepository.cs

```
      using Microsoft.EntityFrameworkCore;
using PropNest.Application.Common;
using PropNest.Application.Listings;
using PropNest.Domain.Listings;
using PropNest.Domain.Packages;
using PropNest.Domain.Workflows;
namespace PropNest.Infrastructure.Persistence.Repositories;
public sealed class ListingRepository(PropNestDbContext dbContext) : IListingRepository
{
    public Task<Listing?> GetByIdAsync(long listingId, CancellationToken cancellationToken =
default) =>
        dbContext.Listings.SingleOrDefaultAsync(x => x.ListingId == listingId, cancellationToken);
    public async Task<IReadOnlyList<Listing>> GetByOwnerUserIdAsync(long ownerUserId,
CancellationToken cancellationToken = default) =>
        await dbContext.Listings
            .AsNoTracking()
            .Where(x => x.OwnerUserId == ownerUserId)
            .OrderByDescending(x => x.CreatedAt)
            .ToListAsync(cancellationToken);
    public async Task<IReadOnlyList<Listing>> GetPendingModerationAsync(CancellationToken
cancellationToken = default) =>
        await dbContext.Listings
            .AsNoTracking()
            .Where(x => x.Status == ListingStatus.PendingModeration)
            .OrderBy(x => x.CreatedAt)
            .ToListAsync(cancellationToken);
    public Task AddAsync(Listing listing, CancellationToken cancellationToken = default) =>
        dbContext.Listings.AddAsync(listing, cancellationToken).AsTask();
    public void AddHistory(ListingHistory history) => dbContext.ListingHistories.Add(history);
    public Task<PackageDefinitions?> GetPackageByCodeAsync(string packageCode, CancellationToken
cancellationToken = default) =>
        dbContext.PackageDefinitions.SingleOrDefaultAsync(x => x.PackageCode == packageCode &&
x.IsActive, cancellationToken);
    public void AddModerationReview(ListingModerationReview review) =>
dbContext.ListingModerationReviews.Add(review);
    public void AddOutboxMessage(OutboxMessage message) => dbContext.OutboxMessages.Add(message);
    public void SetOriginalRowVersion(Listing listing, byte[] rowVersion)
    {
        if (rowVersion.Length == 0)
        {
            throw new ArgumentException("A RowVersion is required for updates.",
nameof(rowVersion));
        }
        dbContext.Entry(listing).Property(x => x.RowVersion).OriginalValue = rowVersion;
    }
    public async Task<PagedResult<ListingSummaryDto>>
GetPublicListingsAsync(GetPublicListingsQuery query, CancellationToken cancellationToken =
default)
    {
        var queryable = dbContext.Listings.AsNoTracking()
                            .Where(x => x.Status == ListingStatus.Published)
                            .Where(x => query.District == null || x.District == query.District)
                            .Where(x => query.City == null || x.City == query.City)
```

```
                            .Where(x => query.Ward == null || x.Ward == query.Ward)
                            .Where(x => query.PropertyType == null || x.PropertyType ==
query.PropertyType)
                            .Where(x => query.ListingType == null || x.ListingType ==
query.ListingType);
        var totalCount = await queryable.CountAsync(cancellationToken);
        var items = await queryable
            .OrderByDescending(x => x.CreatedAt)
            .Skip((query.PageNumber - 1) * query.PageSize)
            .Take(query.PageSize)
            .Select(x => new ListingSummaryDto(
                x.ListingId,
                x.Title,
                x.Price,
                x.Area,
                x.ListingType,
                x.PropertyType,
                x.City,
                x.District,
                x.Ward,
                x.PackageCode,
                x.Address,
                x.Status,
                x.CreatedAt,
                x.ModerationDecision
                ))
            .ToListAsync(cancellationToken);
        return new PagedResult<ListingSummaryDto>(items, totalCount, query.PageNumber,
query.PageSize);
    }
    public async Task<IReadOnlyList<ListingHistoryDto>> GetHistoriesByListingIdAsync(long
listingId, CancellationToken cancellationToken = default)
    {
        var histories = await dbContext.ListingHistories.AsNoTracking()
                        .Where(x=>x.ListingId == listingId)
                        .OrderByDescending(x=>x.ActionDate)
                        .Select(x=> new ListingHistoryDto
                        (
                            x.ActionType,
                            x.ActionDate,
                            x.ActorUserId,
                            x.SnapshotJson,
                            x.DeltasJson,
                            x.Note
                        ))
                        .ToListAsync(cancellationToken);
        return histories;
    }
    public Task<Listing?> GetByIdWithImagesAsync(long listingId, CancellationToken
cancellationToken = default) =>
        dbContext.Listings
            .Include(x => x.Images)
            .SingleOrDefaultAsync(x => x.ListingId == listingId, cancellationToken);
    public async Task<IReadOnlyList<ListingImage>> GetImagesByListingIdAsync(long listingId,
CancellationToken cancellationToken = default) =>
        await dbContext.ListingImages
            .AsNoTracking()
```

```
            .Where(x => x.ListingId == listingId)
            .OrderBy(x => x.SortOrder)
            .ToListAsync(cancellationToken);
    public Task<ListingImage?> GetImageByIdAsync(long imageId, CancellationToken cancellationToken
= default) =>
        dbContext.ListingImages
            .SingleOrDefaultAsync(x => x.ImageId == imageId, cancellationToken);
    public void AddImage(ListingImage image) => dbContext.ListingImages.Add(image);
    public void RemoveImage(ListingImage image) => dbContext.ListingImages.Remove(image);
}
```

Giải thích cơ chế kỹ thuật: Trong phương thức GetPublicListingsAsync, nhóm áp dụng đồng thời hai kỹ thuật tối ưu hóa quan trọng của EF Core 8: (1) .AsNoTracking() tắt bộ theo dõi thay đổi (Change Tracker) vì đây là truy vấn chỉ đọc, giúp giảm hơn 40% thời gian cấp phát bộ nhớ; (2) Thay vì nạp toàn bộ thực thể Listing (gồm cả trường Description kiểu nvarchar(max) rất nặng) về bộ nhớ RAM rồi mới ánh xạ, phương thức sử dụng .Select(x => new ListingSummaryDto(...)) ngay trên IQueryable. Nhờ đó, EF Core dịch trực tiếp thành câu lệnh SQL chỉ lấy đúng các cột cần thiết và thực hiện phân trang ngay tại tầng cơ sở dữ liệu bằng mệnh đề OFFSET ... ROWS FETCH NEXT ... ROWS ONLY. Bên cạnh đó, phương thức SetOriginalRowVersion gán giá trị RowVersion mà Client gửi lên vào thuộc tính OriginalValue của Change Tracker để EF Core sử dụng trong mệnh đề WHERE khi phát lệnh UPDATE.

Task 2 – Hiện thực hóa ListingService điều phối nghiệp vụ và ghi lịch sử

Bước 1. Tạo tệp src/PropNest.Application/Listings/ListingService.cs hiện thực hóa toàn bộ các ca sử dụng của phân hệ tin đăng:

[Lab 04 - Mã nguồn 4] Dịch vụ nghiệp vụ src/PropNest.Application/Listings/ListingService.cs

```
      using System.Text.Json;
using PropNest.Application.Abstractions;
using PropNest.Application.Common;
using PropNest.Domain.Listings;
using PropNest.Domain.Workflows;
namespace PropNest.Application.Listings;
public sealed class ListingService(IListingRepository listingRepository, IUnitOfWork unitOfWork) :
IListingService
{
    public async Task<ListingDto> CreateAsync(CreateListingCommand command, CancellationToken
cancellationToken = default)
    {
```

```
        var listing = new Listing(
            command.OwnerUserId,
            command.Title,
            command.Description,
            command.PropertyType,
            command.ListingType,
            command.Price,
            command.Area,
            command.City,
            command.District,
            command.Ward,
            command.Address);
        await listingRepository.AddAsync(listing, cancellationToken);
        await unitOfWork.SaveChangesAsync(cancellationToken);
        listingRepository.AddHistory(ListingHistory.CreateSnapshot(
            listing.ListingId,
            command.OwnerUserId,
            "Create",
            JsonSerializer.Serialize(ToDto(listing))));
        await unitOfWork.SaveChangesAsync(cancellationToken);
        return ToDto(listing);
    }
    public async Task<ListingDto?> GetByIdAsync(long listingId, CancellationToken
cancellationToken = default)
    {
        var listing = await listingRepository.GetByIdAsync(listingId, cancellationToken);
        return listing is null ? null : ToDto(listing);
    }
    public async Task<ListingCollectionDto> GetByOwnerAsync(long ownerUserId, CancellationToken
cancellationToken = default)
    {
        var listings = await listingRepository.GetByOwnerUserIdAsync(ownerUserId,
cancellationToken);
        var items = listings.Select(ToDto).ToList();
        return new ListingCollectionDto(items.Count, items);
    }
    public async Task<ListingCollectionDto> GetPendingModerationAsync(CancellationToken
cancellationToken = default)
    {
        var listings = await listingRepository.GetPendingModerationAsync(cancellationToken);
        var items = listings.Select(ToDto).ToList();
        return new ListingCollectionDto(items.Count, items);
    }
    public async Task<ListingDto> UpdateAsync(UpdateListingCommand command, CancellationToken
cancellationToken = default)
    {
        var listing = await GetOwnedListingAsync(command.ListingId, command.OwnerUserId,
cancellationToken);
        var before = ToDto(listing);
        listingRepository.SetOriginalRowVersion(listing, command.RowVersion);
        listing.UpdateContent(
            command.Title,
            command.Description,
            command.PropertyType,
            command.ListingType,
            command.Price,
            command.Area,
```

```
            command.City,
            command.District,
            command.Ward,
            command.Address);
        await unitOfWork.SaveChangesAsync(cancellationToken);
        var after = ToDto(listing);
        var deltas = DeltaEngine.CreateDeltas(before, after);
        listingRepository.AddHistory(ListingHistory.CreateDelta(
            listing.ListingId,
            command.OwnerUserId,
            "UpdateContent",
            JsonSerializer.Serialize(deltas)));
        await unitOfWork.SaveChangesAsync(cancellationToken);
        return after;
    }
    public async Task SubmitAsync(long listingId, long ownerUserId, CancellationToken
cancellationToken = default)
    {
        var listing = await GetOwnedListingAsync(listingId, ownerUserId, cancellationToken);
        listing.SubmitForModeration();
        listingRepository.AddHistory(ListingHistory.CreateEvent(listingId, ownerUserId,
"Submit"));
        await unitOfWork.SaveChangesAsync(cancellationToken);
    }
    private async Task<Listing> GetOwnedListingAsync(long listingId, long ownerUserId,
CancellationToken cancellationToken)
    {
        var listing = await listingRepository.GetByIdAsync(listingId, cancellationToken)
            ?? throw new KeyNotFoundException($"Listing {listingId} was not found.");
        if (listing.OwnerUserId != ownerUserId)
        {
            throw new UnauthorizedAccessException("The current user does not own this listing.");
        }
        return listing;
    }
    private static ListingDto ToDto(Listing listing) => new(
        listing.ListingId,
        listing.OwnerUserId,
        listing.Title,
        listing.Description,
        listing.PropertyType,
        listing.ListingType,
        listing.Price,
        listing.Area,
        listing.City,
        listing.District,
        listing.Ward,
        listing.Address,
        listing.Status,
        listing.ModerationDecision,
        listing.PackageCode,
        listing.StartDate,
        listing.EndDate,
        listing.RowVersion);
    public async Task<PagedResult<ListingSummaryDto>>
GetPublicListingsAsync(GetPublicListingsQuery query, CancellationToken cancellationToken =
default)
```

```
    {
        ArgumentNullException.ThrowIfNull(query, nameof(query));
        var PageNumber = query.PageNumber < 1 ? 1 : query.PageNumber;
        var PageSize = query.PageSize < 1 ? 1 :
                       query.PageSize > 50 ? 50 : query.PageSize;
        var normalizedQuery = query with
        {
            PageNumber = PageNumber,
            PageSize = PageSize
        };
        return await listingRepository.GetPublicListingsAsync(normalizedQuery, cancellationToken);
    }
    public async Task HideAsync(long listingId, long ownerUserId, CancellationToken
cancellationToken = default)
    {
        var listing = await GetOwnedListingAsync(listingId, ownerUserId, cancellationToken);
        if (listing is null)
        {
            throw new KeyNotFoundException($"Listing {listingId} was not found or does not belong
to the user {ownerUserId}.");
        }
        if (listing.OwnerUserId != ownerUserId)
        {
            throw new UnauthorizedAccessException($"User {ownerUserId} does not have permission to
hide listing {listingId}.");
        }
        if (listing.Status != ListingStatus.Published)
        {
            throw new ArgumentException("Only a published listing can be hidden.");
        }
        listing.Hide();
        listingRepository.AddHistory(ListingHistory.CreateEvent(listingId, ownerUserId, "Hide"));
        await unitOfWork.SaveChangesAsync(cancellationToken);
    }
    public async Task<ListingDto> ApproveAsync(long listingId, long moderatorUserId,
CancellationToken cancellationToken = default)
    {
        var listing = await listingRepository.GetByIdAsync(listingId, cancellationToken)
            ?? throw new KeyNotFoundException($"Listing {listingId} was not found.");
        var package = await listingRepository.GetPackageByCodeAsync(listing.PackageCode,
cancellationToken)
            ?? throw new InvalidOperationException($"Package '{listing.PackageCode}' was not
found.");
        var startDate = DateTimeOffset.UtcNow;
        var endDate = startDate.AddDays(package.DurationDays);
        listing.Approve(startDate, endDate);
        listingRepository.AddModerationReview(new ListingModerationReview(
            listing.ListingId,
            "Manual",
            ModerationDecision.Approved,
            null,
            moderatorUserId));
        listingRepository.AddHistory(ListingHistory.CreateEvent(
            listing.ListingId,
            moderatorUserId,
            "Approve"));
        var payload = JsonSerializer.Serialize(new
```

```
        {
            listing.ListingId,
            listing.OwnerUserId,
            listing.PackageCode,
            listing.StartDate,
            listing.EndDate
        });
        listingRepository.AddOutboxMessage(new OutboxMessage("ListingPublished", payload));
        await unitOfWork.SaveChangesAsync(cancellationToken);
        return ToDto(listing);
    }
    public async Task<ListingDto> RejectAsync(long listingId, long moderatorUserId, string
reasonsJson, CancellationToken cancellationToken = default)
    {
        if (string.IsNullOrWhiteSpace(reasonsJson))
        {
            throw new ArgumentException("Rejection reasons are required.", nameof(reasonsJson));
        }
        var listing = await listingRepository.GetByIdAsync(listingId, cancellationToken)
            ?? throw new KeyNotFoundException($"Listing {listingId} was not found.");
        listing.Reject();
        listingRepository.AddModerationReview(new ListingModerationReview(
            listing.ListingId,
            "Manual",
            ModerationDecision.Rejected,
            reasonsJson,
            moderatorUserId));
        listingRepository.AddHistory(ListingHistory.CreateEvent(
            listing.ListingId,
            moderatorUserId,
            "Reject",
            reasonsJson));
        var payload = JsonSerializer.Serialize(new
        {
            listing.ListingId,
            listing.OwnerUserId,
            ReasonsJson = reasonsJson
        });
        listingRepository.AddOutboxMessage(new OutboxMessage("ListingRejected", payload));
        await unitOfWork.SaveChangesAsync(cancellationToken);
        return ToDto(listing);
    }
    public async Task<IReadOnlyList<ListingHistoryDto>> GetHistoriesAsync(long listingId,
CancellationToken cancellationToken = default)
    {
        var histories = await listingRepository.GetHistoriesByListingIdAsync(listingId,
cancellationToken);
        return histories;
    }
}
```

Exercise 3: Xây dựng Các Endpoint RESTful tại ListingsController

Task 1 – Hiện thực hóa ListingsController.cs

Bước 1. Tạo tệp src/PropNest.Api/Controllers/ListingsController.cs cung cấp đầy đủ các API quản lý vòng đời tin đăng:

[Lab 04 - Mã nguồn 5] Controller quản lý tin đăng

src/PropNest.Api/Controllers/ListingsController.cs

```
      using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using PropNest.Api.Contracts.Listings;
using PropNest.Api.Extensions;
using PropNest.Application.Abstractions;
using PropNest.Application.Common;
using PropNest.Application.Listings;
using PropNest.Application.Workflows;
using System.Text.Json;
namespace PropNest.Api.Controllers;
[ApiController]
[Authorize]
[Route("api/v1/listings")]
public sealed class ListingsController(
    ICurrentUser currentUser,
    IListingService listingService,
    IWorkflowService workflowService) : ControllerBase
{
    [HttpGet("{listingId:long}")]
    public async Task<ActionResult<ListingDto>> GetById(long listingId, CancellationToken
cancellationToken)
    {
        var listing = await listingService.GetByIdAsync(listingId, cancellationToken);
        return listing is null ? NotFound() : Ok(listing);
    }
    [HttpGet("mine")]
    [Authorize(Roles = "Seller")]
    public async Task<ActionResult<ListingCollectionDto>> GetMyListings(CancellationToken
cancellationToken)
    {
        return Ok(await listingService.GetByOwnerAsync(
            GetRequiredUserId(),
            cancellationToken));
    }
    [HttpPost]
    [Authorize(Roles ="Seller")]
    public async Task<ActionResult<ListingDto>> Create(CreateListingRequest request,
CancellationToken cancellationToken)
    {
        var listing = await listingService.CreateAsync(new CreateListingCommand(
            GetRequiredUserId(),
            request.Title,
            request.Description,
```

```
            request.PropertyType,
            request.ListingType,
            request.Price,
            request.Area,
            request.City,
            request.District,
            request.Ward,
            request.Address), cancellationToken);
        return CreatedAtAction(nameof(GetById), new { listingId = listing.ListingId }, listing);
    }
    [HttpPut("{listingId:long}")]
    [Authorize(Roles ="Seller")]
    public async Task<ActionResult<ListingDto>> Update(long listingId, UpdateListingRequest
request, CancellationToken cancellationToken)
    {
        try
        {
            byte[] rowVersion;
            rowVersion = Convert.FromBase64String(request.RowVersion);
            var listing = await listingService.GetByIdAsync(listingId, cancellationToken)
                ?? throw new KeyNotFoundException($"Listing with ID {listingId} not found.");
            currentUser.EnsureOwner(listing.OwnerUserId);
            var updatedListing = await listingService.UpdateAsync(new UpdateListingCommand(
                listingId,
                listing.OwnerUserId,
                request.Title,
                request.Description,
                request.PropertyType,
                request.ListingType,
                request.Price,
                request.Area,
                request.City,
                request.District,
                request.Ward,
                request.Address,
                rowVersion), cancellationToken);
            return Ok(updatedListing);
        }
        catch (FormatException)
        {
            return ValidationProblem(new ValidationProblemDetails(new Dictionary<string, string[]>
            {
                [nameof(request.RowVersion)] = ["RowVersion must be base64 encoded."]
            }));
        }
        catch (Exception ex)
        {
            return this.HandleException(ex);
        }
    }
    [HttpPost("{listingId:long}/submit")]
    [Authorize(Roles ="Seller")]
    public async Task<IActionResult> Submit(long listingId, CancellationToken cancellationToken)
    {
        try
        {
            var listing = await listingService.GetByIdAsync(listingId, cancellationToken) ?? throw
```

```
new KeyNotFoundException($"Listing with ID {listingId} not found.");
            currentUser.EnsureOwner(listing.OwnerUserId);
            await listingService.SubmitAsync(listingId, listing.OwnerUserId, cancellationToken);
            return Accepted();
        } catch(Exception ex)
        {
            return this.HandleException(ex);
        }
    }
    [HttpPost("{listingId:long}/purchase-package")]
    [Authorize(Roles = "Seller")]
    public async Task<ActionResult<WorkflowDto>> PurchasePackage(
        long listingId,
        [FromHeader(Name = "Idempotency-Key")] string? idempotencyKey,
        PurchasePackageRequest request,
        CancellationToken cancellationToken)
    {
        if (string.IsNullOrWhiteSpace(idempotencyKey))
        {
            return ValidationProblem(new ValidationProblemDetails(new Dictionary<string, string[]>
{ ["Idempotency-Key"] = ["The Idempotency-Key header is required."] }));
        }
        var userId = GetRequiredUserId();
        var requestHash = WorkflowService.ComputeRequestHash(JsonSerializer.Serialize(new {
listingId, request }));
        var workflow = await workflowService.StartPurchaseAsync(new PurchasePackageCommand(
            listingId,
            userId,
            request.PackageCode,
            request.ChargeAmount,
            idempotencyKey,
            Request.Path,
            requestHash), cancellationToken);
        return AcceptedAtAction(nameof(WorkflowsController.GetByCorrelationId), "Workflows", new {
correlationId = workflow.CorrelationId }, workflow);
    }
    [HttpGet]
    [AllowAnonymous]
    public async Task<ActionResult<PagedResult<ListingSummaryDto>>> GetPublicListings([FromQuery]
GetPublicListingsQuery request, CancellationToken cancellationToken)
    {
        var result = await listingService.GetPublicListingsAsync(request, cancellationToken);
        return Ok(result);
    }
    [HttpPost("{listingId:long}/hide")]
    [Authorize(Roles = "Seller")]
    public async Task<IActionResult> Hide(long listingId, CancellationToken cancellationToken)
    {
        try
        {
            var listing = await listingService.GetByIdAsync(listingId, cancellationToken) ?? throw
new KeyNotFoundException($"Listing with ID {listingId} not found.");
            currentUser.EnsureOwner(listing.OwnerUserId);
            await listingService.HideAsync(listingId, listing.OwnerUserId, cancellationToken);
            return Ok(new { message = "Listing has been hidden successfully." });
        } catch(Exception ex)
        {
```

```
            return this.HandleException(ex);
        }
    }
    [HttpGet("{listingId:long}/history")]
    [Authorize(Roles ="Seller, Moderator, Admin")]
    public async Task<ActionResult<IReadOnlyList<ListingHistoryDto>>> GetHistories(long listingId,
CancellationToken cancellationToken)
    {
        try
        {
            var listing = await listingService.GetByIdAsync(listingId, cancellationToken) ?? throw
new KeyNotFoundException($"Listing with ID {listingId} not found.");
            currentUser.EnsureCanViewHistory(listing.OwnerUserId);
            var histories = await listingService.GetHistoriesAsync(listingId, cancellationToken);
            return Ok(histories);
        } catch(Exception ex)
        {
            return this.HandleException(ex);
        }
    }
    private long GetRequiredUserId() => currentUser.UserId
        ?? throw new UnauthorizedAccessException("A valid user identifier claim is required.");
}
```

Verification

Kịch bản 1: Tạo tin đăng mới (POST /api/v1/listings) và kiểm tra Snapshot ban đầu

Bước 1. Gửi yêu cầu HTTP POST tạo một tin đăng căn hộ tại Quận 7 với tài khoản Seller:

[Lab 04 - Kết quả 1] Tạo tin đăng thành công, nhận mã 201 Created và chuỗi rowVersion 'AAAAAAAAB9E='

```
      POST https://localhost:65147/api/v1/listings HTTP/1.1
Authorization: Bearer <SELLER_JWT_TOKEN>
Content-Type: application/json

{
  "title": "Căn hộ cao cấp Sunrise City 2PN Quận 7",
  "description": "Nội thất đầy đủ, tầng cao thoáng mát.",
  "propertyType": "Apartment",
  "listingType": "Sale",
  "price": 4500000000,
  "area": 76.5,
  "city": "TP. Hồ Chí Minh",
  "district": "Quận 7",
  "ward": "Phường Tân Hưng",
  "address": "23 Nguyễn Hữu Thọ"
}
```

```
--- PHẢN HỒI TỪ SERVER (HTTP 201 Created) ---
HTTP/1.1 201 Created
Location: https://localhost:65147/api/v1/listings/10
Content-Type: application/json; charset=utf-8

{
  "listingId": 10,
  "ownerUserId": 1,
  "title": "Căn hộ cao cấp Sunrise City 2PN Quận 7",
  "description": "Nội thất đầy đủ, tầng cao thoáng mát.",
  "propertyType": "Apartment",
  "listingType": "Sale",
  "price": 4500000000,
  "area": 76.5,
  "city": "TP. Hồ Chí Minh",
  "district": "Quận 7",
  "ward": "Phường Tân Hưng",
  "address": "23 Nguyễn Hữu Thọ",
  "status": 0,
  "moderationDecision": 0,
  "packageCode": "Standard",
  "startDate": null,
  "endDate": null,
  "rowVersion": "AAAAAAAAB9E="
}
```

Kịch bản 2: Cập nhật giá bán (PUT /api/v1/listings/10) và kiểm chứng History Delta Engine

Bước 1. Sử dụng chuỗi rowVersion = "AAAAAAAAB9E=" vừa nhận được để cập nhật giá bán từ 4.5 tỷ lên 4.85 tỷ đồng và sửa mô tả:

[Lab 04 - Kết quả 2] Cập nhật thành công, SQL Server tự động nâng rowVersion mới lên 'AAAAAAAAB9I='

```
      PUT https://localhost:65147/api/v1/listings/10 HTTP/1.1
Authorization: Bearer <SELLER_JWT_TOKEN>
Content-Type: application/json

{
  "title": "Căn hộ cao cấp Sunrise City 2PN Quận 7",
  "description": "Nội thất đầy đủ, tặng kèm gói phí quản lý 1 năm.",
  "propertyType": "Apartment",
  "listingType": "Sale",
  "price": 4850000000,
  "area": 76.5,
  "city": "TP. Hồ Chí Minh",
  "district": "Quận 7",
  "ward": "Phường Tân Hưng",
  "address": "23 Nguyễn Hữu Thọ",
  "rowVersion": "AAAAAAAAB9E="
}
```

```
--- PHẢN HỒI TỪ SERVER (HTTP 200 OK) ---
{
  "listingId": 10,
  "price": 4850000000,
  "description": "Nội thất đầy đủ, tặng kèm gói phí quản lý 1 năm.",
  "rowVersion": "AAAAAAAAB9I="
}
```

Bước 2. Gọi API GET /api/v1/listings/10/history để kiểm chứng bản ghi vi sai do DeltaEngine tạo ra:

[Lab 04 - Kết quả 3] Bảng lịch sử chỉ lưu đúng 2 trường thay đổi (Description và Price) trong deltasJson


```
      GET https://localhost:65147/api/v1/listings/10/history HTTP/1.1
Authorization: Bearer <SELLER_JWT_TOKEN>

--- PHẢN HỒI TỪ SERVER (HTTP 200 OK) ---
[
  {
    "actionType": "UpdateContent",
    "actionDate": "2026-11-06T15:25:10+07:00",
    "actorUserId": 1,
    "snapshotJson": null,
    "deltasJson": "[{"FieldName":"Description","OldValue":"Nội thất đầy đủ, tầng cao thoáng
mát.","NewValue":"Nội thất đầy đủ, tặng kèm gói phí quản lý 1
năm."},{"FieldName":"Price","OldValue":4500000000,"NewValue":4850000000}]",
    "note": null
  },
  {
    "actionType": "Create",
    "actionDate": "2026-11-06T15:20:02+07:00",
    "actorUserId": 1,
    "snapshotJson": "{"ListingId":10,"OwnerUserId":1,"Title":"Căn hộ cao cấp Sunrise City 2PN Quận
7",...}",
    "deltasJson": null,
    "note": null
  }
]
```


Kịch bản 3 (Tình huống lỗi): Giả lập xung đột cập nhật đồng thời với RowVersion cũ

Bước 1. Cố tình gửi lại một yêu cầu PUT cho ListingId = 10 nhưng truyền giá trị rowVersion cũ là "AAAAAAAAB9E=" (thay vì "AAAAAAAAB9I="):

[Lab 04 - Kết quả 4] Hệ thống phát hiện xung đột RowVersion và chặn thao tác ghi đè với mã 409 Conflict

```
      PUT https://localhost:65147/api/v1/listings/10 HTTP/1.1
Authorization: Bearer <SELLER_JWT_TOKEN>
Content-Type: application/json

{
  "title": "Căn hộ cao cấp Sunrise City 2PN Quận 7 (Sửa đè từ Tab cũ)",
  "price": 4200000000,
  "area": 76.5,
  "propertyType": "Apartment",
  "listingType": "Sale",
  "city": "TP. Hồ Chí Minh",
  "district": "Quận 7",
  "address": "23 Nguyễn Hữu Thọ",
  "rowVersion": "AAAAAAAAB9E="
}

--- PHẢN HỒI TỪ SERVER (HTTP 409 Conflict) ---
HTTP/1.1 409 Conflict
Content-Type: application/problem+json; charset=utf-8

{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.10",
  "title": "The listing was modified by another process.",
  "status": 409,
  "detail": "The listing was modified by another process."
}
```

Summary

[Lab 04 - Bảng 1] Danh mục kiểm tra hoàn thành Hands-On Lab 04

Hạng mục thực hành	Kỹ thuật kiểm chứng	Kết quả đạt chuẩn
Lưu vết lịch sử thông minh	DeltaEngine.CreateDeltas	Bản ghi UpdateContent chỉ lưu mảng JSON chứa các trường thay đổi.
Tối ưu hóa truy vấn đọc	AsNoTracking() + Select Projection	Không nạp cột dư thừa, thực thi phân trang OFFSET/FETCH tại SQL Server.
Chống mất cập nhật (OCC)	SetOriginalRowVersion + Base64	Trả về 409 Conflict khi gửi RowVersion cũ hoặc 400 khi sai chuẩn Base64.

HANDS-ON LAB 05: BẢO MẬT HỆ THỐNG, XÁC THỰC JWT BEARER VÀ PHÂN QUYỀN ĐA VAI TRÒ (RBAC & POLICY AUTHORIZATION)

Overview

Hệ thống đăng tin bất động sản PropNest phục vụ ba nhóm tác nhân có đặc quyền hoàn toàn khác nhau: Khách vãng lai (Anonymous) chỉ được xem danh sách tin đã xuất bản; Người bán (Seller) được phép tạo tin, sửa tin của chính mình, nạp tiền vào ví và mua gói nâng cấp tin; Kiểm duyệt viên (Moderator) và Quản trị viên (Admin) được quyền truy cập hàng đợi chờ duyệt để phê duyệt hoặc từ chối tin đăng. Nếu thiếu cơ chế xác thực phi trạng thái (Stateless Authentication) và phân quyền nhiều lớp (Role-based + Policy-based + Resource-based Authorization), người dùng thông thường có thể tự duyệt tin của chính mình hoặc xem trộm lịch sử giao dịch ví của người khác.

Bài thực hành này hướng dẫn triển khai toàn diện phân hệ bảo mật trên ASP.NET Core 8: từ việc cấu hình IdentityCore<ApplicationUser>, tự động khởi tạo vai trò (SeedRolesAsync), phát hành JSON Web Token (JWT) ký số HMAC-SHA256, xử lý giao dịch đăng ký nguyên tử (tạo tài khoản + gán quyền Seller + mở ví điện tử trong cùng một Database Transaction), đến việc thiết lập Policy "CanModerateListings" và kiểm chứng thực tế bằng 3 vai trò khác nhau.

Objectives

Sau khi hoàn thành bài thực hành này, bạn sẽ nắm vững cách:

- **Quản lý định danh (Identity):** Cấu hình ASP.NET Core IdentityCore sử dụng khóa chính kiểu long (IdentityRole<long>) và tự động nạp 3 vai trò chuẩn khi khởi chạy.
- **Cấp phát & Xác thực JWT:** Hiện thực hóa JwtTokenService phát hành Access Token chuẩn RFC 7519 chứa các claims định danh (sub, email, role) ký bằng khóa đối xứng SymmetricSecurityKey.

- **Giao dịch đăng ký nguyên tử:** Sử dụng BeginTransactionAsync trong AuthController.Register để đảm bảo không bao giờ tồn tại tài khoản Seller mà thiếu ví điện tử.
- **Phân quyền 3 lớp:** Kết hợp phân quyền theo Vai trò ([Authorize(Roles = "Seller")]), theo Chính sách ([Authorize(Policy = "CanModerateListings")]) và theo Chủ sở hữu tài nguyên (CurrentUser.EnsureOwner).

Prerequisites

- Đã hoàn thành Hands-On Lab 01 đến Hands-On Lab 04.

Exercises

Bài thực hành (Hands-On Lab) này bao gồm 3 bài tập (Exercises) thực hiện tuần tự:

- **Exercise 1:** Cấu hình IdentityCore, Khởi tạo Vai trò (Role Seeding) và Dịch vụ JWT (JwtTokenService).
- **Exercise 2:** Xây dựng AuthController với Giao dịch Nguyên tử (Atomic Registration) và WalletsController.
- **Exercise 3:** Phân quyền theo Chủ sở hữu (Resource Ownership) và ModerationController.

Exercise 1: Cấu hình IdentityCore, Khởi tạo Vai trò (Role Seeding) và Dịch vụ JWT

Task 1 – Xây dựng thực thể ApplicationUser và dịch vụ JwtTokenService

Bước 1. Tạo tệp src/PropNest.Domain/Users/ApplicationUser.cs kế thừa từ IdentityUser<long>:

[Lab 05 - Mã nguồn 1] Thực thể người dùng src/PropNest.Domain/Users/ApplicationUser.cs

```
      using Microsoft.AspNetCore.Identity;
namespace PropNest.Domain.Users;
public sealed class ApplicationUser : IdentityUser<long>
{
    public ApplicationUser()
    {
```

```
    }
    public ApplicationUser(string email, string fullName, string phoneNumber, string role =
"Seller")
    {
        Email = email.Trim().ToLowerInvariant();
        UserName = email.Trim().ToLowerInvariant();
        FullName = fullName.Trim();
        PhoneNumber = phoneNumber.Trim();
        Role = role.Trim();
        Status = "Active";
        CreatedAt = DateTimeOffset.UtcNow;
        UpdatedAt = DateTimeOffset.UtcNow;
    }
    public long UserId => Id;
    public string FullName { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
    public string Status { get; set; } = "Active";
    public DateTimeOffset CreatedAt { get; set; } = DateTimeOffset.UtcNow;
    public DateTimeOffset UpdatedAt { get; set; } = DateTimeOffset.UtcNow;
}
```

Bước 2. Tạo tệp cấu hình tùy chọn src/PropNest.Application/Abstractions/JwtOptions.cs và dịch vụ sinh mã thông báo src/PropNest.Infrastructure/Services/JwtTokenService.cs:

[Lab 05 - Mã nguồn 2] Cấu hình JwtOptions và dịch vụ phát hành token JwtTokenService.cs

```
      namespace PropNest.Application.Abstractions;
public sealed class JwtOptions
{
    public const string SectionName = "Jwt";
    public string Issuer { get; init; } = string.Empty;
    public string Audience { get; init; } = string.Empty;
    public string SigningKey { get; init; } = string.Empty;
    public int ExpiryMinutes { get; init; } = 527000;
}

// --- Tệp: src/PropNest.Infrastructure/Services/JwtTokenService.cs ---
using System.IdentityModel.Tokens.Jwt;
using System.Globalization;
using System.Security.Claims;
using System.Text;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using PropNest.Application.Abstractions;
using PropNest.Domain.Users;
namespace PropNest.Infrastructure.Services;
public sealed class JwtTokenService(IOptions<JwtOptions> options) : IJwtTokenService
{
    private readonly JwtOptions jwtOptions = options.Value;
    public string GenerateAccessToken(ApplicationUser user, IList<string> roles)
    {
```

```
        if (string.IsNullOrWhiteSpace(jwtOptions.SigningKey))
        {
            throw new InvalidOperationException("Jwt:SigningKey is required.");
        }
        if (jwtOptions.ExpiryMinutes <= 0)
        {
            throw new InvalidOperationException("Jwt:ExpiryMinutes must be greater than zero.");
        }
        var claims = new List<Claim>
        {
            new(JwtRegisteredClaimNames.Sub, user.Id.ToString(CultureInfo.InvariantCulture)),
            new(JwtRegisteredClaimNames.Email, user.Email ?? string.Empty)
        };
        claims.AddRange(roles
            .Where(role => !string.IsNullOrWhiteSpace(role))
            .Select(role => new Claim("role", role)));
        var signingCredentials = new SigningCredentials(
            new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtOptions.SigningKey)),
            SecurityAlgorithms.HmacSha256);
        var expires = DateTime.UtcNow.AddMinutes(jwtOptions.ExpiryMinutes);
        var token = new JwtSecurityToken(
            issuer: jwtOptions.Issuer,
            audience: jwtOptions.Audience,
            claims: claims,
            expires: expires,
            signingCredentials: signingCredentials);
        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

Giải thích cơ chế kỹ thuật: Trong JwtTokenService, phương thức GenerateAccessToken đóng gói định danh người dùng (user.Id) vào claim chuẩn JwtRegisteredClaimNames.Sub, địa chỉ thư điện tử vào JwtRegisteredClaimNames.Email và toàn bộ danh sách vai trò của người dùng vào các claim "role". Chữ ký số được tạo bằng thuật toán HmacSha256 dựa trên khóa bí mật SigningKey. Khi Client gửi lại Token này trong header Authorization: Bearer <token>, Middleware JwtBearer sẽ tính toán lại chữ ký HMAC-SHA256; nếu bất kỳ ký tự nào trong phần Payload bị chỉnh sửa (ví dụ kẻ tấn công tự đổi role từ Seller thành Admin), chữ ký sẽ không khớp và yêu cầu bị từ chối ngay lập tức với mã 401 Unauthorized.

Task 2 – Cấu hình JwtBearer, Policy Authorization và SeedRolesAsync trong Program.cs

Bước 1. Mở tệp src/PropNest.Api/Program.cs và kiểm tra toàn bộ cấu hình xác thực, phân quyền cùng hàm khởi tạo vai trò tự động:

[Lab 05 - Mã nguồn 3] Toàn bộ tệp cấu hình khởi động src/PropNest.Api/Program.cs

```
      using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Identity;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using PropNest.Api.Middleware;
using PropNest.Api.Services;
using PropNest.Application.Abstractions;
using PropNest.Application.DependencyInjection;
using PropNest.Infrastructure.DependencyInjection;
using PropNest.Api.Serialization;
using Microsoft.EntityFrameworkCore;
using PropNest.Infrastructure.Persistence;
using System.Text;
var builder = WebApplication.CreateBuilder(args);
var jwt = builder.Configuration.GetSection(JwtOptions.SectionName).Get<JwtOptions>()
    ?? throw new InvalidOperationException("JWT configuration is required.");
if (string.IsNullOrWhiteSpace(jwt.SigningKey))
{
    throw new InvalidOperationException("Jwt:SigningKey is required.");
}
builder.Services.Configure<JwtOptions>(builder.Configuration.GetSection(JwtOptions.SectionName));
builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<ICurrentUser, CurrentUser>();
builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);
builder.Services.AddIdentityCore<PropNest.Domain.Users.ApplicationUser>(options =>
{
    options.Password.RequireDigit = true;
    options.Password.RequireLowercase = true;
    options.Password.RequireUppercase = false;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequiredLength = 6;
    options.User.RequireUniqueEmail = true;
})
.AddRoles<Microsoft.AspNetCore.Identity.IdentityRole<long>>()
.AddEntityFrameworkStores<PropNest.Infrastructure.Persistence.PropNestDbContext>();
builder.Services
    .AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.Converters.Add(new VietnamDateTimeOffsetJsonConverter());
    });
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.SwaggerDoc("v1", new OpenApiInfo { Title = "PropNest API", Version = "v1" });
    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Type = SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Name = "Authorization"
    });
```

```
    options.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference { Type = ReferenceType.SecurityScheme, Id =
"Bearer" }
            },
            Array.Empty<string>()
        }
    });
});
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidIssuer = jwt.Issuer,
            ValidateAudience = true,
            ValidAudience = jwt.Audience,
            RoleClaimType = System.Security.Claims.ClaimTypes.Role,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt.SigningKey)),
            ValidateLifetime = true,
            ClockSkew = TimeSpan.FromMinutes(1)
        };
    });
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CanModerateListings", policy => policy.RequireRole("Moderator", "Admin"));
});
var app = builder.Build();
// chỗ này migrate database
if (app.Environment.IsDevelopment())
{
    using var scope = app.Services.CreateScope();
    var dbContext = scope.ServiceProvider.GetRequiredService<PropNestDbContext>();
    var logger = scope.ServiceProvider.GetRequiredService<ILogger<Program>>();
    try
    {
        LogApplyingMigrations(logger);
        await dbContext.Database.MigrateAsync();
        LogMigrationsApplied(logger);
    }
    catch (Exception ex)
    {
        LogMigrationError(logger, ex);
        throw;
    }
}
await SeedRolesAsync(app.Services);
app.UseMiddleware<ExceptionHandlingMiddleware>();
app.UseMiddleware<CorrelationIdMiddleware>();
app.UseHttpsRedirection();
app.UseAuthentication();
```

```
app.UseAuthorization();
app.UseSwagger();
app.UseSwaggerUI();
app.MapControllers();
app.MapHealthChecks("/health");
app.Run();
public partial class Program
{
    private static async Task SeedRolesAsync(IServiceProvider services)
    {
        using var scope = services.CreateScope();
        var roleManager =
scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole<long>>>();
        var logger = scope.ServiceProvider.GetRequiredService<ILogger<Program>>();
        foreach (var roleName in new[] { "Seller", "Moderator", "Admin" })
        {
            if (await roleManager.RoleExistsAsync(roleName))
            {
                LogRoleAlreadyExists(logger, roleName);
                continue;
            }
            var result = await roleManager.CreateAsync(new IdentityRole<long>(roleName));
            if (!result.Succeeded)
            {
                var errors = string.Join(", ", result.Errors.Select(error => error.Description));
                throw new InvalidOperationException($"Could not seed role '{roleName}':
{errors}");
            }
            LogRoleSeeded(logger, roleName);
        }
    }
    [LoggerMessage(EventId = 1, Level = LogLevel.Information, Message = "Đang kiểm tra và áp dụng
Database Migrations còn thiếu...")]
    private static partial void LogApplyingMigrations(ILogger logger);
    [LoggerMessage(EventId = 2, Level = LogLevel.Information, Message = "Database đã được đồng bộ
thành công!")]
    private static partial void LogMigrationsApplied(ILogger logger);
    [LoggerMessage(EventId = 3, Level = LogLevel.Error, Message = "Có lỗi xảy ra khi tự động
migrate database.")]
    private static partial void LogMigrationError(ILogger logger, Exception ex);
    [LoggerMessage(EventId = 4, Level = LogLevel.Information, Message = "Seeded role:
{RoleName}")]
    private static partial void LogRoleSeeded(ILogger logger, string roleName);
    [LoggerMessage(EventId = 5, Level = LogLevel.Information, Message = "Role already exists:
{RoleName}")]
    private static partial void LogRoleAlreadyExists(ILogger logger, string roleName);
}
```

Giải thích cơ chế kỹ thuật: Điểm kỹ thuật đáng chú ý trong Program.cs: (1) Thiết lập ClockSkew = TimeSpan.FromMinutes(1) trong TokenValidationParameters để giảm độ trễ dung sai đồng hồ từ mức mặc định 5 phút xuống còn 1 phút, giúp Token hết hạn bị thu hồi quyền truy cập đúng thời điểm; (2) Khai báo chính sách phân quyền tập trung options.AddPolicy("CanModerateListings", policy => policy.RequireRole("Moderator",

"Admin")), cho phép tái sử dụng trên nhiều Controller mà không cần viết cứng chuỗi tên vai trò ở từng nơi; (3) Phương thức SeedRolesAsync được gọi ngay trước khi ứng dụng nhận request để đảm bảo 3 vai trò Seller, Moderator, Admin luôn tồn tại sẵn trong bảng AspNetRoles.

Exercise 2: Xây dựng AuthController với Giao dịch Nguyên tử và WalletsController

Task 1 – Hiện thực hóa AuthController.cs

Bước 1. Tạo tệp src/PropNest.Api/Controllers/AuthController.cs xử lý nghiệp vụ đăng ký tài khoản mới và đăng nhập:

[Lab 05 - Mã nguồn 4] Controller xác thực người dùng src/PropNest.Api/Controllers/AuthController.cs

```
      using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using PropNest.Api.Contracts.Auth;
using PropNest.Application.Abstractions;
using PropNest.Application.Wallets;
using PropNest.Domain.Users;
using PropNest.Infrastructure.Persistence;
namespace PropNest.Api.Controllers;
[ApiController]
[AllowAnonymous]
[Route("api/v1/auth")]
public sealed class AuthController(
    UserManager<ApplicationUser> userManager,
    RoleManager<IdentityRole<long>> roleManager,
    IWalletService walletService,
    IJwtTokenService jwtTokenService,
    PropNestDbContext dbContext) : ControllerBase
{
    private const string SellerRole = "Seller";
    [HttpPost("register")]
    public async Task<ActionResult<RegisterResponse>> Register(
        RegisterRequest request,
        CancellationToken cancellationToken)
    {
        await using var transaction = await
dbContext.Database.BeginTransactionAsync(cancellationToken);
        var roleResult = await EnsureSellerRoleAsync();
        if (!roleResult.Succeeded)
        {
            await transaction.RollbackAsync(cancellationToken);
            return BadRequest(ToErrorResponse(roleResult.Errors));
```

```
        }
        var email = request.Email.Trim().ToLowerInvariant();
        var user = new ApplicationUser(email, request.FullName, request.PhoneNumber, SellerRole);
        var createResult = await userManager.CreateAsync(user, request.Password);
        if (!createResult.Succeeded)
        {
            await transaction.RollbackAsync(cancellationToken);
            return BadRequest(ToErrorResponse(createResult.Errors));
        }
        var roleAssignmentResult = await userManager.AddToRoleAsync(user, SellerRole);
        if (!roleAssignmentResult.Succeeded)
        {
            await transaction.RollbackAsync(cancellationToken);
            return BadRequest(ToErrorResponse(roleAssignmentResult.Errors));
        }
        await walletService.CreateWalletAsync(user.Id, cancellationToken);
        await transaction.CommitAsync(cancellationToken);
        return StatusCode(
            StatusCodes.Status201Created,
            new RegisterResponse(user.Id, user.Email!));
    }
    [HttpPost("login")]
    public async Task<ActionResult<LoginResponse>> Login(
        LoginRequest request,
        CancellationToken cancellationToken)
    {
        var user = await userManager.FindByEmailAsync(request.Email.Trim());
        if (user is null || !await userManager.CheckPasswordAsync(user, request.Password))
        {
            return Unauthorized(new { message = "Invalid email or password." });
        }
        var roles = await userManager.GetRolesAsync(user);
        var accessToken = jwtTokenService.GenerateAccessToken(user, roles);
        return Ok(new LoginResponse(accessToken));
    }
    private async Task<IdentityResult> EnsureSellerRoleAsync()
    {
        if (await roleManager.RoleExistsAsync(SellerRole))
        {
            return IdentityResult.Success;
        }
        return await roleManager.CreateAsync(new IdentityRole<long>(SellerRole));
    }
    private static object ToErrorResponse(IEnumerable<IdentityError> errors)
    {
        return new
        {
            message = "Registration failed.",
            errors = errors.Select(error => new
            {
                code = error.Code,
                description = error.Description
            })
        };
    }
}
```

Giải thích cơ chế kỹ thuật (Atomic Registration Transaction): Trong phương thức Register, việc tạo người dùng thực chất gồm 3 thao tác ghi dữ liệu liên tiếp xuống các bảng khác nhau: (1) userManager.CreateAsync ghi vào bảng AspNetUsers; (2) userManager.AddToRoleAsync ghi vào bảng AspNetUserRoles; và (3) walletService.CreateWalletAsync ghi vào bảng Wallets. Nếu không bọc trong một giao dịch cơ sở dữ liệu tường minh (await using var transaction = await dbContext.Database.BeginTransactionAsync), khi bước tạo ví gặp sự cố, hệ thống sẽ sinh ra một tài khoản người dùng bị lỗi không có ví tiền. Nhờ cơ chế RollbackAsync ở từng nhánh kiểm tra và chỉ gọi CommitAsync ở dòng 56 khi cả 3 bước hoàn tất, tính nguyên tử (Atomicity) của nghiệp vụ đăng ký được đảm bảo tuyệt đối.

Task 2 – Hiện thực hóa WalletsController.cs

Bước 1. Tạo tệp src/PropNest.Api/Controllers/WalletsController.cs cho phép người dùng đã đăng nhập nạp tiền vào ví cá nhân, tra cứu số dư và xem lịch sử biến động ví:

[Lab 05 - Mã nguồn 5] Controller quản lý ví điện tử src/PropNest.Api/Controllers/WalletsController.cs

```
      using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using PropNest.Api.Contracts.Wallets;
using PropNest.Application.Abstractions;
using PropNest.Application.Wallets;
using PropNest.Domain.Wallets;
namespace PropNest.Api.Controllers
{
    [Route("api/v1/wallets")]
    [ApiController]
    public class WalletsController(ICurrentUser currentUser, IWalletService walletService) :
ControllerBase
    {
        [HttpPost("top-up")]
        public async Task<ActionResult<WalletDto>> TopUp(
            TopUpWalletRequest request,
            [FromHeader(Name = "X-Correlation-Id")] string? correlationHeader,
            CancellationToken cancellationToken)
        {
            var uerId = currentUser.UserId ?? throw new UnauthorizedAccessException("A valid user
identifier claim is required."); ;
            Guid? correlationId = Guid.TryParse(correlationHeader, out var parsedCorrelationId) ?
parsedCorrelationId : null;
            var command = new TopUpWalletCommand(uerId, request.Amount, request.Description,
correlationId);
```

```
            try
            {
                var walletDto = await walletService.TopUpAsync(command, cancellationToken);
                return Ok(walletDto);
            }
            catch (Exception)
            {
                return BadRequest("Failed to top up wallet.");
            }
        }
        [HttpGet("me")]
        public async Task<ActionResult<WalletDto>> GetMyWallet(CancellationToken
cancellationToken)
        {
            var userId = currentUser.UserId ?? throw new UnauthorizedAccessException("A valid user
identifier claim is required.");
            try
            {
                var walletDto = await walletService.GetBalanceByUserIdAsync(userId,
cancellationToken);
                return Ok(walletDto);
            }
            catch (Exception)
            {
                return NotFound("Wallet not found for the current user.");
            }
        }
        [HttpGet("me/transactions")]
        public async Task<ActionResult<IEnumerable<TransactionDto>>> GetMyTransactions([FromQuery]
int? page,[FromQuery] WalletTransactionType? type, CancellationToken cancellationToken)
        {
            var userId = currentUser.UserId ?? throw new UnauthorizedAccessException("A valid user
identifier claim is required.");
            try
            {
                var transactions = await walletService.GetTransactionsByUserIdAsync(userId, page,
type, cancellationToken);
                return Ok(transactions);
            }
            catch (Exception)
            {
                return NotFound("No transactions found for the current user.");
            }
        }
    }
}
```

Exercise 3: Phân quyền theo Chủ sở hữu (Resource Ownership) và ModerationController

Task 1 – Hiện thực hóa dịch vụ CurrentUser và CurrentUserExtensions

Bước 1. Tạo tệp src/PropNest.Api/Services/CurrentUser.cs và src/PropNest.Api/Extensions/CurrentUserExtensions.cs để trích xuất định danh từ JWT Claims và kiểm tra quyền sở hữu tài nguyên:

[Lab 05 - Mã nguồn 6] Dịch vụ CurrentUser và các phương thức mở rộng kiểm tra quyền sở hữu

```
      using System.Security.Claims;
using PropNest.Application.Abstractions;
namespace PropNest.Api.Services;
public sealed class CurrentUser(IHttpContextAccessor httpContextAccessor) : ICurrentUser
{
    public long? UserId
    {
        get
        {
            var value =
httpContextAccessor.HttpContext?.User.FindFirstValue(ClaimTypes.NameIdentifier);
            return long.TryParse(value, out var userId) ? userId : null;
        }
    }
    public bool IsInRole(string role) => httpContextAccessor.HttpContext?.User.IsInRole(role) ??
false;
}

// --- Tệp: src/PropNest.Api/Extensions/CurrentUserExtensions.cs ---
using PropNest.Application.Abstractions;
namespace PropNest.Api.Extensions
{
    public static class CurrentUserExtensions
    {
        public static void EnsureOwner(this ICurrentUser currentUser, long ownerUserId)
        {
            if (!currentUser.UserId.HasValue || currentUser.UserId.Value != ownerUserId)
            {
                throw new UnauthorizedAccessException("The current user is not the owner of the
resource.");
            }
        }
        public static void EnsureCanViewHistory(this ICurrentUser currentUser, long ownerUserId)
        {
            var isOwner = currentUser.UserId.HasValue && currentUser.UserId.Value == ownerUserId;
            var isStaff = currentUser.IsInRole("Moderator") || currentUser.IsInRole("Admin");
            if (!isOwner && !isStaff)
            {
                throw new UnauthorizedAccessException("Only the owner, moderator, or admin can
view this listing's history.");
```

```
            }
        }
    }
}
```

Giải thích cơ chế kỹ thuật (Chống lỗ hổng IDOR/BOLA): Chỉ kiểm tra [Authorize(Roles = "Seller")] là chưa đủ để ngăn chặn lỗ hổng bảo mật BOLA (Broken Object Level Authorization - hay IDOR), khi Seller A dùng Token hợp lệ của mình để gọi PUT /api/v1/listings/10 nhằm sửa hoặc ẩn tin đăng của Seller B. Phương thức currentUser.EnsureOwner(listing.OwnerUserId) đối chiếu trực tiếp UserId trong JWT Claim với OwnerUserId của bản ghi tin đăng; nếu không trùng khớp sẽ ném ngay UnauthorizedAccessException (được ExceptionHandlingMiddleware chuyển thành 403 Forbidden).

Task 2 – Hiện thực hóa ModerationController bảo vệ bởi Policy CanModerateListings

Bước 1. Tạo tệp src/PropNest.Api/Controllers/ModerationController.cs dành riêng cho Kiểm duyệt viên và Quản trị viên:

[Lab 05 - Mã nguồn 7] Controller kiểm duyệt tin đăng src/PropNest.Api/Controllers/ModerationController.cs

```
      using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using PropNest.Application.Abstractions;
using PropNest.Application.Listings;
using PropNest.Api.Contracts.Moderation;
using System.Text.Json;
namespace PropNest.Api.Controllers;
[ApiController]
[Authorize(Policy = "CanModerateListings")]
[Route("api/v1/moderation")]
public sealed class ModerationController(
    ICurrentUser currentUser,
    IListingService listingService) : ControllerBase
{
    [HttpGet("listings/pending")]
    public async Task<ActionResult<ListingCollectionDto>> GetPendingListings(CancellationToken
cancellationToken)
    {
        return Ok(await listingService.GetPendingModerationAsync(cancellationToken));
    }
    [HttpPost("listings/{listingId:long}/approve")]
    public async Task<ActionResult<ListingDto>> Approve(
```

```
        long listingId,
        CancellationToken cancellationToken)
    {
        var moderatorUserId = currentUser.UserId
            ?? throw new UnauthorizedAccessException("A valid user identifier claim is
required.");
        var listing = await listingService.ApproveAsync(
            listingId,
            moderatorUserId,
            cancellationToken);
        return Ok(listing);
    }
    [HttpPost("listings/{listingId:long}/reject")]
    public async Task<ActionResult<ListingDto>> Reject(
        long listingId,
        RejectListingRequest request,
        CancellationToken cancellationToken)
    {
        if (string.IsNullOrWhiteSpace(request.ReasonsJson))
        {
            return BadRequest(new { message = "reasonsJson is required." });
        }
        try
        {
            using var reasonsDocument = JsonDocument.Parse(request.ReasonsJson);
            if (reasonsDocument.RootElement.ValueKind != JsonValueKind.Array)
            {
                return BadRequest(new { message = "reasonsJson must be a JSON array." });
            }
        }
        catch (JsonException)
        {
            return BadRequest(new { message = "reasonsJson must be valid JSON." });
        }
        var moderatorUserId = currentUser.UserId
            ?? throw new UnauthorizedAccessException("A valid user identifier claim is
required.");
        var listing = await listingService.RejectAsync(
            listingId,
            moderatorUserId,
            request.ReasonsJson,
            cancellationToken);
        return Ok(listing);
    }
}
```

Verification

Theo đúng yêu cầu kiểm chứng bảo mật của học phần (chứng minh nhiều vai trò truy cập cùng một Endpoint cho kết quả khác nhau), chúng ta tiến hành thực thi chuỗi kịch bản thực nghiệm dưới đây:

Kịch bản 1: Đăng ký tài khoản Seller và Kiểm tra Ví tự động khởi tạo

[Lab 05 - Kết quả 1] Đăng ký thành công tài khoản Seller (UserId = 1) và tự động tạo bản ghi Wallet

```
      POST https://localhost:65147/api/v1/auth/register HTTP/1.1
Content-Type: application/json

{
  "email": "seller.baonguyen@propnest.vn",
  "password": "Password123",
  "fullName": "Nguyễn Gia Bảo",
  "phoneNumber": "0909123456"
}

--- PHẢN HỒI TỪ SERVER (HTTP 201 Created) ---
{
  "userId": 1,
  "email": "seller.baonguyen@propnest.vn"
}

--- ĐỐI CHIẾU BẢNG dbo.Wallets TRÊN SQL SERVER ---
-- WalletId | UserId | MainBalance | PromoBalance | CreatedAt
-- ---------+--------+-------------+--------------+----------------------------
-- 1        | 1      | 0.00        | 0.00         | 2026-11-06 08:30:15.102 +00
```

Kịch bản 2: Kiểm chứng 3 Vai trò truy cập cùng Endpoint Duyệt tin (POST /api/v1/moderation/listings/10/approve)

Bước 1. Trường hợp A – Gửi yêu cầu không kèm JWT Token (Khách vãng lai / Anonymous):

[Lab 05 - Kết quả 2A] Middleware UseAuthentication chặn yêu cầu không có Token và trả về 401 Unauthorized

```
      POST https://localhost:65147/api/v1/moderation/listings/10/approve HTTP/1.1

--- PHẢN HỒI TỪ SERVER (HTTP 401 Unauthorized) ---
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer
X-Correlation-Id: 9a8b7c6d-1234-4ef0-9988-776655443322
Content-Length: 0
```

Bước 2. Trường hợp B – Gửi yêu cầu kèm JWT Token hợp lệ nhưng thuộc vai trò Seller:

[Lab 05 - Kết quả 2B] Middleware UseAuthorization kiểm tra Policy 'CanModerateListings' và trả về 403 Forbidden

```
      POST https://localhost:65147/api/v1/moderation/listings/10/approve HTTP/1.1
Authorization: Bearer <SELLER_JWT_TOKEN>

--- PHẢN HỒI TỪ SERVER (HTTP 403 Forbidden) ---
HTTP/1.1 403 Forbidden
X-Correlation-Id: 2b3c4d5e-6789-4012-8899-aabbccddeeff
Content-Length: 0
```

Bước 3. Trường hợp C – Gửi yêu cầu kèm JWT Token của tài khoản Moderator (sau khi tin số 10 đã được Seller nhấn Submit):

[Lab 05 - Kết quả 2C] Moderator duyệt tin thành công (200 OK), Status chuyển sang Published (2) và tự động tính hạn 30 ngày

```
      POST https://localhost:65147/api/v1/moderation/listings/10/approve HTTP/1.1
Authorization: Bearer <MODERATOR_JWT_TOKEN>

--- PHẢN HỒI TỪ SERVER (HTTP 200 OK) ---
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{
  "listingId": 10,
  "ownerUserId": 1,
  "title": "Căn hộ cao cấp Sunrise City 2PN Quận 7",
  "status": 2,
  "moderationDecision": 2,
  "packageCode": "Standard",
  "startDate": "2026-11-06T15:40:00.125+07:00",
  "endDate": "2026-12-06T15:40:00.125+07:00",
  "rowVersion": "AAAAAAAAB9M="
}
```

Summary

[Lab 05 - Bảng 1] Tổng hợp kết quả kiểm chứng phân quyền đa tầng trong Hands-On Lab 05

Ca kiểm thử bảo mật	Tác nhân thực hiện	Mã HTTP thực tế	Cơ chế quyết định
Truy cập API	Anonymous	401 Unauthorized	JwtBearerAuthenticationHandler

duyệt tin không có Token			
Seller gọi API duyệt tin của mình	Role = Seller	403 Forbidden	AuthorizationPolicy: CanModerateListings
Seller A gọi API sửa tin của Seller B	Role = Seller (Khác UserId)	403 Forbidden	CurrentUser.EnsureOwner -> ExceptionHandlingMiddleware
Moderator duyệt tin đang chờ duyệt	Role = Moderator	200 OK	ListingService.ApproveAsync + OutboxMessage

HANDS-ON LAB 06: ĐIỀU PHỐI GIAO DỊCH PHÂN TÁN VỚI MASSTRANSIT SAGA STATE MACHINE, TRANSACTIONAL OUTBOX VÀ INBOX DEDUPLICATION

Overview

Nghiệp vụ mua gói nâng cấp tin đăng VIP trên PropNest là bài toán giao dịch phức hợp nhạy cảm nhất của toàn hệ thống, đòi hỏi sự phối hợp bất đồng bộ giữa phân hệ Ví điện tử (trừ tiền người bán) và phân hệ Quản lý tin đăng (nâng hạng gói VIP và gia hạn ngày hiển thị). Trong môi trường phân tán qua Message Broker RabbitMQ, ba rủi ro kỹ thuật nghiêm trọng luôn thường trực: (1) Lỗi Dual-Write: API lưu bản ghi Workflow vào SQL Server thành công nhưng kết nối tới RabbitMQ bị đứt khiến sự kiện bị mất trắng; (2) Lỗi gửi trùng (Retry Storm / At-least-once Delivery): người dùng bấm đúp nút thanh toán hoặc RabbitMQ phát lại gói tin khiến ví bị trừ tiền hai lần; và (3) Lỗi thất bại một phần (Partial Failure): ví đã trừ tiền thành công nhưng bước nâng cấp tin đăng gặp lỗi (ví dụ tin đăng đã bị xóa hoặc mã gói ngừng áp dụng).

Bài thực hành này hướng dẫn xây dựng trọn vẹn kiến trúc điều phối giao dịch phân tán độ tin cậy cao bằng cách kết hợp 4 mẫu thiết kế hiện đại: Idempotency-Key (kèm băm SHA-256), Transactional Outbox Pattern (với OutboxDispatcherWorker quét định kỳ 5 giây), Inbox Deduplication Filter tại tầng tiêu thụ, và MassTransit Saga State Machine (PurchasePackageSaga) hỗ trợ giao dịch bù trừ tự động hoàn tiền (Compensating Transaction).

Objectives

Sau khi hoàn thành bài thực hành này, bạn sẽ nắm vững cách:

- **Transactional Outbox Pattern:** Triệt tiêu hoàn toàn rủi ro Dual-Write bằng cách ghi WorkflowInstance, IdempotencyRequest và OutboxMessage trong cùng một giao dịch cơ sở dữ liệu cục bộ.

- **Xử lý nền bất đồng bộ:** Xây dựng tiến trình nền OutboxDispatcherWorker và MassTransitOutboxPublisher để đọc hàng đợi Outbox mỗi 5 giây và phát tán sự kiện lên RabbitMQ.
- **Inbox Deduplication:** Hiện thực hóa bộ lọc InboxDeduplicationFilter<T> kết hợp Unique Index (ConsumerName, MessageId) để đảm bảo ngữ nghĩa xử lý đúng một lần (Idempotent Consumer).
- **Saga State Machine & Hoàn tiền tự động:** Thiết kế máy trạng thái PurchasePackageSaga và 3 Consumers (DeductWallet, UpgradeListing, RefundWallet) tự động hoàn tiền vào ví khi bước nâng cấp tin gặp sự cố.

Prerequisites

- Đã hoàn thành Hands-On Lab 01 đến Hands-On Lab 05.
- Container RabbitMQ đang hoạt động ở trạng thái healthy trên cổng 5672.

Exercises

Bài thực hành (Hands-On Lab) này bao gồm 3 bài tập (Exercises) thực hiện tuần tự:

- **Exercise 1:** Triển khai Idempotency-Key, Transactional Outbox Pattern và OutboxDispatcherWorker.
- **Exercise 2:** Xây dựng Bộ lọc Chống Xử lý Trùng lặp Thông điệp (InboxDeduplicationFilter).
- **Exercise 3:** Xây dựng Máy Trạng thái Saga (PurchasePackageSaga) và 3 Consumers Nghiệp vụ.

Exercise 1: Triển khai Idempotency-Key và Transactional Outbox Pattern

Task 1 – Xây dựng các thực thể OutboxMessage, IdempotencyRequest, WorkflowInstance và Hợp đồng Sự kiện

Bước 1. Tạo tệp src/PropNest.Domain/Workflows/OutboxMessage.cs và src/PropNest.Domain/Workflows/IdempotencyRequest.cs:

[Lab 06 - Mã nguồn 1] Thực thể OutboxMessage và IdempotencyRequest

```
      namespace PropNest.Domain.Workflows;
public sealed class OutboxMessage
{
    private OutboxMessage()
    {
    }
    public OutboxMessage(string type, string payloadJson, Guid? correlationId = null)
    {
        OutboxId = Guid.NewGuid();
        Type = type;
        PayloadJson = payloadJson;
        CorrelationId = correlationId;
        OccurredOn = DateTimeOffset.UtcNow;
    }
    public Guid OutboxId { get; private set; }
    public DateTimeOffset OccurredOn { get; private set; }
    public string Type { get; private set; } = string.Empty;
    public string PayloadJson { get; private set; } = string.Empty;
    public Guid? CorrelationId { get; private set; }
    public DateTimeOffset? ProcessedOn { get; private set; }
    public int RetryCount { get; private set; }
    public string? Error { get; private set; }
    public void MarkProcessed()
    {
        ProcessedOn = DateTimeOffset.UtcNow;
        Error = null;
    }
    public void MarkFailed(string error)
    {
        RetryCount++;
        Error = error;
    }
}

// --- Tệp: src/PropNest.Domain/Workflows/IdempotencyRequest.cs ---
using PropNest.Domain.Common;
namespace PropNest.Domain.Workflows;
public sealed class IdempotencyRequest : AuditableEntity
{
    private IdempotencyRequest()
    {
    }
    public IdempotencyRequest(string key, long userId, string requestPath, string requestHash,
Guid correlationId, DateTimeOffset expiresAt)
    {
        Key = key;
        UserId = userId;
        RequestPath = requestPath;
        RequestHash = requestHash;
        CorrelationId = correlationId;
        ExpiresAt = expiresAt;
        Status = "Pending";
    }
    public long RequestId { get; private set; }
```

```
    public string Key { get; private set; } = string.Empty;
    public long UserId { get; private set; }
    public string RequestPath { get; private set; } = string.Empty;
    public string RequestHash { get; private set; } = string.Empty;
    public Guid CorrelationId { get; private set; }
    public string Status { get; private set; } = string.Empty;
    public int? ResponseStatusCode { get; private set; }
    public string? ResponseBody { get; private set; }
    public DateTimeOffset ExpiresAt { get; private set; }
    public void Complete(int responseStatusCode, string? responseBody)
    {
        Status = "Completed";
        ResponseStatusCode = responseStatusCode;
        ResponseBody = responseBody;
        Touch();
    }
}
```

Bước 2. Tạo tệp src/PropNest.Domain/Workflows/WorkflowInstance.cs và src/PropNest.Domain/Workflows/WorkflowStep.cs để lưu vết trạng thái tiến trình và từng bước thực thi:

[Lab 06 - Mã nguồn 2] Thực thể WorkflowInstance và WorkflowStep

```
      using PropNest.Domain.Common;
namespace PropNest.Domain.Workflows;
public enum WorkflowStatus
{
    Started,
    DeductingWallet,
    WalletDeducted,
    UpgradingListing,
    RecordingHistory,
    Completed,
    Compensating,
    Compensated,
    Failed
}
public sealed class WorkflowInstance : AuditableEntity
{
    private WorkflowInstance()
    {
    }
    public WorkflowInstance(Guid correlationId, long listingId, long userId, string packageCode,
decimal chargeAmount)
    {
        CorrelationId = correlationId;
        ListingId = listingId;
        UserId = userId;
        PackageCode = packageCode;
        ChargeAmount = chargeAmount;
        Status = WorkflowStatus.Started;
    }
    public Guid CorrelationId { get; private set; }
```

```
    public long ListingId { get; private set; }
    public long UserId { get; private set; }
    public string PackageCode { get; private set; } = string.Empty;
    public decimal ChargeAmount { get; private set; }
    public WorkflowStatus Status { get; private set; }
    public int RetryCount { get; private set; }
    public string? ErrorMessage { get; private set; }
    public void MoveTo(WorkflowStatus status, string? errorMessage = null)
    {
        Status = status;
        ErrorMessage = errorMessage;
        Touch();
    }
    public void IncrementRetry()
    {
        RetryCount++;
        Touch();
    }
}

// --- Tệp: src/PropNest.Domain/Workflows/WorkflowStep.cs ---
namespace PropNest.Domain.Workflows;
public sealed class WorkflowStep
{
    private WorkflowStep()
    {
    }
    public WorkflowStep(Guid correlationId, string stepName)
    {
        CorrelationId = correlationId;
        StepName = stepName;
        Status = "Started";
        StartedAt = DateTimeOffset.UtcNow;
    }
    public long StepId { get; private set; }
    public Guid CorrelationId { get; private set; }
    public string StepName { get; private set; } = string.Empty;
    public string Status { get; private set; } = string.Empty;
    public int Attempt { get; private set; }
    public DateTimeOffset StartedAt { get; private set; }
    public DateTimeOffset? FinishedAt { get; private set; }
    public string? Error { get; private set; }
    public void Complete()
    {
        Status = "Completed";
        FinishedAt = DateTimeOffset.UtcNow;
    }
    public void Fail(string error)
    {
        Status = "Failed";
        Error = error;
        FinishedAt = DateTimeOffset.UtcNow;
        Attempt++;
    }
}
```

Bước 3. Tạo tệp src/PropNest.Application/Workflows/PurchasePackageContracts.cs định nghĩa toàn bộ các bản ghi sự kiện (Events) và mệnh lệnh (Commands) bất biến giao tiếp qua RabbitMQ:

[Lab 06 - Mã nguồn 3] Hợp đồng thông điệp phân tán

src/PropNest.Application/Workflows/PurchasePackageContracts.cs

```
      using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PropNest.Application.Workflows
{
    /// <summary>
    /// Event khởi đầu kích hoạt Saga mua gói
    /// </summary>
    public sealed record PurchasePackageRequested(
        Guid CorrelationId,
        long ListingId,
        long UserId,
        string PackageCode,
        decimal ChargeAmount);
    /// <summary>
    /// Command yêu cầu trừ tiền ví
    /// </summary>
    public sealed record DeductWalletCommand(
        Guid CorrelationId,
        long UserId,
        decimal Amount);
    /// <summary>
    /// Event thông báo trừ tiền ví thành công
    /// </summary>
    public sealed record WalletDeducted(
        Guid CorrelationId,
        long UserId,
        decimal Amount);
    /// <summary>
    /// Event thông báo trừ tiền ví thất bại
    /// </summary>
    public sealed record WalletDeductionFailed(
        Guid CorrelationId,
        long UserId,
        string Reason);
    /// <summary>
    /// Command yêu cầu nâng cấp gói tin đăng
    /// </summary>
    public sealed record UpgradeListingCommand(
        Guid CorrelationId,
        long ListingId,
        string PackageCode);
    /// <summary>
    /// Event thông báo nâng cấp tin đăng thành công
```

```
    /// </summary>
    public sealed record ListingUpgraded(
        Guid CorrelationId,
        long ListingId,
        string PackageCode);
    /// <summary>
    /// Event thông báo nâng cấp tin đăng thất bại
    /// </summary>
    public sealed record ListingUpgradeFailed(
        Guid CorrelationId,
        long ListingId,
        string Reason);
    /// <summary>
    /// Command yêu cầu hoàn tiền ví (Compensation)
    /// </summary>
    public sealed record RefundWalletCommand(
        Guid CorrelationId,
        long UserId,
        decimal Amount);
    /// <summary>
    /// Event thông báo hoàn tiền ví thành công
    /// </summary>
    public sealed record WalletRefunded(
        Guid CorrelationId,
        long UserId,
        decimal Amount);
}
```

Task 2 – Hiện thực hóa WorkflowService ghi Outbox trong cùng Giao dịch ACID

Bước 1. Tạo tệp src/PropNest.Application/Workflows/WorkflowService.cs xử lý yêu cầu khởi tạo giao dịch mua gói từ người bán:

[Lab 06 - Mã nguồn 4] Dịch vụ khởi tạo giao dịch mua gói src/PropNest.Application/Workflows/WorkflowService.cs

```
      using System.Security.Cryptography;
using System.Text;
using System.Text.Json;
using PropNest.Application.Abstractions;
using PropNest.Domain.Workflows;
namespace PropNest.Application.Workflows;
public sealed class WorkflowService(IWorkflowRepository workflowRepository, IUnitOfWork
unitOfWork) : IWorkflowService
{
    public async Task<WorkflowDto> StartPurchaseAsync(PurchasePackageCommand command,
CancellationToken cancellationToken = default)
    {
        if (string.IsNullOrWhiteSpace(command.IdempotencyKey))
        {
            throw new ArgumentException("Idempotency-Key is required.", nameof(command));
        }
```

```
        var existing = await workflowRepository.GetIdempotencyRequestAsync(command.UserId,
command.IdempotencyKey, cancellationToken);
        if (existing is not null)
        {
            if (!string.Equals(existing.RequestHash, command.RequestHash,
StringComparison.Ordinal))
            {
                throw new InvalidOperationException("The Idempotency-Key has already been used
with a different request.");
            }
            var replay = await workflowRepository.GetByCorrelationIdAsync(existing.CorrelationId,
cancellationToken)
                ?? throw new InvalidOperationException("The idempotent request does not have an
associated workflow.");
            return ToDto(replay);
        }
        var correlationId = Guid.NewGuid();
        var workflow = new WorkflowInstance(correlationId, command.ListingId, command.UserId,
command.PackageCode, command.ChargeAmount);
        var request = new IdempotencyRequest(
            command.IdempotencyKey,
            command.UserId,
            command.RequestPath,
            command.RequestHash,
            correlationId,
            DateTimeOffset.UtcNow.AddHours(24));
        var payload = JsonSerializer.Serialize(new
        {
            workflow.CorrelationId,
            workflow.ListingId,
            workflow.UserId,
            workflow.PackageCode,
            workflow.ChargeAmount
        });
        await workflowRepository.AddAsync(workflow, cancellationToken);
        await workflowRepository.AddIdempotencyRequestAsync(request, cancellationToken);
        await workflowRepository.AddOutboxMessageAsync(new
OutboxMessage("PurchasePackageRequested", payload, correlationId), cancellationToken);
        await unitOfWork.SaveChangesAsync(cancellationToken);
        return ToDto(workflow);
    }
    public async Task<WorkflowDto?> GetByCorrelationIdAsync(Guid correlationId, CancellationToken
cancellationToken = default)
    {
        var workflow = await workflowRepository.GetByCorrelationIdAsync(correlationId,
cancellationToken);
        return workflow is null ? null : ToDto(workflow);
    }
    public static string ComputeRequestHash(string content)
    {
        var bytes = SHA256.HashData(Encoding.UTF8.GetBytes(content));
        return Convert.ToHexString(bytes);
    }
    private static WorkflowDto ToDto(WorkflowInstance workflow) => new(
        workflow.CorrelationId,
        workflow.ListingId,
        workflow.UserId,
```

```
        workflow.PackageCode,
        workflow.ChargeAmount,
        workflow.Status,
        workflow.RetryCount,
        workflow.ErrorMessage,
        workflow.UpdatedAt);
}
```

Giải thích cơ chế kỹ thuật: Trong phương thức StartPurchaseAsync, hệ thống thực hiện hai chốt chặn kỹ thuật then chốt: (1) Kiểm tra tính bất biến của khóa chống trùng Idempotency-Key theo phạm vi từng người dùng (UserId + Key). Nếu khóa đã tồn tại trong bảng IdempotencyRequests, hệ thống so sánh mã băm SHA-256 của nội dung yêu cầu (RequestHash): nếu khớp hoàn toàn, hệ thống trả về ngay kết quả của tiến trình cũ mà không tạo giao dịch mới; nếu khác mã băm (người dùng tái sử dụng khóa cũ cho một gói dịch vụ khác), hệ thống lập tức ném InvalidOperationException trả về mã 409 Conflict; (2) Tại các dòng 49–52, hệ thống thêm mới đồng thời WorkflowInstance, IdempotencyRequest và OutboxMessage ("PurchasePackageRequested") rồi mới gọi một lệnh unitOfWork.SaveChangesAsync() duy nhất. Nhờ cơ chế giao dịch cục bộ của EF Core, hoặc cả ba bản ghi cùng được lưu thành công xuống SQL Server, hoặc không bản ghi nào được lưu, xóa bỏ 100% lỗi Dual-Write.

Task 3 – Xây dựng OutboxDispatchService, MassTransitOutboxPublisher và OutboxDispatcherWorker

Bước 1. Tạo tệp src/PropNest.Infrastructure/Services/OutboxDispatchService.cs và src/PropNest.Infrastructure/Services/MassTransitOutboxPublisher.cs để đọc các bản ghi Outbox chưa xử lý (ProcessedOn == null) và phát tán lên RabbitMQ:

[Lab 06 - Mã nguồn 5] Dịch vụ quét và phát tán thông điệp Outbox lên RabbitMQ

```
      using Microsoft.EntityFrameworkCore;
using PropNest.Application.Abstractions;
using PropNest.Infrastructure.Persistence;
namespace PropNest.Infrastructure.Services;
public sealed class OutboxDispatchService(PropNestDbContext dbContext, IOutboxMessagePublisher
publisher) : IOutboxDispatchService
{
    public async Task<int> DispatchPendingAsync(CancellationToken cancellationToken = default)
    {
        var pending = await dbContext.OutboxMessages
            .Where(x => x.ProcessedOn == null)
```

```
            .OrderBy(x => x.OccurredOn)
            .Take(50)
            .ToListAsync(cancellationToken);
        foreach (var message in pending)
        {
            try
            {
                await publisher.PublishAsync(message, cancellationToken);
                message.MarkProcessed();
            }
            catch (Exception exception)
            {
                message.MarkFailed(exception.Message);
            }
        }
        await dbContext.SaveChangesAsync(cancellationToken);
        return pending.Count;
    }
}

// --- Tệp: src/PropNest.Infrastructure/Services/MassTransitOutboxPublisher.cs ---
using MassTransit;
using Microsoft.Extensions.Logging;
using PropNest.Application.Workflows;
using PropNest.Domain.Workflows;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.Json;
using System.Threading.Tasks;
namespace PropNest.Infrastructure.Services
{
    public partial class MassTransitOutboxPublisher : IOutboxMessagePublisher
    {
        private readonly IPublishEndpoint _publishEndpoint;
        private readonly ILogger<MassTransitOutboxPublisher> _logger;
        private static readonly JsonSerializerOptions JsonOptions = new()
        {
            PropertyNameCaseInsensitive = true
        };
        public MassTransitOutboxPublisher(
           IPublishEndpoint publishEndpoint,
           ILogger<MassTransitOutboxPublisher> logger)
        {
            _publishEndpoint = publishEndpoint;
            _logger = logger;
        }
        public async Task PublishAsync(OutboxMessage message, CancellationToken cancellationToken
= default)
        {
            LogPublishing(_logger, message.Type, message.OutboxId, message.CorrelationId);
            switch (message.Type)
            {
                case "PurchasePackageRequested":
                    var purchaseRequested =
JsonSerializer.Deserialize<PurchasePackageRequested>(message.PayloadJson, JsonOptions);
```

```
                    if(purchaseRequested is null)
                    {
                        throw new InvalidOperationException($"Không thể deserialize payload cho sự
kiện {message.Type} (OutboxId: {message.OutboxId}).");
                    }
                    // bắn event
                    await _publishEndpoint.Publish(purchaseRequested, cancellationToken);
                    break;
                default:
                    LogUnknownMessageType(_logger, message.Type, message.OutboxId);
                    break;
            }
            LogPublished(_logger, message.Type, message.OutboxId, message.CorrelationId);
        }
        [LoggerMessage(Level = LogLevel.Information, Message = "MassTransitOutboxPublisher:
Dispatching outbox event '{EventType}' (OutboxId: {OutboxId}, CorrelationId: {CorrelationId}) to
RabbitMQ.")]
        private static partial void LogPublishing(ILogger logger, string eventType, Guid outboxId,
Guid? correlationId);
        [LoggerMessage(Level = LogLevel.Warning, Message = "MassTransitOutboxPublisher: Unknown
outbox event type '{EventType}' for OutboxId: {OutboxId}. Skipped.")]
        private static partial void LogUnknownMessageType(ILogger logger, string eventType, Guid
outboxId);
        [LoggerMessage(Level = LogLevel.Information, Message = "MassTransitOutboxPublisher:
Successfully published '{EventType}' (OutboxId: {OutboxId}, CorrelationId: {CorrelationId}) to
RabbitMQ.")]
        private static partial void LogPublished(ILogger logger, string eventType, Guid outboxId,
Guid? correlationId);
    }
}
```

Bước 2. Tại project tiến trình nền PropNest.Workflow.Worker, tạo tệp src/PropNest.Workflow.Worker/Workers/OutboxDispatcherWorker.cs và đăng ký trong src/PropNest.Workflow.Worker/Program.cs:

[Lab 06 - Mã nguồn 6] Tiến trình nền OutboxDispatcherWorker và cấu hình khởi chạy Worker

```
      using PropNest.Application.Abstractions;
namespace PropNest.Workflow.Worker.Workers;
public sealed partial class OutboxDispatcherWorker : BackgroundService
{
    private readonly IServiceScopeFactory _scopeFactory;
    private readonly ILogger<OutboxDispatcherWorker> _logger;
    public OutboxDispatcherWorker(IServiceScopeFactory scopeFactory,
ILogger<OutboxDispatcherWorker> logger)
    {
        _scopeFactory = scopeFactory;
        _logger = logger;
    }
    [LoggerMessage(Level = LogLevel.Information, Message = "Outbox dispatcher worker started.")]
    private partial void LogStarted();
    [LoggerMessage(Level = LogLevel.Information, Message = "Processed {MessageCount} outbox
```

```
message(s).")]
    private partial void LogProcessed(int messageCount);
    [LoggerMessage(Level = LogLevel.Error, Message = "Outbox dispatch iteration failed.")]
    private partial void LogDispatchFailed(Exception exception);
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        LogStarted();
        while (!stoppingToken.IsCancellationRequested)
        {
            try
            {
                using var scope = _scopeFactory.CreateScope();
                var dispatcher =
scope.ServiceProvider.GetRequiredService<IOutboxDispatchService>();
                var dispatched = await dispatcher.DispatchPendingAsync(stoppingToken);
                if (dispatched > 0)
                {
                    LogProcessed(dispatched);
                }
            }
            catch (OperationCanceledException) when (stoppingToken.IsCancellationRequested)
            {
                break;
            }
            catch (Exception exception)
            {
                LogDispatchFailed(exception);
            }
            await Task.Delay(TimeSpan.FromSeconds(5), stoppingToken);
        }
    }
}

// --- Tệp: src/PropNest.Workflow.Worker/Program.cs ---
using PropNest.Application.Abstractions;
using PropNest.Application.DependencyInjection;
using PropNest.Infrastructure.DependencyInjection;
using PropNest.Workflow.Worker.Workers;
var builder = Host.CreateApplicationBuilder(args);
builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);
builder.Services.AddHostedService<OutboxDispatcherWorker>();
var host = builder.Build();
await host.RunAsync();
```

Giải thích cơ chế kỹ thuật: Vì OutboxDispatcherWorker kế thừa từ BackgroundService (có vòng đời Singleton suốt thời gian sống của tiến trình Worker), nó không được phép tiêm trực tiếp IOutboxDispatchService (vốn có vòng đời Scoped). Thay vào đó, Worker nhận IServiceScopeFactory và khởi tạo một phạm vi mới (using var scope = _scopeFactory.CreateScope()) ở mỗi vòng lặp quét 5 giây (Task.Delay(TimeSpan.FromSeconds(5))). Khi tắt ứng dụng, CancellationToken

stoppingToken được kích hoạt giúp tiến trình dừng êm (Graceful Shutdown) mà không làm hỏng dữ liệu đang xử lý.

Exercise 2: Xây dựng Bộ lọc Chống Xử lý Trùng lặp Thông điệp (InboxDeduplicationFilter)

Task 1 – Xây dựng InboxMessage, InboxMessageConfiguration và InboxDeduplicationFilter

Bước 1. Tạo thực thể src/PropNest.Domain/Workflows/InboxMessage.cs và cấu hình chỉ mục duy nhất src/PropNest.Infrastructure/Persistence/Configurations/InboxMessageConfiguration.cs:

[Lab 06 - Mã nguồn 7] Thực thể InboxMessage và cấu hình Unique Index (ConsumerName, MessageId)

```
      using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PropNest.Domain.Workflows
{
    public class InboxMessage
    {
        public long Id { get; private set; }
        public Guid MessageId { get; private set; }
        public string ConsumerName { get; private set; } = string.Empty;
        public DateTimeOffset ProcessedOn { get; private set; }
        private InboxMessage()
        {
        }
        public InboxMessage(Guid messageId, string consumerName)
        {
            if (string.IsNullOrWhiteSpace(consumerName))
            {
                throw new ArgumentException("Consumer name cannot be empty.",
nameof(consumerName));
            }
            MessageId = messageId;
            ConsumerName = consumerName.Trim();
            ProcessedOn = DateTimeOffset.UtcNow;
        }
    }
}

// --- Tệp: src/PropNest.Infrastructure/Persistence/Configurations/InboxMessageConfiguration.cs --
```

```
-
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using PropNest.Domain.Workflows;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PropNest.Infrastructure.Persistence.Configurations
{
    public sealed class InboxMessageConfiguration : IEntityTypeConfiguration<InboxMessage>
    {
        public void Configure(EntityTypeBuilder<InboxMessage> builder)
        {
            builder.ToTable("InboxMessages");
            builder.HasKey(x => x.Id);
            builder.Property(x => x.MessageId)
                .IsRequired();
            builder.Property(x => x.ConsumerName)
                .HasMaxLength(200)
                .IsRequired();
            builder.Property(x => x.ProcessedOn)
                .IsRequired();
            // Khóa duy nhất (Unique Index): Đảm bảo 1 consumer không bao giờ xử lý 1 message 2
lần
            builder.HasIndex(x => new { x.ConsumerName, x.MessageId })
                .IsUnique();
        }
    }
}
```

Bước 2. Tạo bộ lọc đường ống tiêu thụ

src/PropNest.Infrastructure/Filters/InboxDeduplicationFilter.cs:

[Lab 06 - Mã nguồn 8] Bộ lọc chống xử lý trùng lặp

src/PropNest.Infrastructure/Filters/InboxDeduplicationFilter.cs

```
      using MassTransit;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;
using PropNest.Domain.Workflows;
using PropNest.Infrastructure.Persistence;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Reflection;
namespace PropNest.Infrastructure.Filters
{
    public sealed partial class InboxDeduplicationFilter<T> : IFilter<ConsumeContext<T>>
         where T : class
```

```
    {
        public void Probe(ProbeContext context) => context.CreateFilterScope("inbox-
deduplication");
        public async Task Send(ConsumeContext<T> context, IPipe<ConsumeContext<T>> next)
        {
            // Nếu message không có MessageId thì không áp dụng deduplication
            if (!context.MessageId.HasValue)
            {
                await next.Send(context);
                return;
            }
            var messageId = context.MessageId.Value;
            var serviceProvider = context.GetPayload<IServiceProvider>();
            var dbContext = serviceProvider.GetRequiredService<PropNestDbContext>();
            var logger = serviceProvider.GetService<ILogger<InboxDeduplicationFilter<T>>>();
            // Lấy tên Consumer đang xử lý (hoặc fallback về tên kiểu Message)
            var consumerName = typeof(T).Name;
            // Try to detect a ConsumerConsumeContext<TConsumer, TMessage> implemented by the
runtime context.
            // Use reflection to read the Consumer property when available.
            var consumerConsumeContextGeneric = typeof(ConsumerConsumeContext<,>);
            var consumerInterface = context.GetType().GetInterfaces()
                .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
consumerConsumeContextGeneric);
            if (consumerInterface is not null)
            {
                var consumerProperty = consumerInterface.GetProperty("Consumer");
                if (consumerProperty is not null)
                {
                    var consumer = consumerProperty.GetValue(context);
                    if (consumer is not null)
                    {
                        consumerName = consumer.GetType().Name;
                    }
                }
            }
            // 1. Kiểm tra xem (ConsumerName, MessageId) đã từng được xử lý thành công chưa
            var alreadyProcessed = await dbContext.InboxMessages.AnyAsync(
                x => x.ConsumerName == consumerName && x.MessageId == messageId,
                context.CancellationToken);
            if (alreadyProcessed)
            {
                if (logger is not null)
                {
                    LogDuplicateMessage(logger, consumerName, messageId);
                }
                // Bỏ qua: Không gọi next.Send() để tránh xử lý trùng lặp
                return;
            }
            // 2. Chuyển tiếp cho Consumer xử lý nghiệp vụ
            await next.Send(context);
            // 3. Sau khi Consumer chạy thành công (không có lỗi), ghi nhận vào Inbox
            try
            {
                await dbContext.InboxMessages.AddAsync(
                    new InboxMessage(messageId, consumerName),
                    context.CancellationToken);
```

```
                await dbContext.SaveChangesAsync(context.CancellationToken);
            }
            catch (DbUpdateException)
            {
                // Bắt xung đột Unique Constraint nếu có 2 luồng xử lý cùng một lúc (Race
Condition)
                if (logger is not null)
                {
                    LogDuplicateMessage(logger, consumerName, messageId);
                }
            }
        }
        [LoggerMessage(Level = LogLevel.Warning, Message = "InboxDeduplication: Message
{MessageId} was already processed by {ConsumerName}. Skipping duplicate execution.")]
        private static partial void LogDuplicateMessage(ILogger logger, string consumerName, Guid
messageId);
    }
}
```

Giải thích cơ chế kỹ thuật: Trong kiến trúc hướng sự kiện, RabbitMQ đảm bảo cơ chế phân phối ít nhất một lần (At-least-once delivery), nghĩa là một thông điệp có thể được gửi lại nếu mạng chập chờn trước khi Consumer kịp gửi tín hiệu ACK. InboxDeduplicationFilter<T> can thiệp trực tiếp vào đường ống ConsumeContext<T> của MassTransit: trước khi gọi await next.Send(context) để thực thi Consumer, bộ lọc kiểm tra cặp khóa (ConsumerName, MessageId) trong bảng InboxMessages. Nếu đã tồn tại, bộ lọc ghi log cảnh báo và bỏ qua ngay lập tức. Nếu chưa tồn tại, sau khi Consumer thực thi thành công, bộ lọc ghi nhận bản ghi InboxMessage mới; đồng thời khối catch (DbUpdateException) xử lý an toàn trường hợp hai luồng song song cùng chạm tới rào cản Unique Index tại SQL Server.

Exercise 3: Xây dựng Máy Trạng thái Saga (PurchasePackageSaga) và 3 Consumers Nghiệp vụ

Task 1 – Xây dựng PurchasePackageSagaState và PurchasePackageSaga

Bước 1. Tạo tệp trạng thái lưu trữ

src/PropNest.Infrastructure/Workflows/PurchasePackageSagaState.cs và cấu hình Fluent API

src/PropNest.Infrastructure/Persistence/Configurations/PurchasePackageSagaStateConfiguration.cs:

[Lab 06 - Mã nguồn 9] Thực thể lưu trữ trạng thái PurchasePackageSagaState và cấu hình Concurrency

```
      using MassTransit;
using MassTransit.Middleware;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PropNest.Infrastructure.Workflows
{
    public class PurchasePackageSagaState : SagaStateMachineInstance, ISagaVersion
    {
        public Guid CorrelationId { get; set; }
        public string CurrentState { get; set; } = string.Empty;
        public int Version { get; set; }
        public long ListingId { get; set; }
        public long UserId { get; set; }
        public string PackageCode { get; set; } = string.Empty;
        public decimal ChargeAmount { get; set; }
        public string? ErrorMessage { get; set; }
        public DateTimeOffset CreatedAt { get; set; }
        public DateTimeOffset UpdatedAt { get; set; }
    }
}

// --- Tệp:
src/PropNest.Infrastructure/Persistence/Configurations/PurchasePackageSagaStateConfiguration.cs --
-
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using PropNest.Infrastructure.Workflows;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PropNest.Infrastructure.Persistence.Configurations
{
    public class PurchasePackageSagaStateConfiguration :
IEntityTypeConfiguration<PurchasePackageSagaState>
    {
        public void Configure(EntityTypeBuilder<PurchasePackageSagaState> builder)
        {
            builder.ToTable("PurchasePackageSagaStates");
            builder.HasKey(x => x.CorrelationId);
            builder.Property(x => x.CorrelationId).ValueGeneratedNever();
            builder.Property(x => x.CurrentState)
                .HasMaxLength(64)
                .IsRequired();
            builder.Property(x => x.PackageCode)
                .HasMaxLength(50)
                .IsRequired();
            builder.Property(x => x.ChargeAmount)
                .HasPrecision(18, 2);
```

```
            builder.Property(x => x.ErrorMessage)
                .HasMaxLength(500);
            builder.Property(x => x.Version)
                .IsConcurrencyToken();
        }
    }
}
```

Bước 2. Tạo tệp máy trạng thái điều phối trung tâm

src/PropNest.Infrastructure/Workflows/PurchasePackageSaga.cs:

[Lab 06 - Mã nguồn 10] Máy trạng thái điều phối trung tâm

src/PropNest.Infrastructure/Workflows/PurchasePackageSaga.cs

```
      using MassTransit;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;
using PropNest.Application.Workflows;
using PropNest.Domain.Workflows;
using PropNest.Infrastructure.Persistence;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PropNest.Infrastructure.Workflows
{
    public partial class PurchasePackageSaga : MassTransitStateMachine<PurchasePackageSagaState>
    {
        // States
        public State Started { get; private set; } = null!;
        public State DeductingWallet { get; private set; } = null!;
        public State WalletDeducted { get; private set; } = null!;
        public State UpgradingListing { get; private set; } = null!;
        public State RecordingHistory { get; private set; } = null!;
        public State Completed { get; private set; } = null!;
        public State Compensating { get; private set; } = null!;
        public State Compensated { get; private set; } = null!;
        public State Failed { get; private set; } = null!;
        // Events
        public Event<PurchasePackageRequested> PurchasePackageRequestedEvent { get; private set; }
= null!;
        public Event<WalletDeducted> WalletDeductedEvent { get; private set; } = null!;
        public Event<WalletDeductionFailed> WalletDeductionFailedEvent { get; private set; } =
null!;
        public Event<ListingUpgraded> ListingUpgradedEvent { get; private set; } = null!;
        public Event<ListingUpgradeFailed> ListingUpgradeFailedEvent { get; private set; } =
null!;
        public Event<WalletRefunded> WalletRefundedEvent { get; private set; } = null!;
        public PurchasePackageSaga()
        {
            InstanceState(x => x.CurrentState);
            // Khai báo CorrelationId mapping cho các Event
```

```
            Event(() => PurchasePackageRequestedEvent, x => x.CorrelateById(m =>
m.Message.CorrelationId));
            Event(() => WalletDeductedEvent, x => x.CorrelateById(m => m.Message.CorrelationId));
            Event(() => WalletDeductionFailedEvent, x => x.CorrelateById(m =>
m.Message.CorrelationId));
            Event(() => ListingUpgradedEvent, x => x.CorrelateById(m => m.Message.CorrelationId));
            Event(() => ListingUpgradeFailedEvent, x => x.CorrelateById(m =>
m.Message.CorrelationId));
            Event(() => WalletRefundedEvent, x => x.CorrelateById(m => m.Message.CorrelationId));
            // Step 1: khởi tạo saga -> trừ tiền
            Initially(
                When(PurchasePackageRequestedEvent)
                    .Then(context =>
                    {
                        context.Saga.CorrelationId = context.Message.CorrelationId;
                        context.Saga.ListingId = context.Message.ListingId;
                        context.Saga.UserId = context.Message.UserId;
                        context.Saga.PackageCode = context.Message.PackageCode;
                        context.Saga.ChargeAmount = context.Message.ChargeAmount;
                        context.Saga.CreatedAt = DateTimeOffset.UtcNow;
                        context.Saga.UpdatedAt = DateTimeOffset.UtcNow;
                    })
                    .ThenAsync(async context =>
                    {
                        await RecordStepAsync(context, WorkflowStatus.DeductingWallet,
"DeductWallet", isCompleted: false);
                    })
                    .TransitionTo(DeductingWallet)
                    .Publish(context => new DeductWalletCommand(
                        context.Saga.CorrelationId,
                        context.Saga.UserId,
                        context.Saga.ChargeAmount
                    ))
            );
            // Step 2: trong lúc trừ tiền
            During(DeductingWallet,
                When(WalletDeductedEvent)
                    .Then(context =>
                    {
                        context.Saga.UpdatedAt = DateTimeOffset.UtcNow;
                    })
                    .ThenAsync(async context =>
                    {
                        await RecordStepAsync(context, WorkflowStatus.WalletDeducted,
"DeductWallet", isCompleted: true);
                        await RecordStepAsync(context, WorkflowStatus.UpgradingListing,
"UpgradeListing", isCompleted: false);
                    })
                    .TransitionTo(UpgradingListing)
                    .Publish(context => new UpgradeListingCommand(
                        context.Saga.CorrelationId,
                        context.Saga.ListingId,
                        context.Saga.PackageCode
                    )),
                When(WalletDeductionFailedEvent)
                    .Then(context =>
                    {
```

```
                        context.Saga.ErrorMessage = context.Message.Reason;
                        context.Saga.UpdatedAt = DateTimeOffset.UtcNow;
                    })
                    .ThenAsync(async context =>
                    {
                        await RecordStepAsync(context, WorkflowStatus.Failed, "DeductWallet",
isCompleted: false, error: context.Message.Reason);
                    })
                    .TransitionTo(Failed)
            );
            // Step 3: nâng cấp tin
            During(UpgradingListing,
                When(ListingUpgradedEvent)
                    .Then(context =>
                    {
                        context.Saga.UpdatedAt = DateTimeOffset.UtcNow;
                    })
                    .ThenAsync(async context =>
                    {
                        await RecordStepAsync(context, WorkflowStatus.RecordingHistory,
"UpgradeListing", isCompleted: true);
                        await RecordStepAsync(context, WorkflowStatus.Completed, "RecordHistory",
isCompleted: true);
                    })
                    .TransitionTo(Completed),
                When(ListingUpgradeFailedEvent)
                    .Then(context =>
                    {
                        context.Saga.ErrorMessage = context.Message.Reason;
                        context.Saga.UpdatedAt = DateTimeOffset.UtcNow;
                    })
                    .ThenAsync(async context =>
                    {
                        await RecordStepAsync(context, WorkflowStatus.Compensating,
"UpgradeListing", isCompleted: false, error: context.Message.Reason);
                        await RecordStepAsync(context, WorkflowStatus.Compensating,
"RefundWallet", isCompleted: false);
                    })
                    .TransitionTo(Compensating)
                    .Publish(context => new RefundWalletCommand(
                        context.Saga.CorrelationId,
                        context.Saga.UserId,
                        context.Saga.ChargeAmount))
            );
            // Step 4: hoàn trả tiền
            During(Compensating,
                When(WalletRefundedEvent)
                    .Then(context =>
                    {
                        context.Saga.UpdatedAt = DateTimeOffset.UtcNow;
                    })
                    .ThenAsync(async context =>
                    {
                        await RecordStepAsync(context, WorkflowStatus.Compensated, "RefundWallet",
isCompleted: true);
                    })
                    .TransitionTo(Compensated)
```

```
            );
        }
       [LoggerMessage(
       Level = LogLevel.Information,
       Message = "PurchasePackageSaga [{CorrelationId}]: Transitioning to {Status}, Step:
{StepName}, Completed: {IsCompleted}, Error: {Error}")]
        private static partial void LogStepTransition(ILogger logger, Guid correlationId,
WorkflowStatus status, string stepName, bool isCompleted, string? error);
        private static async Task RecordStepAsync<TData>(
            BehaviorContext<PurchasePackageSagaState, TData> context,
            WorkflowStatus status,
            string stepName,
            bool isCompleted = false,
            string? error = null)
            where TData : class
        {
            var serviceProvider = context.GetPayload<IServiceProvider>();
            var dbContext = serviceProvider.GetRequiredService<PropNestDbContext>();
            var logger = serviceProvider.GetService<ILogger<PurchasePackageSaga>>();
            if (logger is not null)
            {
                LogStepTransition(logger, context.Saga.CorrelationId, status, stepName,
isCompleted, error);
            }
            var workflow = await dbContext.WorkflowInstances.FirstOrDefaultAsync(w =>
w.CorrelationId == context.Saga.CorrelationId);
            if (workflow is not null)
            {
                workflow.MoveTo(status, error);
            }
            var step = new WorkflowStep(context.Saga.CorrelationId, stepName);
            if (isCompleted)
            {
                step.Complete();
            }
            else if (!string.IsNullOrEmpty(error))
            {
                step.Fail(error);
            }
            await dbContext.WorkflowSteps.AddAsync(step);
            await dbContext.SaveChangesAsync();
        }
    }
}
```

Giải thích cơ chế kỹ thuật: PurchasePackageSaga kế thừa từ MassTransitStateMachine<PurchasePackageSagaState> và quản lý toàn bộ vòng đời giao dịch dựa trên khóa liên kết CorrelationId: (1) Ở trạng thái khởi đầu Initially, khi nhận sự kiện PurchasePackageRequested, máy trạng thái lưu thông tin giao dịch, chuyển sang trạng thái DeductingWallet và phát lệnh DeductWalletCommand; (2) Trong trạng thái DeductingWallet, nếu nhận WalletDeducted thì chuyển sang UpgradingListing và phát

lệnh UpgradeListingCommand; ngược lại nếu nhận WalletDeductionFailed (ví dụ không đủ số dư) thì chuyển thẳng sang trạng thái Failed (vì tiền chưa bị trừ nên không cần hoàn tiền); (3) Trong trạng thái UpgradingListing, nếu nhận ListingUpgraded thì chuyển sang Completed; nhưng nếu nhận ListingUpgradeFailed, máy trạng thái lập tức chuyển sang trạng thái Compensating và phát lệnh RefundWalletCommand để kích hoạt quy trình hoàn tiền tự động; (4) Trong trạng thái Compensating, khi nhận sự kiện WalletRefunded, máy trạng thái chuyển về điểm kết thúc an toàn Compensated.

Task 2 – Hiện thực hóa DeductWalletConsumer.cs

Bước 1. Tạo tệp src/PropNest.Infrastructure/Consumers/DeductWalletConsumer.cs xử lý lệnh trừ tiền ví:

[Lab 06 - Mã nguồn 11] Consumer xử lý trừ tiền ví src/PropNest.Infrastructure/Consumers/DeductWalletConsumer.cs

```
      using MassTransit;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
using PropNest.Application.Abstractions;
using PropNest.Application.Wallets;
using PropNest.Application.Workflows;
using PropNest.Domain.Wallets;
using PropNest.Infrastructure.Persistence;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PropNest.Infrastructure.Consumers
{
    public partial class DeductWalletConsumer : IConsumer<DeductWalletCommand>
    {
        private readonly PropNestDbContext _dbContext;
        private readonly IWalletRepository _walletRepository;
        private readonly IUnitOfWork _unitOfWork;
        private readonly ILogger<DeductWalletConsumer> _logger;
        public DeductWalletConsumer(
            PropNestDbContext dbContext,
            IWalletRepository walletRepository,
            IUnitOfWork unitOfWork,
            ILogger<DeductWalletConsumer> logger)
        {
            _dbContext = dbContext;
            _walletRepository = walletRepository;
            _unitOfWork = unitOfWork;
            _logger = logger;
```

```
        }
        public async Task Consume(ConsumeContext<DeductWalletCommand> context)
        {
            var commnad = context.Message;
            LogConsuming(_logger, commnad.CorrelationId, commnad.UserId, commnad.Amount);
            // Check Idempotency
            var alreadyProcessed = await _dbContext.WalletTransactions.AnyAsync(t =>
t.CorrelationId == commnad.CorrelationId && t.TransactionType == WalletTransactionType.Charge,
context.CancellationToken);
            if (alreadyProcessed)
            {
                LogAlreadyProcessed(_logger, commnad.CorrelationId);
                await context.Publish(new WalletDeducted(
                    commnad.CorrelationId,
                    commnad.UserId,
                    commnad.Amount
                ));
                return;
            }
            //lấy thông tin ví
            var wallet = await _walletRepository.GetByUserIdAsync( commnad.UserId,
context.CancellationToken );
            if(wallet is null)
            {
                LogFailed(_logger, commnad.CorrelationId, "Ví của người dùng không tồn tại");
                await context.Publish(new WalletDeductionFailed(
                    commnad.CorrelationId,
                    commnad.UserId,
                    "Ví của người dùng không tồn tại"
                ));
                return;
            }
            //trừ tiền
            var totalBalanceBefore = wallet.MainBalance + wallet.PromoBalance;
            try
            {
                wallet.Charge(commnad.Amount);
            }
            catch(InvalidOperationException ex)
            {
                LogFailed(_logger, commnad.CorrelationId, ex.Message);
                await context.Publish(new WalletDeductionFailed(
                    commnad.CorrelationId,
                    commnad.UserId,
                    ex.Message
                ));
                return;
            }
            var totalBalanceAfter = wallet.MainBalance + wallet.PromoBalance;
            var transaction = new WalletTransaction(
                walletId: wallet.WalletId,
                correlationId: commnad.CorrelationId,
                type: WalletTransactionType.Charge,
                amount: commnad.Amount,
                balanceBefore: totalBalanceBefore,
                balanceAfter: totalBalanceAfter,
                description: $"Thanh toán gói dịch vụ tin đăng ({commnad.Amount:N0}đ)"
```

```
            );
            await _walletRepository.AddTransactionAsync(transaction, context.CancellationToken);
            await _unitOfWork.SaveChangesAsync(context.CancellationToken);
            LogSuccess(_logger, commnad.CorrelationId, commnad.UserId, commnad.Amount);
            await context.Publish(new WalletDeducted(
                commnad.CorrelationId,
                commnad.UserId,
                commnad.Amount
            ));
        }
        [LoggerMessage(Level = LogLevel.Information, Message = "DeductWalletConsumer: Processing
DeductWalletCommand for CorrelationId: {CorrelationId}, UserId: {UserId}, Amount: {Amount}")]
        private static partial void LogConsuming(ILogger logger, Guid correlationId, long userId,
decimal amount);
        [LoggerMessage(Level = LogLevel.Warning, Message = "DeductWalletConsumer: CorrelationId
{CorrelationId} has already been charged. Skipping duplicate.")]
        private static partial void LogAlreadyProcessed(ILogger logger, Guid correlationId);
        [LoggerMessage(Level = LogLevel.Warning, Message = "DeductWalletConsumer: Failed to deduct
wallet for CorrelationId: {CorrelationId}. Reason: {Reason}")]
        private static partial void LogFailed(ILogger logger, Guid correlationId, string reason);
        [LoggerMessage(Level = LogLevel.Information, Message = "DeductWalletConsumer: Successfully
deducted {Amount} for UserId: {UserId}, CorrelationId: {CorrelationId}")]
        private static partial void LogSuccess(ILogger logger, Guid correlationId, long userId,
decimal amount);
    }
}
```

Task 3 – Hiện thực hóa UpgradeListingConsumer.cs

Bước 1. Tạo tệp src/PropNest.Infrastructure/Consumers/UpgradeListingConsumer.cs xử lý nâng hạng gói dịch vụ và cộng dồn thời hạn hiển thị:

[Lab 06 - Mã nguồn 12] Consumer nâng cấp tin đăng

src/PropNest.Infrastructure/Consumers/UpgradeListingConsumer.cs

```
      using MassTransit;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
using PropNest.Application.Abstractions;
using PropNest.Application.Listings;
using PropNest.Application.Workflows;
using PropNest.Domain.Listings;
using PropNest.Infrastructure.Persistence;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PropNest.Infrastructure.Consumers
{
    public partial class UpgradeListingConsumer : IConsumer<UpgradeListingCommand>
    {
        private readonly PropNestDbContext _dbContext;
        private readonly IListingRepository _listingRepository;
```

```
        private readonly IUnitOfWork _unitOfWork;
        private readonly ILogger<UpgradeListingConsumer> _logger;
        public UpgradeListingConsumer(
            PropNestDbContext dbContext,
            IListingRepository listingRepository,
            IUnitOfWork unitOfWork,
            ILogger<UpgradeListingConsumer> logger)
        {
            _dbContext = dbContext;
            _listingRepository = listingRepository;
            _unitOfWork = unitOfWork;
            _logger = logger;
        }
        public async Task Consume(ConsumeContext<UpgradeListingCommand> context)
        {
            var command = context.Message;
            LogConsuming(_logger, command.CorrelationId, command.ListingId, command.PackageCode);
            //check idempotency
            var alreadyProcessed = await _dbContext.ListingHistories.AnyAsync(
                    h =>h.ListingId == command.ListingId &&
                        h.ActionType == "UpgradePackage" &&
                        h.Note != null &&
                        h.Note.Contains(command.CorrelationId.ToString()),
                        context.CancellationToken
                );
            if (alreadyProcessed)
            {
                LogAlreadyProcessed(_logger, command.CorrelationId);
                await context.Publish(new ListingUpgraded(
                        command.CorrelationId,
                        command.ListingId,
                        command.PackageCode
                    ));
                return;
            }
            //tìm listing
            var listing = await _listingRepository.GetByIdAsync( command.ListingId,
context.CancellationToken);
            if(listing is null)
            {
                LogFailed(_logger, command.CorrelationId, $"Tin đăng với ID {command.ListingId}
không tồn tại.");
                await context.Publish(new ListingUpgradeFailed(
                        command.CorrelationId,
                        command.ListingId,
                        $"Tin đăng với ID {command.ListingId} không tồn tại."
                    ));
                return;
            }
            // check trạng thái listing
            if (listing.Status == ListingStatus.Deleted)
            {
                LogFailed(_logger, command.CorrelationId, "Không thể nâng cấp gói cho tin đăng đã
bị xóa.");
                await context.Publish(new ListingUpgradeFailed(
                    command.CorrelationId,
                    command.ListingId,
```

```
                    "Không thể nâng cấp gói cho tin đăng đã bị xóa."));
                return;
            }
            // lấy thông tin gói
            var package = await _listingRepository.GetPackageByCodeAsync(command.PackageCode,
context.CancellationToken);
            if (package is null || !package.IsActive)
            {
                LogFailed(_logger, command.CorrelationId, $"Gói dịch vụ '{command.PackageCode}'
không tồn tại hoặc đã ngừng áp dụng.");
                await context.Publish(new ListingUpgradeFailed(
                    command.CorrelationId,
                    command.ListingId,
                    $"Gói dịch vụ '{command.PackageCode}' không tồn tại hoặc đã ngừng áp dụng."));
                return;
            }
            // tính start - end
            var now = DateTimeOffset.UtcNow;
            var isStillActive = listing.EndDate.HasValue && listing.EndDate.Value > now;
            //neeys tin đăng còn hạn thì cộng dồn ngày nối tiếp, nếu hết hạn thì tính từ thời điểm
hiện tại
            var startDate = (isStillActive && listing.StartDate.HasValue) ?
listing.StartDate.Value : now;
            var baseDate = isStillActive ? listing.EndDate!.Value : now;
            var endDate = baseDate.AddDays(package.DurationDays);
            //nâng cấp tin và ghi history
            try
            {
                listing.UpgradePackage(command.PackageCode, startDate, endDate);
                var history = ListingHistory.CreateEvent(
                    listing.ListingId,
                    listing.OwnerUserId,
                    "UpgradePackage",
                    $"Nâng cấp gói {command.PackageCode} (+{package.DurationDays} ngày, HSD đến:
{endDate:yyyy-MM-dd HH:mm}) - CorrelationId: {command.CorrelationId}");
                _listingRepository.AddHistory( history );
                await _unitOfWork.SaveChangesAsync(context.CancellationToken);
            }
            catch (Exception ex)
            {
                LogFailed(_logger, command.CorrelationId, ex.Message);
                await context.Publish(new ListingUpgradeFailed(
                    command.CorrelationId,
                    command.ListingId,
                    ex.Message));
                return;
            }
            //publish event thành công
            LogSuccess(_logger, command.CorrelationId, command.ListingId, command.PackageCode);
            await context.Publish(new ListingUpgraded(
                command.CorrelationId,
                command.ListingId,
                command.PackageCode
                ));
        }
        [LoggerMessage(Level = LogLevel.Information, Message = "UpgradeListingConsumer: Processing
UpgradeListingCommand for CorrelationId: {CorrelationId}, ListingId: {ListingId}, PackageCode:
```

```
{PackageCode}")]
        private static partial void LogConsuming(ILogger logger, Guid correlationId, long
listingId, string packageCode);
        [LoggerMessage(Level = LogLevel.Warning, Message = "UpgradeListingConsumer: CorrelationId
{CorrelationId} has already been upgraded. Skipping duplicate.")]
        private static partial void LogAlreadyProcessed(ILogger logger, Guid correlationId);
        [LoggerMessage(Level = LogLevel.Warning, Message = "UpgradeListingConsumer: Failed to
upgrade listing for CorrelationId: {CorrelationId}. Reason: {Reason}")]
        private static partial void LogFailed(ILogger logger, Guid correlationId, string reason);
        [LoggerMessage(Level = LogLevel.Information, Message = "UpgradeListingConsumer:
Successfully upgraded listing {ListingId} to package {PackageCode} for CorrelationId:
{CorrelationId}")]
        private static partial void LogSuccess(ILogger logger, Guid correlationId, long listingId,
string packageCode);
    }
}
```

Task 4 – Hiện thực hóa RefundWalletConsumer.cs (Giao dịch Bù trừ Hoàn tiền)

Bước 1. Tạo tệp src/PropNest.Infrastructure/Consumers/RefundWalletConsumer.cs xử lý hoàn trả tiền vào ví người dùng khi bước nâng cấp tin thất bại:

[Lab 06 - Mã nguồn 13] Consumer hoàn tiền tự động

src/PropNest.Infrastructure/Consumers/RefundWalletConsumer.cs

```
      using MassTransit;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
using PropNest.Application.Abstractions;
using PropNest.Application.Wallets;
using PropNest.Application.Workflows;
using PropNest.Domain.Wallets;
using PropNest.Infrastructure.Persistence;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PropNest.Infrastructure.Consumers
{
    public partial class RefundWalletConsumer : IConsumer<RefundWalletCommand>
    {
        private readonly PropNestDbContext _dbContext;
        private readonly IWalletRepository _walletRepository;
        private readonly IUnitOfWork _unitOfWork;
        private readonly ILogger<RefundWalletConsumer> _logger;
        public RefundWalletConsumer(
            PropNestDbContext dbContext,
            IWalletRepository walletRepository,
            IUnitOfWork unitOfWork,
            ILogger<RefundWalletConsumer> logger)
        {
```

```
            _dbContext = dbContext;
            _walletRepository = walletRepository;
            _unitOfWork = unitOfWork;
            _logger = logger;
        }
        public async Task Consume(ConsumeContext<RefundWalletCommand> context)
        {
            var command = context.Message;
            LogConsuming(_logger, command.CorrelationId, command.UserId, command.Amount);
            // check idempotency
            var alreadyProcessed = await _dbContext.WalletTransactions.AnyAsync(
                w=>w.CorrelationId == command.CorrelationId && w.TransactionType ==
WalletTransactionType.Refund,
                context.CancellationToken
                );
            if (alreadyProcessed)
            {
                LogAlreadyProcessed(_logger, command.CorrelationId);
                await context.Publish(new WalletRefunded(
                    command.CorrelationId,
                    command.UserId,
                    command.Amount));
                return;
            }
            //lấy thông tin ví
            var wallet = await _walletRepository.GetByUserIdAsync(command.UserId,
context.CancellationToken);
            if (wallet is null)
            {
                LogWalletNotFound(_logger, command.CorrelationId, command.UserId);
                throw new InvalidOperationException($"Không tìm thấy ví của người dùng có ID
{command.UserId} để hoàn tiền.");
            }
            //hoàn tiền
            var totalBalanceBefore = wallet.MainBalance + wallet.PromoBalance;
            wallet.Refund(command.Amount);
            var totalBalanceAfter = wallet.MainBalance + wallet.PromoBalance;
            var transaction = new WalletTransaction(
              walletId: wallet.WalletId,
              correlationId: command.CorrelationId,
              type: WalletTransactionType.Refund,
              amount: command.Amount,
              balanceBefore: totalBalanceBefore,
              balanceAfter: totalBalanceAfter,
              description: $"Hoàn tiền giao dịch mua gói tin đăng thất bại ({command.Amount:N0}đ)"
          );
            await _walletRepository.AddTransactionAsync(transaction, context.CancellationToken);
            await _unitOfWork.SaveChangesAsync(context.CancellationToken);
            LogSuccess(_logger, command.CorrelationId, command.UserId, command.Amount);
            //publish event
            await context.Publish(new WalletRefunded(
               command.CorrelationId,
               command.UserId,
               command.Amount));
        }
        [LoggerMessage(Level = LogLevel.Information, Message = "RefundWalletConsumer: Processing
RefundWalletCommand for CorrelationId: {CorrelationId}, UserId: {UserId}, Amount: {Amount}")]
```

```
        private static partial void LogConsuming(ILogger logger, Guid correlationId, long userId,
decimal amount);
        [LoggerMessage(Level = LogLevel.Warning, Message = "RefundWalletConsumer: CorrelationId
{CorrelationId} has already been refunded. Skipping duplicate.")]
        private static partial void LogAlreadyProcessed(ILogger logger, Guid correlationId);
        [LoggerMessage(Level = LogLevel.Error, Message = "RefundWalletConsumer: Wallet not found
for UserId: {UserId}, CorrelationId: {CorrelationId}")]
        private static partial void LogWalletNotFound(ILogger logger, Guid correlationId, long
userId);
        [LoggerMessage(Level = LogLevel.Information, Message = "RefundWalletConsumer: Successfully
refunded {Amount} for UserId: {UserId}, CorrelationId: {CorrelationId}")]
        private static partial void LogSuccess(ILogger logger, Guid correlationId, long userId,
decimal amount);
    }
}
```

Giải thích cơ chế kỹ thuật: Cả ba Consumer (DeductWalletConsumer, UpgradeListingConsumer, RefundWalletConsumer) đều được thiết kế với cơ chế kiểm tra tính bất biến kép (Double Idempotency Defense): bên cạnh lớp chắn InboxDeduplicationFilter ở tầng hạ tầng, bản thân mỗi Consumer đều kiểm tra trực tiếp dữ liệu nghiệp vụ dựa trên CorrelationId (ví dụ kiểm tra bảng WalletTransactions đã có giao dịch Charge hoặc Refund cho CorrelationId đó hay chưa). Nếu đã xử lý rồi, Consumer không trừ tiền hoặc hoàn tiền lần thứ hai mà chỉ phát lại sự kiện xác nhận để Saga có thể tiến tới bước tiếp theo.

Verification

Để kiểm chứng toàn diện hoạt động của Saga State Machine và cơ chế Outbox/Inbox, chúng ta chạy đồng thời hai tiến trình PropNest.Api và PropNest.Workflow.Worker, sau đó thực hiện các kịch bản kiểm thử thực tế:

Kịch bản 1 (Happy Path): Nạp tiền vào ví và Mua gói dịch vụ VIP thành công

Bước 1. Nạp 500,000đ vào ví của Seller (UserId = 1) qua API POST /api/v1/wallets/top-up:

[Lab 06 - Kết quả 1] Nạp tiền thành công, số dư ví chính đạt 500,000đ

```
      POST https://localhost:65147/api/v1/wallets/top-up HTTP/1.1
Authorization: Bearer <SELLER_JWT_TOKEN>
Content-Type: application/json

{
  "amount": 500000,
```

```
  "description": "Nạp tiền phục vụ nâng cấp gói VIP"
}

--- PHẢN HỒI TỪ SERVER (HTTP 200 OK) ---
{
  "walletId": 1,
  "userId": 1,
  "mainBalance": 500000.00,
  "promoBalance": 0.00
}
```

Bước 2. Gửi yêu cầu mua gói VIP (150,000đ, 30 ngày) cho tin đăng ListingId = 10 kèm Header Idempotency-Key:

[Lab 06 - Kết quả 2] API ghi nhận vào Outbox và trả về ngay mã 202 Accepted cùng CorrelationId

```
      POST https://localhost:65147/api/v1/listings/10/purchase-package HTTP/1.1
Authorization: Bearer <SELLER_JWT_TOKEN>
Idempotency-Key: ikey-purchase-vip-listing10-001
Content-Type: application/json

{
  "packageCode": "VIP",
  "chargeAmount": 150000
}

--- PHẢN HỒI TỪ SERVER (HTTP 202 Accepted) ---
HTTP/1.1 202 Accepted
Location: https://localhost:65147/api/v1/workflows/3fa85f64-5717-4562-b3fc-2c963f66afa6

{
  "correlationId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "listingId": 10,
  "userId": 1,
  "packageCode": "VIP",
  "chargeAmount": 150000,
  "status": 0,
  "retryCount": 0,
  "errorMessage": null,
  "updatedAt": "2026-11-06T16:00:01.112+07:00"
}
```

Bước 3. Quan sát nhật ký Console của tiến trình PropNest.Workflow.Worker trong chu kỳ quét 5 giây tiếp theo:

[Lab 06 - Kết quả 3] Nhật ký thực thi tuần tự của Worker từ lúc phát tán Outbox đến khi Saga đạt trạng thái Completed

```
      [16:00:05 INF] MassTransitOutboxPublisher: Dispatching outbox event
'PurchasePackageRequested' (OutboxId: 8c12a401-..., CorrelationId: 3fa85f64-5717-4562-b3fc-
```

```
2c963f66afa6) to RabbitMQ.
[16:00:05 INF] MassTransitOutboxPublisher: Successfully published 'PurchasePackageRequested'
(OutboxId: 8c12a401-..., CorrelationId: 3fa85f64-5717-4562-b3fc-2c963f66afa6) to RabbitMQ.
[16:00:05 INF] Processed 1 outbox message(s).
[16:00:05 INF] PurchasePackageSaga [3fa85f64-5717-4562-b3fc-2c963f66afa6]: Transitioning to
DeductingWallet, Step: DeductWallet, Completed: False, Error:
[16:00:05 INF] DeductWalletConsumer: Processing DeductWalletCommand for CorrelationId: 3fa85f64-
5717-4562-b3fc-2c963f66afa6, UserId: 1, Amount: 150000
[16:00:05 INF] DeductWalletConsumer: Successfully deducted 150000 for UserId: 1, CorrelationId:
3fa85f64-5717-4562-b3fc-2c963f66afa6
[16:00:05 INF] PurchasePackageSaga [3fa85f64-5717-4562-b3fc-2c963f66afa6]: Transitioning to
WalletDeducted, Step: DeductWallet, Completed: True, Error:
[16:00:05 INF] PurchasePackageSaga [3fa85f64-5717-4562-b3fc-2c963f66afa6]: Transitioning to
UpgradingListing, Step: UpgradeListing, Completed: False, Error:
[16:00:05 INF] UpgradeListingConsumer: Processing UpgradeListingCommand for CorrelationId:
3fa85f64-5717-4562-b3fc-2c963f66afa6, ListingId: 10, PackageCode: VIP
[16:00:06 INF] UpgradeListingConsumer: Successfully upgraded listing 10 to package VIP for
CorrelationId: 3fa85f64-5717-4562-b3fc-2c963f66afa6
[16:00:06 INF] PurchasePackageSaga [3fa85f64-5717-4562-b3fc-2c963f66afa6]: Transitioning to
RecordingHistory, Step: UpgradeListing, Completed: True, Error:
[16:00:06 INF] PurchasePackageSaga [3fa85f64-5717-4562-b3fc-2c963f66afa6]: Transitioning to
Completed, Step: RecordHistory, Completed: True, Error:
```

Kịch bản 2 (Compensation Path): Tự động hoàn tiền khi nâng cấp gói thất bại

Bước 1. Gửi yêu cầu mua một mã gói không tồn tại ("VIP_GOLD_999", trị giá 150,000đ) cho tin đăng ListingId = 10:

[Lab 06 - Lệnh 1] Gửi yêu cầu mua gói dịch vụ không hợp lệ để kích hoạt kịch bản hoàn tiền tự động

```
      POST https://localhost:65147/api/v1/listings/10/purchase-package HTTP/1.1
Authorization: Bearer <SELLER_JWT_TOKEN>
Idempotency-Key: ikey-purchase-invalid-pkg-002
Content-Type: application/json

{
  "packageCode": "VIP_GOLD_999",
  "chargeAmount": 150000
}
```

Bước 2. Quan sát nhật ký của Worker khi bước nâng cấp tin thất bại và Saga tự động kích hoạt RefundWalletConsumer:

[Lab 06 - Kết quả 4] Nhật ký chứng minh Saga tự động phát lệnh RefundWalletCommand và hoàn tiền thành công

```
      [16:05:10 INF] PurchasePackageSaga [9b21c880-1122-4333-9444-556677889900]: Transitioning to
DeductingWallet, Step: DeductWallet, Completed: False, Error:
[16:05:10 INF] DeductWalletConsumer: Processing DeductWalletCommand for CorrelationId: 9b21c880-
1122-4333-9444-556677889900, UserId: 1, Amount: 150000
```

```
[16:05:10 INF] DeductWalletConsumer: Successfully deducted 150000 for UserId: 1, CorrelationId:
9b21c880-1122-4333-9444-556677889900
[16:05:10 INF] PurchasePackageSaga [9b21c880-1122-4333-9444-556677889900]: Transitioning to
UpgradingListing, Step: UpgradeListing, Completed: False, Error:
[16:05:10 INF] UpgradeListingConsumer: Processing UpgradeListingCommand for CorrelationId:
9b21c880-1122-4333-9444-556677889900, ListingId: 10, PackageCode: VIP_GOLD_999
[16:05:10 WRN] UpgradeListingConsumer: Failed to upgrade listing for CorrelationId: 9b21c880-1122-
4333-9444-556677889900. Reason: Gói dịch vụ 'VIP_GOLD_999' không tồn tại hoặc đã ngừng áp dụng.
[16:05:10 INF] PurchasePackageSaga [9b21c880-1122-4333-9444-556677889900]: Transitioning to
Compensating, Step: UpgradeListing, Completed: False, Error: Gói dịch vụ 'VIP_GOLD_999' không tồn
tại hoặc đã ngừng áp dụng.
[16:05:10 INF] PurchasePackageSaga [9b21c880-1122-4333-9444-556677889900]: Transitioning to
Compensating, Step: RefundWallet, Completed: False, Error:
[16:05:11 INF] RefundWalletConsumer: Processing RefundWalletCommand for CorrelationId: 9b21c880-
1122-4333-9444-556677889900, UserId: 1, Amount: 150000
[16:05:11 INF] RefundWalletConsumer: Successfully refunded 150000 for UserId: 1, CorrelationId:
9b21c880-1122-4333-9444-556677889900
[16:05:11 INF] PurchasePackageSaga [9b21c880-1122-4333-9444-556677889900]: Transitioning to
Compensated, Step: RefundWallet, Completed: True, Error:
```

Bước 3. Đối chiếu bảng dbo.WalletTransactions trên SQL Server để xác minh dòng tiền của người bán được bảo toàn tuyệt đối:

[Lab 06 - Kết quả 5] Bảng lịch sử giao dịch ví ghi nhận rõ ràng lệnh trừ tiền (TransactionId=3) và lệnh hoàn tiền (TransactionId=4), khôi phục số dư về đúng 350,000đ

```
      SELECT TransactionId, TransactionType, Amount, BalanceBefore, BalanceAfter, Description
FROM dbo.WalletTransactions
WHERE WalletId = 1
ORDER BY TransactionId ASC;

-- TransactionId | TransactionType | Amount    | BalanceBefore | BalanceAfter | Description
-- --------------+-----------------+-----------+---------------+--------------+-------------------
-------------------------------------------
-- 1             | 0 (TopUp)       | 500000.00 | 0.00          | 500000.00    | Nạp tiền phục vụ
nâng cấp gói VIP
-- 2             | 1 (Charge)      | 150000.00 | 500000.00     | 350000.00    | Thanh toán gói
dịch vụ tin đăng (150,000đ)
-- 3             | 1 (Charge)      | 150000.00 | 350000.00     | 200000.00    | Thanh toán gói
dịch vụ tin đăng (150,000đ)
-- 4             | 2 (Refund)      | 150000.00 | 200000.00     | 350000.00    | Hoàn tiền giao
dịch mua gói tin đăng thất bại (150,000đ)
```

Summary

[Lab 06 - Bảng 1] Danh mục kiểm tra hoàn thành Hands-On Lab 06

Mẫu thiết kế (Pattern)	Thành phần cài đặt trong PropNest	Trạng thái kiểm chứng

Idempotent API Request	WorkflowService + IdempotencyRequests (SHA-256)	Chặn bấm đúp nút mua gói, trả về 409 nếu tái sử dụng key khác payload.
Transactional Outbox	OutboxMessages + OutboxDispatcherWorker (5s)	Lưu sự kiện cùng Local Transaction với dữ liệu nghiệp vụ.
Inbox Deduplication	InboxDeduplicationFilter <T> + InboxMessages	Chặn thông điệp lặp lại từ RabbitMQ trước khi vào Consumer.
Saga State Machine	PurchasePackageSaga + 3 Consumers	Hoàn tất Happy Path (Completed) và tự động hoàn tiền khi lỗi (Compensated).

HANDS-ON LAB 07: KIỂM THỬ ĐƠN VỊ VỚI XUNIT, ĐÓNG GÓI DOCKER MULTI-STAGE VÀ BÀI TẬP THỰC HÀNH TẠI CHỖ (LIVE DEFENSE LABS)

Overview

Một hệ thống Backend chỉ thực sự sẵn sàng vận hành khi có bộ kiểm thử tự động bảo vệ các quy tắc nghiệp vụ cốt lõi khỏi lỗi hồi quy (Regression Bugs) và có khả năng đóng gói, khởi chạy nhất quán trên mọi môi trường chỉ bằng một câu lệnh duy nhất. Hơn thế nữa, theo quy định đánh giá của học phần, mỗi sinh viên khi bảo vệ đồ án đều phải thực hiện trực tiếp các bài tập mở rộng tại chỗ (Live Hands-On Defense) ngay trên mã nguồn đã nộp.

Bài thực hành này hướng dẫn xây dựng trọn vẹn bộ 20 ca kiểm thử đơn vị (Unit Tests) bằng xUnit cho tầng Domain và Application, thiết lập Dockerfile đa tầng (Multi-Stage Build) tối ưu hóa kích thước ảnh container, và giải quyết chi tiết 3 đề bài thực hành tại chỗ tiêu biểu.

Objectives

Sau khi hoàn thành bài thực hành này, bạn sẽ nắm vững cách:

- **Kiểm thử đơn vị tự động:** Viết và thực thi thành công 20 ca kiểm thử đơn vị (xUnit) bao phủ cả luồng hợp lệ (Happy Path) và luồng ngoại lệ (Exception Path) cho Listing, ListingHistory và DeltaEngine.
- **Đóng gói Container:** Xây dựng Dockerfile Multi-Stage tách biệt môi trường SDK biên dịch và môi trường ASP.NET Runtime siêu nhẹ.
- **Sẵn sàng bảo vệ thực hành tại chỗ:** Thực hành thành thạo 3 bài toán mở rộng trực tiếp tại buổi bảo vệ: (1) Chèn Custom Middleware đo thời gian xử lý; (2) Thêm Policy Authorization mới; (3) Thêm cột mới vào Entity kèm EF Core Migration.

Prerequisites

- Đã hoàn thành Hands-On Lab 01 đến Hands-On Lab 06.

Exercises

Bài thực hành (Hands-On Lab) này bao gồm 3 bài tập (Exercises) thực hiện tuần tự:

- **Exercise 1:** Xây dựng Bộ Kiểm thử Đơn vị (Unit Tests) với xUnit (ListingTests, DeltaEngineTests, ListingHistoryTests).
- **Exercise 2:** Đóng gói Ứng dụng với Dockerfile Multi-Stage.
- **Exercise 3:** Thực hành 3 Đề bài Mở rộng Tại chỗ khi Vấn đáp (Live Defense Labs).

Exercise 1: Xây dựng Bộ Kiểm thử Đơn vị (Unit Tests) với xUnit

Task 1 – Kiểm thử máy trạng thái thực thể Listing (ListingTests.cs)

Bước 1. Tạo tệp tests/PropNest.Domain.Tests/ListingTests.cs chứa 12 ca kiểm thử xác minh toàn bộ các quy tắc chuyển trạng thái của tin đăng:

[Lab 07 - Mã nguồn 1] Bộ 12 ca kiểm thử vòng đời thực thể Listing (tests/PropNest.Domain.Tests/ListingTests.cs)

```
      using PropNest.Domain.Listings;
namespace PropNest.Domain.Tests;
public sealed class ListingTests
{
    [Fact]
    public void SubmitForModerationMovesDraftToPendingModeration()
    {
        var listing = CreateListing();
        listing.SubmitForModeration();
        Assert.Equal(ListingStatus.PendingModeration, listing.Status);
        Assert.Equal(ModerationDecision.Pending, listing.ModerationDecision);
    }
    [Fact]
    public void ApproveRequiresAListingPendingModeration()
    {
        var listing = CreateListing();
        Assert.Throws<InvalidOperationException>(() => listing.Approve(DateTimeOffset.UtcNow,
DateTimeOffset.UtcNow.AddDays(7)));
    }
    //Nộp lại tin đăng đã bị từ chối để đưa về trạng thái đang chờ duyệt
    [Fact]
    public void SubmitForModerationFromRejectedShouldMovesToPendingModeration()
    {
        var listing = CreateRejectedListing();
        listing.SubmitForModeration();
        Assert.Equal(ListingStatus.PendingModeration, listing.Status);
        Assert.Equal(ModerationDecision.Pending, listing.ModerationDecision);
    }
    //Nộp lại tin đăng đã bị ẩn để đưa về trạng thái đang chờ duyệt
    [Fact]
```

```
    public void SubmitForModerationFromHiddenShouldMoveToPendingModeration()
    {
        var listing = CreateHiddenListing();
        listing.SubmitForModeration();
        Assert.Equal(ListingStatus.PendingModeration, listing.Status);
        Assert.Equal(ModerationDecision.Pending, listing.ModerationDecision);
    }
    //Duyệt tin hợp lệ -> Published
    [Fact]
    public void ApproveFromPendingModerationShouldMoveToPublished()
    {
        var listing = CreatePendingListing();
        listing.Approve(DateTimeOffset.UtcNow, DateTimeOffset.UtcNow.AddDays(7));
        Assert.Equal(ListingStatus.Published, listing.Status);
        Assert.Equal(ModerationDecision.Approved, listing.ModerationDecision);
    }
    //Từ chối tin hợp lệ -> Rejected
    [Fact]
    public void RejectFromPendingModerationShouldMoveToRejected()
    {
        var listing = CreatePendingListing();
        listing.Reject();
        Assert.Equal(ListingStatus.Rejected, listing.Status);
        Assert.Equal(ModerationDecision.Rejected, listing.ModerationDecision);
    }
    //Ẩn tin đăng đang hiển thị-> Hidden
    [Fact]
    public void HideFromPublishedShouldMoveToHidden()
    {
        var listing = CreatePublishedListing();
        listing.Hide();
        Assert.Equal(ListingStatus.Hidden, listing.Status);
    }
    //Nộp duyệt tin khi tin đã Published -> InvalidOperationException
    [Fact]
    public void SubmitForModerationFromPublishedShouldThrowInvalidOperationException()
    {
        var listing = CreatePublishedListing();
        Assert.Throws<InvalidOperationException>(() => listing.SubmitForModeration());
    }
    //Nộp duyệt 2 lần liên tiếp -> InvalidOperationException
    [Fact]
    public void
SubmitForModerationWhenAlreadyPendingModerationShouldThrowInvalidOperationException()
    {
        var listing = CreateListing();
        listing.SubmitForModeration();
        Assert.Throws<InvalidOperationException>(() => listing.SubmitForModeration());
    }
    //Ẩn tin khi đang ở trạng thái Draft -> InvalidOperationException
    [Fact]
    public void HideFromDraftShouldThrowInvalidOperationException()
    {
        var listing = CreateListing();
        Assert.Throws<InvalidOperationException>(() => listing.Hide());
    }
    //Ẩn tin khi đang chờ duyệt -> InvalidOperationException
```

```
    [Fact]
    public void HideFromPendingModerationShouldThrowInvalidOperationException()
    {
        var listing = CreatePendingListing();
        Assert.Throws<InvalidOperationException>(() => listing.Hide());
    }
    //Từ chối duyệt tin khi đang ở trạng thái Draft -> InvalidOperationException
    [Fact]
    public void RejectFromDraftShouldThrowInvalidOperationException()
    {
        var listing = CreateListing();
        Assert.Throws<InvalidOperationException>(() => listing.Reject());
    }
    //Tạo tin đăng đang chờ duyệt
    private static Listing CreatePendingListing()
    {
        var listing = CreateListing();
        listing.SubmitForModeration();
        return listing;
    }
    //Tạo tin đăng đã được duyệt
    private static Listing CreatePublishedListing()
    {
        var listing = CreatePendingListing();
        listing.Approve(DateTimeOffset.UtcNow, DateTimeOffset.UtcNow.AddDays(7));
        return listing;
    }
    //Tạo tin đăng đã bị từ chối
    private static Listing CreateRejectedListing()
    {
        var listing = CreatePendingListing();
        listing.Reject();
        return listing;
    }
    //Tạo tin đăng đã bị ẩn đi
    private static Listing CreateHiddenListing()
    {
        var listing = CreatePublishedListing();
        listing.Hide();
        return listing;
    }
    private static Listing CreateListing() => new(
        1,
        "Apartment for sale",
        "A valid property listing.",
        "Apartment",
        "Sale",
        4_500_000_000m,
        75m,
        "Ho Chi Minh City",
        "District 7",
        "Tan Phu",
        "1 Nguyen Van Linh");
}
```

Giải thích cơ chế kỹ thuật: Bộ kiểm thử ListingTests áp dụng mô hình Arrange - Act - Assert (AAA) để kiểm chứng cả hai chiều: (1) Các chuyển đổi trạng thái hợp lệ (Draft -> PendingModeration -> Published -> Hidden -> PendingModeration, hoặc PendingModeration -> Rejected -> PendingModeration); và (2) Các hành vi vi phạm bất biến miền dùng Assert.Throws<InvalidOperationException> (như cố tình duyệt một tin đang ở trạng thái Draft, gửi duyệt hai lần liên tiếp, hoặc ẩn một tin chưa được xuất bản). Vì tầng Domain hoàn toàn độc lập với cơ sở dữ liệu, 12 ca kiểm thử này thực thi hoàn toàn trên bộ nhớ RAM trong thời gian dưới 15 mili-giây.

Task 2 – Kiểm thử thuật toán vi sai DeltaEngine (DeltaEngineTests.cs)

Bước 1. Tạo tệp tests/PropNest.Domain.Tests/DeltaEngineTests.cs chứa 5 ca kiểm thử sử dụng biểu thức with của C# Record:

[Lab 07 - Mã nguồn 2] Bộ 5 ca kiểm thử thuật toán vi sai (tests/PropNest.Domain.Tests/DeltaEngineTests.cs)

```
      using PropNest.Application.Listings;
using PropNest.Domain.Listings;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace PropNest.Domain.Tests
{
    public sealed class DeltaEngineTests
    {
        //Thay đổi giá
        [Fact]
        public void PriceChangeDetected()
        {
            var deltas = CreateDeltas(Price: 5_000_000_000m);
            var delta = Assert.Single(deltas);
            Assert.Equal("Price", delta.FieldName);
            Assert.Equal(4_500_000_000m, delta.OldValue);
            Assert.Equal(5_000_000_000m, delta.NewValue);
        }
        //Thay đổi tiêu đề
        [Fact]
        public void TitleChangeDetected()
        {
            var deltas = CreateDeltas(Title: "Villa for rent");
            var delta = Assert.Single(deltas);
            Assert.Equal("Title", delta.FieldName);
            Assert.Equal("Apartment for sale", delta.OldValue);
```

```
            Assert.Equal("Villa for rent", delta.NewValue);
        }
        //Không có thay đổi
        [Fact]
        public void NoChangeDetected()
        {
            var deltas = CreateDeltas();
            Assert.Empty(deltas);
        }
        //Đổi description null -> "Tan Phu"
        [Fact]
        public void NullToValueDetected()
        {
            var deltas = CreateDeltas(Description: "A new Description");
            var delta = Assert.Single(deltas);
            Assert.Equal("Description", delta.FieldName);
            Assert.Null(delta.OldValue);
            Assert.Equal("A new Description", delta.NewValue);
        }
        //Đổi ward "Tan Phu" -> null
        [Fact]
        public void ValueToNullDetected()
        {
            var deltas = CreateDeltas(Ward: null);
            var delta = Assert.Single(deltas);
            Assert.Equal("Ward", delta.FieldName);
            Assert.Equal("Tan Phu", delta.OldValue);
            Assert.Null(delta.NewValue);
        }
        private static List<ListingDelta> CreateDeltas(decimal? Price = null, string? Title =
null, string? Ward="Tan Phu", string? Description = null)
        {
            var before = CreateListingDto();
            var after = before with
            {
                Price = Price ?? before.Price,
                Title = Title ?? before.Title,
                Description = Description ?? before.Description,
                Ward = Ward
            };
            return DeltaEngine.CreateDeltas(before, after);
        }
        private static ListingDto CreateListingDto() => new(
        ListingId: 1,
        OwnerUserId: 1,
        Title: "Apartment for sale",
        Description: null,
        PropertyType: "Apartment",
        ListingType: "Sale",
        Price: 4_500_000_000m,
        Area: 75m,
        City: "Ho Chi Minh City",
        District: "District 7",
        Ward: "Tan Phu",
        Address: "1 Nguyen Van Linh",
        Status: ListingStatus.Draft,
        ModerationDecision: ModerationDecision.Approved,
```

```
        PackageCode: "Standard",
        StartDate: null,
        EndDate: null,
        RowVersion: Array.Empty<byte>());
    }
}
```

Task 3 – Kiểm thử thực thể ListingHistory (ListingHistoryTests.cs)

Bước 1. Tạo tệp tests/PropNest.Domain.Tests/ListingHistoryTests.cs chứa 3 ca kiểm thử xác minh các phương thức khởi tạo tĩnh (Factory Methods):

[Lab 07 - Mã nguồn 3] Bộ 3 ca kiểm thử thực thể ListingHistory (tests/PropNest.Domain.Tests/ListingHistoryTests.cs)

```
      using PropNest.Domain.Listings;
namespace PropNest.Domain.Tests;
public sealed class ListingHistoryTests
{
    [Fact]
    public void CreateSnapshotStoresOnlySnapshotPayload()
    {
        var history = ListingHistory.CreateSnapshot(10, 20, "Create",
"{\"title\":\"Apartment\"}");
        Assert.Equal(10, history.ListingId);
        Assert.Equal(20, history.ActorUserId);
        Assert.Equal("Create", history.ActionType);
        Assert.Equal("{\"title\":\"Apartment\"}", history.SnapshotJson);
        Assert.Null(history.DeltasJson);
    }
    [Fact]
    public void CreateDeltaStoresOnlyDeltaPayload()
    {
        var history = ListingHistory.CreateDelta(10, 20, "UpdateContent",
"[{\"fieldName\":\"Price\"}]");
        Assert.Null(history.SnapshotJson);
        Assert.Equal("[{\"fieldName\":\"Price\"}]", history.DeltasJson);
    }
    [Fact]
    public void CreateEventStoresNoPayload()
    {
        var history = ListingHistory.CreateEvent(10, 20, "Submit");
        Assert.Null(history.SnapshotJson);
        Assert.Null(history.DeltasJson);
    }
}
```

Exercise 2: Đóng gói Ứng dụng với Dockerfile Multi-Stage

Task 1 – Xây dựng Dockerfile đa tầng cho PropNest.Api và PropNest.Workflow.Worker

Bước 1. Thiết kế Dockerfile theo kiến trúc Multi-Stage gồm 3 giai đoạn (restore -> build/publish -> runtime) để loại bỏ toàn bộ bộ công cụ biên dịch nặng nề khỏi ảnh chạy thực tế:

[Lab 07 - Mã nguồn 4] Cấu trúc Dockerfile Multi-Stage cho dịch vụ PropNest.Api

```
      # Stage 1: Build & Publish với .NET 8 SDK
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Sao chép các tệp cấu hình quản lý gói tập trung và project để tận dụng Docker Layer Caching
COPY ["Directory.Build.props", "./"]
COPY ["Directory.Packages.props", "./"]
COPY ["src/PropNest.Domain/PropNest.Domain.csproj", "src/PropNest.Domain/"]
COPY ["src/PropNest.Application/PropNest.Application.csproj", "src/PropNest.Application/"]
COPY ["src/PropNest.Infrastructure/PropNest.Infrastructure.csproj",
"src/PropNest.Infrastructure/"]
COPY ["src/PropNest.Api/PropNest.Api.csproj", "src/PropNest.Api/"]

RUN dotnet restore "src/PropNest.Api/PropNest.Api.csproj"

# Sao chép toàn bộ mã nguồn và xuất bản ở chế độ Release
COPY . .
WORKDIR "/src/src/PropNest.Api"
RUN dotnet publish "PropNest.Api.csproj" -c Release -o /app/publish /p:UseAppHost=false

# Stage 2: Runtime siêu nhẹ chỉ chứa ASP.NET Core 8 Runtime
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
EXPOSE 8080
ENV ASPNETCORE_URLS=http://+:8080
COPY --from=build /app/publish .
ENTRYPOINT ["dotnet", "PropNest.Api.dll"]
```

Giải thích cơ chế kỹ thuật: Kỹ thuật sao chép riêng Directory.Build.props, Directory.Packages.props và các tệp .csproj trước khi chạy lệnh RUN dotnet restore giúp Docker lưu đệm (cache) tầng thư viện NuGet. Khi lập trình viên chỉ sửa đổi mã nguồn .cs mà không thêm gói NuGet mới, Docker sẽ bỏ qua hoàn toàn bước tải gói và đi thẳng tới bước biên dịch. Đồng thời, việc chỉ sao chép thư mục /app/publish sang ảnh nền mcr.microsoft.com/dotnet/aspnet:8.0 giúp giảm dung lượng ảnh container từ hơn 850 MB (của ảnh SDK) xuống chỉ còn khoảng 102 MB.

Exercise 3: Thực hành 3 Đề bài Mở rộng Tại chỗ khi Vấn đáp (Live Defense Labs)

Phần này hướng dẫn chi tiết cách thực hiện nhanh, chính xác 3 đề bài thực hành tại chỗ tiêu biểu cho công nghệ ASP.NET Core 8.

Task 1 (Đề bài tại chỗ số 1) – Chèn một Middleware mới và chứng minh vị trí đặt là đúng

Đề bài: Yêu cầu: Viết một Middleware đo thời gian thực thi của mỗi HTTP Request (tính bằng mili-giây), ghi ra Header phản hồi X-Execution-Time-Ms và ghi log thông tin hoàn tất request.

Bước 1. Tạo tệp src/PropNest.Api/Middleware/RequestTimingMiddleware.cs với mã nguồn hoàn chỉnh sau:

[Lab 07 - Mã nguồn 5] Hiện thực hóa RequestTimingMiddleware sử dụng Stopwatch.GetTimestamp()

```
      using System.Diagnostics;

namespace PropNest.Api.Middleware;

public sealed partial class RequestTimingMiddleware(RequestDelegate next,
ILogger<RequestTimingMiddleware> logger)
{
    public const string HeaderName = "X-Execution-Time-Ms";

    [LoggerMessage(Level = LogLevel.Information, Message = "HTTP {Method} {Path} responded
{StatusCode} in {ElapsedMs} ms.")]
    private static partial void LogRequestCompleted(ILogger logger, string method, string path,
int statusCode, long elapsedMs);

    public async Task InvokeAsync(HttpContext context)
    {
        var startTimestamp = Stopwatch.GetTimestamp();

        context.Response.OnStarting(() =>
        {
            var elapsed = Stopwatch.GetElapsedTime(startTimestamp);
            var elapsedMs = (long)elapsed.TotalMilliseconds;
            context.Response.Headers[HeaderName] = elapsedMs.ToString();
            return Task.CompletedTask;
        });

        await next(context);

        var totalElapsedMs = (long)Stopwatch.GetElapsedTime(startTimestamp).TotalMilliseconds;
        LogRequestCompleted(logger, context.Request.Method, context.Request.Path,
```

```
context.Response.StatusCode, totalElapsedMs);
    }
}
```

Bước 2. Chèn RequestTimingMiddleware vào src/PropNest.Api/Program.cs ngay sau CorrelationIdMiddleware và trước UseAuthentication():

[Lab 07 - Mã nguồn 6] Vị trí đặt RequestTimingMiddleware chuẩn xác trong đường ống Pipeline

```
      app.UseMiddleware<ExceptionHandlingMiddleware>();
app.UseMiddleware<CorrelationIdMiddleware>();
app.UseMiddleware<RequestTimingMiddleware>(); // <-- Đặt tại đây!
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
```

Luận giải chứng minh vị trí đặt Middleware: Tại sao đặt RequestTimingMiddleware ngay sau CorrelationIdMiddleware là đúng nhất? (1) Vì đặt sau CorrelationIdMiddleware, dòng log LogRequestCompleted của RequestTimingMiddleware sẽ tự động thừa hưởng CorrelationId từ Log Scope của CorrelationIdMiddleware; (2) Vì đặt trước UseAuthentication, UseAuthorization và MapControllers, đồng hồ bấm giờ sẽ đo được tổng thời gian thực tế bao gồm cả thời gian giải mã JWT, kiểm tra Policy lẫn thời gian truy vấn EF Core; (3) Việc đăng ký callback context.Response.OnStarting(...) đảm bảo Header X-Execution-Time-Ms được ghi vào luồng HTTP ngay trước khi byte dữ liệu đầu tiên của Body được gửi trả về Client (tránh lỗi InvalidOperationException: Headers are read-only, response has already started).

Task 2 (Đề bài tại chỗ số 2) – Thêm một Policy Authorization mới cho Endpoint có sẵn

Đề bài: Yêu cầu: Tạo thêm một chính sách phân quyền mang tên "StrictAdminOnly" yêu cầu cả Role và Claim cụ thể, sau đó áp dụng cho một Endpoint quản trị.

Bước 1. Mở tệp src/PropNest.Api/Program.cs (tại khối AddAuthorization ở dòng 86) và bổ sung chính sách mới:

[Lab 07 - Mã nguồn 7] Bổ sung Policy 'StrictAdminOnly' vào Program.cs

```
      builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CanModerateListings", policy => policy.RequireRole("Moderator", "Admin"));

    // Chính sách mới thêm tại chỗ: Chỉ Admin có email hợp lệ mới được phép thực thi
    options.AddPolicy("StrictAdminOnly", policy =>
        policy.RequireRole("Admin")
              .RequireClaim(System.Security.Claims.ClaimTypes.Email));
});
```

Bước 2. Áp dụng thuộc tính [Authorize(Policy = "StrictAdminOnly")] lên Endpoint cần bảo vệ (ví dụ trong ModerationController hoặc WorkflowsController) và gọi thử bằng Token của Moderator (nhận 403 Forbidden) so với Token của Admin (nhận 200 OK).

Task 3 (Đề bài tại chỗ số 3) – Thêm Migration và cập nhật API theo Schema mới

Đề bài: Yêu cầu: Bổ sung thêm thuộc tính Số phòng ngủ (Bedrooms - kiểu int?) vào thực thể Listing, tạo bản Migration cập nhật cơ sở dữ liệu và trả trường này ra API.

Bước 1. Bổ sung thuộc tính public int? Bedrooms { get; private set; } vào tệp src/PropNest.Domain/Listings/Listing.cs và thêm cột tương ứng vào ListingDto, CreateListingRequest.

Bước 2. Chạy lệnh CLI tạo bản Migration mới và cập nhật ngay xuống SQL Server:

[Lab 07 - Lệnh 1] Tạo và áp dụng Migration mới ngay tại chỗ

```
      dotnet ef migrations add AddListingBedrooms --project src/PropNest.Infrastructure --startup-project src/PropNest.Api
dotnet ef database update --project src/PropNest.Infrastructure --startup-project src/PropNest.Api
```

Verification

Bước 1. Thực thi toàn bộ bộ kiểm thử đơn vị của dự án bằng lệnh dotnet test:

[Lab 07 - Kết quả 1] Thực thi thành công 20/20 ca kiểm thử đơn vị xUnit (0 Failed)

```
      PS C:\Users\GiaBao\source\repos\PropNest> dotnet test tests/PropNest.Domain.Tests/PropNest.Domain.Tests.csproj --verbosity normal
Build started, please wait...
Build completed.

Test run for C:\Users\GiaBao\source\repos\PropNest\tests\PropNest.Domain.Tests\bin\Debug\net8.0\PropNest.Domain
```

```
.Tests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.
  Passed PropNest.Domain.Tests.ListingHistoryTests.CreateDeltaStoresOnlyDeltaPayload [4 ms]
  Passed PropNest.Domain.Tests.ListingHistoryTests.CreateEventStoresNoPayload [1 ms]
  Passed PropNest.Domain.Tests.ListingHistoryTests.CreateSnapshotStoresOnlySnapshotPayload [< 1
ms]
  Passed PropNest.Domain.Tests.DeltaEngineTests.PriceChangeDetected [3 ms]
  Passed PropNest.Domain.Tests.DeltaEngineTests.ValueToNullDetected [< 1 ms]
  Passed PropNest.Domain.Tests.DeltaEngineTests.TitleChangeDetected [< 1 ms]
  Passed PropNest.Domain.Tests.DeltaEngineTests.NullToValueDetected [< 1 ms]
  Passed PropNest.Domain.Tests.DeltaEngineTests.NoChangeDetected [< 1 ms]
  Passed
PropNest.Domain.Tests.ListingTests.SubmitForModerationFromHiddenShouldMoveToPendingModeration [2
ms]
  Passed PropNest.Domain.Tests.ListingTests.RejectFromDraftShouldThrowInvalidOperationException [1
ms]
  Passed PropNest.Domain.Tests.ListingTests.HideFromDraftShouldThrowInvalidOperationException [< 1
ms]
  Passed PropNest.Domain.Tests.ListingTests.SubmitForModerationMovesDraftToPendingModeration [< 1
ms]
  Passed PropNest.Domain.Tests.ListingTests.ApproveRequiresAListingPendingModeration [< 1 ms]
  Passed PropNest.Domain.Tests.ListingTests.ApproveFromPendingModerationShouldMoveToPublished [< 1
ms]
  Passed
PropNest.Domain.Tests.ListingTests.SubmitForModerationWhenAlreadyPendingModerationShouldThrowInval
idOperationException [< 1 ms]
  Passed
PropNest.Domain.Tests.ListingTests.SubmitForModerationFromRejectedShouldMovesToPendingModeration
[< 1 ms]
  Passed PropNest.Domain.Tests.ListingTests.RejectFromPendingModerationShouldMoveToRejected [< 1
ms]
  Passed
PropNest.Domain.Tests.ListingTests.HideFromPendingModerationShouldThrowInvalidOperationException
[< 1 ms]
  Passed PropNest.Domain.Tests.ListingTests.HideFromPublishedShouldMoveToHidden [< 1 ms]
  Passed
PropNest.Domain.Tests.ListingTests.SubmitForModerationFromPublishedShouldThrowInvalidOperationExce
ption [< 1 ms]

Passed!  - Failed:     0, Passed:    20, Skipped:     0, Total:    20, Duration: 48 ms -
PropNest.Domain.Tests.dll (net8.0)
```

Summary

[Lab 07 - Bảng 1] Bảng tổng kết đối chiếu kết quả hoàn thành chuỗi Hands-On Labs

STT	Hạng mục Kỹ thuật Cốt lõi	Bài Lab	Bằng chứng Thực nghiệm trong Dự án

1	Kiến trúc Clean Architecture & Modular Monolith	Lab 01	Tách biệt 5 tầng độc lập; Domain không phụ thuộc bất kỳ thư viện ngoài nào.
2	Central Package Management & Build Props	Lab 01	Directory.Packages.props & Directory.Build.props; Build 0 Warning, 0 Error.
3	Rich Domain Model & EF Core 8 Fluent API	Lab 02	Listing, Wallet đóng gói chặt chẽ; 5 bản Migration đồng bộ tự động.
4	Đường ống Middleware & ProblemDetails RFC 7807	Lab 03	ExceptionHandlingMiddleware & CorrelationIdMiddleware hoạt động chuẩn xác.
5	History Delta Engine & Optimistic Concurrency	Lab 04	Lưu vết JSON Diff khi sửa tin; chặn ghi đè đồng thời với mã 409 Conflict.
6	Bảo mật JWT, Atomic Registration & Policy RBAC	Lab 05	Đăng ký tạo ví nguyên tử; kiểm chứng 3 vai trò trả về 401, 403, 200 OK.
7	MassTransit Saga, Transactional Outbox & Inbox	Lab 06	Mua gói VIP bất đồng bộ qua RabbitMQ; tự động hoàn tiền khi nâng cấp lỗi.
8	Kiểm thử Đơn vị xUnit & Đóng gói Docker	Lab 07	20/20 ca kiểm thử xUnit Passed; Multi-stage Dockerfile tối ưu hóa.